INDUSTRY STARTER PACKS · THE ARCHITECTURE WORKBOOK

Banking

Running the ADM in a bank — a step-by-step workbook for the repository you have been handed

The companion volume. Not something to read — something to work through. Every phase of the ADM as numbered steps: what you produce, which sheet of the repository it goes in, which columns, who signs it, how you know it is done, and the way it usually goes wrong. Written against a real populated metamodel and a real comparison tool, not against the standard in the abstract.

Use it with the learner guide. South African lens. October 2026.

First edition · October 2026

Thotnr Consulting

Contents

BEFORE YOU START

Chapter 1 · How to use this workbook 3

PRELIMINARY PHASE

Chapter 2 · Standing the practice up 23

PHASE A

Chapter 3 · Architecture Vision 46

PHASE B

Chapter 4 · Business Architecture 65

PHASE C

Chapter 5 · Data Architecture 93

Chapter 6 · Application Architecture 119

PHASE D

Chapter 7 · Technology Architecture 148

PHASE E

Chapter 8 · Opportunities and Solutions 175

PHASE F

Chapter 9 · Migration Planning 204

PHASE G

Chapter 10 · Implementation Governance 227

PHASE H

Chapter 11 · Architecture Change Management 253

CONTINUOUS

Chapter 12 · Requirements Management 277

APPENDICES

Appendix A · The sheet-by-sheet reference 300

Appendix B · The report catalogue 325

Appendix C · Workshop packs, checklists and the ninety-day calendar 334

Appendix D · The repository in ArchiMate 4 353

Appendix E · Where to look in volume 1 377

Appendix F · The forms 393

Appendix G · The principles and standards starter set 446

Appendix H · Running the maturity assessment 477

CHAPTER 1

How to use this workbook

One repository, one invented bank, and the change control it does not have — which you add before you load a single record.

One repository, one invented bank, and a hole in its governance that you have to close before you type anything into it. This chapter is the orientation: what you were handed, how the pieces fit, and the handful of conventions that make the other nineteen chapters usable. It contains no steps. Everything from chapter 2 onward does.

> **ARDENT UNION BANK IS INVENTED**
>
> Ardent Union Bank does not exist. The repository says so on its own front page: *"Ardent Union Bank is fictional. The figures are internally consistent and describe no real organisation"* [1]. Treat every number in this volume the way you would treat a worked example in a textbook — a shape to copy, not a benchmark to quote. Table Bay Bank, which appears throughout volume 1, is a separate invention. Do not mix their data, and do not import volume 1's system names — Helm, Quay, Tern, Meridian, Chartroom — into this repository. Ardent Union's applications are called `A01 Core Banking` through `A14 Fraud Detection`.

What you have, and what each thing is for

Thing	What it is	What it is not
`Banking_TOGAF_Metamodel.xlsx`	The repository. 105 sheets: one per object type, one per relationship type, thirteen reports, an industry extension and an integrity check	Not a document. Nothing in it is prose to read front to back
`Banking_Architecture_Comparison_Tool_TEMPLATE.xlsx`	A diff instrument: five paste targets, a `Classification` column that separates design delta from governed drift from ungoverned change, a human `Review` sheet, and a `Gap Candidates` sheet shaped to paste into `029 Gap`	Not populated, and not a gap catalogue. It is a template. It produces candidates; a person classifies them
The two volumes	*Banking — a starter pack for enterprise architects* is the content. This volume is the method	Neither is a reference manual. Volume 1 is read once; this one is opened at a step

Say the second one's name in full when you ask for it. It is a **template**: the insurance edition of this set shipped a comparison workbook already loaded with a worked example, and the banking edition does not. Chapter 8 works Phase E against the template's design and says plainly that both extracts have to be produced by hand, because this repository has no export sheets at all. The insurance repository had three.

The repository is the centre of gravity, so learn its shape first.

- `00 Index` — the catalogue. One row per object type with its record count and its domain.
- **64 object sheets**, `001 Driver` through `066 Architecture Increment`, with `034` and `051` absent. The repository counts *sixty-five* object types because `EXT Industry Extension` is the sixty-fifth, and `00 Index` lists all sixty-five in one table.
- **25 relationship sheets**, `R01` to `R25`.
- **13 report sheets**, `RPT01 Dashboard` to `RPT13 Orphan Analysis`.
- `EXT Industry Extension` — six object types banking needs beyond the TOGAF core. Chapter 2 gives it three steps.
- `ZZ Checks` — twenty-one lines of the repository grading itself. Read it before and after every phase.

The repository's own numbers, and the three places they disagree

`RPT01 Dashboard` is the one-page summary: 65 object types, 878 records, 25 relationship types, 465 relationship instances, 13 reports, 14 applications, 32 interfaces, 19 of them unmanaged, 20 with no logging, 10 direct database reads, 3 platform components past vendor support, 4 security controls absent, 4 control objectives with no evidence, 3 data entities with no lineage, 12 level-1 capabilities [2]. `ZZ Checks` repeats most of those figures with a `Threshold` column beside each, and the thresholds are all zero, because what a healthy estate shows is nothing [3].

Three of those numbers do not survive a count.

> **THREE NUMBERS THAT DISAGREE WITH THE SHEETS BENEATH THEM**
>
> **878 against 846.** Add up the per-sheet record counts in `00 Index` and you get 846 — 840 across the sixty-four `O` sheets plus the six rows of `EXT`. The index banner, `RPT01 Dashboard` and `ZZ Checks` all say 878. Thirty-two records are unaccounted for. The relationship figure, 465, does reconcile exactly. So one total was typed and the other was counted [1][2][3].
>
> **"Every type carries records" against two empty sheets.** `ZZ Checks` line one reads `Object types populated` = 65 with the threshold `Every type carries records`. `00 Index` records `O29 Gap` at 0 records and `O59 Compliance Assessment` at 0 records. The repository contradicts itself two sheets apart.
>
> **`Gaps overdue` = 0.** `RPT01 Dashboard` reports no overdue gaps, which reads as good news. There are no gaps at all. `O29 Gap` is empty, `R12 Gap closed by Work Package` is empty, and `RPT07 Gap Analysis` has a header and nothing under it. A zero computed over an empty set is the most dangerous number in any repository, and this one has three of them: gaps overdue, vacant architecture roles, and records validated against a real organisation.

The point is not that the workbook is sloppy. The point is the habit: where a typed total and a counted total disagree, the count wins, and you should never quote a repository figure to an executive without having counted it once yourself. Chapter 2, step 2.4 makes that a step.

The relationship sheets, and how they are named

The insurance repository named its relationship sheets after their endpoints — `R08 ACT-ROL`. This one names them after what they mean: `R01 Capability decomposition`, `R12 Gap closed by Work Package`, `R21 Role performed by Actor`, `R22 Org Unit owns Capability`. That is easier to read and harder to script against, and several tab names run past Excel's thirty-one-character limit and are cut off mid-word — `R02 Value stream uses Capabilit`, `R05 Application manages Data En`, `R06 Application interfaces with`, `R09 Requirement satisfied by Bu`, `R13 Work Package delivers Build`. **Spell them the way the tab spells them**, truncation included. This volume does, everywhere, and will not say so again.

The shape is consistent. Most relationship sheets have no `Ref` of their own: they open with the source reference, then the source's name, then the target reference, then the target's name, then one or two properties. Only `R06 Application interfaces with` and `R19 Data Entity relates to Data` carry a `Ref` column, because an interface and an entity relationship are things you need to cite.

`00 Index` claims that because every row names both ends by reference *and* by name, a broken link is visible without a lookup [1]. That holds only while somebody keeps the name in step with the reference, and nothing in the workbook does it for them: the name columns hold typed text, not formulas. Treat the name column as a courtesy, not a control, and write yourself the three-line script that checks it.

The ADM as this workbook runs it

phase	the sheets it fills	what proves it
Preliminary	005 009-012 057 058 001-004 031 EXT	ZZ RPT13
A Vision	006-008 009-011 014 031	RPT01 06
B Business	013 015-019 035 R01-R03 R22 R25	RPT03
C Data	020-022 036-044 R05 R19	RPT05
C Appl'n	023 024 045 R04 R06 R20	RPT02 04
D Technology	025-028 032 046-050 R07 R18	RPT10 11 12
E Opportun.	029 052 054 055 R09 R12 R13	RPT07
F Migration	030 053 056 066 R14 R15 R23	RPT08
G Governance	033 059-065 R16 R17 R24	RPT09
H Change	nothing — the sheets are absent	ZZ
Requirements	006 R08 R09	RPT06

64 object sheets, 25 relationship sheets, 13 reports. No phase H sheets exist.

Figure 1.1 The ADM as this volume runs it: each phase, the sheets it fills and the report that shows it landed. Phase H has no sheets in this repository — you build them in chapter 2.

Twenty chapters: one per ADM phase, Requirements Management as a running twelfth, then eight appendices. The phase order is the standard one and this volume does not argue with it [4]. What it adds is the mapping — which sheets a phase fills, and which report tells you it landed.

Ch	Phase	Primary sheets	What shows it landed
2	Preliminary	`O05 Principle`, `O09` – `O12`, `O57 Governance Body`, `O58 Governance Gate`, `O01` – `O04`, `O31 Deliverable`, `EXT Industry Extension`	`ZZ Checks`, `RPT13 Orphan Analysis`
3	A — Vision	`O06 Requirement`, `O07 Constraint`, `O08 Assumption`, `O09` – `O11`, `O14 Value Stream`, `O31 Deliverable`	`RPT01 Dashboard`, `RPT06 Requirements Traceability`
4	B — Business	`O13 Business Capability`, `O15` – `O19`, `O35`, `R01` – `R03`, `R22`, `R25`	`RPT03 Capability Heatmap`
5	C — Data	`O20 Data Entity`, `O21`, `O22`, `O36` – `O44`, `R05`, `R19`	`RPT05 Data Ownership`
6	C — Application	`O23 Application Component`, `O24`, `O45`, `R04`, `R06`, `R20`	`RPT02 Application Portfolio`, `RPT04 Interface Catalogue`
7	D — Technology	`O25` – `O28`, `O32 Standard`, `O46` – `O50`, `R07`, `R18`	`RPT10 Technology Currency`, `RPT11 Security Position`, `RPT12 Standards Conformance`
8	E — Opportunities	`O29 Gap`, `O52`, `O54 Architecture Alternative`, `O55 Building Block`, `R09`, `R12`, `R13`	`RPT07 Gap Analysis`
9	F — Migration	`O30 Work Package`, `O53 Transition Architecture`, `O56 Project`, `O66`, `R14`, `R15`, `R23`	`RPT08 Roadmap`
10	G — Governance	`O33 Architecture Contract`, `O59` – `O65`, `R16`, `R17`, `R24`	`RPT09 Regulatory Coverage`
11	H — Change	Nothing. The sheets this phase needs are not in the file	`ZZ Checks`, and whatever you added in chapter 2
12	Requirements	`O06 Requirement`, `R08`, `R09`	`RPT06 Requirements Traceability`

Two things in that table deserve a second look.

Row eleven is empty on purpose. Phase H is architecture change management, and the four sheets a change-management phase runs on — version control, a change log, change requests against records, and baselines — are not in this repository. Chapter 2 tells you to build them in week one and chapter 11 runs the phase against what you built. This is the volume's largest single piece of advice and the next section is about nothing else.

The report numbering is not the insurance set's. If you have worked that workbook, unlearn two numbers. Here `RPT07` **is** `Gap Analysis`, exactly where you would expect it, and `RPT09` is `Regulatory Coverage`, not a gap-and-roadmap sheet. There is no pair of sheets sharing a name. Thirteen reports, each answering one question, catalogued in appendix B.

How a step is laid out

From chapter 2 onward this volume is numbered steps — around a hundred and fifty of them. Every one has the same eight parts in the same order. Learn the shape once and you can read any step in forty seconds.

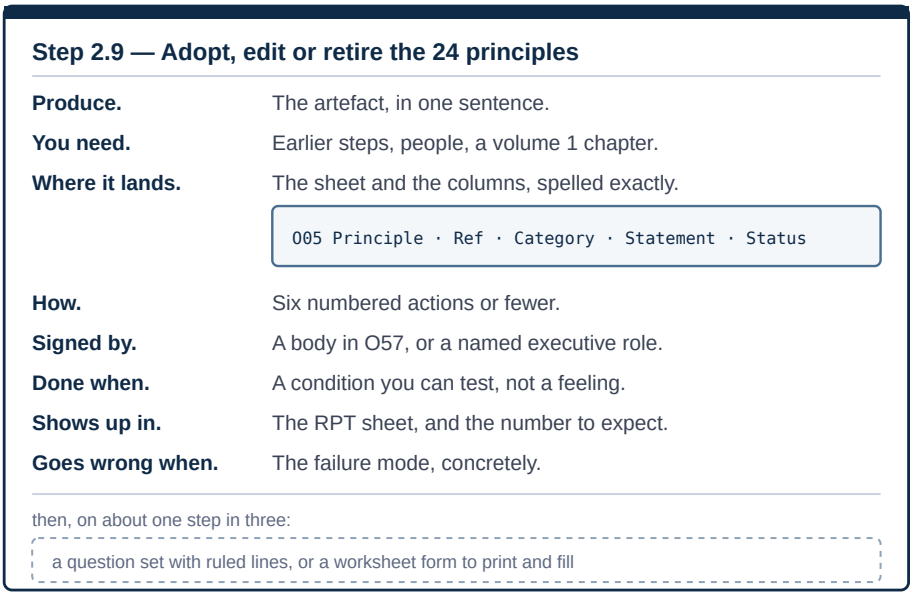

Figure 1.2 The step template. Eight parts, always in this order. You will meet it about a hundred and fifty times.

- **Produce.** The artefact, in one sentence. If you cannot say what the step produces, the step is a meeting.
- **You need.** Inputs: earlier steps, the people whose time you must book, the volume 1 chapter that supplies the content.
- **Where it lands.** The sheet, named exactly, and the columns to fill, with what belongs in each non-obvious one. This is the part the volume exists for.
- **How.** Six numbered actions or fewer. If something needs more than six, it is two steps.
- **Signed by.** A body from `O57 Governance Body` or a named executive role. Architecture nobody signed is a draft forever.
- **Done when.** A condition you can test. "The stakeholders are aligned" is not a condition. "Every `Critical` capability in `O13` has an owning unit in `R22`" is.
- **Shows up in.** The report sheet where the work becomes visible, and what the number should look like if the step went well. This is how you mark your own homework.
- **Goes wrong when.** The real failure mode, stated concretely enough to recognise in your own repository.

About one step in three carries a question set with ruled lines under it, for the judgements only you can make. Some steps carry a worksheet form instead, whose columns are the real columns of the real sheet. The full catalogue forms live in appendix F; the forms inside steps are small working tables for the room. Write in the book. A judgement you did not write down is a judgement you will make differently next month.

The change-control gap, and what to add before you load anything

This is the finding. Every other repository defect in this volume is a record you can correct in an afternoon. This one is structural.

There is no record-keeping anywhere in the file. Not a `Version` column, not a `Record Status`, not an `In Repository Since`, not a `Last Changed`, not a `Changed By`, not a `Change Ref`, not an `Approved By` — on any of the sixty-four object sheets or any of the twenty-five relationship sheets. The word "version" occurs once in 105 sheets, and not as a column header. And there are no `01 Version Control`, `02 Change Log`, `03 Change Requests` or `04 Baselines` sheets to carry what the columns would point at [1][5].

It is easy to mistake one sheet for the mechanism. `O61 Governance Change Request` is not it. It holds five rows with the columns `Ref`, `Change request`, `Against`, `Type`, `Status`, `Expires` — no raised date, no decided date, no previous value, no new value, no approver. Three of its rows are dispensations and scope changes against work packages. The other two, `CR-G03 Raise unfunded building blocks as gaps` and `CR-G04 Record benefit realisation dates per package`, are typed `Correction` and aimed at `Against` = `Repository` [6]. Somebody noticed the repository needed correcting, wrote it down, and had nowhere to apply it. That is the gap, recorded inside the gap.

What to add, before the first record

Four sheets and seven columns. Add them in this order; it takes a morning.

Add this sheet	Columns	What it is for
`01 Version Control`	`Rule`, `Statement`, `Owner` on top; `Workbook Version`, `State As At`, `Requests Raised`, `Approved`, `Open`, `Changes Logged`, `Baselines Taken` below	The rules, stated once, and the repository's current state in eight cells
`02 Change Log`	`Ref`, `Date`, `Sheet`, `Record`, `Column`, `Previous Value`, `New Value`, `Changed By`, `Change Ref`	The evidence base. Never truncated, never sorted destructively
`03 Change Requests`	`Ref`, `Raised`, `Raised By`, `Type`, `Scope`, `Description`, `Status`, `Decided`, `Approved By`, `Decision`	Nothing changes without a row here first
`04 Baselines`	`Ref`, `Name`, `Date`, `Workbook Version`, `Approved By`, `Status`, `Note`, `Changes Since`	A named, dated, approved fixed point to compare against

Then seven columns at the right-hand end of every object and relationship sheet you intend to maintain. Not all sixty-four — only the ones your year-one tailoring decision in step 2.5 puts in scope.

Column	What it tells you	What a bad value looks like
`Version`	`1.0` is the load; the minor number increments once per approved change, so `1.3` means three changes since baseline	Anything above `1.0` with an empty `Change Ref`. Somebody typed over the sheet
`Record Status`	`Baseline`, `Approved`, or `Draft — change pending` while a request is undecided	`Draft — change pending` against a request decided months ago
`In Repository Since`	When the record first appeared. On records added later, the date of the adding request	A date before the first baseline on a record the baseline does not contain
`Last Changed`	The date of the most recent approved change	Equal to `In Repository Since` on a record that has visibly moved
`Changed By`	`Repository load` for baseline rows, otherwise the **role** that made the change	A person's name. Roles outlive people, and `011 Role` is where they live
`Change Ref`	The join back to `03 Change Requests`. This is the audit trail	Empty on anything that is not `1.0`
`Approved By`	`Architecture Board` for anything that engages a principle, `Chief Enterprise Architect` otherwise	A body that does not appear in `057 Governance Body`

Two naming cautions. **Do not prefix your change requests `CR-`.** `061 Governance Change Request` already uses `CR-G01` to `CR-G05` for governance dispensations against designs, and a bare `CR-012` in a `Change Ref` column will be read as one of those by the next architect and by any script. Use `RCR-` — repository change request — or anything else unambiguous. And **do not derive `Version` by hand.** It follows from the count of approved rows in `02 Change Log` against that record, which is a formula, not a judgement.

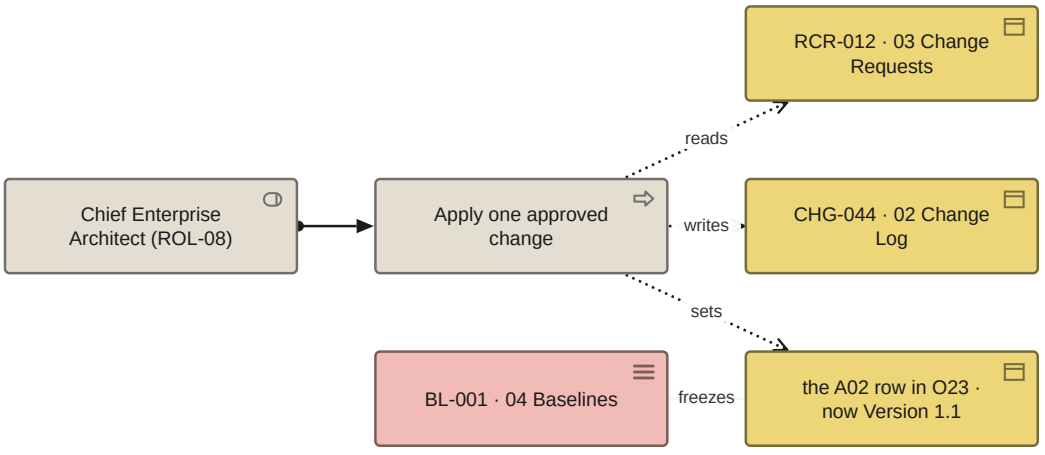

Figure 1.3 The change-control spine you must add, in ArchiMate 4. One approved change: the architect's process reads the change request, writes the log entry and sets the record's version — which is why the version is derived and never typed. The baseline freezes the record at a named version to compare against. None of these four sheets exists in the file as shipped.

What goes wrong if you skip this

Not a style point. Five specific failures, in the order you will meet them.

1. **You cannot answer the only question an executive asks about a repository.** "What has changed since the board saw this?" With no log, the honest answer is that you do not know, and the answer you will give instead is a guess.
2. **Gap analysis loses its fixed point.** The comparison tool template wants a baseline extract in one paste target and a later extract in another, and it wants a label and a date for each. With no `04 Baselines` there is nothing to name, so every difference the tool finds reads as a design delta and nothing can ever be classified as governed drift. The tool's whole value is in that distinction.
3. **The `ZZ Checks` numbers stop being a trend.** Nineteen unmanaged interfaces is not a finding. Nineteen this quarter against twenty-six last quarter is a finding, and nineteen against twelve is an escalation. Without a dated snapshot you have a photograph where you needed a film.
4. **The repository stops being evidence.** Joint Standard 1 of 2023 on IT governance and risk management commenced on 15 November 2024 and expects a bank to show how change to material systems is controlled [7]. If your architecture repository is the thing that says which systems are material, which data entity feeds which return, and who owns what, then a repository with no change history is not evidence of anything. The same applies the first time a supervisor asks how a figure in a BA return was derived.
5. **The `Owner` columns rot silently.** They already have. `O20 Data Entity` names ten owners — `Head of Treasury`, `Head of Lending`, `Head of Financial Crime` and seven more — and not one of them appears in `O11 Role` [8]. Nobody can tell whether those titles were wrong when they were typed or stopped being right afterwards, because there is no date on them.

> **THE WEEK-ONE JOB**
>
> Add the four sheets and the seven columns, load `04 Baselines` with one row describing the repository exactly as you received it, and only then open a sheet to edit it. Chapter 2 makes this steps 2.2 and 2.3 and puts them before everything except fixing the enterprise boundary. If you take one instruction from this chapter, take that one.

The `Ref` conventions already in use

Every object sheet puts the reference in column A and the name in column B, under a four-row preamble: a title, a line reading `Ardent Union Bank · <domain> domain · <n> records`, a blank, then the header row. **The header is row 4.** Any script you write reads from row 5.

The prefixes are set. Do not invent new ones for sheets that have them.

Prefix	Sheet	n	Prefix	Sheet	n
DRV-	O01 Driver	13	WP-	O30 Work Package	12
GOL-	O02 Goal	6	DLV-	O31 Deliverable	20
OBJ-	O03 Objective	14	STD-	O32 Standard	16
MSR-	O04 Measure	16	ACO-	O33 Architecture Contract	8
PRN-	O05 Principle	24	BBB-	O35 Business Building Block	10
REQ-	O06 Requirement	32	REL- ATR- DST- MIG- MAP-	O36 – O40	95
CON-	O07 Constraint	8	SEC- MTD- MDR- DBB-	O41 – O44	42
ASM-	O08 Assumption	6	ABB-	O45 Application Building Block	10
ORG-	O09 Organization Unit	12	TBB-	O46 Technology Building Block	7
ACT-	O10 Actor	18	RME-	O47 Reference Model Entry	11
ROL-	O11 Role	18	ENV- RES- NWL-	O48 – O50	26
LOC-	O12 Location	8	IF-	O52 Implementation Factor	8
C01.01.01	O13 Business Capability	120	TA-1 ALT-A	O53 , O54	8
VS01	O14 Value Stream	5	BBB- DBB- ABB- TBB-	O55 Building Block	37
BSV- BPR- BFN-	O15 – O17	44	PRJ-	O56 Project	12
PRD- CTR-	O18 , O19	18	GB- GT-	O57 , O58	9
E01	O20 Data Entity	14	DEC- CR-G RSK-	O60 – O62	20
LDC- PDC-	O21 , O22	24	CO- POL- EP- INC-	O63 – O66	22
A01	O23 Application Component	14	X01	EXT Industry Extension	6
ISS-	O24 Information System Service	14	—	O29 Gap , O59	0

TSV- LTC- PTC- PSV-	O25 – O28	39

Seven irregularities in that list. None of them is a mistake to tidy; all of them are load-bearing, and you need to know them before you query anything.

Five reference shapes in one file. Most sheets use `XXX-nn`. Four use a bare letter and digits with no hyphen — `C01`, `E01`, `A01`, `VS01`. Three use a single-digit suffix — `GT-0`, `TA-1`, `INC-1`. One uses a letter suffix, `ALT-A` to `ALT-E`. Two use three digits, `REQ-001` and `ATR-001`, expecting to pass ninety-nine. Sort a mixed column as text and `A1` will not land where you expect.

`O13 Business Capability` does not use a prefix at all. References are dotted and unprefixed: `C01`, `C01.01`, `C01.01.01`. The dots carry the level, which is why `R01 Capability decomposition` can hold 108 rows with a `Level` column and no separate hierarchy table. Twelve level-1 capabilities decomposed to level 3, 120 records.

`O55 Building Block` has no reference space of its own. Its thirty-seven rows reuse `BBB-` from `O35`, `DBB-` from `O44`, `ABB-` from `O45` and `TBB-` from `O46` — ten, ten, ten and seven. So a bare `ABB-03` is ambiguous between `O45 Application Building Block` and `O55 Building Block`, and `R09 Requirement satisfied by Bu` and `R13 Work Package delivers Build` both point into the ambiguity. Always name the sheet.

`O46 Technology Building Block` has two columns called `Ref`. Column A holds `TBB-01` to `TBB-07`. Column C, also headed `Ref`, holds a second undeclared space, `T01` to `T07`. Nothing else in the repository uses `T01`. A duplicate header will break any reader that keys on column names, and pandas will silently rename one of them.

`O29 Gap` and `O59 Compliance Assessment` have no established prefix, because they have no rows. You choose. `GAP-` and `CA-` are conventional, and `RPT07 Gap Analysis`, `R12 Gap closed by Work Package` and `R24 Compliance Assessment tests` will accept whatever you type, because every join in this workbook is a text match. Choose once, write it into `01 Version Control`, and never renumber.

`CR-G` is taken, as above. It is the collision you will walk into.

Never renumber anything. References are matched as text from the relationship sheets and from the inline reference columns on about half the object sheets. Renumber `E06` and you break `R05 Application manages Data En`, `R19 Data Entity relates to Data` and the `Entity` columns of `O21`, `O37`, `O39`, `O40` and `O43` — and because the adjacent name column is typed rather than computed, nothing turns red to tell you.

The two volumes, and how to use them together

The division of labour is strict.

Volume 1 is the content. Twelve lines of business, the money, the operating model, thirteen BPMN level-2 processes, the conceptual data model, twenty-five conceptual applications, the real estate, four

reference architectures, the vendors, the regulation, and how to draw all of it in ArchiMate 4. Read it front to back, once. It answers *what is true about banking*.

Volume 2 is the method. It never re-teaches banking. Which sheet, which columns, in what order, signed by whom, and the report that proves it. It answers *what do I do on Tuesday*.

Read volume 1 once before the Preliminary Phase, then keep it open at the chapter the phase names. Each chapter from 2 onward opens with a *Before you start this phase* box, and steps resting on a judgement volume 1 has already worked carry a **Look it up.** line naming the chapter and the section inside it.

One warning, repeated wherever it bites. Volume 1's catalogues are numbered for volume 1 — `P-01` … `P-12` processes, `C-01` … `C-20` capabilities, `R-01` … `R-33` roles, `D-01` … `D-20` data entities, `A-01` … `A-25` conceptual applications, all in its appendix A. This repository is numbered for itself: twelve `C01`-style level-1 capabilities, twenty `BPR-` processes, eighteen `ROL-` roles, fourteen `E01`-style entities, fourteen `A01`-style applications. Same industry, different grain, built independently. **Map by name, never by number.** `A-11` is not `A11`. `D-05` is not `E05`. `C-07` is not `C07`. Twenty-five conceptual applications against fourteen `A` rows is not a coincidence waiting to be aligned; it is two models at different resolutions.

Workbook chapter	Phase	Volume 1 chapters to have read
1 How to use this workbook	Before you start	1, 16
2 Standing the practice up	Preliminary	1, 5, 15, 18
3 Architecture Vision	A	4, 5, 7, 17
4 Business Architecture	B	2, 5, 7, 8, 19
5 Data Architecture	C	9, 10, 15, 19
6 Application Architecture	C	11, 12, 13, 19
7 Technology Architecture	D	12, 13, 14, 15
8 Opportunities and Solutions	E	6, 11, 13, 14
9 Migration Planning	F	12, 14, 17, 18
10 Implementation Governance	G	15, 16
11 Architecture Change Management	H	7, 14, 15
12 Requirements Management	Continuous	10, 15, 16
13 The sheet-by-sheet reference	Reference	16, 20
14 The report catalogue	Reference	6, 15
15 Workshop packs and the calendar	Reference	5, 18
16 The repository in ArchiMate 4	Reference	16, 17
17 Where to look in volume 1	Reference	19, 20
18 The forms	Reference	—
19 Principles and standards starter set	Reference	13, 15, 21
20 Running the maturity assessment	Reference	7, 18

That column is the minimum, not a reading list.

The notation: ArchiMate 4, and the one exception

Every architecture diagram in this volume is ArchiMate 4. Volume 1, chapter 16 (Modelling banking in ArchiMate) teaches the notation — the elements, the five decisions that settle most of a banking model, and the viewpoints worth drawing. This section says only how the notation lands on *these sheets*, which is the part volume 1 cannot tell you, because it does not know which sheets you have.

The repository is a TOGAF content metamodel, not an ArchiMate model. Its sheets are object types and relationship types; ArchiMate is how you draw them. The mapping below is fixed, so that two architects drawing the same corner of the repository produce the same picture [9].

Sheet	ArchiMate 4 element	Sheet	ArchiMate 4 element
`001 Driver`	Driver	`014 Value Stream`	Value Stream
`002 Goal`	Goal	`015 Business Service`	Service
`003 Objective`	Outcome	`016 Business Process`	Process
`004 Measure`	Assessment	`017 Business Function`	Function
`005 Principle`	Principle	`018 Product`	Product
`006 Requirement`	Requirement	`019 Contract`	Business Object, as a Contract
`007 Constraint`	Requirement, dashed	`020 Data Entity`	Business Object
`008 Assumption`	Meaning	`021 Logical Data Component`	Data Object
`009 Organization Unit`	Business Actor	`022 Physical Data Component`	Artifact
`010 Actor`	Business Actor	`023 Application Component`	Application Component
`011 Role`	Role, or Stakeholder	`024 Information System Service`	Service
`012 Location`	Location	`025 Technology Service`	Service
`013 Business Capability`	Capability	`028 Platform Service`	Service

Sheet	ArchiMate 4 element	Sheet	ArchiMate 4 element
`026 Logical Technology Componen`	Node	`055 Building Block`	Grouping
`027 Physical Technology Compone`	Node, Device or System Software	`056 Project`	Work Package
`029 Gap`	Assessment	`057 Governance Body`	Business Actor
`030 Work Package`	Work Package	`058 Governance Gate`	Event
`031 Deliverable`	Deliverable	`062 Delivery Risk`	Assessment
`032 Standard`	Requirement	`063 Control Objective`	Requirement
`041 Security Control`	Node, or Function	`064 Security Policy`	Principle
`048 Environment`	Grouping	`065 Enforcement Point`	Application Interface
`053 Transition Architecture`	Plateau	`066 Architecture Increment`	Plateau
`054 Architecture Alternative`	Course of Action	`EXT X01 – X06`	see chapter 2, step 2.6

Five notes on that mapping, each of which catches somebody.

Five sheets map to `Service`. `015 Business Service`, `024 Information System Service`, `025 Technology Service` and `028 Platform Service` are all the one ArchiMate `Service` element, and so is a business service drawn at the top of a layered view. The layer is carried by what *realises* the service, not by the element. Draw the realiser and the layer is unambiguous.

`027 Physical Technology Compone` takes whichever element fits the thing. A server or an authorisation switch is a `Device`, a database engine is `System Software`, everything else is a `Node`. The sheet's `Class` column — Mainframe, Application, Database and so on — tells you which, and it is the only place that does.

Gaps, measures and risks are all `Assessment`. `029 Gap`, `004 Measure` and `062 Delivery Risk` are statements about the state of affairs, which is what `Assessment` means. A gap is not a `Requirement`; the requirement is the thing that closes it.

`053` and `066` are both `Plateau`, and not the same plateau. `053 Transition Architecture` holds three — `TA-1 Governed and supportable`, `TA-2 Integrable`, `TA-3 Business visible`. `066 Architecture Increment` holds six, landing at months 10, 16, 22, 30, 42 and 56. The transition architecture is the state; the increment is what is added to reach it. Label which sheet each plateau came from, or nobody will reconcile three with six.

A stakeholder is the `Stakeholder` element, not a `Business Actor`. The same `011 Role` row is drawn as a `Role` in a business view and a `Stakeholder` in a motivation view. The view decides, and chapter 3 uses both.

Relationships follow ArchiMate's rules, not the relationship sheet's name. An application **realises** a service. A role is **assigned to** a process. A process **accesses** a business object. A service **serves** a process or an actor. A capability is **composed of** its sub-capabilities. A work package **realises** a deliverable or a plateau. A driver **influences** a goal, a principle **influences** a requirement, a requirement **realises** a goal. Where a relationship sheet joins two rows in a way ArchiMate will not allow, the sheet has found you a modelling error, not a notation problem.

> **THE EXCEPTION: LEVEL-2 PROCESS MODELS ARE BPMN 2.0**
>
> ArchiMate has no swimlanes and no way to show a hand-off from one lane to another. A level-2 banking process is mostly hand-offs — originator to credit, credit to legal, legal back to operations, operations to the scheme — so drawing one in ArchiMate loses the thing worth drawing. Volume 1, chapter 8 (The process model) publishes thirteen level-2 processes in BPMN 2.0 and explains the choice under its own heading, *Why this chapter leaves ArchiMate*. This volume keeps to that. Everything at level 1 and above stays in ArchiMate: `O16 Business Process` as a `Process`, joined to its value stream, its owning role and the objects it touches. Volume 1, chapter 16 (Modelling banking in ArchiMate) sets out where the notation stops under *The honest limits*.

The three ways people wreck this

Modelling everything. The symptom is a repository with more records than relationships, and this one has it: 846 records against 465 relationship instances. **Thirty-three of the sixty-four object sheets are reached by no relationship sheet at all.** About half of those are joined by an inline reference column instead — `O16 Business Process` has a `Value stream` column, `O37 Data Attribute` has an `Entity` column, `O49 Resilience Tier` has an `Application` column — which is a weaker join but a real one. Seventeen are joined to nothing by any reading: `O05 Principle`, `O07 Constraint`, `O08 Assumption`, `O19 Contract`, `O25 Technology Service`, `O26 Logical Technology Componen`, `O28 Platform Service`, `O31 Deliverable`, `O38 Data Set`, `O42 Metadata Element`, `O47 Reference Model Entry`, `O48 Environment`, `O52 Implementation Factor`, `O54 Architecture Alternative`, `O57 Governance Body`, `O58 Governance Gate` and `O62 Delivery Risk`. Twenty-four principles that govern nothing, five alternatives that decide nothing, four gates that test nothing. If you are about to populate a sheet, name the relationship sheet that will join it to something else. If there is not one, do not populate it yet.

And note what `RPT13 Orphan Analysis` does with all of that. It reports **two** orphans: `A13 Data Platform` and `A14 Fraud Detection`, both with the finding "No business service maps to it" [10]. It is a good report and it looks at one join only — applications against business services through `R04 Business Service supported`. The seventeen isolated sheets are invisible to it. The repository's own orphan report is not an orphan report; it is an application orphan report. Chapter 2, step 2.4 tells you to run it anyway, and then to run the count the report does not.

Modelling nothing. The opposite failure, and the more common one. The architect reads the ADM, agrees with it, produces slides, and the repository stays exactly as loaded. The insurance repository caught this with a version-control report that graded each sheet `Unchanged since baseline`. This one cannot, because there is no change log — which is the fourth reason the previous section matters.

Until you build one, the only evidence that you have done anything is the `ZZ Checks` numbers moving, and they move slowly.

Modelling the org chart. The most expensive of the three because it looks like work, and this repository shows you both halves of the mistake. `O09 Organization Unit` holds twelve rows and they are a generic corporate structure — Executive, Finance, Risk & Compliance, Human Resources, Technology, Data & Analytics, Operations, Commercial, Customer Service, **Supply & Logistics**, Internal Audit, Legal. There is no Retail Banking, no Business Banking, no Treasury, no Payments. A bank's organisation chart was replaced by a template's. Then `R22 Org Unit owns Capability` joins the two on the diagonal: `ORG-01` to `C01`, `ORG-02` to `C02`, all the way to `ORG-12` to `C12`. The result is that Human Resources owns Credit Risk Management, Supply & Logistics owns Risk & Capital Management, and Legal owns Channel & Distribution [11]. Twelve rows, twelve lies, and the sheet looks complete.

Capabilities are what the enterprise can do; organisation units are who currently does it. Keeping them in separate sheets joined by `R22` is the point, because the second changes without the first. Reorganise a bank and the capability map should not move. Chapter 2, step 2.1 rebuilds `O09`; chapter 4 rebuilds `R22`. Not the second before the first.

Adapting this to a bank that is not Ardent Union

Most of what you were handed is reusable. Some of it is Ardent Union's and must go.

Keep as shipped	Re-derive for your bank	Do not delete
The sheet structure, the four-row preamble, the header-on-row-4 convention and the `Ref` prefixes	`O09`, `O10`, `O11`, `O12`, `O18`, `O19`, `O23` — your organisation, actors, roles, sites, products, contracts and applications	Nothing. Mark a record superseded; never delete a row that a relationship sheet or an inline reference column points at
`EXT Industry Extension` — the six banking object types, which are right	`O01`–`O04`: your drivers, goals, objectives and measures, and every `Baseline` figure in `O04`	
`O05 Principle` as a starting set of twenty-four to argue with	`O13` levels 2 and 3, and every `Current` and `Target` score	
`O57`, `O58` — five bodies and four gates, renamed to your forums	Every `Owner`, `Head` and `Master system` value in the file	
`O63 Control Objective`'s seven obligations, which are the right seven for a bank	`O29 Gap`, `O30 Work Package`, `O56 Project` and everything downstream of them	

The order matters. Do `O09 Organization Unit` and `O12 Location` first, because `Head` and `Location` values across a dozen other sheets reference them by name. Then `O11 Role`, because `Owner` columns should point at roles. Then `O18 Product`, because products drive the business layer. Then the motivation sheets. Leave capabilities until you have value streams, for the reason chapter 3 gives.

Four bank-specific adjustments you will need almost immediately, all of them metamodel extensions rather than data entry.

Licence class has nowhere to live. `O18 Product` carries four columns — `Ref`, `Product`, `Type`, `Description` — and nothing else. No line of business, no book size, no licence reference. Volume 1, chapter 15 argues under *The Banks Act, and why a licence is a data attribute* that the statutory category of deposit-taker and the product's regulatory treatment are attributes of the product. Add the columns before you load a product.

The money is in the wrong currency. `O30 Work Package` costs read `$7.7m`, `$10.1m`, `$2.0m`; `O54 Architecture Alternative` reads `$24m` to `$88m`. For a bank reporting in rands, a dollar sign in a board paper is a conversation you do not want. Change the symbol, or convert and record the rate and date in `01 Version Control`.

Classification is wrong, not merely absent. `O20 Data Entity` has a `Classification` column and it reads `Public` against `E02 Customer`, `E03 Account` and `E07 Payment Instruction` [12]. Under POPIA that is not a placeholder, it is a finding. Chapter 5 rebuilds the column; until then do not publish `RPT05 Data Ownership` to anybody.

One record contradicts volume 1. `TBB-03` in `O46 Technology Building Block` and `REQ-030` in `O06 Requirement` are both called "API gateway and open banking". South Africa has **no regulated open banking regime** — what exists is bilateral commercial APIs plus SARB Directive 2 of 2024 on screen-scraped EFT credit initiation, with the FSCA's Open Finance position paper only exploring a regime [13]. Rename both to what you mean, usually "API gateway and third-party access with consent management". Two cells, and leaving them is how a false market claim reaches a board pack.

> **READ-ONLY FOR A WEEK**
>
> Before you change a cell, work through `ZZ Checks`, `RPT01 Dashboard` and `RPT13 Orphan Analysis`, then count the records in `00 Index` yourself and count how many object sheets no relationship sheet reaches. Four readings and one count. They tell you how big the repository is, where it is unlinked, what it believes about itself and where that belief is wrong. If you are inheriting a repository rather than starting one, those five numbers are your handover, and chapter 2 opens with them as step 2.4.

How to use this volume day to day

Open it at the step you are on. The part heading names the phase, the step numbers are stable, and chapters 13 to 20 are reference — the sheet-by-sheet description, the report catalogue, the workshop packs and ninety-day calendar, the repository drawn in ArchiMate 4, the index back into volume 1, the forms, the principles and standards starter set, and the maturity assessment.

When a step says volume 1's appendix A already drafts a catalogue, go and start from there, then argue with it. Your bank is not Table Bay and it is certainly not Ardent Union.

THE CHAPTER IN TEN LINES

- The repository is 105 sheets: 64 object sheets, 25 relationship sheets, 13 reports, an index, an industry extension and an integrity check.
- `EXT Industry Extension` is the sixty-fifth object type, which is why the file says 65 and has 64 `O` sheets.
- Three of the repository's own headline numbers do not survive a count: 878 records against 846, “every type carries records” against two empty sheets, and `Gaps overdue` = 0 over an empty `O29 Gap`.
- There is no version column, no change log, no change-request register and no baseline sheet anywhere in the file. Add four sheets and seven columns in week one.
- Do not prefix your own change requests `CR-`. `O61 Governance Change Request` already uses `CR-G01` to `CR-G05`.
- Five reference shapes coexist. `O13` is dotted and unprefixed, `O55` reuses four other sheets' prefixes, and `O46` has two columns called `Ref`.
- Map volume 1's catalogues onto the repository by name, never by number. `A-11` is not `A11`.
- Everything architectural is ArchiMate 4. The single exception is level-2 process models, which are BPMN 2.0, per volume 1 chapter 8.
- Thirty-three object sheets are reached by no relationship sheet; seventeen are joined to nothing at all. `RPT13 Orphan Analysis` sees none of them — it only checks applications against business services.
- `R22 Org Unit owns Capability` is a mechanical diagonal: ORG-01 to C01 through ORG-12 to C12. Human Resources owns Credit Risk Management. Rebuild it.

SOURCES FOR THIS CHAPTER

- **1.** Banking_TOGAF_Metamodel.xlsx, sheet 00 Index
- **2.** Banking_TOGAF_Metamodel.xlsx, sheet RPT01 Dashboard
- **3.** Banking_TOGAF_Metamodel.xlsx, sheet ZZ Checks
- **4.** The Open Group, TOGAF Standard, 10th Edition — Architecture Development Method https://pubs.opengroup.org/architecture/togaf10-doc/adm/
- **5.** Banking_TOGAF_Metamodel.xlsx — full sheet and column inventory, all 105 sheets
- **6.** Banking_TOGAF_Metamodel.xlsx, sheet O61 Governance Change Request
- **7.** Prudential Authority and FSCA, Joint Standard 1 of 2023 — IT governance and risk management, commenced 15 November 2024 https://www.resbank.co.za/en/home/publications/prudential-authority/pa-public-awareness/joint-standards
- **8.** Banking_TOGAF_Metamodel.xlsx, sheets O20 Data Entity and O11 Role
- **9.** The Open Group, ArchiMate Specification — elements, relationships and viewpoints https://pubs.opengroup.org/architecture/archimate3-doc/
- **10.** Banking_TOGAF_Metamodel.xlsx, sheet RPT13 Orphan Analysis
- **11.** Banking_TOGAF_Metamodel.xlsx, sheets O09 Organization Unit and R22 Org Unit owns Capability
- **12.** Banking_TOGAF_Metamodel.xlsx, sheet O20 Data Entity
- **13.** South African Reserve Bank, Directive 2 of 2024 — initiation of EFT credit payments; FSCA Open Finance position paper https://www.resbank.co.za/en/home/publications/publication-detail-pages/prudential-authority/pa-directives/2024

CHAPTER 2

Standing the practice up

The phase that decides who may say no, to what, on what evidence — and, in this repository, the phase that has to build its own change control first.

The Preliminary Phase has a bad reputation because it is usually done as a slide deck. Done properly it decides whether anything else in this volume will work: who may say no, to what, on what evidence, and whether anyone can prove next year what the repository said this year. Fifteen steps, four to six weeks if the sponsorship is real.

Ardent Union Bank, the bank in the repository, is invented and its figures illustrative. Its Preliminary Phase is instructive because it went half right. Sixty-four object sheets were loaded, twenty-four principles written, five bodies and four gates described — and `057 Governance Body`, `058 Governance Gate`, `005 Principle`, `031 Deliverable` and `062 Delivery Risk` are joined to nothing at all, while `DEC-01` to `DEC-04` in `060 Governance Decision` all read `Outstanding` [1][2]. The structure exists; the practice does not.

> **BEFORE STEP 2.1**
>
> Do step 2.4 first if you are inheriting this repository rather than starting it. `ZZ Checks` takes twenty minutes to read and it will change what you do in steps 2.5 and 2.8. Chapter 1 explains why the numbers on it need counting before they are quoted.

> **BEFORE YOU START THIS PHASE**
>
> Read these first and keep volume 1 open at them.
>
> - **Volume 1, chapter 1, *The seven things an architect must understand*.** The industry in one sitting.
> - **Volume 1, chapter 15, *The Banks Act, and why a licence is a data attribute*.** Why the licence is architecture, not reference data.
> - **Volume 1, chapter 5, *The architect's real counterparties*.** The people whose agreement you need and who will never come to an architecture forum.
> - **Volume 1, chapter 18, *Six numbers that tell you whether the estate is healthy*.** What to measure before you have permission to change anything.

The shape of the phase

Steps	What they settle
2.1 – 2.4	Boundary, change control, the first baseline, and the health check
2.5 – 2.8	Tailoring, the industry extension, and the one convention that fixes `Owner`
2.9 – 2.12	Principles, bodies, gates, and the architecture function itself
2.13 – 2.14	Maturity, and whether the enterprise's stated intent holds together
2.15	The deliverable register, and closing against a baseline

Step 2.1 — Fix the enterprise boundary and the licensed entities in scope

Produce. A one-page scope statement naming every legal entity, deposit-taking licence and organisation unit inside the architecture boundary — and, explicitly, the ones outside it.

You need. The group structure from the company secretary, and the licence schedule as the Prudential Authority holds it.

Look it up. Volume 1, chapter 15, *The Banks Act, and why a licence is a data attribute* — four statutory categories of deposit-taker, and what each one may do.

Where it lands. `O09 Organization Unit` and `O12 Location`.

> **`O09 Organization Unit` — five columns, and one of them is wrong for a bank**
>
> `Ref` — the next `ORG-` number, never a reused one. `Type` — the shipped values are Group, Corporate, Line and Assurance. There is no Legal Entity value and no licence column; see the warning below. `Head` — a role title, never a person's name, and it should resolve to a row in `O11 Role`. `Headcount` — an integer, with the date recorded in `02 Change Log`.
>
> The twelve shipped rows are a generic corporate structure: Executive, Finance, Risk & Compliance, Human Resources, Technology, Data & Analytics, Operations, Commercial, Customer Service, **Supply & Logistics**, Internal Audit, Legal. No Retail Banking, no Business Banking, no Treasury, no Payments. The sheet most obviously not about a bank is the sheet most other sheets depend on. Replace all twelve rows.

How.

1. List the licensed entities with the statutory category against each.
2. Decide the boundary. Write down each exclusion with a reason.
3. Load `O12 Location` first — eight rows, `LOC-01 Head Office` to `LOC-08 Disaster Recovery Site`, each with a `Scale` in people. `LOC-07 Cloud Region` reads "88 people", which is what happens when one column measures two kinds of thing.
4. Load `O09` top-down, replacing rather than editing: group, lines of business, group functions, assurance.

5. Stop where a unit could own a capability. Twelve to twenty rows suits a mid-tier bank.
6. Raise a change request for every column the metamodel lacks, starting with licence.

Signed by. The Chief Executive, or the Executive Committee (`GB-04` in `057 Governance Body`). Nobody below that can rule an entity out of scope.

Done when. Every licensed entity either appears in `009` or appears on the written exclusion list with a reason; every `Head` value is a role title that exists in `011 Role`; and the boundary statement carries a date and a signature.

Shows up in. Nothing yet, and that is the first thing to notice. No report counts organisation units, and `R22 Org Unit owns Capability` is the only relationship sheet reaching `009`. The effect appears in chapter 4, when `RPT03 Capability Heatmap` finally has owners behind it.

Goes wrong when. You edit the twelve shipped rows instead of replacing them, and nobody can later tell which units are yours. Or `Head` holds a person's name and the column becomes a graveyard. Or an offshore entity is quietly omitted and `CON-05 Data residency` turns up in Phase D as a surprise rather than a constraint.

> **THERE IS NOWHERE TO RECORD WHICH ENTITY HOLDS WHICH LICENCE**
>
> `009 Organization Unit`'s `Type` column describes organisational shape, not legal standing. A group with a registered bank, a controlling company, a foreign branch and a bancassurance subsidiary has no column in which to say so. Add `Legal Entity`, `Registration Number` and `Licence Category` before you load it, and record the extension in `03 Change Requests` with `Type` = Extension. A three-column change that prevents a year of argument about whether a shared service sits inside the regulated entity.

BEFORE YOU DRAW THE BOUNDARY

Which licensed entity are you most tempted to leave out, and what turns up in Phase D if you do?

Which role — not which person — can rule an entity out of scope, and has that role agreed to be asked?

At what level does a unit stop being capable of owning a capability, and what does that make your deepest `009` row?

Step 2.2 — Add change control before you touch a single record

Produce. Four new sheets and seven new columns, so every later change leaves a trace.

You need. The file and a morning. Do not wait for a governance decision; `GB-05 Architecture Function` already holds the mandate to maintain the repository [1].

Where it lands. Four sheets that do not exist yet, inserted before `001 Driver`.

> **The four sheets to add, and the minimum columns in each**
>
> `01 Version Control` — the rules at the top, one row each with an `Owner`; the state below: `Workbook Version`, `State As At`, `Requests Raised`, `Approved`, `Open`, `Changes Logged`, `Baselines Taken`.
>
> `02 Change Log` — `Ref`, `Date`, `Sheet`, `Record`, `Column`, `Previous Value`, `New Value`, `Changed By`, `Change Ref`. One row per cell changed, never truncated.
>
> `03 Change Requests` — `Ref`, `Raised`, `Raised By`, `Type`, `Scope`, `Description`, `Status`, `Decided`, `Approved By`, `Decision`. Prefix them `RCR-`: `061 Governance Change Request` already holds `CR-G01` to `CR-G05`.
>
> `04 Baselines` — `Ref`, `Name`, `Date`, `Workbook Version`, `Approved By`, `Status`, `Note`, `Changes Since`. The last is a formula over `02 Change Log`.
>
> Then the seven columns at the right-hand end of every sheet in your year-one set: `Version`, `Record Status`, `In Repository Since`, `Last Changed`, `Changed By`, `Change Ref`, `Approved By`. Chapter 1 gives each value set.

How.

1. Insert the four sheets, with the headers above.
2. Write the rules into `01 Version Control` in your own words. The one that matters: `Version` is derived from `02 Change Log` and never typed.
3. Add the seven columns to the sheets your tailoring decision will keep. Leave the rest alone.
4. Set every existing record to `Version` = 1.0, `Record Status` = Baseline, `Changed By` = Repository load, `In Repository Since` = the date you received the file.
5. Lock the seven columns against typing and leave `Version` as a formula.
6. Tell the domain architects, in writing, that an edit without a request is a defect and will be reverted.

Signed by. The Chief Enterprise Architect (`ROL-08`). A working practice, not an architecture decision — waiting for a board slot is how it does not happen.

Done when. The four sheets exist with headers, every record in your year-one set reads `Version` = 1.0 and `Record Status` = Baseline, and `01 Version Control` names a version and a state date.

Shows up in. `01 Version Control`, and from here on every `Change Ref` in this volume. There is no `RPT` sheet for it; add a fourteenth once the log has thirty rows.

Goes wrong when. You defer it until "after the first phase", because the phase is urgent and the housekeeping is not. The first phase is exactly the period in which the repository changes most, and nothing from it will be recoverable.

Step 2.3 — Cut the first baseline, which means building the sheet first

Produce. A named, dated, approved snapshot of the repository as you received it, before you change anything.

You need. Step 2.2. If you have already loaded your own data you are too late for a clean baseline — say so in the `Note` column rather than pretending otherwise.

Where it lands. `04 Baselines`, one row: `BL-001`, `Name` = Preliminary entry baseline, `Date`, `Workbook Version` = 1.0, `Approved By` = Chief Enterprise Architect, `Status` = Current, `Note` = what it is for and what is known to be wrong with it.

How.

1. Save a read-only copy with the baseline reference in the filename.
2. Add the `04 Baselines` row with `Status` = Current. Exactly one row may read Current.
3. Produce the extract by hand. This repository has **no export sheets** — the insurance edition had three — so copy the sheets you care about into a flat `Sheet`, `Record`, `Column`, `Value` layout and keep it beside the baseline copy. Chapter 8 explains why the comparison tool wants that shape.
4. Record what the baseline already knows is wrong: 878 against 846, the two empty sheets, the `Owner` columns.
5. Record the approval in `O60 Governance Decision` if a body gave one.

Signed by. The Chief Enterprise Architect for the first baseline, the Architecture Board (`GB-01`) for every one after.

Done when. `04 Baselines` has exactly one `Current` row; a read-only file exists with that reference in its name; and a flat extract sits beside it.

Shows up in. `01 Version Control`, `Baselines Taken` = 1.

Goes wrong when. You cut the baseline after loading your own data, so you can never show what you inherited. Or two rows read Current, and every later comparison quietly measures against the wrong one.

Step 2.4 — Run `ZZ Checks` and `RPT13 Orphan Analysis` on day one

Produce. Three numbers you can state without opening the file, and a list of defects with a decision against each.

You need. Nothing. This is the first thing you do, before step 2.1 if you are inheriting the repository.

Look it up. Volume 1, chapter 18, *Six numbers that tell you whether the estate is healthy* — the six an architect should be able to quote in a corridor.

Where it lands. Nowhere yet. The readings go into your own notes and then into `03 Change Requests`, one request per genuine defect.

> **`ZZ Checks` — fifteen lines, and how to read them**
>
> Three columns: `Check`, `Result`, `Threshold`. Nearly every threshold is zero, because what a healthy estate shows is nothing. The results: 19 unmanaged interfaces, 20 with no logging, 10 direct database reads, 3 platform components past support, 4 security controls absent, 4 control objectives without evidence, 3 data entities without lineage, 4 work packages past their end date, 0 vacant architecture roles [3]. The closing line is what makes the sheet worth reading: *the thresholds are what a healthy estate would show; the results are what this one does; that distance is the work.*
>
> Three results are green and should not be trusted. `Vacant architecture roles` = 0 because every row of `O10 Actor` reads `Filled` — a statement about the sheet, not the bank. `Records validated against a real organisation` = 0 is the repository declaring its own data illustrative. And `Object types populated` = 65 against "every type carries records" is false: `00 Index` puts `O29 Gap` and `O59 Compliance Assessment` at zero.

How.

1. Read `ZZ Checks` and write down the three worst numbers. For most banks those are unmanaged interfaces, interfaces with no logging, and direct database reads.
2. Count the records yourself. The per-sheet counts in `00 Index` total 846 against a headline of 878 [1][2][3]. Decide which you will quote, and write the decision down.
3. Read `RPT13 Orphan Analysis`. Two orphans, `A13 Data Platform` and `A14 Fraud Detection`, both with the finding "No business service maps to it" [4]. `A13` is marked `Replace` in `O23`, so it is an orphan you are retiring; `A14` is one you are keeping.
4. Then run the count the report does not. For each of the 64 object sheets, ask whether any relationship sheet names it. Thirty-three do not, and seventeen have no inline reference column either — including `O05 Principle`, `O57 Governance Body` and `O58 Governance Gate`.
5. Raise one change request per genuine defect, not one for all of them. A request that says "fix the repository" is never closed.
6. Put the three numbers on the Architecture Board's standing agenda, with last month's beside them.

Signed by. Nobody. This is the architect's own instrument, and `GB-05 Architecture Function` already carries the mandate to maintain the repository.

Done when. You can state the three numbers from memory, and every defect has a change request or a written reason for being left alone.

Shows up in. Itself, next month. A healthy trend is unmanaged interfaces and isolated object types both falling. Nothing in the file computes the trend, which is why step 2.2 came first.

Goes wrong when. The sheet is read once, at the end of a phase, as a formality. By then the defects are six months old. Read it monthly and each one costs ten minutes.

READING YOUR OWN REPOSITORY BEFORE YOU ADD TO IT

Which three numbers on `ZZ Checks` would most embarrass this bank in a supervisory meeting, and who owns each?

Of the seventeen object sheets joined to nothing, which three will you link this quarter, and to what?

Which green number on `ZZ Checks` or `RPT01 Dashboard` is green because the sheet behind it is empty?

Step 2.5 — Tailor the metamodel, and write down what you are not doing

Produce. A three-column decision: of the 64 object sheets, which you populate in year one, which wait, and which stay empty — with a reason against every sheet in the third list.

You need. Steps 2.3 and 2.4, and an honest estimate of architect-days. For four or five people, year one is about sixteen sheets, not sixty-four.

Look it up. Volume 1, chapter 19, *What these are, and what they are not* — why a catalogue that answers no question costs more than it returns.

Where it lands. `03 Change Requests`, one row with `Type` = Restructure, `Scope` listing the object types and `Decision` carrying the three lists. Add a row to `031 Deliverable` for the tailored framework too; the shipped twenty do not include one.

How.

1. Start from the question each sheet answers, not from the sheet. If you cannot name the question, it is year two.
2. For every candidate, name the relationship sheet that will join it to something else. No relationship, no population.

3. Fix year one at around sixteen sheets. A defensible banking set is `001` – `014`, `020`, `023`, `029` – `032`, `057`, `058` and `EXT`.
4. Put the heavy catalogues later. `037 Data Attribute` holds 60 rows joined only by an `Entity` column; `042`, `047` and `052` hold 33 rows between them joined to nothing at all.
5. Write the reason for every sheet you leave empty. "Not needed until Phase D" is a reason; "we never got to it" is what you are preventing.
6. Date the decision and put it in the repository, not a notebook.

Signed by. The Chief Enterprise Architect, noted to the Architecture Board. A scoping decision, not an architecture one.

Done when. Every one of the 64 object sheets is on exactly one of the three lists, inside the file.

Shows up in. `ZZ Checks`, line one. A sheet you deliberately left empty reads zero records, and your successor can find out why in one click.

Goes wrong when. Nobody records the decision, so a successor reads the empty sheets as neglect and fills them. That is how a repository acquires sixty data attributes that nothing references.

THE TAILORING DECISION

Which sheet are you about to populate because it is easy, rather than because you can name the question it answers?

For each sheet you are leaving empty, what reason will a successor accept three years from now?

How many architect-days does year one actually have, and which sheet comes off the list when that number halves?

Step 2.6 — Read `EXT Industry Extension`, and work out why a bank needs six types the core does not have

Produce. A written position on each of the six industry extension types: what it holds, which sheet holds its instances today, and whether that is good enough.

You need. Step 2.5, and volume 1's data chapters, because five of the six are data entities in disguise.

Look it up. Volume 1, chapter 16, *The mapping* — a thousand banking words against forty-two ArchiMate elements, and where the core runs out.

Where it lands. `EXT Industry Extension`: three columns, `Ref`, `Industry extension type`, and `What it holds and why the core metamodel is not enough`, over six rows `X01` to `X06` [5].

Ref	industry extension type	what it holds	where an instance already lives
`X01`	**Product**	terms, pricing, eligibility	O18 Product, 8 rows
`X02`	**Arrangement**	a product held by a customer	O20, E05 Arrangement
`X03`	**Exposure**	amount at risk, by facility	O20, E08 Credit Exposure
`X04`	**Limit**	an authorised ceiling	O20, E10 Limit
`X05`	**Regulatory Report**	a submission, with lineage	O20, E13 Regulatory Report
`X06`	**Payment Scheme**	a clearing arrangement	nowhere. No record exists

Three columns, six rows, no instances. The sheet declares the types; other sheets hold five of them under different names. The sixth is the one a bank least controls.

Figure 2.1 `EXT Industry Extension`, all six rows, with the sheet that already holds each type's instances. Five of the six are recorded elsewhere under a different name. `X06 Payment Scheme` is recorded nowhere — and in a South African bank it is the extension type whose rules the bank least controls.

How.

1. Read the six rows: `X01 Product`, `X02 Arrangement`, `X03 Exposure`, `X04 Limit`, `X05 Regulatory Report`, `X06 Payment Scheme`.
2. Work out why each is there. The TOGAF core has a `Product` but not the *instance* a customer holds — the thing with a balance, a rate, an arrears status and a regulatory treatment. It has no amount at risk to a counterparty, which is the unit Basel III and IFRS 9 compute over, and no ceiling on it. No supervisory submission as an object with lineage. No shared infrastructure the bank participates in without controlling.
3. Find where each type's instances live today. Five already do, under other names: `X01` in `O18 Product`, and `X02`, `X03`, `X04`, `X05` in `O20 Data Entity` as `E05 Arrangement`, `E08 Credit Exposure`, `E10 Limit` and `E13 Regulatory Report`.
4. Notice what is missing. **`X06 Payment Scheme` has no instances anywhere.** Search the file for "scheme" and every hit is a description, never a record.

5. Decide whether each existing home is adequate. For `X03` and `X04` it is usually not: an exposure is a position with a measurement date, and `O20` has no date column.
6. Replace the shipped sentence in the third column with your position. Do not append to it.

Signed by. The Chief Enterprise Architect, with the Chief Risk Officer agreeing `X03` and `X04`, the CFO `X05`.

Done when. Each row names the sheet holding its instances, and `X06` either names a new sheet or states why schemes stay out of scope.

Shows up in. `00 Index`, which counts `EXT` as the sixty-fifth object type and so puts it in the 65 on `ZZ Checks`. Your edit moves no number, which is exactly why this step gets skipped.

Goes wrong when. `EXT` is read as a list of missing sheets and somebody creates six new tabs duplicating five that already hold the data. The extension declares types; it does not demand tabs.

> **THE EXTENSION TYPE WITH NO RECORD ANYWHERE IS THE ONE A BANK CONTROLS LEAST**
>
> A South African bank's payment architecture is written largely outside the bank. SAMOS went ISO 20022 on 17 September 2022, PayShap is ISO 20022 native, Registered Mandates became mandatory on 10 March 2025 with its own 19:00 late collection window, and the rules are now set by the Reserve Bank and PayInc after PASA's recognition was withdrawn in September 2026 [6]. Each is a design constraint no `007 Constraint` row records, and a membership obligation no `019 Contract` row records — `019` holds ten vendor agreements, one per application, with `Counterparty` set to the application's own name. Add the schemes.

Step 2.7 — Extend the metamodel without breaking it

Produce. Either a seventh industry extension type with its object sheet and its relationship sheet, or a written decision that six is enough.

You need. Step 2.6, and the tailoring decision from 2.5 — an extension nobody will populate is worse than none.

Look it up. Volume 1, chapter 13, *The three ways architects misuse one* — the failure modes of adopting a reference architecture are the failure modes of extending a metamodel.

Where it lands. `EXT Industry Extension` for the declaration, a new `O` sheet for the records, a new `R` sheet for the join, and `00 Index` for the count.

How.

1. Test the candidate on three questions. Does the core genuinely lack it? Will it hold more than ten records? Can you name the relationship sheet that will join it? Three yeses or it stays out.
2. Take the next free `X` reference. Never renumber the six that exist.

3. Build the object sheet in the house shape: title row, the `Ardent Union Bank · <domain>` `domain · <n> records` line, a blank, then the header on row 4 with `Ref` in column A and the name in column B.
4. Build the relationship sheet the same way, and name it after what it means, accepting the truncation: `R26 Scheme governs Payment Instruc`.
5. Add the seven record-keeping columns from step 2.2 to both sheets.
6. Update `00 Index` with a row and a count. A type the index does not list is invisible to your successor.

Signed by. The Architecture Board (`GB-01`). A metamodel change engages every domain architect.

Done when. The new type appears in `EXT`, in `00 Index` with a count, in its own object sheet, and in a relationship sheet with real rows.

Shows up in. `00 Index` and `ZZ Checks`, both of which should now read 66 object types. If they still read 65 you added a sheet and not a type.

Goes wrong when. You add the object sheet and not the relationship sheet, and the repository gains its eighteenth isolated catalogue.

Form P-1 — the industry extension position

One row per extension type. Transcribe the last two columns into `EXT Industry Extension`. Six rows as shipped; a seventh only if step 2.7 passes all three tests. Signed by the Chief Enterprise Architect.

Ref	Industry extension type	Instances live in	Adequate?	Action
X01	Product	O18 Product	Example, delete me	Add licence columns

Step 2.8 — Settle one convention for every `Owner` column, and apply it

Produce. A single rule for what an `Owner`, `Head` or `Master system` cell may contain, and a corrected pass over every sheet that has one.

You need. `O11 Role` rebuilt in step 2.1, two hours, and the willingness to leave a cell blank rather than wrong.

Look it up. Volume 1, chapter 10, *Ownership and lineage* — why an owner is a named individual, why a steward is not, and what the difference costs at a supervisory visit.

Where it lands. Everywhere. `O01`, `O02`, `O03`, `O05`, `O20`, `O23` and `O62` all have an `Owner`; `O09` has a `Head`; `O20` also has a `Master system`.

> **One column name, four conventions, no integrity check**
>
> `O20 Data Entity` names ten owners — `Head of Treasury`, `Head of Lending`, `Head of Financial Crime` and seven more — and **not one appears in `O11 Role`** [7]. `O23 Application Component` holds organisation-unit names instead: `A07 Digital Channel` is owned by `Legal`, `A11 General Ledger` by `Customer Service`. `O62 Delivery Risk` names a `Head of Procurement` that exists in no sheet. `O05 Principle` names a governance body, which is at least a real row in `O57`. `O01 Driver` names `Group Executive`, which is a row nowhere.
>
> Nothing on `ZZ Checks` tests any of it.

How.

1. Choose the rule. The one that survives: an `Owner` is a role reference from `O11 Role`, written as reference plus name, exactly as the relationship sheets do it.
2. Extend `O11 Role` to the roles the bank has. Eighteen is not enough; step 3.3 adds five a stakeholder map always misses.
3. Walk every `Owner` column. Where the value resolves to a role, write reference and name. Where it does not, blank it and raise a request.
4. Split `O23`'s single `Owner` into a business owner and a technical owner, both roles.
5. Fix `O20`'s `Master system` to hold an `A` reference. Today `E03 Account` is mastered in `Payment Hub` and `E13 Regulatory Report` in `Loan Origination`.
6. Write the rule into `01 Version Control` so nobody invents a fifth convention.

Signed by. The Chief Enterprise Architect, with the Data Governance Council (`GB-03`) agreeing the data-entity owners — its stated mandate is to designate data owners and resolve disputes [1].

Done when. Every non-blank `Owner` in your year-one set resolves to `O11 Role`, and every `Master system` resolves to `O23`.

Shows up in. `RPT05 Data Ownership`, which reproduces `O20`'s `Owner`, `Master system` and `Lineage` straight through. After this step it stops being a list of plausible titles and becomes a list of people you can telephone.

Goes wrong when. You fill the blanks instead of blanking the wrong ones. A plausible owner is worse than an empty cell, because nobody will ever check it.

Step 2.9 — Adopt, edit or retire the twenty-four principles you were given

Produce. A principles catalogue in which every record has been actively adopted, edited or retired, with a change reference — not a fresh set written from nothing.

You need. Step 2.2 for the change references, and two hours each from the CIO, CDO, CRO, CISO and Chief Compliance Officer.

Look it up. Volume 1, chapter 15, *The Joint Standards, with the numbering got right* — four instruments, three of them numbered "1", and what each obliges a bank to show.

Where it lands. `005 Principle`: `Ref` (`PRN-01` … `PRN-24`), `Principle`, `Category`, `Statement`, `Owner`, `Status`. Five categories — Business 4, Application 4, Data 6, Technology 5, Security 5.

How.

1. Read all twenty-four in one sitting. Half an hour.
2. Apply the motherhood test below. Mark each keep, edit or retire.
3. For an edit, change `Statement` and nothing else first. It is one sentence of consequence, not a policy.
4. Retire by setting `Status`, never by deleting a row.
5. Raise one change request for the pass, with twenty-four rows in `02 Change Log`.
6. Only then write a new principle. A bank needs two the shipped set lacks: one on obligations to payment schemes it does not control, and one on reproducibility of a regulatory figure.

Signed by. The Architecture Board (`GB-01`), whose mandate already includes approving dispensations with an expiry — so it has to own the rules it dispenses from.

Done when. All twenty-four rows carry a `Change Ref` from this pass, and `Status` reflects a decision rather than a load.

Shows up in. Nothing, and that is the finding. **No relationship sheet reaches `005 Principle`,** and there is no principle-impact report. Twenty-four principles govern no requirement and are violated by nothing. Add `R27 Requirement governed by Princ`, or accept that the catalogue is decoration.

Goes wrong when. You write forty new principles from nothing — unobjectionable, unordered, unused. The subtler failure is adopting all twenty-four unchanged, which means you have tested none of them against your own bank.

THE MOTHERHOOD TEST, AND A `STATUS` COLUMN NOBODY ASSESSED

Three questions, and a principle must pass all three. **Would any sane executive argue the opposite?** If not it is a motherhood statement. `PRN-09 Data is an Asset` fails alone; the clause after rescues it — data has an owner, a definition and a lifecycle — because somebody argues the opposite every time a line of business refuses to release a file. **Does it say what to do when two good things conflict?** The shipped set cannot answer: it has no tier or precedence column at all. Add one, or `PRN-06 Technology Independence` and `PRN-17 Control Technical Diversity` will contradict each other in your first platform decision with nothing to break the tie. **Can you name a decision in the last year it would have changed?** If not, it is decoration.

Then look at `Status`. Twenty read `Approved` and four read `Draft` — `PRN-05`, `PRN-10`, `PRN-15`, `PRN-20`. Every fifth row. A pattern, not an assessment. Treat all twenty-four as unassessed.

TESTING THE TWENTY-FOUR PRINCIPLES YOU INHERITED

The row-by-row pass transcribes onto the Preliminary phase form for `005 Principle` in appendix F — keep, edit or retire, with the change reference. This is the thinking that fills it.

Pick three principles. For each, name the decision in the last year it would have changed.

`PRN-06` and `PRN-17` will conflict. Which wins in your bank, and which column records that?

Which principle does your bank break routinely, and who benefits from it being broken?

Step 2.10 — Constitute the five governance bodies, and record what each has refused

Produce. Five bodies with a real cadence, a real membership, and a mandate each has exercised at least once.

You need. Step 2.1, and the existing forums. Most banks have six or seven committees that could do this work and none of them calls itself an architecture board.

Look it up. Volume 1, chapter 5, *The architect's real counterparties* — who decides what in a bank, and which of them will never attend an architecture forum.

Where it lands. `057 Governance Body`: `Ref`, `Governance body`, `Cadence`, `Membership`, `Mandate`. Five rows [1].

> **`057 Governance Body` — five rows, and two things the sheet cannot tell you**
>
> `GB-01 Architecture Board`, monthly: holds `G1` and `G2`, approves dispensations with an expiry. `GB-02 Investment Committee`, quarterly, CFO, CEO, CIO and Chief Enterprise Architect: holds `G0` funding and `G3` benefit confirmation. `GB-03 Data Governance Council`, monthly: designates data owners and resolves disputes. `GB-04 Executive Committee`, monthly: escalation point when bodies cannot agree. `GB-05 Architecture Function`, continuous: publishes criteria, assesses compliance, maintains the repository.
>
> There is no `Standing` or `Last met` column, so the sheet records intent rather than fact. And `GB-05` is a function, not a forum — keep it, because it carries the repository mandate, but do not count it as a body that meets.

How.

1. Map the five rows onto forums that already meet. Renaming an existing committee beats creating a sixth.
2. Add `Standing` and `Last met` columns.
3. For each body, write down the last decision it took that went against somebody. A body that has never refused anything is a mailing list.
4. Check membership against `011 Role`; after step 2.8 every name is a role reference.
5. Agree the escalation path and test it on a live disagreement before you need it.
6. Join the sheet to something: `060 Governance Decision`'s `Body` column holds the body's *name* rather than its `GB-` reference. Fix that and you have the repository's first governance join.

Signed by. The Executive Committee (`GB-04`), the only body that can grant another body a mandate.

Done when. Each row names a forum that has met in the last quarter, with a date, and at least two can point at something they refused.

Shows up in. Nothing. `057` is joined to no relationship sheet and appears in no report. After this step `060 Governance Decision` should resolve to it — seven decisions today, four of them `Outstanding` [2].

Goes wrong when. All five rows are described, nobody meets, and the phase is closed on the strength of the description. That is Ardent Union's position exactly.

Step 2.11 — Adopt the four gates, and write what each one tests

Produce. Four gates with a named holder, a named point in the delivery lifecycle, and an acceptance test that can fail.

You need. Step 2.10, and the delivery lifecycle as it is actually run, from whoever runs it.

Where it lands. `058 Governance Gate`: `Ref`, `Gate`, `When`, `What it tests`. Four rows [8].

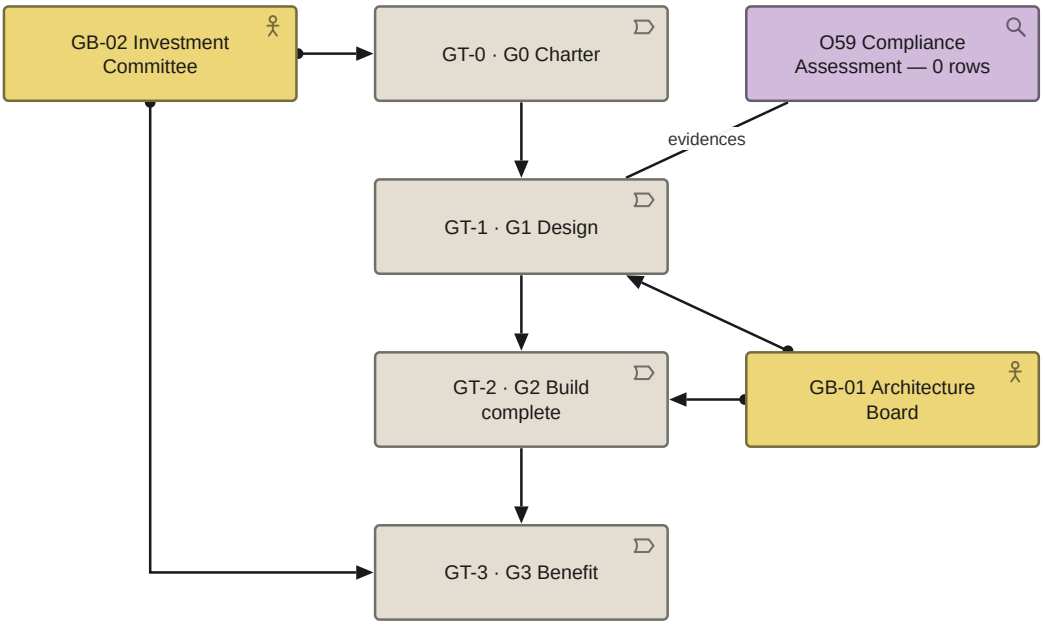

Figure 2.2 The four gates of `058 Governance Gate` in ArchiMate 4. A gate is an `Event` — a point the work either passes or does not — triggering the next; the body that holds it is a `Business Actor` assigned to it. `GB-02` holds `G0` and `G3`, `GB-01` holds `G1` and `G2`, as `057 Governance Body` states in prose. The assessment the gates should write into holds zero rows. Ardent Union Bank is invented.

How.

1. Read the four. `GT-0 G0 Charter` before work starts: scope, acceptance criteria and contract exist. `GT-1 G1 Design` before build: design meets principles, standards and reference architecture. `GT-2 G2 Build complete` before release: every acceptance criterion evidenced. `GT-3 G3 Benefit` after operation: stated benefit confirmed against a date.
2. Match each gate to a point in the delivery process that already stops things. A gate needing a new meeting will not be held.
3. Write the test so it can fail. "Design meets principles" is not testable until the principles have a precedence rule — step 2.9.
4. Add a `Held by` column holding a `GB-` reference. Today the holder is buried in prose in `057`'s `Mandate` column.
5. Agree the dispensation route: who grants one, on what evidence, with what expiry. `CR-G01` is the shipped precedent, a point-to-point file transfer dispensation expiring 2029-06.
6. Record the adoption in `060 Governance Decision`. `DEC-06 Adopt the four-gate regime` already exists and reads `Approved`.

Signed by. The Architecture Board for `G1` and `G2`, the Investment Committee for `G0` and `G3`.

Done when. Every gate names a holder, a lifecycle point, and a test that has failed at least once in rehearsal.

Shows up in. `059 Compliance Assessment`, which is **empty** — zero rows where a working regime would hold one per acceptance criterion per gate, with a `Verdict` and `Evidence`. Your

first real `G1` should put rows in it, and `R24 Compliance Assessment tests` is where the join goes.

Goes wrong when. `G3` is adopted and never held, because benefit confirmation falls due after everyone has moved on. `RSK-07 Benefit not measurable at G3` is already recorded at `Certain` likelihood, which means somebody knew.

Step 2.12 — Stand up the architecture function as an organisation unit and a capability

Produce. An organisation unit, a set of role records and a capability row, so that the architecture function appears in the model it maintains.

You need. Steps 2.1, 2.8 and 2.10.

Look it up. Volume 1, chapter 18, *The three artefacts that buy you credibility* — what to produce in the first ninety days so the function is taken seriously.

Where it lands. `O09 Organization Unit`, `O10 Actor`, `O11 Role`, `O13 Business Capability` and `R22 Org Unit owns Capability`.

How.

1. Add the unit to `O09` with a `Head` of Chief Enterprise Architect. Ardent Union has no architecture unit; the function appears only as `GB-05`.
2. Check the roles. `O11 Role` already holds `ROL-08 Chief Enterprise Architect` and `ROL-09` to `ROL-12`, the four domain architects. That spine is right.
3. Break the actor-role identity. `O10 Actor` and `O11 Role` hold the same eighteen names in the same order and `R21 Role performed by Actor` joins them one to one. An actor is a party, a role is a responsibility; the sheets exist because one party holds several roles and one role outlives several parties.
4. Decide the operating model — centralised, federated, or a small centre with domain architects in the lines — and record it in `O60 Governance Decision`.
5. Add the capability. `O13`'s twelve level-1 rows contain no architecture capability, and it needs one with a `Current` and `Target` you will be measured against.
6. Fill the five `O11 Role` gaps that matter in a bank, which step 3.3 names.

Signed by. The CIO, with the Executive Committee noting it. If the function reports outside technology, the Chief Executive.

Done when. The function has a row in `O09`, a capability row in `O13`, an `R22` join between them, and an actor-to-role mapping that is no longer one to one.

Shows up in. `RPT03 Capability Heatmap`, once the capability has scores. Expect a low current score; the honest figure for a first-year function is 1 or 2.

Goes wrong when. The function is stood up as a reporting line and never modelled — the only part of the enterprise with no capability, no maturity score and no owner, and therefore the first thing cut when budgets move.

Step 2.13 — Run the maturity assessment, and decide where the result lands

Produce. A maturity score per architecture discipline, with evidence against each, in a sheet rather than a slide.

You need. Step 2.12, half a day with each domain architect, and half a day with internal audit.

Look it up. Volume 1, chapter 7, *What the operating model means for architecture* — which disciplines a bank's operating model requires, and at what grade.

Where it lands. This is the problem. **There is no maturity sheet.** `O13 Business Capability`'s `Current` and `Target` columns score business capabilities, not architecture practice. Appendix H holds the instrument and the full column set; the decision here is only where the answer goes.

How.

1. Choose the home. The defensible option is a new `O67 Architecture Maturity` sheet — `Ref`, `Discipline`, `Current`, `Target`, `Evidence`, `Assessed`, `Assessed By` — declared in `00 Index` and governed under step 2.7's rules.
2. The expedient option is rows in `O13` under the architecture capability from step 2.12, reusing `Current` and `Target`. It works, and it mixes two scales in one column. If you take it, record that in `01 Version Control`.
3. Score against evidence, not opinion. "We have a principles catalogue" is not evidence that principles govern anything; step 2.9 showed no sheet reaches `O05`.
4. Score the repository itself as one discipline, using the `ZZ Checks` numbers from step 2.4.
5. Set targets one grade up, not three. Six disciplines moving from 1 to 2 is a good first year.
6. Date the assessment and book the next one twelve months out.

Signed by. The Chief Enterprise Architect, with `ROL-18 Internal Auditor` reviewing the evidence rather than the scores.

Done when. Every discipline has a score, a target and a line of evidence, and the result sits in a sheet with a date on it.

Shows up in. Wherever you put it. None of the thirteen shipped reports reads a maturity score except `RPT03 Capability Heatmap`, and only for business capabilities.

Goes wrong when. The assessment produces a radar chart and is never repeated, so the only thing it establishes is that somebody once had an opinion.

Step 2.14 — Test the motivation layer end to end, and find the missing join

Produce. A motivation layer in which every driver is current, every goal is owned, every objective reaches a measure, and the two halves are joined.

You need. Step 2.2, the board strategy, the last two sets of results, and the finance team, because half the measures are theirs.

Look it up. Volume 1, chapter 1, *The seven things an architect must understand* — the pressures that produce a bank's drivers, and which are permanent. Then volume 1, chapter 6, *The four numbers an executive watches* — because an architecture measure that moves none of them will not be funded.

Where it lands. `O01 Driver` (`Ref`, `Driver`, `Type`, `Description`, `Origin`, `Owner`, `Pressure`), `O02 Goal`, `O03 Objective`, `O04 Measure`, and the three relationship sheets `R08 Requirement derives from Dr`, `R10 Objective measured by Measu` and `R11 Goal realised by Objective`.

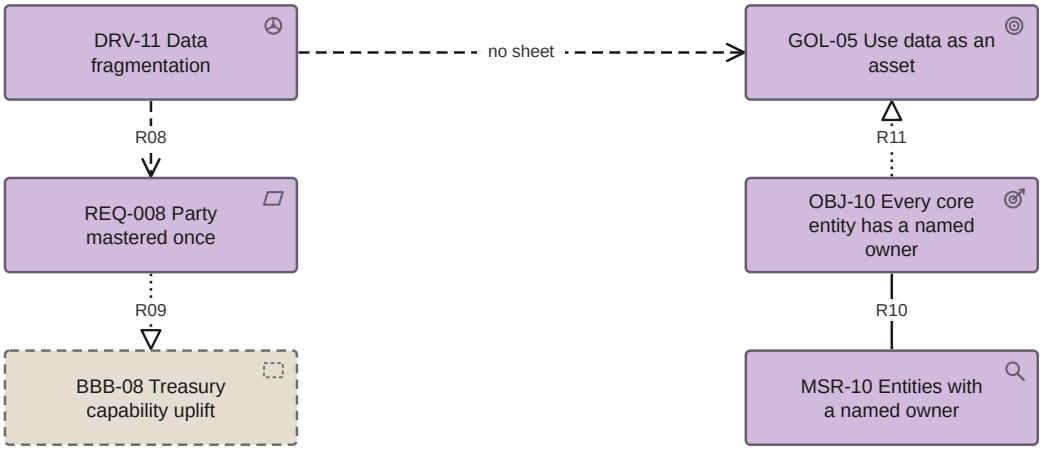

Figure 2.3 The motivation layer as the relationship sheets actually join it. Two chains: a driver influences a requirement (`R08`) which is satisfied by a building block (`R09`), and an outcome realises a goal (`R11`) assessed by a measure (`R10`). Nothing joins a driver to a goal — there is no sheet for it. Step 2.14 adds one.

How.

1. Read the thirteen drivers. Seven are regulatory and external, every one owned by the Chief Compliance Officer at `High` pressure: Basel III/IV, IFRS 9, AML/CFT and sanctions, payment scheme rules, data protection, operational resilience, conduct and fair treatment. Six are internal and business, all owned by `Group Executive` at `Medium`. Seven regulatory drivers at identical pressure owned by one officer is a template, not an assessment — and `Group Executive` is not a role in `O11 Role`.
2. Re-pressure the drivers against what is actually live for a South African bank in 2026: the Joint Standards, CoDI's single customer view, and the post-grey-list mutual evaluation. They do not press equally.

3. Read the six goals. Each is owned by a different executive, which is the one place the shipped data is right. Keep them and check the owners.
4. **Find the missing join.** There is no relationship sheet from a driver to a goal. `R08` joins drivers to requirements, `R11` joins goals to objectives, and nothing connects the two chains. Build `R26 Goal responds to Driver` in the house shape; it takes twenty minutes and is the highest-value sheet of the phase [10].
5. Keep an objective only if you can state the measure that would show it moving. Eleven of the fourteen read `Not started`; delete the ones with no measure rather than carrying them.
6. Re-baseline every measure, add a `Baseline Taken` column with the date, and set a cadence you will keep. Eight monthly measures for a four-person function is seven too many in year one.

Signed by. The Executive Committee (`GB-04`) for drivers and goals — they are the enterprise's statement about itself and the architect does not own them. The Chief Financial Officer for anything with money in it.

Done when. Every driver has a dated `Pressure`, every goal an owning executive role, every objective a measure through `R10`, every measure a unit consistent with its baseline — and a relationship sheet joins drivers to goals.

Shows up in. `RPT06 Requirements Traceability`, which shows each requirement's driver, so a driver you retire leaves requirements pointing at nothing. Nothing reports on measures at all; that is the argument for a fourteenth report, and appendix B says what it should hold.

Goes wrong when. The baselines are left as shipped, somebody quotes `MSR-01` at 23% in a board paper, and a year later nobody can say whether 31% is progress or a different method.

> **Why no baseline in `O04 Measure` can be read**
>
> `MSR-01 Obligations with evidence` is a `%` with a baseline of 23. `MSR-02 Manual regulatory adjustments` is a `count` at 58. `MSR-03 Unit cost to serve` is an `index` at 51. `MSR-06 Cross-sell ratio` is a `ratio` at 67, which is not a ratio. `MSR-13 Recovery objectives tested` is a `count` at 36, against twelve rows in `O49 Resilience Tier`.
>
> Every baseline is a two-digit number between 20 and 67 regardless of unit, and no column records when it was taken. Treat all sixteen as unset. `R10` joins all fourteen objectives to one measure each, cleanly and in order, and that part is worth keeping.

Step 2.15 — Register the deliverables and close the phase against a baseline

Produce. A decision record, a second baseline, a deliverable register that tells the truth, and the first real rows in `O29 Gap`.

You need. Steps 2.1 to 2.14, and one Architecture Board meeting with a decision on the agenda rather than a status update.

Look it up. Volume 1, chapter 18, *What week thirteen should look like* — what a first quarter should have produced, against which to test this phase.

Where it lands. `O31 Deliverable`, `O60 Governance Decision`, `O29 Gap` and `04 Baselines`.

> **`O31 Deliverable` — twenty rows, four columns, and three things it lacks**
>
> `Ref`, `Deliverable`, `Phase`, `Status`. No owner, no due date, no location, no dependency. Thirteen rows read `Approved` and seven `Draft`, and the Approved run from `DLV-01 Architecture Vision` to `DLV-13 Consolidated Gaps` — so the repository claims an approved consolidated gap register while `O29 Gap` is empty. And there is **no row for a Request for Architecture Work**: the register opens at the second document of Phase A rather than the first. Chapter 3, step 3.1 adds it. Add `Owner`, `Due` and `Repository Location` before you move any status.

How.

1. Add the three missing columns and fill them for the Preliminary and Phase A rows.
2. Move a `Status` to Approved only where a location and an approver are both filled. Expect most of the thirteen to come back to Draft, and say so at the Board.
3. Raise the Preliminary gaps in `O29 Gap`: `Ref`, `Gap`, `Domain`, `Building block`, `Severity`, `Status`, `Due`. The first four write themselves — no change control, no principle-to-requirement join, no driver-to-goal join, no payment scheme records.
4. Record the phase decision in `O60 Governance Decision` as a question the Board could have refused.
5. Cut the second baseline, set the first to Superseded, and keep the extract.
6. Check that your three day-one numbers have moved in the right direction.

Signed by. The Architecture Board (`GB-01`), with the Executive Committee noting it.

Done when. `04 Baselines` has exactly one `Current` row dated this month; `O29 Gap` has at least four rows with severities; every `Approved` deliverable has a location; and `O60 Governance Decision` has a new row that is not `Outstanding`.

Shows up in. `RPT07 Gap Analysis`, empty since the file was built and now holding rows. And `ZZ Checks`, where `Work packages past end date` should at last be explicable — four packages read `Past end date` while their `End` dates fall in 2029 to 2031, because the `Delivery` column was typed rather than computed [9].

Goes wrong when. You close the phase while `O57 Governance Body` still describes bodies that have never met and `O59 Compliance Assessment` is still empty. The deliverables exist; the capability does not. That is the exact state of the repository you were handed, and the only thing that distinguishes your Preliminary Phase from Ardent Union's is whether you closed it against evidence.

Before you move to Phase A

THE PRELIMINARY PHASE — A REVIEWER'S QUESTIONS

What has the Architecture Board refused, and where is the refusal recorded? If nothing, `GT-1` does not exist yet, whatever `058 Governance Gate` says.

Which of the five bodies in `057 Governance Body` has met with a quorum in the last quarter, and what is your evidence?

What did this phase raise rather than inherit — which gap, which change request, which number moved on `ZZ Checks`?

Can you reconstruct, from the repository alone, every change made to it since the day you received it? If not, step 2.2 is not finished.

THE CHAPTER IN TEN LINES

- Do step 2.4 first if you inherited the repository: `ZZ Checks` and `RPT13 Orphan Analysis` cost half a day and change the next three steps.
- Add `01 Version Control`, `02 Change Log`, `03 Change Requests` and `04 Baselines` plus seven record-keeping columns before you edit a cell. Prefix your requests `RCR-`, not `CR-`.
- `O09 Organization Unit` holds a generic corporate structure with a Supply & Logistics unit and no Treasury. Replace all twelve rows rather than editing them.
- `EXT Industry Extension` declares six banking object types; five already have instances under other names and `X06 Payment Scheme` has none at all.
- One `Owner` column name holds four conventions. Settle one rule — a role reference from `O11 Role` — and blank what does not resolve.
- No relationship sheet reaches `O05 Principle`, so twenty-four principles govern nothing. The `Status` column reads Draft on every fifth row, which is a pattern, not an assessment.
- `O57 Governance Body` has no `Standing` or `Last met` column, and `O59 Compliance Assessment` is empty. A described gate is not a held gate.
- There is no maturity sheet. Decide where the assessment lands before you run it; appendix H holds the instrument.
- There is no driver-to-goal relationship sheet. The motivation layer is two disconnected chains until you build one.
- Every measure baseline in `O04` is a two-digit number regardless of unit, and no column records when it was taken. Treat all sixteen as unset.

SOURCES FOR THIS CHAPTER

- **1.** Banking_TOGAF_Metamodel.xlsx, sheet O57 Governance Body
- **2.** Banking_TOGAF_Metamodel.xlsx, sheets O60 Governance Decision and O61 Governance Change Request
- **3.** Banking_TOGAF_Metamodel.xlsx, sheet ZZ Checks
- **4.** Banking_TOGAF_Metamodel.xlsx, sheets RPT13 Orphan Analysis and O23 Application Component
- **5.** Banking_TOGAF_Metamodel.xlsx, sheet EXT Industry Extension
- **6.** South African Reserve Bank, national payment system pages — SAMOS ISO 20022 migration, PayShap, Registered Mandates and the withdrawal of PASA's recognition https://www.resbank.co.za/en/home/what-we-do/payments-and-settlements
- **7.** Banking_TOGAF_Metamodel.xlsx, sheets O20 Data Entity, O23 Application Component, O62 Delivery Risk and O11 Role
- **8.** Banking_TOGAF_Metamodel.xlsx, sheet O58 Governance Gate
- **9.** Banking_TOGAF_Metamodel.xlsx, sheet O30 Work Package
- **10.** The Open Group, TOGAF Standard, 10th Edition — Preliminary Phase https://pubs.opengroup.org/architecture/togaf10-doc/adm/

CHAPTER 3

Architecture Vision

Two documents and one signature. This repository holds all three Phase A deliverables marked approved, and no mandate behind any of them.

Phase A is where the architecture gets a mandate or quietly fails to. It produces two documents and one signature: a vision somebody senior will defend, a statement of work saying what is and is not in scope, and an agreement that turns an opinion into authority. Twelve steps, four to six weeks, and most of the effort goes on conversations rather than sheets [6].

Ardent Union Bank is invented and its figures are illustrative — but its Phase A record is worth reading, because the repository claims the phase is finished. `O31 Deliverable` shows `DLV-01 Architecture Vision`, `DLV-02 Statement of Architecture Work` and `DLV-03 Stakeholder Map` all `Approved` [1]. There is no row for a Request for Architecture Work at all, `O29 Gap` is empty, `O59 Compliance Assessment` is empty, and `DEC-01` to `DEC-04` in `O60 Governance Decision` still read `Outstanding` [2]. Three approved Phase A deliverables, no mandate behind them and no gate record in front of them. That is the most common shape of a failed Phase A: the documents exist and nobody is bound by them.

> **BEFORE YOU START THIS PHASE**
>
> Phase A is conversation. These let you hold it without an interpreter.
>
> - **Volume 1, chapter 5, *The architect's real counterparties*.** The people whose agreement you need.
> - **Volume 1, chapter 4, *The canvas for a full-service bank*.** The nine blocks, already filled for a bank this size.
> - **Volume 1, chapter 2, *The twelve lines*.** What each line of business means in practice, and which three are not lines at all.
> - **Volume 1, chapter 17, *The motivation view: why this bank is changing*.** A driver-to-estate argument, worked end to end.

The shape of the phase

Steps	What they settle
3.1	The mandate, and what to do when nobody will write one
3.2 – 3.4	Who the stakeholders are, which ones are missing, and what each one is afraid of
3.5 – 3.6	The business model and the value streams, named before capabilities
3.7	Scope: breadth, depth, horizon and domains
3.8 – 3.9	Constraints and assumptions, with owners and expiry dates
3.10 – 3.12	The vision, the statement of work, and the gate

Step 3.1 — Get a Request for Architecture Work, or write one and have it signed

Produce. A one- or two-page request naming the sponsor, the business problem, the scope, the time box and the authority being granted.

You need. A sponsor. The strategy document. The drivers and goals from step 2.14, because the request should quote two or three of them by reference.

Look it up. Volume 1, chapter 18, *Week one: eight documents, and where they live* — what already exists in the bank that you can quote instead of asking for.

Where it lands. `031 Deliverable`, in a row that does not exist yet.

> **`031 Deliverable` has no row for the document that starts the ADM**
>
> The twenty shipped rows run `DLV-01 Architecture Vision` to `DLV-20 Requirements Catalogue`, and the register begins with the *second* document of Phase A. Add `DLV-21 Request for Architecture Work` with `Phase` = "Preliminary / A" — it is raised in Preliminary and consumed in A — and use the `Owner`, `Due` and `Repository Location` columns you added in step 2.15. A deliverable with no location is a claim.
>
> Do not renumber to put it first. The references are matched as text and `DLV-01` is quoted in your own notes already.

How.

1. Ask the sponsor for it once, in writing, with a template attached.
2. When that produces nothing — and in most banks it does — write it yourself, in their words, in under five hundred words.
3. Quote the drivers and objectives by reference. `DRV-10 Legacy estate` with `MSR-07 Components within vendor support` at 21% says more than two paragraphs.
4. State what you will not do. A request with no exclusions is read as a blank cheque and then resented.

5. Put it in front of the sponsor as a document to sign rather than to write. Signature, date, and a named deputy.
6. Register it with `Status` = Approved and the signature location filled.

Signed by. The Chief Executive, or the executive who owns the problem. Not the CIO on the CEO's behalf, unless the CIO is the sponsor.

Done when. A signed document exists, the `O31` row is Approved with a location, and you can name the person who would answer for it in a board meeting.

Shows up in. `O31 Deliverable`. It is the only register this repository has, and it has no dependency column — so write the dependency into the `Note` you added, or `DLV-01` and `DLV-02` will again read Approved with nothing behind them.

Goes wrong when. You start Phase A without one, on the reasoning that everybody knows what you are doing. Six weeks later the scope is contested, there is no document to point at, and the argument is held in a steering committee by people who were not in the original conversation. Write the two pages.

WHEN NOBODY WILL SIGN

Three fallbacks, in order of preference. **One:** get the request minuted. A decision in `O60 Governance Decision` with a `Body` and a `Status` that is not `Outstanding` is weaker than a signature, but it is evidence. **Two:** attach the architecture work to a funded work package that already has approval — `WP-01 Governance foundations` carries `Approval` = "In flight" at $7.7m and is the obvious host, though note its `Delivery` reads "Past end date" against an `End` of 2030-05, which tells you the column was typed rather than computed [3]. **Three:** write the request, send it, and record the absence of a reply with the date you asked. That is not cynical. It is the difference between a phase that is blocked and a phase that is late.

Step 3.2 — Build the stakeholder map, and break the actor-role identity

Produce. A stakeholder map in which an organisation unit, a party and a responsibility are three different things.

You need. Step 2.1 and step 2.12. The organisation chart, the committee terms of reference, and an hour with the company secretary.

Look it up. Volume 1, chapter 5, *The org chart will not help you* — why the reporting line tells you almost nothing about who decides.

Where it lands. `O09 Organization Unit`, `O10 Actor` (`Ref`, `Actor`, `Type`, `Status`), `O11 Role` (`Ref`, `Role`, `Type`, `Performed by`) and `R21 Role performed by Actor`.

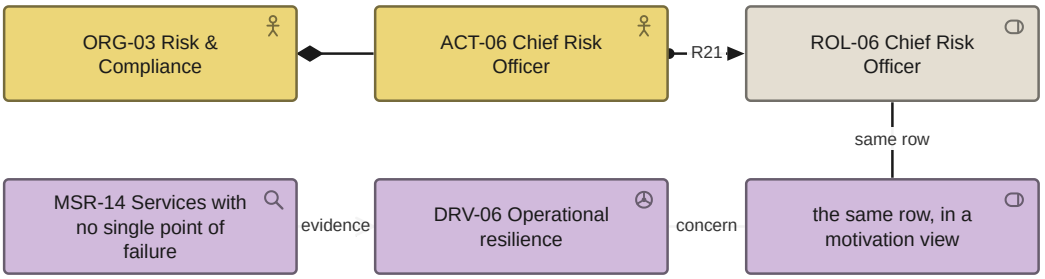

Figure 3.1 Three sheets, four ArchiMate 4 elements. An organisation unit is composed of actors, an actor is assigned to a role (`R21`), and the same `O11 Role` row is drawn as a `Stakeholder` in a motivation view, where the concern it holds is a `Driver` and the evidence is a `Measure` . In the repository as shipped, `O10 Actor` and `O11 Role` hold the same eighteen names and `R21` joins them one to one. Ardent Union Bank is invented.

How.

1. Look at what you were given. `O10 Actor` and `O11 Role` hold **the same eighteen names in the same order**, and `R21` joins them one to one: `ROL-01` to `ACT-01` , `ROL-02` to `ACT-02` , and so on [4]. `Performed by` reads `Person` on all eighteen rows, which tells you nothing.
2. Separate the two. An actor is a party — a person, a team, a committee, an outsourced provider. A role is a responsibility that outlives whoever holds it. The sheets exist because one party holds several roles and one role passes between parties.
3. Rebuild `O10 Actor` with the parties that matter, including the ones outside the bank: the scheme operator, the credit bureau, the external auditor, the managed service provider.
4. Rebuild `O11 Role` with responsibilities, and use the `Type` column for the grouping that is useful to you — Executive, Architecture, Delivery, Operations, Data, Security, Assurance are the shipped values and they are serviceable.
5. Rebuild `R21` so that it is no longer a diagonal. A vacancy is an `ACT-` row with `Status` set accordingly, which is what makes the `Vacant architecture roles` check on `ZZ Checks` worth anything.
6. Join the units to the actors. There is no relationship sheet from `O09` to `O10` , so either add one or accept that the organisation and its people are two unconnected lists.

Signed by. The Chief Executive's office, through the company secretary. A stakeholder map nobody senior has read is a list of names.

Done when. `O10` and `O11` hold different numbers of rows, `R21` is not one to one, and every `ACT-` row has a `Status` that somebody checked this month.

Shows up in. `ZZ Checks` , `Vacant architecture roles` . It reads 0 today because every `O10` row says `Filled` . In a real bank in its first architecture year it will not read 0, and the honest number is the useful one.

Goes wrong when. You accept the one-to-one mapping because it looks tidy. The first reorganisation then breaks both sheets at once, and nobody can tell which half was wrong.

Step 3.3 — Add the five stakeholders a bank's map always misses

Produce. Five role records, five actor records and five named concerns that were not in the repository.

You need. Step 3.2. An introduction to each of the five, which is usually easier to get than you expect, because none of them has ever been asked.

Look it up. Volume 1, chapter 5, *Treasury and markets* and *Finance and regulatory reporting* — what these functions do, and why their systems are nobody's priority until they fail.

Where it lands. `O10 Actor` and `O11 Role`, five new rows each.

Who	What they hold that nobody else does	In the repository today
The treasurer	The funding and liquidity position, the rate view, and the system that holds both	`A10 Treasury Management` reads `Replace` and `Poor` health. `O20` names a `Head of Treasury` as owner of `E07 Payment Instruction`. **No role record**
The chief risk officer	Capital, credit and operational risk appetite, and the veto on anything that moves it	`ROL-06` exists. `C10 Risk & Capital Management` is `Criticality` = Low, which is wrong for a bank
The head of compliance	Every obligation in `O63 Control Objective`, four of which have no mechanism	`ROL-07 Chief Compliance Officer` exists, and owns all seven regulatory drivers in `O01`
The regulatory reporting manager	The BA return calendar, and the lineage from report to source	`A12 Regulatory Reporting` and `E13 Regulatory Report` exist. **No role record**
The payments scheme representative	The bank's obligations to SAMOS, PayShap, Registered Mandates and the card schemes	`DRV-04` and `CO-04` name scheme rules; `CO-04` reads `No mechanism`. **No role record, and no scheme record**

How.

1. Add the three missing roles and their actors. Two of the five — the CRO and the compliance officer — are already in `O11 Role`; the treasurer, the reporting manager and the scheme representative are not.
2. Add the one the repository does not mention at all: the POPIA Information Officer. The role is a statutory one and the obligations in `CO-05 Data protection`, which reads `In breach`, are theirs.
3. For each of the six, write one sentence: what would make them say no. That sentence is their concern and it goes into the vision.
4. Check their systems. `A10 Treasury Management` is `Replace` with `Poor` health; `A12 Regulatory Reporting` is `Retain` with `Adequate`. Both are `Critical` or near it and neither is in a work package.
5. Correct `C10 Risk & Capital Management`, which carries `Criticality` = Low against a `Current` of 4 and a `Target` of 5. A bank cannot have low-criticality capital management.

6. Put all six on the invitation list for step 3.10, and record who declined.

Signed by. The Chief Enterprise Architect, with each new role confirmed by its own executive.

Done when. Every one of the six has a row in `O10` and `O11`, a written concern, and a diary entry.

Shows up in. `RPT09 Regulatory Coverage`, indirectly. Four of its seven obligations read `No mechanism` or `In breach` — `CO-01 Basel III / IV`, `CO-04 Payment scheme rules`, `CO-05 Data protection`, `CO-07 Conduct and fair treatment` [5]. Three of those four belong to a stakeholder the map did not have.

Goes wrong when. The map is built from the technology organisation outwards, so it contains every architect and no treasurer. The treasury system is then the one nobody defended, and it is the one that fails during a liquidity event.

BEFORE YOU FIX THE STAKEHOLDER LIST

Who in this bank can stop your work without attending a single architecture meeting, and have you met them?

For each of the six roles in step 3.3, write the one sentence that would make them say no.

Which stakeholder's system is `Critical` in `O23` and in no work package in `O30`?

Form A-1 — the stakeholder and concern register

One row per stakeholder. The first three columns transcribe into `011 Role`; `Concern` has no column in the repository, so it goes into the Architecture Vision and into `001 Driver` where it is a real pressure. Twelve to twenty rows for a first pass. Signed by the Chief Enterprise Architect.

ROL-	Role	Type	Actor (ACT-)	Concern, one sentence	Decides or advises
ROL-19	Head of Treasury	Executive	ACT-19	Example, delete me	Decides

Step 3.4 — Rank the stakeholders, and record what each one is measured on

Produce. A short list of the stakeholders whose agreement the vision needs, each with the number they are measured on.

You need. Step 3.3. The last two sets of published results, and the executive scorecards if you can get them.

Look it up. Volume 1, chapter 5, *Four questions* — four questions that work on any role in a bank and produce something usable in twenty minutes.

Where it lands. Nowhere directly. There is no stakeholder-concern column anywhere in the repository, which is the honest finding: `001 Driver` has `Origin` and `Owner`, and that is the closest the model gets.

How.

1. Sort the list into three groups: decides, is consulted, is informed. Only the first group gets a meeting.
2. Against each name in the first group, write the measure they are judged on. Match it to an `MSR-` row where you can; `MSR-03 Unit cost to serve` belongs to the CFO, `MSR-13 Recovery objectives tested` to the CRO.

3. Where a stakeholder's measure is not in `O04 Measure`, add it. A concern with no measure is a preference.
4. Record each concern as a driver if it is a genuine pressure on the bank, not merely on the person. `O01 Driver` is the right home and `Pressure` is the right column.
5. Name the one stakeholder most likely to refuse, and write down what you will offer them.
6. Book the meetings before you write a word of the vision.

Signed by. Nobody. This is the architect's own working list and it should not be circulated.

Done when. Every name in the "decides" group has a measure, and at least one concern has become a row in `O01 Driver`.

Shows up in. `O01 Driver` and `O04 Measure`. After this step the driver sheet should no longer read seven regulatory drivers at identical pressure owned by one officer.

Goes wrong when. Everybody is in the "decides" group, so the vision is written to twelve audiences and persuades none.

Step 3.5 — Draw the business model canvas, and find the four blocks with no home

Produce. A one-page canvas for the bank, and a list of the blocks the repository cannot hold.

You need. Step 3.4. Volume 1, chapter 4, which has already drafted a canvas for a full-service bank — start from it and argue with it.

Look it up. Volume 1, chapter 4, *The canvas for a full-service bank* — the nine blocks filled in, with the paragraph on what the canvas leaves out.

Where it lands. Mostly nowhere, and that is the point of the step.

Canvas block	Where it lands in the repository
Value propositions	`018 Product`, eight rows from `PRD-01 Current Account` to `PRD-08 Treasury Deposit`
Channels	`C12 Channel & Distribution`, `A07 Digital Channel`, `LOC-03 Contact Centre`, `LOC-04 Branch Network`
Customer relationships	`015 Business Service`, `Exposure` column (Internal or External)
Key activities	`013 Business Capability` levels 1 and 2, `016 Business Process`
Key resources	`023 Application Component`, `027 Physical Technology Compone`
Key partners	`019 Contract`, which holds ten vendor agreements and nothing else
Customer segments	**Nowhere.** No segment sheet, no segment column on `018`
Revenue streams	**Nowhere.** No money on any product or capability row
Cost structure	**Nowhere** except `030 Work Package Cost`, which is programme cost, not run cost

How.

1. Fill the five blocks the repository supports from the sheets, not from a workshop. They are already written down.
2. Fill the four it does not from the annual report and the segmental disclosure. A bank publishes its segments, its revenue lines and its cost-to-income ratio every six months.
3. Decide whether to extend the metamodel. A `Segment` column on `018 Product` and a `Line of Business` column are the two highest-value additions, and step 2.7 gives the rules.
4. Do not add a revenue column to `013 Business Capability`. Revenue belongs to a product and a segment, not to a capability, and putting it there is how a capability map becomes a budget argument.
5. Test the canvas against the drivers. If `DRV-08 Cost pressure` is real, the cost structure block should show where.
6. Keep the canvas as a deliverable, not a sheet. Add it to `031 Deliverable` with a location.

Signed by. The Chief Commercial Officer or the head of strategy, whose canvas it actually is.

Done when. All nine blocks are filled, four of them from published sources, and the two metamodel extensions are either made or recorded as declined.

Shows up in. Nothing. No report reads the canvas, and nothing in the thirteen would notice its absence. It earns its place by being the artefact the sponsor recognises.

Goes wrong when. The canvas is drawn in a workshop from memory, so it describes the bank the executives would like rather than the one in the results booklet.

Step 3.6 — Name the value streams, and reconcile them against the relationship sheet

Produce. Five to seven value streams with stages you can list, each joined to the capabilities it actually uses.

You need. Step 3.5. Two hours with the chief operating officer, and the five shipped value streams as a starting point.

Look it up. Volume 1, chapter 16, *Capabilities and value streams* — which is which, and why naming the stream before the capability saves a month.

Where it lands. `O14 Value Stream` (`Ref`, `Value stream`, `Scope`, `Stages`, `Capabilities`) and `R02 Value stream uses Capabilit`.

> **O14 and R02 disagree, and the relationship sheet wins**
>
> The five shipped streams are good banking streams: `VS01 Prospect to Customer` (4 stages), `VS02 Apply to Fund` (5), `VS03 Initiate to Settle` (6), `VS04 Arrears to Resolution` (4), `VS05 Trade to Report` (5). Keep the names.
>
> Then look at the two places the capabilities are recorded. `O14`'s own `Capabilities` column reads `C01, C02, C03` against `VS04 Arrears to Resolution`, while `R02 Value stream uses Capabilit` joins `VS04` to `C04`, `C05` and `C06`. They cannot both be right, and `R02` is the one the reports read. The inline column is a stale denormalisation — blank it or compute it.
>
> Then look at the shape of `R02`: a three-wide sliding window, `VS01` to `C01`–`C03`, `VS02` to `C02`–`C04`, up to `VS05` to `C05`–`C07`. So `VS05 Trade to Report` is recorded as using Payments, Deposits and Lending, and not using `C08 Treasury & Liquidity`, `C10 Risk & Capital Management` or `C11 Finance & Regulatory Reporting`. All fifteen rows need rebuilding.
>
> And the `Stages` column holds a count, not a list, while `O16 Business Process` holds exactly four processes per stream regardless — so `VS03`'s six declared stages have four processes behind them.

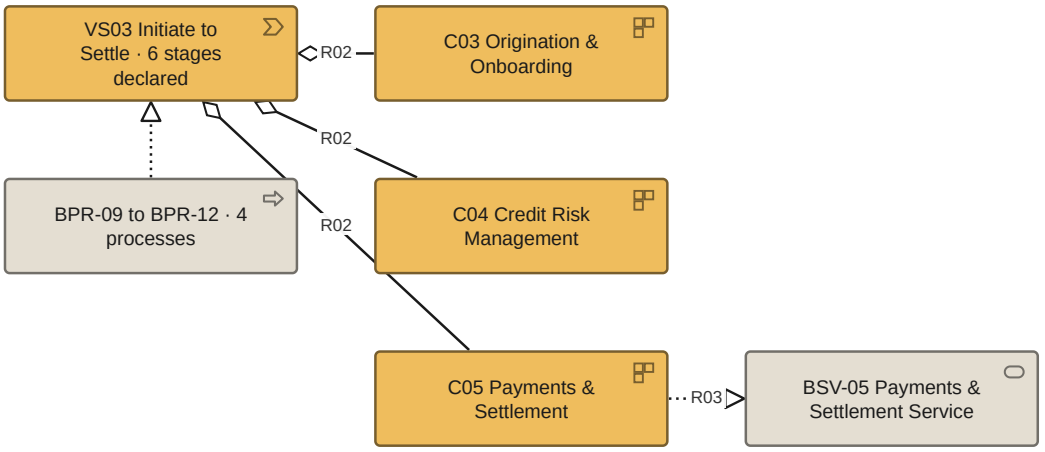

Figure 3.2 `VS03 Initiate to Settle` as `R02 Value stream uses Capabilit` joins it. The stream declares six stages in `O14` and has four processes behind it in `O16`; its inline `Capabilities` column and `R02` name different capabilities. The relationship sheet is the one the reports read.

How.

1. Keep the five names and test them against the twelve lines of business. A bank with a card book and a merchant acquiring book usually needs a sixth stream for disputes and chargebacks.
2. Replace `Stages` with a real list, or add a `Stage` sheet. A count is not a value stream.
3. Rebuild `R02` from the room, one stream at a time, by asking which capabilities the stream could not run without.
4. Blank `O14`'s `Capabilities` column rather than correcting it, and record why in `02 Change Log`.
5. Check `O16 Business Process` against the new stage lists and raise the mismatch as a gap.
6. Leave the capability map itself alone until chapter 4. Value streams first is the whole reason this step is in Phase A.

Signed by. The Chief Operating Officer. Value streams cross organisational boundaries and only the COO sits across all of them.

Done when. Every stream has a stage list, `R02` has been rebuilt from a conversation rather than a pattern, and the inline `Capabilities` column no longer contradicts it.

Shows up in. `RPT03 Capability Heatmap`, once chapter 4 uses these streams to set `Criticality`. A capability that no value stream uses should not be `Critical`, and today `C05 Payments & Settlement` is `Critical` while `C11 Finance & Regulatory Reporting` is also `Critical` with no stream reaching it.

Goes wrong when. You accept the shipped `R02` because it is populated. Fifteen rows of a sliding window look exactly like fifteen rows of analysis.

Step 3.7 — Set the scope in four dimensions, and write the exclusions down

Produce. Four scope statements — breadth, depth, horizon, domains — each with what is out and why.

You need. Steps 3.1 and 3.6.

Look it up. Volume 1, chapter 2, *The twelve lines* — and the section after it on the three things you will be told are lines and are not.

Where it lands. The Statement of Architecture Work in step 3.11. No sheet holds a scope statement.

Figure 3.3 The four scope dimensions, with where each is already half-answered in the repository and what it currently says. All four belong in the Statement of Architecture Work, exclusions included.

How.

1. Breadth: name the lines of business and the legal entities. Use the licence columns you added in step 2.1.
2. Depth: state the lowest level you will model in each domain. Level 2 capabilities, level 1 processes and application components is a defensible first cycle; the shipped `013` goes to level 3 with templated names and it is no use to you.
3. Horizon: pick a date, not a duration. `003 Objective` horizons run 12 months to 3 years and `066 Architecture Increment` runs to month 56, which is four and a half years. Those two cannot both describe the same target.
4. Domains: all four, or say which you are deferring. Deferring technology in a bank with three components past vendor support is not available to you.
5. Write the exclusions as sentences with reasons. "Markets and investment banking are excluded because they run on a separate licence and a separate platform" is a scope statement; "out of scope" is not.
6. Get the exclusions agreed by the executive who owns each excluded area, in writing.

Signed by. The sponsor from step 3.1, with each excluded area's executive acknowledging.

Done when. Four statements exist, every exclusion has a reason and an acknowledgement, and the horizon is a date.

Shows up in. `066 Architecture Increment` and `053 Transition Architecture` in chapter 9, which have to fit inside the horizon you set here. Six increments to month 56 against a three-year horizon is the mismatch to resolve now, on paper, rather than in Phase F.

Goes wrong when. Scope is set by breadth alone. Three months later the capability map is at level 4 in one domain and level 1 in another, and the gap analysis cannot run.

SETTING THE SCOPE

Write your horizon as a date. What is true on that date that is not true today?

Which excluded area will send you work anyway, and which executive has acknowledged the exclusion in writing?

At what level do you stop modelling capabilities this cycle, and what makes level 3 tempting?

Step 3.8 — Record the constraints, with an owner and an expiry

Produce. A constraint catalogue in which every row has an owner, a date and a condition under which it stops being true.

You need. Step 3.7. The finance business partner, the delivery lead, and the operations calendar.

Look it up. Volume 1, chapter 15, *What the regulator actually asks to see* — because half a bank's real constraints are supervisory and arrive as dates.

Where it lands. `007 Constraint`: `Ref`, `Constraint`, `Description`, `Type`. Eight rows, four columns, no owner and no date.

How.

1. Read the eight. They are a sound set: `CON-01 Flat technology budget`, `CON-02 Delivery capacity unrecorded`, `CON-03 Legacy platform skills scarce`, `CON-04 Regulatory change in flight`, `CON-05 Data residency`, `CON-06 Change freeze windows`, `CON-07 Vendor roadmap dependency`, `CON-08 Union and works council consultation`.
2. Add `Owner`, `Raised`, `Review` and `Falsified by` columns. A constraint with no expiry becomes folklore; `CON-02 Delivery capacity unrecorded` is a constraint that one measurement would remove.
3. Add the banking constraints the set is missing. The settlement window and the batch clock constrain every design in the payment estate. The BA return calendar constrains every

change to the reporting chain. Exchange control constrains cross-border flows and no system can infer it.

4. Distinguish a constraint from a decision. `CON-05 Data residency` is a constraint if a statute says so and a decision if the bank chose it; the distinction decides who can lift it.
5. Join the sheet to something. No relationship sheet reaches `O07 Constraint`, so a constraint cannot be shown to bind a work package. Add `R28 Work Package limited by Const` or accept that the catalogue is advisory.
6. Review the whole sheet at every gate, not once a year.

Signed by. The owning executive for each constraint, which is the point of adding the column.

Done when. Every row has an owner, a review date and a falsification condition, and at least three banking-specific constraints have been added.

Shows up in. Nothing. `O07 Constraint` appears in no report and in no relationship sheet, which is precisely why constraints get forgotten between Phase A and Phase E.

Goes wrong when. The constraints are copied into the vision as a paragraph of caveats, so nobody can tell later which ones were real. Or `CON-02` is left as a constraint for three years when a week of measurement would have retired it.

Step 3.9 — Test the assumptions, and give each one a falsification date

Produce. Six to ten assumptions, each with an owner, a test, and a date by which it will be known to be true or false.

You need. Step 3.8.

Look it up. Volume 1, chapter 14, *The corrections — get these right, because the rest of the market gets them wrong* — fifteen things the market believes about banking technology that are not so, every one of which is somebody's assumption.

Where it lands. `O08 Assumption`: `Ref`, `Assumption`, `Status`. Three columns, six rows, and `Status` holds `Open` or `At risk`.

How.

1. Read the six. `ASM-01 Executive sponsorship holds for the programme duration`, `ASM-02 Data owners can be designated within two quarters`, `ASM-03 Vendor selections complete before the dependent phase` (`At risk`), `ASM-04 Delivery capacity can be staffed to plan` (`At risk`), `ASM-05 Regulatory interpretation remains stable`, `ASM-06 Existing licences permit the target deployment model`.
2. Add `Owner`, `Test` and `Resolve By` columns. An assumption with no date is a hope.
3. Write the test so somebody else could run it. `ASM-06` is tested by reading two vendor agreements in `O19 Contract`, and `CTR-01` expires 2027-01, which makes it urgent rather than open.

4. Pair each assumption with the delivery risk it becomes if it fails. `O62 Delivery Risk` already holds the pairs: `ASM-02` fails into `RSK-02`, `ASM-03` into `RSK-01`, `ASM-04` into `RSK-03`. Record the pairing; no relationship sheet does it for you.
5. Add the banking assumptions the set is missing, starting with the one every roadmap rests on: that the core platform will still be supported for the life of the plan. `PTC-01 Core banking mainframe` has a `Support end` of 2028-05 against work packages running to 2031.
6. Review at every gate and close each one as proven or disproven, never as "still open".

Signed by. The Chief Enterprise Architect, with each assumption's owner accepting the test.

Done when. Every assumption has an owner, a test, a resolve-by date and a paired risk.

Shows up in. `O62 Delivery Risk`, where `RSK-01 Integration product not selected` already reads `Certain` likelihood and `High` impact. An assumption whose paired risk is `Certain` is not an assumption. It is a fact you have not accepted.

Goes wrong when. The sheet is filled in Phase A and never reopened, so the roadmap in Phase F rests on six statements nobody has tested in two years.

CONSTRAINTS AND ASSUMPTIONS

Which constraint in `007` would one week of measurement retire, and who has to authorise the week?

For each of the six assumptions, write the test and the date it resolves by.

`PTC-01 Core banking mainframe` loses vendor support in 2028-05 and work packages run to 2031. Which is wrong?

Step 3.10 — Write the Architecture Vision

Produce. A document of eight to twelve pages that a sponsor will defend in front of the board without you in the room.

You need. Steps 3.1 to 3.9. One drawing, which should be a motivation view.

Look it up. Volume 1, chapter 17, *The motivation view: why this bank is changing* — the same argument, worked end to end for an invented bank, with the second draft that fixed the first.

Where it lands. `031 Deliverable`, `DLV-01 Architecture Vision`, which currently reads `Approved` with no location. Move it to Draft, then earn the Approved back.

How.

1. Open with the business problem in the sponsor's words, quoting two drivers and one measure. Not with the ADM.
2. Give the target in one picture. A motivation view — drivers, goals, outcomes, one or two measures — says more than a layered diagram at this stage, and the repository has the content after step 2.14.
3. State the scope and the exclusions from step 3.7, in full, early, and in the same words as the statement of work.
4. Name the stakeholders and their concerns from step 3.3. Six names with one sentence each is the section people actually read.
5. Give the value to the bank in terms of the four numbers an executive watches, not in architecture terms.
6. Close with what you are asking for: the decision, the people, the time box, and the date of the gate.

Signed by. The sponsor, and noted by the Executive Committee (`GB-04`).

Done when. The sponsor has presented it without you, and `DLV-01` carries a location, an approver and a date.

Shows up in. `031 Deliverable`. Expect `DLV-01` to sit at Draft for several weeks; a vision approved in a fortnight was not read.

Goes wrong when. The vision describes the architecture rather than the change, so the board agrees with it and funds nothing. Or it contains a number nobody counted — which is what `MSR-01` at 23% will be until step 2.14 is finished.

Step 3.11 — Write the Statement of Architecture Work

Produce. A short contract: scope, deliverables, roles, time box, gates, and what the architecture function will and will not do.

You need. Step 3.10. The deliverable list from `031` and the gate definitions from `058`.

Look it up. Volume 1, chapter 7, *What the operating model means for architecture* — which parts of the operating model the function can realistically govern in its first cycle.

Where it lands. `031 Deliverable`, `DLV-02 Statement of Architecture Work`, and `033 Architecture Contract`.

How.

1. List the deliverables by `DLV-` reference with a date against each. Phase A through Phase D is seven or eight rows, not twenty.
2. Name the roles by `ROL-` reference, including the business roles whose time you need.
3. Name the gates by `GT-` reference and the body that holds each, which step 2.11 settled.
4. State the time box and the review points. A cycle longer than nine months will be overtaken by the bank.
5. State what you will not do: no solution design, no vendor selection, no project management. This paragraph prevents most of the scope creep in the next six months.
6. Record it as an architecture contract. `O33 Architecture Contract` holds eight rows, every one against a work package and none against the architecture work itself. Add one, with `Compliance status` = Not yet assessable.

Signed by. The sponsor and the Chief Enterprise Architect, with the Investment Committee (`GB-02`) noting it, since it holds `G0` .

Done when. `DLV-02` carries a location and two signatures, and `O33` has a row for this cycle.

Shows up in. `O33 Architecture Contract` and, in chapter 10, `O59 Compliance Assessment` — which is where the acceptance criteria in this document get tested.

Goes wrong when. The statement of work lists deliverables and omits the exclusions, so the function spends Phase B doing solution design for the one programme that shouted loudest.

Step 3.12 — Hold the Phase A gate

Produce. A decision record, a baseline, and the first entries in the compliance assessment sheet.

You need. Steps 3.1 to 3.11, and the Investment Committee's agenda.

Look it up. Volume 1, chapter 13, *What a reference architecture is actually for* — because the gate will ask which reference architecture you are conforming to, and `O47 Reference Model Entry` holds eleven candidates nobody has chosen between.

Where it lands. `O58 Governance Gate` `GT-0` , `O59 Compliance Assessment` , `O60 Governance Decision` , and `04 Baselines` .

How.

1. Put `GT-0 G0 Charter` to the Investment Committee as the gate it is defined as: scope, acceptance criteria and contract exist.
2. Write one `O59 Compliance Assessment` row per acceptance criterion, with `Gate` = `GT-0` , a `Verdict` and `Evidence` . The sheet has been empty since the file was built; these are its first rows.
3. Record the decision in `O60 Governance Decision` with a `Status` that is not `Outstanding` . Four of the seven shipped decisions still read `Outstanding` , including `DEC-01 Adopt the`

`recommended architecture alternative` — which means `ALT-B Foundations first`, marked `Recommended` at $46m, has never been adopted.

4. Raise what Phase A could not settle as gaps in `029 Gap`, with severities.
5. Cut the Phase A exit baseline in `04 Baselines`, supersede the previous one, and keep the extract.
6. Confirm the three `ZZ Checks` numbers from step 2.4 have moved, and say by how much.

Signed by. The Investment Committee (`GB-02`), which holds `G0` funding.

Done when. `059 Compliance Assessment` has rows with verdicts and evidence; `060` has a non-outstanding decision dated this month; `04 Baselines` has one `Current` row; and `DLV-01`, `DLV-02` and `DLV-03` each carry a location and an approver.

Shows up in. `RPT07 Gap Analysis` and `059 Compliance Assessment`, both of which should stop being empty. If `059` is still empty after your gate, the gate was a meeting.

Goes wrong when. The gate is held, minuted as "noted", and nothing is written into the repository. Six months later Phase B is challenged, there is no decision record, and the challenge succeeds.

Before you move to Phase B

PHASE A — A REVIEWER'S QUESTIONS

Who signed the Request for Architecture Work, and what would they say this work is for, in their own words?

Which of the six stakeholders from step 3.3 has read the vision, and which declined to meet you?

Does `059 Compliance Assessment` now have rows in it? If not, what did your gate actually test?

Your horizon is a date. Name three things in `023 Application Component` that will be out of vendor support before it.

THE CHAPTER IN TEN LINES

- The repository marks all three Phase A deliverables `Approved` and holds no Request for Architecture Work row at all. Add `DLV-21`.
- When nobody will sign, take a minute in `O60 Governance Decision`, attach to a funded work package, or record the silence with a date. Not nothing.
- `O10 Actor` and `O11 Role` hold the same eighteen names and `R21 Role performed by Actor` joins them one to one. An actor is a party; a role is a responsibility.
- Three of the five stakeholders a bank's map always misses have no role record: the treasurer, the regulatory reporting manager and the payments scheme representative. So does the POPIA Information Officer.
- `C10 Risk & Capital Management` carries `Criticality` = Low. A bank cannot have low-criticality capital management.
- Four of the nine business model canvas blocks have no home in the repository: customer segments, revenue streams, cost structure, and anything about money on a product row.
- `O14 Value Stream`'s inline `Capabilities` column contradicts `R02`, and `R02` is a three-wide sliding window. Rebuild all fifteen rows from a conversation.
- Set scope in four dimensions and write the exclusions as sentences with reasons. `O03`'s three-year horizons and `O66`'s month-56 increment cannot both be the target.
- `O07 Constraint` and `O08 Assumption` have no owner, no date and no relationship sheet reaching them. Add the columns before you add rows.
- `O59 Compliance Assessment` is empty. If it is still empty after your Phase A gate, the gate was a meeting.

SOURCES FOR THIS CHAPTER

- **1.** Banking_TOGAF_Metamodel.xlsx, sheet O31 Deliverable
- **2.** Banking_TOGAF_Metamodel.xlsx, sheets O29 Gap, O59 Compliance Assessment and O60 Governance Decision
- **3.** Banking_TOGAF_Metamodel.xlsx, sheet O30 Work Package
- **4.** Banking_TOGAF_Metamodel.xlsx, sheets O10 Actor, O11 Role and R21 Role performed by Actor
- **5.** Banking_TOGAF_Metamodel.xlsx, sheets O63 Control Objective and RPT09 Regulatory Coverage
- **6.** The Open Group, TOGAF Standard, 10th Edition — Phase A, Architecture Vision
  https://pubs.opengroup.org/architecture/togaf10-doc/adm/

CHAPTER 4

Business Architecture

The phase that decides whether anybody believes the rest of the work.

Phase B is where the volume stops being about the repository and starts being about the bank. Two sheets decide everything downstream — `O13 Business Capability` and `O16 Business Process` — and both of them arrive already filled, which is worse than arriving empty [1].

Ardent Union Bank, the bank in the repository, is an invention. So is Table Bay Bank, the bank in volume 1. Their numbers do not mix.

> **BEFORE YOU OPEN A SHEET**
>
> Most of volume 1 is business-layer teaching, so this is where it saves the most time. Four passages do most of the work.
>
> - **Volume 1, chapter 5, *Roles against the value chain* and *The credit mandate ladder*.** What each role decides and where its authority stops.
> - **Volume 1, chapter 8, *The L1 process map*.** Twelve L1 processes, each with a trigger, an outcome and an owner.
> - **Volume 1, chapter 16, *Is it a service or a process?*** The five minutes that keep `O15`, `O16` and `O17` apart.
> - **Volume 1, chapter 19, *Business capabilities*, *Roles* and *Business processes*.** The pre-drafted catalogues, with their local numbering.

What you are filling, and in what order

Phase B in this repository is ten object sheets and seven relationship sheets. The object sheets are `O09 Organization Unit`, `O10 Actor`, `O11 Role`, `O13 Business Capability`, `O14 Value Stream`, `O15 Business Service`, `O16 Business Process`, `O17 Business Function`, `O18 Product` and `O19 Contract`. The relationship sheets are `R01 Capability decomposition`, `R02 Value stream uses Capabilit`, `R03 Capability realised by Busi`, `R04 Business Service supported `, `R21 Role performed by Actor`, `R22 Org Unit owns Capability` and `R25 Application supports Produc` [1].

Several of those tab names are cut off. Excel truncates a sheet name at thirty-one characters, so `R02 Value stream uses Capabilit` really is missing its final `y` and `R04 Business Service supported` really does end in a space. Spell them the way the tab spells them or your `VLOOKUP` will fail and you will blame the data.

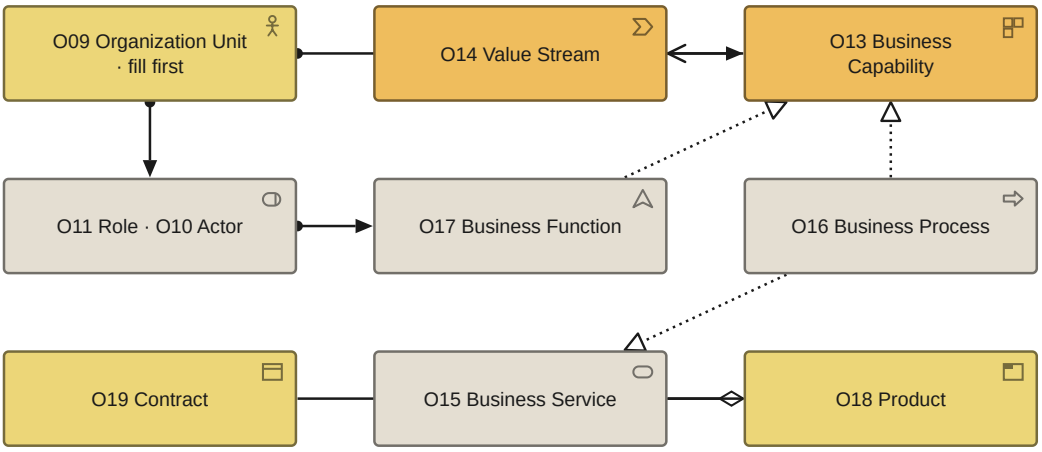

Figure 4.1 The Phase B sheets as one ArchiMate 4 business-layer view, carrying the relationships the seven `R` sheets actually hold. Nothing on it stands alone, which is why the fill order is not negotiable.

The order is not decoration. Skip `O09` and nothing downstream has an owner. Skip `O14` and the capability map has nothing to test it. Skip `O13` and Phases C, D and E have no anchor at all.

> **THE ONE RULE THAT HOLDS FOR EVERY SHEET IN THIS CHAPTER**
>
> Fill a relationship sheet in the same sitting as the second of its two endpoint sheets. A relationship backlog is not a backlog — it is a decision nobody made. Here, `R02` and the `Capabilities` column on `O14` were filled at different times by different logic and now disagree about two of five value streams. Nobody noticed, because nothing reads both.

The steps

Step 4.1 — Fix the Phase B boundary in writing before you open a sheet

Produce. One page: which value streams and which capability levels are in scope for this iteration, and what "out of scope" means operationally.

You need. The approved Statement of Architecture Work from Phase A (`O31 Deliverable`, `DLV-02`), the thirteen rows of `O01 Driver`, and twenty minutes with the sponsor.

Where it lands. Not in a sheet. `O31 Deliverable` carries `DLV-04 Business Architecture`, `DLV-05 Capability Map` and `DLV-06 Value Stream Definition`, all three of which the repository already marks `Approved`. The scope page goes into `DLV-04`.

How.

1. Read the twelve rows of `O09 Organization Unit` and mark each one in or out. There is no `Parent` column, so the hierarchy you think you see is recorded nowhere.
2. Decide the level you will assess. `O13` holds 120 rows across three levels; assessing level 3 honestly is 72 conversations.

3. Name the value streams in scope from the five rows of `O14 Value Stream`. Three is a defensible iteration; five is a quarter.
4. Write the exclusions as consequences — no workshop, no score, no gap raised — and make the sponsor say them out loud before you write them down.

Signed by. The sponsor named on the Statement of Architecture Work, countersigned by the Chief Enterprise Architect (`ROL-08`, and `GB-05 Architecture Function` on `O57 Governance Body`).

Done when. Every row of `O09` is marked in or out, and the capability level is a number on a page somebody signed.

Shows up in. Nowhere in the thirteen reports, which is the point. Scope is the one Phase B artefact this repository cannot hold, so it lives in a deliverable.

Goes wrong when. Scope is set by capability count rather than by value stream. "The whole map" is 120 rows at three levels, which is four months. Three value streams and the level 2 capabilities they touch is five weeks.

FIXING THE PHASE B BOUNDARY

Which value streams are in this iteration, and what makes them the ones?

At which capability level will you score, and who agreed that number?

For each exclusion, what is the operational consequence, and who said it out loud?

Step 4.2 — Load the organisation and the actors, and refuse to call either the architecture

Produce. A complete `O09 Organization Unit` list with one head and one headcount per unit, an `O10 Actor` sheet that includes the parties outside the bank, and a written note of where the chart lies.

You need. The HR structure, the group annual report, and volume 1 chapter 5.

Look it up. Volume 1, chapter 5, *The org chart will not help you*, and *The architect's real counterparties*, for the parties banks leave off a repository.

Where it lands.

> **`O09 Organization Unit`**
>
> `Ref`, `Organisation unit`, `Type`, `Head`, `Headcount`. `Type` is the honest column — Ardent Union uses `Group`, `Corporate`, `Line` and `Assurance`, and the gap between `Corporate` and `Line` tells you whether a unit can be told what to do. `Head` is a position title, never a person's name. There is **no `Parent` column and no `Location` column**: add `Parent` before you load a row.

> **`O10 Actor`**
>
> `Ref`, `Actor`, `Type`, `Status`. `Type` groups actors for reporting — the shipped values are `Executive`, `Architecture`, `Delivery`, `Operations`, `Data`, `Security` and `Assurance`. `Status` is `Filled` or vacant. There is **no `Internal/External` column and no `Population` column**, so an actor's count and its side of the bank's boundary cannot be recorded. Add both.

How.

1. Add `Parent` to `O09`, then load top down so it never points at a blank. Set `Type` from the group structure, not the reporting line: a line of business that reports to a group function is still a line of business. Take `Headcount` from payroll.
2. Write what each unit is accountable for in one sentence that does not reuse its own name, and mark the units with no decision rights.
3. Add the external actors. Ardent Union has eighteen and every one is an employee. There is no Customer, no Prudential Authority, no Financial Sector Conduct Authority, no payment scheme, no credit bureau, no correspondent bank. Phase G cannot trace an obligation to a party that is not on the sheet.
4. Separate the parties volume 1 insists on separating. Account holder, mandate holder, guarantor and beneficial owner are four actors, not one customer.

Signed by. Chief People Officer (`ORG-04`) for the structure, Chief Risk Officer (`ORG-03`) for the external counterparty set, Chief Enterprise Architect for the load.

Done when. Every `Parent` resolves, every `Head` is a title, headcounts sum to a number Finance recognises, and `O10` contains at least one party the bank does not employ.

Shows up in. Nothing reads `O09` except `R22 Org Unit owns Capability`, so a unit with no `R22` row is one you recorded and never used. `ZZ Checks` carries `Vacant architecture roles`, currently `0` against a threshold of `0` — a check that cannot fail tells you nothing.

Goes wrong when. You treat `O09` as the shape of the architecture. It is the shape of the payroll, and here not even that. Ardent Union's twelve units carry 2,012 people, of whom `ORG-11 Internal Audit` has 259 and `ORG-07 Operations` — under the Chief Operating Officer, in a bank with branches and a contact centre — has 32. There is no Retail Banking, no Business Banking, no Treasury and no Credit, and there is an `ORG-10 Supply & Logistics` with 168 people. That list came from a generic enterprise template and nobody read it back. Mark it as a finding in `DLV-04` on day one, before anyone builds an owner map on top of it.

Step 4.3 — Build the role catalogue the bank actually staffs, then join it to the actors

Produce. A role catalogue where every row is a position whose authority differs from its neighbour's, and `R21 Role performed by Actor` showing who holds what.

You need. `O10` complete. Volume 1 appendix A drafts a banking role set as `R-01` to `R-33` — load it and argue with it.

Look it up. Volume 1, chapter 19, *Roles*, for the thirty-three drafted roles.

Where it lands.

> **`O11 Role`**
>
> `Ref`, `Role`, `Type`, `Performed by`. `Type` is the grouping — `Executive`, `Architecture`, `Delivery`, `Operations`, `Data`, `Security`, `Assurance` — and you will need to add banking types: `Credit`, `Branch`, `Relationship`, `Collections`, `Treasury`, `Product`. `Performed by` holds a kind, not a reference: all eighteen rows read `Person`.

How.

1. Load volume 1's `R-01` … `R-33` as candidates. **Map by name, never by number.** Thirty-three drafted roles against eighteen `ROL-` rows is not a renumbering, and `R-21` in volume 1 has nothing to do with `R21 Role performed by Actor` in the repository. That collision of two `R-` schemes has caught people already.
2. Delete every candidate the bank does not staff, and add every position the repository missed. Ardent Union's eighteen roles are all executive, architecture, delivery or assurance. There is no Credit Analyst, no Credit Manager, no Branch Manager, no Relationship Manager, no Teller, no Collections Agent, no Dealer. A bank's role catalogue with no banking roles in it is the clearest sign of a template fill.
3. Apply the one test: does this row's authority differ from the row above it? "Senior Credit Analyst" is a role when its mandate differs. "Senior Branch Administrator" usually is not.
4. Set `Performed by` to `Organisation` where a partner, an agency or an outsourced desk performs the role, and note the arrangement in `DLV-04`.
5. Fill `R21` in the same sitting, and expect it to stop being one-for-one as soon as the catalogue is honest.

Signed by. Each role's accountable executive. Credit roles by the Chief Risk Officer (`ORG-03`), branch and channel roles by the Chief Commercial Officer (`ORG-08`), never by the architect.

Done when. Every row is a position with distinct authority, and the count of roles no longer equals the count of actors.

Shows up in. Nowhere. `O11` feeds no report here, which is worth a line in the findings: an unreported sheet rots quietly.

Goes wrong when. Actor and role collapse into one list, which is what has happened here. `ACT-01 Chief Executive` and `ROL-01 Chief Executive`, eighteen rows each, identical

names, identical order, joined one-for-one by `R21`: the actor sheet is a copy of the role sheet with a different prefix, and the model has lost the only thing the distinction is for — that one person holds several roles and one role can be held by a partner's staff. The second failure is roles copied from HR job titles, which gives seniority grades instead of authority bands.

Form B-1 — Role catalogue, banking roles the repository is missing

One row per position whose authority differs. Columns are the real `O11 Role` columns in sheet order. A first pass for a full-service bank usually needs twenty-five to thirty-five rows. Signed by each role's accountable executive.

Ref	Role	Type	Performed by
ROL-19	Credit Analyst — example, delete me	Credit	Person

Continue on a second copy of this form. The full catalogue lives in appendix F.

Step 4.4 — Record the credit mandate ladder, which the repository has nowhere to put

Produce. The bank's delegated credit authority, written as data: which role may approve what exposure, up to what rand limit, against which product, and what happens above the ceiling.

You need. The signed delegated authority schedule from the Credit Committee. The role catalogue from step 4.3. Volume 1 chapter 5, which treats the ladder at length.

Look it up. Volume 1, chapter 5, *The credit mandate ladder*, and chapter 8, *10. Business credit, and the mandate ladder*, for the ladder as a process. Volume 1, chapter 16, *The credit decision, and the mandate ladder*, for how to model it.

Where it lands. The honest answer is **nowhere, as shipped.** `O11 Role` has four columns and none holds a number. `O13 Business Capability` has `Criticality` but no authority. `O07 Constraint` could hold "no single signature above R15m" as prose, but a constraint is not a

limit table. `EXT Industry Extension` names `X04 Limit` as a type this sector needs beyond the TOGAF core, and then gives it no sheet.

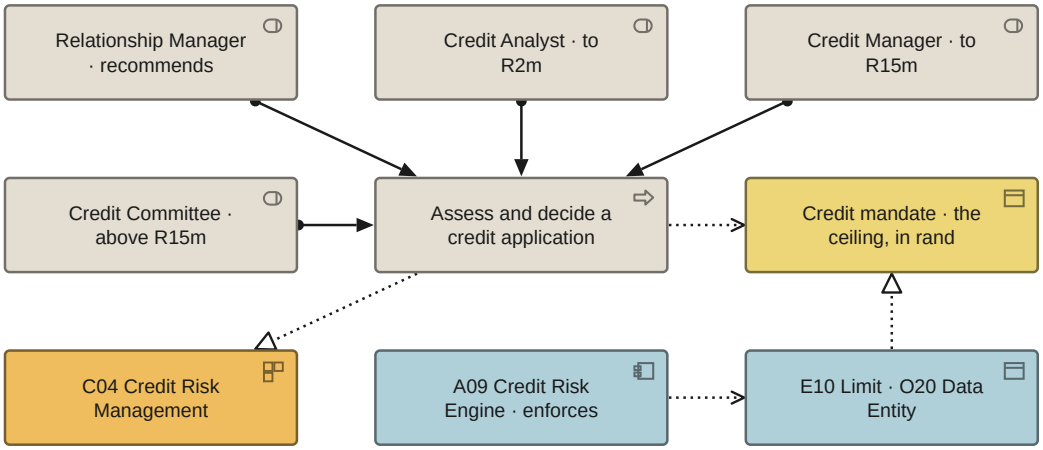

Figure 4.2 The credit mandate ladder, which the repository has no column for. Four roles assigned to one decision, the rand ceiling as a business object, and the limit recorded as data in one place and enforced in another.

So you add it. Three options, in order of how much you should like them:

Option	What you do	When it is right
Extend `O11 Role`	Add `Authority band`, `Limit ZAR`, `Product scope`, `Mandate source`	A single ladder, under twenty bands. Keeps authority next to the role
New sheet `O67 Credit Mandate`	`Ref`, `Role`, `Product`, `Limit ZAR`, `Secured`, `Joint signature`, `Escalates to`, `Mandate source`	Several ladders by product and segment, which is the normal case
Use `E10 Limit` on `O20`	Model the limit as data the engine enforces; keep the authority schedule outside the repository	The limits change monthly and the repository would always be stale

Take the second for a full-service bank, the first for a single-product lender. Either way add `Mandate source` — the board resolution or committee minute that grants the authority — because in Phase G that column is the evidence.

How.

1. Get the signed schedule. Not the slide, the signed schedule, with its effective date.
2. Create the columns or the sheet. One row per band per product, with the rand ceiling as a number, not as text.
3. Record `Escalates to` as a `Ref`, so the ladder is traversable and a dual-signature rule shows as two rows that both point up.
4. Join each band to `C04 Credit Risk Management` and to the process that exercises it.

5. Record where the limit is **enforced**, which is almost never where it is **recorded**. `A09 Credit Risk Engine` and `E10 Limit` on `O20 Data Entity` are the two rows Phase C will need.

Signed by. The Chief Risk Officer (`ORG-03`), against the board resolution that delegates the authority. This is the one Phase B signature that is a legal act and not a courtesy.

Done when. Every band has a rand ceiling, an escalation target and a named mandate source, and the ladder reconciles to the signed schedule line for line.

Shows up in. Nowhere, until you put it somewhere. Once `C04` carries the ladder, a `Current` of 2 against a `Target` of 4 on `RPT03` becomes a statement about whether the bank can prove who approved what.

Goes wrong when. The ladder lives in a policy document, the engine enforces different numbers, and nobody compares them. That is the most common serious finding in a bank's credit estate, and it is invisible to a repository with no column for a rand limit.

THE MANDATE LADDER, WRITTEN DOWN

Which role in this bank may approve the largest unsecured exposure on a single signature, and what is the rand number?

Where is that number enforced in software, and who last reconciled the enforced value to the signed schedule?

Which column or sheet are you adding to hold it, and who signed that decision?

Step 4.5 — Write the value streams, then reconcile `O14` against `R02`

Produce. Five to seven value streams, each triggered by somebody outside the bank, each with its stages named and the same capability set in both places it is recorded.

You need. Scope from step 4.1. The business model work from Phase A.

Look it up. Volume 1, chapter 7, *The value chain, end to end*, and chapter 16, *Capabilities and value streams*.

Where it lands.

> **`O14 Value Stream`**
>
> `Ref`, `Value stream`, `Scope`, `Stages`, `Capabilities`. `Scope` is one sentence written from outside the bank. `Stages` holds **text, not a number** — the shipped values read `4 stages`, `5 stages`, `6 stages` — and the stage names are nowhere. Put them in `Scope` or add a `Stage names` column. `Capabilities` holds a comma-separated list, which duplicates `R02 Value stream uses Capabilit` and is the reason the two disagree.

How.

1. Start from the triggering party. If you cannot name who outside the bank starts it, it is a process and belongs in `O16`.
2. Write `Scope` in the trigger's words. `VS02 Apply to Fund` reads "Credit application through decision, agreement and disbursement" — the bank's sentence, not the applicant's.
3. Name the stages. `VS03 Initiate to Settle` says `6 stages` and no sheet says what they are.
4. Reconcile `Capabilities` against `R02` row by row. Three of five agree. `VS04 Arrears to Resolution` lists `C01, C02, C03` on `O14` and `C04, C05, C06` in `R02`; `VS05 Trade to Report` lists `C02, C03, C04` against `R02`'s `C05, C06, C07`. Make `R02` authoritative, because reports can read a relationship sheet, and formula-link or delete the other.
5. Add the value stream the bank will tell you is internal. Regulatory reporting is triggered by a supervisor and is a value stream.

Signed by. The executive who owns each stream's outcome, in one sitting so they see each other's streams.

Done when. Every stream has a triggering party who is not an employee, stage names exist, and `O14` and `R02` agree for all five rows.

Shows up in. `RPT01 Dashboard` counts relationship instances. `R02` carries fifteen — exactly three per stream — which is the signature of a generated fill, not a workshop.

Goes wrong when. Nobody looks at `VS04 Arrears to Resolution` long enough to notice that neither version of its capability list includes `C07 Lending & Collections`. A collections value stream that does not use the collections capability is not a modelling slip — it is evidence that no collections manager was in the room.

Step 4.6 — Apply the four tests before you accept a single capability row

Produce. A pass or fail against four tests for every row you intend to keep, and a deletion list for the rest.

You need. `O13` as delivered. Volume 1 appendix A's `C-01` … `C-20`.

Look it up. Volume 1, chapter 19, *Business capabilities*, and chapter 16, *Is it an element at all?*

Where it lands. Not in a sheet. The pass list becomes the `O13` you keep; the deletion list goes in `DLV-05 Capability Map` as an appendix, because somebody will ask why their capability disappeared.

THE FOUR TESTS. A ROW THAT FAILS ONE IS NOT A CAPABILITY

Test	What it asks	A row that fails it
1 · Noun, not verb	Is the name a thing the bank can do, with no tense?	"Assess a credit application" is a process. It belongs in `O16`
2 · Survives a reorganisation	Move the division, outsource the desk — is it still needed?	"Operations" dies with `ORG-07`. It was a function
3 · One owner can be named	Exactly one unit, accountable, able to say no	Two owners means the boundary is wrong, not that it is shared
4 · Can be scored and invested in	Could you write a cheque against it and measure the result?	"Payments Assurance" cannot be funded, because nobody can say what it is

How.

1. Load volume 1's `C-01` … `C-20` as candidates. **Map by name, never by number** — and say out loud that `C-01` … `C-20` is a local twenty-row set written for teaching, not BIAN's 586 business capabilities. If the bank is a BIAN member, map to the Service Landscape separately and keep both; if it is not, note that BIAN publishes no licence and availability is not permission [6].
2. Run the four tests against all 120 rows of `O13`. The twelve level 1 rows pass. Almost nothing below them does.
3. Write the deletion list with a reason per row, so the conversation is about the reason.
4. Write the one sentence that decides membership, and put it at the top of `DLV-05`.

Signed by. Chief Enterprise Architect, with the twelve level 1 owners agreeing the test set before it is applied to their areas.

Done when. Every surviving row has four passes, and every deleted row has a written reason.

Shows up in. `RPT03 Capability Heatmap` reads only the twelve level 1 rows, so the 108 below them are invisible to every report here. That is why they were never questioned.

Goes wrong when. You accept the delivered decomposition because it is tidy. `O13`'s levels 2 and 3 are generated: each level 1 capability has exactly three level 2 children drawn from a rotating word list — `Administration`, `Optimisation`, `Onboarding`, `Reporting`, `Assurance`, `Support`, `Planning`, `Execution`, `Monitoring`, `Design` — and each of those has exactly two level 3 children, `– Define` and `– Operate`. Twelve, thirty-six, seventy-two. The result includes `Treasury Onboarding`, `Payments Assurance` and `Deposits Planning`, none of which a

banker would recognise, and `Customer Management Onboarding — Operate`, which fails tests 1 and 4 at once. Delete all 108 and rebuild.

THE SENTENCE THAT DECIDES A CAPABILITY BOUNDARY

Write the one sentence that decides whether a row belongs in `013`. Fifteen words or fewer.

Which three level 1 capabilities will you decompose this iteration, and why those three?

Who will be asked to defend this map, and to whom?

Step 4.7 — Rebuild the decomposition, then prove it with `R01`

Produce. A two-level map under each level 1 capability in scope, where every child is a capability its parent's owner recognises, and `R01 Capability decomposition` mirrors it exactly.

You need. The pass list from step 4.6, two hours per level 1 area, and the owner in the room.

Look it up. Volume 1, chapter 2, *The twelve lines*, for the decomposition the lines of business force, and chapter 11, *The twenty-five*, for where capability and application stop matching.

Where it lands.

> **`013 Business Capability`**
>
> `Ref`, `Capability`, `Level`, `Parent`, `Description`, `Criticality`, `Current`, `Target`, `Disposition`. `Ref` uses dotted notation — `C04.02.01` — which carries the tree in the key and makes a renumber a find-and-replace across six sheets. `Level` is `1`, `2` or `3`. `Parent` is a `Ref`, blank only at level 1. `Description` must say what the capability does without reusing its own name, which 108 of the 120 shipped rows fail ("Customer Management Administration within customer management").

How.

1. Decompose from the owner's language, not from a word list. Under `C04 Credit Risk Management` a bank recognises origination scoring, affordability assessment, limit setting, rating and provisioning, collateral valuation, and portfolio monitoring. It does not recognise "Credit Risk Management Design".

2. Keep siblings at one level of abstraction. If one child is a quarter of the parent and another a twentieth, the boundary is wrong.
3. Stop at the level you can score. Three to seven children per parent, two levels below level 1 at most.
4. Rewrite `Description` for every row you keep.
5. Fill `R01` in the same sitting. Five columns — `Parent`, `Parent name`, `Child`, `Child name`, `Level` — and it is the one relationship sheet here that is internally correct, with 108 rows mirroring `O13`'s `Parent` exactly. Keep it that way; it is the check you will use for the rest of the volume.

Signed by. The level 1 owner for their own subtree. The Chief Enterprise Architect signs the map as a whole.

Done when. Every `Parent` resolves, `R01`'s row count equals `O13`'s rows at levels 2 and 3, and no `Description` reuses its own `Capability` value.

Shows up in. `RPT01 Dashboard` reports `Level 1 capabilities: 12`, decomposed to level 3, and `ZZ Checks` reports the record count. Neither tests whether the children mean anything, which is why the generated tree survived to delivery.

Goes wrong when. You renumber. The dotted `Ref` appears in `O13`, `R01`, `R02`, `R03`, `R22`, `O15`, `O17` and `O35 Business Building Block`. Insert a capability mid-sequence and you have eight sheets to repair. Append instead, and let the numbers be non-contiguous.

Step 4.8 — Assign exactly one accountable owner per capability

Produce. `R22 Org Unit owns Capability` with one row per assessed capability, each naming a unit that can say no.

You need. `O09` and `O13` complete. The level 1 owners in one room.

Where it lands.

> **`R22 Org Unit owns Capability`**
>
> `Org unit`, `Unit name`, `Capability`, `Capability name`. Four columns, no `Basis`, no `Accountable since`. One row per capability; a second row for the same capability is the finding, not the model.

How.

1. Delete the shipped twelve rows and start again. They are a diagonal: `ORG-01` → `C01` through `ORG-12` → `C12`. The result is that Human Resources owns Credit Risk Management, Technology owns Payments & Settlement, Legal owns Channel & Distribution, Supply & Logistics owns Risk & Capital Management, and Internal Audit owns Finance & Regulatory Reporting.

2. Treat that last one as more than a data error. An internal audit function that owns the capability it must audit independently is a governance breach, and if it survives into `DLV-04` the Audit Committee will ask you about it.
3. Assign ownership from accountability, not headcount. Ask the candidate one question: can you stop a project in this capability? If not, they are not the owner.
4. Where two units both claim it, split the capability or escalate to `GB-04 Executive Committee`. Do not record two rows.
5. Record ownership below level 1 only where the level 1 owner has genuinely delegated it.

Signed by. `GB-04 Executive Committee` on `O57 Governance Body` ratifies the map as a set. Individual rows are signed by the accepting unit head.

Done when. Every assessed capability has exactly one `R22` row, accepted in writing.

Shows up in. `RPT13 Orphan Analysis` is meant to surface unreferenced objects. It lists two — `A13 Data Platform` and `A14 Fraud Detection`, both applications, both found by testing `R04` alone. It does not test `O13` for a missing `R22` row, so a capability with no owner is not an orphan as far as this repository is concerned. Run that check by hand, every phase.

Goes wrong when. Ownership goes to the unit that does the work rather than the unit accountable for the outcome. In a bank those differ most often in credit, where the line originates and Risk owns the decision.

OWNERSHIP, TESTED

For your three largest capabilities: can the named owner stop a project? Name the last time one did.

Which capability has two claimants, and who will settle it?

Step 4.9 — Publish the scale, then score `Current` so the numbers survive contact

Produce. A one-page scale circulated three days before the first session, and then a `Current` score for every capability in scope, each with its evidence recorded and every contested score noted.

You need. The capability owner and two or three practitioners per area, two hours per level 1 area, and a facilitator who is not the owner. Nothing from the repository: the scale is method, and it has to be settled before anybody scores.

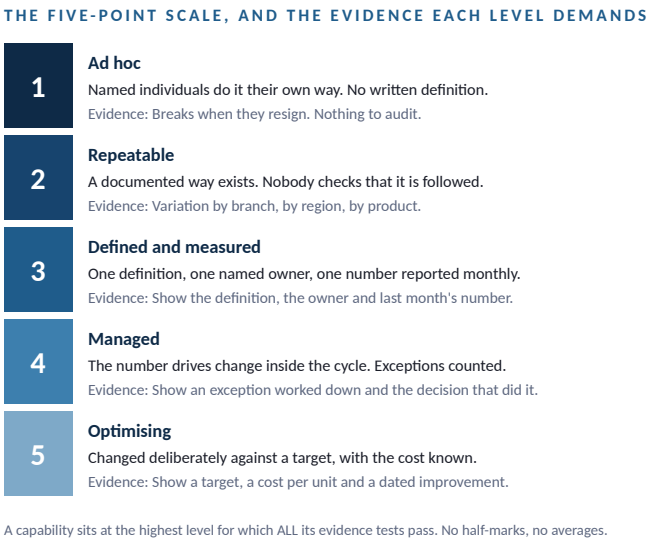

Figure 4.3 The scoring scale to publish three days before the first workshop. `013` already holds `Current` values 1 to 4 and `Target` values up to 5, so five levels is the scale you inherit.

Where it lands. The scale goes in `DLV-05 Capability Map` as its first appendix — it is not a repository object, since `032 Standard` holds technology standards, not scoring methods. The scores go in one column.

> **`013 Business Capability`**
>
> `Current` only. Leave `Target`, `Criticality` and `Disposition` for step 4.10 — three different decisions by three different people, and collapsing them into one session is how you get a wish list. There is **no `Evidence` column and no `Assessed on` column**. Add both, or keep the evidence in the workshop pack and accept that in six months nobody will know where a 3 came from.

How.

1. Fix the scale at five levels, because `013` already holds `Current` 1 to 4 and `Target` 2 to 5 — a scale you inherit beats one you impose. Write each level's evidence test in the bank's terms: level 3 is not "defined", it is "one written definition, one named owner, and a number somebody reports monthly".

2. State the two rules that make it defensible. A level cannot be claimed without its evidence, named in the session and not promised after it. And a capability sits at the highest level for which **all** its tests pass, never the average — half-marks are how a scale becomes a negotiation. Publish it with the participant list, so nobody arrives arguing about the instrument.
3. Score blind. Every participant writes a number for every capability on paper, alone, before anybody speaks. Use Form B-2.
4. Reveal all scores at once. Where the spread is one level, take the lower and move on. Where it is two or more, stop and make the claimant produce the evidence: "we have a documented process" becomes "show me the document and tell me who last changed it". Most level 4 claims fall to 2 in that exchange.
5. Record contested scores with both numbers and the reason, and score what exists this morning rather than what goes live next quarter.

Signed by. `GB-01 Architecture Board` notes the scale before scoring begins. The capability owner signs their own `Current` scores — not the architect and not the sponsor, because in step 4.10 the owner will be asked to fund the gap.

Done when. Every participant had the scale before the session, every capability in scope has a score, every score has a named piece of evidence, and the contested list is written down.

Shows up in. `RPT03 Capability Heatmap`, twelve rows, `Current` against `Target`, with `Gap` computed as the difference. If every score is 3 or 4 you did not run an assessment, you ran a presentation.

Goes wrong when. Scores are inherited. Ardent Union's 120 `Current` values run 1 to 4 with no relationship to the tree: `C03 Origination & Onboarding` scores 2 while its child `C03.01 Origination Planning` scores 4 and its grandchild `C03.01.01` also scores 4. A parent cannot be less mature than every one of its children unless the roll-up means nothing — and here it means nothing, because no formula computes a parent from its children. Apply the roll-up rule in appendix H, or delete the parent scores.

Form B-2 — Blind scores for one capability area

One copy per level 1 capability, one column per participant. Scores transcribe into `Current` on `013 Business Capability` after the reveal. Usually eight to twelve rows per area. Signed by the capability owner.

Ref	Capability	Blind score	Agreed score	Evidence named
C04.01	Affordability assessment — example, delete me	2	2	NCA affordability policy v4, Mar 2026

Step 4.10 — Set `Target`, `Criticality` and `Disposition` as three separate decisions

Produce. A `Target` maturity per capability agreed with the sponsor, a `Criticality` grade agreed with Risk, and a `Disposition` that follows from both rather than from arithmetic.

You need. `Current` complete. The sponsor for `Target`, the Chief Risk Officer for `Criticality`, and the investment committee's appetite for `Disposition`.

Where it lands.

> **`013 Business Capability`**
>
> `Criticality` takes `Critical`, `High`, `Medium`, `Low`. `Target` takes 1 to 5. `Disposition` takes `Sustain` or `Build` — only those two values appear in 120 rows, which is a smaller vocabulary than the work needs. Add `Replace` and `Divest`, and note the addition in `DLV-05`.

How.

1. Set `Criticality` from impact, not affection. A capability is `Critical` when its failure stops the bank banking, breaches a licence condition or breaks a scheme obligation. Ardent Union grades `C10 Risk & Capital Management` as `Low` — in a bank where `CO-01 Basel`

`III / IV` on `O63 Control Objective` reads `In breach`. The two statements cannot both stand.

2. Make `Criticality` inherit. In the shipped data it does not: `C01 Customer Management` is `Critical` and its child `C01.01.02` is `Low`, while `C10` is `Low` and its grandchild `C10.01.01` is `Critical`. A child can be less critical than its parent. It cannot be more.
3. Set `Target` against appetite. A `Target` of 5 on forty-three of 120 rows is not a target, it is a mood. Ask the sponsor what they will fund and set it there.
4. Set `Disposition` as a judgement. In the shipped data it is arithmetic: every level 1 row with a gap of 1 reads `Sustain` and every row with a gap of 2 reads `Build`, no exceptions in twelve rows. `Disposition` should carry what the gap cannot express — that a capability is differentiating, or outsourced, or being exited.
5. Record the differentiating capabilities. There is no `Differentiating` column; add one, or you cannot tell a capability that must be excellent from one that must merely work.

Signed by. Sponsor signs `Target`. Chief Risk Officer (`ORG-03`) signs `Criticality`. `GB-02 Investment Committee` signs `Disposition`, because a disposition is a funding statement.

Done when. No capability is graded more critical than its parent, every `Target` has a named signatory, and at least one `Disposition` disagrees with the gap.

Shows up in. `RPT03 Capability Heatmap` carries all four columns plus the computed `Gap`. `RPT01 Dashboard` does not aggregate maturity, so `RPT03` is the whole picture.

Goes wrong when. All three columns are filled in one session by one group. `Criticality` drifts to whoever is loudest, `Target` drifts to 5, and `Disposition` becomes a restatement of the gap — which is precisely what happened here.

Step 4.11 — Read `RPT03` and turn it into a shortlist, not a heatmap

Produce. A ranked shortlist of six to ten capabilities for Phase E, with the reason each one is on it, and the gaps raised in `O29 Gap`.

You need. `O13` complete through step 4.10.

Where it lands. The shortlist goes in `DLV-04 Business Architecture`. The gaps go in `O29 Gap` — `Ref`, `Gap`, `Domain`, `Building block`, `Severity`, `Status`, `Due` — which currently holds **zero records** across the whole repository.

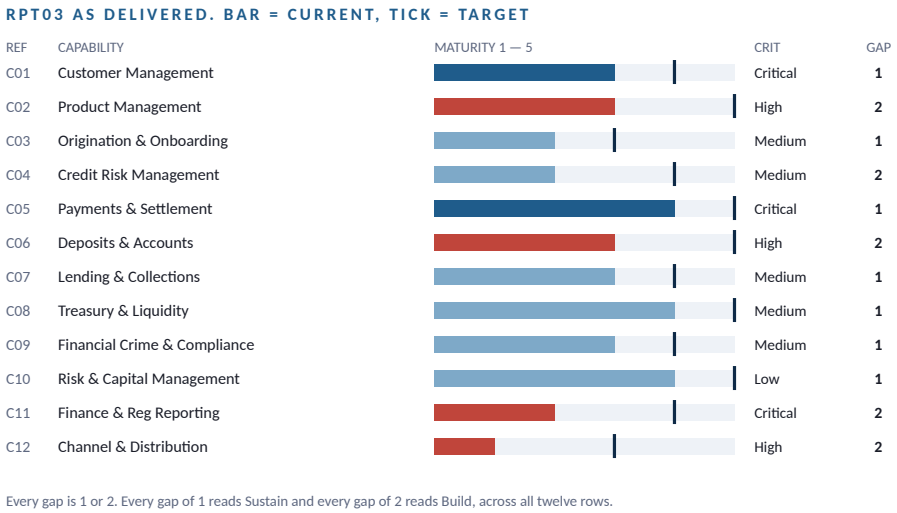

Figure 4.4 `RPT03 Capability Heatmap` exactly as shipped: twelve level 1 rows, no gap larger than 2, and a `Disposition` column that is arithmetic rather than judgement. The 108 rows below level 1 appear in no report at all.

How.

1. Read `RPT03` as delivered. Twelve rows, gaps of 1 or 2, nothing larger. `C12 Channel & Distribution` is worst at `Current` 1 against `Target` 3; `C11 Finance & Regulatory Reporting` is `Critical` at `Current` 2.
2. Rank by `Criticality` first and `Gap` second, never the reverse. A gap of 2 on a `Low` capability is a distraction; the same gap on a `Critical` one is the programme.
3. Cross-check against `O35 Business Building Block`. Ten of the twelve level 1 capabilities have a `BBB-` row. The two without are `C11` and `C12` — the `Critical` one with a gap of 2, and the one with the lowest `Current` score in the bank.
4. Raise a gap per shortlisted capability in `O29 Gap`, with `Domain` = `Business`, `Severity` graded, and a `Due` date somebody owns.
5. Stop at ten. A shortlist of thirty is a capability map with a new name.

Signed by. `GB-02 Investment Committee`, which is the body that will fund it.

Done when. `O29 Gap` has `Domain` = `Business` rows, and `RPT07 Gap Analysis` is not empty.

Shows up in. `RPT07 Gap Analysis` and, once Phase E links them, `R12 Gap closed by Work Package`. Note that `RPT01 Dashboard` reads `Gaps overdue: 0` — true, and true because there are no gaps at all.

Goes wrong when. The heatmap is the deliverable. A coloured grid is not a decision. The test of the shortlist is whether `O29 Gap` has rows in it when you leave the room.

READING THE HEATMAP HONESTLY

Which capability on your shortlist is there because of its criticality rather than its gap?

Which capability did an executive want on the list, and what evidence kept it off?

How many rows did you add to `029 Gap`? If the answer is none, what did Phase B find?

Step 4.12 — Separate business functions from business processes before you load either

Produce. A clean `017 Business Function` sheet and a clean `016 Business Process` sheet, with a written rule for which sheet a new row goes in.

You need. `013` complete. Volume 1 chapter 16, which settles this in five minutes.

Look it up. Volume 1, chapter 16, *Is it a service or a process?*, and chapter 8, *L0, L1, L2, L3 — and the warning*.

Where it lands.

> **`017 Business Function`**
>
> `Ref`, `Business function`, `Capability`, `Description`. A function is continuous behaviour grouped by skill — it runs, it does not finish. `Capability` is the `013 Ref` it performs against. There is no `Organisation unit` column, so the chain to a unit runs `009` → `R22` → `013` → `017`, three joins long.

This is the commonest Phase B error in any sector and it is worse in a bank, because banking vocabulary uses one word for all four things. "Payments" is a capability, a function, a process and a department, and a repository that cannot tell them apart produces a model in which "Payments" appears four times and means none of them.

Sheet	The element	The test	Ardent Union's row
`O13 Business Capability`	Capability	Survives a reorganisation	`C05 Payments & Settlement`
`O17 Business Function`	Function	Runs continuously, never finishes	`BFN-05 Payments & Settlement`
`O16 Business Process`	Process	Has a start, an end and a count per year	`BPR-09 Initiate to Settle — stage 1`
`O15 Business Service`	Service	Somebody outside asks for it by name	`BSV-05 Payments & Settlement Service`

How.

1. Read the test column, in that order, for every row you intend to create.
2. Delete the one-to-one. Ardent Union's twelve functions, twelve services and twelve level 1 capabilities carry the same twelve names in the same order, differing only by the word "Service" and the prefix. Three sheets recording one list is the same fact written three times, and it misleads every reader who assumes three sheets mean three things.
3. Rebuild `O17` from the organisation's real groupings of skill. A bank has a credit function, a payments operations function, a treasury middle office, a financial crime function. It does not have a function per capability.
4. Where a function spans two capabilities, create two rows. `Capability` holds one `Ref`.

Signed by. Chief Operating Officer (`ORG-07`) for the operational functions, Chief Enterprise Architect for the model.

Done when. `O13` level 1, `O17` and `O15` have three different row counts.

Shows up in. Nowhere. Neither `O16` nor `O17` feeds any of the thirteen reports — there is no process report at all. State that absence in `DLV-04` : a sheet nobody reads will drift.

Goes wrong when. Somebody maps one function to one capability because it makes the spreadsheet symmetrical. Symmetry in Phase B is almost always a generated fill.

Step 4.13 — Load the process inventory from the real L1 map

Produce. `O16 Business Process` holding the bank's actual level 1 processes, each tied to a value stream, each graded for automation.

You need. `O14` complete. Volume 1 chapter 8, which publishes twelve level 1 banking processes and thirteen BPMN level 2 models — start from those, not from a blank sheet.

Look it up. Volume 1, chapter 8, *The L1 process map*, for the twelve, and *1. Onboard a party and complete FICA* onwards for the level 2 models. The trap is in the same chapter, *The trap: modelling the process you were told about*.

Where it lands.

> **`O16 Business Process`**
>
> `Ref`, `Business process`, `Value stream`, `Description`, `Automation`. `Value stream` is an `O14 Ref`. `Automation` takes `Manual`, `Partly automated`, `Automated`. There is **no `Volume p.a.`, no `Cycle time`, no `Owner` and no `Level`** — so you cannot tell a level 1 process from a level 3 task, compute effort, or say who owns one. Add `Level`, `Owner` and `Volume p.a.` before you load, because volume converts a process into hours, and hours move funding.

How.

1. Delete the twenty shipped rows. They are placeholders: `BPR-01 Prospect to Customer — stage 1` through `BPR-20 Trade to Report — stage 4`, four per value stream, with `Description` reading "Stage 1 of prospect to customer." No process is named, no trigger recorded, no outcome stated.
2. Note the contradiction while you delete. `O14` says `VS03 Initiate to Settle` has `6 stages` and `VS02` and `VS05` have `5 stages`. `O16` gives all five streams exactly four. The two sheets disagree about three of five streams.
3. Load volume 1's twelve level 1 processes, mapped by name to your value streams. **Map by name, never by number** — volume 1's `P-01` … `P-12` are its own identifiers and the repository's `BPR-` prefix is unrelated.
4. Set `Automation` from observation, not from the system inventory. A process with a system in it is not automated if a person re-keys the output.
5. Record `Volume p.a.` from the operational reports. For a mid-tier South African bank, payment instructions and card authorisations run orders of magnitude above everything else, and that ratio is usually the Phase E argument.

Signed by. Each process owner. In a bank the honest answer is often that nobody owns the end-to-end process, and recording that beats inventing an owner.

Done when. Every process has a trigger, an outcome, a level, a volume and either a named owner or an explicit note that it has none, and the per-stream count reconciles to `O14`.

Shows up in. Nowhere. A process appears only as a `Value stream` foreign key, so an inconsistent `O16` is invisible until Phase C asks which process touches which entity.

Goes wrong when. You model the process you were told about rather than the one that runs. The test is cheap: ask the practitioner what they do when the system says no.

Step 4.14 — Define the business services and test them for channel independence

Produce. `O15 Business Service` holding services a party outside the bank can ask for, graded for exposure and criticality, with `R03` and `R04` filled truthfully.

You need. `O13` and `O16` complete. The application list from `O23 Application Component` to fill `R04`.

Look it up. Volume 1, chapter 7, *Channels, and what each one can actually do*, and chapter 16, *Is it a service or a process?*

Where it lands.

> **`O15 Business Service`**
>
> `Ref`, `Business service`, `Capability`, `Description`, `Exposure`, `Criticality`. `Exposure` takes `Internal` or `External` and is the column that tests whether the row is a service at all. `Criticality` takes the `O13` vocabulary — except that across twelve rows it never takes the value `Critical`, in a bank where `O13` grades twenty-one capabilities `Critical`.

How.

1. Apply the one test: can a party outside the bank ask for this, by name, without knowing how the bank is organised? If not, it is a function.
2. Set `Exposure` from channel reality. Eight of twelve services are `Internal`, including `BSV-05 Payments & Settlement Service` — in a bank whose customers initiate payments. Meanwhile `BSV-02 Product Management Service` is `External`, which would mean customers ask the bank to manage its own catalogue.
3. Reconcile `Criticality` against `O13`. Service and capability disagree on nine of twelve rows. `C05 Payments & Settlement` is `Critical`; `BSV-05 Payments & Settlement Service` is `Low`. One of those is wrong and it is not the capability.
4. Fill `R03 Capability realised by Busi` as you go. As shipped it is one-for-one across twelve rows, which is the symmetry test failing again.
5. Fill `R04 Business Service supported` against the real estate, and check every row. As shipped it is an offset diagonal: `BSV-05 Payments & Settlement Service` supported by `A05 Collections & Recovery`, `BSV-08 Treasury & Liquidity Service` by `A08 AML / Sanctions Screening`, `BSV-09 Financial Crime & Compliance Service` by `A09 Credit Risk Engine`, `BSV-12 Channel & Distribution Service` by `A12 Regulatory Reporting`. Every row looks plausible and is wrong, which is the dangerous kind.

Signed by. The capability owner for the service definition; the application owner named in `O23` for the `R04` row.

Done when. Every service passes the outside-party test, `Criticality` agrees with its capability or the disagreement is written down, and every `R04` row has been read aloud by somebody who knows the system.

Shows up in. `RPT13 Orphan Analysis` tests `R04` and nothing else, which is how `A13 Data Platform` and `A14 Fraud Detection` come to be the only recorded orphans. Fixing `R04` changes that list, so re-read `RPT13` after this step, not before.

Goes wrong when. `R04` is filled from a vendor diagram. The test for a wrong row: which team would you call when the service fails? If it is not the team that owns the application in the row,

the row is wrong.

Step 4.15 — Register the products and the contracts that constrain them

Produce. `O18 Product` holding the bank's sellable products, `R25 Application supports Produc` holding the system that actually administers each one, and `O19 Contract` holding the third-party agreements that bind the architecture.

You need. The product catalogue from Product and Pricing, and the vendor contract register from Procurement.

Look it up. Volume 1, chapter 16, *Product, and the catalogue underneath it*. For the word itself, chapter 3, *Product*.

Where it lands.

> **`O18 Product`**
>
> `Ref`, `Product`, `Type`, `Description`. `Type` takes `Deposit`, `Lending`, `Payments`, `Wholesale`. There is **no `Owner`, no `Status`, no `Line of business`, and nothing to say whether a product is configured or coded** — the single most useful product fact an architect can record, because it decides whether a new product is a release or a configuration change. Add `Config or code`.

How.

1. Load from the catalogue, not the system. A product the bank still services but no longer sells needs a row and needs `Status` to say so.
2. Fill `R25 Application supports Produc` from the administering system, and check it. As shipped it is another offset diagonal: `PRD-02 Savings Account` supported by `A04 Loan Origination`, `PRD-04 Mortgage` by `A10 Treasury Management`, `PRD-05 Credit Card` by `A13 Data Platform`, `PRD-08 Treasury Deposit` by `A08 AML / Sanctions Screening`. Eight rows, eight wrong.
3. Notice what `R25` does to `RPT13`. The orphan report says `A13 Data Platform` is referenced by nothing, yet `R25` gives it a row — because the orphan test reads `R04` only. Trust the report less than the sheets.
4. Load `O19 Contract` — `Ref`, `Contract`, `Counterparty`, `Type`, `Expiry` — and fix `Counterparty`. All ten rows name the **application** as the counterparty (`CTR-01` reads `Core Banking`), not the legal entity you would sue. Put the vendor's registered name there.
5. Resolve `Type` against `Expiry`. Seven contracts are `Perpetual` and all seven carry an `Expiry` date. A perpetual licence with a support renewal is two facts; record them in two columns.

Signed by. Head of Product for `O18`; General Counsel (`ORG-12`) for `O19`, because a contract register with the wrong counterparty is a legal exposure, not a modelling defect.

Done when. Every product has an administering system its product manager recognises, every contract names a legal entity, and `Type` and `Expiry` no longer contradict each other.

Shows up in. `RPT02 Application Portfolio` for the application side. `O19` feeds no report, so expiry surfaces nowhere until Phase D reads `Support end` on `O27 Physical Technology Compone`.

Goes wrong when. Products are loaded at marketing granularity. "Gold Current Account" and "Platinum Current Account" are one product with two pricing tiers unless they are administered differently — and if they are, that is the finding.

Step 4.16 — Close Phase B through `RPT03` and `RPT13`

Produce. A signed Phase B result: the capability map, the value streams, the process inventory, the gaps raised, and a written list of every repository defect you found and did not fix.

You need. Steps 4.1 to 4.15 complete for the capabilities in scope.

Where it lands. `O31 Deliverable` rows `DLV-04 Business Architecture`, `DLV-05 Capability Map` and `DLV-06 Value Stream Definition`. All three are already marked `Approved`, against the data this chapter has been taking apart. Set them back to draft, or your sign-off means nothing.

How.

1. Re-run `ZZ Checks`. It reports `Object types populated: 65` against the threshold "Every type carries records" — and `O29 Gap` holds zero records, so that check passes while being false. Record the discrepancy.
2. Read `RPT03 Capability Heatmap` top to bottom and ask whether you would defend each of the twelve rows to the Audit Committee.
3. Run the three checks `RPT13 Orphan Analysis` does not: capabilities with no `R22` owner, value streams whose `O14 Capabilities` disagrees with `R02`, and applications referenced by `R25` but not `R04`.
4. Count your `O29 Gap` rows with `Domain` = `Business`. Fewer than four, after all of this, and you have under-reported.
5. Write the defect list. It is the most valuable artefact Phase B produces here, and it belongs in front of `GB-01 Architecture Board`, not in an appendix.

Signed by. `GB-01 Architecture Board` approves the result at gate `GT-1 G1 Design` on `O58 Governance Gate`. The sponsor signs the shortlist.

Done when. `RPT07 Gap Analysis` has business-domain rows, the three deliverables carry a status you set yourself, and the defect list has a named owner per line.

Shows up in. `RPT01 Dashboard`, `RPT03`, `RPT07` and `RPT13`. If `RPT07` is still empty, Phase B produced a description, not an architecture.

Goes wrong when. Phase B closes with a beautiful capability map and an empty gap sheet. That combination is not a clean estate. It is a phase that did not look.

What a finished Phase B looks like in the numbers

Reading	As delivered	After Phase B
`013` rows at levels 2 and 3	108, generated from a word list	As many as you can score, each one named by its owner
`013` `Target` of 5	43 of 120 rows	However many the sponsor will fund
`013` `Disposition` values	2 (`Sustain` , `Build`), derived from `Gap`	4, at least one disagreeing with the gap
`010` external actors	0 of 18	Customers, supervisors, schemes and bureaux on the sheet
`011` banking roles	0 of 18	The credit ladder, the branch, the dealing desk
`014` vs `R02` agreement	3 of 5 streams	5 of 5
`016` processes with a trigger and an outcome	0 of 20	All of them
`029 Gap` rows, `Domain` = `Business`	0	4 or more

The distance between those two columns is Phase B. Nothing in the first column is a bug in Excel: every sheet is structurally sound, almost every foreign key resolves, `ZZ Checks` passes most of its thresholds, and `00 Index` states 878 records and 465 relationship instances, which is true. The repository is complete and almost entirely untrue, and that is the normal condition of a populated metamodel you did not fill yourself [1][2][3].

PHASE B — A REVIEWER'S QUESTIONS

Name the capability on your map whose boundary you expect to be argued about, and the sentence that settles it.

Which repository defect did you find and decide not to fix, and who agreed to carry it?

If the Chief Risk Officer reads `RPT03` and disagrees with one row, which row is it?

How many business-domain gaps did Phase B raise, and who owns the largest one?

THE CHAPTER IN TEN LINES

- Phase B is ten object sheets and seven relationship sheets. The relationships are the part that lies.
- Several `R` tab names are truncated at thirty-one characters. Spell them as the tab spells them.
- `O09 Organization Unit` has no `Parent` column, twelve generic units and no Retail, Credit or Treasury.
- `O10 Actor` and `O11 Role` hold the same eighteen names in the same order. One of them is wrong.
- The role catalogue contains no banking roles, so the credit mandate ladder has nowhere to live. Add it.
- A rand limit has no column anywhere in the repository. Extend `O11` or add a mandate sheet, and record `Mandate source`.
- `O14`'s `Capabilities` column and `R02` disagree for two of five value streams. Make `R02` authoritative.
- Four tests for a capability: noun not verb, survives a reorganisation, one nameable owner, scorable.
- `O13`'s 108 rows below level 1 are generated from a word list and appear in no report. Delete and rebuild.
- Publish the maturity scale and its evidence before the workshop, score blind, then argue from evidence.
- `Criticality` does not inherit in the shipped data. A child may be less critical than its parent, never more.
- `Disposition` is derived from `Gap` on all twelve level 1 rows. That is arithmetic, not a decision.
- `R22` is a diagonal: Human Resources owns Credit Risk Management and Internal Audit owns Finance.
- `O13`, `O17` and `O15` carry the same twelve names. Three sheets recording one list is not a model.
- `O16` has no `Volume p.a.`, no `Owner` and no `Level`. Add all three before you load a row.
- `O29 Gap` holds zero records. A Phase B that raises no business gap did not look.

SOURCES FOR THIS CHAPTER

- **1.** Banking_TOGAF_Metamodel.xlsx, Phase B sheets O09–O19 and R01, R02, R03, R04, R21, R22, R25 (Ardent Union Bank, illustrative data)
- **2.** Banking_TOGAF_Metamodel.xlsx, sheets RPT03 Capability Heatmap, RPT07 Gap Analysis, RPT13 Orphan Analysis, RPT01 Dashboard and ZZ Checks
- **3.** Banking_TOGAF_Metamodel.xlsx, sheets O29 Gap (zero records), O31 Deliverable, O35 Business Building Block, O57 Governance Body and O58 Governance Gate
- **4.** The Open Group — TOGAF Standard, 10th Edition: ADM Methods, Phase B Business Architecture https://pubs.opengroup.org/architecture/togaf10-doc/adm/
- **5.** APQC — Process Classification Framework and process frameworks overview https://www.apqc.org/process-performance-management/process-frameworks
- **6.** BIAN — Service Landscape 14.0 release notes, 20 February 2026 (322 Service Domains, 586 business capabilities; no published licence) https://bian.org/wp-content/uploads/2026/02/BIAN-v14.0-Release-Notes-v1.0_-Final-Version.pdf
- **7.** The Open Group — Publication Y201, the BIAN reference model expressed in ArchiMate https://publications.opengroup.org/y201
- **8.** National Credit Regulator — National Credit Act affordability assessment regulations, GoN 202 of 2015 (regulation 23A) https://www.gov.za/sites/default/files/gcis_document/201503/38557rg10382gon202.pdf

CHAPTER 5

Data Architecture

One column decides this phase. The other seven are description.

One column decides this phase. `Owner` on `O20 Data Entity` is the whole data architecture; the other seven columns describe it [1]. If you leave this chapter with fourteen named accountable people and nothing else, Phase C has succeeded.

Ardent Union Bank, the bank in the repository, is an invention, as is Table Bay Bank in volume 1. Their data does not mix.

> **BEFORE YOU OPEN A SHEET**
>
> Volume 1 does the teaching. This chapter does the filling.
>
> - **Volume 1, chapter 9, *The spine: a party, an agreement, an account*.** The fifteen-entity spine and the arguments about each.
> - **Volume 1, chapter 10, *Ownership and lineage*.** Why `Owner` is the column that matters.
> - **Volume 1, chapter 10, *Classification, consent and POPIA*.** The four columns POPIA actually needs.
> - **Volume 1, chapter 15, *CoDI, and a regulation that is really a data specification*.** The worked case of a regulator specifying a data structure.

What you are filling

The data half of Phase C is eleven object sheets and two relationship sheets: `O20 Data Entity`, `O21 Logical Data Component`, `O22 Physical Data Component`, `O36 Data Entity Relationship`, `O37 Data Attribute`, `O38 Data Set`, `O39 Data Migration`, `O40 Source To Target Mapping`, `O42 Metadata Element`, `O43 Master Data Rule`, `O44 Data Building Block`, plus `R05 Application manages Data En` and `R19 Data Entity relates to Data` — the last two truncated at Excel's thirty-one characters, so spell them as the tabs spell them.

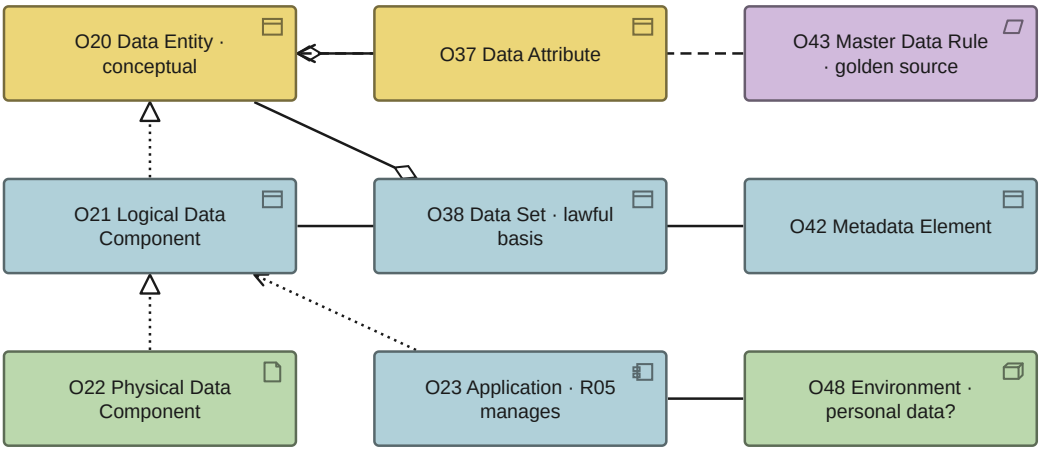

Figure 5.1 The data half of Phase C as one ArchiMate 4 view. Conceptual at the top, logical in the middle, physical at the bottom, with the three sheets that qualify an entity rather than describe it down the right.

One report reads any of it. `RPT05 Data Ownership` is `O20` with `Definition` dropped, and it carries a computed footer: "3 of 14 entities have no lineage from report back to source." The footer is correct, and it is the only automated data-quality statement this repository makes about its own data layer.

> **THE RULE FOR THIS PHASE**
>
> Fill `Owner` and `Master system` in the same sitting, with both candidate owners in the room. An `Owner` without a `Master system` is a person with no system; a `Master system` without an `Owner` is a system with no person. The repository has both filled for all fourteen entities and almost every pairing is wrong, which is worse than either blank.

The steps

Step 5.1 — Inherit Phase B and set the entity scope

Produce. One page: which entities this iteration covers, at which layer, and which are explicitly deferred.

You need. The Phase B result — `O13 Business Capability`, `O14 Value Stream` and `O16 Business Process` as you left them. The fourteen rows of `O20 Data Entity` as delivered.

Look it up. Volume 1, chapter 9, *How many*, for the number of conceptual entities a bank needs, and *The afternoon that produces one* for how long it takes.

Where it lands. `DLV-07 Data Architecture` on `O31 Deliverable`, which is marked `Approved` in the repository you were given. Set it to draft.

How.

1. List the entities each in-scope value stream touches. If `VS02 Apply to Fund` is in scope, so are Party, Arrangement, Credit Exposure, Collateral and Limit.
2. Decide the layer. Conceptual (`O20`) for everything in scope; logical (`O21`) for what a project will build against; physical (`O22`) only for the systems Phase E will touch.
3. Decide the attribute boundary now. `O37 Data Attribute` holds sixty rows and will hold six hundred if you let it.
4. Write the deferral list with a date, so a deferred entity is a decision and not an oversight.

Signed by. The Chief Data Officer (`ORG-06` , `ACT-05`), with `GB-03 Data Governance Council` noting the scope.

Done when. Every row of `O20` is marked in, out or deferred, and the attribute boundary is a written list of entity names.

Shows up in. `RPT05 Data Ownership` shows all fourteen entities regardless of scope, so the scope page is the only place the boundary exists.

Goes wrong when. Scope is set by sheet rather than by value stream, and three weeks go on `O37` while `Owner` on `O20` is still a job title nobody holds.

SETTING THE DATA SCOPE

Which entities does this iteration cover, and which value stream put each one in scope?

Which entities get attribute-level treatment, and what makes those the ones?

What is deferred, until when, and who agreed?

Step 5.2 — Read the entity catalogue, then reconcile it against volume 1

Produce. A conceptual entity list the business recognises, with a definition per entity that a lawyer and a developer would both accept.

You need. `O20` as delivered. Volume 1 chapter 9's fifteen-entity spine and appendix A's `D-01` … `D-20` .

Look it up. Volume 1, chapter 19, *Conceptual data entities*, for the drafted twenty, and chapter 9, *Three tests for whether something is an entity*.

Where it lands.

> **O20 Data Entity**
>
> `Ref`, `Data entity`, `Class`, `Definition`, `Classification`, `Owner`, `Master system`, `Lineage`. `Class` takes `Master`, `Transactional` or `Derived` and decides how the entity is governed. `Definition` is the column that settles arguments — it must say what is in and what is out, not restate the name. `Lineage` takes `Full`, `Partial` or `Absent` and is an assertion somebody has to be able to demonstrate.

How.

1. Load volume 1's `D-01` … `D-20` as candidates. **Map by name, never by number.** Twenty drafted entities against fourteen `E` rows is not a renumbering, and the repository's `Ref` scheme is its own.
2. Apply volume 1's three tests to every row. Ardent Union's fourteen hold up better than most of its sheets: Party, Account, Product, Arrangement, Transaction, Payment Instruction, Credit Exposure, Collateral, Limit, Financial Instrument, General Ledger Entry, Regulatory Report and Alert / Case are all defensible for a bank.
3. Check `Class` against what you intend to do with each entity. `E13 Regulatory Report` is correctly `Derived`, which makes its lineage claim a statement about every upstream entity.
4. Rewrite `Definition` where it is circular, and add what is missing. There is no Channel, no Document, no Consent, no Mandate and no Position. Mandate matters most here, because Registered Mandates and DebiCheck are mandate-driven rails and a mandate is not an instruction.
5. Resolve the duplication with `EXT Industry Extension`. `X03 Exposure`, `X04 Limit` and `X05 Regulatory Report` on `EXT` are already `E08`, `E10` and `E13` on `O20`. Decide which sheet is authoritative; do not maintain both.

Signed by. Chief Data Officer for the list; each `Owner` signs their own `Definition`.

Done when. Every `Definition` states an inclusion and an exclusion, and the entity count is the one the business recognises rather than the one that shipped.

Shows up in. `RPT05 Data Ownership` carries `Class` and drops `Definition`, so a circular definition is invisible in the report and visible only on the sheet.

Goes wrong when. Entities are added because a system has a table of that name. `E11 Financial Instrument` earns its place in a bank with a trading book and is dead weight in one without.

Step 5.3 — Split Party from Customer before you draw a single relationship

Produce. A written decision on whether Party and Customer are one entity or two, and the relationship that follows from it.

You need. `O20` read. Volume 1 chapter 9 on exactly this question.

Look it up. Volume 1, chapter 9, *Party, and why Customer is usually a bad entity*, and chapter 3, *Customer*, for the four things the word means in one bank. The failure mode is in chapter 1, *The customer who exists five times*.

Where it lands. `O20 Data Entity` rows `E01 Party` and `E02 Customer`, and the `O36` / `R19` row that joins them.

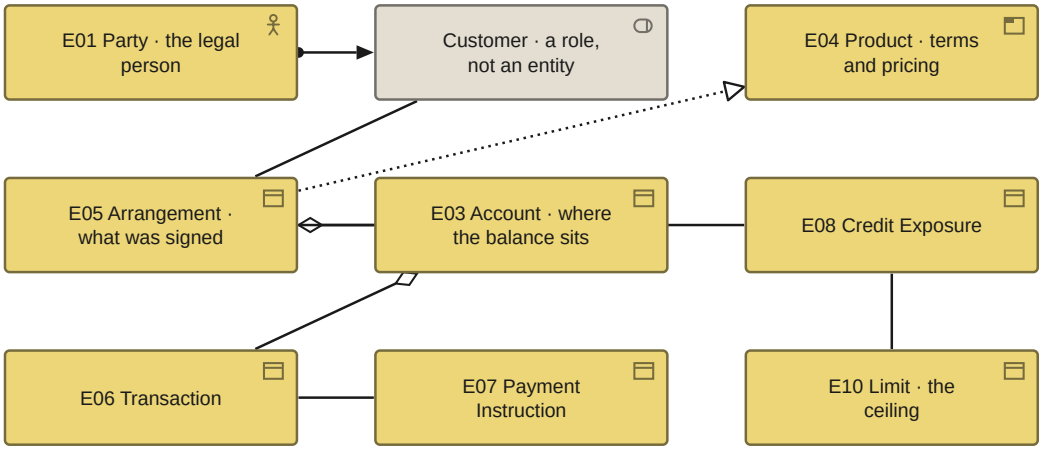

Figure 5.2 The spine done properly: Party is the actor, Customer is a role it plays, and the arrangement sits between the product and the account. Compare it with `REL-01` as delivered, which joins Party to Customer one-to-one.

How.

1. Read the shipped relationship. `REL-01` says Party `has` Customer, cardinality `1:1`. A one-to-one between two master entities says they are the same entity wearing two names.
2. Decide which model you are in. Either Party is the entity and Customer is a role it plays, in which case the cardinality is one party to many roles; or Customer is a genuine subtype, in which case name the attributes Party does not have.
3. Take the first. A party in a bank is simultaneously an account holder, a guarantor on somebody else's facility, a signatory on a company account, a beneficial owner and often an employee. All five are roles. None is a copy of the party.
4. Record the consequence for `043 Master Data Rule`. If Customer is a role, a duplicate customer is a duplicate party and the matching rule belongs on Party alone.
5. Write the sentence the bank will be held to, because CoDI asks for it in step 5.12 and the regulator asks for it in Phase G.

Signed by. `GB-03 Data Governance Council`, which `O57 Governance Body` describes as the body that "designates data owners and resolves disputes". This is the dispute it exists for.

Done when. Party and Customer are either one entity or two with a written reason, and no `1:1` relationship survives between two `Master` entities.

Shows up in. `RPT05 Data Ownership` lists both as `Master`, with different owners and different master systems — `E01 Party` owned by Head of Customer Management in Collections &

Recovery, `E02 Customer` owned by Head of Financial Crime in Regulatory Reporting. Two owners and two systems for what the relationship sheet says is one thing.

Goes wrong when. Nobody decides, both entities stay, and every downstream system builds its own join. That is how a bank gets the same person five times, which volume 1 opens on.

Step 5.4 — Settle ownership and authority, entity by entity

Produce. One named accountable person per entity and one named system of record, both agreed in the room, with the contested ones written down.

You need. `O20` and the entity list from step 5.2. The candidate owners. Two hours, and a chair who can close an argument.

Look it up. Volume 1, chapter 10, *Ownership and lineage*, and *What "single version of the truth" has to mean*.

Where it lands.

> **`O20 Data Entity`**
>
> `Owner` and `Master system`. `Owner` holds a **position title**, not a person's name — there is no `Owner role` column and no `Steward` column, so if you want a steward as well as an owner, add one. `Master system` holds the application's **name**, not its `O23 Ref`, which means the join to the application portfolio is on a text field and will break the first time a system is renamed. `Lineage` takes `Full`, `Partial` or `Absent`.

This is the step the phase turns on, and the shipped data is the clearest teaching example of getting it wrong. `E02 Customer` is owned by the Head of Financial Crime and mastered in Regulatory Reporting. `E03 Account` is owned by the Head of Lending and mastered in the Payment Hub. `E11 Financial Instrument` is mastered in Fraud Detection, `E13 Regulatory Report` in Loan Origination. Each pairing names a position and an application that exist in the bank, which is why nothing flags them.

How.

1. Ask one question per entity: who gets the call when this data is wrong? Not who maintains it, not who reports on it — who is accountable for it being right.
2. Reject a function as an answer. "Data & Analytics" is not an owner; the Chief Data Officer is a sponsor, not an owner. An owner is a business position with a budget.
3. Name the system of record from where the data is first created with authority, not from where it is most convenient to read. `A13 Data Platform` is never a master; it is a copy.
4. Record contested entities with both claims. Party and Customer will be contested. Account will be contested between the core and the payments hub. Those two arguments are the data architecture.

5. Set `Lineage` honestly. Three of fourteen entities read `Absent` — `E08 Credit Exposure`, `E10 Limit` and `E11 Financial Instrument`. Two of those feed capital and the third feeds credit. An `Absent` on a Basel input is a reportable finding, not a tidy-up.

Signed by. Each `Owner` signs their own row, in person. `GB-03 Data Governance Council` ratifies the set and settles the contested ones.

Done when. Every entity has a position title that a named human currently holds, every `Master system` is a system that creates the data, and the contested list is empty or escalated.

Shows up in. `RPT05 Data Ownership`, all seven columns, and its computed footer counting entities with no lineage. That footer is the number to quote at `GB-01 Architecture Board`.

Goes wrong when. Ownership is assigned in a spreadsheet rather than in a room. A column filled by the architect is one the owner has never seen, and the first time it matters — a regulator asking who signed off a figure — the answer is nobody.

Form C-1 — Deciding the contested sources

One row per contested entity. Columns transcribe into `Owner` and `Master system` on `O20 Data Entity`. A first pass usually contests four to six entities. Signed by `GB-03 Data Governance Council`.

Ref	Data entity	Owner	Master system	Lineage
E03	Account — example, delete me	Head of Deposits	Core Banking	Partial

OWNERSHIP, TESTED

For `E01 Party`: who gets the call when a customer exists twice, and have they agreed to that in writing?

Which entity has two claimants, and what will `GB-03` decide on?

Which entity's `Lineage` reads `Full` without anybody having demonstrated it end to end?

Step 5.5 — Reconcile the three sheets that each name a system of record

Produce. One answer per entity to the question "where does this live", and formula links or deletions so the answer is stored once.

You need. `O20`, `O43 Master Data Rule` and `R05 Application manages Data En` open side by side.

Look it up. Volume 1, chapter 19, *Application to data, with the system of record*.

Where it lands. `O20` `Master system`, `O43` `Golden source`, and `R05` rows with `Role` = `Master`. Three sheets, one fact.

THREE SHEETS, ONE FACT: WHERE DOES THIS ENTITY LIVE?

ENTITY		O20 MASTER SYSTEM	O43 GOLDEN SOURCE	R05 MASTER ROW
E01	Party	Collections & Recovery	Regulatory Reporting	Collections & Recovery
E02	Customer	Regulatory Reporting	Collections & Recovery	Regulatory Reporting
E03	Account	Payment Hub	Payment Hub	Payment Hub
E04	Product	Credit Risk Engine	Digital Channel	Credit Risk Engine
E05	Arrangement	Card Management	Payment Hub	Card Management
E06	Transaction	Loan Origination	Core Banking	Loan Origination
E07	Payment Instruction	Payment Hub	Data Platform	Payment Hub
E08	Credit Exposure	Loan Origination	Regulatory Reporting	Loan Origination
E09	Collateral	Digital Channel	Regulatory Reporting	Digital Channel
E10	Limit	CRM	Digital Channel	CRM

Shaded: O43 disagrees with the other two. Nine of ten rows. E01 and E02 have the same two answers, swapped.

Figure 5.3 `O20 Data Entity`, `O43 Master Data Rule` and `R05 Application manages Data En` each name the system of record. They agree for one entity in ten.

How.

1. Build the comparison for every entity. As delivered, `O20` and `R05` agree for all fourteen. `O43 Golden source` disagrees with both for nine of the ten entities it covers; `E03 Account` is the single exception.
2. Read the worst pair out loud. `O20` masters `E01 Party` in Collections & Recovery and `E02 Customer` in Regulatory Reporting. `O43` has the same two answers for the same two entities, swapped.
3. Pick one as authoritative. Make it `O20`, because `RPT05 Data Ownership` reads it and nothing reads `O43`'s `Golden source`.
4. Replace `O43 Golden source` with a formula to `O20`, or delete the column. A fact stored twice diverges; a fact stored twice where no report compares the copies diverges silently.
5. Note the four entities `O43` ignores. `E11` through `E14` have no master data rule, and two of them — Regulatory Report and Alert / Case — carry the strongest regulatory interest.

Signed by. Chief Data Officer, with the `Owner` of each reconciled entity confirming the single answer.

Done when. `O20` `Master system`, `O43` `Golden source` and `R05`'s `Master` rows give the same system for every entity, by construction rather than by coincidence.

Shows up in. Nowhere automatically. No report in this repository compares the three, which is precisely why they diverged. Add the comparison to your own checklist and run it every phase.

Goes wrong when. Somebody fixes one sheet. Then two agree and one does not, and the next reader picks whichever supports the decision they already made.

Step 5.6 — Redraw the entity relationships with cardinality that means something

Produce. A relationship set where every row has a verb a business person would use and a cardinality somebody has tested against real data.

You need. `O20` settled. `O36` and `R19` as delivered.

Look it up. Volume 1, chapter 9, *Agreement, Product and the thing called Account*, and *The several things the word "Account" means*.

Where it lands.

> **`O36 Data Entity Relationship`**
>
> `Ref`, `From entity`, `Relationship`, `To entity`, `Cardinality`. **`R19 Data Entity relates to Data` holds the identical eleven rows with the identical `REL-` references** — the same fact in two sheets with no formula between them. Keep `R19`, because relationship sheets are what reports read, and make `O36` a view of it or delete one.

How.

1. Read the eleven shipped rows as a chain: `E01` → `E02` → `E03` →…→ `E12`, each entity joined only to the next in `Ref` order, with verbs cycling through `has`, `references`, `is composed of`, `is derived from` and cardinalities cycling through `1:1`, `N:M`, `1:N`, `N:1`.
2. Note what that produces. "Account is composed of Product" inverts the containment. "Product is derived from Arrangement" reverses the dependency — an arrangement is an instance of a product, not the reverse. "Arrangement has Transaction, N:1" says many arrangements share one transaction. "Collateral is composed of Limit" is not a sentence anybody in credit would say.
3. Note the two entities in no row at all. `E13 Regulatory Report` and `E14 Alert / Case` appear in neither sheet, which makes them orphan entities `RPT13 Orphan Analysis` cannot detect, because it tests applications only.
4. Rebuild from the spine. An Arrangement realises a Product's terms for a Party; an Account holds balances under an Arrangement; a Transaction posts to an Account; a Payment Instruction produces Transactions; an Exposure aggregates Arrangements; a Limit constrains an Exposure.
5. Test every cardinality against a query. If nobody can run the count, write `unverified` rather than a number.

Signed by. The `Owner` of the entity on the `From` side of each row.

Done when. Every verb survives being read aloud to a banker, every cardinality has been counted or marked unverified, and every entity appears in at least one row.

Shows up in. Nowhere. There is no entity-relationship report among the thirteen, so this sheet is only ever as good as the last person who read it.

Goes wrong when. The relationships are drawn from the physical schema. A foreign key in a core banking system is a performance decision from 1998, not a conceptual relationship.

Step 5.7 — Fill the logical and physical layers without collapsing them into one

Produce. Logical components that group entities by how they will be built, and a physical inventory that records the estate as it is.

You need. `O20` settled. The database inventory from Technology (`ORG-05`).

Look it up. Volume 1, chapter 12, *15. The data platform*, and *19–21. Party, documents and identity*.

Where it lands.

> **`O21 Logical Data Component`**
>
> `Ref` , `Logical data component` , `Entity` , `Description` . One `Entity` `Ref` per row, so an entity spanning two components needs two rows.

> **`O22 Physical Data Component`**
>
> `Ref` , `Physical data component` , `Application` , `Type` . `Application` is an `O23` `Ref` . `Type` takes `Relational` , `Document` , `Time series` , `File` . There is **no `Entity` column**, so `O22` cannot be joined to `O20` except through the application, and no `Location` or `Environment` column, so you cannot say where the data physically sits — which POPIA section 72 makes a question you will be asked [4].

How.

1. Delete the one-to-one in `O21` . Twelve components for twelve entities, each named " `<entity>` store" and described as "Logical grouping of `<entity>` data", is `O20` with a prefix change.
2. Group instead. A party store holding Party, the roles it plays and its identity documents is one logical component spanning three entities. That is what the layer is for. `E13 Regulatory Report` and `E14 Alert / Case` have no component at all, so the two most reportable entities in the bank have no logical home.
3. Load `O22` from the inventory and sanity-check `Type` . The shipped values rotate: `A03 Card Management` on `Time series` , `A04 Loan Origination` on `Document` , `A06 CRM` on `File` , `A11 General Ledger` on `Time series` . A general ledger on a time-series store and a CRM on flat files are claims nobody made on purpose.
4. Note the two applications with no physical data component. `A13 Data Platform` and `A14 Fraud Detection` have none, and `O20` names Fraud Detection as the master system for `E11 Financial Instrument` . A master with no recorded store cannot be a master.

Signed by. Chief Information Officer (`ORG-05`) for `O22` ; Chief Data Officer for `O21` .

Done when. `O21` has fewer rows than `O20`, every entity has a logical home, and every `O22` row's `Type` has been confirmed by the team that runs the system.

Shows up in. `RPT02 Application Portfolio` indirectly, through `Application`. Neither sheet feeds a report of its own.

Goes wrong when. The logical layer is generated from the conceptual layer and the physical layer from the application list — both of which happened here. Two generated layers leaves Phase E nothing real to plan against.

Step 5.8 — Go to attribute level only where it earns it

Produce. Attributes for the entities a project will build against, with keys and classification that survive scrutiny.

You need. The attribute boundary from step 5.1.

Look it up. Volume 1, chapter 10, *Attributes, where they earn their place*, and *Keys, and why a South African account number is not one*.

Where it lands.

> **`037 Data Attribute`**
>
> `Ref`, `Entity`, `Attribute`, `Type`, `Key`, `Classification`. `Entity` holds the entity's **name**, not its `Ref`, so a renamed entity breaks the join. `Key` is `Yes` or `No`. `Type` takes `String`, `Code`, `Date`. There is **no `Mandatory` column, no `Length`, no `Source system` and no `Valid values`**, so a code attribute cannot carry its domain.

How.

1. Delete the shipped sixty rows. They are ten entities times the same six attributes — `Identifier`, `Name`, `Status`, `Effective date`, `Owner`, `Classification` — with no banking attribute anywhere. No account number, no BIC, no South African identity number, no exposure amount, no interest rate, no currency, no scheme reference, no mandate reference.
2. Fix the key problem while you delete. Every entity has `Key` = `Yes` on both `Identifier` and `Name`, which is two primary keys per entity and a name as one of them. A name is never a key.
3. Add the attributes that carry regulatory weight: the identity number and its verification basis, the account number and the branch code behind it, the product code that decides the BA return line, the exchange control category, the currency and the rate basis.
4. Resolve attribute classification against entity classification. The shipped data rotates them independently, so `Party.Identifier` is `Public` while `E01 Party` is `Confidential`, and `Account.Identifier` is `Restricted` while `E03 Account` is `Public`. An attribute may be

more sensitive than its entity; the entity must never be less sensitive than its most sensitive attribute.

5. Stop. Four entities at attribute level is a week. Fourteen is a quarter, and the last ten will not be read.

Signed by. The entity `Owner`, with the Information Officer confirming classification.

Done when. Every entity at attribute level has exactly one key, no attribute is more sensitive than its entity, and at least one attribute per entity would be recognised by somebody who works in that part of the bank.

Shows up in. Nowhere. `037` feeds no report, which is why sixty generic rows survived.

Goes wrong when. The team models attributes for all fourteen entities because the sheet exists. Attributes earn their place by being argued about, and nobody argues about `Status`.

Form C-2 — Attributes that earn their place

One copy per entity, for the four entities inside your attribute boundary. Columns are the real `037 Data Attribute` columns in sheet order. Eight to twelve rows per entity. Signed by the entity `Owner`.

Ref	Entity	Attribute	Type	Key	Classification
ATR-061	Party	Identity number — example, delete me	String	Yes	Confidential

Step 5.9 — Treat POPIA as columns, not as a policy document

Produce. A classification and retention position per entity that an Information Officer would sign, plus the two section-specific findings POPIA forces in a bank.

You need. `020` with `Classification` set. `048 Environment`. The bank's retention schedule.

Look it up. Volume 1, chapter 15, *POPIA, with the banking bite*, and chapter 10, *Classification, consent and POPIA*, *Section 71: a fully automated decline is a regulated act*, *Transfers, consent and marketing*, and *Retention: the longest clock wins*. Do not re-derive the law here — volume 1 has it.

Where it lands. `O20 Classification` holds the grade. Everything else POPIA needs has no column: **there is no `Personal data`, no `Special personal information`, no `Retention`, no `Lawful basis` and no `Cross-border` column anywhere in the repository.** Add three to `O20`, one to `O38` and a location to `O22`.

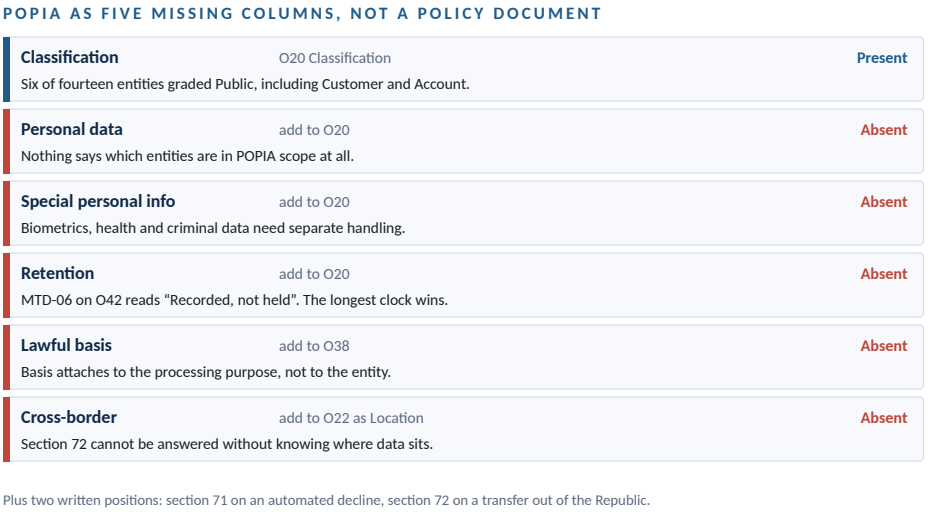

Figure 5.4 What POPIA needs from the repository, and which of it the repository can hold. `Classification` exists and is wrong; the other four positions have no column at all.

How.

1. Fix `Classification` first. Six of fourteen entities are graded `Public`, including `E02 Customer`, `E03 Account`, `E07 Payment Instruction`, `E08 Credit Exposure` and `E14 Alert / Case`. A financial-crime alert graded `Public` is not a classification error — it is a tipping-off exposure and a POPIA breach in the same cell.
2. Add `Personal data` as a flag and `Special personal information` as a second one, because POPIA treats them differently and a bank holds both.
3. Add `Retention` per entity and apply volume 1's rule that the longest clock wins. In a bank the National Credit Act, FICA and the Companies Act all run different clocks over the same record.
4. Record the section 71 position. A fully automated credit decline with no human intervention is a regulated act, which means the architecture must be able to name the decision, the data behind it and the route to human review. `A09 Credit Risk Engine` is where that lives, and nothing in the repository records whether a human reviews its declines [3].
5. Record the section 72 position. Any personal information leaving South Africa needs a lawful basis, and `O22 Physical Data Component` has no location column, so you cannot answer the question from the repository. Add `Location` to `O22` and cross-reference `O12 Location` [4].

6. Read `048 Environment` and stop. It records `Personal data present` = `No` for `ENV-01 Production` and `Yes` for `ENV-02 Pre-production` and `ENV-04 Development`. Production holding no personal data while development does is either a data-masking success nobody has heard of or the two values are inverted. Resolve it before anybody quotes the sheet.

Signed by. The Information Officer, who is the statutory role under POPIA and PAIA, with the Chief Risk Officer (`ORG-03`) on the section 71 position.

Done when. No entity holding personal data is graded `Public`, every entity has a retention period with a named statutory basis, and the section 71 and section 72 positions are written down.

Shows up in. `RPT05 Data Ownership` carries `Classification`. `RPT09 Regulatory Coverage` records `CO-05 Data protection` with `Evidence` = `In breach`, which is the repository agreeing with you before you start.

Goes wrong when. POPIA is treated as a policy appended to the architecture. It is four columns and two questions, and a repository that cannot answer them has not described the estate.

CLASSIFICATION, AND WHAT IT COSTS

Which entity is graded `Public` in the repository and holds personal information? List them.

For your longest retention clock: which statute sets it, and which system actually holds the record that long?

Does any fully automated decline reach a customer without human review? Who confirmed that?

Step 5.10 — Name the data sets and the lawful basis the sheet cannot hold

Produce. A data set register where every set has a purpose, a lawful basis and a named consumer.

You need. `038` as delivered. The entity list from step 5.2.

Where it lands.

> **`038 Data Set`**
>
> `Ref`, `Data set`, `Description`. **Three columns.** There is no `Entities`, no `Lawful basis`, no `Classification`, no `Owner`, no `Retention` and no `Consumer`. As shipped, `038` is a list of six names and cannot carry a POPIA position at all. Add `Entities`, `Lawful basis`, `Owner` and `Retention` before you load a row.

How.

1. Read the six shipped sets. `DST-01 Regulatory reporting set` through `DST-06 Quality and compliance set` are plausible names with no content behind them.
2. Add `Entities` as a list of `020` `Ref`s, so a set is a defined collection and not a label.
3. Add `Lawful basis` and fill it per set, not per entity, because the basis attaches to the processing. `DST-02 Customer analytics set` is the row that fails: analytics on customer data usually runs on legitimate interest or consent, and banks rarely record which.
4. Name a consumer per set, and cross-reference `041 Security Control`. A set nobody consumes is a copy, and a copy holding personal information with no control behind it is a liability.

Signed by. The Information Officer for `Lawful basis`; the consuming business owner for the set itself.

Done when. Every set names its entities, its basis, its owner and its consumer.

Shows up in. Nowhere. `038` feeds no report, and `ZZ Checks` does not count data sets.

Goes wrong when. `038` is filled with the names of reporting marts. A data set in POPIA terms is a processing purpose, not a database.

Step 5.11 — Write master data rules a developer could build

Produce. Match, survivorship and validation rules specific enough to implement, with a threshold and a tie-break.

You need. The Party decision from step 5.3. `043` as delivered.

Look it up. Volume 1, chapter 10, *Reference and master data*, and chapter 1, *The customer who exists five times*.

Where it lands.

> **`043 Master Data Rule`**
>
> `Ref`, `Entity`, `Rule type`, `Description`, `Golden source`. `Rule type` takes `Validation`, `Survivorship`, `Golden source`, `Match rule`. `Description` is where the rule has to live, and there is no `Threshold`, no `Tie-break`, no `Owner` and no `Effective date` column.

How.

1. Read the shipped ten and note the shape. Every `Description` reads "Rule governing `<entity>` mastering." Six of the ten are `Rule type` = `Survivorship` and exactly one is a `Match rule` — and you cannot have survivorship without matching, because survivorship decides which of two matched records wins.
2. Write the Party match rule first, because party duplication is the problem volume 1 returns to in almost every chapter. State the fields, the comparison, the threshold and what happens in the grey band between auto-merge and auto-reject.
3. Write the survivorship rule as a field-by-field precedence, not as a system precedence. The most recent address may come from the digital channel while the verified identity number comes from onboarding.
4. State the tie-break, in writing, because it is the rule that gets invoked at 2am. "Most recently updated" is a tie-break. "The core is authoritative" is not, because the core holds older data for some fields.
5. Extend `043` to the four uncovered entities, and reconcile `Golden source` against `020` as step 5.5 requires.

Signed by. The entity `Owner`, with `GB-03 Data Governance Council` approving the threshold, because a match threshold is a risk appetite statement.

Done when. A developer could implement every rule from the `Description` alone, and every `Survivorship` row has a matching `Match rule` row for the same entity.

Shows up in. Nowhere. No report reads `043`, which is how ten tautologies survived to delivery.

Goes wrong when. The rules are written as principles. "Party is mastered once" is a goal. "Match on identity number exact, plus surname fuzzy above 0.85, auto-merge above 0.95, queue between 0.85 and 0.95" is a rule.

THE SURVIVORSHIP RULE, WRITTEN TO BE BUILT

Write the Party match rule: fields, comparison, auto-merge threshold, manual-review band.

What is the tie-break when two records match and both were updated today?

Step 5.12 — Build CoDI's single customer view as the worked example

Produce. The data requirements CoDI imposes, mapped onto `O20` and `O43`, with the columns and rules the repository is missing.

You need. Steps 5.3, 5.4 and 5.11 complete. The CoDI coverage and reporting rules, and the single customer view calculations document [5][6].

Look it up. Volume 1, chapter 15, *CoDI, and a regulation that is really a data specification*, and *The single customer view*; and chapter 10, *CoDI's single customer view: a regulation that is really a data model*. Volume 1 flags one open question about submission format after 30 September 2025 — do not assume past it.

Where it lands. `O20 Data Entity` gains columns and `O43 Master Data Rule` gains rows. Nothing in the repository as delivered can compute a single customer view.

This is the best teaching case in South African banking because it is not a principle. It is a regulator specifying a record structure with a pass/fail test, and it forces specific things into specific columns.

What CoDI requires	Where it lands	What is missing today
One record per qualifying depositor, across every deposit system	`E01 Party` as the resolved identity, `O43` match rule	Party and Customer are `1:1` and mastered in two systems
Qualifying-product flag	A column on `O18 Product`, not on the account	`O18` has `Type` only, with no qualifying flag
Joint-account sharing ratio	An attribute on `E05 Arrangement` in `O37`	No sharing-ratio attribute; the rules default to equal shares
Personal and business accounts of one sole proprietor, kept apart	A role distinction on `E01 Party`	`E02 Customer` is a copy of Party, so the distinction cannot be made
Related-party identification for the bank's own executives and audit partners	A party-to-party relationship plus an employment role	No party-to-party relationship exists in `O36` or `R19`
Balance rules: gross, accrued interest in, fees out, negatives as zero	Derivation rules on `E03 Account`	No derivation column on `O20` and no rule rows in `O43`

How.

1. Read the coverage rules and the calculations document, and list every field they prescribe. The field set goes down to a reliability-of-identification flag per depositor.
2. Add the qualifying flag to the product master, not to the account. Cover follows the product's qualification, and an account-level flag drifts.
3. Add the sharing-ratio attribute even though most banks have never captured it. The default of equal shares is a reporting answer, not a data answer, and the gap belongs in `O29 Gap`.
4. Write the related-party rule. A bank must identify, in its own depositor data, which depositors are its executives, directors, their related parties and its lead audit partners. That

is a party-to-party relationship the repository has no row for.

5. Raise the gaps. Every missing column in the table above is an `O29 Gap` row with `Domain` = `Data` and a named owner.

Signed by. Chief Financial Officer (`ORG-02`) owns the submission; Chief Data Officer owns the data structure; the `Owner` of `E01 Party` owns the identity resolution.

Done when. The bank can state, for a chosen reporting date, how it would compute one record per qualifying depositor and which of the prescribed fields it cannot currently produce.

Shows up in. `RPT09 Regulatory Coverage` through `CO-05 Data protection` and the deposit obligations; `RPT07 Gap Analysis` through the gaps you raise.

Goes wrong when. The single customer view is treated as a reporting project. It is a party resolution project with a report at the end, and the report is two numbers.

Step 5.13 — Record the metadata position, including what is absent

Produce. An honest statement of which metadata the bank holds, which it has recorded and does not hold, and which is missing.

You need. `O42` as delivered.

Where it lands.

> **`O42 Metadata Element`**
>
> `Ref` , `Metadata element` , `Category` , `What it records` , `Coverage` . `Category` takes `Business` , `Governance` , `Technical` , `Operational` . `Coverage` is the useful column and takes `Full` , `Partial` , `Recorded, not held` and `Absent` .

How.

1. Read `Coverage` before anything else. Four elements read `Recorded, not held` — `MTD-06 Retention rule` , `MTD-08 Lineage` , `MTD-09 Schema` and `MTD-10 Primary key` . That value is the most honest cell in the repository: the metadata element exists as a concept and the content does not exist anywhere.
2. Cross-check against the claims elsewhere. `O20 Lineage` reads `Full` for eight entities while `MTD-08 Lineage` reads `Recorded, not held` . Both cannot be true. One of the two is the finding.
3. Do the same for retention. `MTD-06` says retention is recorded and not held, and step 5.9 has you adding a `Retention` column to `O20` . Those are the same gap seen from two sheets.
4. Set `MTD-11 Physical location` from `O22` , which has no location column — so it stays `Absent` until step 5.7 adds one.
5. Decide what you will actually hold. A metadata element with `Coverage` = `Partial` and no plan to improve it is a liability in Phase G, when somebody asks for the evidence.

Signed by. Chief Data Officer, with `GB-03 Data Governance Council` noting the absences.

Done when. Every `Coverage` value is one somebody can demonstrate, and every `Recorded, not held` has either become `Partial` or become an `O29 Gap` row.

Shows up in. Nowhere. `O42` feeds no report, and `ZZ Checks` counts only `Data entities without lineage`, which reads `3` against a threshold of `0`.

Goes wrong when. `Coverage` is set optimistically. `Full` on `MTD-07 Consent basis` in a bank that cannot say which processing runs on consent is the kind of claim that survives until an inspection.

Step 5.14 — Complete application-to-data, and repair the key column

Produce. `R05 Application manages Data En` with a usable key on both ends, and a role per row that says more than master or consumer.

You need. `O20` and `O23 Application Component`.

Look it up. Volume 1, chapter 19, *Process to data, as CRUD*, and *Application to data, with the system of record*.

Where it lands.

> **`R05 Application manages Data En`**
>
> `Application`, `Application name`, `Data entity`, `Entity name`, `Role`. `Role` takes `Master` or `Consumer` — two values, which is not a CRUD map. There is no `Create`, `Read`, `Update`, `Delete` detail and no `Interface` column.

How.

1. Fix the key before anything else. The sheet has twenty-eight rows in two blocks. The fourteen `Consumer` rows correctly put the application's `Ref` in the `Application` column. The fourteen `Master` rows put the application's **name** there instead, so `Application` reads `Collections & Recovery` where it should read `A05`. Those fourteen cannot be joined to `O23` on `Ref` at all, which makes the `00 Index` claim that "every relationship row names both ends by reference and by name" false for half this sheet.
2. Extend `Role` to a CRUD vocabulary, or add four columns. A `Consumer` that updates is not a consumer, and the difference decides where a control goes.
3. Fill the real pattern. Twenty-eight rows across fourteen applications and fourteen entities means exactly one master and one consumer each. A real bank has several consumers per entity and the count runs into the hundreds.
4. Reconcile the `Master` block against `O20 Master system` — they agree today, so fix them together or they will stop agreeing — and record the ten direct database reads `ZZ Checks` counts against a threshold of `0` as rows of their own.

Signed by. Each application `Owner` in `O23`, with the entity `Owner` confirming the `Master` rows.

Done when. Every row's `Application` value is an `O23` `Ref`, every entity has more than one consumer or a written reason why not, and the ten direct database reads each have a row.

Shows up in. `RPT13 Orphan Analysis` reports `A13 Data Platform` and `A14 Fraud Detection` as orphans because no business service maps to them — yet `R05` gives `A13` a row and `O20` names `A14` as a master system. The orphan report tests one relationship sheet and misses this one.

Goes wrong when. `R05` is filled from the integration catalogue. An interface says two systems talk; it does not say which one is authoritative for the data crossing it.

Step 5.15 — Stub the migration and the mapping while the room is still warm

Produce. A first-pass migration register and a source-to-target skeleton, recorded now and completed in Phase E.

You need. `O20` settled, `O22` loaded, and the systems the roadmap is likely to touch.

Where it lands.

> **`O39 Data Migration`**
>
> `Ref`, `Entity`, `From system`, `To system`, `Status`. `Status` takes `Not started`, `Planned`, `In progress`. There is **no `Volume`**, **no `Cutover date`**, **no `Reconciliation`** and **no `Owner`**, so a migration row cannot be planned from this sheet alone.

> **`O40 Source To Target Mapping`**
>
> `Ref`, `Entity`, `Source field`, `Target field`, `Rule`. `Rule` takes `Direct`, `Lookup`, `Transform`. `Source field` and `Target field` hold the application name plus the word "field" — `Payment Hub field` — so the sheet is structurally present and semantically empty.

How.

1. Delete the shipped rows in both sheets. `O39`'s six migrations and `O40`'s twelve mappings are offset chains through the application list, and they contradict each other: `MIG-01` migrates Party from Payment Hub to Loan Origination while `MAP-01` maps Party from a Payment Hub field to a Card Management field. Neither system is Party's master under `O20` or `O43`.
2. Record only the migrations the Phase B shortlist implies, and add `Volume`, `Cutover` and `Reconciliation` to `O39`. Reconciliation is the column that decides whether a migration can be signed off.
3. Put real field names in `O40`, even for three fields per entity. Three real rows beat twelve placeholders.

4. Mark both sheets as Phase C stubs in `DLV-07`, with a completion owner and date, so Phase E inherits them rather than rediscovering them.

Signed by. The receiving application's `Owner` for each migration row; the entity `Owner` for the mapping rule.

Done when. Every `039` row has a source and target that `020` agrees are plausible, and every `040` row names a field that exists.

Shows up in. Nowhere directly. `RPT08 Roadmap` reads `030 Work Package`, so a migration becomes visible only once Phase E gives it a package.

Goes wrong when. The mapping sheet is filled before the master system argument is settled. Then every mapping has to be redone, and the second version is always done under time pressure.

Step 5.16 — Run the Phase C data checkpoint

Produce. A signed data architecture, the gaps raised, and the defect list.

You need. Steps 5.1 to 5.15 complete for the entities in scope.

Where it lands. `DLV-07 Data Architecture` and `DLV-08 Data Model` on `031 Deliverable`, both of which arrive marked `Approved`. `044 Data Building Block` holds ten `DBB-` rows, one per entity for `E01` to `E10`, each with the acceptance criterion "No `<entity>` record without a named owner" — a criterion that cannot fail, because `020` gives every entity an owner string. Rewrite the criteria so they can fail.

How.

1. Re-run the eight checks nothing automates: `020` against `043` on the golden source; `036` against `R19` for drift; entities with no relationship; entities with no logical component; attribute classification against entity classification; `020 Lineage` against `MTD-08`; `048 Environment` on personal data; and `R05`'s `Master` block for the key problem.
2. Read `RPT05 Data Ownership` aloud, row by row, with the owners present. If anybody is hearing their own name for the first time, the phase is not finished.
3. Read `ZZ Checks`. `Data entities without lineage` reads `3` against a threshold of `0`; `Direct database reads` reads `10` against `0`. Both are Phase C findings.
4. Raise gaps in `029 Gap` with `Domain` = `Data`. The sheet holds zero records across the entire repository, so any number above zero is an improvement and fewer than five is under-reporting.
5. Rewrite the `044 Data Building Block` acceptance criteria to something testable, and set the two deliverables to a status you chose.

Signed by. `GB-03 Data Governance Council` approves the data architecture; `GB-01 Architecture Board` approves it at gate `GT-1 G1 Design` on `058 Governance Gate`.

Done when. `RPT05` has fourteen rows each naming a person who has signed, `RPT07 Gap Analysis` has data-domain rows, and every `Lineage` value of `Full` has been demonstrated end to end once.

Shows up in. `RPT05`, `RPT07`, `RPT09 Regulatory Coverage` and `ZZ Checks`.

Goes wrong when. Phase C closes on a beautiful entity-relationship diagram and an `Owner` column the owners have never read. The diagram is the artefact; the column is the architecture.

What a finished Phase C data layer looks like

Reading	As delivered	After Phase C
Entities whose `Owner` has signed	0 of 14	14 of 14
`020` and `043` agreeing on the system of record	1 of 10	10 of 10, by formula
Entities graded `Public` while holding personal data	6 of 14	0
`Lineage` = `Absent`	3 of 14, two of them Basel inputs	0, or each one a gap with an owner
Entities in no relationship row	2 of 14	0
POPIA columns on `020`	1 of 5	5 of 5
`037` rows that are banking attributes	0 of 60	Every one
`R05` rows joinable on `Ref`	14 of 28	28 of 28
`029 Gap` rows, `Domain` = `Data`	0	5 or more

Nothing in the left column is a broken spreadsheet. Every sheet is structurally sound and `00 Index` truthfully reports 878 records. What the left column shows is a data architecture describing a filing cabinet rather than a bank, and Phase C is the distance between the two columns [1][2].

PHASE C, THE DATA HALF — A REVIEWER'S QUESTIONS

Name the entity whose owner was hardest to settle, and what `GB-03` decided.

Which `Lineage` value of `Full` have you personally traced from report back to source?

If the regulator asked today how the bank computes a single customer view, what is the honest answer?

How many data-domain gaps did Phase C raise, and which is the largest?

THE CHAPTER IN TEN LINES

- `Owner` on `O20 Data Entity` is the data architecture. Fill it in a room, never in a spreadsheet.
- `RPT05 Data Ownership` is `O20` with `Definition` dropped, and its lineage footer is the only computed data-quality statement in the repository.
- `REL-01` joins Party to Customer `1:1`, which says they are one entity. Decide which, in writing.
- `O20`, `O43 Golden source` and `R05`'s `Master` rows each name a system of record and agree for one entity in ten.
- `O36 Data Entity Relationship` and `R19 Data Entity relates to Data` hold identical rows with identical refs. Keep one.
- The eleven shipped relationships are a daisy chain in `Ref` order with rotating verbs. `E13` and `E14` appear in none.
- `O21` is `O20` with a prefix change. A logical layer that mirrors the conceptual layer earns nothing.
- `O22` has no `Entity` and no `Location` column, so POPIA section 72 cannot be answered from it.
- `O37`'s sixty rows are six generic attributes times ten entities, with two keys each and a name as one of them.
- Six of fourteen entities are graded `Public`, including `E14 Alert / Case`. That is a POPIA and a tipping-off finding in one cell.
- `O48 Environment` records personal data absent from production and present in development. Resolve it before quoting it.
- POPIA needs `Personal data`, `Special personal information`, `Retention`, `Lawful basis` and a location. None exist.
- CoDI's single customer view is a record structure with a pass/fail test, and it forces a qualifying flag onto the product, not the account.
- `O43`'s ten rules are tautologies, with six survivorship rows and one match rule. Survivorship without matching is nothing.
- `R05` puts the application name where the `Ref` belongs in fourteen of twenty-eight rows, so half the sheet will not join.
- `O29 Gap` holds zero records. A Phase C that raises no data gap did not look.

SOURCES FOR THIS CHAPTER

- **1.** Banking_TOGAF_Metamodel.xlsx, Phase C data sheets O20–O22, O36–O40, O42–O44 and R05, R19 (Ardent Union Bank, illustrative data)
- **2.** Banking_TOGAF_Metamodel.xlsx, sheets RPT05 Data Ownership, RPT07 Gap Analysis, RPT09 Regulatory Coverage, RPT13 Orphan Analysis and ZZ Checks
- **3.** Protection of Personal Information Act 4 of 2013, section 71 — automated decision making https://popia.co.za/section-71-automated-decision-making/
- **4.** Protection of Personal Information Act 4 of 2013, section 72 — transfers of personal information outside the Republic https://popia.co.za/section-72-transfers-of-personal-information-outside-republic/
- **5.** South African Reserve Bank / CoDI — Coverage and Reporting Rules (final) https://www.resbank.co.za/content/dam/sarb/what-we-do/financial-stability/resolution-planning/CoDI%20Coverage%20and%20Reporting%20Rules%20Final.pdf
- **6.** South African Reserve Bank / CoDI — Single customer view (SCV) calculations document, v0.2 https://www.resbank.co.za/content/dam/sarb/what-we-do/deposit-insurance/SCV%20calculations%20document-V0.2.pdf
- **7.** The Open Group — TOGAF Standard, 10th Edition: ADM Methods, Phase C Information Systems Architectures https://pubs.opengroup.org/architecture/togaf10-doc/adm/
- **8.** Basel Committee on Banking Supervision — Principles for effective risk data aggregation and risk reporting (BCBS 239), January 2013 https://www.bis.org/publ/bcbs239.pdf

CHAPTER 6

Application Architecture

The half of Phase C where the arguments get settled, because an application has a contract, a run team and a name on it.

Phase C holds data and applications in one phase [11], and in a bank they fail differently. The data half argues about meaning. The application half argues about money, because an application has a vendor agreement, a run team, a disposition somebody has already told the board about, and a name on it. You can be vague about what Party means. You cannot be vague about who owns the core.

Sixteen steps, spined on `023 Application Component`. **Ardent Union Bank**, the bank in the repository, is an invention, as is volume 1's **Table Bay Bank**; this chapter uses only Ardent's numbers. Two mechanical warnings: the relationship tabs here are named after **what they mean** rather than their endpoints, and several are cut at Excel's thirty-one characters — `R05 Application manages Data En`, `R06 Application interfaces with`, `R18 Standard applies to Applica`. Type them as the tab types them. And the first column of every object sheet is `Ref`, not `ID`.

> **BEFORE YOU START THIS PHASE**
>
> - **Volume 1, chapter 11, *The twenty-five*.** The conceptual application model, `A-01` to `A-25`.
> - **Volume 1, chapter 11, *The patterns*.** Usually one, usually several, usually bought, usually built, usually missing.
> - **Volume 1, chapter 12, *The shape of the estate*.** What a South African bank actually runs.
> - **Volume 1, chapter 19, *Conceptual applications* and *Capability to application — the coverage map*.** The pre-drafted catalogue and a worked coverage grading.
>
> Run `ZZ Checks` first and keep the numbers: 65 object types, 878 records, 465 relationship instances. Six of its results are findings this chapter produces [10].

Start at the model, not the estate

There is a reliable way to waste six weeks. Somebody sends an export from the service-management tool, it has four hundred rows, a hundred of them printers, and you clean it. At the end you own a tidy list and still cannot say whether it is complete, because nothing in it says what ought to have been there.

Volume 1 chapter 11 says. Twenty-five conceptual application capabilities, `A-01` to `A-25`, described by what they do with no product name in them. Walk them in order and ask one question of each: *what does this job in this bank, and how well*. A capability with nothing behind it is a gap you can name in week one. The five usually absent matter more than the twenty usually present, because they are absent for the same reason everywhere: `A-05` product definition and pricing, `A-04` party master with relationship and mandate, `A-09` rail adaptation and clearing, `A-18` regulatory reporting and

submission, A-24 document, evidence and communication. All five are **shared**, and no product line funds a capability it must share with four others — a funding problem dressed as an architecture problem, and it predicts most of what follows.

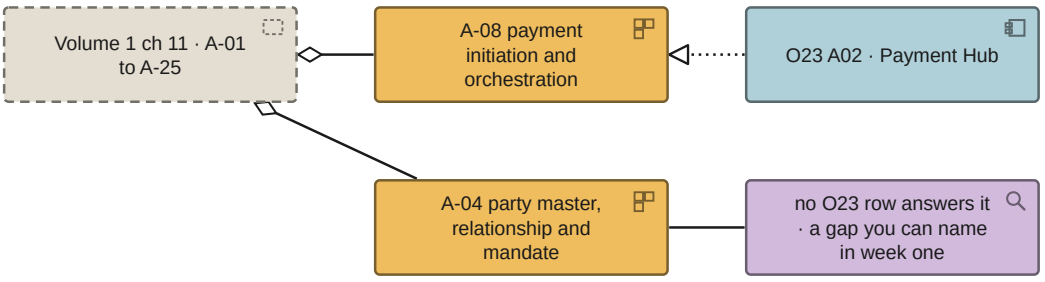

Figure 6.1 The direction of travel, in ArchiMate 4. Volume 1 chapter 11's twenty-five conceptual application capabilities aggregate into the model; a real 023 component realises one of them. A capability nothing realises is a gap you can name before the inventory is finished — and in Ardent Union Bank four of the five usually-missing capabilities are among them.

Ardent's 023 holds fourteen applications [1], and read against the twenty-five the shape is immediate. A01 Core Banking answers A-06 and A-07; A02 Payment Hub answers A-08; A03 Card Management answers A-10 and probably A-11; A07 Digital Channel answers A-01 and A-02 together, which is one application doing two jobs. Then the holes: nothing answers A-04, A-05, A-09, A-24 or A-25. Four of the five usually-missing are missing here too, and the fifth, A-18, is claimed by A12 Regulatory Reporting, whose only recorded interfaces are files and direct reads.

> **THE SPINE OF THE APPLICATION HALF OF PHASE C**
>
> Walk the conceptual model. Map real systems onto it. Record what you find in 023. Never let the inventory define the model: an inventory has survivor bias built in, because the systems easiest to see are the ones somebody already watches.

Step 6.1 — Lay the conceptual model across the estate

Produce. A one-page mapping from A-01 … A-25 to the systems you believe deliver each, blanks left visibly blank.

You need. Volume 1 chapters 11 and 12. Ninety minutes each with the heads of retail, business banking, payments, credit, finance and financial crime.

Look it up. Volume 1, chapter 11, *The table*, gives all twenty-five with purpose, user roles, data mastered and typical system. Map onto 023 **by name, never by number**: here A08 is the sanctions screening system, not A-08.

Where it lands.

> **047 Reference Model Entry**
>
> Four columns: `Ref`, `Reference model entry`, `Description`, `Status`. Eleven rows, ten `Adopted`, one `Candidate` [9]. The nearest thing to a home for the conceptual model's answers, and not quite that: the rows are target patterns, not jobs to be done. Add nothing yet; keep the mapping as a working page and bring the holes to step 6.15.

How.

1. Copy the twenty-five into a working list, the five usually-missing at the top.
2. Against each, write the system you expect and the person who would know, before you research — your prior, on the record, so you can be wrong about it.
3. Hold the six interviews, one question per entry: *what does this job here?*
4. Two systems, record two. A spreadsheet, record the spreadsheet. No answer, `None`.
5. Count the `None` rows. For Ardent it is at least five, and that count opens the reading.

Signed by. Chief Enterprise Architect (`ROL-08`), with the Chief Information Officer (`ROL-04`) acknowledging the system column.

Done when. All twenty-five are mapped or `None` with a reason, and the five usually-missing each carry a written verdict.

Shows up in. Nothing yet, which is the point: the page exists before any report can be trusted. It reappears at step 6.16.

Goes wrong when. You map the model onto the estate rather than the estate onto the model. The tell: every capability gets one system and nothing reads `None`.

The inventory itself

`O23 Application Component` is ten columns and that is all: `Ref`, `Application`, `Type`, `Description`, `Disposition`, `Criticality`, `Health`, `In service`, `Users`, `Owner` [1]. No cost, hosting, vendor, lifecycle or version columns — this repository carries no record-keeping columns anywhere, which the Preliminary chapter deals with.

Group	Columns	Who fills it	When
Identity	`Ref`, `Application`, `Type`, `Description`	Architect	Step 6.2
Accountability	`Owner`, `Criticality`	Architect with CIO	Step 6.3
State and size	`Disposition`, `Health`, `In service`, `Users`	Owning exec with finance	Steps 6.3, 6.4

`Type` takes three values at Ardent: twelve `Package`, one `Custom` (`A07`), one `Platform` (`A13`). A useful column badly used — it records build provenance, which dates, rather than whether the row is running or designed. To hold a target application, and in a bank you always must, add a column. Do not borrow `Type`.

Step 6.2 — Open a row for every application you can name

Produce. `O23` populated with `Ref`, `Application`, `Type` and `Description` for every application, including the ones that do not exist yet.

You need. Step 6.1. The service-management list if one exists. `O09 Organization Unit` and `O11 Role` from Phase B, because step 6.3 cannot fill `Owner` against empty sheets.

Where it lands.

> **`O23 Application Component`**
>
> `Ref` is `Ann`, sequential, never reused, never renumbered. `Application` is what people in the building call it, not the vendor's product name — Ardent's read `Core Banking`, `Payment Hub`, `Card Management`. `Description` is one sentence on what it administers: `A01` reads "Accounts, balances, interest and posting", the ledger boundary in six words. `Type` from `Package`, `Custom`, `Platform`.

How.

1. One row per entry from step 6.1, named as the business names it.
2. Write `Description` about what it holds and posts, not about its technology.
3. Keep the count honest: fourteen rows counts applications, not servers. Four hundred means an asset register.
4. Keep vendor product names out, or the catalogue becomes a procurement document and dates within a year, and leave the other five columns empty until the next two steps.

Signed by. Chief Enterprise Architect.

Done when. Every row has four identity columns and no two rows describe the same boundary.

Shows up in. `RPT02 Application Portfolio`, which reprints `O23` minus `Description`: nine columns, fourteen rows, no computed column, no footer [2]. It adds nothing to `O23`, so any assessment you want you compute or add as columns.

Goes wrong when. The boundary is drawn round a licence rather than a job. Twelve finance workbooks that together assemble the BA returns are one row: one application, badly implemented.

Step 6.3 — Fix the accountability columns before anybody reads the sheet

Produce. `Owner` and `Criticality` filled, each defensible in one sentence, plus a list of what you changed.

You need. Step 6.2. `O09 Organization Unit`, twelve rows, and `O15 Business Service` with its own `Criticality`.

Look it up. Volume 1, chapter 12, *The shape of the estate*, before accepting anybody's criticality rating for a payments hub.

Where it lands.

> **`O23 Application Component`**
>
> `Owner` holds an organisation unit — but in the shipped file it holds the unit's **name** as free text rather than its `ORG-` reference: `Executive`, `Customer Service`, `Operations`, `Commercial`, `Internal Audit`, `Legal`, `Data & Analytics`, `Finance`, `Risk & Compliance` [1]. Change it to the reference or you cannot join it to anything. `Criticality` is `Critical`, `High`, `Medium` or `Low`, on one test: if this stops for a working day, does the bank breach a customer obligation, a scheme rule or a supervisory deadline?

How.

1. Replace every `Owner` name with the matching `O09` reference. No matching unit is a finding, not a typo.
2. Reject an owner who cannot be accountable. `Internal Audit` (`ORG-11`) owns `A05 Collections & Recovery` and `A08 AML / Sanctions Screening`; `Legal` (`ORG-12`) owns `A07 Digital Channel` [1]. Audit owning two systems it must assess is an independence breach, and belongs in the catalogue as one.
3. Apply the test row by row. Ardent reads three `Critical` — `A03`, `A11`, `A12` — and `A02 Payment Hub` reads `Low`. A hub that clears and settles cannot be `Low`; its outage is reportable the same day.
4. Cross-check `O15`: `BSV-05 Payments & Settlement Service` also reads `Low` [3]. When two sheets agree on something impossible, the generator agreed with itself.
5. Record what you changed and who confirmed it, as the defence for month four.

Signed by. Chief Information Officer for `Criticality`, with Executive Committee (`GB-04`) as escalation when two units claim one application.

Done when. Every `Owner` resolves to a live `O09` reference, no assurance unit owns an operational application, and every `Critical` and `Low` carries its one-sentence test.

Shows up in. `RPT02`, columns `Criticality` and `Owner`. If `Low` still sits against a clearing or settlement system, the column was not assessed.

Goes wrong when. `Owner` becomes a committee. An owner is a unit with a budget and a head, which `O09` gives you in its own column.

BEFORE YOU LET ANYONE QUOTE RPT02

Which application has an owner that cannot be accountable for it, and who should own it instead?

Which `Criticality` value would you have to defend to a supervisor, and what is the sentence?

`Owner` holds names, not references. What breaks the first time an organisation unit is renamed?

Step 6.4 — Add the five columns the sheet does not have

Produce. `O23` extended with run cost, hosting, vendor class, lifecycle and state, each with a stated source, and a catalogue note saying the columns were added.

You need. Step 6.3. Run cost per application from the ledger. `O19 Contract`, ten rows, and the agreements themselves.

Where it lands.

> **`O23 Application Component`**
>
> Five new columns after `Owner`, so nothing already written moves. `Annual run cost (Rm)` is licence plus support plus infrastructure plus the run team. `Hosting` is `On-premise`, `Cloud` or `SaaS`, from where the compute runs rather than who operates it. `Vendor class` is a class, not a name: `In-house`, `Banking package vendor`, `Specialist vendor`, `To be selected`. `Lifecycle` is factual — `Current`, `Legacy`, `Emerging`, `Declining` — and stays apart from `Disposition`, which is intentional.

How.

1. Take cost from the ledger, not the budget, and reconcile against the technology run line.
2. Fill `Vendor class` from `O19 Contract`, reading that sheet while you are in it: its `Counterparty` column holds the **application name**, not a vendor — `CTR-01`'s counterparty is "Core Banking" [10]. No vendor is named anywhere in this repository.
3. Note the contracts that do not exist. The ten cover `A01` to `A10`; `A11`, `A12`, `A13` and `A14` have none, and two of those four are `Critical` [1][10].
4. Check contract type against expiry. Six of the ten read `Perpetual` and carry an `Expiry` date: that is a support term, not a licence term, and somebody will read it as a cliff.
5. Sanity-check `Users`. The rows sum to 2,497 named users against 2,012 staff, which is plausible; `A10 Treasury Management` at 304 users is not, being more people than the whole Finance unit's 150.

Signed by. Chief Financial Officer (`ROL-02`) for cost, Chief Information Officer for the rest, Architecture Board (`GB-01`) approving the additions so the next architect knows they are not in the shipped file.

Done when. Cost reconciles within five per cent and the added columns are listed with their provenance.

Shows up in. `RPT02` only if you extend that sheet too — it is a reprint, so a column added to `O23` alone is invisible to everybody who reads reports.

Goes wrong when. Cost is allocated rather than measured, which makes everything downstream a function of headcount.

Finding the applications nobody declares

Every estate holds applications no inventory has, because the inventory was built from what technology operates and these are what the business operates. Three rules find most of them, and each produces evidence for a sheet rather than an opinion for a slide.

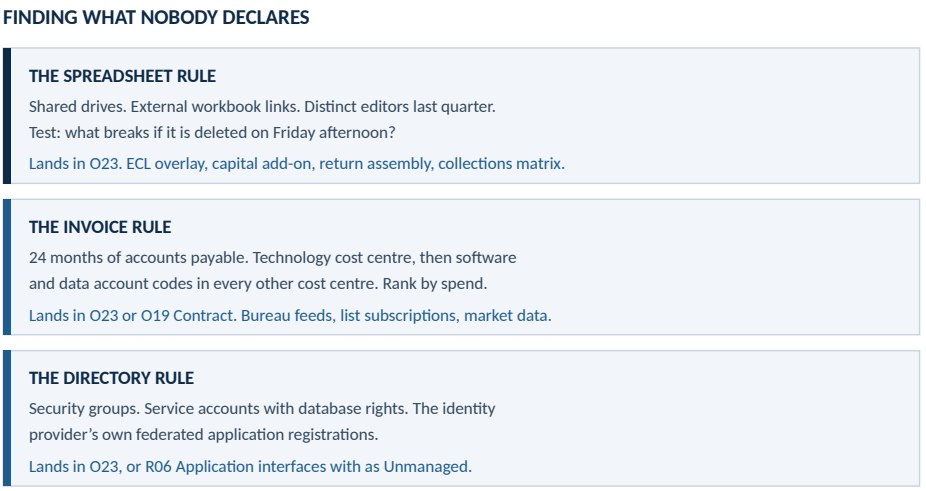

Figure 6.2 The three discovery rules, what each one reads and where the finding lands. Run all three: each misses a different class of undeclared application.

The spreadsheet rule. A spreadsheet is an application when three things are true at once: more than one person edits it, it has a refresh rhythm, and something downstream depends on its output. Ask the owner one question — *what happens if this is deleted on Friday afternoon?* — and open an `O23` row if the answer names a BA return, a payment run, an impairment number, a scheme settlement or a provision. In a South African bank the classes that most often sit in a spreadsheet are the expected-credit-loss overlay, the capital add-on reconciliation, the regulatory return assembly layer and the collections treatment matrix. Ardent's `A12 Regulatory Reporting` is `Critical` , has no contract, and its only recorded interfaces are files and direct reads [1][4]; a spreadsheet layer under it is the likeliest thing in this estate.

The invoice rule. Pull twenty-four months of accounts payable from the general ledger, Ardent's `A11`, and filter twice: on cost centre equals technology, which finds what you know, then on every software, licence, subscription, data-feed and professional-services account code **across all other cost centres**, which finds what you do not. Shadow applications are paid for by the business, which is why the technology asset register lacks them. In a bank this rule finds a predictable set — credit bureau feeds, sanctions and politically-exposed-person list subscriptions, market data, model validation tooling, dialler capacity. Every vendor with a recurring payment and no `O23` row is an application you have not recorded, a contract missing from `O19 Contract`, or a service nobody uses. Five hundred thousand rand a year is a defensible materiality line at Ardent's size.

The directory rule. Export security groups, service accounts and the identity provider's application registrations. A security group named after a system with no `O23` row means the system exists and somebody grants access to it. A service account with database rights and no owning application is almost always an interface, and belongs in `R06 Application interfaces with` as `Unmanaged`. And the federated application list will be longer than your `SaaS` count every time; in a bank it also finds the regulator and scheme portals staff reach with named credentials, which are architecture objects whether or not anybody bought them.

Step 6.5 — Run the three discovery rules and close the inventory

Produce. A dated discovery log, new `O23` rows, and a written statement of what you chose not to record and why.

You need. Step 6.4's reconciliation. Read access to accounts payable, the identity provider, and the finance, credit-risk and regulatory-reporting shared drives. One analyst for four days.

Where it lands.

> **`O23 Application Component`**
>
> New rows, with how each was found written into `Description`. Provenance matters more here than anywhere else, because these are the rows that get challenged. Where `Hosting` and `Vendor class` are unknown, record an estimate and flag it rather than leaving the row out.

How.

1. Spreadsheet rule: shared drives, external workbook links, distinct editors last quarter, then interview the top twenty by dependency.
2. Invoice rule: twenty-four months, technology cost centre first, then software and data account codes everywhere else, ranked by spend.
3. Directory rule: security groups, service accounts with database rights, federated application list.
4. Give every candidate one of three outcomes: a new `O23` row, a new `O19 Contract` row, or rejected with a reason.
5. Log every rejection. The rejections are what make the inventory defensible six months later.

Signed by. Chief Information Officer, with the Chief Financial Officer confirming the extract was complete.

Done when. The cost reconciliation closes within five per cent and the count discovery found that the original list lacked is written down. Zero means a rule was not really run.

Shows up in. `RPT02` row count, and `RPT13 Orphan Analysis`, which grows before it shrinks: a newly found application has no business service mapped to it yet.

Goes wrong when. You find nine spreadsheets and open nine rows. An application is a capability boundary, not a file.

WHAT THE THREE RULES TURNED UP

How many applications did discovery find that the original list did not have? If zero, which rule did you not really run?

Which spreadsheet would stop a BA return, a scheme settlement or an impairment close if it were deleted on Friday?

Which vendor takes a recurring payment from a business cost centre with no `023` row and no `019 Contract` row?

Working table — the discovery log

One row per candidate, rejections included — they are what make the inventory defensible later. `Found by` is the rule: spreadsheet, invoice or directory. A first pass usually needs twelve to twenty rows. Signed by the Chief Information Officer, with finance confirming the extract was complete.

Candidate	Found by	Evidence	Outcome	Reason
ECL overlay workbook	Spreadsheet	5 editors, monthly close	New `023` row	Feeds the impairment number

Assessing the portfolio

The repository gives you one assessment column, `Health`: Ardent reads two `Healthy`, four `Adequate`, three `At risk`, five `Poor` [1][2]. One column cannot carry a portfolio assessment, because it collapses two independent questions — is this worth having, and is it built well — into a number nobody can argue with productively. Add two columns, score them separately, keep `Health` as the operations view it is, and use the same method unchanged every year or the trend means nothing.

Business value, out of 12. Four sub-scores of 0 to 3, scored by the business owner:

Sub-score	0	3
Balance-sheet or revenue exposure touched	none	the majority of advances or transaction revenue
Regulatory or scheme necessity	none	a return or a settlement cannot be produced without it
Differentiation	commodity	a reason customers choose this bank
Breadth of capability support	one level-1 capability	five or more

Technical fitness, out of 10. Five sub-scores of 0 to 2, scored by the owning engineer, each with its evidence row named:

Sub-score	Evidence	0	2
Platform support currency	`O27` `Support end` via `R07`	past, or inside twelve months	beyond five years
Standards conformance	`R18` `Conformance`	any `Breach`	`Conformant` throughout
Integration style	`R06` `Type`, `Managed`	a `Direct DB read` row exists	every row `Managed`
Interface observability	`R06` `Logged`	any `No`	every row `Yes`
Recoverability	`O49` `Tested`	`Never tested`	tested inside the cadence

Take differentiation from the level-1 rows of `O13 Business Capability` the application supports, not from the owner's view of their own system, and breadth from step 6.10 — which is why the workshop runs a second pass after the coverage work. Never let one person score both axes: an owner scoring value will score fitness low to attract investment, and an engineer scoring fitness will score value high to protect the team.

Step 6.6 — Score both axes, in one workshop with two scorers

Produce. `Business value` 0 to 12 and `Technical fitness` 0 to 10 for every row, sub-scores in the workshop pack, every extreme sub-score naming its evidence.

You need. Step 6.5. `O13 Business Capability`, 120 rows; `O18 Product`, eight rows; `O27 Physical Technology Compone`, `R18`, `R06`, `O49 Resilience Tier`. The named business owner and the owning engineer per application.

Look it up. Volume 1, chapter 11, *As the basis for the application portfolio*, for what the conceptual model is and is not evidence of when somebody argues their system is strategic. Then volume 1, chapter 12, *The legacy question*, before anybody proposes replacing a core on fitness grounds alone.

Where it lands.

> **`O23 Application Component`**
>
> Both scores as integers in the columns added at step 6.4. The sub-scores have no home here: keep them in the workshop pack attached to `DLV-09`, and say in the catalogue that the sheet holds the total and the pack the working. Where evidence does not exist, score 0 and note it — a recovery test that never ran is a fitness problem, not an unknown.

How.

1. Pre-fill exposure and necessity from `O18 Product` and `O63 Control Objective`, so the room argues two sub-scores rather than four. Run it in `Ref` order, ninety minutes per axis; anything over three minutes is parked.
2. Join `R07` to `O27` for `Support end`, then look at what you joined: `R07` has fourteen rows, all `Environment` = `Production`, and it is positional — `A01` to `PTC-01` through `A12` to `PTC-12`, then `A13` back to `PTC-01` [1]. It records `A05 Collections & Recovery` on the HSM estate and `A07 Digital Channel` on a storage array. Rebuild it at step 7.2 before scoring against it.
3. Take conformance from `R18`, where seven of fourteen applications appear at all [8]; integration style and observability from `R06`, which holds ten `Direct DB read` rows and twenty of thirty-two interfaces unlogged [4]; recoverability from `O49`, where ten of twelve rows read `Never tested` and two applications have no row.
4. Resolve parked rows offline with the two who disagreed, circulate, then write the totals.

Signed by. The owning executive per application for value; Chief Information Officer for fitness, on the record of the application domain architect (`ROL-11`).

Done when. Every row has two integers, the parked list is empty, and every sub-score of 0 or 2 names its evidence row. Middle scores may be judgement; the extremes may not.

Shows up in. `O23`, and `RPT02` once you extend it.

Goes wrong when. Everything scores 7 or 8 on value, which is a room scoring importance; or fitness tracks age, which is one input to one sub-score. `A03 Card Management` went live in 2023 and reads `Poor`; `A11 General Ledger` went live in 2013 and reads `Healthy` [1].

DISPOSITION AGAINST HEALTH · ARDENT UNION BANK, 14 ROWS

Disposition	Poor	At risk	Adequate	Healthy
Retain	1 A03	2 A01, A14	2 A05, A12	1 A11
Invest	2 A02, A06	—	—	1 A09
Consolidate	—	1 A04	—	—
Replace	1 A10	—	1 A13	—
Retire	1 A07	—	1 A08	—

Four cells in red, five applications: three Poor systems carry Retain or Invest, and two Adequate ones carry Retire or Replace. None of the five has a recorded explanation.

Figure 6.3 `O23 Application Component`'s own `Disposition` and `Health` columns crossed, with Ardent Union Bank's fourteen applications counted into each cell. Ardent is an invention; the arithmetic is the shipped file's [1][2].

Step 6.7 — Derive the grid and test the disposition against it

Produce. A quadrant per row, a verdict on whether the recorded `Disposition` agrees with it, and one sentence per disagreement.

You need. Step 6.6, and a `Disposition` column set and locked **before** the scores were revealed. Otherwise this step tests nothing.

Where it lands.

> **`O23 Application Component`**
>
> Two added columns, `Quadrant` and `Disposition agrees`. Value 6 or more is high, fitness 5 or more is high. High and high is `Invest`; high value and low fitness is `Migrate`; low value and high fitness is `Tolerate`; low and low is `Eliminate`. The verdict is `Yes` when the recorded disposition sits in the quadrant's permitted set — `Invest` permits `Invest` and `Retain`, `Migrate` permits `Replace` and `Consolidate`, `Tolerate` permits `Retain`, `Eliminate` permits `Retire` — and `No` otherwise. Ardent's vocabulary maps onto those sets without extension.

How.

1. Compute the quadrant mechanically. Never adjust a score to move an application into a quadrant somebody prefers.
2. Compare each disposition against the permitted set and write `Yes` or `No`.
3. Hold a ten-minute conversation per `No` and write one sentence: change the score, change the disposition, or keep both and record why. Silence is the only wrong answer.

4. Run the same test against `Health`, which you already have. Three of the five `Poor` rows carry `Retain` or `Invest` — `A03 Card Management` (`Critical`, `Poor`, `Retain`), `A02 Payment Hub` and `A06 CRM` (both `Poor`, `Invest`) — and two carry `Adequate` with `Retire` or `Replace`, `A08` and `A13` [2]. Five disagreements in fourteen, before you have scored anything.
5. Carry the disagreements into Phase E as a named input to `054 Architecture Alternative`.

Signed by. Architecture Board (`GB-01`), monthly. Its mandate in `057 Governance Body` is to hold gates `G1` and `G2` and approve dispensations with an expiry — the right power here, and not a funding power.

Done when. Every row has a quadrant and a verdict, and every `No` an explanation somebody signed.

Shows up in. `RPT02`, columns `Disposition` and `Health`. The quadrant and the verdict need your added columns.

Goes wrong when. The disposition is set after the scores. Then the verdict reads `Yes` everywhere and the test only agrees with itself.

WHERE THE GRID AND THE DISPOSITION DISAGREE

Were the dispositions locked before the scores were revealed? If not, what does the verdict actually test?

`A03 Card Management` is `Critical`, three years in service, `Health` = `Poor`, `Disposition` = `Retain`. Which of those four is wrong?

Which application scores into `Eliminate` and has a defender? Who is it, and what are they protecting?

Working table — the disposition disagreements

One row per disagreement, from either test: quadrant against `Disposition`, or `Health` against `Disposition`. The explanation is one sentence written in the ten-minute conversation. Ardent produces five from `Health` alone. Carry what survives into `054 Architecture Alternative`. Signed by the Architecture Board.

Ref	Quadrant	Disposition	Value / fitness	Why they disagree
A02	Migrate	Invest	9 / 3	Hub rebuild already contracted

THE WORKSHOP THAT PRODUCES ALL OF THIS

One session, three hours, nine people: the Chief Enterprise Architect chairing, the application and data domain architects, the Chief Information Officer, the finance business partner, the Chief Risk Officer or a delegate, and two business owners rotating by application group. Agenda: dispositions read out from `023` and locked; value scored; break; fitness scored; the grid computed live on the sheet; disagreements listed and **not** resolved in the room. Resolution is a two-week follow-up with named owners. Locking the dispositions first is the one mechanic that makes the test worth running, and it costs nothing but discipline.

Services, business services, and what coverage can and cannot show

An application component is something you buy and run. An information system service is something other things call. Separate sheets let you replace the first without renegotiating the second.

Step 6.8 — Name the information system services and record which are published

Produce. `024 Information System Service` populated, with `Style` and `Published` filled from observation rather than intent.

You need. Step 6.2. `015 Business Service`, twelve rows.

Where it lands.

O24 Information System Service

Six columns: `Ref`, `Information system service`, `Application`, `Description`, `Style`, `Published`. `Style` from `API`, `Event`, `Batch`, `File`. `Published` is `Published` or `Unpublished`, tested by whether a team outside the owning team can find the contract and call it without asking a person. `Application` is one `Ann` reference, not several.

How.

1. Walk `O15` and ask what automated element delivers each business service.
2. Name the service after the job, not the system. Ardent's fourteen rows are all `<Application> Service`, described as "Service exposed by `<Application>`" [3] — filled mechanically from `O23`, which tells you nothing. `Core Banking Service` is a system with a word after it; `Post An Entry Service` is a service.
3. Set `Style` from what it is today: Ardent reads four `API`, three `Event`, three `Batch`, four `File`, a credible mix once the names are fixed.
4. Set `Published` honestly — Ardent reads seven and seven — then cross-check `R06`. `ISS-01 Core Banking Service` is `Published` with `Style` = `API`, yet every recorded interface into `A01` is a file, a batch or a direct database read [3][4]. One sheet is wrong, and it is not `R06`.
5. Where a business service has no automated element, say so rather than inventing one.

Signed by. Chief Enterprise Architect, with the application domain architect.

Done when. Every row names a job rather than a system, every `Published` claim can be shown by somebody outside the owning team, and every `Style` agrees with `R06`.

Shows up in. Nothing: there is no service report among the thirteen. Say so in the catalogue, because a sheet with no report over it decays quietly.

Goes wrong when. One service is recorded as provided by two applications. That is two services or a duplication finding; never one service with two providers.

Step 6.9 — Join business services to applications and grade the coverage

Produce. `R04 Business Service supported` rebuilt from evidence and honestly graded.

You need. Steps 6.2 and 6.8. `O15 Business Service` and `R03 Capability realised by Busi`.

Where it lands.

R04 Business Service supported

Four columns: `Business service`, `Service name`, `Application`, `Application name`. The tab name is truncated and ends in a space. One row per application taking part in delivering the service. There is no coverage column, so add `Coverage` (`Full` or `Partial`) and `Note` — `Partial` with a note is where this sheet earns its keep.

How.

1. Read what is there first. `R04` holds twelve rows, one per business service, each pointing at exactly one application, and it is positional: `BSV-05 Payments & Settlement Service` is recorded as supported by `A05 Collections & Recovery`, `BSV-08 Treasury & Liquidity Service` by `A08 AML / Sanctions Screening`, `BSV-12 Channel & Distribution Service` by `A12 Regulatory Reporting` [3]. Two applications appear nowhere: `A13` and `A14`.
2. Delete the generated rows. A wrong relationship is worse than a missing one, because reports read it as evidence and nobody re-derives it.
3. Rebuild application by application. An application's owner knows what it supports; a business service's owner rarely knows what supports it. Expect two to five rows per service, not one.
4. Grade `Coverage`. `Full` means end to end for the whole book; anything else is `Partial` with a note saying what is missing — one segment, one channel, read-only.
5. Count `Partial` against `Full`. A composite bank whose only complete party view sits in the warehouse reads heavily `Partial`, and that is the honest shape rather than a data-quality problem.

Signed by. Chief Enterprise Architect, with each business service owner confirming its rows.

Done when. Every `O15` row has at least one `R04` row and every `Partial` has a note.

Shows up in. `RPT13 Orphan Analysis`, which reads this sheet and nothing else for its application findings, and indirectly `RPT03 Capability Heatmap`.

Goes wrong when. Everything reads one-to-one. Four channels each reaching payments is four rows, three of them `Partial`, plus a duplication finding.

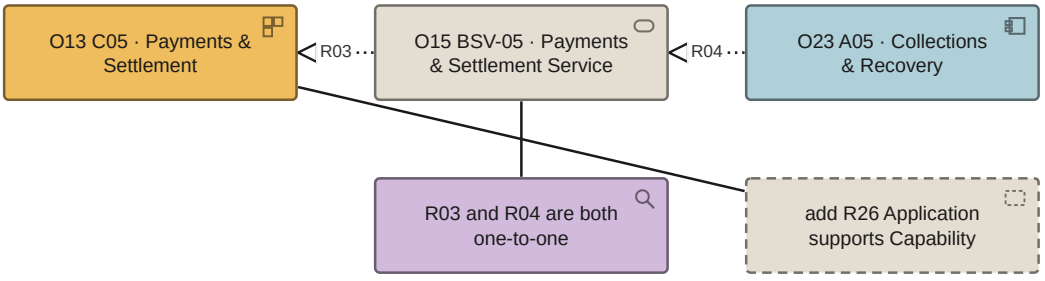

Figure 6.4 Why coverage reads clean and duplication cannot be seen. `R03` joins one capability to one business service and `R04` joins that service to one application, so the two-hop path is a bijection. The sheet to add is `R26 Application supports Capability`, populated at level 3.

Step 6.10 — Read capability coverage, and name the duplication the repository cannot express

Produce. A coverage reading at level 1, the duplication finding in business terms, and the sheet you have to add to hold it.

You need. Step 6.9. `O13 Business Capability`, `R03 Capability realised by Busi`, `RPT03 Capability Heatmap`.

Look it up. Volume 1, chapter 19, *Capability to application — the coverage map*, grades each capability full, partial or gap. Take the grading, match by name.

Where it lands. Nothing existing, and say so plainly in the catalogue: **there is no application-to-capability relationship sheet in this repository.** The only path from capability to application is two hops, `O13` → `R03` → `O15` → `R04` → `O23`, and because `R03` and `R04` are both one-to-one in the shipped file, every capability appears to have exactly one application and duplication can never be seen. Add `R26 Application supports Capability`, columns `Application`, `Application name`, `Capability`, `Capability name`, `Support`, `Note`, and populate it at level 3.

How.

1. Map at level 3. At level 1 every application supports everything and nothing can be read; Ardent's seventy-two level-3 rows are a workable afternoon.
2. Work application by application, for the reason in step 6.9.
3. Read `RPT03 Capability Heatmap` for the `Gap` column, `Target` minus `Current` on the level-1 rows. The widest are `C12 Channel & Distribution` at 1 against 3 and `C11 Finance & Regulatory Reporting` at 2 against 4, both `Build`, and `C11` is `Critical` [6].
4. Count applications per level-1 capability. Three or more is a duplication cluster; raise one gap per cluster, not one per application in it.
5. Name it in business terms. "Each channel has its own payment initiation path" is a finding somebody funds; "four applications touch payments" is an observation nobody does.

Signed by. Architecture Board (`GB-01`), with the Data Governance Council (`GB-03`) notified where the duplication is a mastering problem rather than a functional one.

Done when. Every level-3 capability marked `Critical` or `High` has an application or a named gap, and every cluster a one-sentence business statement.

Shows up in. `RPT03 Capability Heatmap`, and `RPT07 Gap Analysis` once you have gaps — which in the shipped file you do not. `O29 Gap` holds **zero records** and `RPT07` is a header with no rows [10]. That is not a clean estate; it is an empty sheet, and the dashboard's `Gaps overdue` figure of zero is arithmetic over nothing.

Goes wrong when. Duplication is treated as automatically wrong. Two lending books for two very different customer sets can be correct; the question is which is authoritative for what.

COVERAGE, DUPLICATION AND THE EMPTY GAP SHEET

Which level-1 capability has three or more applications behind it, and what is the one business sentence that would fund fixing it?

`029 Gap` holds no records and `RPT07` is empty. What are the first five gaps you would write, and which sheet does each trace to?

Which duplication is deliberate — two books, two regimes, two reasons — and who will confirm that in writing?

Data authority, interfaces, requirements and standards

Step 6.11 — Record which application manages which data entity, and who is authoritative

Produce. `R05 Application manages Data En` rebuilt with a role per row, and a reading of `RPT05 Data Ownership` naming every contested entity.

You need. `020 Data Entity` from chapter 5, fourteen rows.

Look it up. Volume 1, chapter 19, *Application to data, with the system of record*, bolds the record master per entity; take the grading and match by name. Then volume 1, chapter 9, *Party, and why Customer is usually a bad entity*, before accepting that `E01 Party` and `E02 Customer` are two entities with two masters.

Where it lands.

> **`R05 Application manages Data En`**
>
> Five columns: `Application`, `Application name`, `Data entity`, `Entity name`, `Role`. `Role` is `Master` or `Consumer` in the shipped file. Add an `Authoritative` column taking `Yes`, `No`, `Contested` or `Target authority` — the shape that keeps a contest visible instead of letting whoever edits last resolve it.

How.

1. Read the sheet before touching it. Its twenty-eight rows split fourteen `Master` and fourteen `Consumer`, and the `Master` rows carry the application **name** where a reference belongs — `Collections & Recovery`, not `A05` — while the `Consumer` rows carry references properly [5]. `00 Index` claims every relationship row names both ends by reference and by name. For half of `R05` that is untrue.

2. Notice why: the `Master` rows were generated from `O20`'s own `Master system` column, which also holds names. So `R05` and `O20` cannot disagree, and the sheet that should catch two systems claiming one entity instead copies one of them.
3. Rebuild from the systems. Each writeable copy is a `Master` row, read-only replicas are copies, and where two or more claim the master, mark **all of them** `Contested` and resolve at `GB-03`, not in the sheet.
4. Read the result against sense. Ardent records `E01 Party` mastered by `A05 Collections & Recovery`, `E02 Customer` by `A12 Regulatory Reporting`, `E03 Account` by `A02 Payment Hub`, `E12 General Ledger Entry` by `A09 Credit Risk Engine`, `E13 Regulatory Report` by `A04 Loan Origination` [5]. `A01 Core Banking` masters nothing and appears once, as a consumer of Party. A bank whose core masters no entity has either a very unusual architecture or a generated join.
5. Read `RPT05 Data Ownership` and know what it is: it reprints `O20` without `Definition` and adds one footer line, three of fourteen entities with no lineage from report back to source [5]. It does not compute contested authority and has no column for it, so that reading is a column you add and a paragraph you write.

Signed by. Chief Data Officer (`ROL-05`) through the Data Governance Council (`GB-03`), whose mandate in `O57` is exactly this: designates data owners and resolves disputes.

Done when. Every `R05` row carries a reference in both key columns, and no entity has two uncontested masters.

Shows up in. `RPT05 Data Ownership`, columns `Master system` and `Lineage`, and `RPT06 Requirements Traceability` through `REQ-008` to `REQ-015`.

Goes wrong when. Somebody resolves the contest in the sheet. Writing the target application into the master column erases the finding — which is the whole reason `GB-03` exists.

TWO CLASSIFICATIONS FOR ONE THING

`O20 Data Entity` classifies `E01 Party` as `Confidential` and `E02 Customer` as `Public` [5]. Those are the same people. `E03 Account`, `E07 Payment Instruction`, `E11 Financial Instrument` and `E14 Alert / Case` also read `Public`, which would make a balance and a financial-crime case publishable. Principle `PRN-14 Classification Travels with Data` says classification is a property of the data and not its location; six rows of `O20` say otherwise. Raise it as one finding against the classification scheme, not six against the rows.

Step 6.12 — Build the interface register and count the unmanaged

Produce. `R06 Application interfaces with` complete, the unmanaged count, the unlogged count, and the direct-read list named row by row. This is the finding that gets a design gate funded.

You need. Step 6.2, plus the scheduler's job list, the broker's queue list, the file-transfer schedule and the database link and service-account inventory from step 6.5.

Look it up. Volume 1, chapter 12, *How these systems actually talk*, and *The batch clock and the business day* — know which window each nightly transfer sits in before judging it.

Where it lands.

> **`R06 Application interfaces with`**
>
> Nine columns: `Ref`, `Source`, `Source name`, `Target`, `Target name`, `Type`, `Frequency`, `Managed`, `Logged`. `Ref` is `INT-nnn`. `Type` from `API`, `Event`, `Batch`, `File`, `Direct DB read`. `Frequency` from `Real time`, `Hourly`, `Daily`, `Nightly`. `Managed` is a four-part test: the interface traverses a managed integration layer, is published in a catalogue, is versioned, and is monitored. All four, or it is `Unmanaged`. `Logged` is a different question — a managed interface with no log is a real and common combination.

How.

1. Collect from four sources — scheduler, broker, file transfer, database links — because each misses a different class.
2. One row per logical interface, not per file. Twenty files on one nightly schedule between two systems is one row at `Frequency` = `Nightly`.
3. Apply the `Managed` test strictly. An API call with no gateway, no catalogue entry and no monitoring is `Unmanaged` however modern it looks.
4. Check `Type` against `Managed`. Ardent has ten `Direct DB read` rows and two of them, `INT-007` and `INT-013`, are marked `Managed` [4]. A direct read of another application's store cannot be managed, because there is no contract to manage. Then check `Type` against `Frequency`: `INT-005` is a `File` at `Real time`, and `INT-011` and `INT-014` are `Batch` at `Real time` [4]. A real-time batch is a column filled without being read.
5. Count, and quote `RPT04 Interface Catalogue`, whose footer does the arithmetic: nineteen of thirty-two unmanaged, twenty carrying no logging [4]. `RPT04` reprints `R06` with references resolved to names and adds that one line. It computes nothing else.

Signed by. Chief Information Officer, with the integration lead named as evidence owner.

Done when. Every interface has a `Ref`, a `Managed` verdict, a `Logged` verdict and a `Type` that agrees with its `Frequency`, and the direct reads are listed separately for the Board.

Shows up in. `RPT04 Interface Catalogue`, footer; and `ZZ Checks`, three of whose rows are this step — unmanaged 19 against a threshold of 0, no logging 20 against 0, direct database reads 10 against 0 [10].

Goes wrong when. The count flatters. A soft test turns nineteen into four and the design gate never gets created. `058 Governance Gate` holds `GT-1 G1 Design`, before build, on whether the design meets principles, standards and the reference architecture. That gate exists and nineteen unmanaged interfaces got past it, which means it was not operating.

THE UNMANAGED INTERFACES

The full register transcribes onto the Phase C interface form in appendix F. These questions decide what goes in the last two columns.

Applying the four-part test strictly, what is your unmanaged percentage — and what was it before you applied it strictly?

Which direct database read would double the cost of replacing the system at the far end, and is there a contract to hold a replacement to?

Twenty interfaces carry no log. Which of them moves money, and who would notice a missing message?

Step 6.13 — Trace requirements to applications, through the building blocks

Produce. `R09 Requirement satisfied by Bu` rebuilt, and a statement of which requirements reach an application and which stop at a capability.

You need. `O06 Requirement`, thirty-two rows, and `O55 Building Block`, thirty-seven rows, of which `O45 Application Building Block` holds the ten application ones.

Where it lands.

> **`R09 Requirement satisfied by Bu`**
>
> Five columns: `Requirement`, `Requirement name`, `Building block`, `Block name`, `Traced`. Note what this sheet does not do: it joins a requirement to a **building block**, never directly to an application. The path to an application is two hops, through `O45`'s own `Application` column — say so in the catalogue, because an architect hunting a requirement-to-application sheet will not find one.

How.

1. Read the shipped join and reject it. The thirty-two rows are positional: `REQ-001 Evidence Basel III / IV` satisfied by `BBB-01 Customer Management capability uplift`; `REQ-008 Party mastered once` by `BBB-08 Treasury & Liquidity capability uplift`; `REQ-031 Adopt Event streaming` by `TBB-01 Real-time payment rails` [7]. Each wrong in a way a reader skims past.
2. Rebuild requirement by requirement. The six integration requirements, `REQ-016` to `REQ-021`, name their application in their own title and belong against `ABB-01` to `ABB-06`; the eight mastering requirements belong against `DBB-01` to `DBB-08`; the seven regulatory

requirements trace to whatever block produces the evidence — and for most of them no such block exists, which is the honest answer.

3. Set `Traced` from the join, not from hope. Ardent reads twenty-four `Yes` and eight `No`, and the eight are every fourth row, which is a generator's signature rather than an assessment.
4. Read `RPT06 Requirements Traceability` and know its shape: it reprints `O06` without `Statement`, adds one column, `Reaches a block`, and one footer line [7].
5. Count what the application building blocks cover. `O45` holds ten rows, for `A01` to `A10`; `A11`, `A12`, `A13` and `A14` have none, and two of those four are `Critical` [1][7].

Signed by. Chief Enterprise Architect, with each requirement's owner from `O06`.

Done when. Every requirement traces to a block whose acceptance criterion could be tested, or is listed as untraced with a reason.

Shows up in. `RPT06 Requirements Traceability`, column `Reaches a block`.

Goes wrong when. Every requirement traces to a target block that was going to be built anyway, which proves the programme internally consistent and nothing else.

> **THE ACCEPTANCE CRITERION ALREADY IN THE FILE**
>
> Each of `O45`'s ten rows carries the same acceptance criterion in different words: "No unmanaged interface to or from `<application>`" [7]. That is testable against `R06`, and today nine of the ten fail. The one that passes is `ABB-10 Treasury Management target`, because `A10` is the single application in Ardent's estate that no unmanaged interface touches. Nine failing acceptance criteria, computable in one filter, is the most useful page in the Phase C pack — and it was in the repository before you arrived.

Step 6.14 — Assess standards conformance, application by application

Produce. `R18 Standard applies to Applica` extended to cover every application in scope, with a verdict and a note per pairing.

You need. Step 6.2. `O32 Standard`, sixteen rows.

Where it lands.

> **`R18 Standard applies to Applica`**
>
> Five columns: `Standard`, `Standard name`, `Application`, `Application name`, `Conformance`. `Conformance` from `Conformant`, `Partial`, `Breach`, `Not assessed`. Add a `Note` column, mandatory on anything other than `Conformant` — a verdict without a note is a complaint.

How.

1. Count the coverage first. The sixteen rows pair each standard with exactly one application, and only odd-numbered applications appear: `A01`, `A03`, `A05`, `A07`, `A09`, `A11`, `A13`. Seven of fourteen applications carry no verdict of any kind [8].
2. Assess only where the standard's scope reaches the application. Do not assess a container image standard against a mainframe. `O32` has no scope column, which is the gap step 7.5 fills.
3. Notice that `O32` carries its own `Conformance` column and that `R18`'s verdict is identical to it for every standard [8]. The repository holds one verdict per standard, not one per pairing, and so cannot record that one application conforms where its neighbour breaches. The per-pairing verdict belongs in `R18`; `O32`'s column should become a roll-up or go.
4. Write the note. Ardent has none: four standards read `Breach` and nothing says why.
5. Read `RPT12 Standards Conformance` and know that it is a verbatim reprint of `O32` — same five columns, same sixteen rows, no footer, no rate, no coverage count [8]. Every number you want from it, you compute.

Signed by. Architecture Board (`GB-01`), on evidence assembled by the domain architect.

Done when. Every `Critical` and `High` application is assessed against every standard whose scope covers it, and every verdict other than `Conformant` has a note.

Shows up in. `RPT12 Standards Conformance`. Quote a coverage count rather than a conformance rate: today seven applications of fourteen, with four of sixteen standards in `Breach` and no explanation anywhere.

Goes wrong when. The rate is reported as a score. It is a ratio over whichever pairings somebody chose to assess.

The reference model, BIAN, and the cheapest finding in the set

Step 6.15 — Reconcile the reference model entries, and decide where BIAN fits

Produce. `047 Reference Model Entry` reconciled against what the estate runs, every `Adopted` claim evidenced, and a one-page position on BIAN.

You need. Step 6.1, and volume 1 chapter 13 for the reference architectures. Do not re-derive that chapter.

Look it up. Volume 1, chapter 13, *BIAN — the Banking Industry Architecture Network*, and *What BIAN costs, and the thing most people get wrong*. Then *How BIAN relates to TOGAF and ArchiMate* in the same chapter, for the only mapping question that matters: BIAN Service Domains are not TOGAF building blocks and the translation is manual.

Where it lands.

> **O47 Reference Model Entry**
>
> `Ref`, `Reference model entry`, `Description`, `Status`. `Status` is `Adopted` or `Candidate`. Extend it to `Adopted`, `Candidate`, `Rejected`, `Not applicable`: a reference model with no rejections was never argued with.

How.

1. Test every `Adopted` row against evidence. Ardent reads ten `Adopted`, one `Candidate` [9]. `RME-04 Event streaming` is `Adopted` and `R06` records four `Event` interfaces out of thirty-two; `RME-06 Data lineage platform` is `Adopted` and `RPT05` reports three entities with no lineage at all. An adopted pattern with no instance is a `Candidate`.
2. Reconcile `O47` against `O26 Logical Technology Componen`. The first seven rows of `O47` and all seven rows of `O26` are the same patterns with the same descriptions and two different status vocabularies [9], and they disagree twice: `ISO 20022 message backbone` is `Candidate` in `O47` and `Current` in `O26`; `Data lineage platform` is `Adopted` in `O47` and `Target` in `O26`. Pick one as authoritative — `O26` for what runs, `O47` for what is approved — and make the other point at it.
3. Read `RME-03 API gateway and open banking` carefully. It is `Adopted`, described as "Consent-managed third-party access". **South Africa has no regulated open banking regime.** What a South African bank has is bilateral commercial APIs, plus SARB Directive 2 of 2024 on screen-scraped credit initiation. Rewrite the row to say what it is — a bank-owned API programme — or an executive will read a regulatory obligation into a row that carries none.
4. Decide the BIAN question in writing, once. Service Landscape 14.0 carries 322 Service Domains and 586 business capabilities, and BIAN publishes no licence: availability is not permission [12]. Use it as a vocabulary check against `O13` and as a package-fit lens, name the release you used, and keep its content out of the repository.
5. Where ISO 20022 is the reference, keep it in `O47` as a pattern and let the message detail live with the payments team. It is a reference data model wearing a message format's clothes.

Signed by. Chief Enterprise Architect, with Architecture Board (`GB-01`) approving every status change.

Done when. Every `Adopted` row names an instance in the estate, `O47` and `O26` agree or one defers to the other, and the open banking row says what it means.

Shows up in. No report reads `O47` here — a reason to keep it short and review it on a fixed cadence.

Goes wrong when. The reference model is filled in as aspiration and marked `Adopted`. Then you hold two target architectures and no baseline.

Step 6.16 — Run the orphan analysis, then publish the Application Portfolio Catalog

Produce. A reading of `RPT13 Orphan Analysis` with a question raised against every row, and the Phase C application deliverables closed out.

You need. Steps 6.1 to 6.15, and `O31 Deliverable` rows `DLV-09 Application Architecture` and `DLV-10 Interface Catalogue`, both phase `C`, both currently `Approved` [10].

Where it lands.

> **`O31 Deliverable`**
>
> Four columns: `Ref`, `Deliverable`, `Phase`, `Status`. No owner, no due date, no location: add `Owner`, `Due` and `Repository location` or the register cannot be chased. `Status` runs `Approved` or `Draft`.

How.

1. Read `RPT13 Orphan Analysis` first, because it is the cheapest finding in the whole set. Four columns, two rows: `A13 Data Platform`, `Replace`, "No business service maps to it"; `A14 Fraud Detection`, `Retain`, the same finding [9]. The sheet's own closing line is the right stance — an orphan is not always wrong, it is always worth a question.
2. Ask the question of each. A data platform with no business service is usually a mapping failure: it serves reporting, which nobody made a service. A fraud detection engine with no business service in a bank is a missing business service, because fraud intervention is something the bank does for customers and the supervisor asks about. One row is a sheet problem, the other an architecture problem, and `RPT13` cannot tell you which.
3. Re-run `RPT13` after step 6.9. If the rebuilt `R04` is honest, both rows clear and new ones appear — the applications with no *justifiable* service rather than no recorded one.
4. Assemble the catalogue in six sections: inventory from `O23`; assessment from your added columns; coverage and duplication from step 6.10; data authority from `R05` read next to `RPT05`; interfaces from `RPT04`; standards coverage from `R18`.
5. Put five numbers on the front page. For Ardent: fourteen applications; nineteen unmanaged interfaces of thirty-two; twenty unlogged; seven of fourteen applications with no standards verdict; nine of ten application building blocks failing their own acceptance criterion.
6. List what is deliberately not in it — rejected discovery candidates, parked scores, relationships you deleted as generated rather than observed, the columns and sheets you added — then take it to Architecture Board (`GB-01`) for approval and to Investment Committee (`GB-02`) as information. `GB-02` holds `G0` funding and cannot judge architecture compliance; `GB-01` can and cannot fund.

Signed by. Chief Information Officer as owner, approved by Architecture Board (`GB-01`).

Done when. `DLV-09` and `DLV-10` carry a status you believe, every front-page number traces to a sheet in one step, and `RPT13` has a written question against every row.

Shows up in. `RPT13 Orphan Analysis` and `031 Deliverable`. Note the state you inherited: `DLV-09` and `DLV-10` both read `Approved`, and so does `DLV-13 Consolidated Gaps` — over a gap sheet with no records in it [10]. An approved deliverable above an empty sheet is the most expensive kind of green.

Goes wrong when. The catalogue is a slide deck. A catalogue is a register with a reading attached; if the numbers are not live from the sheets, you cannot answer the first question in the room, which is whether they are current.

> **THE HANDOVER TO PHASE D**
>
> Three of the five technical-fitness sub-scores are inputs from chapter 7, not outputs of chapter 6: support currency depends on `027 Physical Technology Compone` and a rebuilt `R07`, recoverability on `049 Resilience Tier`, and the standards evidence on step 7.5 giving every standard a scope. Run step 6.6 with steps 7.2, 7.4 and 7.12 as one piece of work with two owners, or accept that part of it is scored against evidence that does not exist yet.

PHASE C, THE APPLICATION HALF — A REVIEWER'S QUESTIONS

What are your five front-page numbers, and can each be traced to a sheet in one step?

Which relationships did you delete as generated rather than observed, and who signed the deletion off?

Which application masters a data entity that its disposition commits you to replacing, and does any work package name the destination?

`RPT02`, `RPT05`, `RPT11`, `RPT12` and `RPT13` reprint their source sheets and add a footer at most. Which numbers in your catalogue did you compute yourself, and is that said on the page?

THE CHAPTER IN TEN LINES

- Walk the conceptual model from volume 1 chapter 11 and map real systems onto it, never the reverse.
- Four of the five usually-missing capabilities are missing in Ardent too — and all five are shared.
- O23 is ten columns: no cost, no hosting, no vendor, no lifecycle. Add five and say that you did.
- Owner holds unit names, not ORG- references, and Internal Audit owns two operational systems.
- Three discovery rules — spreadsheet, invoice, directory. Run all three; zero finds means one was skipped.
- Health is one column doing two jobs. Score business value out of 12 and technical fitness out of 10, separately.
- Lock the disposition before revealing the scores, or the agreement test only agrees with itself.
- R04 and R03 are both one-to-one, so capability coverage reads clean and duplication cannot be seen.
- Nineteen of thirty-two interfaces unmanaged, twenty unlogged, ten direct database reads — two of them marked Managed.
- RPT02, RPT05, RPT11, RPT12 and RPT13 reprint their source sheets and add a footer at most. Compute the rest yourself.

SOURCES FOR THIS CHAPTER

- **1.** Banking_TOGAF_Metamodel.xlsx, sheets O23 Application Component, O19 Contract and R07 Application deployed on Tec
- **2.** Banking_TOGAF_Metamodel.xlsx, sheet RPT02 Application Portfolio
- **3.** Banking_TOGAF_Metamodel.xlsx, sheets O15 Business Service, O24 Information System Service and R04 Business Service supported by Application
- **4.** Banking_TOGAF_Metamodel.xlsx, sheets R06 Application interfaces with Application and RPT04 Interface Catalogue
- **5.** Banking_TOGAF_Metamodel.xlsx, sheets O20 Data Entity, R05 Application manages Data Entity and RPT05 Data Ownership
- **6.** Banking_TOGAF_Metamodel.xlsx, sheets O13 Business Capability, R03 Capability realised by Business Service and RPT03 Capability Heatmap
- **7.** Banking_TOGAF_Metamodel.xlsx, sheets O06 Requirement, O45 Application Building Block, O55 Building Block, R09 Requirement satisfied by Building Block and RPT06 Requirements Traceability
- **8.** Banking_TOGAF_Metamodel.xlsx, sheets O32 Standard, R18 Standard applies to Application and RPT12 Standards Conformance
- **9.** Banking_TOGAF_Metamodel.xlsx, sheets O47 Reference Model Entry, O26 Logical Technology Component and RPT13 Orphan Analysis
- **10.** Banking_TOGAF_Metamodel.xlsx, sheets O29 Gap, O31 Deliverable, O57 Governance Body, O58 Governance Gate, RPT01 Dashboard, RPT07 Gap Analysis and ZZ Checks
- **11.** The Open Group, the TOGAF Standard — Architecture Development Method overview https://www.opengroup.org/togaf
- **12.** BIAN, “Service Landscape 14.0 Release Notes”, final version 20 February 2026 (as covered in volume 1, chapter 13) https://bian.org/wp-content/uploads/2026/02/BIAN-v14.0-Release-Notes-v1.0_-Final-Version.pdf

CHAPTER 7

Technology Architecture

Four sheets, four questions: what the estate offers, what kind of thing offers it, which product on which contract, and where it sits.

Phase D is the phase architects either skip or drown in. Skipped, it becomes a server list. Drowned in, it becomes a configuration database with worse tooling. The repository keeps you out of both by forcing a separation the estate itself does not have: service, logical component, physical component, place. Four sheets, four questions. What does the estate offer. What kind of thing offers it. Which product, on which contract, with which support end date. And where does it sit.

Fourteen steps. **Ardent Union Bank**, the bank in the repository, is an invention, as is volume 1's **Table Bay Bank**. Two tab names here are cut off at Excel's thirty-one characters, and the missing letters are not a typo in your writing: spell them `O26 Logical Technology Componen` and `O27 Physical Technology Compone`.

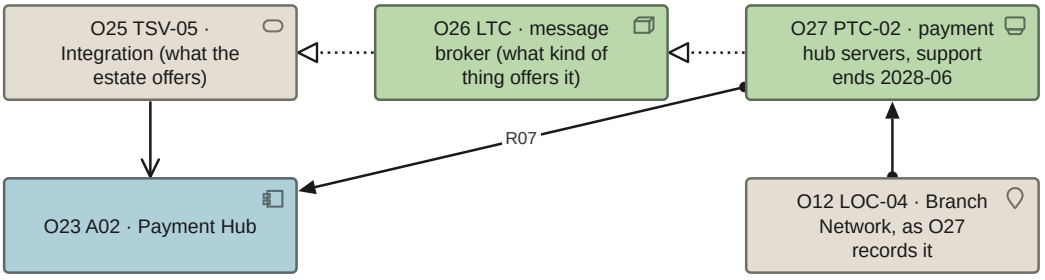

Figure 7.1 The four Phase D questions as ArchiMate 4, with the two relationships you have to rebuild. `R07 Application deployed on Tec` and `O27`'s own `Location` column are both positional in the shipped file, and they disagree with each other.

BEFORE YOU START THIS PHASE

- **Volume 1, chapter 14, *The research is dated, and here is what goes stale first.*** Which vendor facts to distrust, and how fast.
- **Volume 1, chapter 14, *The cost lines that are not the licence.*** What a platform actually costs once it is running.
- **Volume 1, chapter 15, *The Joint Standards, with the numbering got right.*** Four instruments, three numbered "1". Get this right before you cite one.
- **Volume 1, chapter 15, *What the regulator actually asks to see.*** The list this phase has to be able to produce on a day of somebody else's choosing.

Run `ZZ Checks` before and after. Four of its rows are Phase D findings: platform components past support 3 against a threshold of 0, security controls absent 4 against 0, control objectives without evidence 4 against 0, vacant architecture roles 0 against 0 [8].

Services before things

`025 Technology Service` is the sheet a business reader can follow. Four columns only — `Ref`, `Technology service`, `Description`, `Category` — and twelve rows at Ardent, each a technical capability the estate offers rather than a product that offers it: `Compute`, `Storage`, `Network`, `Identity`, `Integration`, `Database`, `Backup and Recovery`, `Monitoring`, `Data Platform`, `End User Compute`, `Security Operations`, `Automation` [1].

The value of the sheet is that it survives a product change. When the message broker is replaced, `TSV-05 Integration` does not move; `027` does. What the sheet lacks is an availability target, an owner and a baseline-or-target flag. Add the first two before anything else in this phase: an availability target with no owner is a wish, and a service with no owner cannot be assessed.

Step 7.1 — Name the technology services, and fix the Category column

Produce. `025 Technology Service` populated, with an availability target and an owning organisation on every row.

You need. `023 Application Component` from chapter 6, the operations team's service catalogue if one exists, and `009 Organization Unit` and `011 Role`.

Look it up. Volume 1, chapter 14, *The research is dated, and here is what goes stale first*, for which facts about a market to write down and which to leave to the sourcing team.

Where it lands.

> **`025` Technology Service**
>
> `Name` is the capability, not the product: `Database`, not the engine. `Category` groups them, and in the shipped file it does so badly — `TSV-05 Integration` is categorised `Data`, `TSV-11 Security Operations` is `Data`, `TSV-12 Automation` is `Data` and `TSV-09 Data Platform` is `Infrastructure` [1]. Agree a vocabulary — `Compute`, `Data`, `Integration`, `Security`, `Resilience`, `Network`, `Platform` — and apply it. Add `Availability target` as a percentage, set from the most demanding consumer, `Owner` as an `ORG-` reference and `State` as `Baseline` or `Target`.

How.

1. Start from what applications need, not from what infrastructure owns. Twelve to twenty rows is right for a bank; Ardent has twelve.
2. Re-categorise every row against the agreed vocabulary and count the changes. Four of twelve is what Ardent needs, and that count belongs in the catalogue.
3. Set `Availability target` from the most demanding consumer, which in a bank means the payment and card path, not the average.
4. Mark as `Target` the services that do not exist yet: recording them as services rather than projects is what lets a requirement point at one.
5. Keep the consumer list out of this sheet. `R07 Application deployed on Tec` holds it, badly, and duplicating it guarantees the two disagree.

Signed by. Chief Information Officer (`ROL-04`), with each consuming business service's owner acknowledging the availability target.

Done when. Every row has a category from the agreed vocabulary, an availability target and an owning organisation, and no row names a vendor product.

Shows up in. No report among the thirteen reads `025`. This sheet is read by people, not reports, so set its review cadence deliberately.

Goes wrong when. The sheet is the product list with the products renamed. `Database` is a technology service; a particular engine is one physical realisation of it.

Step 7.2 — Record what each application runs on, and refuse the join you were given

Produce. `R07 Application deployed on Tec` rebuilt from the deployment record, with the dependency graded and non-production environments present.

You need. Steps 7.1 and 6.2, the configuration record, the deployment pipeline's target list, and the operations team's own view of what runs where.

Where it lands.

> **R07 Application deployed on Tec**
>
> Five columns: `Application`, `Application name`, `Platform component`, `Component name`, `Environment`. Add `Dependency` taking `Critical` (the application stops) or `High` (it degrades), because without it the resilience work in step 7.12 has nothing to prioritise against.

How.

1. Read the shipped join and reject it. Fourteen rows, one per application, all `Environment` = `Production`, and the pairing is positional: `A01` to `PTC-01` through `A12` to `PTC-12`, then `A13` back to `PTC-01` and `A14` to `PTC-02` [1]. It records `A05 Collections & Recovery` deployed on the HSM estate, `A07 Digital Channel` on an enterprise storage array and `A10 Treasury Management` on the ATM estate controller. None of those is possible.
2. Delete it and rebuild. Expect two to five components per application: a runtime, a database, an integration component and usually a file-transfer or scheduling component.
3. Record non-production. Every shipped row says `Production`, so the repository holds no record of where anything is tested, and `048 Environment` lists four environments nothing points at.
4. Grade `Dependency` honestly. Most rows will be `Critical`, because an application without its runtime does not degrade, it stops — so the `High` rows are the ones to get right.
5. Count the applications with no traceable component after the rebuild, and say so rather than filling the hole.

Signed by. Chief Information Officer, with the service management lead (`ROL-15`) as evidence owner.

Done when. Every application has at least one component per environment it runs in, every `Critical` application has a `Critical` row, and the deleted rows are listed in the change you raised.

Shows up in. `RPT10 Technology Currency` indirectly, and the fitness scoring at step 6.6, which joins through here to `027` for `Support end`.

Goes wrong when. The generated join is left in place because it looks complete. A wrong relationship is worse than a missing one: `RPT10` and the fitness score both read it as evidence.

THE JOIN YOU WERE HANDED

Which three rows of `R07` are physically impossible, and who in operations will confirm the real answer?

Every row reads `Production`. Where is each `Critical` application actually tested, and what records that?

After the rebuild, which application has no traceable platform component — and what does that do to its fitness score?

Logical and physical, kept apart

A logical component is a kind of thing. A physical component is a specific thing with a vendor, a class, a support end date and a place. Ardent holds seven logical rows and twelve physical ones [1], with different status vocabularies: `O26` uses `Current`, `Emerging`, `Target`, while `O27` uses `Position`, reading `Supported` or `Past support`.

One problem with `O26` is worth knowing before you open it. Its seven rows are the same seven patterns as the first seven rows of `047 Reference Model Entry`, with the same descriptions in the `Rationale` column [4]. One sheet was filled from the other, and they disagree twice. Step 6.15 settles which is authoritative; this step uses that answer.

Step 7.3 — Write the logical component catalogue, and reconcile it with the reference model

Produce. `026 Logical Technology Componen` populated with classes of technology, each providing a named service, and the two disagreements with `047` resolved.

You need. Step 7.1. Step 6.15's decision on which of `026` and `047` is authoritative.

Where it lands.

> **`026 Logical Technology Componen`**
>
> Four columns: `Ref`, `Logical technology component`, `Rationale`, `Status`. `Status` from `Current`, `Emerging`, `Target`. Add `Provides service` as a `TSV-` reference — without it no logical component can be tied to the service layer and `O25` floats free.

How.

1. One row per class of technology you deliberately run, not per product. A row with a version number in it belongs in `027`.

2. Replace the reference-model patterns with component classes. Ardent's seven read `Real-time payment rails`, `ISO 20022 message backbone`, `API gateway and open banking`, `Event streaming`, `Tokenisation and HSM`, `Data lineage platform`, `Active-active data centres` [4] — patterns and targets, not classes. The classes a bank runs are a mainframe, an application runtime, a relational database, a message broker, an event platform, an identity provider, a hardware security module, a container platform and a lakehouse.
3. Fix the two disagreements with `O47`: `ISO 20022 message backbone` is `Candidate` in `O47` and `Current` in `O26`; `Data lineage platform` is `Adopted` in `O47` and `Target` in `O26` [4]. A platform cannot be both adopted and a target.
4. Set `Provides service` and expect duplicates. Two logical components providing one service honestly records two ways of doing one thing, and `PRN-17 Control Technical Diversity` is what you assess it against.
5. Use `Emerging` freely: approved and not yet widespread is a position, not a failure.

Signed by. Chief Enterprise Architect (`ROL-08`), with the technology domain architect (`ROL-12`) per category.

Done when. Every `O25` service has at least one logical component, every row is a class rather than a pattern, and `O26` and `O47` no longer contradict each other.

Shows up in. Nothing directly, which is a reason to review it on a date rather than on a prompt.

Goes wrong when. The sheet becomes a second reference model, which is what it is today. Then you have two target architectures and no catalogue of what you run.

Step 7.4 — Record the physical estate, with support dates from the contract

Produce. `O27 Physical Technology Compone` populated, every `Support end` taken from a contract and every `Position` consistent with it.

You need. Step 7.3, `O19 Contract`'s ten rows, the infrastructure cost breakdown from finance, and `O12 Location` from step 7.9 — or run the two steps together.

Look it up. Volume 1, chapter 14, *The cost lines that are not the licence*, before you accept an infrastructure cost that is only the licence.

Where it lands.

> **O27 Physical Technology Compone**
>
> Seven columns: `Ref`, `Physical technology component`, `Vendor`, `Class`, `Support end`, `Position`, `Location`. `Vendor` in the shipped file holds a class rather than a name — `IBM-class`, `Oracle-class`, `VMware-class`, `Hyperscaler`, `Microsoft`, and plain `Vendor` for seven of the twelve rows [3]. Keep it that way in a published example and put the real name in your own copy. `Support end` is `YYYY-MM` from the contract. `Position` is `Supported` or `Past support`. Add `Annual cost (Rm)`, `Lifecycle` and `Standard status` — the last of these is what step 7.7 needs and the sheet does not have.

How.

1. One row per node or cluster, not per server. Twelve rows is the right order for a bank of Ardent's size; three hundred means an asset register.
2. Take `Support end` from your contract, not the vendor's public lifecycle page. Extended support you are paying for and extended support you could buy are different facts, and only one is yours.
3. Make `Position` agree with `Support end`, which in the shipped file it does not. Three rows read `Past support`: `PTC-10 ATM estate controller` at 2026-10, `PTC-05 HSM estate` at 2026-11, `PTC-06 Branch server estate` at 2026-12 [3]. Two of those dates are still in the future, so until somebody says which column is wrong the three-component finding rests on one date.
4. Add `Annual cost (Rm)` from the infrastructure ledger. Without it the currency paper at step 7.7 has a deadline and no number, and a deadline with no number does not get funded.
5. Check `Location` against sense. Ardent puts the HSM estate in a cloud region and both the branch server estate and the enterprise storage array in a contact centre, while only two of twelve components sit in the Primary Data Centre and one in the Secondary [3].

Signed by. Chief Information Officer, with the Chief Financial Officer (`ROL-02`) on cost.

Done when. Every component has a class, a location, a support end date traced to a contract, and a `Position` that agrees with that date.

Shows up in. `RPT10 Technology Currency`, which reprints `O27` in full and adds one footer line: three components are running past vendor support today [3]. It computes nothing else, and in particular does not cross support dates with standard status, because `O27` has no standard-status column.

Goes wrong when. `Support end` is filled from the vendor's website and `Position` from somebody's impression. Then the two columns drift apart, which is the state you inherited.

On the vendor column

`027`'s `Vendor` values are deliberately classes — `IBM-class`, `Oracle-class`, `VMware-class`, `Hyperscaler` — which is the right choice for a published example. When you put real names in your own copy, volume 1 chapter 14's corrections list applies: ownership moves faster than any architecture document, and a product attributed to last year's owner reads as carelessness to the one person in the room who knows. Nothing here is a recommendation of a product.

The standards catalogue, and the finding it produces

`032 Standard` is sixteen rows at Ardent and spans more than technology: five `Security`, four `Data`, two `Integration`, two `Platform`, two `Delivery`, one `Governance` [5]. `Status` reads nine `Current`, four `Emerging` and three `Deprecated`, and `Conformance` reads five `Partial`, four `Breach`, four `Not assessed` and three `Conformant`.

Two structural problems. The sheet has **no owner column, no scope column and no review date**, so a standard can be neither assessed nor chased. And conformance is held on the standard rather than on the pairing, so the repository cannot record that one component conforms where its neighbour breaches. Worse for Phase D: `R18 Standard applies to Applica` joins standards to **applications only**, and no standard-to-component relationship exists anywhere in the twenty-five relationship sheets — so no technology component here can be assessed against a standard at all.

Step 7.5 — Write the standards catalogue and give conformance somewhere to live

Produce. `032 Standard` with an owner, a scope and a review date on every row, and a new relationship sheet that can hold a per-component verdict.

You need. `005 Principle` from the Preliminary Phase — a standard with no principle behind it is a preference. `011 Role` for owners.

Look it up. Volume 1, chapter 13, *ISO 20022 — a reference data model wearing a message format's clothes*, before you write a messaging standard that is really a data standard.

Where it lands.

> **`032 Standard`**
>
> Five columns today: `Ref`, `Standard`, `Category`, `Status`, `Conformance`. Add `Owner` as a `ROL-` reference and never an organisation; `Applies to` as a testable scope — `All new interfaces`, `New workloads`, `Cloud workloads`, `All persistence` — and `Review due` as `YYYY-MM` inside eighteen months. `Applies to` = "everything" is an unassessable standard.

> **R26 Standard applies to Component**
>
> A sheet you add, because none exists. Columns `Standard`, `Standard name`, `Component`, `Component name`, `Conformance`, `Note`, with `Conformance` from `Conformant`, `Partial`, `Breach`, `Not assessed` and `Note` mandatory on anything other than `Conformant`.

How.

1. Write a standard only where you can name what it applies to.
2. Set `Owner` to a role from `O11` and check the role is filled. Note while you are there that `O05 Principle` records `Architecture Board` as the owner of all twenty-four principles, including `PRN-13 Data Ownership is Individual`, which reads "An owner is a person, not a committee" [7]. Do not repeat that here.
3. Deal with the deprecated rows. `STD-02 Event schema standard`, `STD-12 Coding and review standard` and `STD-14 Naming convention` are `Deprecated`, and `R18` still assesses applications against all three [5]. Either the deprecation is wrong or the assessments are waste.
4. Decide what `O32`'s `Conformance` column becomes — a roll-up computed from the pairing sheet, or nothing. It cannot stay a hand-entered verdict once per-pairing verdicts exist.
5. Stagger `Review due` so the Board does not face sixteen reviews in one month.

Signed by. Architecture Board (`GB-01`), which publishes criteria and may grant dispensations with an expiry.

Done when. Every standard has an owner who exists, an `Applies to` naming a testable scope and a `Review due` inside eighteen months, and the component relationship sheet exists.

Shows up in. `RPT12 Standards Conformance`, a verbatim reprint of `O32` — same five columns, same sixteen rows, no footer, no rate, no coverage count [5]. Every number you quote from it, you computed.

Goes wrong when. The catalogue is written as aspiration and marked `Current`. Then step 7.6 reports breaches against standards nobody agreed to, and the Board rejects the report rather than the estate.

Step 7.6 — Assess conformance, component by component

Produce. The component conformance sheet populated, with a note on every verdict that is not `Conformant`, and a coverage count rather than a rate.

You need. Steps 7.4 and 7.5.

Where it lands.

> **`R26 Standard applies to Component`**
>
> One row per standard-and-component pairing you assessed. Assess only where the standard's `Applies to` covers the component: a container image standard against a mainframe is not a finding, it is a category error.

How.

1. Assess the components under every `Critical` and `High` application first, reached through the rebuilt `R07`.
2. Write a `Note` for every `Partial` and `Breach`. A verdict without a note is a complaint; Ardent's sixteen `R18` rows carry no notes at all, and four read `Breach` [5].
3. Do not assess everything. Deliberate coverage of what matters beats a matrix nobody reads, provided you say what you left out and why.
4. Report the coverage count. In `R18` as shipped only the odd-numbered applications appear — `A01`, `A03`, `A05`, `A07`, `A09`, `A11`, `A13` — so seven of fourteen applications have no verdict of any kind, and no component has one.
5. Carry the verdicts into step 6.6's fitness sub-score and into the checkpoint pack.

Signed by. Architecture Board (`GB-01`), on evidence assembled by the technology domain architect.

Done when. Every component carrying a `Critical` application is assessed against every standard whose scope covers it, and every non-conformance has a note a reader could act on.

Shows up in. `RPT12 Standards Conformance`, once you extend it to read the pairing sheet. As shipped it reads `O32` only.

Goes wrong when. The conformance rate is reported as a score. It is a ratio over whichever pairings somebody chose to assess.

O27 BY SUPPORT END DATE · WHAT RPT10 REPRINTS

Ref	Component	Support end	Position	
PTC-10	ATM estate controller	2026-10	Past support	
PTC-05	HSM estate	2026-11	Past support	← ?
PTC-06	Branch server estate	2026-12	Past support	← ?
PTC-01	Core banking mainframe	2028-05	Supported	
PTC-02	Payment hub servers	2028-06	Supported	
PTC-07	Enterprise storage array	2029-02	Supported	
PTC-11	Cloud landing zone	2029-06	Supported	
PTC-08	Virtualisation cluster	2029-11	Supported	
PTC-04	Core database cluster	2030-01	Supported	
PTC-03	Card authorisation switch	2031-02	Supported	
PTC-12	Backup and archive	2031-08	Supported	
PTC-09	Data warehouse appliance	2031-09	Supported	

Three rows read Past support. Two of the three support-end dates are still in the future, marked above. Until somebody says which column is wrong, the finding rests on one date.

No cost column. No standard-status column. Both have to be added before step 7.7.

Figure 7.2 `O27 Physical Technology Compone` sorted by `Support end`, which is what `RPT10 Technology Currency` reprints before adding its one footer line. Ardent Union Bank is an invention; the dates and positions are the shipped file's [3].

Step 7.7 — Produce the technology-currency finding

Produce. A one-page paper, by component, of what falls out of vendor support and when, with the annual cost, the standard status and the applications affected. This is the Phase D artefact that gets budget approved.

You need. Step 7.4 for `Support end` and the added `Standard status` and `Annual cost (Rm)` columns, step 7.6 for conformance, the rebuilt `R07`, and `O19 Contract` for notice periods.

Look it up. Volume 1, chapter 14, *What a core replacement costs, and how long it takes*, before anybody puts a replacement date on a core platform.

Where it lands.

> **`O29 Gap`**
>
> Seven columns: `Ref`, `Gap`, `Domain`, `Building block`, `Severity`, `Status`, `Due`. The sheet holds **zero records** in the shipped file, and `RPT07 Gap Analysis` is a header with no rows [8]. The currency finding is the first gap to write: `Domain` = `Technology`, `Building block` the `TBB-` reference, `Severity` from the criticality of the applications affected.

How.

1. Filter `O27` on `Support end` inside thirty-six months and sort ascending. Ardent's earliest three are `PTC-10` at 2026-10, `PTC-05` at 2026-11 and `PTC-06` at 2026-12 [3].
2. Cross `Support end` with the `Standard status` column you added. A component both out of support and contained is a decision already taken and unfunded; one out of support and still current in the standards is a standard nobody updated. This cross is the finding, and

the shipped file cannot produce it, because `RPT10` has no standard-status column to cross and `R18` points at applications rather than components.

3. Pull the applications per component from the rebuilt `R07` and the dependency grade with them. On the shipped positional join the three past-support components carry `A05`, `A06` and `A10`, none of them `Critical` — which is itself evidence that the join is generated, because a bank's oldest platforms carry its oldest and most critical books.
4. Add the notice period from `O19 Contract`. A support agreement expiring 2027-01 with twelve months' notice is a decision due now, not next year; `CTR-01 Core Banking vendor agreement` expires 2027-01 [1].
5. Total the annual cost of the affected components. That number, next to the applications and the dates, is the paper.

Signed by. Chief Information Officer presenting; Investment Committee (`GB-02`) deciding, since it holds `G0` funding and the Architecture Board cannot approve funding [8].

Done when. Every component with `Support end` inside thirty-six months appears with its cost, applications, standard status and contract notice period, and each has either a funded work package or a recorded risk acceptance.

Shows up in. `RPT10 Technology Currency` footer, and `RPT07 Gap Analysis` once `O29 Gap` has rows in it.

Goes wrong when. The paper leads with the standard rather than the date. "Non-conformant with `STD-10`" is an architecture opinion. "`PTC-06 Branch server estate`, support ends 2026-12, the branch network runs on it, notice period six months, and the position column has read `Past support` since before the date" is a fact with a deadline.

THE CURRENCY PAPER, BEFORE YOU PRESENT IT

Two of the three `Past support` rows have support end dates in the future. Which column is wrong, and who corrects it?

For each component, what is the earliest decision date once you count the contract notice period back from expiry?

`027` has no cost column. What is the annual cost of the three past-support components, and where did that number come from?

Form D-1 — the technology currency paper

One row per component with `Support end` inside thirty-six months, sorted by date. `Notice` comes from the matching `019 Contract` row and the decision date is expiry minus notice. The last column is a funded work package or a recorded risk acceptance — never "monitor". A first pass usually needs four to eight rows. Presented by the Chief Information Officer, decided by `GB-02`.

Ref	Support end	Cost (Rm)	Apps on it	Notice	Funded or accepted
PTC-06	2026-12		1	6 months	Risk accepted, expires 2027-03

Step 7.8 — Publish the platform services and record who consumes them

Produce. `028 Platform Service` populated with a consumption model, and a relationship sheet joining each platform service to the applications that use it.

You need. Steps 7.1 and 7.4. A view on which services a delivery team can obtain without raising a project.

Where it lands.

> **`028 Platform Service`**
>
> Three columns only: `Ref`, `Platform service`, `Description`. Ardent holds eight rows — `Container platform`, `API gateway`, `Event backbone`, `Secrets management`, `CI/CD`, `Observability`, `Data catalogue`, `Feature store` [1]. Nothing in the twenty-five relationship sheets references a `PSV-` row, so this is the only object sheet in the repository that is joined to nothing at all. Add `Technology service` as a `TSV-` reference, `Consumption model` as `Self-service` or `Managed`, and `State` as `Baseline` or `Target`.

How.

1. Define a platform service only where a delivery team can actually obtain it. Otherwise it is a technology service with a brochure.
2. Set `Consumption model` from how it is obtained today, not the target operating model. If obtaining it needs a ticket and a conversation, it is `Managed`, and saying so is what makes the gap visible.
3. Mark `State` = `Target` where no product is selected. `API gateway` is one of those: `O65 EP-03 API gateway` records that the product is not yet selected [6], while `O41 SEC-05 Interface logging` records a control `In place` at that same gateway.
4. Add the consumer join and populate it, so a `Target` platform service with ten consuming applications reads as a dependency rather than a deliverable.

Signed by. Chief Information Officer.

Done when. Every platform service names a technology service and a consumption model, every `Self-service` claim can be demonstrated, and the consumer join exists.

Shows up in. Nothing today. After the join, Phase E work-package shaping.

Goes wrong when. The sheet stays joined to nothing. An object sheet nothing references is invisible to every report, and `RPT13 Orphan Analysis` reads applications only.

Where it all actually sits

Step 7.9 — Place the estate, and reconcile the two location columns

Produce. `O12 Location` with a criticality and a real scale, and `R20 Application located at Loca` rebuilt so that it agrees with `O27`'s own `Location` column.

You need. Step 7.4. The facilities list. `O09 Organization Unit` with headcounts.

Where it lands.

> **`O12 Location`**
>
> Four columns: `Ref`, `Location`, `Type`, `Scale`. Ardent holds eight rows: `Head Office`, `Operations Centre`, `Contact Centre`, `Branch Network`, `Primary Data Centre`, `Secondary Data Centre`, `Cloud Region`, `Disaster Recovery Site` [2]. Add `City`, `Country` and `Criticality` in the same tier language as `O23`. `Criticality` comes from what happens if the place is unavailable, not from how many people sit in it.

> **`R20 Application located at Loca`**
>
> Four columns: `Application`, `Application name`, `Location`, `Location name`. One row per place an application runs. Add `Role` from `Primary`, `Secondary`, `Disaster recovery`, `Distributed`, or the sheet cannot tell a live site from a standby one.

How.

1. Fix `Scale` first. It reads `275 people` for the Primary Data Centre and `205 people` for the Disaster Recovery Site, against `248 people` for the whole branch network [2]. The eight rows total 1,101 against 2,012 staff in `O09`, so half the bank is unplaced.
2. Rebuild `R20`. The shipped sheet is positional — `A01` to `LOC-01` through `A08` to `LOC-08`, then `A09` back to `LOC-01` [2] — which records the sanctions screening system as running only at the disaster recovery site.
3. Reconcile `R20` against `O27`'s `Location` column through `R07`. The two disagree for most applications, and only one can be right: an application cannot be somewhere its component is not.
4. Mark the recovery site's role explicitly, and record a cloud region as a location with no headcount rather than leaving it out.
5. Record the applications and components with no location rather than guessing.

Signed by. Chief Information Officer, with facilities confirming the headcounts.

Done when. Every `O27` component and every application has a location or a recorded reason, `R20` agrees with `O27`, and `O12`'s scale figures reconcile to `O09`.

Shows up in. No report reads `O12` or `R20`, so the reconciliation is something you do and write down rather than something a sheet will flag.

Goes wrong when. A cross-border site is recorded as an office and nothing else. A foreign branch or a cloud region outside the country is also a data-transfer position, and `CO-05 Data protection` already reads `In breach` [6].

Step 7.10 — Record the environments and the personal data in them

Produce. `048 Environment` populated honestly, with the personal-data flag set from a check rather than from an assumption.

You need. Step 7.9. The non-production estate's actual configuration, which usually nobody holds centrally.

Where it lands.

> **`048 Environment`**
>
> Four columns: `Ref`, `Environment`, `Description`, `Personal data present`. Ardent holds four rows — `Production`, `Pre-production`, `Test`, `Development` — and the flag reads **`No` for Production**, `Yes` for Pre-production, `No` for Test, `Yes` for Development [2]. A bank's production environment holds personal data by definition, so that row is wrong and the two `Yes` rows are the control finding. Add `Where it runs` as a `LOC-` reference, `Data classification`, and `Provenance` saying how the row came to exist.

How.

1. Fix the Production row, and treat its being wrong as the headline: a sheet that says production holds no personal data will be quoted by somebody who has not read it.
2. Set the flag for each non-production environment from a check of the last refresh, not from the masking policy.
3. Cross-check `041 SEC-07 Non-production masking`, `In place` at `Provisioning`, against `065 EP-05 Non-production provisioning`, which reads "Most refreshes are unmasked" [6]. Both cannot be true, and `048` says personal data is present in two of four environments.
4. Fill `Provenance` with `Authored` where an architect wrote it from conversation, and change it only when somebody verifies it against a system. Half of what an architect records about an estate is hearsay until checked, and the honest move is to publish it marked as hearsay.
5. Join environments to locations, which nothing in the repository does today.

Signed by. Chief Enterprise Architect, with the Chief Information Security Officer on non-production.

Done when. Every environment names where it runs, what classification of data it holds and how the row came to exist, and the production row is correct.

Shows up in. `RPT11 Security Position` indirectly, through `SEC-07`.

Goes wrong when. `Provenance` is left blank because the architect is confident. A blank provenance column makes every row look equally verified.

Step 7.11 — Record the network links, and de-duplicate them

Produce. `050 Network Link` with one row per link, every external party represented, and the duplicate rows resolved.

You need. Step 7.9. The network team's site and partner connection list.

Where it lands.

> **`050 Network Link`**
>
> Five columns: `Ref`, `From`, `To`, `Bandwidth`, `Diverse path`. Add `What it carries` as a sentence about business content rather than bandwidth, and `Provenance`. `Diverse path` is `Yes` or `No` and it is the column `PRN-04 Business Continuity` is assessed against — the principle reads "Operations continue despite the loss of any single component" [7].

How.

1. Resolve the duplicates first. `NWL-01` and `NWL-09` are both Operations Centre to Contact Centre, one `Diverse path` = `Yes` and one `No`; `NWL-02` and `NWL-10` are both Contact Centre to Branch Network, at 100 Mbps and 10 Gbps, again with opposite verdicts [2]. Four rows describing two links, each pair contradicting itself.
2. Read the link that matters most. `NWL-04`, Primary Data Centre to Secondary Data Centre, reads `Diverse path` = `No` [2]. That is the one link a bank's continuity position depends on. It is an `O29 Gap` row and a Joint Standard 1 of 2023 exposure, because that standard requires a geographically separate disaster recovery site and annual testing including site loss [9].
3. Add the external links. The ten shipped rows form a closed ring between Ardent's own sites: no payment scheme, no interbank utility, no supervisor submission path, no partner, no public internet. Each of those is an architecture object in a South African bank.
4. Write `What it carries` in business terms: who is at the far end, how many of them, which application they reach.
5. Set `Provenance` on every row.

Signed by. Chief Information Officer.

Done when. Every external party that reaches an application has a link, no pair of rows describes the same link twice, and every link says what it carries.

Shows up in. No report reads `O50`, so the diversity finding has to be carried by hand into `O29 Gap` and the checkpoint pack.

Goes wrong when. The sheet becomes a network diagram in table form. Three thousand merchants reaching the acquiring path is an architecture fact; the circuit type is not.

Resilience, and what a supervisor will ask for

O23 CRITICALITY AGAINST O49 TIER AND TEST STATUS

Ref	Application	O23	O49 tier	Objective	Tested
A03	Card Management	Critical	Tier 1	24 hours	Never tested
A11	General Ledger	Critical	Tier 3	1 hour	Never tested
A12	Regulatory Reporting	Critical	Tier 2	1 hour	Never tested
A02	Payment Hub	Low	Tier 3	15 min	Never tested
A01	Core Banking	High	Tier 2	4 hours	Tested, no date
A13	Data Platform	High	—	—	no O49 row
A14	Fraud Detection	Medium	—	—	no O49 row

Three Critical applications on three different tiers. The Tier 1 rows carry 24-hour objectives while a Tier 3 payment hub carries fifteen minutes. Ten of twelve rows read Never tested, the two that do not carry no date, and two applications have no row at all.

Figure 7.3 `049 Resilience Tier` read against `023 Application Component`'s own `Criticality` column. Nothing in the repository forces the two to agree, so the first job of step 7.12 is to make them [2].

`049 Resilience Tier` is what makes `PRN-04 Business Continuity` testable. Without it the principle is a sentence. With it, it is twelve rows with a tier, a recovery objective and a test status — and in the shipped file all three columns argue with something else.

Volume 1 chapter 15 sets out what the Prudential Authority and the FSCA expect. Two Joint Standards apply to banks and this phase has to evidence both. **Joint Standard 1 of 2023**, on information technology governance and risk management, commenced 15 November 2024 and obliges an IT asset inventory, a register of critical applications, backup with restore testing at least annually, recovery time *and* recovery point objectives, a geographically separate disaster recovery site and annual resilience testing including site loss [9]. **Joint Standard 2 of 2024**, on cybersecurity and cyber resilience, commenced 1 June 2025 with twelve months to comply fully, and adds an inventory of information assets, an assessment of risks to critical operations, and least privilege with multi-factor authentication for users reaching critical functions and applications holding sensitive information [10].

THE NUMBERING ERROR IN CIRCULATION

Joint Standard 1 of 2024 is outsourcing by insurers and does not apply to banks [11]. You will see it listed as a banking outsourcing obligation in learner guides and compliance summaries, and it is wrong. Banks' outsourcing obligations come from the Banks Act Regulations and Prudential Authority instruments, and from the third-party provisions inside the two technology Joint Standards. Volume 1 chapter 15 sets out the four instruments, three of them numbered "1". Check the number before you put it in a control objective.

Read against that list, four sheets here are regulatory artefacts rather than architecture documents: `O23` is the register of critical applications, `O27` the IT asset inventory, `O49` the recovery-objective and testing record, and `O12` with `O50` the separate-site evidence. Each is a data product with an owner, not an appendix to a policy.

Step 7.12 — Set the resilience tiers and make them evidence something

Produce. `049 Resilience Tier` with a tier that agrees with criticality, a recovery time and a recovery point objective, a test cadence and a test date.

You need. `O23` with `Criticality` from step 6.3, `O13 Business Capability` with its own `Criticality`, and the continuity team's recovery objectives if they exist.

Look it up. Volume 1, chapter 15, *Resolution, and the test you take over a weekend*, for what a recovery position has to survive, and *What the regulator actually asks to see* for the list.

Where it lands.

> **`049 Resilience Tier`**
>
> Five columns: `Ref`, `Application`, `Tier`, `Recovery objective`, `Tested`. One objective column cannot hold both a recovery time and a recovery point, and Joint Standard 1 of 2023 requires both [9] — so split it into `RTO` and `RPO`. `Tested` holds a word, `Tested` or `Never tested`, which cannot evidence an annual cadence; replace it with `Test cadence` and `Last tested` as a date.

How.

1. Reconcile `Tier` against `O23 Criticality` before anything else. Ardent's three `Critical` applications land on three different tiers: `A03 Card Management` is `Tier 1`, `A12 Regulatory Reporting` is `Tier 2` and `A11 General Ledger` is `Tier 3` [2]. Two of the three are wrong, and nothing in the repository forces them to agree.
2. Fix the inversion in `Recovery objective`. The `Tier 1` rows read `24 hours` for `A03`, `A04` and `A05`, while the `Tier 3` row for `A02 Payment Hub` reads `15 min` [2]. The most critical tier carries the loosest objective, which is the column filled without being read.
3. Set `RTO` and `RPO` from the business consequence, then check whether the estate can meet them. A `Tier 1` RPO of minutes against a nightly batch is a gap, and recording both numbers is the point.
4. Cover every application. There are twelve rows for fourteen applications: `A13 Data Platform` and `A14 Fraud Detection` have no tier — the same two the orphan analysis flags.
5. Write `Never tested` where it is true. Ten of twelve rows do [2], and the two that read `Tested` carry no date, so neither can evidence a cadence.

Signed by. Chief Risk Officer (`ROL-06`), who owns continuity, with the Chief Information Officer on feasibility.

Done when. Every `Critical` application is `Tier 1`, every row has an RTO, an RPO, a cadence and either a test date or an explicit `Never tested`, and the untested `Tier 1` rows are named in the checkpoint pack.

Shows up in. No report reads `049` and `ZZ Checks` does not count it, so the untested position has to be carried into the Phase D pack by hand.

Goes wrong when. The tiers are copied from the continuity plan without the dates. A tier with an objective and no test date is a target, and an inspector reads it as one.

RESILIENCE YOU CAN DEMONSTRATE

Two of three `Critical` applications are not `Tier 1`. Which is right, the criticality or the tier, and who decides?

Where the recovery objective is minutes and the data moves in a nightly batch, which number is wrong — the target or the estate?

Ten of twelve rows read `Never tested` and the two that do not carry no date. What would the next test cost, and when is it scheduled?

Form D-2 — the resilience tier worksheet

One row per application. Ardent has fourteen, so continue on a second copy of this form. `Tier` must agree with `023 Criticality`. `RTO` and `RPO` are the two columns `049` does not have and Joint Standard 1 of 2023 requires. `Last tested` is a date or the words `Never tested` — nothing in between. Signed by the Chief Risk Officer.

Ref	Application	Tier	RTO	RPO	Last tested
RES-02	Payment Hub	1	15 min	0	Never tested

Security as architecture

Security here is not a control list bolted on at the end. It is five sheets answering five different questions, and the chain between them is what makes a position auditable.

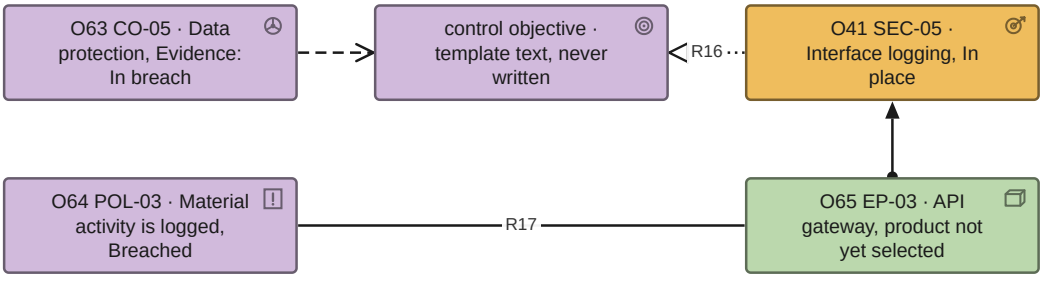

Figure 7.4 The security chain, and where it breaks. `R16 Control Objective met by Co` copies its `Position` from `063`'s `Evidence` rather than from `041`'s own `Position`, so an obligation can read `Evidenced` while the control named as meeting it reads `Absent`. Three of Ardent's seven do.

Sheet	Question it answers
`063 Control Objective`	What must be true, and which instrument says so
`064 Security Policy`	What the bank has committed to
`041 Security Control`	What is actually implemented
`065 Enforcement Point`	Where the control is applied in the architecture
`R16` and `R17`	Whether the objective is met and the policy is in fact enforced

The chain is broken in three places in the shipped file, and the breaks are the most useful findings in Phase D.

Step 7.13 — Record security as architecture, and repair the chain

Produce. `063`, `064`, `041`, `065`, `R16 Control Objective met by Co` and `R17 Policy enforced at Enforcem` consistent with each other, read through `RPT11 Security Position` and `RPT09 Regulatory Coverage`.

You need. Volume 1 chapter 15 for the instruments, `006 Requirement`, steps 7.4, 7.9 and 7.10, and the Chief Information Security Officer and Chief Compliance Officer (`ROL-07`) in one room.

Look it up. Volume 1, chapter 15, *POPIA, with the banking bite*, for the obligation `020`'s classification columns are assessed against.

Where it lands.

> **`063 Control Objective`**
>
> Five columns: `Ref`, `Obligation`, `Source`, `Control objective`, `Evidence`. Two columns are not doing their job. `Source` restates the obligation instead of naming the instrument that creates it — for `CO-01 Basel III / IV` it is a six-word summary of the capital and liquidity ratios [6] — so no row names the Banks Act, the Regulations relating to Banks, Joint Standard 1 of 2023, Joint Standard 2 of 2024, POPIA or FICA. And `Control objective` holds template filler: "Control that evidences Basel III / IV." Write both properly. `Evidence` is the honest position, from `Evidenced`, `In breach`, `No mechanism`.

> **`065 Enforcement Point`**
>
> Four columns: `Ref`, `Enforcement point`, `Acts on`, `Position`. Ardent's five positions are the model for how to write this column: `EP-01` "Newly established", `EP-02` "Enforces authentication, not attribution", `EP-03` "Product not yet selected", `EP-04` "Classification not applied at read", `EP-05` "Most refreshes are unmasked" [6]. Each is a sentence somebody can act on.

How.

1. Repair `R16` first, because it is positional — `CO-01` to `SEC-01`, `CO-02` to `SEC-02`, through to `CO-07` to `SEC-07` [6] — which records Basel III/IV as met by identity lifecycle management, IFRS 9 by design review and data protection by interface logging. Every one of the seven is wrong.
2. Then find what the positional join hides. `R16`'s `Position` column copies `O63`'s `Evidence`, not `O41`'s `Position`, so `CO-02`, `CO-03` and `CO-06` all read `Evidenced` while the controls named as meeting them — `SEC-02 Design review`, `SEC-03 Privileged access management`, `SEC-06 Periodic access review` — all read `Absent` in `O41` [6]. **Three obligations are recorded as evidenced by controls the repository records as absent.** That sentence is the Phase D security finding.
3. Repair `R17` the same way. It is positional, `POL-01` to `EP-01` through `POL-04` to `EP-04`, which leaves `EP-05 Non-production provisioning` enforcing nothing — even though `POL-01` ("Personal information is protected wherever it is", scoped across production and non-production) is precisely the policy `EP-05` exists for. And `POL-04`'s own `Scope` text says it is "Enforced at the design gate" while `R17` enforces it at the data platform [6].
4. Test every `Met` position. `POL-02` reads `Met` at `EP-02`, whose own position says it "Enforces authentication, not attribution", while the control behind it, `SEC-04 Attributable access`, reads `Partial`. `POL-04` reads `Met` at a gate recorded as "Newly established", behind `SEC-02 Design review`, which reads `Absent`. Neither policy is met.
5. Test every `In place` control the same way. `SEC-05 Interface logging` reads `In place` at the API gateway, which has no product selected, while `RPT04 Interface Catalogue` reports twenty of thirty-two interfaces carrying no logging. `SEC-07 Non-production masking` reads `In place` at provisioning, which reports most refreshes unmasked.
6. Read the two reports for what they are. `RPT11 Security Position` reprints `O41` and adds one line, four controls absent; `RPT09 Regulatory Coverage` reprints `O63` and adds one line, four obligations with no mechanism or a mechanism in breach [6]. Both lines are correct arithmetic, and neither touches the contradictions above.

Signed by. Chief Information Security Officer for `O41`, `O64` and `O65`; Chief Compliance Officer for `O63`'s sources; Architecture Board (`GB-01`) for the published position.

Done when. Every control objective names a real instrument, every `Evidenced` position is evidenced by a control that is not `Absent`, every policy names an enforcement point that exists, and every enforcement point enforces at least one policy.

Shows up in. `RPT11 Security Position`, `RPT09 Regulatory Coverage`, and the `ZZ Checks` rows `Security controls absent` and `Control objectives without evidence`, both reading 4 against a threshold of 0 [8].

Goes wrong when. The enforcement point is named but not assessed. A policy enforced at a gate that does not exist is not enforced, and `EP-03`'s position — product not yet selected — is the reason nineteen interfaces reached production unmanaged.

THE SECURITY CHAIN, UNSOFTENED

Which three obligations are recorded as evidenced by controls the repository records as absent, and who signed each off?

Which policy reads `Met` and is enforced at a point that explicitly does not enforce it?

`063 Source` names no instrument. Write the instrument for each of the seven obligations, and check the Joint Standard numbers before you do.

Step 7.14 — Publish the Technology Standards Catalogue and pass the Phase D checkpoint

Produce. The Phase D deliverables closed out, and a checkpoint pack the Architecture Board can approve or refuse.

You need. Steps 7.1 to 7.13. `031 Deliverable` rows `DLV-11 Technology Architecture` and `DLV-12 Technology Standards`, both phase `D`, both currently `Approved` [8].

Where it lands.

> **`031 Deliverable`**
>
> Four columns: `Ref`, `Deliverable`, `Phase`, `Status`. No owner, no due date, no location — add them or the register cannot be chased. Never move a due date to match the day you finished.

How.

1. Assemble the catalogue in four sections: the standards from `032` with their new owners and scopes; conformance from the component sheet you added; the currency paper from step 7.7; the resilience position from `049`.
2. Put six numbers on the front page. For Ardent: sixteen standards, three `Deprecated` and still being assessed against; four in `Breach` with no note; three components past support against one defensible date; twelve resilience tiers for fourteen applications, ten never tested; four security controls absent; four obligations with no mechanism or a mechanism in breach [3][5][6][8].
3. State the provenance. Environments, network links and resilience tiers here were authored, not observed; say so on the front page rather than in a footnote.
4. List the relationships you deleted as generated rather than observed — `R07`, `R20`, `R16`, `R17` — and who approved each deletion. That list is the most important page for whoever

inherits the repository.

5. Take the catalogue to Architecture Board (`GB-01`) and the currency paper on to Investment Committee (`GB-02`). Check `O58 Governance Gate` while you are there: `GT-1 G1 Design` tests whether a design meets principles, standards and the reference architecture, which is only testable once step 7.5 has given every standard a scope.
6. Re-run `ZZ Checks` and compare against the run you kept at the start of chapter 6.

Signed by. Chief Information Officer as owner, approved by Architecture Board (`GB-01`).

Done when. `DLV-11` and `DLV-12` carry a status you believe, every front-page number traces to a sheet in one step, and the Board has either approved the catalogue or named what is missing.

Shows up in. `O31 Deliverable` , `RPT10 Technology Currency` , `RPT12 Standards Conformance` , `RPT11 Security Position` and `ZZ Checks` .

Goes wrong when. The catalogue is published without the currency paper. The standards catalogue alone is a governance document; with the dates, the costs and the applications attached it is a budget submission.

THE PHASE D CHECKPOINT, IN NINE LINES

Every `O25` service has a logical and a physical realisation, and you can name the ones that do not. Every `O27` component has a support end date traced to a contract and a `Position` that agrees with it. Every component carrying a `Critical` application is assessed against the standards whose scope covers it. Every standard has an owner who exists and a scope that can be tested. Every component and every application has a location, and `O27` and `R20` agree. Every environment names its data classification and its provenance. No two `O50` rows describe the same link. Every `Critical` application is `Tier 1` with an RTO, an RPO and a test date. And every `Evidenced` control objective is evidenced by a control that is not `Absent` .

PHASE D — A REVIEWER'S QUESTIONS

Which generated joins did you delete, and is the list of them inside the published catalogue?

Which rows were authored rather than observed, and who would have to verify each one?

Which obligation in `063` would you have to answer in an inspection next month, and what evidence exists today?

Joint Standard 1 of 2023 and Joint Standard 2 of 2024 apply to this bank. Which sheet evidences each of their artefact requirements, and which requirement has no sheet at all?

THE CHAPTER IN TEN LINES

- O25 has no availability target and no owner. Add both before anything else in the phase.
- R07 is positional and puts collections on the HSM estate. Delete it and rebuild from the deployment record.
- O26's seven rows are O47's first seven patterns, and they disagree twice. Decide which sheet is authoritative.
- Two of the three Past support rows in O27 have support end dates in the future.
- There is no standard-to-component relationship sheet, so no component here can be assessed against a standard.
- RPT10 and RPT12 reprint O27 and O32 and add a footer at most. The currency cross has to be built by hand.
- O48 says production holds no personal data, and says two non-production environments do.
- Four O50 rows describe two links, each pair contradicting itself; the data-centre interconnect is not diverse.
- Joint Standard 1 of 2023 and Joint Standard 2 of 2024 apply to banks. Joint Standard 1 of 2024 is insurers' outsourcing.
- Three control objectives read Evidenced, met by controls the same repository records as Absent.

SOURCES FOR THIS CHAPTER

- **1.** Banking_TOGAF_Metamodel.xlsx, sheets O25 Technology Service, O26 Logical Technology Component, O28 Platform Service, O19 Contract and R07 Application deployed on Technology
- **2.** Banking_TOGAF_Metamodel.xlsx, sheets O12 Location, O48 Environment, O49 Resilience Tier, O50 Network Link, O09 Organization Unit, O23 Application Component and R20 Application located at Location
- **3.** Banking_TOGAF_Metamodel.xlsx, sheets O27 Physical Technology Component and RPT10 Technology Currency
- **4.** Banking_TOGAF_Metamodel.xlsx, sheets O26 Logical Technology Component and O47 Reference Model Entry
- **5.** Banking_TOGAF_Metamodel.xlsx, sheets O32 Standard, R18 Standard applies to Application and RPT12 Standards Conformance
- **6.** Banking_TOGAF_Metamodel.xlsx, sheets O41 Security Control, O63 Control Objective, O64 Security Policy, O65 Enforcement Point, R16 Control Objective met by Control, R17 Policy enforced at Enforcement Point, RPT09 Regulatory Coverage and RPT11 Security Position
- **7.** Banking_TOGAF_Metamodel.xlsx, sheets O05 Principle and O11 Role
- **8.** Banking_TOGAF_Metamodel.xlsx, sheets O29 Gap, O31 Deliverable, O57 Governance Body, O58 Governance Gate, RPT07 Gap Analysis and ZZ Checks
- **9.** Joint Standard 1 of 2023 — IT governance and risk management (commenced 15 November 2024) https://lawlibrary.org.za/akn/za/act/standard/sarb/2023/1/eng@2024-11-15
- **10.** Moonstone, "Cybersecurity Joint Standard commencement date confirmed" (Joint Standard 2 of 2024 effective 1 June 2025, twelve months to comply) https://www.moonstone.co.za/cybersecurity-joint-standard-commencement-date-confirmed/
- **11.** Joint Standard 1 of 2024 — Outsourcing by Insurers (in operation 1 December 2024; applies to insurers, not banks) https://www.resbank.co.za/content/dam/sarb/publications/prudential-authority/pa-public-awareness/covid-19-response/2024/joint-comms-1-of-2024/Joint%20Standard%201%20of%202024%20Outsourcing%20by%20Insurers.pdf

CHAPTER 8

Opportunities and Solutions

The repository hands you twelve work packages, thirty-seven building blocks and a roadmap. It hands you an empty gap register. Everything in this chapter is about putting the missing half back underneath the half you were given.

Phase E is where the architecture stops describing the bank and starts committing money [10]. You leave it with a gap register, building blocks carrying acceptance criteria, work packages, transition states you can stop at, costed alternatives and a funded roadmap.

Ardent Union Bank hands you the opposite: the *outputs* of Phase E with the *input* missing. `030 Work Package` holds twelve packages worth $64.5m, `055 Building Block` thirty-seven blocks with acceptance criteria, `053 Transition Architecture` three states, `054 Architecture Alternative` five options. `031 Deliverable` records `DLV-13 Consolidated Gaps` as `Approved` [1][7].

And `029 Gap` holds nothing. Zero records — so `RPT07 Gap Analysis` is empty, `R12 Gap closed by Work Package` has zero instances, and sixty-four and a half million dollars of work is justified by nothing you can point at [1][2].

That is this chapter. Build the register that should have come first, then go back through the realization sheets and keep only what survives contact with it.

Ardent Union Bank is invented and its data is real within this workbook. Table Bay Bank is volume 1's invented bank and appears only when pointing back at volume 1.

> **THE COSTS ARE IN DOLLARS**
>
> The `Cost` column of `030 Work Package` and of `056 Project` reads `$7.7m`, `$10.1m`, `$2.0m`. A South African bank's investment paper is in rands. Convert before the first review, state the rate and date once, and never run two currencies in one pack.

BEFORE YOU START THIS PHASE

The comparison tool finds differences between two things you wrote down. What a bank is *structurally* missing comes from volume 1.

- **Volume 1, chapter 11, *The patterns*.** The five application capabilities that are usually missing entirely, and why all five are shared.
- **Volume 1, chapter 14, *Build, buy or configure*.** Sourcing per capability, and the question that actually decides it.
- **Volume 1, chapter 17, *Gaps, and the roadmap in two plateaus*.** What a two-plateau roadmap looks like when it is honest.
- **Volume 1, chapter 7, *The reporting and regulatory calendar*.** The dates Phase E cannot move, which is most of them.

Run `ZZ Checks` and write its fifteen result values down before you touch anything. Cheapest before-and-after evidence you will produce, and several of its rows become gaps in step 8.9 [8].

Part one — the comparison, and what it can honestly tell you

A `Banking_Architecture_Comparison_Tool_TEMPLATE.xlsx` is built for this set. It takes two extracts of the repository and separates three things a naive diff reports identically: a **design delta**, where the target deliberately states something different and you have a genuine gap; **governed drift**, where the baseline moved under an approved change and the target is now stale; and **ungoverned change**, where a value moved and nothing explains it [3].

There are no export sheets. The insurance repository shipped three; this one ships none. Nothing computes an extract for you, so you build both by hand [1].

There is no change log sheet either — no `02 Change Log`, no `Change Ref` or `Approved By` column anywhere. The fifth paste target has nothing to paste from until change control exists, which is chapter 2's step. Say the consequence out loud: with an empty change log the tool cannot tell governed drift from ungoverned change, so every real difference classifies as `UNGOVERNED CHANGE` [1][3].

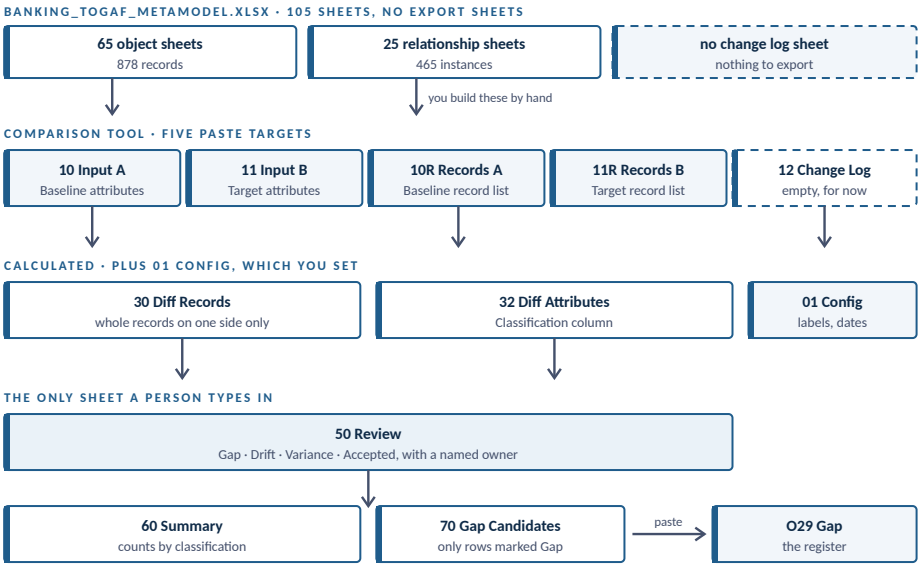

Figure 8.1 Producing the comparison by hand. This repository has no export sheets and no change log sheet, so the first two boxes are built with a query or a formula and the fifth paste target stays empty. Nothing reaches `029 Gap` without passing through `50 Review`.

> **THE COMPOSITE KEY IS THE WHOLE TRICK**
>
> `O13|C05.03.01|Current` is three things joined by a pipe: sheet, record, attribute. Every match in the tool is a lookup on that one string, and the record-level key drops the third part.
>
> Two consequences. The tool's column is headed `ID` and this repository's first column is `Ref`: put the `Ref` value in the `ID` column and do not argue with the header. And renumber a catalogue between snapshots and every key changes, so the whole catalogue reports as removed and re-added — the trap in step 8.3, and `O13`'s 120 rows on the four-part `C01.01.01` scheme are where it bites.

Step 8.1 — Produce the two extracts by hand

Produce. Four blocks of pasted values: baseline attributes, target attributes, and the two record lists.

You need. A named, dated copy of the repository as the baseline; a decision on what snapshot B is; the extract scope below; somebody who can write a Power Query unpivot, or patience with a formula.

How.

1. Copy the workbook, rename it with a date, treat the copy as read-only. There is no `04 Baselines` sheet, so the filename and a line in the deliverable are your only baseline record.
2. For every sheet in the extract scope, build one row per record per material attribute: `Key | Kind | Type | Type Name | ID | Name | Attribute | Value`. `Kind` is `Object` or `Relationship`; `Type` is the sheet prefix.
3. Build the record list the same way, one row per record: `Key | Kind | Type | Type Name | ID | Name`. Relationship sheets go in it too — a link on one side only is a real finding, and

`30 Diff Records` is the only place it shows.

4. Paste Special, Values Only, into `10`, `11`, `10R` and `11R`. Never link: external references break on rename and depend on the reader enabling link updates.
5. Leave `12 Input Change Log` empty and note on `01 Config` that it is empty and why.
6. Check the counts. The repository states 878 records and 465 relationship instances, so a record list materially short of 1,343 rows per side has a sheet missing [1][8].

Signed by. Chief Enterprise Architect, `ROL-08`, recorded as `Prepared by` on `01 Config`.

Done when. `60 Summary` reads the same `Records in A` and `Records in B` as your own row counts, and no cell in any of the five input sheets holds a formula.

Shows up in. `60 Summary` — `Records in A`, `Records in B`, `Attribute rows compared`, `Change log entries supplied`. The last one will read zero, and that is correct here.

Goes wrong when. Somebody extracts `Description` too. A diff across free text buries every real finding under rewording, and `013` alone carries 120 descriptions written to a template.

Where it lands.

> **`10 Input A Baseline · 11 Input B Target · 10R Records A · 11R Records B`**
>
> Five paste targets, values-only. `10` and `11` take the attribute extract; `10R` and `11R` the record lists — without them `30 Diff Records` is silently empty, which looks like a result. `12 Input Change Log` stays empty here.

The extract scope is yours to judge. Extract the attributes a gap could turn on, and nothing else:

Sheet	Attributes in the extract	Why it is material
`013 Business Capability`	`Criticality`, `Current`, `Target`, `Disposition`	The largest single source of candidate gaps
`020 Data Entity`	`Classification`, `Owner`, `Master system`, `Lineage`	A changed master system is a changed architecture
`023 Application Component`	`Disposition`, `Criticality`, `Health`, `Owner`	`Disposition` is the portfolio decision
`027 Physical Technology Compone`	`Support end`, `Position`	A support date that moves moves the roadmap
`030 Work Package`	`Cost`, `Start`, `End`, `Approval`, `Delivery`	The roadmap, and the rows that disagree with themselves
`049 Resilience Tier`	`Tier`, `Recovery objective`, `Tested`	Eleven of twelve read `Never tested`
`R06 Application interfaces with`	`Type`, `Frequency`, `Managed`, `Logged`	Nineteen unmanaged, twenty unlogged, ten direct reads

Add `032 Standard` on `Status` and `Conformance`, and `R18 Standard applies to Applica` on `Conformance`. Several tab names are truncated at thirty-one characters; spell them as the tabs do.

Step 8.2 — Set the configuration before you read a single result

Produce. A dated, labelled, attributed comparison that states which question it is answering.

You need. The baseline filename and date from step 8.1; the date snapshot B was drawn; a decision on mode.

How.

1. Set `Snapshot A label` to the baseline's filename and date. There is no baseline register here, so the label is the register.
2. Set `Snapshot A date` to the date you froze the copy, not the date you ran the tool.
3. Set `Snapshot B label` and `Snapshot B date` to the state you extracted and the day you extracted it.
4. Set `Comparison mode`. `Baseline vs Target` makes an unexplained difference a gap candidate; `Snapshot vs Snapshot` makes it an ungoverned change. The mode changes no formula — it changes what you may conclude, and with no change log it is the only thing separating the two readings.
5. Set `Prepared by` to a role from `011 Role` and `Prepared on` to today. A comparison with no preparer is not evidence.

Signed by. Chief Enterprise Architect, `ROL-08`.

Done when. `60 Summary` echoes all five configuration fields, and the dates bracket a period you can describe in one sentence.

Shows up in. `60 Summary`, top block — `Snapshot A`, `Snapshot A date`, `Snapshot B`, `Snapshot B date` and `Mode` all read back from here. A blank in that block is a comparison nobody can date.

Goes wrong when. Snapshot B is dated the day you ran the tool, not the day you took the extract. Those are a fortnight apart, somebody edits a sheet between them, and you defend a difference that arrived after your own cut.

Where it lands.

> **`01 Config`**
>
> `Snapshot A label`, `Snapshot A date`, `Snapshot B label`, `Snapshot B date`, `Comparison mode`, `Prepared by`, `Prepared on`. Nothing is pre-filled, and `60 Summary` reads back from here.

BEFORE YOU RUN THE COMPARISON, ANSWER THESE

What exactly is snapshot B? The target architecture you drew in Phases B to D, or the repository as it stands today? Write one sentence.

If a difference turns out to be unexplained, who do you go and ask? Name the role, not the team.

What will you do with the count of unexplained differences, given that no change log exists to explain any of them?

Step 8.3 — Read the record-level differences

Produce. Whole records present on one side only, triaged into additions, removals and renumbering noise.

You need. Steps 8.1 and 8.2.

How.

1. Filter `30 Diff Records` on `Status`. Read the two blocks separately: `From A` walks the baseline, `From B` walks the target and reports only what the baseline lacked, so nothing is double-counted.
2. Group `REMOVED in B` by `Type`. Where nearly every record of a type is removed and a similar number added, compare the `ID` columns — a changed identifier shape means renumbering, not removal.
3. Read the relationship rows alone. `R06 Application interfaces with` carries thirty-two instances, and a row present only in B is a new integration between two systems nobody decided to integrate. That is the most useful finding this sheet produces.
4. Account for every genuine removal as decommissioning, omission or error, and say which.

Signed by. Chief Enterprise Architect, `ROL-08`.

Done when. Every `REMOVED in B` row is accounted for, and the renumbered ones are excluded from any count you quote out loud.

Shows up in. `60 Summary` — `Records added` and `Records removed` against `Records in A`. Those two numbers size a catalogue build. They do not size a gap register.

Goes wrong when. You quote the raw count. Renumber `O13` from `C01` to a new scheme and the sheet reports 120 removals and 120 additions out of 878 records. Nothing moved. A register

built off that number raises a hundred capability gaps that do not exist.

Where it lands.

`30 Diff Records`

`Scope | Key | Kind | Type | Type Name | ID | Name | In A | In B | Status`

`Scope` is `From A` or `From B`; `In A` and `In B` are 1 or 0 from a match on the record key; `Status` is `Present both`, `REMOVED in B` or `ADDED in B`. Relationships are compared here and only here.

Step 8.4 — Read the classification column, and know what it can mean

Produce. Attribute-level differences sorted into the six values the tool computes.

You need. Step 8.3 complete, so that renumbering noise is already identified.

How.

1. Filter `32 Diff Attributes` on `Classification` and read the groups in this order: `UNGOVERNED CHANGE`, `Added in B`, `Record removed in B`, `Governed drift + further change`, `Governed drift`, `Same`.
2. For every `UNGOVERNED CHANGE` row, read `Value in A` against `Value in B` and decide which story fits: a deliberate design decision nobody logged, or somebody editing a cell. Different owners, different remedies.
3. A design delta is deliberate and somebody can name the decision. It earns a gap and eventually a work package.
4. An ungoverned change is a governance defect. It earns a change request against `061 Governance Change Request`. Writing it into `029 Gap` funds somebody's unlogged edit, the worst outcome available from this tool.
5. Read `Added in B` as the usual shape of a genuine delta — the target says what the baseline did not.

Signed by. Chief Enterprise Architect, `ROL-08`; the domain architect for the sheet concerned co-signs their own domain.

Done when. Every row not classified `Same` has been read by a person and sorted into design delta, ungoverned change or noise.

Shows up in. `60 Summary`, the classification split. Read the split before the totals. A split that is almost entirely `UNGOVERNED CHANGE` is telling you about your change control, not about your architecture.

Goes wrong when. Somebody takes the `UNGOVERNED CHANGE` count to an audit committee as a count of unauthorised changes. Here it counts *unexplained* differences, because nothing exists to explain any of them. **Look it up.** Volume 1, chapter 15, *What the regulator actually asks to see*.

Where it lands.

32 Diff Attributes

`Scope | Key | Type | Type Name | ID | Name | Attribute | Value in A | Value in B | Differs | Logged change | Change Request | Logged new value | Classification`

`Differs` is 1 or 0; `Logged change` matches the key against `12 Input Change Log`; `Classification` is `Same`, `Governed drift`, `Governed drift + further change`, `UNGOVERNED CHANGE`, `Added in B` or `Record removed in B`. No judgement about significance — that is step 8.5.

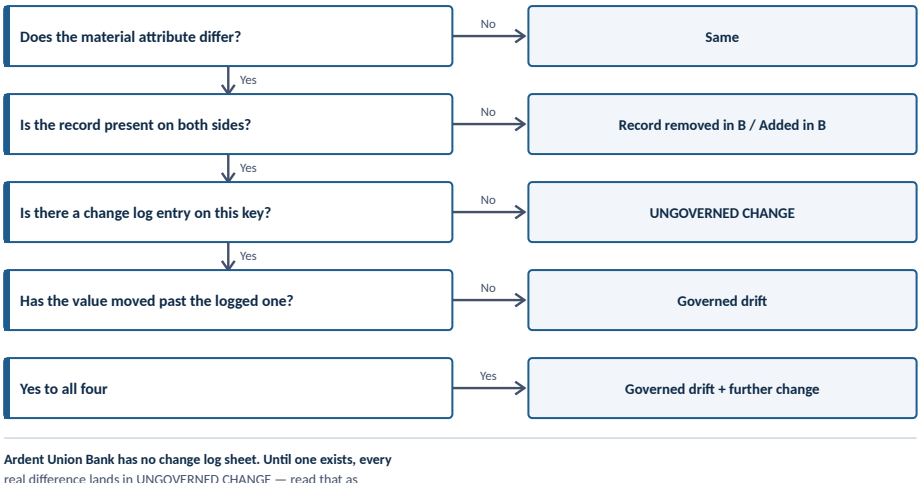

Figure 8.2 What the `Classification` column decides: four questions, six answers, no judgement about significance. The band at the foot is the banking-specific consequence of an empty `12 Input Change Log`.

Step 8.5 — Classify every difference on `50 Review`, with a name against it

Produce. A human verdict on each difference, with a rationale and an owner.

You need. The output of steps 8.3 and 8.4; `011 Role`; ninety minutes with each domain architect.

How.

1. Copy each difference needing a decision from `32 Diff Attributes` into `50 Review`: key, type name, `Ref`, attribute, tool classification.
2. Set `Your classification` to one of four: **Gap**, a real architectural difference; **Drift**, the baseline moved and the target is now stale; **Variance**, a modelling difference with no architectural meaning; **Accepted**, deliberate and fine.
3. Write `Rationale` as a sentence a stranger can read in eighteen months. "Payment Hub disposition moved from Retire to Invest after the scheme mandate assessment" is a rationale. "Agreed" is not.
4. Set `Owner` to a role from `011 Role`. All eighteen are `Performed by: Person`, so there is no excuse for a committee.

5. Set `Action`: a Gap goes to `O29`, a Drift reissues a target artefact, a Variance is a modelling correction, an Accepted row gets a line in the deliverable so nobody raises it next quarter.

Signed by. Each row by its named `Owner`; the sheet as a whole by the Chief Enterprise Architect.

Done when. Every row has a `Your classification`, a `Rationale` of more than four words and an `Owner` that is a role from `O11 Role`.

Shows up in. `60 Summary`, the review split — Gap, Drift, Variance, Accepted. And `70 Gap Candidates`, which carries only the Gap rows.

Goes wrong when. The architect classifies all of it alone in an afternoon. The rule is the point of the sheet: **a difference is not a finding until a named person says it is.** A tool that presents raw differences as findings loses its audience permanently, and quickly.

Where it lands.

> **`50 Review`**
>
> `Key | Type Name | ID | Attribute | Tool classification | Your classification | Rationale | Owner | Action`. The only sheet anyone types in, and nothing reaches `70 Gap Candidates` until a row here says `Gap`.

Three worked rows, so the shape is concrete.

Key	Values	Verdict	Owner		
`O23\	A07\	Disposition`	`Retire`, `Health Poor`, 260 users	Gap — a retiring front end carrying the most users is a date, not a disposition	Chief Information Officer
`O49\	RES-02\	Tested`	`Never tested` against a 15-minute objective	Gap — an untested fifteen-minute objective is an assertion	Head of Operations
`O20\	E01\	Master system`	`Collections & Recovery` masters `Party`	Variance in the diff, finding in the architecture — step 8.8	Chief Data Officer

Step 8.6 — Lift `70 Gap Candidates` and set the severity yourself

Produce. A block of gap candidates shaped to paste into `O29 Gap`, with a severity you can defend.

You need. `50 Review`; `O63 Control Objective`; `RPT09 Regulatory Coverage`; the regulatory calendar.

How.

1. Read `70 Gap Candidates`. It carries only rows a person classified `Gap`, and `Severity` arrives as a placeholder because the tool has no basis on which to set it.
2. Set `Severity` on **consequence, not size**. Critical: a supervisory submission, a settlement or a statutory notice fails. High: a control objective in `O63` has no mechanism, or a `Critical` application has no tested recovery. Medium: cost or change lead time. Low: modelling hygiene.
3. Cross-check `RPT09 Regulatory Coverage`: `CO-01 Basel III / IV` and `CO-05 Data protection` read `In breach`, `CO-04 Payment scheme rules` and `CO-07 Conduct and fair treatment` read `No mechanism` [4]. Any gap touching those four is Critical before you argue about anything.
4. Set `Due` off the regulatory calendar, not the programme's convenience.
5. Decide where the tool's `Owner` and `Source difference` go, because `O29 Gap` has no owner column. Either add `Owner` and `Source` — the addition chapter 2 makes for change control — or keep the comparison workbook as the attachment of record and cite its `50 Review` key in the gap text.

Signed by. Chief Enterprise Architect proposes; `GB-01 Architecture Board` sets severity on anything Critical.

Done when. No row carries a placeholder severity, and every Critical row names the obligation or the service behind it.

Shows up in. `RPT07 Gap Analysis`, once step 8.10 has loaded `O29`. `Severity` and `Due` are two of its seven columns.

Goes wrong when. Severity is set on how annoying the gap is to the architect. It is the ranking an investment committee will use to cut your programme, so set it as though somebody will take the bottom third away. They will.

Where it lands.

> **`70 Gap Candidates`**
>
> Shaped to paste into `O29 Gap`. Give candidates a distinct `Ref` prefix while they are candidates, so a half-finished paste cannot be mistaken for the register. `Building block` stays empty until step 8.11.

Part two — the three sources of gap the tool cannot find

A diff compares two things you wrote. It cannot find what neither says, and in a bank the gaps outside it are usually the larger half.

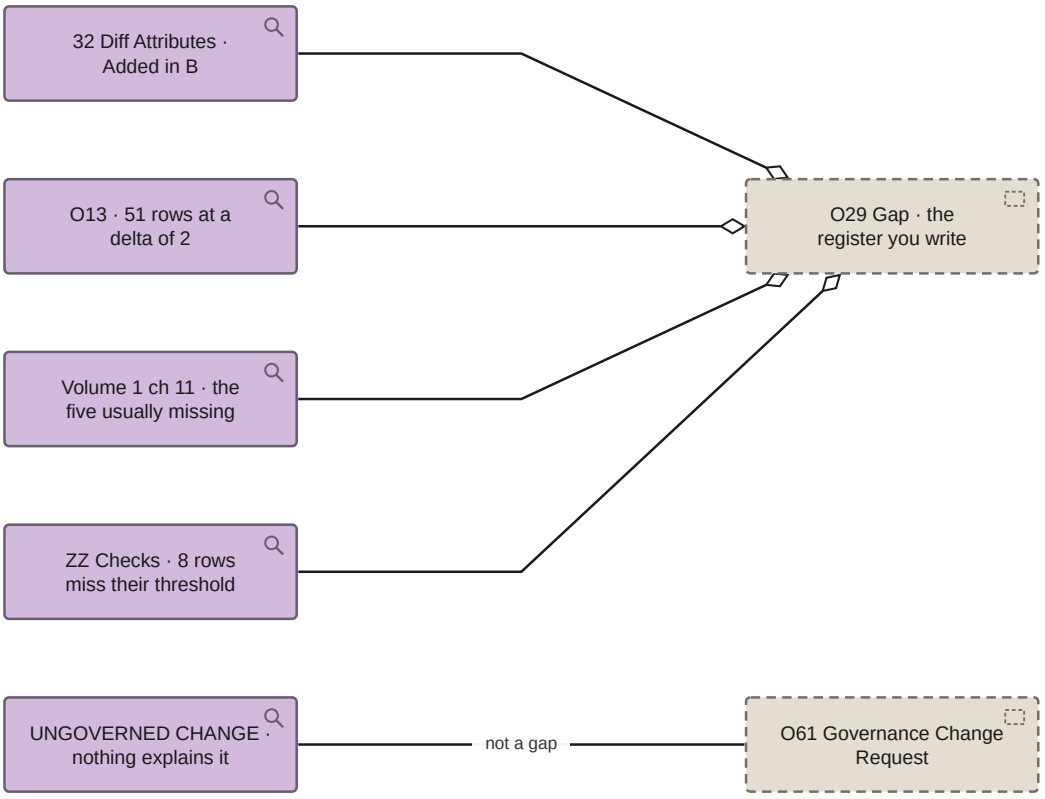

Figure 8.3 Four sources aggregate into the gap register. The fifth is the one that looks like a gap and is not: an ungoverned change earns a change request against `061`, not a work package.

Step 8.7 — Derive capability gaps from the maturity delta, not from the heatmap

Produce. A ranked list of capability gaps from `O13 Business Capability`.

You need. `O13`; `RPT03 Capability Heatmap`; `R22 Org Unit owns Capability` for the owner.

How.

1. Compute `Target` minus `Current` for all 120 rows of `O13` — twelve at level 1, thirty-six at level 2, seventy-two at level 3 [1].
2. Read what the sheet is telling you before you use it. **Every one of the 120 rows has `Target` above `Current`**: sixty-nine at a delta of one, fifty-one at two. And `Disposition` reads `Build` on exactly those fifty-one and `Sustain` on the other sixty-nine. The column is the arithmetic restated, so re-derive disposition yourself.
3. Filter to delta of two **and** `Criticality` `Critical` or `High`. Twenty-nine rows, and a gap list you can defend in a room.
4. Read the five level-1 rows in it: `C02 Product Management` 3 to 5, `C04 Credit Risk Management` 2 to 4, `C06 Deposits & Accounts` 3 to 5, `C11 Finance & Regulatory Reporting` 2 to 4 and `Critical`, `C12 Channel & Distribution` 1 to 3.
5. Then go two levels down, because `RPT03 Capability Heatmap` shows level 1 only — twelve rows of 120. Six level-3 rows are `Critical` at a delta of two, among them `C05.03.01`

`Payments Assurance — Define` and `C06.03.02 Deposits Execution — Operate`.

6. Aggregate to the level you will fund. One gap per row is 120 gaps and a register nobody reads.

Signed by. The owning executive from `R22 Org Unit owns Capability` per level-1 capability; `GB-01 Architecture Board` for the set.

Done when. Every gap names a capability `Ref`, a delta and a criticality, and the count is one a board will sit through.

Shows up in. `RPT03 Capability Heatmap` — its `Gap` column is the delta, and after this step every row at `Gap` 2 with `Criticality` `Critical` or `High` has an `O29` row behind it.

Goes wrong when. You take `RPT03` as the gap analysis. Twelve rows of level-1 summary, where `C12 Channel & Distribution` at 1 against 3 is the only alarming one — while `C06.03.02`, also at 1 and also `Critical`, is invisible.

Look it up. Volume 1, chapter 19, *Capability to application — the coverage map* — and map by name, never by number. Volume 1's `C-01…C-20` are local identifiers; this repository runs `C01…C12` with its own decomposition and the two do not line up.

Where it lands.

> **`O29 Gap`**
>
> For a capability gap, `Domain` is `Business` and `Gap` is a shortfall with a number in it: "Finance & Regulatory Reporting at maturity 2 against a target of 4; Critical." Not "improve regulatory reporting".

Form 8-A — capability gap candidates

One row per gap you will fund, from the twenty-nine-row filter in step 8.7. Columns are `029 Gap`'s own in sheet order, less `Status` — type that into Excel as `Open`. A first pass usually needs eight to fourteen rows. Signed by the owning executive from `R22`.

Ref	Gap	Domain	Building block	Severity	Due
GAP-01	Example, delete me — C11 at maturity 2 against target 4	Business		Critical	2028-06

Step 8.8 — Test the five capabilities a bank is usually missing entirely

Produce. Five explicit findings, each either "present and evidenced" or a gap.

You need. Volume 1 chapter 11; `O23`; `O20`; `O18 Product`; `RPT09 Regulatory Coverage`; `049 Resilience Tier`.

How.

1. Take the five by name, not by number. Volume 1's `A-nn` identifiers are local; the repository has its own `Ref` scheme.
2. **Product definition, catalogue and pricing.** Nothing in `O23`'s fourteen applications describes it; `O18 Product` is eight names; and `O20` masters `E04 Product` in `Credit Risk Engine`. A credit engine is not a product master. Gap, Application domain.
3. **Party master, relationship and mandate.** `O20` masters `E01 Party` in `Collections & Recovery` and `E02 Customer` in `Regulatory Reporting` — two masters, neither a party master, for two entities that are one spine. The bank's central data problem in an application costume. Gap, Data domain.
4. **Rail adaptation, clearing and settlement.** `A02 Payment Hub` exists, which beats most banks. Then read the rest of the row: `Criticality Low`, `Health Poor`, in service since

2010, and `RES-02` gives it a fifteen-minute recovery objective, `Never tested`. Gap, Technology domain.

5. **Regulatory reporting and submission.** `A12 Regulatory Reporting` is `Critical` and `Adequate`, `E13 Regulatory Report` has `Full` lineage — yet `RPT09` shows `CO-01 Basel III / IV` in breach and `CO-04 Payment scheme rules` with no mechanism. A reporting tool over a mart is not this capability; the submission lifecycle, the lineage and the resubmission record are. Gap, Data domain.
6. **Document, evidence and communication.** Nothing holds a rendered artefact — no application, no `O20` entity, no `O38 Data Set` — so the bank cannot reproduce what a customer received. Gap, Application domain.

Signed by. Chief Enterprise Architect; the Chief Data Officer co-signs the party and reporting findings.

Done when. All five carry a written verdict, including the ones you decided are present, with the evidence named.

Shows up in. `O29 Gap`, and then `RPT13 Orphan Analysis` indirectly — an application named for a capability it does not have is usually also an application nothing references.

Goes wrong when. You conclude a capability is present because an application carries its name. `A12 Regulatory Reporting` is named for it and `RPT09` says four obligations have no mechanism or a mechanism in breach. The name is not the evidence.

Look it up. Volume 1, chapter 11, *The patterns*, and the sentence under it that matters most: all five are **shared**. No product line funds a capability it must share with four others, and no product-line executive is measured on it. That is a funding-model problem presented as an architecture problem, and it is why these five end up in one programme with one owner.

Where it lands.

> **`O29 Gap` · `O55 Building Block`**
>
> Each becomes one gap, not several, and one building block at step 8.11. A shared capability split across four blocks is one nobody will fund.

THE FIVE, AGAINST YOUR OWN BANK

Which of the five does your bank genuinely have, with evidence somebody outside the architecture team would accept?

For the ones it does not have — which product line's budget would have had to pay for it, and who decided not to?

Which single one of the five, if it existed, would close the most rows in your gap register?

Step 8.9 — Harvest the integrity findings as gaps with numbers attached

Produce. A set of gaps whose acceptance criterion is already written, because `ZZ Checks` wrote it.

You need. `ZZ Checks`; `RPT13 Orphan Analysis`; `O04 Measure`; `O05 Principle`.

How.

1. Read `ZZ Checks`: fifteen rows, each with a `Result` and a `Threshold`, and a closing line that says the thresholds are what a healthy estate shows, the results are what this one does, and the distance is the work [8].
2. Write every row where `Result` differs from `Threshold` as a gap. The threshold is the acceptance criterion and the result the baseline, so these gaps are testable the day you write them.
3. Read `RPT13 Orphan Analysis`: `A13 Data Platform`, `Replace`, and `A14 Fraud Detection`, `Retain`, both with no business service mapping to them. An orphan is not always wrong; it is always worth a question.
4. Ask about `A14` first. A fraud engine no business service maps to means the fraud response path is not modelled, and the path from detection to freeze is what a dispute body measures. **Look it up.** Volume 1, chapter 1, *5. Payments is shared infrastructure, and in South Africa it is its own country*.
5. Tie each gap to the measure that will show it closing, and name the principle it offends — checking that principle's `Status` first.

Signed by. Chief Enterprise Architect; `GB-05 Architecture Function` owns the check sheet.

Done when. Every `ZZ Checks` row missing its threshold has a gap or a written reason it does not need one.

Shows up in. `RPT01 Dashboard`, which carries the same counts across twenty-four rows and is the sheet an executive actually looks at.

Goes wrong when. You raise a gap against a principle still `Draft`. `PRN-05 Common Use Applications`, `PRN-10 Data is Shared`, `PRN-15 Requirements-Based Change` and `PRN-20 Least Privilege` are all `Draft` in `005` [6], and a gap raised against a draft principle gets argued away in the meeting. Get those four approved or stop citing them.

Look it up. Volume 1, chapter 18, *Six numbers that tell you whether the estate is healthy*.

Where it lands.

> **`029 Gap`**
>
> One gap per failing check, with the number in the `Gap` text: "Nineteen of thirty-two application interfaces are unmanaged, against a threshold of zero."

`ZZ Checks` row	Result against threshold	The measure it closes against
Interfaces unmanaged	19 against 0	`MSR-08 Managed interfaces`, baseline 27%
Interfaces with no logging	20 against 0	`MSR-08`, and `OBJ-08` at 18 months
Direct database reads	10 against 0	`PRN-10 Data is Shared` — still `Draft`
Platform components past support	3 against 0	`MSR-07 Components within vendor support`, baseline 21%
Control objectives without evidence	4 against 0	`MSR-01 Obligations with evidence`, baseline 23%
Data entities without lineage	3 against 0	`MSR-11 Lineage coverage`, baseline 47%
Work packages past end date	4 against 0	Not a gap — a plan defect, step 9.12

Part three — from gaps to blocks to packages

Step 8.10 — Write the gap register, and set the numbering scheme

Produce. `029 Gap` populated, for the first time in this repository's life.

You need. The candidates from 8.6, the capability gaps from 8.7, the five from 8.8 and the integrity gaps from 8.9; `005 Principle`.

How.

1. Set the `Ref` scheme and write it down. The sheet is empty, so nothing constrains you and nothing records your choice either. `GAP-01` upward, zero-padded, never reused.
2. Write `Gap` as a shortfall with a quantity and a consequence. "Party and Customer are mastered in two different systems, neither of which is a party master" is a gap. "Improve data management" is a budget request.
3. Set `Domain` to `Business`, `Data`, `Application` or `Technology`, spelled as `O55`'s `Domain` column spells it — the only thing joining the two sheets.
4. Leave `Building block` empty on this pass; you cannot fill it until 8.11, so the register is written twice.
5. Set `Status` to `Open` and `Due` to a date somebody else chose — a return cycle, a scheme mandate, a support-end date from `O27`.
6. Sanity-check the count. More than about thirty rows and you are recording differences; fewer than ten, against nineteen unmanaged interfaces and twelve untested recovery objectives, and you are being polite.

Signed by. `GB-01 Architecture Board` approves the register; each row's owning executive approves its own row.

Done when. `RPT07 Gap Analysis` has rows in it, every row has a `Severity` and a `Due`, and `031 Deliverable` can honestly carry `DLV-13 Consolidated Gaps` as `Approved` — which today it does with nothing behind it.

Shows up in. `RPT07 Gap Analysis`, all seven columns. And `RPT01 Dashboard`'s `Gaps overdue` row, which currently reads 0 because the register is empty rather than because nothing is late.

Goes wrong when. One gap per difference. The tool will hand you dozens of rows on `049 Resilience Tier` alone, because eleven of twelve applications read `Never tested`. That is one gap with eleven instances, not eleven gaps.

Where it lands.

> **`029 Gap`**
>
> `Ref` — your scheme, recorded in the deliverable. `Gap` — the shortfall, with a number. `Domain` — one of four. `Building block` — the block that closes it, filled on the second pass. `Severity` — set on consequence. `Status` — `Open`, `In progress`, `Closed`. `Due` — a date somebody outside the programme chose. There is no `Owner` column and no `Principle` column; step 8.6 decided where those live.

BEFORE THE REGISTER IS SIGNED

Which gap would you be most embarrassed to have missed if a supervisor read this sheet next month?

Which gap are you raising because it is real, and which because it funds something you already wanted?

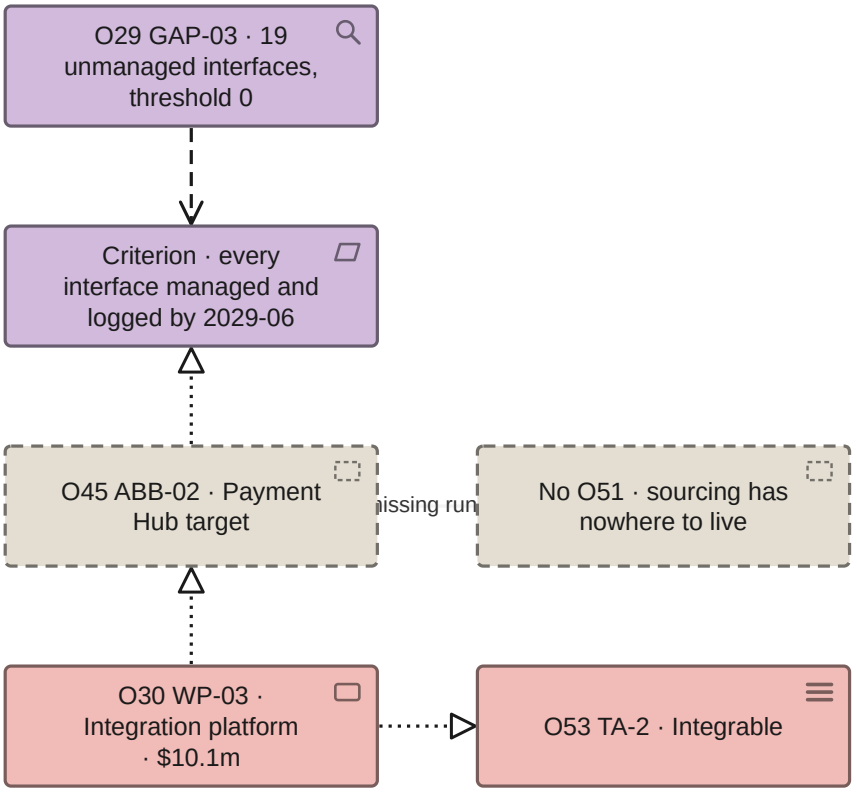

Figure 8.4 The Phase E ladder in ArchiMate 4, one chain worked down. The gap influences the acceptance criterion, the building block realises it, the work package realises the block and the transition state. The dashed box is the rung this repository does not have.

Step 8.11 — Author the building blocks in their domain sheets

Produce. One building block per gap, with an acceptance criterion that is testable, in the right domain sheet.

You need. `029` from 8.10; `035`, `044`, `045`, `046`, `055`; `006 Requirement`; `R09 Requirement satisfied by Bu`.

How.

1. Write each block in its domain sheet. `O35` keys on `Capability`, `O44` on `Entity`, `O45` on `Application`, `O46` on a technology service `Ref` — and `O46`'s second column is *also* headed `Ref`, holding `T01…T07`. Spell it as the sheet does and know which is which.
2. Mirror into `O55 Building Block`, the consolidated thirty-seven-row register with a `Domain` column and the same `Ref` values. Author once, mirror once.
3. Write `What it must do` as behaviour, not as a project name, and `Acceptance criteria` as something a person could test on a Tuesday. The shipped ones are instructive: `ABB-01` reads "No unmanaged interface to or from Core Banking" and `R06` can prove it; `BBB-01` reads "Customer Management measured and owned", and all ten business blocks read that way. Rewrite those ten with a number and a date.
4. Check whether the baseline already passes. `DBB-01` reads "No party record without a named owner" and `O20` already shows `E01 Party` owned by `Head of Customer Management` — so it passes today while the real problem, two masters for one spine, goes untouched. **A criterion the baseline already meets is not a criterion.**
5. Trace back. `R09 Requirement satisfied by Bu` carries thirty-two instances and eight read `No` on `Traced`; `RPT06 Requirements Traceability` reports the same eight. Each is either a gap you have not written or a requirement you should delete.

Signed by. The domain architect for the sheet, `ROL-09` to `ROL-12`; the Chief Enterprise Architect for `O55`.

Done when. `O55` holds one block per gap, every `Acceptance criteria` cell carries a number or a date, and `R09`'s eight `No` rows are zero or explained.

Shows up in. `RPT06 Requirements Traceability`, `Reaches a block` — twenty-four `Yes` and eight `No` today, against a target of thirty-two.

Goes wrong when. Nobody notices there is no `O51`. Sixty-five object sheets, with `O34` and `O51` absent, means **there is no Solution Building Block sheet**. The ladder from architecture building block to solution building block to work package is one rung shorter here, and the build-buy-configure decision has nowhere to live. Put it in `What it must do` or add a `Sourcing` column, and say which in the deliverable.

Look it up. Volume 1, chapter 14, *Build, buy or configure* — and the question that decides it, which is not "is this differentiating" but "does this carry a local specification that changes faster than a vendor's release train".

Where it lands.

`O35 · O44 · O45 · O46 · O55 Building Block`

`O35 — Ref | Business building block | Capability | What it must do | Acceptance criteria`, ten rows. `O44` keys on `Entity`, `O45` on `Application`, `O46` on a technology service `Ref`, seven rows. `O55 Building Block — Ref | Domain | Building block | What it must do | Acceptance criteria` — is the thirty-seven-row union.

Form 8-B — building block and the criterion that tests it

One row per block, in `O55 Building Block`'s own column order. A criterion with no number, date or count goes back. Eight to twelve rows is a page; continue on a second copy. Signed by the domain architect for the `Domain` named.

Ref	Domain	Building block	What it must do	Acceptance criteria
ABB-11	Application	Example, delete me — managed integration	Carry every application interface	0 of 32 interfaces unmanaged by 2029-06

Step 8.12 — Delete `R13` and rebuild it

Produce. `R13 Work Package delivers Build` rebuilt so that each block is delivered by exactly one package.

You need. `O55`; `O30`; `R13` as shipped, which you are about to discard.

How.

1. Read `R13` as shipped first, because it teaches you something about generated repositories. Thirty-seven instances allocated round-robin: `WP-01 Governance foundations` is recorded as delivering `BBB-01 Customer Management capability uplift`, `DBB-03 Account mastering`, `ABB-05 Collections & Recovery target` and `TBB-07 Active-active data centres` — four unrelated blocks across four domains, while `O30` records `WP-01`'s `Domains` as `Business` alone [1][2].
2. Delete the thirty-seven rows and rebuild from the block side, not the package side: for each block in `O55`, name the one package that delivers it.
3. Carry `Domain` on the relationship row, as the columns require.
4. Check two counts by hand. Every block appears exactly once — two packages delivering one block means neither owns the criterion. Every package appears at least once — a package delivering no block is a project looking for a justification.

5. Reconcile `O30`'s `Domains` against the domains of the blocks each package now carries. If they disagree it is usually `Domains` that is wrong.

Signed by. Chief Enterprise Architect; `GB-01 Architecture Board` for the set.

Done when. Row count of `R13` equals row count of `O55`, no block appears twice, and no package is absent.

Shows up in. Nowhere, and that is worth knowing. Thirteen reports and none is a building-block coverage report. Compute the two counts yourself, or add a row to `ZZ Checks` and keep it there.

Goes wrong when. You leave the shipped rows because they look populated. Populated is not true, and `RPT08 Roadmap` will present a package whose blocks have nothing to do with it.

Where it lands.

> **`R13 Work Package delivers Build`**
>
> `Work package | Package name | Building block | Domain | Block name`. The tab name is truncated; the sheet's title row reads "Work Package delivers Building Block". Thirty-seven instances, all wrong.

Step 8.13 — Re-cut the work packages, then link gap to package

Produce. `O30 Work Package` you can defend, and `R12 Gap closed by Work Package` populated for the first time.

You need. `O29`; `O55`; `R13` rebuilt; `O30` as shipped; `O52 Implementation Factor` for what will get in the way.

How.

1. Read the twelve shipped packages. Plausible as a set — governance, data ownership, integration, currency, security, analytics, core renewal, automation, resilience, compliance, customer experience, supplier integration — and each tagged to exactly one domain, three per domain. That symmetry is the tell. Real packages cross domains, because "data ownership and mastering" with no application change is a committee with a mandate.
2. Re-cut so each package delivers a coherent set of blocks, and name every domain it touches in `Domains` as a comma list.
3. Keep the count in the low teens. Twelve is about right for $64.5m; twenty-five is a portfolio pretending to be an architecture.
4. Fill `R12 Gap closed by Work Package` — one row per gap-package pair. Zero instances today.
5. Count both ways and put both numbers in the pack. Gaps with no package are the unfunded architecture; packages closing no gap are what an investment committee will remove, and better you than them.

6. Go back to `O29` and fill the `Building block` column you left empty at 8.10.

Signed by. Chief Enterprise Architect for the shape; `GB-02 Investment Committee` for the money at `GT-0 G0 Charter`.

Done when. Every row of `O29` appears at least once in `R12` and `RPT07`'s `Work package` column has no blanks.

Shows up in. `RPT07 Gap Analysis`, whose last column is the join through `R12`, empty today because `R12` is empty.

Goes wrong when. `O30` has no `Owner`, no dependency column and nowhere to record why a package is cut the way it is. That argument is what nobody can reconstruct eighteen months later, and here it has to live in the Phase E deliverable — one paragraph per package, or the next architect re-cuts them from scratch.

Look it up. Volume 1, chapter 12, *The legacy question*.

Where it lands.

`O30 Work Package · R12 Gap closed by Work Package`

`O30` — `Ref | Work package | Domains | Cost | Start | End | Approval | Delivery`. `Approval` is `Approved`, `In flight` or `Not started`; `Delivery` is `Not started`, `In flight`, `Delivered` or `Past end date`. `R12` — `Gap | Gap name | Severity | Work package | Package name`, zero instances.

Step 8.14 — Define transition states you can actually stop at

Produce. `O53 Transition Architecture` with exit criteria written out, and `R15` complete.

You need. `O30` re-cut; `R15 Transition contains Work Pa`; `O52 Implementation Factor`.

How.

1. Read the three shipped states: `TA-1 Governed and supportable`, `TA-2 Integrable`, `TA-3 Business visible`. The sequence is right; the content is not finished.
2. Fix the exit criteria. The column holds "5 exit criteria", "6 exit criteria", "7 exit criteria" — a count is not something you can pass. Write them out, or say the cell is a pointer into the deliverable.
3. Account for the five packages in no state. `R15` carries seven instances covering `WP-01`, `02`, `04`, `05`, `03`, `07` and `11`, leaving `WP-06`, `WP-08`, `WP-09`, `WP-10` and `WP-12` — $29.9m — in no transition at all. Either they sit in a state or they are business-as-usual outside the programme.
4. Fix the overlap. On the shipped dates `TA-1` runs 2027-12 to 2030-05, `TA-2`'s single package 2028-07 to 2029-07, and `TA-3` starts 2028-03 — before either state it depends on has finished.

5. Make each state testable by a non-architect. `TA-1` exits when no component runs past vendor support, every data entity has a named individual owner and the repository is the record. `TA-2` exits when production traffic crosses managed integration and the ten direct database reads are gone.

Signed by. `GB-01 Architecture Board`; `GB-04 Executive Committee` is the audience for the overlap finding.

Done when. Every package in `030` appears in `R15` or has a written reason not to, and each state's exit criteria are testable statements rather than a count.

Shows up in. `RPT08 Roadmap`, by `Start` and `End`. The report has no transition column, so the states live in `053` and in your figure.

Goes wrong when. The three states are drawn as three equal thirds. They are not. `TA-1` is the longest and the least visible, and a sponsor who has not been told that loses patience at month fourteen.

Look it up. Volume 1, chapter 16, *The roadmap, and what is true at the end of each year*, and chapter 17, *Gaps, and the roadmap in two plateaus*.

Where it lands.

`053 Transition Architecture · R15 Transition contains Work Pa`

`053 — Ref | Transition architecture | Description | Work packages | Exit criteria`, three records, where `Work packages` is a comma list duplicating what `R15` holds properly. `R15 — Transition | Name | Work package | Package name`, seven instances against twelve packages.

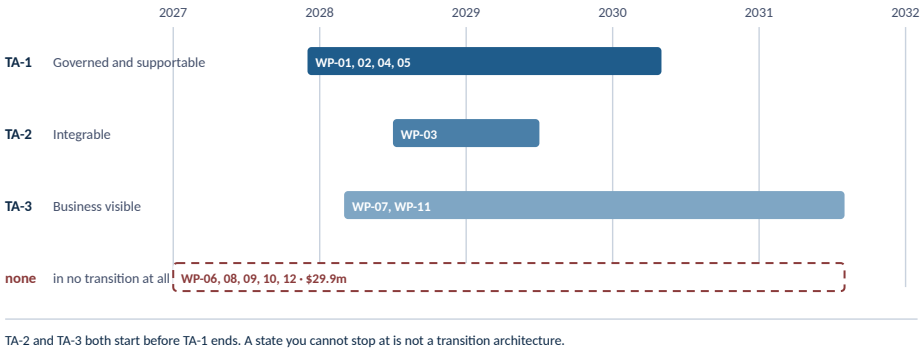

Figure 8.5 The three transition states on the dates `030` actually carries. `TA-2` and `TA-3` both start before `TA-1` finishes, and five packages worth $29.9m sit in no state at all.

BEFORE YOU TAKE THE ROADMAP ANYWHERE

Which transition state could the bank stop at and still be better off than today? Write it in one sentence, in the language your sponsor uses.

Which five work packages are in no state, and for each, is it in the programme or out of it?

What happens to the roadmap if the state before the one you are in slips a quarter?

Step 8.15 — Put three options to the board, not one plan

Produce. `054 Architecture Alternative` reduced to three live options with a reconciled cost.

You need. `054` as shipped; `030` 's costs; `052` ; volume 1 chapter 14.

How.

1. Read the five shipped alternatives: `ALT-A Do nothing beyond compliance` $24m Rejected; `ALT-B Foundations first` $46m Recommended; `ALT-C Business value first` $52m Rejected; `ALT-D Big bang replacement` $88m Rejected; `ALT-E Compliance led` $31m `Partly adopted` [5].
2. Deal with `Partly adopted` first. It is not a decision — it is the minute of a meeting that ran out of time. Adopted, rejected, or a component of the option you recommend: pick one.
3. Reconcile the money. `ALT-B` is $46m and `030` 's packages total $64.5m. Either the alternatives are drawn at a different scope or one number is wrong, and the board will find the $18.5m before you finish the slide.
4. Reduce to three: the one you recommend, the cheapest credible one, and the one the loudest executive wants. Three options is a decision; five is a survey.
5. Give each the same four things: cost, what the bank gets, what it gives up, and the date the first business-visible thing lands. The last is what the conversation turns on.
6. Keep `ALT-D` in the pack and reject it with evidence rather than opinion.

Signed by. Chief Enterprise Architect writes it; `GB-02 Investment Committee` chooses; `GB-04 Executive Committee` is told.

Done when. Three options, each with a cost that ties to `O30`, and no `Recommendation` value that is a shade of grey.

Shows up in. `O54`'s `Recommendation` column and `O30`'s `Approval` column, which should read `Not started` on every package that exists only in a rejected option.

Goes wrong when. One plan goes to the board. A board given one plan does not approve a plan; it approves or defers a request, and deferral is the default. Three options makes the meeting about which, not whether.

Look it up. Volume 1, chapter 14, *What a core replacement costs, and how long it takes*. Nedbank's Managed Evolution — a 2016 start, R11.7bn, the core delivered last at end-2024 — and Barko live on a Temenos software-as-a-service core in under six months are both real and not the same scope. Say which your `ALT-D` resembles.

Where it lands.

`O54 Architecture Alternative`

`Ref | Alternative | Description | Cost | Recommendation`, five records. No basis-of-cost column, no benefit column, no date column. Keep the option paper outside the repository and let `O54` be its index.

Option	Cost	What the bank gets	What it gives up
`ALT-B` Foundations first	$46m, reconciled	A governable, supportable, integrable estate by `TA-2`	Nothing a customer notices for the first two years
`ALT-E` Compliance led	$31m	Four control objectives moved off `No mechanism`	The integration layer, so every later change costs more
`ALT-C` Business value first	$52m	A visible channel change inside eighteen months	Builds on nineteen unmanaged interfaces and ten direct database reads

Part four — reading the result

Step 8.16 — Read `RPT07` and `RPT08`, then issue the Phase E pack

Produce. The three Phase E deliverables, and two reports that reconcile.

You need. Everything above; `O31 Deliverable`; `ZZ Checks` run a second time.

How.

1. Read `RPT07 Gap Analysis` three ways: rows with no `Work package`, rows past `Due`, and the severity spread across `Domain`. If every Critical gap sits in one domain, either that domain is in trouble or your severity scale is.

2. Read `RPT08 Roadmap`, then its footer, which says four work packages are past their end date — and check the footer against the dates, because it does not survive the check. `WP-01` ends 2030-05 and reads `Past end date`; `WP-08` ends 2031-08 and reads `Delivered`. Both the footer and `ZZ Checks` compute off `Delivery`, and step 9.12 fixes it properly.
3. Set `O31 Deliverable` for `DLV-13`, `DLV-14` and `DLV-15`. `DLV-13` reads `Approved` today with an empty gap sheet behind it; set it to `Draft` when you start and move it when the register is real.
4. Run `ZZ Checks` again and put before and after side by side. Phase E should change none of the estate counts, but it should move `Gaps overdue` off zero-because-empty.
5. Take the pack to `GB-01 Architecture Board` for the architecture and `GB-02 Investment Committee` for the money. `GB-02` meets quarterly, so the pack has a date and missing it costs three months.

Signed by. `GB-01 Architecture Board` approves `DLV-13` and `DLV-14`; `GB-02 Investment Committee` approves the envelope at `GT-0 G0 Charter`.

Done when. `RPT07` has no blank `Work package` cells, `RPT08`'s footer agrees with its own `End` column, and all three Phase E rows of `O31` carry a status you would defend in writing.

Shows up in. `RPT01 Dashboard`, which an executive reads and you do not control. Its `Gaps overdue` row gets read first.

Goes wrong when. The pack goes forward with `RPT08`'s footer intact. Four packages past their end date, in a repository whose earliest package starts in 2027, invites exactly the question you cannot answer.

Look it up. Volume 1, chapter 6, *The four numbers an executive watches*.

Where it lands.

> **`RPT07 Gap Analysis · RPT08 Roadmap · O31 Deliverable`**
>
> `RPT07` — seven columns over `O29` and `R12`, empty until this chapter. `RPT08` — seven columns over `O30` plus a computed footer. `O31` — `Ref | Deliverable | Phase | Status`, `DLV-13` to `DLV-15` in phase `E`.

FOR THE REVIEWER, BEFORE THIS PHASE IS SIGNED OFF

Take any three rows of `029 Gap`. For each, can you name the difference, the measurement or the volume-1 pattern it came from?

Take any two rows of `055 Building Block`. Does the baseline already meet the acceptance criterion? If so, what is the block for?

Which work package closes no gap, and why is it still in the plan?

What does the `UNGOVERNED CHANGE` count on `60 Summary` mean in a repository with no change log, and does the pack say so?

THE CHAPTER IN TEN LINES

- Phase E turns differences into a funded programme. The gap register sets the quality of everything after it.
- `O29 Gap` is empty, `R12` has zero instances, and `DLV-13 Consolidated Gaps` reads `Approved`. Twelve work packages worth $64.5m rest on nothing.
- This repository has no export sheets and no change log sheet. You build both extracts by hand, and the fifth paste target stays empty.
- With an empty change log, `UNGOVERNED CHANGE` means unexplained, not unauthorised. Say so in the pack.
- A design delta is deliberate and earns a work package. An ungoverned change is a governance defect and earns a change request against `O61`.
- A difference is not a finding until a named person says so on `50 Review`, with a rationale longer than four words.
- Every one of `O13`'s 120 rows has a maturity delta, and `Disposition` is just the delta restated. Re-derive it. `RPT03` shows level 1 only.
- The five capabilities a bank is usually missing are all shared, which makes them a funding problem dressed as an architecture problem.
- There is no `O51`, so no Solution Building Block sheet. The build-buy-configure decision needs a column you add or a paragraph you write.
- `RPT08`'s footer says four packages are past their end date while their `End` dates are in the future. Fix `Delivery` before anybody quotes it.

SOURCES FOR THIS CHAPTER

- **1.** Banking_TOGAF_Metamodel.xlsx, sheets O29 Gap, O30 Work Package, O53 Transition Architecture, O55 Building Block and O13 Business Capability
- **2.** Banking_TOGAF_Metamodel.xlsx, sheets R12 Gap closed by Work Package, R13 Work Package delivers Build and R15 Transition contains Work Pa
- **3.** Banking_Architecture_Comparison_Tool_TEMPLATE.xlsx, sheets 01 Config, 10 Input A Baseline, 11 Input B Target, 10R Records A, 11R Records B, 12 Input Change Log, 30 Diff Records, 32 Diff Attributes, 50 Review, 60 Summary and 70 Gap Candidates
- **4.** Banking_TOGAF_Metamodel.xlsx, sheets RPT07 Gap Analysis, RPT08 Roadmap, RPT03 Capability Heatmap, RPT09 Regulatory Coverage and RPT13 Orphan Analysis
- **5.** Banking_TOGAF_Metamodel.xlsx, sheet O54 Architecture Alternative
- **6.** Banking_TOGAF_Metamodel.xlsx, sheets O05 Principle, O06 Requirement, O11 Role, O57 Governance Body and O58 Governance Gate
- **7.** Banking_TOGAF_Metamodel.xlsx, sheets O31 Deliverable, O35 Business Building Block, O44 Data Building Block, O45 Application Building Block and O46 Technology Building Block
- **8.** Banking_TOGAF_Metamodel.xlsx, sheets ZZ Checks, 00 Index and RPT01 Dashboard
- **9.** SARB Prudential Authority — Directive D6/2025: returns to be submitted to the Prudential Authority from 1 July 2025 https://www.resbank.co.za/content/dam/sarb/publications/prudential-authority/pa-deposit-takers/banks-directives/2025/d6-2025/D6-2025%20-%20Directive%20returns%20to%20be%20submitted%20to%20the%20PA%20from%201%20July%202025.pdf
- **10.** The Open Group, TOGAF Standard, 10th Edition — ADM Phase E, Opportunities and Solutions https://pubs.opengroup.org/architecture/togaf10-doc/adm/

CHAPTER 9

Migration Planning

A roadmap can be wrong quietly. A plan cannot: the moment you write a date against a project with a named lead, somebody can check. In a bank, the people who will check it own the dates you have just collided with.

Phase F is where the roadmap acquires names, dates and money, and therefore becomes checkable [13]. This chapter is about making it checkable by you before it is checked by the people who own the dates you are about to collide with.

Ardent Union Bank arrives at Phase F better equipped than it arrived at Phase E. `056 Project` carries twelve projects with costs, dates and leads. `R14 Project delivers Work Packa` carries twelve instances. `052 Implementation Factor` carries eight factors. `062 Delivery Risk` carries eight risks with owners. `066 Architecture Increment` carries six increments with the month each lands [1][2][3].

Read them together and three things come apart. One project per package, exactly, twelve for twelve — nothing has been shaped, only transcribed. The increments land on months the projects delivering them do not support. And no sheet anywhere in the repository records a dependency between two projects or two packages. Ardent Union Bank is invented and its data is real within this workbook; Table Bay Bank is volume 1's bank and appears only as a pointer back.

> **BEFORE YOU START THIS PHASE**
>
> - **Volume 1, chapter 7, *The reporting and regulatory calendar*.** The dates you cannot move, and why they are monthly rather than annual.
> - **Volume 1, chapter 7, *The business day*.** Settlement windows, the night batch and the cut-off table that is not public.
> - **Volume 1, chapter 10, *Time: three dates, not two*.** Value date, posting date, system timestamp — the reason data migration in a bank is harder than it looks.
> - **Volume 1, chapter 14, *The cost lines that are not the licence*.** What a cost estimate is missing when it only has a licence in it.

Part one — from packages to projects

Step 9.1 — Shape the work packages into projects

Produce. `056 Project` with a lead against every row, and `R14` rebuilt to match.

You need. `030 Work Package` re-cut in step 8.13; `011 Role` for the leads; `052`; the delivery function's view of concurrent capacity.

How.

1. Read the shipped twelve. Each project carries one package, same name, same cost, same dates. `PRJ-03 Integration platform` delivers `WP-03 Integration platform` at $10.1m [1]. That is a transcription, not a plan.
2. Combine where combining removes a dependency rather than managing it. If `WP-11 Customer experience` cannot start until `WP-03 Integration platform` delivers, running them as one project with one lead removes a hand-off.
3. Split where a package has two different acceptance cadences — a regulatory-dated half and a discretionary half should not share a lead.
4. Set `Lead` from `O11 Role`. Then read the shipped leads, because they will not survive a glance: `PRJ-04 Technology currency` is led by the `Security Architect`, `PRJ-05 Access and security` by the `Internal Auditor`, and `PRJ-07 Core platform renewal` by the `Chief Executive` [1]. An internal auditor cannot lead a project they will later assure.
5. Rebuild `R14 Project delivers Work Packa` — `Project | Project name | Work package | Package name | Cost` — carrying `Cost` on the relationship row where a project splits a package's budget.
6. Write the argument for each shape into the Phase F deliverable, because `O56` has no column for it. One paragraph per project, naming what combining or splitting removed.

Signed by. Head of Delivery, `ROL-13`, for the shape; `GB-02 Investment Committee` for the money.

Done when. Every project has a lead who can be in the room, no lead sits in an assurance role, and every package appears in `R14`.

Shows up in. `RPT08 Roadmap` indirectly, through `O30`'s dates, which must now agree with `O56`'s.

Goes wrong when. Twelve projects stay twelve projects because the sheet already looked complete. One-to-one is the default, not an answer, and it hides every dependency inside a package boundary.

Look it up. Volume 1, chapter 5, *The architect's real counterparties*.

Where it lands.

`O56 Project · R14 Project delivers Work Packa`

`O56` — `Ref | Project | Work package | Cost | Start | End | Lead`. One `Work package` column, so a project holding several packages records the primary one here and the rest in `R14`. No column for why the project is cut this way.

WHY THE PROGRAMME IS CUT THIS WAY

Take your three largest projects. If you cannot answer the first question, the project is the wrong shape.

Why is this one project rather than two? Write the sentence you would say to a new executive.

What does combining or splitting it remove from the dependency graph?

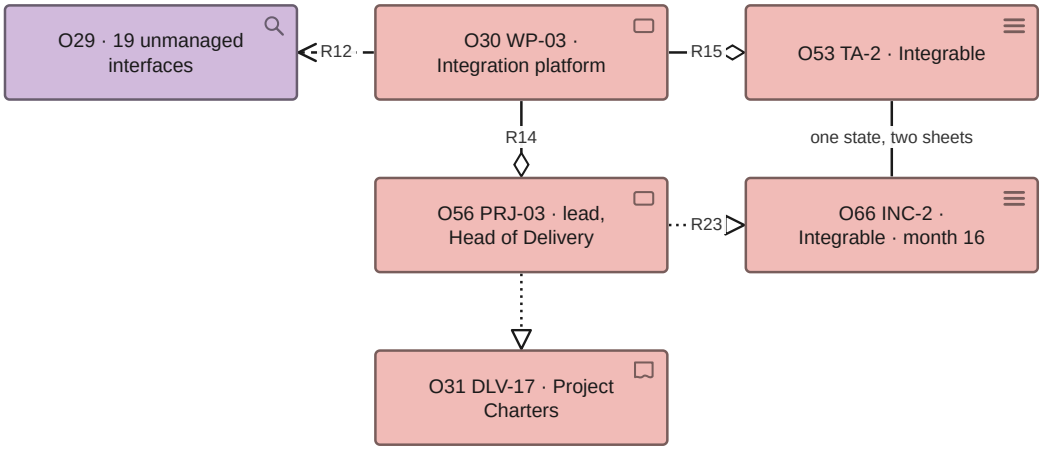

Figure 9.1 One chain through Phase F. A project and a work package are both the Work Package element, so the project aggregates the packages it carries. `TA-2` and `INC-2` are the same state recorded on two sheets, which is the finding on the right-hand edge.

Step 9.2 — Record the implementation factors, each with its deduction

Produce. `O52 Implementation Factor` complete — everything that will shape delivery, and what follows from each.

You need. Phase B to E findings; `O07 Constraint`; `O08 Assumption`; `O62 Delivery Risk`.

How.

1. Read the eight shipped factors. The categories are `Dependency`, `Constraint`, `Resource`, `Assumption` and `Risk`, and the `Deduction` column is the point of the sheet: `IF-01 Vendor selection not made` deducts "Blocks integration work until resolved" [2].
2. Rewrite the vague ones. `IF-03 Peak trading freeze` deducts "No change in the peak window" — which window, how long, whose rule? A deduction without a date is an observation. Step 9.9 gives you the dates.

3. Add the factors the sheet is missing, and there are at least three: no project-to-project dependency can be recorded anywhere in the repository; `O04 Measure` has no target column, so no benefit can be stated as a number; and `O30`'s `Delivery` column disagrees with its own `End` dates on four rows.
4. Make each `Deduction` an instruction. "Blocks mastering work" is a deduction. "Data owners not designated" on its own is a fact.
5. Cross-check against `O62 Delivery Risk`. `RSK-01 Integration product not selected` is `Certain` and `High` — a certainty is not a risk, it is an issue, and it belongs here as a factor with a deduction rather than on the risk sheet.

Signed by. Head of Delivery; Chief Enterprise Architect co-signs anything that changes the architecture rather than the plan.

Done when. Every row has a `Deduction` that tells somebody to do or not do something, and every `Certain` row of `O62` has a matching factor here.

Shows up in. The sequencing argument in the Implementation and Migration Plan. There is no report over `O52`, so if it is not in the plan document it is invisible.

Goes wrong when. The sheet collects observations. Eight rows of true statements with no deduction is a risk log with the consequences removed.

Look it up. Volume 1, chapter 7, *What the operating model means for architecture*.

Where it lands.

> **`O52 Implementation Factor`**
>
> `Ref | Implementation factor | Category | Deduction`. Four columns: no source, no owner. Name the source inside the factor text — "`ZZ Checks`: three components past vendor support" — or the next reader cannot check it.

Step 9.3 — Sequence the programme, and record the dependency the repository cannot hold

Produce. A sequence with a critical path, and a written record of every dependency that shaped it.

You need. `O56` with dates; `O52`; `O53 Transition Architecture`; `R15 Transition contains Work Pa`.

How.

1. Establish the fact first: **there is no dependency sheet.** Twenty-five relationship sheets, and not one relates a project to a project or a package to a package. `R14` links project to package, `R15` transition to package, `R23` increment to project — and nothing links a prerequisite to a dependant [2].

2. So record dependencies in two places and keep them in step: as `O52` factors with a `Dependency` category and a deduction naming both ends, and as a dependency table in the plan document. Do not invent a sheet in the shipped file without saying so in the Preliminary chapter's list of additions.
3. Derive the sequence from the transition states, not from the project list. `TA-1 Governed and supportable` has to exit before `TA-2 Integrable` can be claimed, and `TA-2` before `TA-3 Business visible`.
4. Read the shipped dates against that order. `WP-10 Compliance mechanisms` starts 2027-01, `WP-02` 2027-12, and then nine of twelve packages start in 2028 — a programme that begins almost everything in one year has not been sequenced.
5. Compute the critical path by hand across the longest chain and state its length in months. `WP-07 Core platform renewal` and `WP-08 Process automation` both run to 2031-08, fifty-six months from the earliest start.
6. Name the three packages that could start now with no prerequisite, and start those.

Signed by. Head of Delivery; `GB-01 Architecture Board` for the order of the transition states.

Done when. Every project has a predecessor named or is explicitly declared independent, and the critical path has a length in months somebody has written down.

Shows up in. `RPT08 Roadmap`, by `Start`. Sort it on `Start` and the clustering is visible in one glance.

Goes wrong when. The sequence is drawn from what the business wants first. `TA-3 Business visible` holds `WP-07` and `WP-11`, and on the shipped dates it starts in 2028-03, before the foundations it rests on exist.

Where it lands.

> **`O52 Implementation Factor · O53 Transition Architecture`**
>
> One `O52` row per dependency, `Category` `Dependency`, `Deduction` naming both ends and the consequence: "`PRJ-11` cannot start before `PRJ-03` delivers managed integration; eleven months of float consumed.

Part two — money, benefit, increments and risk

Step 9.4 — Put a basis under every cost, and fix the currency

Produce. Costs in `O30` and `O56` that reconcile to each other and to `O54`, in one currency.

You need. `O30`; `O56`; `O54 Architecture Alternative`; the finance business partner.

How.

1. Convert to rands. The `Cost` column of `O30 Work Package` and of `O56 Project` reads `$7.7m`, `$10.1m`, `$2.0m`, and `O54` reads `$24m` to `$88m`. State the rate and the date once, in the plan, and never let two currencies appear in one pack.
2. Reconcile the three totals. `O30`'s twelve packages sum to $64.5m. `O54`'s recommended `ALT-B Foundations first` reads $46m. An $18.5m difference that nobody has explained is the first question an investment committee asks.
3. Reconcile project to package. `R14` carries a `Cost` column and so do both object sheets, so the same number is recorded three times and can disagree in three places.
4. Write the basis outside the repository, because there is no basis column. Licence, implementation, integration, data migration, parallel running, decommissioning, internal effort, contingency — and say which of the eight each estimate contains.
5. Flag the estimates that are a single round number. `$2.0m` and `$10.1m` sitting next to each other in the same sheet are not the same kind of number.

Signed by. Chief Financial Officer, `ROL-02`; `GB-02 Investment Committee` accepts the total.

Done when. One currency, three totals that reconcile, and every cost carrying a named basis in the plan document.

Shows up in. `RPT08 Roadmap`, `Cost`. It is the column an executive adds up in their head while you are talking.

Goes wrong when. The estimate is the licence plus a multiplier. **Look it up.** Volume 1, chapter 14, *The cost lines that are not the licence*.

Where it lands.

> **`O30 Work Package · O56 Project · R14 Project delivers Work Packa`**
>
> `Cost` on `O30` is the package envelope, on `O56` the project budget, on `R14` the split where one project funds part of a package. Decide which is authoritative and derive the other two from it.

Step 9.5 — Make every benefit trace to a measure, or delete it

Produce. One benefit per project, each tied to a row of `O04 Measure`, with arithmetic.

You need. `O02 Goal`; `O03 Objective`; `O04 Measure`; `R10 Objective measured by Measu`; `R11 Goal realised by Objective`.

How.

1. Start from the objective, not from the project. `O03` carries fourteen objectives against six goals, each with a `Horizon` [6].
2. Find the measure. `R10 Objective measured by Measu` carries fourteen instances linking objective to measure, and `O04` carries sixteen measures with a `Baseline` and a `Cadence`.

3. Notice the hole, because it changes what you can promise: **`O04` has a `Baseline` column and no target column.** `MSR-08 Managed interfaces` has a baseline of 27 and nowhere to record what it should reach. `O03` has a `Horizon` and no quantity either. So the quantified benefit cannot live in this repository at all — it lives in the plan, and `O04` holds only the baseline it will be measured from.
4. Write the arithmetic. "`MSR-07 Components within vendor support` from 21% to 100% by 2029-05 removes three unsupported components and the operational risk capital held against them" is a benefit. "Improved currency" is not.
5. Delete any benefit that does not trace. A benefit with no measure behind it is a sentence somebody wrote to get a paper through a committee, and at `GT-3 G3 Benefit` it is the reason the project is recorded as not having delivered.
6. Check the cadence. `MSR-02 Manual regulatory adjustments` is `Quarterly`, so a benefit claimed at month three has no reading behind it.

Signed by. Each measure's owner for their own row; Chief Financial Officer for the total; `GB-02 Investment Committee` confirms at `GT-3`.

Done when. Every project has one benefit, one measure, one piece of arithmetic and one realisation date.

Shows up in. `RPT01 Dashboard` eventually, and `O60 Governance Decision` at `G3`.

Goes wrong when. `RSK-07 Benefit not measurable at G3` is already on the risk sheet, marked `Certain` and owned by the Chief Financial Officer [3]. Somebody has already told you this will happen. This step is how you make them wrong.

Look it up. Volume 1, chapter 6, *Cost-to-income and jaws*.

Where it lands.

> **`O04 Measure · O03 Objective`**
>
> `O04` — `Ref | Measure | Unit | Domain | Baseline | Cadence`, sixteen measures, no target column. `O03` — `Ref | Objective | Goal | Horizon | Status`, fourteen objectives, no quantity. The number you are promising lives in the plan, and the plan has to say so.

Form 9-A — benefit, traced to a measure

One row per project. An empty arithmetic or realisation cell means the benefit comes out of the project. The measure's owner signs its own row, the Chief Financial Officer accepts the total, `GB-02` confirms it at `GT-3 G3 Benefit`.

Project	Objective	Measure	Arithmetic	Realisation date
PRJ-03	OBJ-08	MSR-08	27% to 100% managed, 19 interfaces routed	2029-07

Step 9.6 — Write the architecture increments, and fix the ones that cannot be true

Produce. `O66 Architecture Increment` as the readable version of the roadmap, and `R23` rebuilt.

You need. `O56` with dates; `O53`; `O66`; `R23 Increment delivered by Proj`.

How.

1. Read the six shipped increments and what each adds: `INC-1 Governed and supportable` at month 10, `INC-2 Integrable` at 16, `INC-3 Compliant by mechanism` at 22, `INC-4 Business visible` at 30, `INC-5 Analytics and evidence` at 42, `INC-6 Core renewal` at 56 [1].
2. Read `R23` next, and read it properly. Six instances, allocated in order: `INC-6 Core renewal` is recorded as delivered by `PRJ-11 Customer experience`, while `PRJ-07 Core platform renewal` is recorded against `INC-4 Business visible` [2]. The links are mechanical and two of the six are plainly wrong.
3. Convert each `Lands` month into a date from the programme start and compare it with the end date of the project `R23` names. On the shipped data `INC-1` lands at month 10 and `PRJ-01` ends at month 40; `INC-5` lands at month 42 and `PRJ-09` ends at month 24.
4. Rebuild `R23` from the project side: for each increment, name every project that contributes, and set `Lands` to the month the last contributor finishes.
5. Write `What is added` as mechanisms that will exist, in the business's words. `INC-1` reads "Repository, ownership, currency, access" — which is honest and tells an executive that nothing a customer notices happens for ten months. Keep that honesty.
6. Reconcile against `O53`. `TA-1`, `TA-2` and `TA-3` carry the same three names as `INC-1`, `INC-2` and `INC-4`. The same state is recorded on two sheets; decide which is authoritative and

say so.

Signed by. Chief Enterprise Architect; `GB-04 Executive Committee` is the audience that most needs this sheet.

Done when. No increment lands before the last project delivering it ends, and `R23` names every contributing project.

Shows up in. Nothing computes over `O66`. It is the sheet you put on a slide, which is exactly why its dates have to be right.

Goes wrong when. Increments are drawn one per project. They are not the same thing: an increment is what becomes true, and several projects usually have to land before anything does.

Look it up. Volume 1, chapter 16, *The roadmap, and what is true at the end of each year*.

Where it lands.

`O66 Architecture Increment · R23 Increment delivered by Proj`

`O66` — `Ref | Increment | Lands | What is added`. Four columns: no note, no conditionality, no benefit. Where an increment depends on a vendor selection or a budget, say so inside `What is added`. `R23` — `Increment | Increment name | Lands | Project | Project name`, carrying `Lands` on the relationship too.

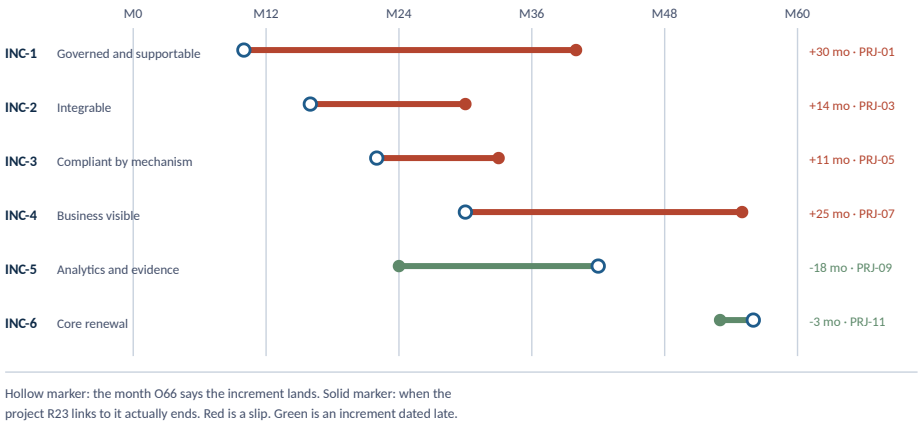

Figure 9.2 The six increments against the projects `R23` says deliver them. Four of the six land months before the project that delivers them ends, and `INC-5` is dated eighteen months after its project finishes. The links are mechanical; rebuild them.

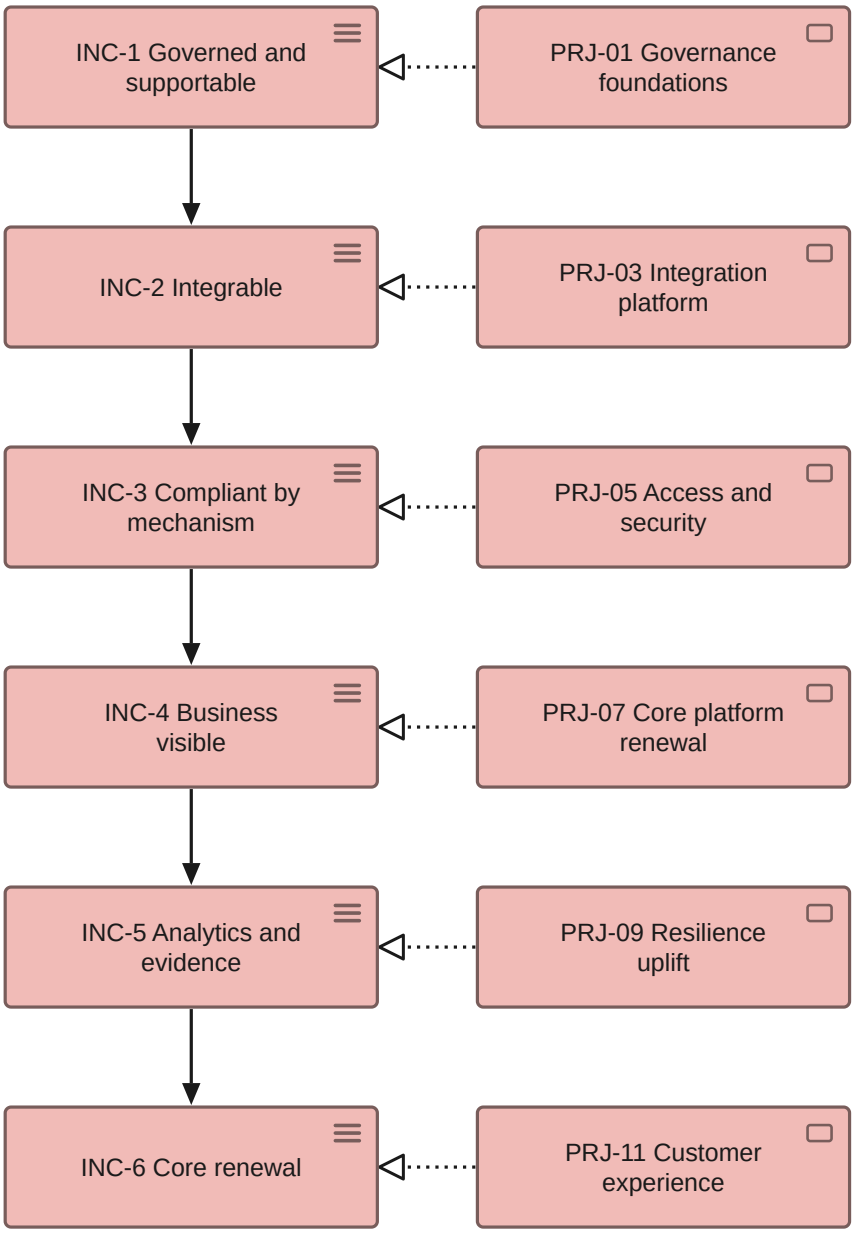

Figure 9.3 `R23` as the repository ships it: six increments as plateaus, each triggering the next, each realised by exactly one project. Two of the six pairings are wrong — `INC-6 Core renewal` against `PRJ-11 Customer experience`, and `PRJ-07 Core platform renewal` against `INC-4`.

Step 9.7 — Record delivery risk with a consequence, not an adjective

Produce. `062 Delivery Risk` where every row says what happens if it lands.

You need. `062`; `052`; `007 Constraint`; `008 Assumption`.

How.

1. Read the eight shipped risks and their `Likelihood` values: three read `Certain` — `RSK-01 Integration product not selected`, `RSK-07 Benefit not measurable at G3` and `RSK-`

`08 Unmanaged interfaces proliferate` [3].

2. Move the three `Certain` rows off this sheet. A certainty is an issue. It belongs in `052` with a deduction, or in the plan as a decision already needed.
3. For every remaining row, write the consequence into the `Risk` text, because the sheet has no consequence column: "`RSK-04 Legacy platform failure before renewal` — `Possible`, `Critical`; a core outage inside the renewal window stops the programme and the bank's month-end."
4. Check the owner is the person who can act. `RSK-02 Data owners not designated in time` is owned by the Chief Data Officer, which is right; `RSK-03 Delivery capacity below plan` by the Head of Delivery, also right.
5. Set a trigger date for each — the date by which the risk must be closed or it becomes an issue. There is no column, so it goes in the plan's risk table, and the plan is reviewed monthly against it.

Signed by. Each row's `Owner`; Head of Delivery for the sheet.

Done when. No row reads `Certain`, and every row names what breaks and when you will know.

Shows up in. `ZZ Checks` has no risk row, so nothing computes here either. The monthly delivery report is where this sheet earns its place.

Goes wrong when. Likelihood and impact are adjectives and the row stops there. `Possible`, `Critical` tells a reader nothing about what to do on Monday.

Look it up. Volume 1, chapter 14, *The two fully documented failures, and what they actually teach*.

Where it lands.

> **`062 Delivery Risk`**
>
> `Ref | Risk | Likelihood | Impact | Owner`. `Likelihood` runs `Unlikely` to `Certain`, `Impact` `Medium` to `Critical`. No mitigation, no trigger date, no consequence column — three things that live in the plan.

Step 9.8 — Tie constraints and assumptions to what they hold up

Produce. `007 Constraint` and `008 Assumption` each pointing at the project it would invalidate.

You need. `007`; `008`; `056`; `052`.

How.

1. Read the eight constraints. Several are already sharp: `CON-05 Data residency` — certain data may not leave the jurisdiction; `CON-06 Change freeze windows` — peak periods are

closed to change; `CON-08 Union and works council consultation` — operational change requires consultation [4].

2. Read the six assumptions and their `Status`. Two read `At risk`: `ASM-03 Vendor selections complete before the dependent phase` and `ASM-04 Delivery capacity can be staffed to plan` [4]. An assumption at risk that holds up a project on the critical path is the most important line in the plan.
3. For each constraint and assumption, name the projects it binds. Neither sheet has a column for it, so the link is an `O52` factor and a column in the plan's assumption register.
4. Give every assumption a test and a date: who confirms it, by when, and what the plan does if the answer is no.
5. Deal with `CON-02 Delivery capacity unrecorded` directly. A plan that assumes capacity nobody has measured is a plan with an unmeasured denominator, and `IF-06 Concurrent capacity unknown` already says so.

Signed by. Head of Delivery; the Chief Compliance Officer for `CON-05` and the regulatory constraints.

Done when. Every `At risk` assumption has a test, an owner and a date, and every constraint names at least one project.

Shows up in. `ZZ Checks` has no row for assumptions, so again: the plan document or nowhere.

Goes wrong when. `CON-04 Regulatory change in flight` stays at that level of generality. **Look it up.** Volume 1, chapter 15, *The Joint Standards, with the numbering got right* — four instruments, three of them numbered "1", and getting the number wrong in a plan is the kind of error a compliance reviewer remembers.

Where it lands.

> **`O07 Constraint · O08 Assumption`**
>
> `O07` — `Ref | Constraint | Description | Type`, with `Type` running `Financial`, `Resource`, `External`, `Regulatory`, `Operational`, `Organisational`. `O08` — `Ref | Assumption | Status`, three columns. No owner, no test, no date: all three belong in the plan.

Part three — the bank's year, and the data

Step 9.9 — Fit the plan to the bank's calendar, not the programme's

Produce. A set of closed windows, each recorded as an `O52` factor with a date rule.

You need. Volume 1 chapter 7; the regulatory reporting team; payments operations; the card team; finance.

How.

1. **The monthly return cycle.** `BA 900`, the balance sheet of deposit-taking institutions, is due within fifteen business days after the month-end it relates to, and the BA suite is determined by directive — currently `D6/2025`, effective 1 July 2025 — with a signed certification form behind each submission [9]. Deduction: nothing touching the ledger, the core or the reporting chain cuts over in the first fifteen business days of a month.
2. **The deposit-insurance submission.** The Corporation for Deposit Insurance requires a monthly file within ten working days of month-end, with a chief executive and chief financial officer declaration within three working days of submission [10]. Deduction: the per-depositor aggregation must run correctly on both the old and the new product structure through any cut-over, not just after it.
3. **The financial close.** A release that lands mid-close must be reversible without posting into a closed period. Posting time is not mutable history, so "we will back it out" is not available.
4. **The payment-scheme windows.** Domestic settlement runs around the clock with a dedicated night window for clearing-house batches, and the authoritative submission and cut-off schedule is not public — it lives in PayInc's product and service manuals, supplied under agreement [11]. Deduction: you cannot cut over a rail when PayInc has a freeze, and you cannot find the freeze calendar on a website. Get it from the bank's payments operations team and put the real dates in the plan.
5. **Card-scheme mandate releases.** Twice a year, non-negotiable, and they consume the card change capacity for the window around them. Deduction: no discretionary card work inside a release window, and the mandate assessment is an input to the plan rather than a surprise.
6. **The quarterly and annual dates.** Regulatory capital and risk-weighted assets, the liquidity coverage and net stable funding ratios, large exposures and Pillar 3, with the remaining Basel III post-crisis reforms in force from 1 July 2025 [12]. Annually, audited statements and the resubmission of annual returns once finalised. Deduction: treasury and risk-engine changes do not land in a quarter-end month.

Signed by. Chief Financial Officer and Chief Compliance Officer jointly; Head of Delivery accepts the resulting dates.

Done when. Every closed window is an `O52` row with a `Deduction` that names months or days, and no project start or cut-over in `O56` falls inside one.

Shows up in. `O56 Project` and `O30 Work Package` dates, and in the count of dates you had to move, which is the number worth quoting.

Goes wrong when. The plan treats the calendar as a risk. It is not a risk. It is a set of dates other people own, published in advance, and a plan that collides with them was not checked.

Look it up. Volume 1, chapter 7, *The reporting and regulatory calendar*, and *The business day*. For the card cycle, chapter 12, *7–9. Cards, end to end*.

Where it lands.

052 Implementation Factor

One row per window. `Category` `Constraint` . `Deduction` carries the rule: "No core or ledger release in business days 1 to 15 of any month." `IF-03 Peak trading freeze` and `IF-04 Regulatory deadline` are the two shipped rows this step replaces with something actionable.

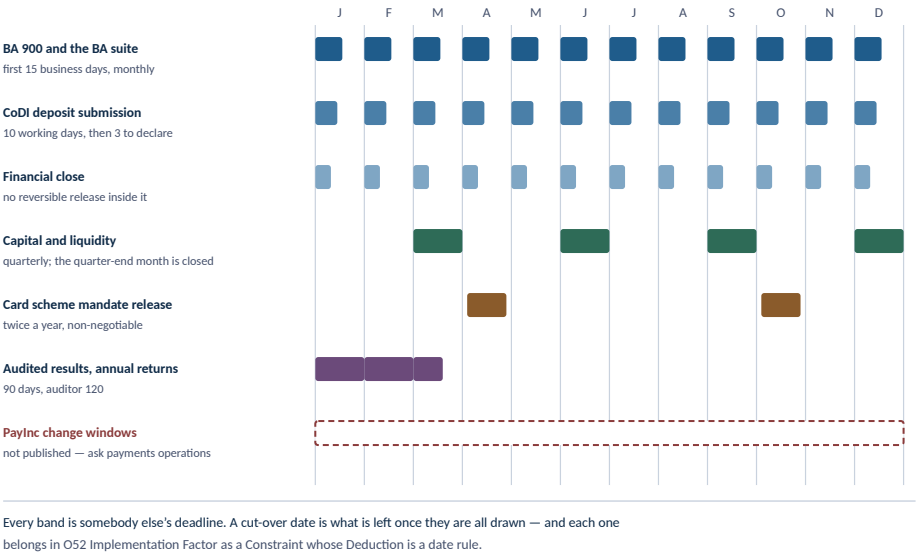

Figure 9.4 A bank's year from the point of view of somebody who needs a cut-over date. Every band is a deadline somebody else owns, and the one that matters most is the dashed band at the foot, because its dates are not published.

THE WINDOWS, BEFORE YOU COMMIT A DATE

Which closed window did you not know about until somebody told you in a meeting? Who owns it?

How many project start or cut-over dates in `056` fall inside a window you have now recorded?

Where will the PayInc change calendar live, and who re-reads it each quarter?

Step 9.10 — Plan each data migration as its own piece of work

Produce. `039 Data Migration` rewritten, and a migration plan per entity that `039` indexes.

You need. `039`; `020 Data Entity`; `R05 Application manages Data En`; `049 Resilience Tier`; the data owners.

How.

1. Read the six shipped rows and do not use them. `MIG-01` moves `Party` from `Payment Hub` to `Loan Origination`; `MIG-03` moves `Account` from `Loan Origination` to `CRM` [7]. Neither is credible, and both contradict `O20`'s own `Master system` column, which masters `E01 Party` in `Collections & Recovery` and `E03 Account` in `Payment Hub`.
2. Rebuild `From system` from `O20`'s `Master system` column and from `R05`, not from the migration sheet. If the two disagree, the data architecture is wrong and Phase C has to be reopened before a date is set.
3. Work out what `O39` cannot hold, because it is most of what matters: record counts, history depth, the as-at date, the reconciliation rule, the cut-over window, the rollback point and the dual-running period. Five columns, none of them these. So each migration is a plan in its own right and `O39` is the index. Say that in the deliverable.
4. Set the history depth per entity, and set it from the obligation rather than from the system. The initial-recognition credit state of an exposure has to survive the life of the exposure, because deterioration is measured against origination — twenty years for a mortgage, on a field most core systems overwrite. Arrears state and days-past-due are a history, not a status. Transaction history carries three dates and losing one of them makes interest irreproducible.
5. Write the reconciliation rule before the mapping. Count, sum, and a balance that ties to the general ledger at a stated as-at date. A migration that reconciles on row count has not reconciled.
6. Plan the dual run. Both systems computing the same balance on the same day, differences explained rather than tolerated, for at least one full month-end and one full return cycle.

Signed by. Chief Data Officer, `ROL-05`, and the entity's `Owner` from `O20`; Chief Financial Officer for anything that reaches the ledger.

Done when. Every row of `O39` names a credible source and target, and each has a migration plan behind it with a reconciliation rule and a rollback point.

Shows up in. `RPT05 Data Ownership`, which carries `Master system` and `Lineage` and will contradict `O39` until this step is done.

Goes wrong when. The migration is planned as a data movement and not as a history reconstruction. In a bank that makes it the critical path, every time: the account exists, the balance is right, and the arrears history that proves the bank treated the customer correctly is gone.

Look it up. Volume 1, chapter 10, *Time: three dates, not two*, and *Why history must be reconstructible*. And chapter 9, *Transaction and Position*.

Where it lands.

O39 Data Migration

`Ref | Entity | From system | To system | Status`. Five columns and nothing for volume, depth, reconciliation, window or rollback. Treat it as the index to six plans, numbered so the index points somewhere.

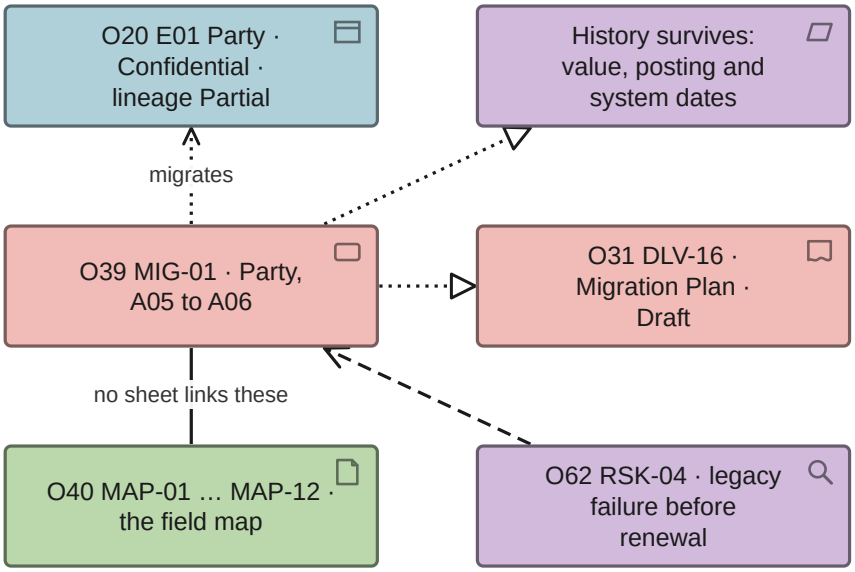

Figure 9.5 One migration, end to end. The requirement on the right is the one a bank adds and an insurer does not: three dates per entry, all of which have to survive. Nothing in the repository relates `040` to `039`, so the mapping and the migration are joined only by the entity name.

Form 9-B — the migration index, one row per entity

Columns are `039 Data Migration`'s own, in sheet order, with the plan reference you assign. Depth, reconciliation rule, cut-over window and rollback point live in the plan, not here — the form's last column is what points at them. Six to ten rows. Signed by the Chief Data Officer and the entity owner from `020`.

Ref	Entity	From system	To system	Status	Plan ref
MIG-01	Party	Collections & Recovery		Planned	DMP-01

Step 9.11 — Map source to target, field by field, with the rule written out

Produce. `040 Source To Target Mapping` with real field names and rules that can be executed.

You need. `039` rebuilt; `037 Data Attribute`; `043 Master Data Rule`; `020`'s `Classification` column.

How.

1. Read the twelve shipped rows. `Source field` and `Target field` read "Payment Hub field" and "Card Management field" — placeholders, not fields [7]. Replace them from `037 Data Attribute`, which carries sixty attributes with a `Type`, a `Key` flag and a `Classification`.
2. Extend the rule vocabulary. `Rule` runs `Direct`, `Lookup` and `Transform`, and three values are not enough: you also need `Derive`, `Default`, `Carry unchanged` and `Reject`. A row with no rule for what happens to a value that fails validation has no answer on cut-over night.
3. Write the expression. A `Transform` with no expression behind it is not a mapping. Either the expression goes in `Rule` or it lives in the mapping document with a reference here.
4. Check classification on every row. `E01 Party` and `E10 Limit` are `Confidential`, and `E04 Product`, `E06 Transaction`, `E09 Collateral` and `E12 General Ledger Entry` are `Restricted` [8]. A mapping that moves a `Confidential` field into a target with weaker

control is a security finding, not a data finding, and `PRN-14 Classification Travels with Data` is the principle it offends.

5. Reconcile against `043 Master Data Rule`, which names a `Golden source` per entity. If the migration target is not the golden source, one of the two is wrong.

Signed by. Domain Architect, Data, `ROL-10`; the Security Architect, `ROL-17`, for any row touching `Confidential` or `Restricted` data.

Done when. No placeholder field names remain, every `Transform` has an expression or a reference, and every row's classification has been checked against the target's controls.

Shows up in. `RPT05 Data Ownership`, and `048 Environment`, whose `Personal data present` column is what decides whether a test migration is allowed at all.

Goes wrong when. The mapping is built from the target system's data dictionary. It has to be built from the source, because what you cannot find in the source is the gap.

Look it up. Volume 1, chapter 10, *Classification, consent and POPIA*.

Where it lands.

> **`040 Source To Target Mapping`**
>
> `Ref | Entity | Source field | Target field | Rule`, twelve rows of placeholders. Keep one row per field, not one per entity: twelve rows for fourteen entities means nothing has been mapped.

Step 9.12 — Reconcile `Delivery` against `End` before anybody quotes the roadmap

Produce. `030`'s `Approval` and `Delivery` columns consistent with its own dates.

You need. `030`; `RPT08 Roadmap`; `ZZ Checks`; today's date.

How.

1. Read `RPT08`'s footer: four work packages are past their end date. Then read the `End` dates behind it. `WP-01` ends 2030-05 and `Delivery` reads `Past end date`. `WP-08` ends 2031-08 and `Delivery` reads `Delivered`. `WP-09` ends 2029-01 and reads `Past end date` [1].
2. Decide which column is authoritative. `Start` and `End` are dates and can be checked; `Delivery` is an assertion. Dates win.
3. Reset `Delivery` from the dates and from the delivery function's actual position: `Not started`, `In flight`, `Delivered` or `Past end date`, with `Past end date` reserved for a package whose `End` is genuinely behind today.
4. Reset `Approval` the same way. Five packages read `In flight` and three read `Not started`, and `WP-03` reads `Not started` on `Approval` while carrying the largest budget in the programme at $10.1m.
5. Re-run `ZZ Checks`. `Work packages past end date` should fall to the number you can defend, and if it falls to zero say so, because a zero that follows a four needs an explanation

in the same sentence.

Signed by. Head of Delivery; Chief Enterprise Architect for the repository.

Done when. `RPT08`'s footer agrees with its own `End` column, and `ZZ Checks` and `RPT01 Dashboard` report the same number.

Shows up in. `RPT08 Roadmap`'s footer, `ZZ Checks` and `RPT01 Dashboard` — three places computing off one column, which is why one wrong column is three wrong reports.

Goes wrong when. Nobody checks, the pack goes out, and an executive adds up that a programme starting in 2027 already has four packages past their end date. You will not get the second question.

Where it lands.

> **`030 Work Package`**
>
> `Approval` and `Delivery` are the two assertion columns; everything else on the sheet is a fact that can be checked. Treat them as owned by the Head of Delivery and updated monthly, and write that down, because nothing in the repository enforces it.

THE TWO COLUMNS NOBODY OWNS

Who updates `Approval` and `Delivery` on `030`, and on what day of the month?

Three reports compute off `Delivery`. Which of the three will an executive see first?

Step 9.13 — Assemble the plan and take it through the funding gate

Produce. `DLV-16 Migration Plan` and `DLV-17 Project Charters`, through `GT-0 G0 Charter`.

You need. Everything above; `031 Deliverable`; `033 Architecture Contract`; `057 Governance Body`; `058 Governance Gate`.

How.

1. Assemble the plan: projects with leads and dates, the dependency table the repository cannot hold, implementation factors with deductions, costs with a basis, benefits with arithmetic, increments, risks, assumptions with tests, the closed-window calendar, and the migration plans.

2. Write the charter per project against what `GT-0 G0 Charter` tests: scope, acceptance criteria and a contract exist before work starts [5].
3. Create the `O33 Architecture Contract` row per project — `Ref | Architecture contract | Work package | Compliance status` — because the gate tests that a contract exists, and Phase G tests against it.
4. Set `O31 Deliverable`: `DLV-16 Migration Plan` and `DLV-17 Project Charters`, both `Draft` today.
5. Take the funding to `GB-02 Investment Committee`, which holds `G0` funding and `G3` benefit confirmation and meets quarterly [5]. Quarterly means the pack is due on a date; missing it costs three months of a programme whose critical path is already fifty-six.
6. Take the architecture to `GB-01 Architecture Board`, monthly, which holds `G1` and `G2` and approves dispensations with an expiry. Phase G starts the day after.

Signed by. `GB-02 Investment Committee` for funding at `GT-0`; `GB-01 Architecture Board` for the architecture content; Head of Delivery owns the plan thereafter.

Done when. Every project has a charter and a contract row, `O31` carries both Phase F deliverables at the status they have actually reached, and a dated `G0` decision exists in `O60 Governance Decision`.

Shows up in. `O60 Governance Decision` and `O33 Architecture Contract`. A funded programme with no `O60` row is a programme somebody will later claim was never approved.

Goes wrong when. The plan is taken to the board without the dependency table, because the repository has no sheet for it and it therefore got left out. That table is the plan. Everything else is schedule.

Look it up. Volume 1, chapter 15, *The BA returns, and the change that moved them out of the law* — the return set can now change in writing, which is exactly the kind of thing a four-year plan has to be able to absorb.

Where it lands.

> **`O31 Deliverable` · `O33 Architecture Contract` · `O58 Governance Gate`**
>
> `O31` — `DLV-16 Migration Plan` and `DLV-17 Project Charters`, phase `F`. `O33` — `Ref | Architecture contract | Work package | Compliance status`, twelve rows keyed on package rather than project. `O58`'s `GT-0 G0 Charter` tests that scope, acceptance criteria and a contract exist before work starts.

FOR THE REVIEWER, BEFORE THE PLAN IS FUNDED

Take the longest project. Name its predecessor, and say where that dependency is recorded now that no sheet holds it.

Take any benefit in the plan. Which `004 Measure` is it against, what is the baseline, and who reads the measure?

Which closed window does the plan collide with, and what did you move?

For the migration on the critical path — what is the reconciliation rule, and what is the rollback point?

THE CHAPTER IN TEN LINES

- Phase F charters the work. One project per work package is a transcription, not a plan — combine where it removes a dependency, split where acceptance cadences differ.
- No sheet in this repository records a dependency between two projects or two packages. Keep it in `O52` with a deduction and in the plan, and say you have done so.
- `O52 Implementation Factor` is where the ideal sequence meets the real one. A factor with no `Deduction` is an observation.
- `O04 Measure` has a `Baseline` and no target column, so the quantified benefit lives in the plan. A benefit with no measure behind it is fiction, and `RSK-07` already predicts it.
- Costs are in dollars across `O30`, `O56` and `O54`, and `ALT-B`'s $46m does not reconcile with $64.5m of packages.
- `R23` pairs increments to projects mechanically: `INC-6 Core renewal` is delivered by `PRJ-11 Customer experience`. Four of six increments land before their project ends.
- Three of eight rows in `O62 Delivery Risk` read `Certain`. A certainty is an issue, not a risk.
- The regulatory calendar is not a risk. BA returns inside fifteen business days, the deposit-insurance file inside ten working days, quarter-end capital, card mandate releases twice a year, and PayInc windows that are not published.
- Data migration is the critical path in a bank because history has to survive: three dates per entry, initial-recognition credit state for the life of the exposure, and arrears as a history rather than a status.
- `O30`'s `Delivery` column disagrees with its own `End` dates on four rows, and three reports compute off it.

SOURCES FOR THIS CHAPTER

- **1.** Banking_TOGAF_Metamodel.xlsx, sheets O56 Project, O30 Work Package and O66 Architecture Increment
- **2.** Banking_TOGAF_Metamodel.xlsx, sheets O52 Implementation Factor, R14 Project delivers Work Packa, R15 Transition contains Work Pa and R23 Increment delivered by Proj
- **3.** Banking_TOGAF_Metamodel.xlsx, sheet O62 Delivery Risk
- **4.** Banking_TOGAF_Metamodel.xlsx, sheets O07 Constraint and O08 Assumption
- **5.** Banking_TOGAF_Metamodel.xlsx, sheets O57 Governance Body, O58 Governance Gate, O31 Deliverable, O33 Architecture Contract and O60 Governance Decision
- **6.** Banking_TOGAF_Metamodel.xlsx, sheets O02 Goal, O03 Objective, O04 Measure, R10 Objective measured by Measu and R11 Goal realised by Objective
- **7.** Banking_TOGAF_Metamodel.xlsx, sheets O39 Data Migration, O40 Source To Target Mapping, O37 Data Attribute and O43 Master Data Rule
- **8.** Banking_TOGAF_Metamodel.xlsx, sheets O20 Data Entity, R05 Application manages Data En and RPT05 Data Ownership
- **9.** SARB Prudential Authority — Directive D6/2025, returns to be submitted to the Prudential Authority from 1 July 2025; and Annexure 2, requirements for the new BA 900 https://www.resbank.co.za/content/dam/sarb/publications/prudential-authority/pa-deposit-takers/banks-directives/2025/d6-2025/D6-2025%20-%20Directive%20returns%20to%20be%20submitted%20to%20the%20PA%20from%201%20July%202025.pdf
- **10.** SARB / CODI — Corporation for Deposit Insurance Member Bank Data Handbook, and the single customer view calculations document https://www.resbank.co.za/content/dam/sarb/what-we-do/deposit-insurance/CODI%20-%20Member%20Bank%20Data%20Handbook.pdf
- **11.** SARB — SAMOS PFMIs Public Disclosure 2024 (operating windows, night window, intraday liquidity); BankservAfrica — Key Rules and Procedures (submission times held in the product and service manuals) https://www.resbank.co.za/content/dam/sarb/what-we-do/payments-and-settlements/domestic-settlement-services-samos/SAMOS%20PFMIs%20Public%20Disclosure%202024.pdf
- **12.** SARB Prudential Authority — Annual Report 2025/26, policy and regulatory initiatives (Basel III post-crisis reforms in force 1 July 2025) https://www.resbank.co.za/content/dam/sarb/publications/reports/pa-annual-reports/2026/chapters/governance-and-strategy/policy-regulate-25-26.pdf
- **13.** The Open Group, TOGAF Standard, 10th Edition — ADM Phase F, Migration Planning https://pubs.opengroup.org/architecture/togaf10-doc/adm/

CHAPTER 10

Implementation Governance

The phase where the architecture either refuses something, or admits that it cannot.

BEFORE YOU START THIS PHASE

Phase G is where the supervisor and the architecture practice meet. Volume 1 carries the obligations; this chapter carries the gates that produce the evidence, and re-teaches no instrument.

- **Chapter 15, *Regulation, risk and what the regulator inspects*** — *What inspection actually looks like*; *The Joint Standards, with the numbering got right*; *King IV, and the board's version of the same question*.
- **Chapter 5, *Who does what*** — *Internal audit, and the control functions the law requires*: the three functions the gates must connect to.
- **Chapter 16, *Modelling banking in ArchiMate*** — *A regulation, from driver to control*: the chain a gate walks when it asks where a criterion came from.

Ardent Union Bank is an invention. Its data is real within this workbook and nowhere else.

What you are building in this phase

Step	Produce	Lands in
10.1	The change-control spine, before any gate	Four new sheets, five new columns
10.2	Four gates, each with a holder and an evidence list	`058 Governance Gate`
10.3	A contract per package in delivery	`033 Architecture Contract`
10.4	A charter that `G0` can refuse	`030 Work Package`, `055 Building Block`
10.5	A design verdict before build	`059 Compliance Assessment` at `G1`
10.6	One verdict per acceptance criterion	`059`, `R24 Compliance Assessment tests`
10.7	A compliance status derived, not asserted	`033 Compliance status`
10.8	The two control reports read together	`RPT09 Regulatory Coverage`, `RPT11 Security Position`
10.9	A decision-rights boundary that holds	`057 Governance Body`
10.10	A dispensation that expires on a date	`061 Governance Change Request`, `060 Governance Decision`
10.11	A `G3` that tests benefit, not delivery	`030`, `004 Measure`
10.12	The board's duty mapped to sheets and columns	`057`, `060`, fifteen named sheets
10.13	A briefing with the Joint Standard numbering right	`063 Control Objective`, `001 Driver`
10.14	A gate calendar that runs monthly	`057 Cadence`, `059`

> **TWO CONVENTIONS FOR THIS CHAPTER AND THE NEXT**
>
> **A column marked (add) does not exist in the shipped file.** Step 10.1 carries the specification. Do not pretend a column is there, and do not fill one that is not.
>
> **The gates exist as rows and have never been held.** `058` carries four records; `059 Compliance Assessment` and `R24 Compliance Assessment tests` carry **zero** [3]. This chapter describes what you are building, not what you inherited.

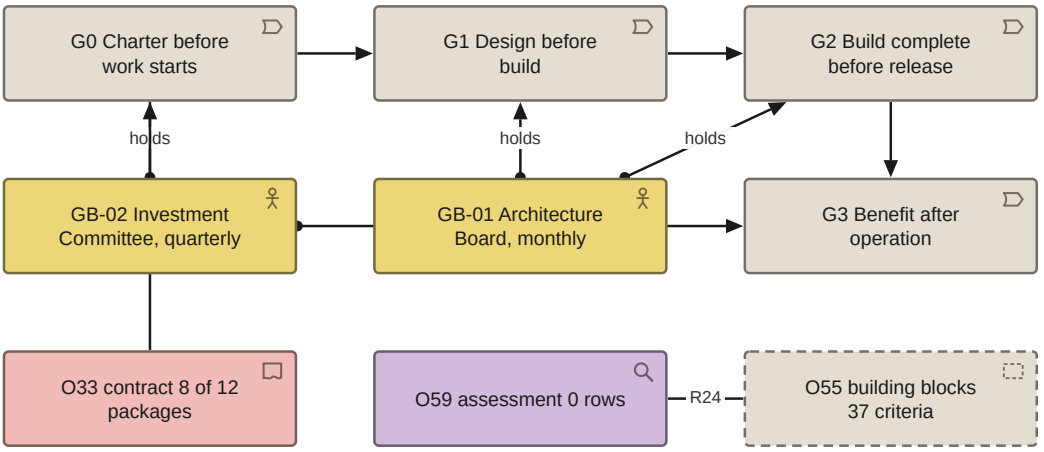

Figure 10.1 The four gates in `058 Governance Gate` as events in the delivery sequence, with the body in `057` assigned to each. The contract, the criteria and the assessment are what each gate reads. `059 Compliance Assessment` is drawn with its real row count: zero. The assignments are the `Held by` column step 10.2 adds — until it exists, no gate has an owner the model can resolve.

The spine this phase stands on

Step 10.1 — Add the change-control spine, or stop here

Produce. Four new sheets and five new columns, in place before a single gate is held.

You need. The workbook and an hour. Chapter 1 told you to do this before loading a record. If you did, check it against the specification below. If you did not, this is where the omission turns expensive: a gate verdict that cannot be dated or attributed is not a verdict.

Where it lands. Four new sheets, positioned before `00 Index`:

- `01 Version Control` — one row per published version. `Version`, `Date`, `Author`, `Approved by`, `Scope of change`, `Change refs`, `Baseline ref`.
- `02 Change Log` — one row per field that moved. `Log ref`, `Date`, `Sheet`, `Record ref`, `Field`, `Old value`, `New value`, `Change ref`, `Changed by`, `Reason`.
- `03 Change Requests` — one row per request. `Change ref`, `Raised`, `Raised by`, `Against`, `Type`, `Description`, `Classification`, `Impact`, `Decision body`, `Decided`, `Status`, `Expires`, `Decision ref`.
- `04 Baselines` — one row per baseline cut. `Baseline ref`, `Date`, `Label`, `Scope`, `Record count`, `Approved by`, `Decision ref`, `Supersedes`.

Then **five columns on every object sheet**, `O01` to `O66`, appended after each sheet's last column so the `Ref`-first shape is undisturbed: `Version`, `Record Status`, `Changed By`, `Change Ref`, `Approved By`. `Record Status` takes `Proposed`, `Approved`, `Superseded` or `Retired`; `Change Ref` resolves to `03 Change Requests`. `In Repository Since` and `Last Changed` are optional — both derive from `02 Change Log` and neither is worth maintaining by hand.

How.

1. Insert the four sheets using this file's own header convention: title row 1, subtitle row 2, headers row 4, data from row 5. A new sheet that breaks it breaks anything built over the workbook.
2. Append the five columns to all 65 object sheets in one pass. `O34` and `O51` do not exist — a loop from `O01` to `O66` fails on the two holes.
3. Back-fill `Record Status` for all 878 existing records with `Proposed`. Nothing here has been through a gate, so nothing is `Approved`.
4. Create version `0.1`, `Scope of change` reading *as delivered, illustrative data, nothing approved* — `ZZ Checks` agrees, with *Records validated against a real organisation* reading `0` [9]. Raise the whole operation as the first row of `03 Change Requests`.

Signed by. `GB-01 Architecture Board`, which `O57` mandates to approve *dispensations with an expiry* [4] — unrecordable without these sheets.

Done when. `02 Change Log` has at least one row carrying a `Change Ref` that resolves to `03 Change Requests`.

Shows up in. Nowhere yet. None of the thirteen `RPT` sheets reads change control.

Goes wrong when. You skip it and reach step 10.6 regardless. `O59` then fills with verdicts carrying no date and no assessor, and chapter 11's comparison tool classifies every one as ungoverned change.

BEFORE YOU ADD A SINGLE GATE RECORD

Which of the four sheets already exists in your copy, and which are you about to create?

Who signed the five-column addition, and on what date — a name and a date, not a team?

What is the one question about a past decision your repository cannot answer today?

The gates

Step 10.2 — Give each of the four gates a holder and a named evidence list

Produce. Four rows in `O58` that a body can actually refuse at.

You need. `O58`'s four rows, `O57`'s five bodies, and `O31 Deliverable`, where `DLV-18 Architecture Contract` and `DLV-19 Compliance Assessment` both read `Draft` against phase `G` [8] — the two drafts this chapter turns into records.

Where it lands.

> **`O58 Governance Gate`**
>
> Real columns: `Ref`, `Gate`, `When`, `What it tests`.
>
> Add: `Held by` (**add**) — the `O57 Ref` of the body that can say no, not its name. `Evidence required` (**add**) — the artefact a refusal would be argued over. `Why it exists` (**add**) — the thing that happened because the gate did not.

Keep the four rows as delivered: `GT-0 G0 Charter` *before work starts*, testing *scope, acceptance criteria and contract exist*; `GT-1 G1 Design` *before build*, testing *design meets principles, standards and reference architecture*; `GT-2 G2 Build complete` *before release*, testing *every acceptance criterion evidenced*; `GT-3 G3 Benefit` *after operation*, testing *stated benefit confirmed against a date* [1]. The holder is what is missing. `O57`'s `Mandate` states the allocation in prose [4], and prose in one sheet is not a join to another.

Which gates are missing in practice: all four. `O59` and `R24` are empty [3], yet `O33` records `ACO-01` and `ACO-07` as `Non-compliant` [2] — two verdicts with no assessment behind them. Treat both as assertions and re-derive at step 10.7.

How.

1. Add the three columns. `Held by` = `GB-02` for `GT-0` and `GT-3`, `GB-01` for `GT-1` and `GT-2`. Money gates to the funders, design gates to the architects.
2. Write `Evidence required` as a named artefact: for `GT-1`, *the interface design and the `R06` rows it will create*; for `GT-2`, *the `O59` verdict per criterion with `Evidence` filled*.
3. Write `Why it exists` from the file. `GT-1` exists because 19 of 32 interfaces are unmanaged and 20 unlogged [9] — nobody refused those designs, because no gate was held.
4. `DEC-06 Adopt the four-gate regime` already reads `Approved` [5]. Add the date and rationale; `O60` has no `Date` column, which is step 11.3's problem.
5. Name the person who chairs each gate. A delegate who never attends is not a member.

Signed by. `GB-01 Architecture Board`, with `GB-04 Executive Committee` as the recorded *escalation point when bodies cannot agree* [4].

Done when. Every `O58` row has a `Held by` holding a `GB-` reference that appears in `O57`.

Shows up in. Nothing computes over `O58`, and `RPT01 Dashboard`'s nineteen measures count no gates [7]. Track `O59`'s row count instead.

Goes wrong when. `Held by` holds a name rather than a `Ref`. Someone renames the board, the join dies silently, and none of `ZZ Checks`' fifteen checks tests a governance join [9].

Step 10.3 — Write the architecture contract for each package actually in delivery

Produce. One `O33` row per package with a delivery team, each with two parties and a date.

You need. `O30`'s twelve packages, `O56 Project`'s twelve `Lead` values, and `R13 Work Package delivers Build` for what each package owes.

Where it lands.

> **`O33 Architecture Contract`**
>
> Real columns: `Ref`, `Architecture contract`, `Work package`, `Compliance status`.
>
> Add: `Parties` (**add**) — delivering organisation and gate-holding body, slash-separated. `Signed` (**add**) — the date a named human signed. `Expiry` (**add**) — past the package end date, never on it. `Status basis` (**add**) — one sentence saying where the status came from. `Review cadence` (**add**). `Last review` (**add**).

A four-column contract is a pointer. `O33` says which package a contract belongs to and asserts a status; it does not say who is bound, when they agreed, or on what basis. Eight contracts cover `WP-01` to `WP-08` [2]. `WP-09` to `WP-12` have none — and `WP-10 Compliance mechanisms` reads `Delivery` = `Delivered` [6]. A package that delivered with no contract and no assessment is the most quotable finding in this repository.

How.

1. Create `ACO-09` to `ACO-12`, one per uncovered package.
2. Set `Parties` from `O56`'s `Lead` and the gate-holding body. `PRJ-09` reads *Domain Architect, Business* — an architect delivering a package the board will assess. Record that in `Status basis`.
3. Set `Compliance status` to `Not yet assessable` wherever `O59` has no rows for the package's blocks, with `Status basis` reading *no assessment exists*. Six of eight already do; make all twelve honest.
4. Set `Expiry` later than `O30`'s `End`. `WP-07` ends `2031-08`.
5. For `WP-10`, raise a change request instead of a contract. You cannot contract retrospectively — record what shipped, assess it at `G2` against `O55`, and let the verdict stand where a contract would have been.

Signed by. Both parties in `Parties`. A contract with one signature is a memo with better formatting.

Done when. Every `O30` row with `Delivery` other than `Not started` has a contract or a recorded reason why not. Seven qualify today, and `WP-09`, `WP-10` and `WP-11` have no contract.

Shows up in. `O33` itself, which no report reads. Twelve packages, twelve contracts, or you have a finding.

Goes wrong when. The contract is filed with the vendor agreement and treated as commercial. `O19 Contract` holds ten vendor agreements. `O33` holds the architecture's right to refuse a build.

Step 10.4 — Make G0 refuse a charter whose criteria were written rather than inherited

Produce. One rule, applied at `G0`, that does most of this phase's work.

You need. `O55 Building Block` — all 37 records carry an `Acceptance criteria` value [10] — and `R13`, which resolves 37 block-to-package links.

Look it up. Volume 1, chapter 16, *Modelling banking in ArchiMate*, *Capabilities and value streams* — why a capability-level criterion and a project-level criterion are different sentences, and which one governs.

Where it lands. No sheet. You are testing `O30 Work Package` against `O55` and refusing where they disagree.

The rule: a charter's acceptance criteria are the `Acceptance criteria` strings of the blocks the package delivers, copied verbatim. Not improved. Not summarised. A project wanting different criteria raises a change request against `O55`, decided at `GB-01`, not negotiated at the gate.

`O55`'s criteria are uneven. `ABB-02` reads *No unmanaged interface to or from Payment Hub* — testable. `BBB-01` reads *Customer Management measured and owned* — not. Ten of the 37 are of the second kind. Fix those before `G0`, never during it.

Check `R13` before you trust it. Its 37 rows allocate blocks to packages round-robin, `WP-01` to `WP-12` and back again, so `WP-03 Integration platform` is recorded as delivering `BBB-03 Origination & Onboarding capability uplift`, `DBB-05 Arrangement mastering` and `ABB-07 Digital Channel target` — three blocks in three domains, allocated by position rather than by argument. Re-argue the allocation before you assess a package against it.

How.

1. List the package's blocks from `R13`; pull their criteria from `O55`.
2. Ask one question of each criterion: could a reasonable person disagree about whether it was met? If not, rewrite it in `O55` under a change request before the gate sits.
3. Compare the charter's criteria to that list as sets. Refuse on any difference in either direction.
4. Refuse where `O30 Approval` reads `Not started` and work has begun. `WP-03` and `WP-12` both do [6].
5. Record the rule once in `O60`, beside `DEC-06`, rather than re-arguing it per charter.

Signed by. `GB-02 Investment Committee`, which holds `G0` and the funding behind it.

Done when. The charter's criteria set equals the `O55` set for its blocks, and every criterion passes the disagreement test.

Shows up in. `R13`, where every block the charter claims should appear against the package `Ref`. One that does not is a block nobody has funded.

Goes wrong when. `O30 Approval` is read as a gate verdict. Its values — `Approved`, `In flight`, `Not started` — mix funding with delivery, and `Delivery` carries delivery separately. Neither records a gate. That is what `O59` is for.

Step 10.5 — Build the G1 design review, which has never produced a record

Produce. A design verdict per acceptance criterion, taken before any build starts.

You need. The design, `O55`'s criteria, `O32 Standard` filtered to what applies, and `O05 Principle` — four of its 24 principles read `Draft` [11], and a draft principle carries no refusal.

Look it up. Volume 1, chapter 12, *The application landscape*, *How these systems actually talk* — the integration patterns a bank's design review must ask about by name.

Where it lands.

> **`O59 Compliance Assessment`**
>
> Real columns: `Ref`, `Building block`, `Block name`, `Acceptance criterion`, `Gate`, `Verdict`, `Evidence`.
>
> Add: `Assessed` (**add**) — the date. `Assessor` (**add**) — a named person. `Recorded in` (**add**) — the `O31 Deliverable` where a reader can check you.

At `G1`, `Evidence` is a statement about a design artefact, not a test result — *the sequence diagram routes every Payment Hub call through the gateway*, not *19 of 32 interfaces are unmanaged*. The second belongs at `G2`.

How.

1. Create one `O59` row per acceptance criterion for the package's blocks, `Gate` = `G1`. For `WP-03` that is three blocks as `R13` has it, and more once you have re-argued the allocation.
2. Set `Verdict` to one of three values and no more: `Compliant`, `Partial`, `Non-compliant`. A `Partial` needs a dated remediation in `Evidence` or it is `Non-compliant`.
3. Check the design against `O32` where `Status` is `Current` — nine are. Three are `Deprecated`, and conforming to one of those is a finding, not a pass.
4. Where the design departs deliberately, stop. That is a dispensation, step 10.10, not a verdict.
5. Write the matching `R24` row. The sheet is empty and its header is the specification.

Signed by. `GB-01 Architecture Board`, monthly per `O57 Cadence` [4].

Done when. Every criterion has a `G1` row with non-empty `Evidence`, and no row reads `Partial` without a date.

Shows up in. `059`'s row count, rising from zero. `MSR-15 Architecture compliance rate` carries a baseline of 51 per cent [12] with nothing in the file that could have computed it. Re-derive it once `059` has rows.

Goes wrong when. `G1` becomes a presentation. Nobody writes a verdict against a criterion, and the build is later assessed at `G2` against criteria it was never designed to meet. The record is the gate: a meeting with no rows in `059` did not happen.

THE DEPARTURE YOUR NEXT DESIGN REVIEW WILL FIND

Which standard in `032` will the next design breach — and is it `Current`, `Emerging` or `Deprecated`?

Which of the 37 criteria in `055` could you not take a verdict against today, and why?

Who will be in the room who can say no, by name?

Assessing compliance

Step 10.6 — Assess one acceptance criterion at a time, never one project at a time

Produce. `059` rows at the grain the sheet was designed for: one per criterion per gate.

You need. `055`'s 37 blocks and their criteria, the gate, and the evidence.

Where it lands. `059 Compliance Assessment` and `R24 Compliance Assessment tests`, both empty as delivered [3].

The arithmetic tells you what you are committing to. Thirty-seven blocks, one criterion each, two gates that take a verdict per criterion — `G1` and `G2` — is **74 rows** at full coverage, plus 24 package-grain verdicts at `G0` and `G3`. That is the whole register. Repositories carry one assessment per project instead because one row per project can be set to `Compliant` without reading anything.

Project-grain hides the shape of the failure. Take a package carrying ten criteria: if two fail on interface logging and eight pass, a project-grain verdict says `Partial` and gives you nothing to act on. Criterion-grain says which two, and the two are a conversation with a named team.

Form G-1 — compliance assessment, one row per criterion

Feeds `059 Compliance Assessment`, columns in sheet order. `Assessed` and `Assessor` are the two additions from step 10.5 and go straight into Excel. A first pass on one work package usually needs eight to twelve rows. Signed by the chair of the body holding the gate.

Ref	Building block	Acceptance criterion	Gate	Verdict	Evidence
CA-01	ABB-02	No unmanaged interface to or from Payment Hub	G1	Non-compliant	Example, delete me

How.

1. Take one package. Pull its blocks from `R13` and their criteria from `055`.
2. Create one row per criterion at the gate being held. Reference them `CA-01` upward; the sheet has no records and no prefix convention, so set one and keep it.
3. Fill `Evidence` with something a stranger could check — a report name and a number, a test identifier, a document and a page.
4. Write the matching `R24` row, which is how the assessment resolves to the block.
5. Stop when the package's criteria are exhausted. Do not assess another package's criterion because it came up in conversation.

Signed by. `GB-01` for `G1` and `G2`; `GB-02` for `G0` and `G3`.

Done when. Every criterion for the package has exactly one row at the gate held, and `R24`'s row count equals `059`'s.

Shows up in. `059` and `R24` only. `ZZ Checks` counts no assessments; add a check row and state in `Threshold` what a healthy number is.

Goes wrong when. One row per project, with `Verdict` taken from the project manager's status report. The repository then records compliance nobody tested, and the first audit finding is that architecture signed off work it did not assess.

Step 10.7 — Derive the contract's compliance status instead of asserting it

Produce. A `Compliance status` on every `O33` row computed from `O59`, with the derivation written down.

You need. `O59` with rows in it, and `R13` to resolve blocks to packages.

Where it lands. `O33 Architecture Contract` — `Compliance status`, plus `Status basis` (**add**) from step 10.3.

Verdicts across the package's criteria at the latest gate held	`Compliance status`
No `O59` rows exist for any of the package's blocks	`Not yet assessable`
Every criterion `Compliant`	`Compliant`
Any criterion `Non-compliant`	`Non-compliant`
No `Non-compliant`, at least one `Partial` with a dated remediation	`Partial`
At least one `Partial` with no date	`Non-compliant`

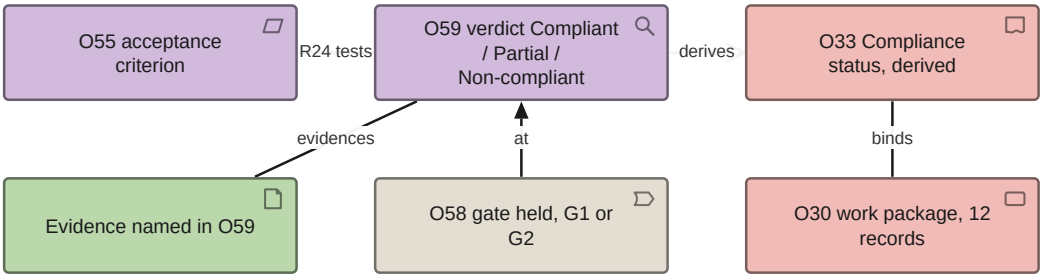

Figure 10.2 How `O33`'s `Compliance status` is derived rather than asserted. The verdict is taken per acceptance criterion at a gate, with evidence named; the contract status is computed from the set of verdicts by the fixed table in step 10.7. `ACO-01` and `ACO-07` currently read `Non-compliant` with nothing in this chain behind them.

How.

1. For each of the twelve packages, count its criteria and its verdicts. Six of eight contracts currently read `Not yet assessable`, which is correct, because no assessments exist [2].
2. Re-derive `ACO-01` and `ACO-07`. Both read `Non-compliant` with nothing behind them. Produce the assessments that justify it, or set them to `Not yet assessable` and say so in `Status basis`.

3. Write `Status basis` as one sentence naming the evidence: *derived from 10 G1 verdicts on 2026-11-14, two Non-compliant*. Set `Last review` to that date and re-derive monthly.
4. Never set `Compliance status` in a meeting. Compute it, bring the number, and let the meeting argue with the verdicts instead.

Signed by. `GB-05 Architecture Function`, mandated to *publish criteria, assess compliance, maintain the repository* [4]. This is the one status architecture sets alone.

Done when. Every `O33` status can be reproduced from `O59` by anyone holding the table above.

Shows up in. `O33` itself. There is no compliance position report here, so produce the twelve-row table by hand for the board pack.

Goes wrong when. A delivery lead negotiates the status. The moment it is negotiable, `O59` is decoration and the contract is a document. Derivation is what makes the number refusable.

Step 10.8 — Read RPT09 and RPT11 together, and act on the distance between them

Produce. A ranked list of obligations whose control is absent, with an owner against each.

You need. `RPT09`, `RPT11`, `O63 Control Objective`, `O41 Security Control` and `R16 Control Objective met by Co`.

Look it up. Volume 1, chapter 15, *Regulation, risk and what the regulator inspects*, *What the regulator actually asks to see* — the questions these two reports answer.

Where it lands. `O29 Gap` — fill `Ref`, `Gap`, `Domain`, `Building block`, `Severity`, `Status`, `Due`. The sheet is empty [17], so the failing obligations become its first rows. Nothing lands in the two reports; both compute from `O63` and `O41`.

`RPT09` lists seven obligations — three `Evidenced`, two `In breach`, two `No mechanism` — and closes *4 obligations have no mechanism or a mechanism in breach* [14]. `RPT11` lists eight controls — two `In place`, two `Partial`, four `Absent` — and closes *A control that is absent is not a control* [15].

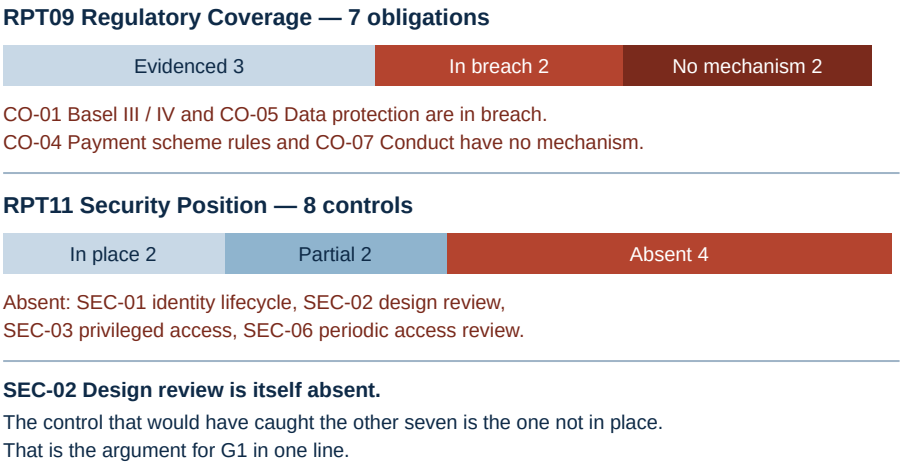

Figure 10.3 `RPT09 Regulatory Coverage` and `RPT11 Security Position` to scale, as shipped. Four of seven obligations have no mechanism or a mechanism in breach; four of eight controls are absent. The two reports are joined by `R16`, whose seven pairings are positional and must each be re-argued before either report is quoted.

The two are joined by `R16`, and the join is the finding. `R16` pairs each objective with exactly one control in sheet order: `CO-01 Basel III / IV` met by `SEC-01 Identity lifecycle`; `CO-05 Data protection` by `SEC-05 Interface logging` [16]. Read the first aloud. Identity lifecycle management does not evidence capital adequacy. The pairings are positional artefacts of generation, and all seven must be re-argued before either report is quoted.

How.

1. Take the four obligations whose evidence is `No mechanism` or `In breach` — `CO-01`, `CO-04`, `CO-05`, `CO-07` — and raise each as an `O29 Gap` row.
2. Re-argue every `R16` pairing. An objective may need several controls; add rows rather than compressing.
3. For each absent control, find the package that would deliver it. `WP-05 Access and security` covers three of the four. `WP-10 Compliance mechanisms` reads `Delivered` while four objectives have no mechanism — raise that as a change request.
4. Name an owner per obligation. `O01 Driver` gives all seven regulatory drivers to the Chief Compliance Officer [18], which is a single point of failure.
5. Put two numbers in every monthly pack: obligations without working evidence, and controls absent.

Signed by. `GB-01 Architecture Board`, with compliance present. The verdict is architectural; the obligation is not the architect's to interpret.

Done when. Every `RPT09` obligation not reading `Evidenced` has an `O29` row and a named owner who is not the Chief Compliance Officer by default.

Shows up in. `RPT09`'s closing count, which should fall. Not quickly: `O63`'s `Control objective` column reads *Control that evidences Basel III / IV.* and six variants of that sentence [19]. Those are placeholders; chapter 12 step 12.8 replaces them.

Goes wrong when. `RPT09` goes into a board pack unchanged. It reads like an assessment and it is a template.

READING THE TWO CONTROL REPORTS HONESTLY

Which of the seven `R16` pairings is defensible as it stands, and which would you not say out loud?

Which absent control in `RPT11` has no work package behind it at all?

What number will you quote to the board, and which cell did it come from?

Who may decide what

Step 10.9 — Write down what each body may not decide

Produce. A decision-rights boundary that survives an executive who disagrees with it.

You need. `057`'s five bodies and `060`'s seven decisions.

Where it lands.

> **`057 Governance Body`**
>
> Real columns: `Ref`, `Governance body`, `Cadence`, `Membership`, `Mandate`.
>
> Add: `May decide` (**add**) — the gates and decision types it holds. `May not decide` (**add**) — the harder and more useful column. `Chair` (**add**) — a named person.

Ref	Body	Cadence	Mandate as recorded [4]
GB-01	Architecture Board	Monthly	G1 and G2; dispensations with an expiry
GB-02	Investment Committee	Quarterly	G0 funding and G3 benefit confirmation
GB-03	Data Governance Council	Monthly	Designates data owners, resolves disputes
GB-04	Executive Committee	Monthly	Escalation when bodies cannot agree
GB-05	Architecture Function	Continuous	Publishes criteria, assesses compliance, maintains the repository

`May not decide` is where the work is. `GB-01` may not grant a dispensation without an expiry, alter an `O55` criterion a contract already binds, or set `Compliance status` — that is derived. `GB-02` may not overturn a `G1` or `G2` verdict; it may fund a remediation or cancel a package, which are different acts. `GB-03` may not designate an owner who is a committee: `PRN-13 Data Ownership is Individual` reads *An owner is a person, not a committee* and is `Approved` [11]. `GB-04` may not take an architecture decision; it breaks deadlocks. `GB-05` may not approve its own assessment.

How.

1. Add the three columns and fill `May decide` from the existing `Mandate` text, unchanged.
2. Write `May not decide` as specific prohibitions, each traceable to a principle in `O05` or a gate in `O58`.
3. Check `O60`'s seven decisions against the boundary — four `Outstanding`, three `Approved` [5]. `DEC-07 Grant dispensation for point-to-point transfer` is within `GB-01`'s mandate.
4. Where a decision was taken by the wrong body, do not reverse it quietly. Record it, send it to the body that owns it, and note the irregularity in `Status basis` on any contract it touched.
5. Name the chairs. A body with no named chair has nobody who can refuse.

Signed by. `GB-04 Executive Committee`, because the boundary binds bodies and only the escalation point can set it.

Done when. Every `O57` row has a non-empty `May not decide`, and every `O60` decision sits inside the mandate of the body in its `Body` column.

Shows up in. `O60`, whose `Body` column holds body *names* and not `GB-` references — another join to repair.

Goes wrong when. The Executive Committee starts taking architecture decisions because it meets monthly and the Investment Committee meets quarterly. Cadence becomes authority.

Step 10.10 — Grant a dispensation that expires on a date, or do not grant it

Produce. A time-bound dispensation with a compensating control and a named owner.

You need. The verdict that would otherwise be `Non-compliant`, the principle or standard being departed from, and the date the departure ends.

Where it lands.

> **`061 Governance Change Request`**
>
> Real columns: `Ref`, `Change request`, `Against`, `Type`, `Status`, `Expires`.
>
> `Type` already takes `Dispensation`; keep that value and invent no second word for it. `Expires` already exists, which makes this the one place in the repository where a time-bound decision is recordable as delivered.
>
> Add: `Raised` (**add**) — the date. `Compensating control` (**add**) — what holds the risk until expiry. `Owner` (**add**) — who closes it. `Decision ref` (**add**) — the `060` row.

Five requests exist. `CR-G01 Dispensation for point-to-point file transfer`, against `WP-04`, `Granted`, expiring `2029-06`. `CR-G02` a `Scope` change. `CR-G03` and `CR-G04` `Correction` against `Repository`. `CR-G05 Refuse or re-baseline the integration package`, type `Compliance`, status `Board decision required` [20]. Four carry no `Expires` — correctly, because only a dispensation needs one.

`CR-G01` is both model and warning: a dispensation expiring mid-2029 is a departure the bank lives with for over two years. `DEC-07` says the board took it knowingly. Check the expiry is shorter than the life of the thing it excuses, and that the date sits in the diary of the person in `Owner`.

How.

1. Write the dispensation as a sentence naming what is departed from and what is granted instead. `CR-G01` does this well.
2. Set `Expires`. No date, no dispensation — the verdict stays `Non-compliant` and the build does not proceed.
3. Write `Compensating control` and link it to `041` where one applies. A file transfer excused from the integration standard needs logging, and `SEC-05 Interface logging` reads `In place` [15].
4. Create the `060` row so the grant has a decision behind it, and put its `Ref` in `Decision ref`.
5. Diarise the expiry. On that date the dispensation lapses and the original verdict applies.

Signed by. `GB-01 Architecture Board`, mandated to approve *dispensations with an expiry* [4]. No other body may grant one.

Done when. Every `061` row typed `Dispensation` has a future `Expires` date and an `Owner` who knows about it.

Shows up in. `O61` sorted by `Expires` — a standing agenda item for `GB-01`. One row today; expect six to ten in a working practice.

Goes wrong when. A dispensation is granted verbally and recorded as a `Partial` verdict. It then never expires, and the departure becomes the standard. Every practice that has lost its standards lost them this way.

BEFORE YOU GRANT THE NEXT DISPENSATION

What exactly is being departed from — a principle in `O05`, a standard in `O32`, or a criterion in `O55`?

What date does it expire, and what happens on that date if nothing has changed?

Who holds the risk until then, by name, and do they know?

Benefit, and the banking overlay

Step 10.11 — Make G3 test benefit, which O30 cannot currently express

Produce. A `G3` verdict per package against a stated benefit and a date.

You need. `O30`, `O04 Measure`, `O03 Objective` and `R10 Objective measured by Measu`.

Where it lands.

> **`O30 Work Package`**
>
> Real columns: `Ref`, `Work package`, `Domains`, `Cost`, `Start`, `End`, `Approval`, `Delivery`.
>
> There is **no benefit column of any kind**. Twelve packages carry costs from `$1.6m` to `$10.1m`, roughly `$64m` in total, and not one states what the money buys.
>
> Add: `Benefit measure` (**add**) — the `O04` `Ref` the package moves. `Benefit target` (**add**). `Benefit starts` (**add**) — when measurement begins. `Benefit confirmed` (**add**) — when `GB-02` agreed it landed.

The repository knows this is the gap. `O62` carries `RSK-07 Benefit not measurable at G3`, likelihood `Certain`, owned by the Chief Financial Officer [13]. `O61` carries `CR-G04 Record benefit realisation dates per package`, status `Approved` [20]. The fix has been approved and not done.

How.

1. Add the four columns, then pick the measure each package moves. `WP-03` moves `MSR-08 Managed interfaces`, baseline 27 per cent; `WP-02` moves `MSR-10 Entities with a named owner`, baseline 26.
2. Set `Benefit target` as a number on that measure, not a percentage improvement. `MSR-08` to 90 per cent is a target; *improve integration* is not.
3. Set `Benefit starts` after `End`. `MSR-08`'s `Cadence` is `Quarterly`, so the first honest reading is a quarter after go-live.
4. Refuse at `G0` any charter whose package has no `Benefit measure`. That is the rule that stops this recurring.
5. Hold `G3` on `Benefit confirmed`, not on `Delivery = Delivered`. `WP-08` and `WP-10` both read `Delivered` [6] and neither has been to a benefit gate.

Signed by. `GB-02 Investment Committee`, quarterly, mandated for *G3 benefit confirmation* [4].

Done when. Every package has a `Benefit measure` resolving to `O04`, and every delivered package has a `Benefit confirmed` date or a `G3` verdict of `Non-compliant`.

Shows up in. `RPT08 Roadmap`, which carries cost and dates and no benefit column either. Until you add one, the roadmap report is a spend plan.

Goes wrong when. `G3` is held on delivery evidence. The package shipped, so it passed. Two years later the measure has not moved and nobody can say which package was meant to move it.

Step 10.12 — Map the board's duty to sheets and columns, and decide what a supervisor gets

Produce. A one-page map from the board's duty to the columns that evidence it, with a named control-function seat at `GB-01`.

You need. `O57`, `O60`, the fifteen sheets below, and volume 1, chapter 15.

Look it up. Volume 1, chapter 15, *Regulation, risk and what the regulator inspects*, *King IV, and the board's version of the same question* — Principle 12, apply-and-explain, and why the governing body owns this.

Look it up. Volume 1, chapter 5, *Who does what*, *Internal audit, and the control functions the law requires* — the three functions that must see your gate records without asking you.

Where it lands. No sheet holds the map. It names columns across fifteen existing sheets, and the only thing you write into the repository is the missing values it exposes, plus attendees on `O57 Membership` and a row in `O60`.

King IV reaches a listed bank through the JSE Listings Requirements as apply-and-explain, and **Principle 12** puts the governance of technology and information with the governing body [21]. **Joint Standard 1 of 2023** is the enforceable version of the same duty, and it obliges named artefacts: a board-approved IT strategy reviewed annually with quarterly action-plan review, an

IT risk framework with a risk register, an IT asset inventory, a register of critical applications, recovery objectives, annual resilience testing and notification of material incidents [22]. The Prudential Authority asks for data, not diagrams, so every cell in the map below names a sheet and a column.

What a supervisor asks for	Sheet	Column	Tested at
Register of critical applications	`023 Application Component`	`Criticality`, `Owner`, `Health`	`G1`
IT asset inventory	`027 Physical Technology Compone`	`Vendor`, `Support end`, `Position`	`G2`
Recovery objectives, and whether tested	`049 Resilience Tier`	`Recovery objective`, `Tested`	`G2`
Who owns each data entity	`020 Data Entity`	`Owner`, `Classification`, `Lineage`	`G1`
Lineage from report to source	`040 Source To Target Mapping`	`Target field`, `Rule`	`G2`
Which interfaces are logged	`R06 Application interfaces with`	`Managed`, `Logged`	`G2`
Obligation to control to evidence	`063`, `R16`, `RPT09`	`Control objective`, `Evidence`	`G2`
Security control position	`041`, `RPT11`	`Position`, `Enforced at`	`G1`
Standards and conformance	`032`, `RPT12`	`Status`, `Conformance`	`G1`
Who decided, and when	`060 Governance Decision`	`Decision`, `Body`, `Status`	all
Departures, and until when	`061`	`Type`, `Expires`	`G1`
Personal data outside production	`048 Environment`	`Personal data present`	`G1`
Third-party arrangements and expiry	`019 Contract`	`Counterparty`, `Expiry`	`G0`
Capability maturity against target	`013`, `RPT03`	`Current`, `Target`	`G0`
Repository integrity	`ZZ Checks`	`Result` against `Threshold`	all

What the repository does **not** hold is the IT risk register. `062 Delivery Risk` holds delivery risk, not IT risk. Say that rather than blurring it.

How.

1. Walk the table and check each named column is populated. Several are not: `063`'s control objectives are placeholders and three data entities have no lineage [9].

2. Where a column is thin, decide whether to fill it or to record where the evidence lives. Both are acceptable. Silence is not.
3. Give the three control functions a standing seat. `GB-01`'s `Membership` names architects, the CIO and the CISO [4]; risk, compliance and internal audit are absent. Add them as attendees who may not vote and may always object in writing.
4. Put the map in front of internal audit before a supervisor sees it, and put the annual strategy review and quarterly action-plan review into `O57`'s cadence.
5. Record the map in `O60`, beneath `DEC-05`, which already reads `Approved` [5].

Signed by. `GB-04 Executive Committee` on behalf of the board, with `GB-01` maintaining it.

Done when. Every row names a column that has values in it, or names where the evidence lives instead, and the three control functions have sat in a `GB-01` meeting.

Shows up in. `ZZ Checks`, the closest thing here to a self-assessment: fifteen rows, of which eight fail their threshold [9].

Goes wrong when. The practice prepares for inspection rather than being inspectable. If the map takes two weeks to assemble, the map is not the problem — the columns are.

Step 10.13 — Get the Joint Standard numbering right before you brief anyone

Produce. An instrument table in `O63 Control Objective`, numbered correctly.

You need. `O63`, `O01 Driver` and volume 1, chapter 15.

Look it up. Volume 1, chapter 15, *Regulation, risk and what the regulator inspects*, *The Joint Standards, with the numbering got right* — the table, the dates and the error in circulation.

Where it lands. `O63 Control Objective` — fill `Ref`, `Obligation`, `Source`, `Control objective`, `Evidence`. Put the instrument in `Obligation` and its commencement date in `Source`. `O01 Driver` carries the same obligations as drivers; keep the two consistent.

Instrument	Subject	Applies to banks	Commenced
Joint Standard 1 of 2023	IT governance and risk management	Yes	15 November 2024 [22]
Joint Standard 2 of 2024	Cybersecurity and cyber resilience	Yes	1 June 2025, twelve months to comply fully [23]
Joint Standard 1 of 2024	Outsourcing by **insurers**	**No**	1 December 2024 [24]
Joint Standard 1 of 2025	CCP and trade-repository exemption criteria	Derivatives activity	October 2025 [22]

> **THE ERROR YOU WILL BE HANDED IN A PAPER**
>
> **Joint Standard 1 of 2024 is an insurance outsourcing standard and does not apply to banks** [24]. You will see it cited as a banking outsourcing obligation in learner guides, vendor decks and compliance summaries. It is wrong. Banks' outsourcing obligations come from the Banks Act Regulations, Prudential Authority directives and guidance, and the third-party provisions inside the two technology Joint Standards. Volume 1 could not confirm which current Prudential Authority instrument governs bank outsourcing, or its number. Neither can this workbook. Until you have confirmed it, write *the applicable Prudential Authority requirement* and not a number.

On notification. Both standards oblige notification of material incidents — material IT incidents under Joint Standard 1 of 2023, material cyber incidents under Joint Standard 2 of 2024 [22] [23]. Volume 1 could not confirm a deadline for either; the sources state the duty without a timeframe. POPIA breach notification runs separately, to the Information Regulator and to affected data subjects: one incident, two notifications, two formats. A **material change** to a material or outsourced arrangement carries its own notification expectation under the Banks Act regime — the duty whose instrument number volume 1 could not pin down. Record all of it the same way: a notification duty is a control objective in `O63` with evidence in `RPT09`, tested at `G2`.

How.

1. Replace `O63`'s generic rows where they cover technology governance. `CO-06 Operational resilience` reads *Impact tolerances for important business services* with `Evidence` = `Evidenced` [19]; split it into the Joint Standard 1 of 2023 artefacts that actually evidence it.
2. Add one obligation row per instrument, with the commencement date in `Source`.
3. Add a notification obligation with its own control objective and evidence. Nothing in the repository records an incident notification path.
4. Where you cannot confirm an instrument, say so in the cell. *Prudential Authority requirement, instrument number unconfirmed* beats a wrong number.
5. Brief the material-change and material-incident duties to the delivery leads. Architecture will not be the function that notices the incident.

Signed by. `GB-01 Architecture Board` with compliance, who own the interpretation.

Done when. Nothing the practice issues cites Joint Standard 1 of 2024 as applying to the bank, and every Joint Standard cited carries its commencement date.

Shows up in. `RPT09`, which will grow past seven rows. That growth is the point: seven generic obligations is not a regulatory universe for a South African bank.

Goes wrong when. Somebody writes the numbering from memory. Three instruments numbered "1" across three years. Write it from the table every time.

Running it

Step 10.14 — Run the gate calendar monthly, from the cadence already in the file

Produce. A dated gate calendar for the next quarter, with named attendees.

You need. `057`'s `Cadence`, `030`'s `Start` and `End`, and `066 Architecture Increment`'s six increments.

Where it lands. No sheet holds a calendar. Drive it from `057 Cadence`: `GB-01` monthly, `GB-02` quarterly, `GB-03` monthly, `GB-04` monthly, `GB-05` continuous [4].

Form G-2 — this month's gate agenda

Feeds nothing; drives `059`. One row per gate to be held this month. `Package` from `030`, `Gate` from `058`, `Criteria` as the count of `055` criteria to be assessed — that number is how long the meeting will take. Signed by the chair of the body in `Body`.

Date	Body	Package	Gate	Criteria	Outcome
2026-11-14	GB-01	WP-03	G1	3	Example, delete me

How.

1. Lay the twelve packages against the calendar using `Start` and `End`. Four are past their end date [9] and need a `G2` now, not a `G0`.
2. Set each gate on the holding body's cadence. `G0` and `G3` are quarterly because `GB-02` is — a real constraint on how fast a package can start.
3. Publish the quarter ahead. A gate the team hears about a week out is a gate they will ask to defer.
4. Cap each meeting by criteria count. Ten criteria is an hour of honest argument.
5. Record every held gate as `059` rows the same day. Not the same week.

Signed by. The chair of each body, per `057 Chair` (**add**).

Done when. The next three months of gates are dated, and every `O30` package appears at a gate or carries a recorded reason why not.

Shows up in. `O59`'s row count, growing monthly. If it stops growing, the calendar is running and the gates are not.

Goes wrong when. The calendar is built from the delivery plan instead of the governance cadence. The gate then moves whenever the project moves, which is the same as no gate.

When the answer is to refuse

Phase G is the only phase where architecture's main output is a refusal. The repository you were handed shows what an estate looks like when nothing was ever refused: 19 unmanaged interfaces, 20 unlogged, 10 direct database reads, three components past vendor support, four security controls absent, four packages past their end date [9]. None of that is a technology failure. Each is a design nobody said no to.

The refusal has to be cheap to make and expensive to overturn. Cheap, because an architect who needs permission to refuse stops refusing. Expensive, because a refusal overturnable in a corridor is a suggestion. The four gates, the criterion-grain assessment, the derived status and the expiring dispensation produce that asymmetry, and all four are rows in sheets you now know.

`ZZ Checks` closes with a line worth reading twice: *The thresholds are what a healthy estate would show. The results are what this one does. That distance is the work* [9]. A gate held without a record closes none of it.

PHASE G — A REVIEWER'S QUESTIONS

Which gate, held once in the next ninety days, would have refused the worst thing in your estate?

How many rows are in your `059` today, and how many should there be given the blocks in scope?

Name the decision the Architecture Board took last quarter that it had no mandate to take. What did you do?

Which dispensation in `061` expires next, and who holds the risk until then?

THE CHAPTER IN TEN LINES

- Phase G produces refusals, dispensations and evidence. A practice that cannot refuse made every earlier phase decorative.
- Add `01 Version Control`, `02 Change Log`, `03 Change Requests` and `04 Baselines`, plus `Version`, `Record Status`, `Changed By`, `Change Ref` and `Approved By` on all 65 object sheets, before any gate is held.
- `O58` carries four gates and `O59` carries zero assessments. Every gate in this repository exists on paper and has never been held.
- `O33` has four columns and needs `Parties`, `Signed`, `Expiry` and `Status basis` before it can govern. Eight contracts cover twelve packages.
- Assess one acceptance criterion at a time. 37 blocks across `G1` and `G2` is 74 rows — a register, not a burden.
- Derive `Compliance status` from `O59` by a fixed table. The moment it is negotiable, the contract is a document.
- `O61` already carries `Type` = `Dispensation` and an `Expires` column. No date, no dispensation.
- `O30` has no benefit column at all. `RSK-07 Benefit not measurable at G3` is recorded as `Certain` and `CR-G04` to fix it is `Approved` and undone.
- For banks: Joint Standard 1 of 2023 on IT governance and risk, commenced 15 November 2024, and Joint Standard 2 of 2024 on cyber, commenced 1 June 2025. Joint Standard 1 of 2024 is insurers' outsourcing and does not apply.
- King IV Principle 12 and Joint Standard 1 of 2023 put technology governance with the board. The evidence is columns in your sheets, not a policy document.

SOURCES FOR THIS CHAPTER

- **1.** Banking_TOGAF_Metamodel.xlsx, sheet O58 Governance Gate
- **2.** Banking_TOGAF_Metamodel.xlsx, sheet O33 Architecture Contract
- **3.** Banking_TOGAF_Metamodel.xlsx, sheets O59 Compliance Assessment and R24 Compliance Assessment tests, both 0 records
- **4.** Banking_TOGAF_Metamodel.xlsx, sheet O57 Governance Body
- **5.** Banking_TOGAF_Metamodel.xlsx, sheet O60 Governance Decision
- **6.** Banking_TOGAF_Metamodel.xlsx, sheet O30 Work Package
- **7.** Banking_TOGAF_Metamodel.xlsx, sheet RPT01 Dashboard
- **8.** Banking_TOGAF_Metamodel.xlsx, sheet O31 Deliverable
- **9.** Banking_TOGAF_Metamodel.xlsx, sheet ZZ Checks
- **10.** Banking_TOGAF_Metamodel.xlsx, sheet O55 Building Block
- **11.** Banking_TOGAF_Metamodel.xlsx, sheet O05 Principle
- **12.** Banking_TOGAF_Metamodel.xlsx, sheet O04 Measure
- **13.** Banking_TOGAF_Metamodel.xlsx, sheet O62 Delivery Risk
- **14.** Banking_TOGAF_Metamodel.xlsx, sheet RPT09 Regulatory Coverage
- **15.** Banking_TOGAF_Metamodel.xlsx, sheets RPT11 Security Position and O41 Security Control
- **16.** Banking_TOGAF_Metamodel.xlsx, sheet R16 Control Objective met by Co
- **17.** Banking_TOGAF_Metamodel.xlsx, sheets O29 Gap and RPT07 Gap Analysis, both 0 records
- **18.** Banking_TOGAF_Metamodel.xlsx, sheet O01 Driver
- **19.** Banking_TOGAF_Metamodel.xlsx, sheet O63 Control Objective
- **20.** Banking_TOGAF_Metamodel.xlsx, sheet O61 Governance Change Request
- **21.** Werksmans, King IV guide (apply-and-explain, seventeen principles, Principle 12 on technology and information) https://werksmans.com/wp-content/uploads/2025/04/Werksmans_KingIV_Booklet_V2.pdf
- **22.** Joint Standard 1 of 2023 — IT governance and risk management (commenced 15 November 2024) https://lawlibrary.org.za/akn/za/act/standard/sarb/2023/1/eng@2024-11-15
- **23.** Moonstone, "Cybersecurity Joint Standard commencement date confirmed" (Joint Standard 2 of 2024 effective 1 June 2025) https://www.moonstone.co.za/cybersecurity-joint-standard-commencement-date-confirmed/
- **24.** Joint Standard 1 of 2024 — Outsourcing by Insurers (in operation 1 December 2024; applies to insurers, not banks) https://www.resbank.co.za/content/dam/sarb/publications/prudential-authority/pa-public-awareness/cov
- **25.** The Open Group, The TOGAF Standard, 10th Edition — Architecture Development Method, Phase G: Implementation Governance https://pubs.opengroup.org/togaf-standard/adm/

CHAPTER 11

Architecture Change Management

Every change is either governed or discoverable later as a defect. There is no third state.

BEFORE YOU START THIS PHASE

Phase H is the phase that decides whether anything you built in A to G is still true next year. It has almost no new modelling in it and a great deal of discipline.

- **Chapter 19, *The cross-maps*** — *Keeping these alive*: the maintenance argument, which this chapter turns into a cadence.
- **Chapter 15, *Regulation, risk and what the regulator inspects*** — *The payment system, and who writes the rules now*: where a scheme mandate comes from, and who tells you.
- **Chapter 7, *The operating model*** — *The reporting and regulatory calendar*: the dates that make a Phase H cycle predictable rather than reactive.
- **Chapter 14, *Technology providers*** — *What a core replacement costs, and how long it takes*: why a core milestone is a Phase H trigger and not a project status update.
- **Chapter 18, *Your first ninety days*** — *Six numbers that tell you whether the estate is healthy*: the numbers step 11.9 turns into a standing report.

Ardent Union Bank is an invention. Its data is real within this workbook and nowhere else.

What you are building in this phase

Step	Produce	Lands in
11.1	A change request raised before any cell moves	`03 Change Requests` **(add)**, `061`
11.2	A log line per field change, carrying a reference	`02 Change Log` **(add)**
11.3	A decision taken at a body and recorded as a record	`060 Governance Decision`
11.4	A classification, decided by three questions	`03 Change Requests` `Classification`
11.5	Two extracts, produced by hand	The comparison tool's paste targets
11.6	The ungoverned-change count, and what you did about it	The tool's `Review` sheet
11.7	A cadence tied to something that already happens	`057` `Cadence`
11.8	A baseline cut for a reason, not on a schedule	`04 Baselines` **(add)**
11.9	Repository health as a standing monthly report	`ZZ Checks` , `RPT13` , `RPT06`
11.10	Volume 1's cross-maps still true	`R01` – `R25` , by name not by number
11.11	A trigger-to-action table the business can see	`018` , `019` , `027` , `063`
11.12	A support expiry answered as a dated change	`027` , `019`
11.13	Four numbers and two decisions for the board	The annual review pack

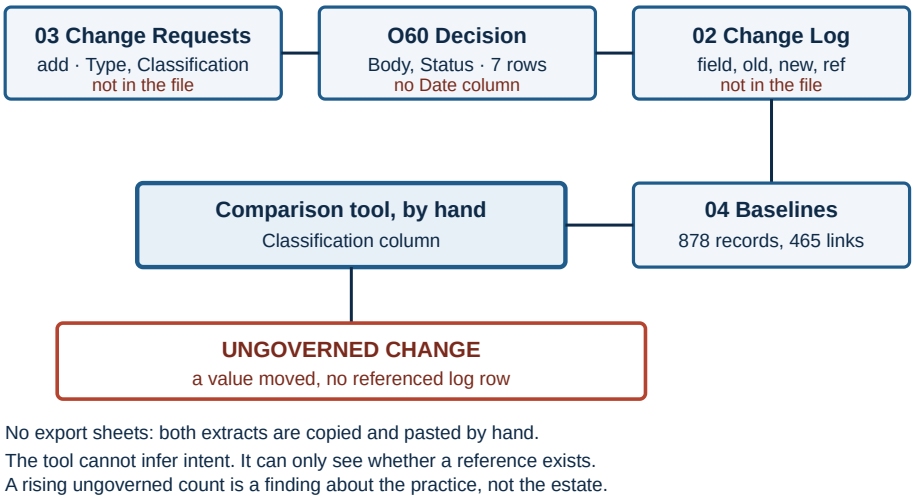

Figure 11.1 The change pipeline, and the single place it is enforced. Three of the five boxes do not exist in the repository as shipped; step 10.1 adds them. A field value that moves without a change log row carrying a `Change Ref` is classified as ungoverned change by the comparison tool, with no appeal.

The pipeline you have to build

Everything in this chapter depends on three sheets that do not exist in the file you were handed. Step 10.1 specified them. If you skipped it, stop and do it, because Phase H without a change log is a

monthly conversation about whether something changed, conducted from memory.

Step 11.1 — Raise the change request before you touch a cell

Produce. One row in `03 Change Requests` (**add**), raised before any value moves.

You need. The request, whoever asked for it, and the sheets it will touch.

Where it lands. `03 Change Requests` (**add**) — `Change ref`, `Raised`, `Raised by`, `Against`, `Type`, `Description`, `Classification`, `Impact`, `Decision body`, `Decided`, `Status`, `Expires`, `Decision ref`.

`Against` names what the change is aimed at: a work package `Ref`, a sheet name, or `Repository` for a change to the model itself. `O61 Governance Change Request` already uses exactly that convention — `CR-G01` against `WP-04`, `CR-G03` and `CR-G04` against `Repository` [2]. Follow it.

The two sheets divide by subject, and keeping them apart matters. `O61` holds **governance** change requests: dispensations, scope changes to a package, corrections to the governance model, compliance escalations. `03 Change Requests` holds **repository** change requests: anything that moves a value in an object or relationship sheet. A dispensation goes in `O61` and gets a row in `03` only if it also changes a record.

How.

1. Give the request a reference and a date. `CR-001` upward for repository changes; `O61` continues its own `CR-G0n` series.
2. Write `Description` as what will be different afterwards, not as the problem. *Set `A07 Digital Channel Disposition` to `Retire` and raise the two interfaces it owns as gaps* is a description. *Digital channel strategy* is not.
3. Fill `Impact` with the sheets and row counts the change touches. If you cannot, the change is not scoped and should not be decided.
4. Set `Decision body` from `O57`, and only to a body whose `May decide` covers it — step 10.9.
5. Leave `Status` at `Raised` until a body has sat. `O61` carries five statuses today: `Granted`, `Approved`, `Raised`, `Approved`, `Board decision required` [2]. That last one is the honest value when nobody has the mandate.

Signed by. Whoever is in `Raised by`. A request with no named raiser gets refused at intake.

Done when. No value in the repository has moved since the last baseline without a `Change ref` pointing at a row here.

Shows up in. `02 Change Log`, where every row's `Change Ref` must resolve to a request in this sheet. An unresolvable reference is the same defect as no reference.

Goes wrong when. The request is written after the change, to tidy the record. The dates give it away, and so does the comparison tool: a log row dated after the value moved tells you the practice is reconstructing rather than governing.

Step 11.2 — Log every field change, with its reference, or the tool will classify it for you

Produce. One row in `02 Change Log` (**add**) per field that moved.

You need. The change request, the sheet and `Ref` of the record, and the old value.

Where it lands. `02 Change Log` (**add**) — `Log ref`, `Date`, `Sheet`, `Record ref`, `Field`, `Old value`, `New value`, `Change ref`, `Changed by`, `Reason`.

The rule, and it is the only rule in this chapter that has no exceptions: **every field change carries a `Change Ref`, or the comparison tool will later classify it as ungoverned change.** Not *may*. The tool has no other way to tell a governed edit from an unexplained one. A value that differs between two extracts, with no log row carrying a reference, is by definition ungoverned.

One row per field, not per record. Changing `A07 Digital Channel`'s `Disposition` and `Health` in one sitting is two log rows against one request. The grain matters because the comparison tool diffs at field level and a record-grain log cannot be reconciled against it.

How.

1. Before editing, copy the old value into `Old value`. Afterwards is too late; the cell is gone.
2. Set `Sheet` to the tab name exactly as the tab spells it, truncations included — `O26 Logical Technology Componen`, `O27 Physical Technology Compone`, `R05 Application manages Data En`. Several tab names are cut at Excel's 31 characters and the log must match the tab, not the intended name.
3. Set `Record ref` to the `Ref` value, never the name. Names change; that is often the change being logged.
4. Fill `Change Ref`. If there is no request, stop editing and go back to step 11.1.
5. Write `Reason` in one clause. *Vendor confirmed support end moved* is a reason. *Update* is not.

Form H-1 — change log, one row per field

Feeds `02 Change Log` (**add**), columns in sheet order. `Changed by` and `Reason` are typed straight into Excel once the row is transcribed. A working month usually fills ten to thirty rows; more than twelve on this form means continue on a second copy. Signed by the Chief Enterprise Architect.

Date	Sheet	Record ref	Field	Old value	New value
2026-11-14	O23 Application Component	A07	Disposition	Retire	Example, delete me

Signed by. `Changed by`, a named person. The architecture function owns the log even when the change came from elsewhere.

Done when. The log's row count for the period equals the number of differing fields the comparison tool finds between the last baseline and now.

Shows up in. The comparison tool's `Classification` column. A clean month shows governed drift and design deltas, and nothing marked ungoverned.

Goes wrong when. Somebody fixes a typo without logging it, on the grounds that a typo is not a change. Next month the tool reports an ungoverned change, somebody spends an afternoon on it, and the answer is a typo. The cost of that afternoon is why the rule has no exceptions.

Step 11.3 — Decide it at a body, and record the decision as a record

Produce. One row in `060 Governance Decision` per decision taken.

You need. The request, the body from `057`, and the date.

Where it lands.

> **`O60 Governance Decision`**
>
> Real columns: `Ref`, `Decision`, `Body`, `Status`.
>
> Add: `Date` (**add**) — without it the sheet cannot order two decisions on the same question. `Rationale` (**add**) — one sentence. `Change ref` (**add**) — the request decided. `Supersedes` (**add**) — the `DEC-` row this replaces.

Four columns is not a decision record. `O60` tells you that the Investment Committee has four `Outstanding` decisions and the Architecture Board has three `Approved` ones [1], and nothing about when, why, or what they replaced. `DEC-01 Adopt the recommended architecture alternative` has been `Outstanding` since the file was built, and there is no column that would tell you for how long.

Note also that `Body` holds a body *name* — `Investment Committee`, `Architecture Board`, `Data Governance Council` — and not an `O57 Ref`. Add the `Ref` alongside or accept that the join is by string match.

How.

1. Create the row at the meeting, not afterwards. `Decision` is a sentence in the past tense: *granted a dispensation for point-to-point file transfer until 2029-06*.
2. Set `Date` to the meeting date and `Body` to the body that sat. Put the `GB-` reference in too.
3. Write `Rationale` in one sentence, and make it the reason that would be given to someone who disagrees.
4. Put the `Change ref` in, both ways: the request's `Decision ref` points here and this row's `Change ref` points back.
5. Where this decision replaces an earlier one, fill `Supersedes`. A decision that silently contradicts `DEC-06 Adopt the four-gate regime` is how a governance framework dies.

Signed by. The chair of the body in `Body`.

Done when. Every `O61` and `03 Change Requests` row with `Status` other than `Raised` has a `Decision ref` resolving to an `O60` row with a date.

Shows up in. `O60` sorted by `Date`, which becomes the governance activity record this repository does not otherwise have.

Goes wrong when. Decisions are minuted in a document and the sheet is updated quarterly from the minutes. The sheet then lags, and the lag is invisible because there is no date column to expose it.

Step 11.4 — Classify the change as simplification, incremental or re-architecting

Produce. A `Classification` value on every request, set by answering three questions about the repository rather than by judgement.

You need. The request, and the sheets its `Impact` names.

Where it lands. `03 Change Requests` (**add**) — `Classification`. Three values only.

TOGAF's three classes are useful here precisely because the repository can answer them. Ask the questions in order and stop at the first yes.

Question about the repository	If yes	Why
Does the change only remove rows, or set a `Disposition` to `Retire` or `Consolidate`, with no new `055` building block?	**Simplification**	Nothing new is being built. `023`'s `Disposition` column already carries `Retire` twice and `Consolidate` once.
Does it change values inside existing records, or add rows to existing sheets, without changing which capability any application supports in `R04`?	**Incremental**	The shape holds; the content moves.
Does it change `R04 Business Service supported`, `R03 Capability realised by Busi`, or any `055` acceptance criterion a contract binds?	**Re-architecting**	The target architecture itself is in question, and that is a Phase A re-entry, not a Phase H edit.

The third test is the one that earns its place. A request that moves which application supports a business service is not a change to the repository; it is a change to the architecture, and it goes back round the ADM. `033` binds eight contracts to acceptance criteria [4]; a request that alters one of those criteria is re-architecting whatever its author calls it.

How.

1. Run the three questions in order. Record the answer, not the class, in `Impact` so the next reader can check the reasoning.
2. Route by class: simplification to `GB-05 Architecture Function`, incremental to `GB-01 Architecture Board`, re-architecting to `GB-02 Investment Committee` and then to Phase A.
3. Refuse any request whose author has pre-classified it as incremental when question three is a yes. That re-labelling is the commonest governance failure in Phase H.
4. Count the three classes monthly. A practice with no re-architecting requests in a year is either very stable or not asking question three.
5. Where a request spans classes, split it. A single request that both retires an application and changes a capability mapping is two requests.

Signed by. `GB-05 Architecture Function` sets the class; `GB-01` may overturn it.

Done when. Every request has a class and the reasoning behind it is written down.

Shows up in. The class mix. Ardent Union's `061` would classify as one dispensation, one scope change, two corrections and one compliance escalation [2] — no re-architecting, in an estate with a core renewal package in flight. That absence is itself the finding.

Goes wrong when. The class is chosen to pick the decision body. An architect who wants a fast answer calls it incremental; one who wants cover calls it re-architecting. The three questions

exist to take that choice away.

THE CHANGE YOU MADE WITHOUT A REQUEST

Name one value you changed in the repository in the last month with no change request behind it.

Under the three questions, what class was it — and which body should have decided it?

What stops the next one happening: a rule, a habit, or a person?

Drift you were not told about

Step 11.5 — Produce the two extracts by hand, because this repository cannot export

Produce. Two extracts in the comparison tool's paste format: the baseline and the current state.

You need. `Banking_Architecture_Comparison_Tool_TEMPLATE.xlsx`, the repository, and the baseline you cut at step 11.8.

Where it lands. The tool's five paste targets. Chapter 8 of this volume covers the tool in full; this step is the Phase H use of it.

Say this plainly to whoever asks for the numbers: **the extracts must be produced by hand.** This repository has no export sheets. The insurance repository shipped three; this one shipped none, and there is nothing in `00 Index`'s structure that would generate a flat extract. You select the columns, copy, and paste. That is a half-hour of work per run and it is the reason the cadence in step 11.7 is monthly rather than weekly.

How.

1. Decide the scope of the run and write it down. Five object sheets is a useful monthly run: `O13 Business Capability`, `O23 Application Component`, `O20 Data Entity`, `O27 Physical Technology Compone`, `O32 Standard`.
2. For each, copy `Ref` plus the columns that carry state — for `O23` that is `Disposition`, `Criticality`, `Health`, `Owner`; for `O13` it is `Criticality`, `Current`, `Target`, `Disposition`.
3. Paste the baseline extract and the current extract into the matching paste targets. Keep the column order identical between the two or the diff is meaningless.

4. Run the tool and read `Classification` first. Nothing else in the output means anything until you know which mode the run was in.
5. Save the run with its date beside the baseline it used. A diff without its baseline named is a number nobody can reproduce.

Signed by. `GB-05 Architecture Function`. Nobody else should be producing these extracts, because whoever produces them controls what the diff can see.

Done when. The two extracts have identical column sets and the run completes with a `Classification` on every differing row.

Shows up in. The tool's `Review` sheet, which is where a person decides what each ungoverned change actually was.

Goes wrong when. The scope changes between runs. Last month's run covered four sheets and this month's covers six, so the ungoverned count rises and nobody knows whether the estate got worse or the net got wider. Fix the scope, change it only by a change request, and record the change.

Step 11.6 — Read the ungoverned count as a practice failure, not an estate failure

Produce. A number, a trend, and a named action per ungoverned change.

You need. The tool's `Review` sheet and `02 Change Log`.

Where it lands. The `Review` sheet, and then `02 Change Log` for anything you log retrospectively.

The tool's `Classification` separates three things: a **design delta**, where baseline and target differ because they are meant to; **governed drift**, where a value moved and a log row explains it; and **ungoverned change**, where a value moved and nothing explains it. Only the third is a finding, and it is a finding about your practice rather than about the bank's systems.

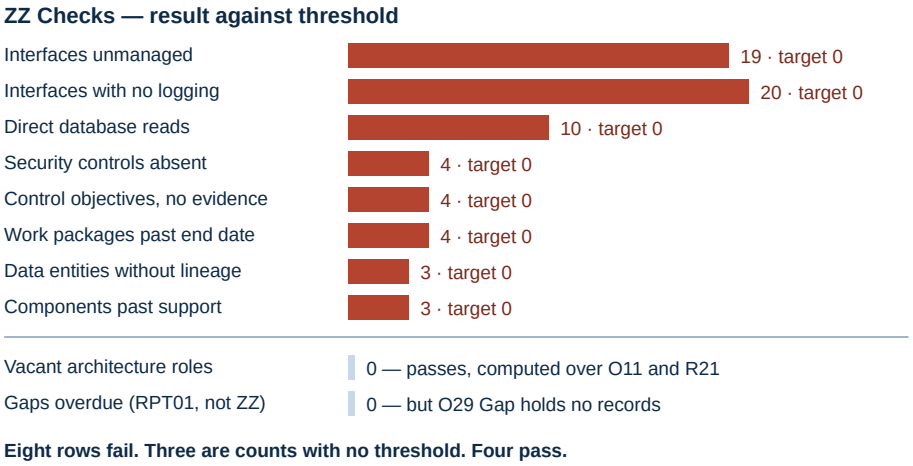

Figure 11.2 `ZZ Checks` as shipped: eight of its fifteen rows fail their threshold, three are counts with no threshold, and four pass. The two zeros that pass are worth a second look, and the `Gaps overdue` reading below them comes from `RPT01 Dashboard` rather than `ZZ Checks` — it reads zero because `O29 Gap` holds no records at all. Read every zero in this repository twice.

This distinction gets lost in rooms. An executive sees *fourteen ungoverned changes* and hears *fourteen things went wrong in the estate*. What it means is *fourteen things changed and nobody wrote them down* — which may be fourteen harmless corrections or one serious unapproved re-architecture. The `Review` sheet exists so a person says which, in writing, within the month.

How.

1. Read the count. Then read the trend against the last three runs, because the level means less than the direction.
2. For each ungoverned row, write one of four dispositions in `Review`: a real difference to raise as an `O29 Gap`; a defensible change to log retrospectively with a reason; an error to reverse; or a recorded authority that differs from practice, which goes to `GB-01`.
3. Log retrospectively where you decided to. Mark the log row as retrospective in `Reason` — do not back-date it.
4. Report the count and the dispositions together, always. The count alone invites the wrong conclusion.
5. Set a target. Zero ungoverned changes is the only honest target, and a practice that is missing it by one or two a month is working.

Form H-2 — ungoverned change review

Feeds the comparison tool's `Review` sheet, and `02 Change Log` (**add**) for anything logged retrospectively. One row per ungoverned change the run found. `Disposition` takes one of four values: `Gap`, `Log retrospectively`, `Reverse`, `To GB-01`. Signed by the Chief Enterprise Architect, monthly.

Sheet	Record ref	Field	What moved	Disposition	Named to
O13 Business Capability	C01.02	Current	1 to 2	Log retrospectively	Example, delete me

Signed by. `GB-05` produces it; `GB-01 Architecture Board` receives it monthly.

Done when. Every ungoverned row has a disposition and a name against it, within the month.

Shows up in. The next run's count, which should fall. If it rises for three runs, the pipeline in steps 11.1 to 11.3 is not being used and no amount of tooling will fix that.

Goes wrong when. The count is reported as an estate health measure. It is a measure of whether your own change process is being followed. Those are different conversations with different people.

READING THE UNGOVERNED COUNT

What is your current ungoverned count, and what was it on the previous two runs?

Of the current ones, how many are real differences and how many are unlogged corrections?

Which person made the change that produced the largest ungoverned finding, and have you spoken to them?

Step 11.7 — Set the cadence, and tie each run to something that already happens

Produce. A dated cadence for the comparison run, the health report and the baseline review.

You need. `057`'s `Cadence` values and the bank's own calendar.

Where it lands. No sheet holds a cadence beyond `057`. Record the Phase H cadence as a decision in `060` so it has the same standing as the gate regime.

Activity	Cadence	Tied to
Comparison run and `Review`	Monthly	The `GB-01 Architecture Board` meeting, which `057` says is monthly [3]
Repository health report, step 11.9	Monthly	The same meeting, same pack
Baseline review, step 11.8	Quarterly	The `GB-02 Investment Committee` meeting [3]
Cross-map refresh, step 11.10	Quarterly	Whichever process forum already reviews process change
Annual architecture review, step 11.13	Annually	The board's IT strategy review, which Joint Standard 1 of 2023 requires annually [6]

How.

1. Attach each activity to a meeting that already exists. A cadence with its own new meeting will be cancelled within two quarters.
2. Put the monthly run before the board pack deadline, not after. A number that arrives late is a number nobody uses.
3. Record the cadence in `060` with a date, so a successor can see when it was agreed.
4. Diarise the quarterly baseline review even in quarters where you expect not to cut one. The review decides; the schedule does not.
5. Reconcile against `019 Contract`'s expiries and `027`'s `Support end` dates once a quarter, because those are the dates that force work regardless of cadence.

Signed by. `GB-01 Architecture Board`.

Done when. Every activity in the table has a named meeting and a date in the next quarter.

Shows up in. `060`, as a dated decision, and in the fact that the monthly pack contains the same sections every month.

Goes wrong when. The cadence is set to the architecture function's convenience rather than to the bank's calendar. Volume 1 chapter 7 lays out the reporting calendar; a Phase H run scheduled into a month-end close will not happen.

Baselines and health

Step 11.8 — Cut a baseline when a decision needs a fixed reference, and not otherwise

Produce. A row in `04 Baselines` (**add**), with a reason.

You need. The decision that needs a fixed reference, and a clean `02 Change Log` up to that point.

Where it lands. `04 Baselines` (**add**) — `Baseline ref`, `Date`, `Label`, `Scope`, `Record count`, `Approved by`, `Decision ref`, `Supersedes`.

`Record count` is the sheet's most useful column, because it makes a baseline falsifiable. The file as delivered would baseline at **878 records across 65 object types and 465 relationship instances** [5]. Any later claim about what the baseline contained can be checked against those three numbers.

When to cut one. A gate decision that will be argued about later. A re-architecting request entering Phase A. A merger or book purchase. The start of a transition architecture — `053 Transition Architecture` carries exit criteria per transition, and those criteria need a fixed reference. A supervisory engagement.

When not to. Monthly, because the calendar said so. Baselining on a schedule produces a stack of references nobody uses and makes every diff ambiguous: a change that crossed three baselines appears three times and is reconciled none of them. Cut a baseline when a decision needs one, and `Supersedes` tells the reader which reference it replaced.

How.

1. Close the change log first. A baseline cut over unlogged changes bakes them in and they become invisible for ever.
2. Record `Scope` precisely: `all object and relationship sheets`, or a named subset. A baseline of unclear scope is not a baseline.
3. Count the records and write the number. If it does not match `ZZ Checks`' `Records` row, find out why before cutting.

4. Get `Approved by` from the body whose decision needs it, and put that decision in `Decision ref`.
5. Archive a copy of the workbook at the baseline, named for the `Baseline ref`. The sheet records the fact; the file is the evidence.

Signed by. The body named in `Approved by`, usually `GB-01` or `GB-02`.

Done when. The baseline's three counts are recorded and a file exists that reproduces them.

Shows up in. Every subsequent comparison run, which names its baseline. A run that cannot name one is not a comparison.

Goes wrong when. A baseline is cut to make a bad diff go away. The ungoverned changes do not disappear; they become the new truth, and the next person to look cannot tell.

BEFORE YOU CUT THE NEXT BASELINE

What decision needs this baseline as a fixed reference? Name it.

Is your change log closed to the cut date — and if not, what is unlogged?

Which baseline does this supersede, and is anything still being measured against that one?

Step 11.9 — Read the three health reports as a standing item, with targets

Produce. Three numbers in the monthly pack, each with a target and a reading.

You need. `ZZ Checks`, `RPT13 Orphan Analysis` and `RPT06 Requirements Traceability`.

Look it up. Volume 1, chapter 18, *Your first ninety days*, *Six numbers that tell you whether the estate is healthy* — the readings this report formalises, and why six is the right number.

Where it lands. No sheet. The report is a page of the monthly pack, built from three sheets.

Reading	Where	Today	Target	What a bad number means
Checks failing their threshold	`ZZ Checks`	8 of 15	0	Each failing row is a named defect with a sheet behind it [5]
Unmanaged interfaces	`ZZ Checks`	19 of 32	0	Integration is happening outside the managed path, so nobody can see what crossed
Interfaces with no logging	`ZZ Checks`	20	0	You cannot answer a supervisory question about data movement
Components past support	`ZZ Checks`, `O27`	3	0	Unpatchable today, not next year
Orphans	`RPT13 Orphan Analysis`	2	under 5	An object nothing references is either mis-modelled or unowned
Requirements reaching a block	`RPT06`	24 of 32	all `Must` rows	A requirement reaching no block is a requirement nothing will deliver [7]

Read the zeros with suspicion. `RPT01 Dashboard` reports `Gaps overdue` as `0` [8], which looks healthy and is not: `O29 Gap` carries **no records at all**, so the zero means nothing has been recorded rather than nothing is late. The same applies to `Vacant architecture roles`, also `0`. A zero computed over an empty sheet is not a result.

`RPT13`'s two orphans are worth the minute they take. `A13 Data Platform`, disposition `Replace`, and `A14 Fraud Detection`, disposition `Retain`, both carry the finding *No business service maps to it* [9]. For `A13` that may be correct, since a data platform serves analytics rather than a business service. For `A14`, a fraud engine with no business service mapped is a modelling gap in `R04`, and the sheet's own closing line puts it well: *An orphan is not always wrong. It is always worth a question.*

How.

1. Build the six-row table once and fill the `Today` column monthly. Do not redesign it; the value is in the series.
2. Where a number moves, name the change request that moved it. A number that improves with no request behind it is an ungoverned change wearing good news.
3. Recompute `ZZ Checks` after every load. Its results are computed over the workbook, so a stale sheet reports last month's estate.
4. Investigate every orphan, every month. Two is a conversation; ten is a modelling problem.
5. Put the table in the same place in the pack every month so the reader finds it without being told.

Signed by. `GB-05 Architecture Function`, received by `GB-01`.

Done when. The table has three consecutive months of readings and somebody has asked about a movement without being prompted.

Shows up in. The pack itself, and in `O04 Measure`, where `MSR-08 Managed interfaces` carries a baseline of 27 per cent and `MSR-15 Architecture compliance rate` 51 per cent [10]. Reconcile your readings against those baselines or correct the baselines.

Goes wrong when. The report becomes a dashboard with twenty numbers. Six readings that somebody acts on beat twenty that nobody reads.

Step 11.10 — Keep volume 1's cross-maps alive when a process or system changes

Produce. A refreshed set of cross-map rows in the relationship sheets, after any change that touches a process or an application.

You need. Volume 1 appendix A, and the relationship sheets `R01` to `R25`.

Look it up. Volume 1, chapter 19, *The cross-maps*, *Keeping these alive* — the maintenance argument. This step is that argument on a cadence.

Where it lands. `R02 Value stream uses Capabilit`, `R03 Capability realised by Busi`, `R04 Business Service supported`, `R05 Application manages Data En`, `R22 Org Unit owns Capability`.

Map by name, never by number. Volume 1's `P-01` … `P-12`, `C-01` … `C-20`, `R-01` … `R-33`, `D-01` … `D-20` and `A-01` … `A-25` are local identifiers in that book. This repository uses its own `Ref` scheme — `C01` to `C12` at level 1 in `O13`, `A01` to `A14` in `O23`. The two look similar enough to be joined by accident and they do not correspond. Match on the name, check the definition, and record the pairing once in a mapping column you add rather than re-deriving it every quarter.

How.

1. After any change request whose `Impact` names an application or a process, re-check the five relationship sheets above.
2. Start from `R04`, which is the map that decides whether an application still supports the service it was built for. Twelve rows today, against fourteen applications in `O23`.
3. Where a capability's `Current` or `Target` moved in `O13`, check `R22` still names the right owning unit. An uplift nobody owns does not happen.
4. Re-run `RPT13 Orphan Analysis` afterwards. A broken cross-map usually shows up as a new orphan before anyone notices the join is gone.
5. Log every cross-map row you add or remove. These are the easiest rows in the repository to edit silently, and the most damaging.

Signed by. `GB-01 Architecture Board` for the application and capability maps; `GB-03 Data Governance Council` where `R05` changes.

Done when. No application in `O23` is absent from `R04`, and no level-1 capability in `O13` is absent from `R22`.

Shows up in. `RPT13 Orphan Analysis`, where the orphan count should stay low. There is no capability-coverage report in this repository — `RPT07` is `Gap Analysis` and it is empty — so build the coverage reading by hand from `R03` and `R04`.

Goes wrong when. The cross-maps are treated as volume 1's content rather than the repository's. They are the joins. An estate whose joins are stale has a catalogue, not a model.

THE CROSS-MAP THAT IS ALREADY STALE

Which application in `O23` is missing from `R04 Business Service supported` — and should it be?

Which volume 1 catalogue did you load by number rather than by name, and what did that cost?

Who notices when a process changes, and how long before the relationship sheets hear about it?

Banking triggers

Step 11.11 — Run a Phase H cycle on each banking trigger, without discretion

Produce. A trigger-to-action table, posted where the business reads things.

You need. `O18 Product`, `O19 Contract`, `O27 Physical Technology Compone`, `O63 Control Objective`, and volume 1, chapter 15 for the instruments.

Look it up. Volume 1, chapter 15, *Regulation, risk and what the regulator inspects*, *The payment system, and who writes the rules now* — who issues a scheme mandate now that PASA's recognition has been withdrawn, and therefore who tells you.

Where it lands. `03 Change Requests` (**add**), with `Against` and `Type` set per the table.

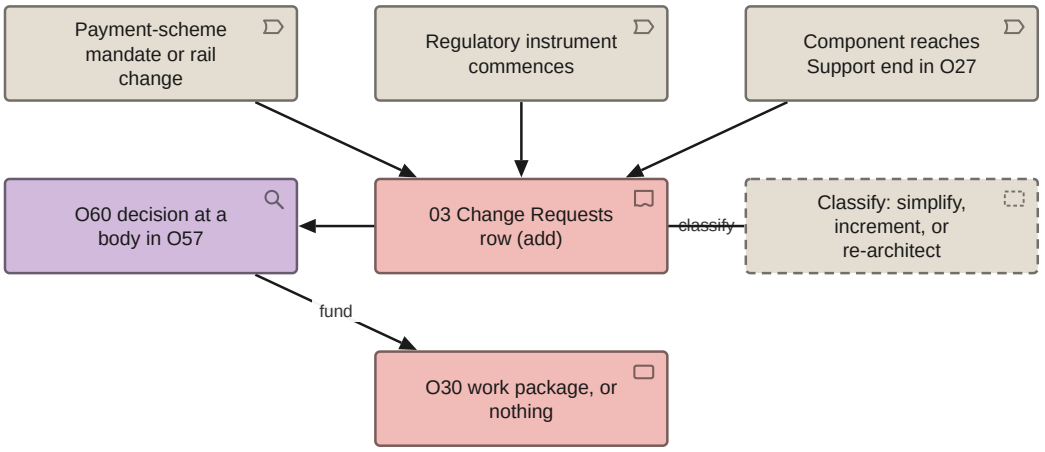

Figure 11.3 Three of the eight banking triggers drawn as what they are: events outside the architecture practice that force a change request, which is then classified and decided. Only a re-architecting classification re-enters the ADM at Phase A; the other two classes end in a work package or in nothing. The trigger is never the architect noticing.

Trigger	First action	Sheets that change	Decided by
A new product launches	Raise a request typed `Addition` against `O18`	`O18 Product`, `R25 Application supports Produc`, `O15 Business Service`, `O13`	`GB-01`, plus `GB-02` if a platform change follows
A payment-scheme mandate or rail change	Raise against `O32 Standard` and the payments applications	`O32`, `R18 Standard applies to Applica`, `O23`, `R06`	`GB-01` with the payments owner
An interchange determination	Reassess pricing-dependent products and the card estate	`O18`, `O23` (`A03 Card Management`), `O04 Measure`	`GB-02`, with Finance
A regulatory instrument commences	Add the obligation and its control objective	`O01 Driver`, `O63`, `O06 Requirement`, `R16`	`GB-01` with Compliance
A core replacement milestone	Re-check the transition's exit criteria before the milestone, not after	`O53 Transition Architecture`, `O66 Architecture Increment`, `O30`	`GB-02`, then `GB-01`
A merger or book purchase	Cut a baseline, then open a re-architecting request	`04 Baselines` **(add)**, `O09`, `O23`, `O20`, `O56`	`GB-02`, then Phase A
A bureau or SACRRA specification change	Reassess the credit data submission and its lineage	`O20 Data Entity`, `O40 Source To Target Mapping`, `O39 Data Migration`	`GB-03 Data Governance Council`
A component reaches `Support end` in `O27`	See step 11.12	`O27`, `O19 Contract`, `O32`	`GB-01`

Three of these are specific to a South African bank and worth naming. The **payment-scheme** trigger now arrives from the SARB and from PayInc rather than from PASA, whose recognition was withdrawn; volume 1 chapter 15 has the detail and the dates. The **interchange determination** is a regulator-set price that moves product economics without any system changing, which is why it is a Phase H trigger and not a pricing exercise. The **SACRRA** trigger is the one architects miss: credit data submission through the Central Data Transmission Hub is a mandatory monthly obligation, and a layout change to it is a data architecture change with a deadline attached.

How.

1. Post the table where the business reads things, not in the repository. The trigger has to be recognised by someone who has never opened the workbook.
2. Name a person in the business per trigger. An architecture function that finds out about the product launch at the launch has no Phase H.
3. For the regulatory trigger, add the obligation to `O63` and the requirement to `O006` in the same sitting. Chapter 12 step 12.7 is the method.
4. For the merger trigger, cut the baseline before anything else. A book purchase with no fixed reference produces a diff nobody can read for two years.
5. Review the table annually. Triggers change: a rail that did not exist three years ago now carries mandated volume.

Signed by. `GB-01 Architecture Board` for all eight; `GB-02` where the trigger moves money.

Done when. Each trigger has a named business person who raises the request, and a date on which you last spoke to them.

Shows up in. `03 Change Requests`' `Raised by` column, which should show business roles and not only the Chief Enterprise Architect.

Goes wrong when. Every trigger is discovered by the architect. All eight are known to somebody three months before they reach the repository. The work of this step is not the table; it is the eight conversations that make the table fire.

Form H-3 — trigger owners and how you hear

Feeds nothing; drives `03 Change Requests`. One row per trigger from step 11.11. The third column is a mechanism — a standing agenda item, a committee paper you are copied on — and the last is the date you held the conversation, not the date you intend to. Signed by the Chief Enterprise Architect.

Trigger	Who raises it	How you hear	Conversation held
New product launches	Head of Product	Product committee paper	Example, delete me

Step 11.12 — Answer a support expiry as a dated change, not a renewal conversation

Produce. A change request per component inside twelve months of `Support end`.

You need. `027 Physical Technology Compone` and `019 Contract`.

Look it up. Volume 1, chapter 14, *Technology providers*, *The cost lines that are not the licence* — what a support-expiry remediation actually costs, before you quote a renewal fee as the alternative.

Where it lands. `03 Change Requests` (**add**), and `029 Gap` where no package covers it.

`027` carries twelve components with a `Support end` date and a `Position` column reading `Supported` or `Past support`. Three read `Past support` today: `PTC-05 HSM estate` (`2026-11`), `PTC-06 Branch server estate` (`2026-12`) and `PTC-10 ATM estate controller` (`2026-10`) [11]. Read those dates against the month you are in. The ATM estate controller's support ended this month, and the HSM estate — the thing that holds card and payment keys — ends next month.

`019 Contract` is the other clock. Ten vendor agreements with `Expiry` dates, the earliest being `CTR-01 Core Banking vendor agreement` at `2027-01` [12]. A core agreement expiring in fourteen months, against a core renewal package running to `2031-08` in `030`, is a gap with a date on it.

How.

1. Sort `O27` by `Support end` and take everything inside twelve months. Three past, and check how many fall inside the next year.
2. Raise one request per component, typed `Addition` or `Reassessment`, with the expiry date in `Description`. The date is the point.
3. Check `R07 Application deployed on Tec` for what runs on it. A component past support with critical applications on it is a different conversation from one with none.
4. Where no `O30` package covers the remediation, raise an `O29 Gap` with `Due` set to the support end date, not to a convenient quarter.
5. Reconcile against `O19`. A component past support whose vendor agreement has also expired has no support path at all, contractual or technical.

Signed by. `GB-01 Architecture Board`, with the Head of Operations where `O62`'s `RSK-04 Legacy platform failure before renewal` applies [13].

Done when. Every `Past support` row in `O27` has a change request and a dated remediation, and `ZZ Checks`' `Platform components past support` row has a plan against its `0` threshold.

Shows up in. `RPT10 Technology Currency`, which is `O27` with the same columns, and `ZZ Checks`.

Goes wrong when. Support expiry is handled as a procurement renewal. The vendor extends support for a fee, the date moves, nothing is architected, and the component is still past its economic life three renewals later. `O01 Driver`'s `DRV-10 Legacy estate` describes exactly that outcome: *Core systems are past their economic life and constrain change* [14].

Step 11.13 — Put four numbers and two decisions in front of the board once a year

Produce. An annual architecture review pack: four numbers, two decisions, one page each.

You need. The twelve monthly health reports, `ZZ Checks`, `RPT09 Regulatory Coverage`, and the comparison runs for the year.

Look it up. Volume 1, chapter 15, *Regulation, risk and what the regulator inspects*, *King IV, and the board's version of the same question* — why this is an obligation and not a courtesy.

Where it lands. `O60 Governance Decision`, for the two decisions the board takes. The pack itself lives outside the repository; every number in it names a sheet.

The four numbers.

1. **Obligations without working evidence.** From `RPT09`: four of seven today [15]. The board's own accountability under Joint Standard 1 of 2023 attaches to this number [6].
2. **Checks failing their threshold.** From `ZZ Checks`: eight of fifteen [5]. One line each, with the trend.
3. **Ungoverned changes for the year.** From the comparison runs. The level and the direction.
4. **Components past vendor support.** From `O27`: three, with the dates and what runs on them [11].

The two decisions. First: does the architecture target still hold, or does something go back to Phase A? Second: what is explicitly accepted for another year — which gap, which dispensation, which past-support component — and who holds that risk by name. A board that accepts nothing has not been told the truth.

How.

1. Build the pack from the monthly reports. If you cannot, the monthly reports were not the right ones.
2. Keep each number to one line and one sheet reference. The appendix can carry the rest.
3. Write the two decisions as motions, so the board can pass or amend them rather than discuss them.
4. Record both in `O60` with dates, and link them to the requests they decide.
5. Hold it against the board's annual IT strategy review, which Joint Standard 1 of 2023 already requires [6]. Do not create a second meeting.

Signed by. The board, with `GB-04 Executive Committee` tabling it.

Done when. `O60` carries two dated board decisions and one of them accepts something named.

Shows up in. `O60`, and in next year's monthly reports, where an accepted risk should appear as a dated item rather than a recurring complaint.

Goes wrong when. The pack becomes a presentation of the architecture. The board does not need the architecture; it needs four numbers it is accountable for and two decisions only it can take.

What Phase H actually produces

Nothing, on a good month. That is the point and it is the hardest thing to defend in a budget conversation. Phase H produces the continued truth of everything else: a repository whose numbers can be trusted, a change log that lets you reconstruct how a value came to be what it is, and a count of ungoverned changes low enough that the count itself is believable.

The repository you were handed has none of that, and its own `ZZ Checks` sheet says so in fifteen rows. Eight fail their threshold, three are counts with no threshold, and four pass. Four work packages are past their end date. Nineteen interfaces are unmanaged. None of those numbers moved on its own, and none of them was logged when it moved.

PHASE H — A REVIEWER'S QUESTIONS

Pick a value in your repository. Can you say who changed it, when, and under what request?

What is your ungoverned-change count, and is it rising, flat or falling across three runs?

Which of the eight banking triggers would you currently find out about on the day it happens?

When did you last cut a baseline, and what decision needed it?

Of the four annual numbers, which one would you least like to show the board — and why is it still true?

THE CHAPTER IN TEN LINES

- Phase H needs `01 Version Control`, `02 Change Log`, `03 Change Requests` and `04 Baselines`, none of which ship with this repository. Step 10.1 specifies them.
- Every field change carries a `Change Ref`, or the comparison tool classifies it as ungoverned change. The rule has no exceptions, including typos.
- One log row per field, not per record, because the tool diffs at field level.
- `O60 Governance Decision` has four columns and no `Date`. Add `Date`, `Rationale`, `Change ref` and `Supersedes` before the sheet can order two decisions on one question.
- Classify by three questions about the repository. A change that moves `R04 Business Service supported` is re-architecting, whatever its author calls it.
- The extracts are produced by hand. This repository has no export sheets.
- A rising ungoverned count is a practice failure, not an estate failure. Report the count and the dispositions together or the room draws the wrong conclusion.
- Cut a baseline when a decision needs a fixed reference. Never on a schedule, and never to make a bad diff disappear.
- Read every zero twice. `Gaps overdue` reads `0` because `O29 Gap` is empty, not because nothing is late.
- Eight banking triggers force a Phase H cycle. Each needs a named person in the business, not an architect who notices.

SOURCES FOR THIS CHAPTER

- **1.** Banking_TOGAF_Metamodel.xlsx, sheet O60 Governance Decision
- **2.** Banking_TOGAF_Metamodel.xlsx, sheet O61 Governance Change Request
- **3.** Banking_TOGAF_Metamodel.xlsx, sheet O57 Governance Body
- **4.** Banking_TOGAF_Metamodel.xlsx, sheets O33 Architecture Contract and O55 Building Block
- **5.** Banking_TOGAF_Metamodel.xlsx, sheet ZZ Checks
- **6.** Joint Standard 1 of 2023 — IT governance and risk management (commenced 15 November 2024; board-approved IT strategy reviewed annually) https://lawlibrary.org.za/akn/za/act/standard/sarb/2023/1/eng@2024-11-15
- **7.** Banking_TOGAF_Metamodel.xlsx, sheet RPT06 Requirements Traceability
- **8.** Banking_TOGAF_Metamodel.xlsx, sheets RPT01 Dashboard and O29 Gap
- **9.** Banking_TOGAF_Metamodel.xlsx, sheet RPT13 Orphan Analysis
- **10.** Banking_TOGAF_Metamodel.xlsx, sheet O04 Measure
- **11.** Banking_TOGAF_Metamodel.xlsx, sheets O27 Physical Technology Compone and RPT10 Technology Currency
- **12.** Banking_TOGAF_Metamodel.xlsx, sheet O19 Contract
- **13.** Banking_TOGAF_Metamodel.xlsx, sheet O62 Delivery Risk
- **14.** Banking_TOGAF_Metamodel.xlsx, sheet O01 Driver
- **15.** Banking_TOGAF_Metamodel.xlsx, sheet RPT09 Regulatory Coverage
- **16.** The Open Group, The TOGAF Standard, 10th Edition — Architecture Development Method, Phase H: Architecture Change Management https://pubs.opengroup.org/togaf-standard/adm/

CHAPTER 12

Requirements Management

A requirement exists to be tested at a gate. Anything that cannot be is a preference in requirement's clothing.

BEFORE YOU START

Requirements Management runs for the life of the practice, not for a phase. It is the discipline that decides whether anything in the repository can be refused at a gate.

- **Chapter 16, *Modelling banking in ArchiMate*** — *A regulation, from driver to control*: the chain this chapter builds in sheets.
- **Chapter 15, *Regulation, risk and what the regulator inspects*** — *What the regulator actually asks to see; CoDI, and a regulation that is really a data specification*.
- **Chapter 13, *The reference architecture*** — *BCBS 239 — the principles that force the data architecture*: a principle that genuinely governs requirements, as a worked case.
- **Chapter 7, *The operating model*** — *The reporting and regulatory calendar*: where most mid-phase requirements actually come from.

Ardent Union Bank is an invention. Its data is real within this workbook and nowhere else.

What you are building, continuously

Step	Produce	Lands in
12.1	An honest reading of the register you inherited	`006 Requirement`
12.2	An acceptance criterion per requirement	`006` `Acceptance criterion` **(add)**
12.3	A spine that joins, with its break repaired	`R08` , `R11` , `R10` , `R26` **(add)**
12.4	The necessity test applied to every row	`003 Objective`
12.5	The governing principle, recorded as a relationship	`005 Principle` , `R27` **(add)**
12.6	Fewer principles, all of them working	`005` `Status`
12.7	An instrument turned into a requirement	`063 Control Objective` , `R16` , `006`
12.8	Instrument-level obligations instead of placeholders	`063`
12.9	A requirement bound to a block and a package	`R09` , `R13`
12.10	The honesty score: traced against tested	`RPT06` , `RPT09` , `059`
12.11	A refusal in writing, with the reason in a column	`006` `Refused` , `Reason` **(add)**
12.12	A register short enough to be read	`006` `Status`

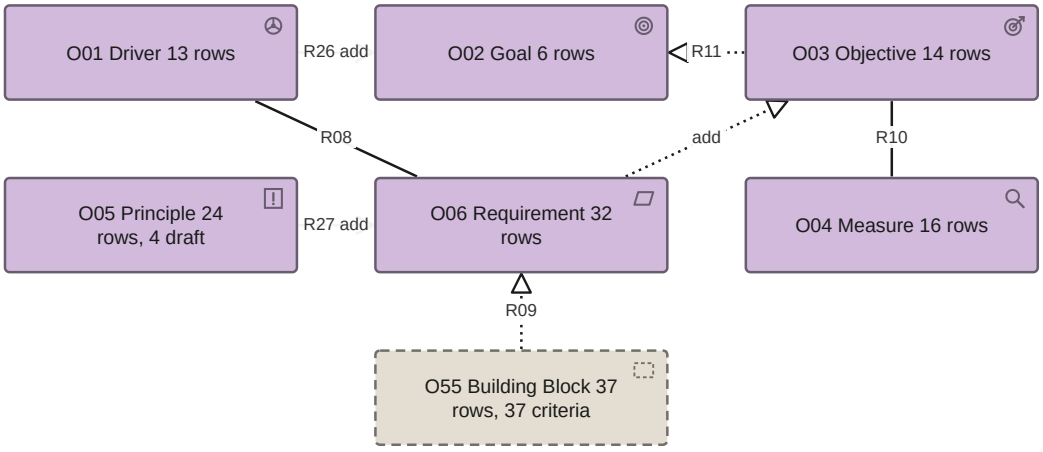

Figure 12.1 The motivation spine as ArchiMate 4 draws it, with the repository's break shown. The driver influences the goal — and nothing in this repository records that influence, because there is no sheet for it. `R26 Driver addressed by Goal` is an addition, not a tab you will find. `004 Measure` draws as an Assessment because ArchiMate 4 has no measure element.

The register

Step 12.1 — Read the register you inherited before you add anything to it

Produce. A one-page reading of `006 Requirement` naming what is there and what is not.

You need. `O06`, `R08 Requirement derives from Dr`, and twenty minutes.

Where it lands.

> **`O06 Requirement`**
>
> Real columns: `Ref`, `Requirement`, `Statement`, `Category`, `Priority`, `Driver`, `Status`.
>
> Add: `Acceptance criterion` (**add**) — step 12.2. `Objective` (**add**) — step 12.4. `Raised` (**add**), `Refused` (**add**) and `Reason` (**add**) — step 12.11.

Thirty-two records [1]. The shape is clean and the content is generated, and you need to know which is which before you load a single row of your own.

Dimension	What the file holds
`Category`	Regulatory 7, Data 8, Integration 6, Business 6, Technology 5
`Priority`	Must 15, Should 12, Could 5
`Status`	Approved 21, Proposed 11
`Driver`	Every row carries one, resolving to `O01 Driver`

Two things are missing and both matter. **There is no acceptance criterion column**, so no requirement in this sheet can be tested at a gate as written. And **there is no link to an objective** — `Driver` is the only upward reference, and a driver is a pressure rather than a commitment. A requirement that reaches a driver has established that somebody cares. It has not established that the bank has agreed to do anything.

Read the statements too. `REQ-001` reads *The architecture must produce evidence that Basel III / IV is met, without manual reconciliation.* That is a good requirement sentence. `REQ-022` reads *Customer Management exposes measures that the executive can act on*, which is a sentence nobody can fail. Six of the 32 are of the second kind — the whole `Business` category.

How.

1. Count the five dimensions above and write the counts down. They are your opening position.
2. Read all 32 statements. Mark each as testable or not. Expect six to eight failures.
3. Add the five columns. Do not fill them yet.
4. Note the duplication: `O06`'s `Driver` column and `R08`'s 32 rows hold the same fact. Keep both, and decide now which is authoritative — the relationship sheet, because that is what anything built over the workbook will read.
5. Log the column addition as a change request. Chapter 11 step 11.1 is the method.

Signed by. `GB-05 Architecture Function`. Reading the register is not a governance act; filling it is.

Done when. The five counts are written down and every statement is marked testable or not.

Shows up in. `RPT06 Requirements Traceability`, which carries the same 32 rows plus a `Reaches a block` column [2]. Read it alongside `O06` from the start.

Goes wrong when. The 32 rows are treated as the bank's requirements. They are a worked example in a sector-shaped repository. Replace them with the client's own, keep the shape, and say so to whoever asks why the count dropped.

Step 12.2 — Write the acceptance criterion, or do not call it a requirement

Produce. A testable criterion against every `Must` and `Should` row.

You need. `O06`, `O55 Building Block`'s `Acceptance criteria` column, and `R09 Requirement satisfied by Bu`.

Look it up. Volume 1, chapter 16, *Modelling banking in ArchiMate*, *The honest limits* — what a model can and cannot carry, and why the criterion belongs in the register rather than in a specification nobody keeps.

Where it lands. `O06 Requirement` — `Acceptance criterion` (**add**).

The criteria already exist in the repository; they are in the wrong sheet. `O55` carries a criterion for all 37 building blocks [3] — *No unmanaged interface to or from Payment Hub*, *No party record without a named owner*, *Real-time payment rails in production for at least one service*. Where `R09` joins a requirement to a block, inherit the block's criterion verbatim rather than writing a new one. Two sentences describing the same test, in two sheets, will diverge within a quarter.

Where no block exists yet, write the criterion anyway and write it as a test. The form that works: a measurable noun, a comparison, and a date or a count. *Managed interfaces to Payment Hub at 100 per cent, measured on* `R06`*'s* `Managed` *column* is a criterion. *Integration improved* is a topic.

Form R-1 — requirement acceptance criteria

Feeds `006 Requirement`. Columns are the sheet's real columns plus `Acceptance criterion` (**add**). `Statement` is left to type straight into Excel — it is too long for this column width. One pass on the `Must` rows usually needs twelve to fifteen rows. Signed by the Chief Enterprise Architect.

Ref	Requirement	Category	Priority	Driver	Acceptance criterion
REQ-016	Core Banking integration managed	Integration	Should	DRV-10	Example, delete me

How.

1. Take the 15 `Must` rows first. Nothing else in the register matters until those are testable.
2. For each, check `R09` for a block. If one exists, copy its `055` criterion exactly.
3. Where no block exists, write the criterion in the form above and raise the missing block as an `029 Gap`.
4. Re-read the seven untestable statements from step 12.1. A requirement whose criterion cannot be written is either badly stated or is not a requirement — decide which, in writing.
5. Set `Status` to `Approved` only once the criterion exists. Eleven rows read `Proposed` today; several should stay there.

Signed by. `GB-01 Architecture Board`, which will be asked to refuse against these criteria at `G1` and `G2`.

Done when. Every `Must` row has a criterion that somebody could fail.

Shows up in. `059 Compliance Assessment`, which takes one verdict per criterion. A register with no criteria produces an empty `059` — which is exactly what the file shows [4].

Goes wrong when. The criterion is written by the team that will be assessed against it. They will write one they can pass. The criterion comes from the architecture; the evidence comes from the team.

THE CRITERION YOU CANNOT YET WRITE

Which of your `Must` requirements has no criterion you can state as a test? Write the sentence you are stuck on.

Is the block missing, or is the requirement badly stated?

Who would have to agree to the criterion before `GB-01` could refuse against it?

The motivation spine

Step 12.3 — Trace the requirement up the spine, and repair the break you find

Produce. A spine that joins from requirement to measure, with the missing relationship added.

You need. `001 Driver`, `002 Goal`, `003 Objective`, `004 Measure`, and the relationship sheets `R08`, `R10`, `R11`.

Look it up. Volume 1, chapter 16, *Modelling banking in ArchiMate*, *A regulation, from driver to control* — the ArchiMate chain this spine is the spreadsheet form of.

Where it lands. The three relationship sheets that exist, plus one that does not:

- `R08 Requirement derives from Dr` — 32 instances, requirement to driver [5].
- `R11 Goal realised by Objective` — 14 instances, six goals to fourteen objectives [6].
- `R10 Objective measured by Measu` — 14 instances, objective to measure [7].
- `R26 Driver addressed by Goal` (**add**) — columns `Driver`, `Driver name`, `Goal`, `Goal name`, matching the house pattern of every other `R` sheet.

The break is between Driver and Goal. Nothing in this repository joins the thirteen drivers in `001` to the six goals in `002`. `002` has no `Driver` column and there is no relationship sheet for it. So the upward path from a requirement stops dead at the driver, and the downward path from a goal stops at the objective. The spine is in two halves that never meet.

That is why `R26` has to be added rather than worked around. Thirteen drivers against six goals is a morning's work — `DRV-01` through `DRV-07`, all seven regulatory drivers, address `GOL-01 Protect the licence to operate`; `DRV-08 Cost pressure` addresses `GOL-02 Reduce cost to serve`; `DRV-11 Data fragmentation` addresses both `GOL-05 Use data as an asset` and `GOL-03 Grow revenue from existing customers`. One driver may address several goals; that is what a relationship sheet is for.

In ArchiMate 4 terms the joins are not all the same relationship, and the sheets should say so. A driver *influences* a goal. An objective *realises* a goal. A requirement *realises* an objective. A measure is not an element at all — ArchiMate has no measure, so `O04 Measure` draws as an Assessment associated with the objective it quantifies. Record the relationship type in the sheet if you intend to generate views from it.

How.

1. Create `R26` with the four columns and fill it from `O01` and `O02`, one row per driver-goal pair.
2. Check `R11` covers every goal. Six goals, fourteen objectives, and `GOL-01` carries two — that is a thin spine for a licence-protection goal.
3. Check `R10` covers every objective. Fourteen of fourteen do [7], which is the healthiest join in this repository.
4. Walk one requirement all the way up and write the chain down. `REQ-005 Evidence Data protection` to `DRV-05 Data protection` to `GOL-01` to `OBJ-01 Every obligation has a named control and evidence` to `MSR-01 Obligations with evidence`, baseline 23 per cent [8].
5. Where the walk fails, record where it failed. A broken spine is a finding, not a modelling inconvenience.

Signed by. `GB-01 Architecture Board`, with the goal owners in `O02` — `GOL-01` is the Chief Executive's, `GOL-05` the Chief Data Officer's [9].

Done when. Every driver appears in `R26`, and a requirement picked at random can be walked to a measure in four hops.

Shows up in. Nothing yet; no report traverses the spine. Build the walk by hand for three requirements and put it in the monthly pack once.

Goes wrong when. The break is papered over by treating `Driver` as a proxy for strategy. A driver is a pressure the bank is under. A goal is something it has decided to do about it. Confusing the two produces requirements that trace upward to nothing anyone has committed to.

Step 12.4 — Apply the necessity test, and admit which requirements are preferences

Produce. An `Objective` value on every requirement, and a list of the rows that cannot have one.

You need. `003 Objective`'s fourteen records and the completed spine from step 12.3.

Where it lands. `006 Requirement` — `Objective` (**add**), holding an `OBJ-` reference.

The test is one question, asked of the objective rather than of the requirement: **would this objective still be met if this requirement were dropped?** If yes, the requirement is optional and its `Priority` should say so. If no, the requirement is necessary and the objective depends on it. If the question cannot be asked because no objective is named, the requirement traces to nothing the bank has committed to — and a requirement that traces to no objective is somebody's preference, however strongly held.

Worked on the file: `REQ-001 Evidence Basel III / IV` against `OBJ-01 Every obligation has a named control and evidence`. Drop the requirement and the objective fails for at least one obligation. So `REQ-001` is necessary, and its `Priority` of `Must` is earned. Now `REQ-028 Adopt Real-time payment rails`, `Priority` `Could`. Which of the fourteen objectives fails without it? None of them mentions payments or availability. That does not make it a bad idea; it makes it an unsponsored one, and the honest response is to find the objective it serves, propose a new one, or leave it at `Could` and stop presenting it as a requirement.

THE REQUIREMENT THAT TRACES TO NOTHING

Name a requirement in your register that no objective depends on. Who asked for it?

Is the honest fix a new objective, a lower priority, or a deletion?

Which of your `Must` rows would survive the necessity test if you asked it out loud at a board?

How.

1. For each `Must` and `Should` row, name the objective and put its `Ref` in `Objective`.
2. Ask the necessity question of each pairing and record the answer in one clause.
3. Where no objective fits, do one of three things and record which: propose an objective to `GB-01`, drop the `Priority` to `Could`, or retire the row per step 12.12.
4. Count the rows with no objective. Expect five to ten in a register you inherited; if it is thirty, the register was built from a wish list.
5. Re-check `R10` afterwards. An objective with no measure cannot answer the necessity question either.

Signed by. `GB-01 Architecture Board`; the objective owner where a new objective is proposed.

Done when. Every `Must` row names an objective, and every row with no objective has a recorded disposition.

Shows up in. `RPT06 Requirements Traceability`, which traces to drivers and blocks but not to objectives. Add the column to your own reading of it.

Goes wrong when. The objective is chosen to justify the requirement. The question is asked of the objective, in that direction, for exactly this reason.

Step 12.5 — Attach the governing principle, and record it as a relationship

Produce. A principle reference against each requirement, in a relationship sheet.

You need. `O05 Principle`'s 24 records and `O06`.

Look it up. Volume 1, chapter 13, *The reference architecture, BCBS 239 — the principles that force the data architecture* — a case where a principle genuinely constrains requirements rather than decorating them.

Where it lands. `R27 Principle governs Requirement` (**add**) — columns `Principle`, `Principle name`, `Requirement`, `Requirement name`, `Effect`. There is no such sheet today; nothing in `R01` to `R25` joins `O05` to anything at all.

That absence is the finding. Twenty-four principles, each with a `Statement`, an `Owner` of `Architecture Board` and a `Status` [10] — and no row anywhere in the repository connects one to a requirement, a design, a standard or a verdict. A principle that nothing references is a well-written sentence.

Some of the 24 would do real work if joined. `PRN-03 Compliance is Designed, Not Added` reads *Obligations are architected in, not bolted on* and governs all seven `Regulatory` requirements. `PRN-10 Data is Shared` reads *Data is shared through published interfaces, not copies* and governs the eight `Data` requirements and the six `Integration` ones. `PRN-13 Data Ownership is Individual` governs every data-mastering requirement and is the principle that refuses a committee as an owner. `PRN-17 Control Technical Diversity` governs the five `Technology` adoption requirements.

The `Effect` column is what keeps this honest: `Constrains`, `Requires` or `Forbids`. A principle that produces no constraint, requirement or prohibition on any row is not governing anything.

How.

1. Create `R27` and work down `O06`, not down `O05`. Start from the requirement and ask which principle constrains it.
2. Fill `Effect` for every row. If you cannot, delete the row — the join was decorative.
3. Note which principles never appear. That list is step 12.6's input.
4. Check `Status`. Four of the 24 read `Draft` — `PRN-05`, `PRN-10`, `PRN-15`, `PRN-20` [10] — and a draft principle cannot carry a refusal at a gate. `PRN-10 Data is Shared` being draft

while eight data requirements depend on it is the sharpest example.

5. Take the four drafts to `GB-01` for approval or deletion. Leaving them draft is the worst of the three options.

Signed by. `GB-01 Architecture Board`, which `O05` already names as the owner of all 24.

Done when. Every `Must` requirement has at least one `R27` row with a non-empty `Effect`.

Shows up in. `R27`'s row count against `O05`'s 24. A principle with no rows is a candidate for retirement.

Goes wrong when. Every requirement is joined to `PRN-01 Primacy of Principles`, which reads *Applies to every part of the enterprise without exception*. That is a meta-principle and it governs nothing specific. Joining to it is how a practice pretends to have done this step.

Step 12.6 — Retire the principles that govern nothing

Produce. A shorter principle set, every member of which appears in `R27`.

You need. `R27` from step 12.5 and `O05`.

Where it lands. `O05 Principle` — `Status`, which already takes `Approved` and `Draft`. Add `Retired` as a third value rather than deleting rows, so a past decision taken under a principle stays readable.

Twenty-four principles is more than a practice can hold in its head, and the test is not elegance but use. Keep a principle if it appears in `R27` with an `Effect`, or if it has been cited in an `O59` verdict or an `O61` dispensation. Retire it otherwise.

`O05`'s set is recognisably TOGAF's starter list with a banking overlay. `PRN-01` to `PRN-04` are the generic four; `PRN-09` to `PRN-14` are the data principles and are the strongest of the set; `PRN-19` to `PRN-23` are security and include `PRN-23 Models are Governed`, which reads *Analytical and AI models are governed like any other asset* and will be the most argued-over row in the sheet within two years.

How.

1. List the principles with no `R27` rows. Take the list to `GB-01` as a single decision, not as 24 conversations.
2. For each, ask whether it has ever been cited in a refusal. If not, retire it.
3. Set `Status` to `Retired` and record the decision in `O60 Governance Decision` with a date.
4. Approve or delete the four `Draft` rows in the same sitting.
5. Keep the survivors to a number somebody can recite. Twelve is defensible; twenty-four is a document.

Signed by. `GB-01 Architecture Board`, in one decision.

Done when. No principle reads `Draft`, and every `Approved` principle appears in `R27`.

Shows up in. `R27` and `O05` read together. The ratio of principles to joins is the measure.

Goes wrong when. Retirement is resisted because the set looks thorough. A thorough principle set that nobody cites costs a page of the governance pack and buys nothing.

Regulation as requirements

Step 12.7 — Turn a regulatory instrument into a requirement, not a compliance memo

Produce. One chain per obligation: instrument, control objective, architecture requirement, building block, verdict.

You need. `O63 Control Objective`, `R16 Control Objective met by Co`, `O06`, `O55` and `O01`.

Look it up. Volume 1, chapter 15, *Regulation, risk and what the regulator inspects*, *What the regulator actually asks to see* — the end of the chain, which is where this one has to land.

Where it lands. `O63 Control Objective` — `Ref`, `Obligation`, `Source`, `Control objective`, `Evidence` — and `R16`, plus a requirement row in `O06`.

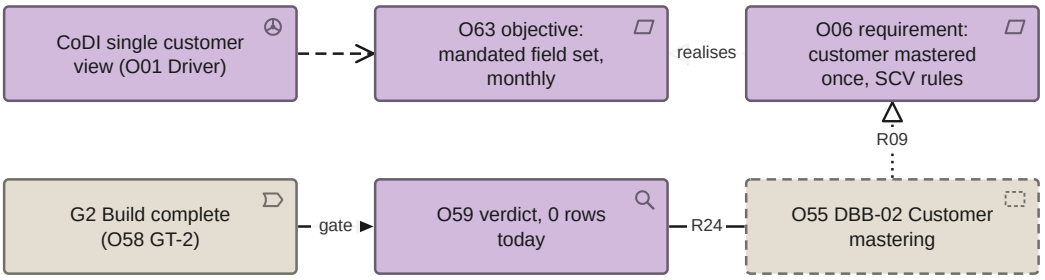

Figure 12.2 One South African instrument end to end. CoDI's single customer view is a regulator-specified data structure, not a principle: the obligation is a driver, the control objective and the architecture requirement are both Requirements with the second realising the first, a data building block realises the requirement, and the verdict is taken at `G2`. Four of seven control objectives in `O63` have no working mechanism today, and `O59` holds no verdicts at all.

The chain, and the reason it matters: an obligation arrives as law. A **control objective** states what must be true for the obligation to be met. An **architecture requirement** states what the architecture must do for the control objective to hold. A **building block** is what delivers it. A **verdict** at `G2` says whether it did. Break the chain anywhere and the obligation becomes a compliance memo — a document asserting that the bank complies, with nothing in the architecture that would notice if it stopped.

The repository has the sheets and not the chain. `O63` holds seven obligations whose `Control objective` column reads *Control that evidences Basel III / IV.* and six variants of that sentence [11]. `R16` pairs each with one `O41 Security Control` row, positionally: `CO-01 Basel III / IV` met by `SEC-01 Identity lifecycle` [12]. And `O06` carries `REQ-001` to `REQ-007`, one per

obligation, each reading *The architecture must produce evidence that X is met, without manual reconciliation* — which is the right sentence at the wrong altitude. It is a programme statement, not a testable requirement.

How.

1. Take one obligation. Write the control objective as what must be true, in the bank's own words, with the instrument's section where one applies.
2. Write one or more requirements in `O06` beneath it. For data protection that is retention per data category, a lawful transfer basis per destination, and masking outside production — three requirements, not one.
3. Join each requirement to the obligation. `R16` joins the control objective to a control; add the requirement reference to `O06`'s `Statement` or create the join explicitly.
4. Bind each requirement to a block in `R09` — step 12.9 — so something delivers it.
5. Set the gate. A regulatory requirement is tested at `G2` with evidence, never at `G1` with a design intention.

Signed by. `GB-01 Architecture Board` with the compliance function, who own the interpretation of the instrument. The architect owns the requirement, not the law.

Done when. The obligation can be walked to a verdict in five hops, and every hop is a row.

Shows up in. `RPT09 Regulatory Coverage`, where `Evidence` should read `Evidenced` only when an `O59` verdict says so. Three of seven read `Evidenced` today with no `O59` rows in existence [13].

Goes wrong when. The obligation is handled by writing a policy. The policy is necessary and it is not architecture. The test is whether anything in the estate would behave differently.

Step 12.8 — Replace the seven generic obligations with instrument-level rows

Produce. An `O63` that names South African instruments instead of categories.

You need. `O63`, `O01 Driver`, and volume 1, chapter 15 for every instrument below.

Look it up. Volume 1, chapter 15, *Regulation, risk and what the regulator inspects*, *CoDI, and a regulation that is really a data specification*, and its subsection *The single customer view* — the clearest case of an instrument that is a data specification wearing a regulation's clothes.

Where it lands. `O63 Control Objective`, which grows from seven rows to roughly twenty.

The seven rows as shipped are categories: `Basel III / IV`, `IFRS 9`, `AML / CFT and sanctions`, `Payment scheme rules`, `Data protection`, `Operational resilience`, `Conduct and fair treatment` [11]. Categories cannot be tested. Instruments can. The table below is the starting set for a South African bank; volume 1 chapter 15 carries the regulation itself and this workbook does not repeat it.

Instrument	Control objective, in outline	Evidence a `062` would accept	Volume 1 chapter 15 section
Banks Act 94 of 1990 and the BA returns	Returns submitted on the Authority's determined content and deadlines	Submission log with lineage to `040`	*The BA returns, and the change that moved them out of the law*
Basel III, with the output floor	Risk-weighted assets computed on both approaches and reconciled	Both runs, same reporting date, one reconciliation	*Basel III, the output floor, and the calculation you run twice*
IFRS 9	Expected credit loss reproducible for a past reporting date	Model version, inputs and output, re-run	*IFRS 9, and the most data-hungry accounting standard a bank faces*
FICA 38 of 2001	Customer due diligence refreshed on a risk-based cycle	Dated CDD record per party, with the risk rating that set the cycle	*FICA, the risk-based approach, and life after the grey list*
Sanctions screening	Screening and reporting inside the statutory clock	Screening log with timestamps	*Sanctions screening, and the twenty-four-hour clock*
National Credit Act	Affordability assessment retained and reproducible	The assessment as it was at decision time	*The National Credit Act, which turns lending into a retained record*
POPIA	Lawful basis, retention and transfer basis per data category	`048 Environment Personal data present`, plus retention rules in `043`	*POPIA, with the banking bite*
CoDI single customer view	Monthly submission on the regulator's mandated field set	The submission, with aggregation and joint-account rules evidenced	*CoDI, and a regulation that is really a data specification*
Conduct Standard 3 of 2020	Complaints categorised to the prescribed categories	Complaint records with category, and the dormancy notifications	*The Conduct Standard for Banks*

Two of these deserve a sentence because architects consistently under-rate them. **CoDI's single customer view is a regulator-specified data structure**, not a design principle — aggregation rules, joint-account shares, beneficiary look-through, a mandated field set and a monthly submission. It belongs in `020 Data Entity` and `043 Master Data Rule`, not in a policy. And the **National Credit Act** turns a lending decision into a record the bank must be able to reproduce years later, which is a bitemporality requirement on the credit data, not a workflow requirement.

Form R-2 — obligation to requirement

Feeds `O63 Control Objective` and `O06 Requirement`. The first four columns are `O63`'s real columns; the last is the `O06` reference you create in the same sitting. A first pass on the South African instrument set needs about twenty rows, so continue on a second copy. Signed by the Chief Enterprise Architect and the compliance function.

Ref	Obligation	Source	Control objective	Evidence	O06 ref
CO-08	POPIA s.14	1 July 2021	Example, delete me	No mechanism	

How.

1. Keep the seven category rows as parents if that helps, and add instrument rows beneath them. Renumber `CO-` cleanly; do not reuse a retired reference.
2. Put the instrument's commencement date in `Source`. A control objective with no date cannot be managed against a transition.
3. Write `Evidence` as what exists, not what should. `No mechanism`, `In breach` and `Evidenced` are the three values the sheet already uses [11].
4. Add the requirement rows in `O06` in the same sitting. An obligation added without a requirement is a longer list, not more control.
5. Re-run `RPT09` and expect the failing count to rise before it falls. Twenty instruments honestly assessed look worse than seven categories optimistically assessed, and it is the first true number the bank has had.

Signed by. `GB-01 Architecture Board` with compliance; the Chief Compliance Officer owns all seven regulatory drivers in `O01` today [14], which is a concentration worth splitting as you go.

Done when. No row in `O63` names a category where an instrument exists, and every row has a commencement date in `Source`.

Shows up in. `RPT09 Regulatory Coverage`, longer and worse, which is the point.

Goes wrong when. The expanded list is built from a compliance register without being joined to requirements. Then the architecture practice owns a longer list it cannot act on, and compliance owns a list it thinks architecture is handling.

THE INSTRUMENT YOU HAVE NOT TURNED INTO A REQUIREMENT

Which South African instrument applies to your bank and has no row in `O63` at all?

For one obligation you do hold: name the control objective, the requirement and the block.

Which obligation is recorded as `Evidenced` on the strength of a policy rather than a verdict?

Into delivery, and back out

Step 12.9 — Bind the requirement to a building block and a work package

Produce. An `R09` row per requirement, re-argued rather than inherited.

You need. `R09 Requirement satisfied by Bu`, `O55`, and `R13 Work Package delivers Build`.

Where it lands. `R09 Requirement satisfied by Bu` — `Requirement`, `Requirement name`, `Building block`, `Block name`, `Traced`.

Every row in `R09` as shipped is a positional artefact and must be re-argued. The sheet pairs `REQ-001` with `BBB-01`, `REQ-002` with `BBB-02`, and so on down both sheets in order [15]. So `REQ-001 Evidence Basel III / IV` is recorded as satisfied by `BBB-01 Customer Management capability uplift`, and `REQ-005 Evidence Data protection` by `BBB-05 Payments & Settlement capability uplift`. Neither pairing means anything. The generator walked two lists in parallel.

This is the most dangerous sheet in the repository, because it looks right. Thirty-two rows, both ends named, a `Traced` column with 24 `Yes` and 8 `No`, and a report computing off it. A reader who does not open `O55` will believe the Basel requirement has a home.

The `Traced` column is useful and should be kept honest: `Yes` means the block genuinely satisfies the requirement and the block is in `R13` against a package. `No` means one of those is missing, and `RPT06`'s `Reaches a block` column carries the same eight `No` values [2].

How.

1. Clear `R09` or mark every row unverified. Then rebuild it, one requirement at a time, from the requirement's own criterion.
2. For each requirement, find the block whose `O55 Acceptance criteria` matches the test you wrote at step 12.2. `REQ-009 Customer mastered once` belongs with `DBB-02 Customer mastering`, whose criterion reads *No customer record without a named owner*.
3. A requirement may need several blocks. Add rows; do not force a one-to-one.
4. Check each block appears in `R13 Work Package delivers Build` against a package in `O30`. `R13`'s 37 rows are allocated round-robin and need the same re-argument as `R09`. If no package genuinely carries the block, the requirement has a design and no funding, and that is an `O29 Gap`.
5. Set `Traced` only when both the block and the package exist. A `Yes` that means *we drew a line* is worse than a `No`.

Signed by. `GB-01 Architecture Board`, as part of `G1`.

Done when. Every `Must` requirement has at least one `R09` row you can defend out loud, and `Traced` is honest.

Shows up in. `RPT06 Requirements Traceability`, whose `Reaches a block` column should move from 24 of 32 to every `Must` row [2].

Goes wrong when. The sheet is accepted as delivered. It will pass a cursory review, satisfy a traceability question, and fail the first time somebody asks which block actually evidences Basel.

Step 12.10 — Read the gap between traced and tested as the practice's honesty score

Produce. Two numbers in the monthly pack, with the distance between them named.

You need. `RPT06 Requirements Traceability`, `RPT09 Regulatory Coverage`, and `O59 Compliance Assessment`.

Where it lands. No sheet. Two readings, one line each.

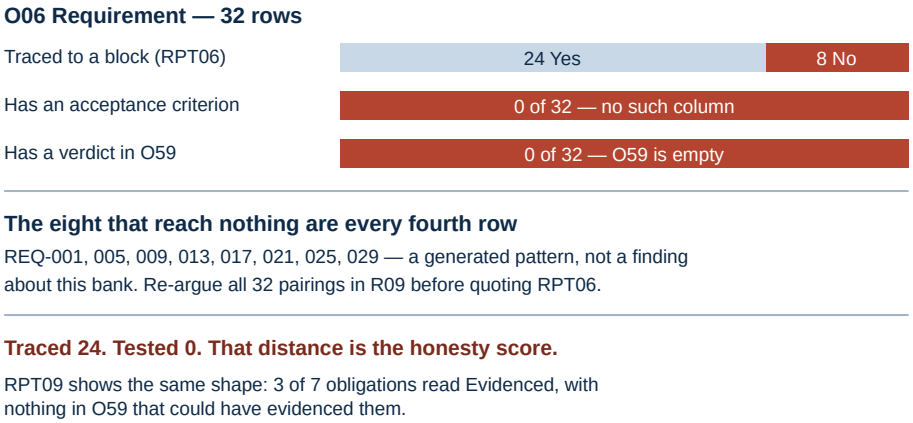

Figure 12.3 The honesty score, from the shipped file. Twenty-four of thirty-two requirements reach a building block in `RPT06`; none has a verdict against its criterion, because `059 Compliance Assessment` holds no records and `O06` has no acceptance criterion column. Tracing is drawing lines. Testing is asking for proof.

Traced is how many requirements reach a building block: 24 of 32 in `RPT06` [2]. **Tested** is how many have a verdict against their criterion in `059`: zero, because `059` has no records [4]. The distance between 24 and 0 is the honesty score, and it is the single most useful number this chapter produces.

A traced requirement has a home. A tested requirement has evidence. Practices accumulate the first and report it as though it were the second, because tracing is drawing lines and testing is asking a delivery team for proof. `RPT09` shows the same pattern on the regulatory side: three of seven obligations read `Evidenced` [13] with nothing in `059` that could have evidenced them.

Reading	Where	Today	Target
Requirements traced to a block	`RPT06 Reaches a block`	24 of 32	every `Must` row
Requirements with a verdict	`059` row count	0	every `Must` row, at `G1` or `G2`
Obligations with working evidence	`RPT09 Evidence`	3 of 7, unverified	every obligation
`Must` rows with an acceptance criterion	`O06` **(add)**	0	15 of 15

How.

1. Compute the four readings monthly, from the sheets named. None takes more than a minute once the columns exist.
2. Report traced and tested on adjacent lines, always. Separating them is how the first gets mistaken for the second.
3. Where tested lags traced by more than a quarter, name the packages that owe verdicts rather than the requirements that lack them.
4. Reconcile against `004 Measure`. `MSR-01 Obligations with evidence` has a baseline of 23 per cent [8] and `RPT09` implies 43 per cent. One of those is wrong, and finding out which is

a useful morning.

5. Put the honesty score in the annual board pack — chapter 11 step 11.13 — as one of the four numbers.

Signed by. `GB-05 Architecture Function`, received by `GB-01`.

Done when. The four readings have three consecutive months of history.

Shows up in. `RPT06` and `O59`. Both are small sheets and neither lies, which is why they are the right pair.

Goes wrong when. Traceability is reported as compliance. A hundred per cent traced register with an empty `O59` means every requirement has been assigned to somebody and none has been checked.

TRACED AGAINST TESTED

What are your four readings this month, and which sheet did each come from?

Which work package owes the most verdicts, and when did you last ask for them?

Does `O04 Measure`'s baseline agree with your `RPT09` reading? If not, which is wrong?

Step 12.11 — Assess a requirement that arrives mid-phase, and refuse it in writing

Produce. A dated intake decision on every requirement that arrives after the phase it belongs to.

You need. `O06`, the current phase from `O31 Deliverable`, and `O30 Work Package`.

Look it up. Volume 1, chapter 7, *The operating model*, *The reporting and regulatory calendar* — where most mid-phase requirements come from, and why they are predictable rather than sudden.

Where it lands. `O06 Requirement` — `Raised` (**add**), `Refused` (**add**) and `Reason` (**add**). `Refused` takes a date or is empty; `Reason` is one sentence.

Mid-phase requirements are not an exception; they are the normal case. The question is never whether to accept one but what it costs. Four tests, in order, and the first failure is the answer:

1. **Does it have a criterion?** If not, it is not a requirement yet. Send it back; do not refuse it.

2. **Does it trace to an objective?** Step 12.4's test. If not, it is a preference and goes to `GB-01` as a proposal for a new objective, not as a requirement.
3. **Is it regulatory with a commencement date?** Then it is accepted, whatever the phase, and something else moves. Volume 1 chapter 15 carries the dates; they are published years ahead.
4. **Does it change a criterion a contract already binds?** Then it is re-architecting under chapter 11 step 11.4 and re-enters at Phase A.

When to refuse. When the requirement fails test 2 and its sponsor will not take it to `GB-01` for an objective. When accepting it would break an `033` contract mid-delivery and the sponsor will not fund the renegotiation. When it duplicates a row already in the register at a different priority. Refuse in the column, with a date and a reason, and never by not replying.

How.

1. Date every arrival in `Raised`. Without it you cannot tell a mid-phase requirement from an original one six months later.
2. Run the four tests and record which one it failed.
3. Where you refuse, fill `Refused` with the date and `Reason` with one sentence that names the test it failed.
4. Where you accept mid-phase, say what moves. A requirement accepted with no consequence was not assessed.
5. Review refusals quarterly at `GB-01`. A refusal nobody revisits becomes a grudge.

Form R-3 — mid-phase requirement intake

Feeds `006 Requirement`'s added `Raised`, `Refused` and `Reason` columns. `Failed test` takes 1, 2, 3 or 4 from the four tests above, or `none` where the requirement is accepted. A quarter rarely needs more than eight rows. Signed by the chair of `GB-01 Architecture Board`.

Raised	Requirement	Sponsor	Failed test	Decision	What moves
2026-11-14	Example, delete me	Head of Payments	2	Refused	

Signed by. `GB-01 Architecture Board`. An architect may send a requirement back for a criterion; only the board refuses one.

Done when. Every row in `O06` has a `Raised` date, and every refusal has a date and a reason.

Shows up in. `O06` filtered on `Refused`. That filter is a quarterly agenda item and a useful thing to show a sponsor who believes architecture never says no.

Goes wrong when. Refusal happens by silence. The requirement stays `Proposed` for a year, everybody assumes somebody is handling it, and it surfaces at a gate as a surprise.

Step 12.12 — Retire requirements, so the register stays worth reading

Produce. A register where every row is live, and a record of the ones that are not.

You need. `O06` and `O59`.

Where it lands. `O06 Requirement` — `Status`, which takes `Approved` and `Proposed` today. Add `Met`, `Superseded` and `Retired` as values rather than deleting rows.

A requirement leaves the register for one of four reasons and the record should say which. **Met** — a `G2` verdict in `O59` reads `Compliant` against its criterion, so the row is history rather than work. **Superseded** — a later requirement covers it, and the later reference goes in the earlier row's `Reason`. **Refused** — step 12.11. Or the **driver went away**: the obligation was repealed, the product withdrawn, the goal dropped.

The fourth case is handled worst. When a driver in `O01` changes, nothing walks down the spine to find the requirements that depended on it. Make that walk part of the Phase H trigger table: a request against `O01` or `O02` always lists the affected `O06` rows in its `Impact`.

Keep the live register small enough to read in a sitting. Thirty-two rows is readable; three hundred is a database nobody opens, and a register nobody opens cannot refuse anything.

How.

1. After every `G2`, set `Status` to `Met` on every requirement whose criterion got a `Compliant` verdict.
2. Quarterly, read the `Proposed` rows. Eleven today [1]. A row that has been `Proposed` for a year is either refused or forgotten; decide which.
3. When a driver changes, walk `R08` down to the requirements and dispose of each.
4. Never delete a row that a decision in `O60` or a verdict in `O59` references. Retire it and keep the reference resolvable.
5. Report the live count monthly alongside the honesty score. A rising live count with a flat tested count is a register running away from the practice.

Signed by. `GB-05 Architecture Function` for `Met`; `GB-01 Architecture Board` for `Retired` and `Superseded`.

Done when. No row has been `Proposed` for more than two quarters without a disposition.

Shows up in. `RPT06`, which carries every row in `006` and will shrink as rows retire. A traceability report that only ever grows is not being maintained.

Goes wrong when. Requirements are kept because deleting them feels like losing work. The cost is paid by every future reader who has to decide which rows still matter, and by the gate that assesses against a criterion nobody meant any more.

The one test for this discipline

Pick a requirement. Walk it up: criterion, objective, goal, driver. Walk it down: building block, work package, project, verdict. Eight hops, eight rows, all of them in sheets named in this chapter. If the walk completes, the requirement is managed. If it breaks, the break is the work.

The repository you were handed completes neither walk. Upward, it stops at the driver because nothing joins `001` to `002`. Downward, it stops at the building block because `059` is empty and `R09`'s pairings are positional. Both ends are a morning's work each, and nothing else in this volume matters as much.

REQUIREMENTS MANAGEMENT — A REVIEWER'S QUESTIONS

Pick one `Must` requirement. Walk it up to a measure and down to a verdict. Where did it break?

How many of your requirements have an acceptance criterion somebody could fail?

Which principle in `005` governs nothing, and why is it still `Approved`?

Traced against tested: what are your two numbers, and what is the distance?

Name the last requirement you refused, the date, and the reason you wrote in the column.

THE CHAPTER IN TEN LINES

- `O06 Requirement` has 32 rows, no acceptance criterion column and no link to an objective. Add `Acceptance criterion` and `Objective` before anything else.
- The criteria already exist, in `O55 Building Block`. Inherit them through `R09`; never write a second sentence for the same test.
- The motivation spine breaks between Driver and Goal. Nothing joins `O01` to `O02`, so add `R26 Driver addressed by Goal`.
- The necessity test is asked of the objective: would it still be met without this requirement? A requirement that traces to no objective is somebody's preference.
- Nothing in `R01`–`R25` joins `O05 Principle` to anything. Add `R27 Principle governs Requirement` with an `Effect` column, or admit the principles are decoration.
- Four of 24 principles read `Draft`, including `PRN-10 Data is Shared`, on which eight data requirements depend. A draft principle carries no refusal.
- An instrument becomes architecture through four hops: obligation, control objective, requirement, building block — then a verdict at `G2`.
- `O63`'s seven rows are categories, not instruments. Replace them with the South African instruments and expect `RPT09` to look worse before it looks better.
- `R09`'s 32 pairings are positional: `REQ-001 Evidence Basel III / IV` is recorded as satisfied by `BBB-01 Customer Management capability uplift`. Re-argue every row.
- Traced 24, tested 0. The distance between those two numbers is the practice's honesty score, and it belongs in the board pack.

SOURCES FOR THIS CHAPTER

- **1.** Banking_TOGAF_Metamodel.xlsx, sheet O06 Requirement
- **2.** Banking_TOGAF_Metamodel.xlsx, sheet RPT06 Requirements Traceability
- **3.** Banking_TOGAF_Metamodel.xlsx, sheet O55 Building Block
- **4.** Banking_TOGAF_Metamodel.xlsx, sheets O59 Compliance Assessment and R24 Compliance Assessment tests, both 0 records
- **5.** Banking_TOGAF_Metamodel.xlsx, sheet R08 Requirement derives from Dr
- **6.** Banking_TOGAF_Metamodel.xlsx, sheet R11 Goal realised by Objective
- **7.** Banking_TOGAF_Metamodel.xlsx, sheet R10 Objective measured by Measu
- **8.** Banking_TOGAF_Metamodel.xlsx, sheet O04 Measure
- **9.** Banking_TOGAF_Metamodel.xlsx, sheets O02 Goal and O03 Objective
- **10.** Banking_TOGAF_Metamodel.xlsx, sheet O05 Principle
- **11.** Banking_TOGAF_Metamodel.xlsx, sheet O63 Control Objective
- **12.** Banking_TOGAF_Metamodel.xlsx, sheets R16 Control Objective met by Co and O41 Security Control
- **13.** Banking_TOGAF_Metamodel.xlsx, sheet RPT09 Regulatory Coverage
- **14.** Banking_TOGAF_Metamodel.xlsx, sheet O01 Driver
- **15.** Banking_TOGAF_Metamodel.xlsx, sheet R09 Requirement satisfied by Bu
- **16.** The Open Group, The TOGAF Standard, 10th Edition — Architecture Development Method, Requirements Management https://pubs.opengroup.org/togaf-standard/adm/
- **17.** CoDI, Single customer view calculations document (joint accounts, beneficiary accounts, aggregation rules) https://www.resbank.co.za/content/dam/sarb/what-we-do/deposit-insurance/SCV%20calculations%20document.pdf
- **18.** Conduct Standard 3 of 2020 (Banks), FSCA — prescribed retail complaint categories and dormancy notification https://lawlibrary.org.za/akn/za/act/standard/fsca/2020/3/eng@2020-07-03

APPENDIX A

The sheet-by-sheet reference

One hundred and five sheets, what each holds, the Ref prefix in column A, the two sheets that are empty, and the eleven things this repository does not have.

You are holding 105 sheets. Twelve of them matter every week, thirty matter in the phase you are in, and the rest you open once, read, and leave alone. This appendix tells you which is which, what each sheet holds, what the first column actually contains, and when in the ADM you fill it.

Ardent Union Bank is an invented South African bank. Its data is real inside this workbook and nowhere else. Where this appendix quotes a number, the number is the one in the file as shipped.

The shape of the repository in one page

Six families of sheet, in the order they sit along the tab bar [1].

Family	Sheets	Count	What it is for	Do you type in it?
Index	`00 Index`	1	The repository's catalogue of itself: what it is, how it is structured, and a record count per sheet	Only the counts, and only when you change what is in the sheets
Objects	`001 Driver` to `066 Architecture Increment`	64	One sheet per metamodel entity. `Ref` in column A, the name in column B, always	Yes — this is where the architecture lives
Extension	`EXT Industry Extension`	1	Six object types banking needs beyond the TOGAF core	Yes, and carefully — see the rule below
Relationships	`R01 Capability decomposition` to `R25 Application supports Produc`	25	One sheet per relationship type, each named after what the link means rather than after its endpoints	Yes — and it is the half people skip
Reports	`RPT01 Dashboard` to `RPT13 Orphan Analysis`	13	Presentations over the object and relationship sheets. Appendix B takes them one at a time	No — read only
Checks	`ZZ Checks`	1	Fourteen integrity counts against the threshold a healthy estate would show	No — read before and after every phase

One plus sixty-four plus one plus twenty-five plus thirteen plus one is 105.

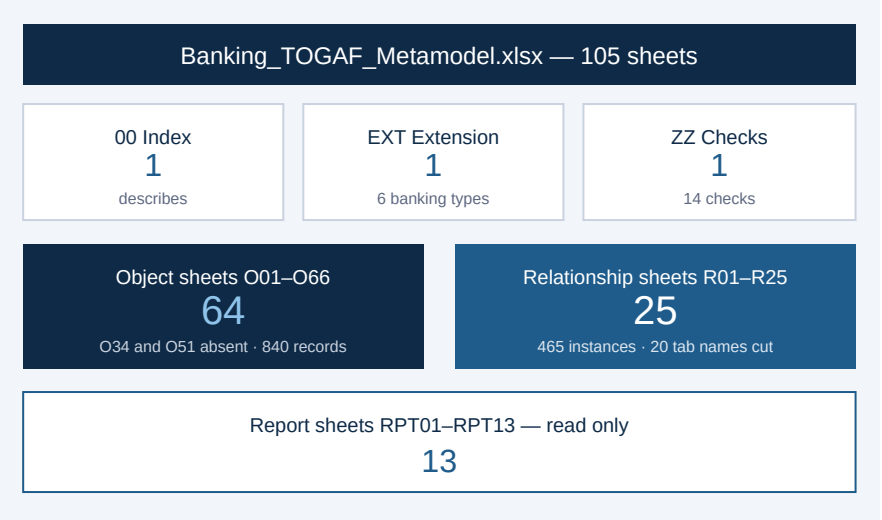

Figure 13.1 The six families of sheet. Objects and relationships are typed and typed into; reports and checks compute; the index and the extension describe. One hundred and five sheets in all.

`00 Index` describes itself as "65 object types · 25 relationship types". **Both are right**, which is worth saying because it is not true of every repository in this series. The 65 is the 64 `O` sheets plus `EXT Industry Extension` counted once, and the index itemises all 65 by name with a record count against each. The 25 is exactly the number of `R` sheets. If you have worked another industry repository and learned to distrust the header, unlearn it here — about the counts of *types*, this file is honest [1].

Two other claims on the same header are not.

> **TWO NUMBERS IN THE DOCUMENTATION DO NOT SURVIVE ARITHMETIC**
>
> **The record count is overstated by 38.** `00 Index`, `RPT01 Dashboard` and `ZZ Checks` all state 878 records. The index's own itemised list sums to **846** — 840 rows across the 64 `O` sheets plus six on `EXT`. Each itemised figure is correct against its sheet; the total is not. Quote 846, or 840 for the core only [1] [3].
>
> **"Every type carries records" is false.** `O29 Gap` and `O59 Compliance Assessment` hold nothing, which the index itself records as "0 records" on both rows. Sixty-three of sixty-five are populated, and `R12 Gap closed by Work Package` and `R24 Compliance Assessment tests` are header-only in consequence [6].

The object numbering has two holes. **`034` and `051` do not exist.** No sheet is named either, nothing references them, and `00 Index` does not itemise them. Nothing is missing from the model, but a macro that walks `O01` to `O66` will fail twice. Walk the sheet names, never the numbers — `R` and `RPT` have no holes today and will acquire them the moment somebody extends the file [2].

The object catalogue

Six tables, grouped by the domain label `00 Index` puts against each sheet. Four columns need a word of explanation before you read them.

`Ref` prefix is read off column A of the shipped file. There is no `Code` column on `00 Index` to disagree with, which removes one trap, but the prefixes are not derivable from the sheet names either and several are irregular — where one has no hyphen, no hyphen is correct. **The columns that matter** lists the columns carrying judgement; `Ref` and the name are on every sheet. **ADM phase · year one?** gives the phase that fills the sheet, then a judgement for a practice standing up from nothing: **yes** means it earns its keep inside twelve months, **if relevant** means fill it when the phase forces it, **later** means a mature practice needs it and a new one does not. **ArchiMate 4 element** is the element to use when the rows are drawn rather than listed, from the forty-two in the house notation; where none fits, the cell says so. Appendix D, *The repository in ArchiMate 4*, carries the reasoning and wins where the two differ.

Motivation — why the architecture exists

Sheet	What it holds	Ref prefix	The columns that matter	ADM phase · year one?	ArchiMate 4 element
`001 Driver`	Thirteen conditions forcing the bank to act — Basel III/IV, IFRS 9, AML/CFT, scheme rules, data protection, resilience, conduct	`DRV-`	`Type`, `Origin`, `Owner`, `Pressure`	Preliminary, A · yes	Driver
`002 Goal`	Six things the bank has decided to be true	`GOL-`	`Owner`, `Status`	A · yes	Goal
`003 Objective`	Fourteen time-bounded statements realising a goal	`OBJ-`	`Goal`, `Horizon`, `Status`	A · yes	Outcome
`004 Measure`	Sixteen indicators with a baseline and a cadence	`MSR-`	`Unit`, `Domain`, `Baseline`, `Cadence`	A · yes	none — the values sit on the Outcome
`005 Principle`	Twenty-four principles across five categories	`PRN-`	`Category`, `Statement`, `Owner`, `Status`	Preliminary · yes	Principle
`006 Requirement`	Thirty-two requirements, each tracing to a driver	`REQ-`	`Statement`, `Category`, `Priority`, `Driver`, `Status`	A, then C and D · yes	Requirement
`007 Constraint`	Eight things that rule approaches out	`CON-`	`Type`	A · yes	none — carry as a Requirement
`008 Assumption`	Six things taken as true pending test	`ASM-`	`Status`	A · yes	none — repository only

`005` is filled well in one respect and thinly in another. All twenty-four principles carry a `Statement` and an `Owner` of `Architecture Board`, which is more than most practices manage. What they do not carry is a tie-break: no tier, weight or precedence column, so when `PRN-01 Primacy of Principles` meets a principle saying the opposite, nothing in the repository settles it. Put the rule in the

`Statement` of one principle rather than adding a column — a column you invent is a column no report reads [4].

`O06` is the better-built half: all thirty-two requirements name a `Driver`, which is why `RPT06 Requirements Traceability` can compute at all.

Business — who does what, and what the business does

Sheet	What it holds	Ref prefix	The columns that matter	ADM phase · year one?	ArchiMate 4 element
`009 Organization Unit`	Twelve units with a head and a headcount	`ORG-`	`Type`, `Head`, `Headcount`	Preliminary · yes	Business Actor
`010 Actor`	Eighteen named positions, each `Filled` or not	`ACT-`	`Type`, `Status`	B · yes	Business Actor
`011 Role`	Eighteen roles, one per actor	`ROL-`	`Type`, `Performed by`	B · yes	Role
`012 Location`	Eight sites with a headcount	`LOC-`	`Type`, `Scale`	B · if relevant	Location
`013 Business Capability`	120 capabilities on three levels: 12 L1, 36 L2, 72 L3	`C01` to `C12.03.02`	`Level`, `Parent`, `Criticality`, `Current`, `Target`, `Disposition`	B · yes	Capability
`014 Value Stream`	Five streams, each with a stage count and a capability list	`VS01` to `VS05`	`Scope`, `Stages`, `Capabilities`	B · yes	Value Stream
`015 Business Service`	Twelve services, one per L1 capability	`BSV-`	`Capability`, `Exposure`, `Criticality`	B · yes	Service
`016 Business Process`	Twenty processes, four per value stream	`BPR-`	`Value stream`, `Automation`	B · yes	Process
`017 Business Function`	Twelve functions, one per L1 capability	`BFN-`	`Capability`	B · if relevant	Function
`018 Product`	Eight banking products across four types	`PRD-`	`Type`	B · yes	Product
`019 Contract`	Ten vendor agreements with an expiry	`CTR-`	`Counterparty`, `Type`, `Expiry`	B, then G · if relevant	Business Object — a contract
`035 Business Building Block`	Ten business ABBs with acceptance criteria	`BBB-`	`Capability`, `What it must do`, `Acceptance criteria`	B, E · yes	Grouping

Three things about `013`, the sheet you will spend the most hours in. The hierarchy is in the `Ref` and the dots are load-bearing: `C01`, `C01.01`, `C01.01.01`. That is what lets `Level` and the whole of `R01`

`Capability decomposition` be derived rather than typed. The assessment columns `Current`, `Target` and `Disposition` are filled on **all three levels**, which gives you three numbers for the same thing and no rule about which wins — decide that L3 is the assessment and the levels above are a roll-up, write that down, and recompute. And the shipped scores do not respect the hierarchy: `C01.01.01` is `High` under a `Critical` parent, `C02.03.01` is `Critical` under a `Medium` one. Child criticality was assigned independently.

`O14`'s `Stages` column holds a count — "4 stages" — not the stages, which are in `O16 Business Process`, four or five placeholder rows per stream. Rename them as the business names them in your first Phase B session.

> **`O19 CONTRACT` NAMES APPLICATIONS, NOT SUPPLIERS**
>
> `CTR-01 Core Banking vendor agreement` has `Counterparty` = `Core Banking`, which is the application, not the company that sells it. The same substitution runs down all ten rows. Fill the real counterparty name before anybody uses this sheet for a vendor review, and keep the application in a separate thought — the repository has no column for it, and `R18 Standard applies to Applica` is the only place application and obligation meet.

Data — the domain where banks lose the argument

Sheet	What it holds	Ref prefix	The columns that matter	ADM phase · year one?	ArchiMate 4 element
`020 Data Entity`	Fourteen entities: Party, Customer, Account, Arrangement, Exposure, Limit and the rest	`E01` to `E14`	`Class`, `Definition`, `Classification`, `Owner`, `Master system`, `Lineage`	C (data) · yes	Business Object
`021 Logical Data Component`	Twelve logical stores, one per entity	`LDC-`	`Entity`	C (data) · yes	Data Object
`022 Physical Data Component`	Twelve physical stores against an application	`PDC-`	`Application`, `Type`	C (data), D · yes	Artifact
`036 Data Entity Relationship`	Eleven entity-to-entity relationships with cardinality	`REL-`	`From entity`, `Relationship`, `To entity`, `Cardinality`	C (data) · yes	Association
`037 Data Attribute`	Sixty attributes, six each for ten of the fourteen entities	`ATR-`	`Entity`, `Attribute`, `Type`, `Key`, `Classification`	C (data) · yes	none — attributes are properties
`038 Data Set`	Six curated sets: regulatory, analytics, operational, finance, risk, compliance	`DST-`	`Description`	C (data) · if relevant	Data Object
`039 Data Migration`	Six entity moves with a from, a to and a status	`MIG-`	`From system`, `To system`, `Status`	E, F · if relevant	none — a Work Package on the Migration view
`040 Source To Target Mapping`	Twelve field mappings with a transformation rule	`MAP-`	`Source field`, `Target field`, `Rule`	F · later	none — Association
`042 Metadata Element`	Fourteen metadata categories with a coverage judgement	`MTD-`	`Category`, `What it records`, `Coverage`	C (data) · yes	none — repository only
`043 Master Data Rule`	Ten mastering rules with a golden source	`MDR-`	`Rule type`, `Description`, `Golden source`	C (data) · yes	none — carry as a Requirement
`044 Data Building Block`	Ten data ABBs, one per master entity	`DBB-`	`Entity`, `What it must do`, `Acceptance criteria`	C (data), E · yes	Grouping

`O20` is the most consequential sheet in the file and the one with the most to argue with. Six of the fourteen entities are classified `Public`, among them `E02 Customer`, `E03 Account` and `E07 Payment Instruction` — a position no South African bank can hold for five minutes. Several `Master system` assignments are not credible either: `E01 Party` mastered in `Collections & Recovery`, `E02 Customer` in `Regulatory Reporting`, `E11 Financial Instrument` in `Fraud Detection`. Three entities read `Lineage` = `Absent`, which is the one figure on the sheet to believe, because `RPT05 Data Ownership` and `ZZ Checks` both compute it and agree [5]. Treat the rest as a worked example of a sheet filled by somebody who had the structure and not the business; appendix C's Pack 2 is the session that fixes it.

`O37` covers ten entities and stops: `E11` to `E14` have no attributes. Defensible as a scope — key attributes on master entities only — but say so in the sheet header rather than letting a reader assume the model is complete.

> **`O36` AND `R19` HOLD THE SAME ELEVEN ROWS**
>
> `O36 Data Entity Relationship` and `R19 Data Entity relates to Data` carry the identical eleven `REL-` records under slightly different headers. Keep `R19` as the relationship of record and `O36` as what your conceptual data model is drawn from, and change both or neither — nothing keeps them in step.

Application and technology — what runs

Sheet	What it holds	Ref prefix	The columns that matter	ADM phase · year one?	ArchiMate 4 element
`023 Application Component`	Fourteen applications with disposition, criticality and health	`A01` to `A14`	`Type`, `Disposition`, `Criticality`, `Health`, `In service`, `Users`, `Owner`	C (application) · yes	Application Component
`024 Information System Service`	Fourteen services exposed by applications	`ISS-`	`Application`, `Style`, `Published`	C (application) · yes	Service
`045 Application Building Block`	Ten application ABBs with acceptance criteria	`ABB-`	`Application`, `What it must do`, `Acceptance criteria`	C (application), E · yes	Grouping
`025 Technology Service`	Twelve technology services in four categories	`TSV-`	`Category`	D · yes	Service
`026 Logical Technology Componen`	Seven logical components with a rationale	`LTC-`	`Rationale`, `Status`	D · yes	Node
`027 Physical Technology Compone`	Twelve components with a vendor support end date	`PTC-`	`Vendor`, `Class`, `Support end`, `Position`, `Location`	D · yes	Node, Device or System Software
`028 Platform Service`	Eight platform services	`PSV-`	`Description`	D · if relevant	Service
`046 Technology Building Block`	Seven technology ABBs	`TBB-`	`What it must do`, `Acceptance criteria`	D, E · yes	Grouping or Node
`048 Environment`	Four environments, each flagged for personal data	`ENV-`	`Description`, `Personal data present`	D · yes	Grouping
`049 Resilience Tier`	Twelve applications with a tier, an objective and a test state	`RES-`	`Tier`, `Recovery objective`, `Tested`	D · yes	none — a property of the Application Component
`050 Network Link`	Ten site-to-site links with a diverse-path flag	`NWL-`	`Bandwidth`, `Diverse path`	D · if relevant	Path

`023` is the sheet every executive will ask to see, and you should spot three oddities before they do. `A02 Payment Hub` is `Criticality` = `Low` in a bank whose `C05 Payments & Settlement` capability

is `Critical`. `A03 Card Management` is `Critical`, `Health` = `Poor` and `In service` 2023 — the newest system is one of the sickest. And `Owner` holds organisation-unit names from `O09`, not people: `Executive`, `Legal`, `Customer Service`. An owner that is a department is not an owner.

> **`046` HAS TWO COLUMNS CALLED `REF`, AND THE SECOND ONE POINTS NOWHERE**
>
> `046 Technology Building Block` reads `Ref | Technology building block | Ref | What it must do | Acceptance criteria`. The first `Ref` holds `TBB-01` to `TBB-07`. The third column, also headed `Ref`, holds `T01` to `T07` — a key space **no other sheet uses**. The logical components it appears to point at are in `O26`, whose refs are `LTC-01` to `LTC-07`. Rename the column `Logical component` and restate the values as `LTC-nn`, or delete it. A duplicate header also defeats `INDEX/MATCH` by name.

`049` reverses its own scale: `Tier 1` carries recovery objectives of 24 hours and 4 hours, `Tier 3` carries 15 minutes and 1 hour. Decide which end is the strict end, apply it, then read the column that matters — **ten of the twelve read `Never tested`**. An untested recovery objective is an intention, and it is the finding the Prudential Authority reaches for first.

`048` says `Production` holds no personal data and `Development` and `Pre-production` do. For a bank that is backwards, and it is the kind of error that survives for years because nobody reads the column. Correct it before `SEC-07 Non-production masking` is taken seriously.

Realisation and delivery — what gets done about it

Sheet	What it holds	Ref prefix	The columns that matter	ADM phase · year one?	ArchiMate 4 element
`029 Gap`	**Empty.** Seven columns, no rows	none — column A unfilled	`Domain`, `Building block`, `Severity`, `Status`, `Due`	E · yes, urgently	none in the forty-two — carry on a Plateau
`030 Work Package`	Twelve packages with cost, dates and two state columns	`WP-`	`Domains`, `Cost`, `Start`, `End`, `Approval`, `Delivery`	E, F · yes	Work Package
`031 Deliverable`	Twenty TOGAF deliverables by phase, `Approved` or `Draft`	`DLV-`	`Phase`, `Status`	all phases · yes	Deliverable
`032 Standard`	Sixteen standards with a lifecycle and a conformance verdict	`STD-`	`Category`, `Status`, `Conformance`	Preliminary, D · yes	none — carry as a Requirement
`033 Architecture Contract`	Eight contracts, one per work package	`ACO-`	`Work package`, `Compliance status`	G · yes	Business Object — a contract
`052 Implementation Factor`	Eight dependencies, constraints and risks against the plan	`IF-`	`Category`, `Deduction`	E · yes	Assessment
`053 Transition Architecture`	Three transition states with work packages and an exit-criteria count	`TA-1` to `TA-3`	`Work packages`, `Exit criteria`	E, F · yes	Plateau
`054 Architecture Alternative`	Five costed options, one recommended	`ALT-A` to `ALT-E`	`Cost`, `Recommendation`	A, E · yes	Course of Action
`055 Building Block`	All 37 building blocks in one list, with a `Domain` column	`BBB-` `DBB-` `ABB-` `TBB-`	`Domain`, `What it must do`, `Acceptance criteria`	E · yes	Grouping
`056 Project`	Twelve projects, one per work package, each with a lead	`PRJ-`	`Work package`, `Cost`, `Start`, `End`, `Lead`	F · yes	Work Package

O62 Delivery Risk	Eight risks with likelihood, impact and an owner	RSK-	Likelihood, Impact, Owner	E, F, G · yes	Assessment
O66 Architecture Increment	Six increments landing at months 10 to 56	INC-1 to INC-6	Lands, What is added	E, F · yes	Plateau

O55 is not a fifth sheet of building blocks. It is **the same 37 records** as O35 (10), O44 (10), O45 (10) and O46 (7), re-presented with a Domain column and re-using their refs exactly, so the same Ref lives on two sheets. R13 Work Package delivers Build keys on O55, so that is the one the delivery half of the model reads. Decide which is master — the domain sheets or the union — write it in the header, and only ever edit the master.

O30 carries two state columns and neither is computed. Approval reads Approved, In flight or Not started; Delivery reads Not started, In flight, Delivered or Past end date. They disagree freely: WP-01 is In flight and Past end date at once, WP-08 is Delivered against an End of 2031-08. Every Start is 2027 or later, so **nothing in the sheet can be past an end date against any real calendar**. Those four rows are a typed opinion, and RPT08 Roadmap and ZZ Checks both repeat them [8].

> **THE COSTLIEST ARITHMETIC IN THE REPOSITORY**
>
> O54 recommends ALT-B Foundations first at **$46m** and marks ALT-E Compliance led ($31m) Partly adopted. The twelve work packages in O30 sum to **$64.5m** — 40% more than the recommended option — and DEC-01 Adopt the recommended architecture alternative is still Outstanding on O60. The plan exceeds an option nobody has approved. Reconcile the three before Phase F and present the reconciliation rather than the roadmap. Every figure in the file is in dollars, which for a South African bank is the first thing to change [8].

Governance and security — the half that makes the rest count

Sheet	What it holds	Ref prefix	The columns that matter	ADM phase · year one?	ArchiMate 4 element
`047 Reference Model Entry`	Eleven reference-model entries, ten `Adopted`, one `Candidate`	`RME-`	`Description`, `Status`	Preliminary, D · yes	Grouping
`057 Governance Body`	Five bodies with cadence, membership and mandate	`GB-`	`Cadence`, `Membership`, `Mandate`	Preliminary · yes	Business Actor or Collaboration
`058 Governance Gate`	Four gates, G0 charter to G3 benefit	`GT-0` to `GT-3`	`When`, `What it tests`	Preliminary, G · yes	Event
`059 Compliance Assessment`	**Empty.** Seven columns, no rows	none — column A unfilled	`Acceptance criterion`, `Gate`, `Verdict`, `Evidence`	G · yes, urgently	Assessment
`060 Governance Decision`	Seven decisions, four `Outstanding`	`DEC-`	`Body`, `Status`	all phases · yes	none — Assessment, with an ADR outside
`061 Governance Change Request`	Five requests: a dispensation, a scope change, two corrections, one escalation	`CR-G`	`Against`, `Type`, `Status`, `Expires`	H · yes	none — repository only
`063 Control Objective`	Seven obligations with a control objective and an evidence verdict	`CO-`	`Source`, `Control objective`, `Evidence`	Preliminary, G · yes	Goal or Requirement
`041 Security Control`	Eight controls with a position and an enforcement point	`SEC-`	`What it does`, `Position`, `Enforced at`	D, G · yes	none — carry as a Requirement
`064 Security Policy`	Four policies with a scope and a position	`POL-`	`Scope`, `Position`	Preliminary · yes	Principle
`065 Enforcement Point`	Five points, each with an honest note on what it does not yet do	`EP-`	`Acts on`, `Position`	D · yes	Node or Business Interface

`057` is the sheet the rest of this volume signs steps against. `GB-01 Architecture Board` holds G1 and G2 and grants dispensations with an expiry; `GB-02 Investment Committee` holds G0 funding and G3 benefit; `GB-03 Data Governance Council` designates data owners; `GB-04 Executive Committee` is the escalation; `GB-05 Architecture Function` is continuous and maintains the repository. Use those names in every signature line — they are the only governance names the file recognises.

`O61` is the most useful small sheet here, because it tells you what the previous occupant knew. `CR-G03 Raise unfunded building blocks as gaps` is `Raised` and `O29 Gap` is empty: the request to populate the gap register exists and has not been actioned. `CR-G05 Refuse or re-baseline the integration package` reads `Board decision required`. `CR-G01` is a dispensation for point-to-point file transfer expiring 2029-06, the only dated one in the file. Read all five on your first day.

`O33` asserts `ACO-01` and `ACO-07` are `Non-compliant` while `O59` holds no assessments at all. A compliance verdict with no assessment behind it is an opinion in a column. Raise the assessments or blank the verdicts.

The relationship catalogue

Twenty-five sheets, each named after what the link means. That naming beats endpoint codes and it has one cost: **twenty of the twenty-five tab names are cut at Excel's 31-character limit**, so the name you see is not the name the sheet wants. Spell them as the tab spells them in any formula, `INDIRECT` or macro. `R04 Business Service supported ` ends in a space, which is invisible and defeats a hand-typed comparison.

The five that fit are `R01 Capability decomposition`, `R11 Goal realised by Objective`, `R12 Gap closed by Work Package`, `R21 Role performed by Actor` and `R22 Org Unit owns Capability`. Two object sheets are cut as well — `O26 Logical Technology Componen` and `O27 Physical Technology Compone` — and `RPT06 Requirements Traceability` is exactly 31 characters but complete. Sheets marked † below are cut [2].

`R22 Org Unit owns Capability` is a business link and sits in the first table below, with the organisation set it belongs to. Only `R06` and `R19` have a `Ref` column. The rest key on both ends, by reference and by name, in adjacent columns — the design decision that makes a broken link visible without a lookup. If a name disagrees with the ref beside it, the row is wrong and you can see it from across the room.

Motivation and organisation

Sheet (as the tab spells it)	From → To	What the link means	The payload column	ArchiMate 4 relationship
`R08 Requirement derives from Dr` †	`O06` → `O01`	This requirement exists because of that driver	none — the link is the payload	Influence
`R09 Requirement satisfied by Bu` †	`O06` → `O55`	This requirement is delivered by that building block	`Traced`	Realization
`R10 Objective measured by Measu` †	`O03` → `O04`	This objective is judged by that indicator	`Unit`	Association
`R11 Goal realised by Objective`	`O02` → `O03`	This goal is achieved through that objective	none	Realization
`R21 Role performed by Actor`	`O11` → `O10`	This role is held by that person	`Status`	Assignment
`R22 Org Unit owns Capability`	`O09` → `O13`	This unit is accountable for that capability	none	Assignment

`R09` is what `RPT06 Requirements Traceability` computes over, and `Traced` is the only judgement in the motivation set. Twenty-four rows read `Yes`, eight read `No`, and there is exactly one row per requirement — so eight requirements reach no building block. Those eight are promises nobody is keeping.

Business and data

Sheet (as the tab spells it)	From → To	What the link means	The payload column	ArchiMate 4 relationship
`R01 Capability decomposition`	`O13` → `O13`	This capability contains that one	`Level`	Composition
`R02 Value stream uses Capabilit` †	`O14` → `O13`	This stream needs that capability to run	none	Serving
`R03 Capability realised by Busi` †	`O13` → `O15`	This capability is delivered as that service	none	Realization
`R04 Business Service supported` †	`O15` → `O23`	This service runs on that application	none	Serving
`R05 Application manages Data En` †	`O23` → `O20`	This application holds that entity	`Role` — `Master` or `Consumer`	Access
`R19 Data Entity relates to Data` †	`O20` → `O20`	This entity relates to that one	`Cardinality`	Association
`R25 Application supports Produc` †	`O18` → `O23`	This product is administered by that application	none	Realization

`R05` is the sheet to fix first and the fault is mechanical. Twenty-eight rows: fourteen `Master`, fourteen `Consumer`. The `Consumer` rows key correctly on `A01` to `A14`. The `Master` rows put the **application name in the key column** — `Collections & Recovery` where `A05` belongs — so column A holds two kinds of value and no lookup resolves the sheet in one pass. Replace the names with refs. Twenty minutes, and nothing downstream is trustworthy until you do.

Application, technology and delivery

Sheet (as the tab spells it)	From → To	What the link means	The payload column	ArchiMate 4 relationship
R06 Application interfaces with †	023 → 023	This application sends data to that one	Type, Frequency, Managed, Logged	Flow
R07 Application deployed on Tec †	023 → 027	This application runs on that component	Environment	Assignment
R20 Application located at Loca †	023 → 012	This application runs at that site	none	Association
R18 Standard applies to Applica †	032 → 023	This standard binds that application	Conformance	Association
R13 Work Package delivers Build †	030 → 055	This package produces that building block	Domain	Realization
R14 Project delivers Work Packa †	056 → 030	This project executes that package	Cost	Composition
R15 Transition contains Work Pa †	053 → 030	This transition state needs that package	none	Aggregation
R23 Increment delivered by Proj †	066 → 056	This increment arrives through that project	Lands	Realization
R12 Gap closed by Work Package	029 → 030	This gap is closed by that package	Severity	Association
R24 Compliance Assessment tests †	059 → 055	This assessment tested that building block	Verdict	Association
R16 Control Objective met by Co †	063 → 041	This obligation is evidenced by that control	Position	Realization
R17 Policy enforced at Enforcem †	064 → 065	This policy is applied at that point	Position	Assignment

R06 is the richest sheet in the file and the one that funds work. Thirty-two interfaces, four judgement columns each. Nineteen read Unmanaged, twenty read Logged = No, ten are Direct DB read — one application reading another's store. RPT01 Dashboard, RPT04 Interface Catalogue and ZZ Checks all compute those three counts and all three agree with the raw rows. Believe them.

Three relationship sheets map positionally rather than semantically, so check before you quote. R16 pairs CO-01 with SEC-01, CO-02 with SEC-02 on down the list, which records Basel III/IV as evidenced by Identity lifecycle. R09 pairs REQ-001 with BBB-01 the same way. R25 has PRD-08 Treasury Deposit administered by AML / Sanctions Screening. The rows are structurally valid and semantically arbitrary: the structure is the gift, the pairings are yours to redo [6].

`R12` and `R24` are header-only because `029` and `059` are empty. They are not broken. They are waiting.

`EXT Industry Extension`

Six rows, three columns: `Ref`, `Industry extension type`, and a third column headed `What it holds and why the core metamodel is not enough`. The refs run `X01` to `X06` [7].

Ref	Type	What it holds	Why the core is not enough
`X01`	Product	A banking product with terms, pricing and eligibility	TOGAF's Product is a bundle of services. A banking product is a priced legal construct with regulated maxima, eligibility rules and a withdrawal process
`X02`	Arrangement	An instance of a product held by a customer	The core has no concept between Product and Contract. An arrangement holds the balance, the rate and the status, and it is what a core banking system administers
`X03`	Exposure	Amount at risk to a counterparty, by facility	Nothing in the core carries a measured financial position. Exposure is the unit Basel III, IFRS 9 and every concentration limit compute on
`X04`	Limit	An authorised ceiling on exposure or activity	A control that is also a data object, enforced in several systems and breached in one. TOGAF has Constraint, which is a statement, not a number with a hierarchy
`X05`	Regulatory Report	A supervisory submission with its data lineage	A deliverable has an approver. A regulatory report has a submission date, a legal basis and a lineage obligation back to source
`X06`	Payment Scheme	A clearing and settlement arrangement the bank participates in	An external actor, a rulebook, a calendar and a settlement obligation at once. No core element holds that combination, and a payments estate is unreadable without it

The six exist because a bank's architecture is unstateable without them. Strike `Arrangement` and you cannot say what a core banking system does. Strike `Exposure` and `Limit` and the credit and capital half of the bank has no objects. Strike `Payment Scheme` and the rails become anonymous interfaces. Volume 1, chapter 9, *The spine: a party, an agreement, an account*, argues the same case from the data side.

Five of the six already appear as rows in `O20 Data Entity`: `E04 Product`, `E05 Arrangement`, `E08 Credit Exposure`, `E10 Limit` and `E13 Regulatory Report`. **`EXT` declares the types; `O20` holds the records.** Nothing in the file states that, so state it in your own header. `X06 Payment Scheme` has no home in `O20` at all, which is the first extension gap to close — a bank in SAMOS, PayShap, DebiCheck, Registered Mandates and the card schemes needs those five as records somewhere, and `O19 Contract` is the wrong shelf.

THE RULE FOR EXTENDING THE EXTENSION

Add a row to `EXT` only when all three are true. **One**, it is an object the business names and argues about, not an attribute of something already modelled. **Two**, no sheet among the sixty-four can hold it without distortion — the test being whether you would have to lie in the `Definition` column. **Three**, you can name the sheet it will get, the `Ref` prefix it will use and the relationship sheet that will connect it, before you add the row.

Then add the sheet. A new object type needs its own `Onn` sheet with `Ref` in column A and the name in column B, at least one `Rnn` sheet joining it to something that exists, a row in `00 Index` with a record count, and a line in `EXT`. Four edits, all four or none, recorded as one change request.

`ZZ Checks`

Three columns — `Check`, `Result`, `Threshold` — and fourteen rows. Five of the rows are inventory, nine are findings, and the sheet closes with the one sentence in the repository that states its own purpose: *the thresholds are what a healthy estate would show; the results are what this one does; that distance is the work.*

What it tests	Result in the shipped file	Threshold	How to read it
Object types populated, records, relationship types, relationship instances, reports	65, 878, 25, 465, 13	"Every type carries records", 13 reports	Inventory. Two of the five are wrong — see the warning above. `465` and `13` are correct
Interfaces unmanaged	19	0	Of 32. The integration business case in one cell
Interfaces with no logging	20	0	You cannot evidence what crossed. Reads directly on Joint Standard 2 of 2024
Direct database reads	10	0	Ten places where changing one application breaks another silently
Platform components past support	3	0	Running unsupported today, with dates in `027`
Security controls absent	4	0	Of eight. A control that is absent is not a control
Control objectives without evidence	4	0	Of seven. Counts `In breach` as well as `No mechanism` — see appendix B
Data entities without lineage	3	0	Of fourteen. Cannot trace a report to its source
Vacant architecture roles	0	0	The only clean finding. All eighteen roles in `R21` read `Filled`
Work packages past end date	4	0	Typed, not computed. Every date in `030` is 2027 or later
Records validated against a real organisation	0	0 — illustrative data throughout	The sheet's own disclaimer, and the reason you replace the records before you quote any of them

Run it twice per phase: before you start, so you know what you inherited, and at the end, so the difference is attributable to you. Photograph the sheet — the file holds no history, so the only record of what the checks said last month is the copy you kept.

Three of the counts are not computed from the rows beneath them, `Work packages past end date` most clearly. Treat `ZZ Checks` as a hand-maintained summary that happens to be right about nine things. The first improvement you can make to this repository is to make all fourteen a formula.

The change-control columns this repository does not have

This is the shortest section in the appendix and the one with the largest consequence.

There are no record-keeping columns on any object or relationship sheet. No `Version`, no `Record Status`, no `In Repository Since`, no `Last Changed`, no `Changed By`, no `Change Ref`, no `Approved By`. **And there are no governance sheets**: no `01 Version Control`, no `02 Change Log`, no `03 Change Requests`, no `04 Baselines`. The repository holds the present tense and nothing else.

It cannot tell you what it said last quarter, who changed a maturity score, or whether anybody approved it.

Add all eleven before you load a single client record. Not after the first phase — before, because retrofitting a change log means inventing history.

Add this column	On	What it is for
`Version`	all `O` and `R` sheets	`1.0` at load, minor increment per approved change. Derived from the change log, never typed
`Record Status`	all `O` and `R` sheets	`Baseline`, `Approved`, or `Draft` while a request against the row is open
`In Repository Since`	all `O` and `R` sheets	The date the record first existed. Set once, never touched
`Last Changed`	all `O` and `R` sheets	The date of the most recent approved change. Equal to the above on an untouched row
`Changed By`	all `O` and `R` sheets	The role that made the change — `Chief Data Officer`, `Domain Architect, Data`. A person, not a system
`Change Ref`	all `O` and `R` sheets	The change request that authorised it. A populated `Changed By` with a blank `Change Ref` is an ungoverned change, and that is the number you report
`Approved By`	all `O` and `R` sheets	`Architecture Board` for anything engaging a principle, `Chief Enterprise Architect` otherwise. Approval is an act — if `GB-01` has not met, do not type its name

Add this sheet	What goes in it	Why
`01 Version Control`	The repository version, the versioning policy in prose, the current baseline, and a count of changes awaiting approval	Somebody will ask "which version is this" in your first month. Without this sheet the answer is a filename
`02 Change Log`	One row per field changed: sheet, `Ref`, column, previous value, new value, date, who, change ref	This is the sheet that makes the other ten work. Previous and new value in every row is what lets you reconstruct any past state. Never truncate it
`03 Change Requests`	The register: what is being asked, against which rows, who raised it, which body decides, the decision and the date	`061 Governance Change Request` holds five governance requests against work packages and the repository. It is not a change register for records and it will not scale to one
`04 Baselines`	Named, dated, approved snapshots with the change-log row number at which each was cut	A baseline is a row number and a signature, not a second copy of the workbook. Keeping a copy of the file as "the baseline" is how two repositories start

Three consequences follow, and they are why this page matters more than the sixty-four sheets above it.

First, **`ZZ Checks` cannot measure its own drift.** It reports the state of the estate and nothing about the state of the record. Add `Records changed this month`, `Changes with no change ref` and `Rows still at baseline`, and the checks sheet starts governing the repository as well as the architecture.

Second, **nothing is evidence.** Joint Standard 1 of 2023 on IT governance and risk commenced 15 November 2024 and expects a bank to show how an architectural decision was taken and by whom. A spreadsheet with no change history is not that showing. Volume 1, chapter 15, *The Joint Standards, with the numbering got right*, has the instruments.

Third, **the comparison tool has nothing to compare.** A baseline-versus-target diff needs a stated baseline. Until `04 Baselines` exists, every comparison runs against whatever the file held the day you copied it.

> **DO NOT START BY FILLING SHEETS**
>
> The temptation on day one is to fix `020`'s classifications or score `013`. Resist it for a week. Add the four sheets and the seven columns, raise change request number one against something that does not matter, and learn the discipline on a low-stakes row. Every correction made before the log exists is a correction with no author.

Naming and `Ref` rules

These are read off the data, not from documentation, because the file carries none.

Rule	Why
`Ref` in column A, name in column B. Never reorder.	Every relationship sheet resolves both ends by position from these two columns. Reordering breaks the workbook silently and the reports still compute
One prefix per sheet, except `055`	`055` carries four — `BBB-`, `DBB-`, `ABB-`, `TBB-` — because it is the union of four sheets
Pad to the width of the set	Two digits under a hundred rows, three for `006` (`REQ-001`) and `037` (`ATR-001`). `REQ-1` and `REQ-001` are different strings to a lookup
Five sheets drop the hyphen: `013`, `014`, `020`, `023`, `EXT`	`C01`, `VS01`, `E01`, `A01`, `X01` — the five most-referenced sheets in the file. The short form is deliberate; do not normalise it
Three sheets drop the padding: `053`, `058`, `066`	`TA-1`, `GT-0`, `INC-1`. Under ten rows each. If one grows past nine, renumber the sheet in one change
`054` numbers with letters: `ALT-A` to `ALT-E`	Alternatives are not ordered by preference and letters stop a reader inferring that they are
Hierarchy goes in the `Ref`, dotted	`C01`, `C01.01`, `C01.01.01`. `013` is the only sheet that does it, and the dots let `Level` and all of `R01` be derived
Names are nouns, title case, no trailing full stop	Names appear twice in every relationship row and in all thirteen reports
Never reuse a retired `Ref`	Once the change log exists it keys on sheet plus `Ref` plus column. Reusing `A07` makes two applications one record across the history
Refs, not names, in key columns	`R05` breaks this, and `020`, `039`, `040` and `043` all carry application names where a ref belongs

What the repository does not hold, and where that content lives

A repository is a register of facts about the architecture. It is not the architecture.

Not in the repository	Where it lives	The link between them
Diagrams. No sheet holds a picture and no report draws one	`Banking_ArchiMate4_Views.pptx`, shipped with the workbook	A view is a selection over the repository. Every element on one should resolve to a `Ref` on an `0` sheet; if it does not, either the view is wrong or the sheet is incomplete
Narrative and argument	The Architecture Definition Document and the twenty deliverables on `031 Deliverable`	`031` holds the record of the document — `Phase` and `Status` — not its text. Thirteen `Approved`, seven `Draft`
Decisions, in full. `060` holds one line and a body	Architecture decision records, referenced from the change register	Context, options, rationale and consequences are an ADR's job. The repository carries the pointer and the verdict
Any history at all	Nowhere. This is the gap the four governance sheets close	Until they exist, the only record of a past state is a dated copy of the file
The extracts the comparison tool needs	Nowhere — there are **no export sheets** here	Build the five extracts by hand: copy the columns, Paste Special values only. Never link the two workbooks
Code, configuration and schemas	The estate. `022` names the store, `037` stops at sixty attributes	Key attributes only: identity, classification, and anything carrying a regulatory consequence
Project plans. `056` holds shape, cost and a lead	The programme's planning tool	`RPT08 Roadmap` is an architectural test of a plan, not a plan
The stages of a value stream	`016 Business Process`	`014`'s `Stages` column holds a count, and `016`'s rows are placeholders until you rename them
The exit criteria of a transition state	Nowhere. `053` holds a count — "5 exit criteria"	Write them out. A state whose exit criteria are a number cannot be tested, and Phase F has nothing to gate on

The last two rows are the same fault twice: a column holding a count where the content belongs. When you find the third, you know what to do with it.

THE CHAPTER IN TEN LINES

- 105 sheets: one index, 64 object sheets, one extension, 25 relationship sheets, 13 reports, one checks sheet.
- 00 Index is right about 65 object types and 25 relationship types. It is wrong about 878 records — the itemised list sums to 846.
- O34 and O51 do not exist. Walk sheet names, never numbers.
- O29 Gap and O59 Compliance Assessment are empty, so R12, R24 and RPT07 are too. ZZ Checks claims every type carries records.
- Derive every Ref prefix from column A. Five sheets drop the hyphen (C01, VS01, E01, A01, X01) and three drop the padding.
- Twenty of the 25 relationship tab names are cut at 31 characters, and R04 Business Service supported ends in a space.
- R05 holds application names in a key column on half its rows. Fix that before you trust any data-ownership report.
- O55 Building Block re-uses the refs of O35, O44, O45 and O46. The same Ref lives on two sheets — pick the master.
- EXT declares six banking object types; O20 holds the records for five of them. X06 Payment Scheme has no records anywhere.
- There is no change control. Add seven columns and four sheets before you load a single client record, not after the first phase.

SOURCES FOR THIS CHAPTER

- **1.** Banking_TOGAF_Metamodel.xlsx, sheet 00 Index — header counts and the itemised record list
- **2.** Banking_TOGAF_Metamodel.xlsx, the 105 sheet names as Excel stores them
- **3.** Banking_TOGAF_Metamodel.xlsx, sheets RPT01 Dashboard and ZZ Checks
- **4.** Banking_TOGAF_Metamodel.xlsx, sheet O05 Principle
- **5.** Banking_TOGAF_Metamodel.xlsx, sheets O20 Data Entity, R05 Application manages Data En and RPT05 Data Ownership
- **6.** Banking_TOGAF_Metamodel.xlsx, sheets O29 Gap, O59 Compliance Assessment, R12 Gap closed by Work Package, R24 Compliance Assessment tests and R16 Control Objective met by Co
- **7.** Banking_TOGAF_Metamodel.xlsx, sheet EXT Industry Extension
- **8.** Banking_TOGAF_Metamodel.xlsx, sheets O30 Work Package, O54 Architecture Alternative, O60 Governance Decision and O61 Governance Change Request
- **9.** The Open Group, TOGAF Standard, Architecture Development Method https://pubs.opengroup.org/architecture/togaf-standard/

APPENDIX B

The report catalogue

Thirteen reports, the one question each answers, what the number means when it is bad, and the three of the thirteen that actually compute something.

Thirteen report sheets, which is a small set and an honest one. They sit at the right-hand end of the tab bar, after `R25` and before `ZZ Checks`, and they are the only sheets in the repository you read rather than fill. A typed cell on an `RPT` sheet is a broken report.

This appendix gives each one five things: the question it answers in a sentence, the ArchiMate 4 view its content belongs on if you are going to draw it rather than tabulate it, when to read it, what good looks like, and what a bad number actually means. Use it to stop reading reports in tab order, which is the order they were built and not the order they are useful.

Ardent Union Bank is invented. Its numbers are real inside this workbook and describe no real bank.

> **WHAT THESE REPORTS ACTUALLY COMPUTE, BEFORE YOU TRUST THE WORD "REPORT"**
>
> Open them and look, because only three of the thirteen compute anything at all [1].
>
> **`RPT03 Capability Heatmap`** adds a `Gap` column — `Target` minus `Current` — over the twelve level-1 rows of `O13`. **`RPT06 Requirements Traceability`** joins `O06 Requirement` to the `Traced` column of `R09 Requirement satisfied by Bu` and renames it `Reaches a block`. **`RPT13 Orphan Analysis`** is an anti-join: applications in `O23` with no row in `R04 Business Service supported`.
>
> Six more — `RPT02`, `RPT05`, `RPT08`, `RPT09`, `RPT10`, `RPT11`, `RPT12` — are their source object sheet re-presented with one or two columns dropped, plus a single summary sentence in the last row. `RPT04` is `R06` with the name columns shown instead of the refs. `RPT01 Dashboard` is the only aggregate. `RPT07 Gap Analysis` is a header and nothing else.
>
> That is not a criticism of the file. A re-presentation that drops `Definition` and adds "3 of 14 entities have no lineage from report back to source" is a useful thing to hand an executive. It is a warning about what will happen when you replace the records: **nothing recomputes.** Write the formulas yourself, in the order this appendix reads them, and the sheets become reports.

> **IF YOU HAVE WORKED THE INSURANCE REPOSITORY, TWO NUMBERS HAVE MOVED**
>
> In this file `RPT07` is **`Gap Analysis`** and `RPT09` is **`Regulatory Coverage`**. In the insurance repository the gap report is `RPT09 Gaps and Roadmap` and `RPT07` is `Capability Coverage`. The two sets share nothing but the prefix. There are thirteen reports here, not thirty-two, and no report in this file is called `Building Block Coverage` at all. Read the tab, never the number you remember [1].

The repository about itself

Two reports that describe the record rather than the bank. Read both on your first morning and neither in front of an executive.

Sheet	The question it answers	ArchiMate 4 view	Read when	What good looks like	What a bad number means
`RPT01 Dashboard`	How big is the repository, and what are the nine findings a first pass over this estate produces?	Layered	Preliminary, then monthly. The first sheet you open and the one to quote from most carefully	Relationship instances comfortably above record count; every finding row either zero or falling	465 instances against 846 records is 0.55 links per record — the objects are catalogued and barely joined. Nine of the eighteen rows are findings and seven of those are non-zero
`RPT13 Orphan Analysis`	Which applications does no business service map to?	none — this is about the record, not the architecture	Preliminary, then after every load	Empty, or every row carrying a reason	`A13 Data Platform` and `A14 Fraud Detection` are the two rows. Both are real systems that genuinely serve no business service in `R04` — the finding is that `R04` is incomplete, not that the systems are spare

`RPT01` states **878 records** and so do `00 Index` and `ZZ Checks`. The itemised list on `00 Index` sums to 846. Appendix A has the arithmetic. Until you have recomputed it, quote the findings from this dashboard and not the inventory.

Two of its rows are worth reading twice because they look clean and are not. **`Gaps overdue | 0`** is true only because `029 Gap` is empty — there are no gaps to be overdue. **`Work packages past end date | 4`** is typed into `030`, not computed: every `Start` in that sheet is 2027 or later, so nothing in it can be past an end date against any calendar. The one genuinely clean row is `Vacant architecture roles | 0`, which all eighteen rows of `R21 Role performed by Actor` confirm [6].

`RPT13` is titled "Objects that nothing else references" and tests exactly one thing: applications against business services. Nothing else in the repository is checked for orphanhood. Extend it — unreferenced data entities, building blocks no work package delivers, standards no application is bound by — and it becomes the most useful sheet in the file, which is what its own header note claims it already is.

The estate — what runs, how it connects, how old it is

Three reports, and together they are the technology business case.

Sheet	The question it answers	ArchiMate 4 view	Read when	What good looks like	What a bad number means
`RPT02 Application Portfolio`	What applications exist, how healthy are they, and what is meant to happen to them?	Application Structure	Phase C (application), then quarterly	Every row has a `Disposition`, a named `Owner` and a `Health` that agrees with its `Criticality`	Five of fourteen read `Health = Poor`. `A03 Card Management` is `Critical`, `Poor` and in service since 2023 — the newest system is one of the sickest, which means the problem is not age
`RPT04 Interface Catalogue`	What integrates with what, how, and is any of it managed or logged?	Application Cooperation	Phase C (application), then at every design gate	`Managed = Managed` and `Logged = Yes` on every row; no `Direct DB read` anywhere	19 of 32 unmanaged, 20 with no logging and 10 direct database reads. Any change to one application can break three others silently. This is the number that funds an integration platform
`RPT10 Technology Currency`	Which platform components are past vendor support, and when do the rest fall off?	Implementation and Deployment	Phase D, then at every vendor roadmap review	No `Support end` date inside eighteen months under a component carrying a critical application	`PTC-05 HSM estate`, `PTC-06 Branch server estate` and `PTC-10 ATM estate controller` read `Past support` today. Dated, costed and unarguable — this is the report that funds currency work

`RPT02` is `O23` with the `Description` column dropped. Its `Owner` column holds organisation-unit names, not people, which is the first thing to fix because a portfolio with departmental owners has no owners. The disposition spread — six `Retain`, three `Invest`, two `Replace`, two `Retire`, one `Consolidate` — is a plausible shape, but check it against `A02 Payment Hub`, which is `Invest` and `Criticality = Low` in a bank whose payments capability is `Critical`.

`RPT04` is the one to read on day one. It shows the names rather than the refs, which makes it readable aloud in a meeting, and its footer computes the two figures that matter: *19 of 32 interfaces are unmanaged and 20 carry no logging* [4]. Pair it with `RPT11 Security Position`, where `SEC-05 Interface logging` is the one control reading `In place`. A control that is in place over twenty interfaces that carry no logging is a control covering twelve of them.

`RPT10` carries `Position` as a stated verdict rather than a date comparison. Replace it with a formula against `Support end` and today's date, because the three `Past support` rows have end dates in 2026 and the sheet will go stale silently in 2028 when `PTC-01 Core banking mainframe` and `PTC-02 Payment hub servers` roll off.

The business — the one report an executive reads without translation

Sheet	The question it answers	ArchiMate 4 view	Read when	What good looks like	What a bad number means
`RPT03 Capability Heatmap`	Where is business capability weak relative to how much the bank depends on it?	Capability Map	Phase B, then once a year at most	`Gap` under 1 on anything `Critical`; `Build` concentrated where criticality is high	`C12 Channel & Distribution` at `Current` 1, `Target` 3, `Gap` 2 and `Criticality` `High`. `C11 Finance & Regulatory Reporting` at `Current` 2 against `Critical`. Five of twelve carry a gap of 2

This is the only report in the file with a genuinely computed column, and `Gap` is it [2]. Read the five rows carrying a gap of 2 — `C02 Product Management`, `C04 Credit Risk Management`, `C06 Deposits & Accounts`, `C11 Finance & Regulatory Reporting`, `C12 Channel & Distribution` — and then read the one row that should worry you more than any of them: `C11` is `Critical` at `Current` 2. A bank that is weak at regulatory reporting is weak at the thing its licence depends on.

Two cautions. The sheet shows level 1 only, twelve rows, while `O13` holds 120 rows scored on all three levels. The level-1 scores were not derived from the levels below them, so a level-1 figure here is an assertion and not a roll-up — recompute it before you present it. And `C05 Payments & Settlement` is `Critical` at `Current` 4, the best score on the sheet, in an estate where the payment hub is `Health` = `Poor` and `Criticality` = `Low`. One of those two sheets is wrong and the heatmap is the one people will believe.

Volume 1, chapter 2, *The lines of business*, is the substance behind the twelve names. Match on name, never on number — the repository's `C01` to `C12` and volume 1's own capability list were built independently.

The data — the strongest case in the repository

Sheet	The question it answers	ArchiMate 4 view	Read when	What good looks like	What a bad number means
`RPT05 Data Ownership`	Who owns each entity, which system masters it, and can a report be traced back to source?	Information Structure	Phase C (data), then at every data-ownership session	A named person in `Owner`, a credible `Master system`, `Classification` matching the entity's sensitivity, `Lineage` = `Full`	Three entities read `Lineage = Absent`: `E08 Credit Exposure`, `E10 Limit`, `E11 Financial Instrument`. Those three are the inputs to capital, impairment and concentration limits, and none of them can be traced to source

`RPT05` is `O20` with the `Definition` column dropped, and its footer computes the one figure to carry into a meeting: *3 of 14 entities have no lineage from report back to source.* Volume 1, chapter 15, *CoDI, and a regulation that is really a data specification*, explains why a South African bank cannot leave that at three.

Everything else on this sheet needs arguing with before it leaves the building. Six entities are classified `Public`, among them `E02 Customer`, `E03 Account` and `E07 Payment Instruction`. `E01 Party` is mastered in `Collections & Recovery` and `E11 Financial Instrument` in `Fraud Detection`. Several `Owner` values are plausible job titles attached to the wrong entity — `E03 Account` owned by the `Head of Lending`, `E14 Alert / Case` owned by the `Head of Lending` as well. Read it as a worked example of the shape the sheet takes, and run appendix C's Pack 2 before you quote a single row.

Obligation, control and conformance — the licence half

Three reports that answer the only question a supervisor really asks: can you show me that this is met, and by what?

Sheet	The question it answers	ArchiMate 4 view	Read when	What good looks like	What a bad number means
`RPT09 Regulatory Coverage`	For each obligation, what is the control objective and what evidences it?	Motivation	Preliminary, then at every Architecture Board	`Evidence = Evidenced` on every row, with the evidence named	Two rows read `No mechanism` and two read `In breach`: `CO-01 Basel III / IV` and `CO-05 Data protection` in breach, `CO-04 Payment scheme rules` and `CO-07 Conduct and fair treatment` with no mechanism at all
`RPT11 Security Position`	Which security controls exist, where are they enforced, and which are absent?	Technology	Phase D, then quarterly	No row reading `Absent`; every control naming a real enforcement point	Four of eight read `Absent`, three of them identity controls enforced at the identity provider. A control that is absent is not a control
`RPT12 Standards Conformance`	Does anything in the estate conform to the standards the estate has set itself?	Technology	Preliminary and Phase D, then quarterly	`Conformance = Conformant` wherever `Status = Current`	Four rows read `Breach` and four read `Not assessed`. `STD-04 Data retention`, `STD-05 Identity and access` and `STD-06 Encryption` are all `Current` and all in breach — three standards the bank wrote and does not follow

`RPT09` is `O63 Control Objective` verbatim, and its `Control objective` column is a placeholder: every row reads "Control that evidences <the obligation>". Rewrite those seven cells first. They are the only seven cells in the repository that a supervisor would read as a statement of intent, and at the moment they state nothing.

The dashboard's row **`Control objectives with no evidence | 4`** is this sheet's footer with a looser label. Two of the four have no mechanism; two have a mechanism in breach. Those are different problems with different owners, and the one number hides the difference. Split it when you write the formula.

`RPT11` and `RPT12` should be read together with `R16 Control Objective met by Co`, which maps obligations to controls positionally — `CO-01` to `SEC-01`, `CO-02` to `SEC-02` — so the file records Basel III/IV as evidenced by `Identity lifecycle`. Redo that mapping before either report is quoted. The counts are right; the pairings are not.

`O64 Security Policy` has no report of its own. `POL-03 Material activity is logged and observable` reads `Breached`, which is the single most defensible finding in the repository given 20 interfaces with no logging, and nothing surfaces it. Add it to `RPT11`.

Requirement, gap and plan — the delivery half

Sheet	The question it answers	ArchiMate 4 view	Read when	What good looks like	What a bad number means
`RPT06 Requirements Traceability`	Does every requirement reach a building block something will actually deliver?	Requirements Realization	Phase A, then at every requirements review and every gate	`Reaches a block = Yes` on everything, and every `Must` traced before any `Could`	Eight of thirty-two read `No`, and they are the wrong eight: `REQ-001 Evidence Basel III / IV`, `REQ-005 Evidence Data protection` and `REQ-009 Customer mastered once` are all `Must` and reach nothing
`RPT07 Gap Analysis`	Every gap, its severity, and the package that closes it	Implementation and Migration	Phase E, then at every board	A gap per finding, each with a severity, a due date and a work package	**The sheet is empty.** Seven headers, no rows. Not a clean estate — an unpopulated register
`RPT08 Roadmap`	What is being spent, over what period, and what state is each package in?	Implementation and Migration	Phase E and F, then monthly	`Approval` and `Delivery` consistent on every row; costs reconciling to the approved alternative	Twelve packages summing to **$64.5m** against the $46m of `ALT-B`, the option `O54` recommends and `DEC-01` has not yet approved

`RPT07` is the most important empty sheet you will ever be handed, and the repository tells you why. `O61 Governance Change Request` row `CR-G03 Raise unfunded building blocks as gaps` reads `Raised`. Somebody saw that the building blocks in `O55` are not all funded by a package in `R13`, raised the request to turn that into a gap register, and left. Populating `O29 Gap` is therefore not your idea — it is an outstanding instruction, and that is a much easier thing to get approved [3].

Do not read a zero on this sheet as good news. `ZZ Checks` has no row for gaps, `RPT01` reports `Gaps overdue | 0`, and both are vacuously true. An empty gap register means Phase E has not been done, not that there is nothing to do.

`RPT08` drops `O30`'s `Domains` column and keeps both state columns, which disagree: `WP-08 Process automation` is `Delivery` = `Delivered` with an `End` of 2031-08, `WP-01 Governance foundations` is `Approval` = `In flight` and `Delivery` = `Past end date` at once. Replace `Delivery` with a formula over `End` and today's date before anybody plans against it. Then do the arithmetic the sheet does not: twelve costs, one total, compared against `O54 Architecture Alternative` and `O60 Governance Decision`. A roadmap costing 40% more than an unapproved option is the finding, and it is one addition away [5].

`RPT06` is the best-built report in the file. One row per requirement, a real join to `R09`, and a footer that states the principle plainly: *a requirement that reaches no building block is a requirement nothing will deliver.* Work the eight `No` rows in priority order and the sheet is clean in a week.

The five reports for a quarterly architecture board

One page, five panels, three decisions. These five and not the others.

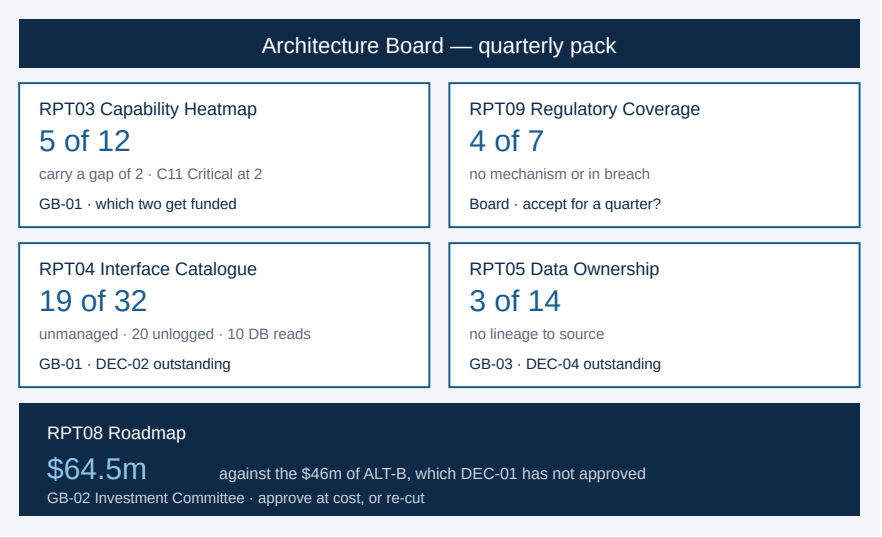

Figure 14.1 Five reports, one page. Each panel names the body that can act on it and the Outstanding decision it maps to.

#	Report	What it puts on the page	The decision it forces
1	`RPT03 Capability Heatmap`	Twelve rows, a criticality, a gap. `C11 Finance & Regulatory Reporting` `Critical` at `Current` 2	Which two capabilities get investment this year. Nothing else in the file frames that question in the business's own words
2	`RPT09 Regulatory Coverage`	Seven obligations, four of them with no mechanism or a mechanism in breach	Whether the board accepts that position for another quarter, in minutes, by name
3	`RPT04 Interface Catalogue`	19 of 32 unmanaged, 20 unlogged, 10 direct database reads	Whether the integration platform is funded. `DEC-02 Select the integration product` is `Outstanding` and `RSK-01` rates that `Certain`
4	`RPT05 Data Ownership`	Fourteen entities, three with no lineage, several with no credible master	Who owns `E01 Party`. `GB-03 Data Governance Council` exists precisely to take that decision and `DEC-04` is still `Outstanding`
5	`RPT08 Roadmap`	Twelve packages, $64.5m, four states that contradict their own dates	Whether the plan is approved at the cost it actually carries, or re-cut to the $46m of `ALT-B`

Why those five. Each one is a finding a named body already has the mandate to resolve: `GB-01 Architecture Board` for the heatmap and the interfaces, `GB-02 Investment Committee` for the roadmap, `GB-03 Data Governance Council` for data ownership, and the board as a whole for the regulatory position. Each has a figure the room cannot argue away without changing the record. And four of the five map onto an `Outstanding` row in `O60 Governance Decision`, which turns the pack from a status report into an agenda.

Why not the others. `RPT01 Dashboard` repeats a record count that is wrong and will cost you the first ten minutes of the meeting defending arithmetic rather than architecture — bring three of its rows inside the other panels instead. `RPT02 Application Portfolio` is a list of fourteen systems with no decision attached; `RPT03` carries the same argument in the business's language. `RPT10`, `RPT11` and `RPT12` belong to a technology or security forum, where the audience can act on them, and they lose their force in a general meeting. `RPT06` is an architect's working sheet and nobody else will read a thirty-two-row traceability table. `RPT07` is empty and `RPT13` has two rows — take both to the board the quarter *after* you have populated them, as evidence the practice is working.

One rule for the pack. Every panel carries the date you read it, because the repository holds no history and no version. Until `01 Version Control` and `04 Baselines` exist, the date on your slide is the only provenance any of these numbers has.

THE CHAPTER IN TEN LINES

- Thirteen reports, not thirty-two. RPT07 here is Gap Analysis and RPT09 is Regulatory Coverage — the opposite of the insurance repository.
- Only RPT03, RPT06 and RPT13 compute. Seven are their source sheet re-presented with a summary line; RPT01 is the only aggregate.
- Replace the records and nothing recomputes. Write the formulas before you load client data.
- RPT07 Gap Analysis is empty, and CR-G03 on O61 is the outstanding instruction to fill it.
- RPT01's Gaps overdue | 0 and Work packages past end date | 4 are both artefacts, not findings.
- RPT04 is the report that funds an integration platform: 19 of 32 unmanaged, 20 unlogged, 10 direct database reads.
- RPT03's Gap is the only computed column in the file, and its level-1 scores were not rolled up from the levels below.
- RPT08 sums to $64.5m against the $46m of the recommended and still-unapproved ALT-B.
- RPT09's seven Control objective cells are placeholders. Rewrite them before a supervisor reads them.
- Board pack: RPT03, RPT09, RPT04, RPT05, RPT08. Each maps to a body with a mandate and an Outstanding row on O60.

SOURCES FOR THIS CHAPTER

- **1.** Banking_TOGAF_Metamodel.xlsx, sheets RPT01 Dashboard to RPT13 Orphan Analysis
- **2.** Banking_TOGAF_Metamodel.xlsx, sheets RPT03 Capability Heatmap and O13 Business Capability
- **3.** Banking_TOGAF_Metamodel.xlsx, sheets RPT07 Gap Analysis, O29 Gap, R12 Gap closed by Work Package and O61 Governance Change Request
- **4.** Banking_TOGAF_Metamodel.xlsx, sheets RPT04 Interface Catalogue, R06 Application interfaces with and RPT11 Security Position
- **5.** Banking_TOGAF_Metamodel.xlsx, sheets RPT08 Roadmap, O30 Work Package, O54 Architecture Alternative and O60 Governance Decision
- **6.** Banking_TOGAF_Metamodel.xlsx, sheets ZZ Checks and R21 Role performed by Actor
- **7.** The Open Group, TOGAF Standard, Architecture Development Method https://pubs.opengroup.org/architecture/togaf-standard/

APPENDIX C

Workshop packs, checklists and the ninety-day calendar

Five workshops that fill named cells, ten checklists you can tick against the file, and thirteen weeks that end in arithmetic a programme director cannot argue with.

Volume 1, chapter 18, *Your first ninety days*, runs the learning track: what is already public, the eight documents to read in week one, who to meet in what order, what to refuse, and the politics. This appendix is the other half — the repository track. It assumes a populated workbook landed on your desk on day one and you have to make it true. The two tracks are designed to be worked in the same thirteen weeks and this one repeats none of that chapter's content.

Ardent Union Bank is invented. Business role names below are volume 1, chapter 5's, *Who does what*; architecture roles and governance bodies are the repository's own, from `011 Role` and `057 Governance Body` [1].

Five workshop packs

A workshop that fills no cells was a meeting. Each pack names the cells.

Look it up. Volume 1, chapter 5, *The architect's real counterparties*, names the eight people whose problems you can solve. Four of them chair a pack below.

Pack 1 — Capability maturity

Fills. `013 Business Capability`: `Criticality`, `Current`, `Target`, `Disposition`, on the level-3 rows of one level-1 area. Six rows per level-2 group, eighteen per area.

Who must be in the room. The line executive who owns the area and can be held to the score — for `C05 Payments & Settlement` the **payments operations** manager; for `C04 Credit Risk Management` the **credit manager / sanctioner** and a **credit risk modeller**; for `C11 Finance & Regulatory Reporting` the **regulatory reporting analyst** and the **financial controller**; for `C12 Channel & Distribution` the **digital channel product owner** and a **branch manager**. Plus the relevant domain architect (`ROL-09` to `ROL-12`) holding the pen, and one person from finance who can say what the area costs. Never more than nine. A tenth turns an assessment into a presentation.

How long. Three hours for one level-1 area. Do not attempt two in a day: scoring drifts after lunch and the afternoon area always scores higher.

Send three working days out. The eighteen level-3 rows for that area, `Capability` and `Description` only, assessment columns blank. The maturity scale, five lines, one per level. `RPT03 Capability Heatmap` so the room sees what the file currently claims about their area. Volume 1, chapter 2, *The*

lines of business, drafts the capability names — start there, argue with it, and **match on name, never on number**.

Running order	Minutes	What happens
1. Boundaries	25	Read each name and description aloud. The room recognises it or renames it. Renaming is allowed; adding is not
2. Criticality first	30	`Critical`, `High`, `Medium`, `Low`. This is dependence, not performance
3. Current	60	One to five, with a named example behind every score
4. Target	35	Where it must be by the horizon on `003 Objective`, not where it would be pleasant
5. Disposition	20	`Sustain` or `Build`. Falls out of the gap
6. The parent rows	10	Agree that levels 1 and 2 will be rolled up from here, not scored separately

Facilitation moves that work. Score `Criticality` before `Current` — once a room has admitted a capability is weak it will not then admit it is critical, because the two together sound like an accusation. Ask for evidence rather than a score: "what would I see if I watched this being done for a week?" and let the number follow. When two people differ by two points, write both down and move on; the disagreement is the finding. And open by showing the room `C05 Payments & Settlement` at `Current` 4 — the best score in the file — next to `RPT02`, where the payment hub reads `Health` = `Poor`. It establishes in ninety seconds that the sheet is not scripture.

Three ways it derails. First, **everything becomes critical**. Cap it: no more than a third of the eighteen rows. Second, **the room scores the parents**. All 120 rows arrive with `Current` and `Target` filled on all three levels and the level-1 figures were not derived from anything — say in the first five minutes that levels 1 and 2 will be recomputed and that scoring them twice produces two answers. Third, **somebody names a system**. The moment a vendor or an application enters the conversation you are assessing software. Park it for Pack 3.

Done when. Every level-3 row in the area carries four judgements, the level-1 row is a computed roll-up, and the executive who owns it repeats the gap figure in their own words.

BEFORE YOU RUN PACK 1

What is the one sentence that decides whether a row belongs in `013` at all?

Which two of the twelve level-1 areas will you assess first, and why those two?

Who will be asked to defend each score, and to whom?

Pack 2 — Data ownership and authority

Fills. `O20 Data Entity: Definition`, `Classification`, `Owner`, `Master system`, `Lineage`. Then the `Role` column of `R05 Application manages Data En` for every application touching the entity — and the key column of those rows, which currently holds application names on fourteen of twenty-eight.

Who must be in the room. This pack lives or dies on the attendance list. The **Chief Data Officer** chairs it as `GB-03 Data Governance Council`. Every candidate master needs the person accountable for that application present, because "do you master Party?" has to be answered by somebody who can be wrong in public. The **information officer (POPIA)** for `Classification`, which is a statutory role and not optional. The **data engineer**, who knows which source systems lie. A **financial crime analyst** for `E01 Party`, `E02 Customer` and `E14 Alert / Case`, and the **regulatory reporting analyst** for `E13 Regulatory Report`. The `ROL-16 Data Steward` takes the pen.

How long. Four hours, five entities. Fourteen entities is three sessions. Do the master spine first — `E01 Party`, `E02 Customer`, `E03 Account`, `E05 Arrangement` — because every later session is easier once Party is settled.

Send five working days out. `RPT05 Data Ownership` as shipped, with the six `Public` classifications highlighted and the three `Absent` lineage rows circled. The `R05` extract showing who claims to master what. One proposed definition per entity, one sentence each — half the session goes on agreeing what the word means. And volume 1, chapter 9, *Party, and why Customer is usually a bad entity*, to everyone who will argue that Customer is the master.

Running order	Minutes	What happens
1. The definition	60	One sentence per entity, agreed aloud. `E01 Party` and `E02 Customer` take half of this on their own
2. Classification	40	`Public`, `Internal`, `Restricted`, `Confidential`, with the information officer holding the pen. Six rows change today
3. The master	60	One application per entity. If the room cannot agree, write `CONTESTED` and name the decision
4. The owner	30	A named person, not a department and not a job family
5. Lineage	30	`Full`, `Partial`, `Absent`, judged against whether a regulatory figure can be traced to source

Facilitation moves that work. Put the classification question before the mastering question: once a room has agreed `E03 Account` is `Restricted`, the claim that it is mastered in `Payment Hub` gets tested harder. Make `CONTESTED` a respectable answer and say so in the first minute — a contested entity recorded is worth more than a master invented, because it names a decision somebody has to take and `DEC-04 Designate data owners` is already `Outstanding` on `O60`. Ask the information officer to read `O48 Environment` out loud, where `Production` is flagged as holding no personal data; the laughter does more for the session than any slide.

Three ways it derails. First, **the room masters by system of record rather than by system of authority**, and you end up with `E02 Customer` mastered in `Regulatory Reporting` exactly as shipped. Ask instead: when two systems disagree about this record, which one is right? Second, **ownership is given to a committee**. A committee cannot be wrong, so it cannot own anything. Third, **somebody solves it with a product**. A master data platform is a Pack 3 and a Phase E conversation; authority is a decision and it is free.

Done when. Every entity has one sentence, one classification, one named person, one master or the word `CONTESTED`, and a lineage verdict. `R05` keys on `A01` to `A14` throughout, and `RPT05` can be read to the board without an apology.

YOUR DATA OWNERSHIP SESSIONS

The five entities in session one, and the date:

The person who will chair as `GB-03`, and the information officer by name:

The entities you expect to record `CONTESTED`, and what decision each one names:

Pack 3 — Application portfolio scoring

Fills. `O23 Application Component`: `Disposition`, `Criticality`, `Health`, `Owner`, `Users`. Then `O49 Resilience Tier` `Tier`, `Recovery objective` and `Tested`, because the same people are in the room and it takes twenty minutes more.

Who must be in the room. The `ROL-11 Domain Architect, Application` chairs. The **core banking engineer** for `A01`, the **payments engineer** for `A02` and `A03`, the **integration and API engineer** for all fourteen, the **data engineer** for `A13`, and the **security and identity engineer** for `A08` and `A14`. One business voice who can speak to value: the relevant **product owner** or the **head of operations** (`ROL-14`). The **Head of Service Management** (`ROL-15`) brings the incident record, which is the only evidence for `Health` anybody will accept.

How long. Three hours for fourteen applications. Fifteen minutes each on the five that matter and five minutes each on the rest.

Send three working days out. `RPT02 Application Portfolio` as shipped. `RPT04 Interface Catalogue` filtered per application, because an application with six unmanaged interfaces cannot be scored `Healthy`. `RPT10 Technology Currency` with the three `Past support` rows marked. Twelve months of incident and change-failure counts from service management — not opinions, counts.

Running order	Minutes	What happens
1. Ownership	20	Replace every departmental `Owner` with a named person. Fourteen rows, no debate
2. Criticality	30	Against the capability it supports in `R04`, not against how much the room likes it
3. Health	60	`Healthy`, `Adequate`, `At risk`, `Poor`, evidenced by incidents, change failures, interface state and support dates
4. Disposition	45	`Retain`, `Invest`, `Consolidate`, `Replace`, `Retire`. Then test each against cost and criticality
5. Resilience	25	Tier, recovery objective, and the date each was last tested. Ten rows currently read `Never tested`

Facilitation moves that work. Score `Criticality` from `R04` rather than from the room, which settles `A02 Payment Hub` before the argument starts — it is shipped as `Low` under a `Critical` capability. Make `Health` require three pieces of evidence or it reverts to `At risk`. When a room wants to `Retire` something, ask which business service in `R04` loses its support and what happens to the data in `R05`; half the retirements become consolidations in one question. Put `A03 Card Management` up early: `Critical`, `Poor`, in service since 2023. A new system in poor health means the problem is integration or operations, not age, and that reframes the whole session.

Three ways it derails. First, **everything is retained**, because retention needs no business case. Insist on a reason per row and a named owner for each reason. Second, **the room scores cost**. There is no cost column on `O23` and adding one invites a number nobody can source — cost belongs to `O30 Work Package`. Third, **disposition is set by vendor roadmap**. A vendor's end-of-support date is an input to `Health`, not a disposition; `RPT10` holds the dates and `O19 Contract` holds the expiries, and both are evidence rather than decisions.

Done when. Fourteen rows each with a named person, a criticality that agrees with `R04`, a health score with three pieces of evidence behind it, and a disposition somebody will defend at `GB-02 Investment Committee`. `RPT13 Orphan Analysis` has either lost `A13` and `A14` or gained a reason against each.

Pack 4 — Gap classification

Fills. The `Review` sheet of `Banking_Architecture_Comparison_Tool_TEMPLATE.xlsx`, then `Gap Candidates`, then `O29 Gap` — which is empty today, and which `CR-G03 Raise unfunded building blocks as gaps` on `O61` already instructs you to populate.

Before the session, the extracts. This repository has **no export sheets**. There is nothing to copy out in one action, so the five extracts are produced by hand: open each source sheet, select the columns the paste target names, copy, and Paste Special values only into the tool. Never link the two workbooks — an external reference breaks on rename and depends on a user enabling updates. Allow half a day and do it twice, once for baseline and once for target.

Who must be in the room. Small. The `ROL-08 Chief Enterprise Architect` and the two domain architects whose sheets produced the differences. One person from delivery who knows what actually

shipped — the **Head of Delivery** (`ROL-13`). The `ROL-18 Internal Auditor` for one hour, not four, because the question "was this change governed?" is their question and they will answer it faster than anyone.

How long. Two hours for the first sixty differences, then an hour per further sixty. The first session is slow because the room is learning the three classifications.

Send two working days out. The tool with both inputs pasted and the diff computed. The three classifications, with one worked example each. And this instruction: nobody classifies in advance.

Running order	Minutes	What happens
1. The three words	20	A **design delta** is an intended change to the target. **Governed drift** is a real change with an approval behind it. **Ungoverned change** is a real change with nothing behind it
2. First pass	45	Classify every difference. No discussion over thirty seconds — park anything slower
3. The parked rows	30	Work the parked list. These are the findings
4. Gap candidates	20	Every ungoverned change and every unclosed design delta becomes a candidate row
5. Severity and due	15	`Severity` and `Due` per candidate, then paste into `O29 Gap`

Facilitation moves that work. Classify before you explain: the explanation is where the room negotiates, and a first pass of thirty seconds a row produces a better distribution than an argued one. Keep the ungoverned-change count visible on a flipchart, because that single number is the business case for the change control appendix A tells you to add. When somebody says "that was agreed", ask where — and if the answer is a meeting rather than a record, it is ungoverned change, and that is not an accusation, it is the finding.

Three ways it derails. First, **everything is a design delta**, because that classification blames nobody. Hold the line: a delta that nothing in the record authorised is drift at best. Second, **the room tries to fix the architecture** instead of classifying the difference. Gaps get raised here and solved in Phase E. Third, **the extracts were taken on different days**, which generates dozens of false differences and destroys the room's confidence by the fortieth row. Take both extracts in one sitting and write the date on each.

Done when. Every difference carries one of three words, `Gap Candidates` holds a row per finding, `O29 Gap` is no longer empty, `R12 Gap closed by Work Package` has rows, and `RPT07 Gap Analysis` computes something for the first time.

Pack 5 — Roadmap sequencing

Fills. `O30 Work Package` `Start` , `End` , `Approval` ; `O53 Transition Architecture` `Work packages` and real written exit criteria in place of a count; `O56 Project` `Lead` ; and `O66 Architecture Increment` `Lands` .

Who must be in the room. `GB-02 Investment Committee` members or their delegates — the **Chief Financial Officer**, the **Chief Information Officer** and the `ROL-08 Chief Enterprise Architect`. The **Head of Delivery** (`ROL-13`), who owns capacity. The **payments engineer**, whose scheme and rail change calendar is the single most useful number for sequencing and the one least often asked for. The **treasurer / head of ALM** if any package touches liquidity reporting.

How long. Three hours, and schedule a second session a fortnight later. The first produces a sequence; the second survives contact with the delivery plan.

Send five working days out. `RPT08 Roadmap` with the cost total computed — **$64.5m** across twelve packages — and `O54 Architecture Alternative` beside it, showing `ALT-B` recommended at **$46m** and `DEC-01` still `Outstanding` on `O60`. `O52 Implementation Factor`, all eight rows, because those are the constraints the sequence has to obey. `O62 Delivery Risk`, where `RSK-01 Integration product not selected` is rated `Certain`.

Running order	Minutes	What happens
1. The arithmetic	25	$64.5m against $46m. Settle whether the plan is re-cut or the alternative re-approved before sequencing anything
2. Dependencies	45	Read `O52`. `IF-01` blocks integration, `IF-02` blocks mastering. Draw the order those two force
3. Transition states	45	Which packages belong to `TA-1`, `TA-2`, `TA-3`, and the exit criteria for each — written out, not counted
4. Capacity	30	The Head of Delivery says how many packages can run at once. `CON-02` records that this is unknown; fix that here
5. Dates and leads	35	`Start`, `End` and a named `Lead` per project. Then check `O66` increments still land in order

Facilitation moves that work. Open with the cost arithmetic rather than the sequence, because a room that has not settled the number will re-sequence around it all afternoon. Make the exit criteria the deliverable: `O53` ships with "5 exit criteria" in a cell and no criteria anywhere, and a transition state you cannot test is a transition state Phase F cannot gate on. Ask the payments engineer for next year's mandatory scheme change before anybody proposes a date. And test the sequence arithmetically: if `TA-1` needs four packages and capacity is two at a time, the critical path is longer than the room thinks and that is subtraction, not opinion.

Three ways it derails. First, **the sequence is set by funding year** rather than by dependency, which produces a plan where integration precedes the product selection that enables it. Second, **everything starts in the first transition state**, because nobody wants to be last. Use the exit criteria as the test: a package that is not needed to exit `TA-1` is not in `TA-1`. Third, **capacity is assumed**. `CON-02 Delivery capacity unrecorded` and `IF-06 Concurrent capacity unknown` both say the number does not exist. Get it in the room or the plan is fiction.

Done when. Twelve packages have dates that obey `O52`, three transition states have written exit criteria, every project has a named lead, `O66`'s six increments land in dependency order, and the cost total reconciles to a decision recorded in `O60`.

Quality checklists, one per ADM phase

Tick these against the repository, not against your memory of it. Every line is an "every X has a Y" assertion and every one is checkable in under a minute.

Preliminary Phase

✓	The assertion	Check it against
	The four governance sheets exist: `01 Version Control`, `02 Change Log`, `03 Change Requests`, `04 Baselines`	the tab bar
	Every object and relationship sheet carries the seven record-keeping columns	any `O` sheet, columns to the right
	Every one of the 24 principles has a `Statement`, an `Owner` and a `Status`	`O05`
	Every one of the 5 governance bodies has a `Cadence`, `Membership` and `Mandate`	`O57`
	Every one of the 4 gates names what it tests and when	`O58`
	Every standard has a `Status` and a `Conformance`, and no `Current` standard is unassessed	`O32`, `RPT12`
	`ZZ Checks` has been read, dated and kept	`ZZ Checks`

Phase A — Architecture Vision

✓	The assertion	Check it against
	Every goal has an `Owner` who is a person	`O02`
	Every objective realises a goal and has a `Horizon`	`O03`, `R11`
	Every objective has at least one measure, and every measure a `Baseline` and `Cadence`	`R10`, `O04`
	Every requirement names a driver	`O06`, `R08`
	Every driver has an `Owner` and a `Pressure`	`O01`
	Every `Must` requirement reaches a building block	`RPT06`
	No assumption has been `Open` for more than a quarter	`O08`

Phase B — Business Architecture

✓	The assertion	Check it against
	Every level-1 capability decomposes to level 3	`O13`, `R01`
	Every assessed capability has `Criticality`, `Current`, `Target`, `Disposition`	`O13`
	Level-1 and level-2 scores are rolled up, not separately typed	`O13`, `RPT03`
	Every value stream's stages are named rows, not a count	`O14`, `O16`
	Every value stream names the capabilities it actually uses	`R02`
	Every capability has an owning organisation unit	`R22`
	Every role has an actor and every actor a `Status`	`R21`, `O10`
	Every business building block has an acceptance criterion that can be failed	`O35`

Phase C — Data Architecture

✓	The assertion	Check it against
	Every entity has a `Definition` in the business's own words	`O20`
	No entity holding personal data is classified `Public`	`O20`, `RPT05`
	Every entity has a named person as `Owner`, not a department	`O20`
	Every entity has exactly one `Master` row, keyed by ref	`R05`
	Every entity has attributes, or a stated reason it has none	`O37`
	Every master entity has a rule with a `Golden source`	`O43`
	Every entity feeding a regulatory report has `Lineage = Full`	`O20`, `RPT05`
	Every metadata element has a `Coverage` verdict	`O42`

Phase C — Application Architecture

✓	The assertion	Check it against
	Every application has `Disposition`, `Criticality`, `Health` and a named `Owner`	`O23`, `RPT02`
	Every application supports a business service, or appears on `RPT13` with a reason	`R04`, `RPT13`
	Every interface has `Type`, `Frequency`, `Managed` and `Logged`	`R06`, `RPT04`
	No `Direct DB read` exists without a dated dispensation	`R06`, `O61`
	Every application declares the entities it masters and consumes	`R05`
	Every product is administered by an application	`O18`, `R25`
	Every information system service names its application and `Published` state	`O24`

Phase D — Technology Architecture

✓	The assertion	Check it against
	Every physical component has a `Support end` date and a computed `Position`	`O27`, `RPT10`
	Every application is deployed on a component, with an `Environment`	`R07`
	Every application has a tier, a recovery objective and a test date	`O49`
	Every environment answers `Personal data present` correctly	`O48`
	Every network link answers `Diverse path`	`O50`
	No security control reads `Absent` without a gap against it	`O41`, `RPT11`
	Every security policy is enforced at a named point	`O64`, `R17`
	Every standard applies to at least one application	`O32`, `R18`

Phase E — Opportunities and Solutions

✓	The assertion	Check it against
	Every `ZZ Checks` result above threshold has a gap	`ZZ Checks`, `029`
	Every gap has `Domain`, `Building block`, `Severity`, `Status`, `Due`	`029`, `RPT07`
	Every gap is closed by a work package	`R12`
	Every building block is delivered by a work package	`055`, `R13`
	Every work package has `Cost`, `Start`, `End`, `Domains`	`030`
	Work package costs sum to the chosen alternative's cost	`030`, `054`, `060`
	Every implementation factor states a `Deduction` that changed the plan	`052`
	Every transition state has written exit criteria	`053`

Phase F — Migration Planning

✓	The assertion	Check it against
	Every work package has a project with a named `Lead`	`056`, `R14`
	Every transition state contains at least one work package	`R15`
	Every increment has a `Lands` month and a delivering project	`066`, `R23`
	Every data migration has a from, a to and a `Status`	`039`
	Every delivery risk has `Likelihood`, `Impact` and an `Owner` who has seen it	`062`
	Every Phase E and F deliverable has moved off `Draft`	`031`
	`Delivery` is computed from `End`, not typed	`030`, `RPT08`

Phase G — Implementation Governance

✓	The assertion	Check it against
	Every work package has an architecture contract	`033`
	Every `Compliance status` is backed by an assessment	`033`, `059`
	Every acceptance criterion has a verdict	`059`, `R24`
	Every gate held has a dated decision	`058`, `060`
	Every dispensation has an `Expires` date	`061`
	Every control objective has evidence, or a gap	`063`, `RPT09`
	No `Non-compliant` contract is in flight without a board decision	`033`, `060`

Phase H and Requirements Management

✓	The assertion	Check it against
	Every record change has a change-log row and a `Change Ref`	`02 Change Log`
	No record has a `Changed By` and a blank `Change Ref`	every `O` sheet
	Every `Outstanding` decision has a body and a next meeting date	`060`, `057`
	Every change request has a `Status` and a named owner	`061`
	Every requirement reaches a building block	`RPT06`
	`ZZ Checks` has been re-run and every delta explained	`ZZ Checks`
	`EXT` is unchanged, or was extended by the four-edit rule	`EXT`, `00 Index`

The ninety-day calendar

Thirteen weeks against the repository. Volume 1, chapter 18, runs the learning track in the same weeks — weeks one and two sit beside *Week one: eight documents, and where they live* and *Weeks one to three: who to meet, in what order*; weeks three to eight beside *What to draw first, and why it must be small* and *What to refuse in month one*; weeks nine onwards beside *Six numbers that tell you whether the estate is healthy*. Read *The politics, honestly* before week one and again in week six, and *What week thirteen should look like* at the start, not the end.

One rule before week one. **Weeks one and two touch no sheet.** Everything after week two changes the record, and until the change log exists in week three there is no author for any change you make.

Figure 15.1 The repository track across thirteen weeks. Weeks one and two touch no sheet; week three adds the change control the file does not have; the five packs run from week five.

Week	What you do in the repository	Sheets you touch
1	Read `00 Index`, `RPT01 Dashboard` and `ZZ Checks` end to end. Then `O60` and `O61` — seven decisions, four `Outstanding`, five change requests. Write down every number that surprises you	none — read only
2	Read `RPT04`, `RPT05`, `RPT09`, `RPT13`. Add the index's itemised record counts and compare with the stated 878. Check which `ZZ Checks` results are computed and which are typed	none — read only
3	Add `01 Version Control`, `02 Change Log`, `03 Change Requests`, `04 Baselines` and the seven record-keeping columns. Raise `CR-001` against something that does not matter	the four new sheets
4	Fix the mechanical faults, one change request each: `R05`'s name-keyed rows, `O46`'s duplicate `Ref` column, `O48`'s personal-data flags, `O19`'s counterparties	`R05`, `O46`, `O48`, `O19`
5	Pack 1 on `C11 Finance & Regulatory Reporting` and `C12 Channel & Distribution` — the worst two scores in the file	`O13`, `R01`
6	Pack 1 on two more areas. Recompute `RPT03` as a roll-up. Take `C11` to the financial controller and the regulatory reporting analyst	`O13`, `RPT03`
7	Pack 2 on the master spine: `E01`, `E02`, `E03`, `E05`. Settle Party or record it `CONTESTED`	`O20`, `R05`, `O43`
8	Pack 2 on the remaining ten entities. Fix the six `Public` classifications with the information officer	`O20`, `O37`, `O42`, `RPT05`
9	Fill `R06` properly with the integration and API engineer: `Type`, `Frequency`, `Managed`, `Logged` on all 32 rows. Then read `RPT04` and build the integration case	`R06`, `RPT04`
10	Pack 3. Fourteen applications, named owners, evidenced health. Then `O49`: twelve tiers and ten missing test dates	`O23`, `O49`, `RPT02`
11	Remap `R16` with the compliance officer — it pairs `CO-0n` to `SEC-0n` positionally — and rewrite the seven placeholder `Control objective` cells	`O63`, `O41`, `R16`, `RPT09`
12	Build the five extracts by hand, run the comparison tool, run Pack 4 on the first sixty differences, paste the agreed gaps	`O29`, `R12`, `RPT07`
13	Pack 5. Write `O53`'s exit criteria out. Reconcile `RPT08` to `O54` and `DEC-01`. Cut `BL-001` on `04 Baselines`	`O30`, `O53`, `O56`, `O66`, `04 Baselines`

What you can show, and when

Point	What you put in front of people	Why it lands
End of week 2	One page. The repository's own numbers against its own documentation: 878 records stated, 846 itemised; "every type carries records" against two empty sheets; four `Outstanding` decisions and `CR-G03` raised and unactioned. And the nine findings on `RPT01` that *are* computed and correct	It costs nothing, needs nobody's cooperation, and establishes that you read the record before forming an opinion. It also gets `GB-01` to approve something in your second week, which is a habit worth starting
End of week 6	`RPT03 Capability Heatmap` for four level-1 areas, scored by the executives who own them and rolled up properly. Alongside it, `RPT05` showing three entities with no lineage into the numbers those executives sign	Four executives have scored their own area and will defend the figure. The lineage finding opens the data conversation with their fingerprints on it rather than yours
End of week 13	`RPT07 Gap Analysis` computing for the first time, every gap with a severity, a due date and a package. `RPT08` reconciled: $64.5m against $46m, with a recommendation. `BL-001` cut and approved	A gap register with named owners is what makes the practice real. The cost reconciliation is the first thing you will have produced that a programme director cannot argue with, because it is addition

YOUR NINETY DAYS

Your start date, and your week 2, week 6 and week 13 show dates:

The three sheets you have decided not to touch in year one, and why:

The person who will approve `CR-001`, by name, and the body they sit on:

The scope you have agreed for the thirteen weeks — which lines of business, which capabilities, which applications are out:

If you only do five things

You will not get the time to do all of it. Nobody does. These five, in this order, in the first six months, and the repository stays true enough to be worth keeping.

#	Do this	Sheets	Why this one
1	**Add change control and make every change go through it.** Four sheets, seven columns, one change request per edit, including your own corrections	`01` – `04`, every `0` and `R` sheet	Everything else can be caught up later. This cannot, because retrofitting a change log means inventing history. It is also the only thing in the file that would answer a Joint Standard 1 of 2023 question about how a decision was taken
2	**Fix the four mechanical faults.** `R05`'s mixed key column, `046`'s duplicate `Ref`, `048`'s inverted personal-data flags, `019`'s application-as-counterparty	`R05`, `046`, `048`, `019`	Half a day in total, and until `R05` keys consistently nothing you compute about data authority is trustworthy. These are the cheapest credible wins in the repository
3	**Assess capability at level 3 with the people who own it, and roll it up.** `Criticality` before `Current`, every score evidenced	`013`, `RPT03`	It is the only artefact here a line executive reads without translation, and it is the input to every prioritisation argument you will have for three years. Done properly it also kills the level-1 figures that were never derived from anything
4	**Settle data authority, or record it as contested.** One master per entity, one named person, one classification that survives the information officer	`020`, `R05`, `RPT05`	Unresolved data authority is the most expensive thing in a banking estate and the hardest to retrofit. `DEC-04` is already `Outstanding`, so you are completing an instruction rather than starting an argument
5	**Fill `Managed` and `Logged` on all 32 interfaces, then populate `029 Gap`.** The second follows from the first	`R06`, `029`, `RPT04`, `RPT07`	Nineteen unmanaged and twenty unlogged is the number that funds an integration platform, and it tells you which applications cannot be changed safely. `CR-G03` already asks for the gap register, which makes it the easiest thing in this list to get approved

What all five have in common: none of them needs a budget, none needs a vendor, and every one of them ends with a number somebody else has to act on. That is the whole method.

THE CHAPTER IN TEN LINES

- Five packs, each naming the cells it fills. A workshop that fills no cells was a meeting.
- Pack 1 scores O13 at level 3 only and rolls up. The shipped level-1 scores were never derived from anything.
- Pack 2 fills O20 and fixes R05, where fourteen of twenty-eight rows key on an application name.
- Pack 3 scores O23 with evidence from service management, and picks up O49's ten missing test dates.
- Pack 4 needs hand-built extracts — this repository has no export sheets — and ends with O29 Gap no longer empty.
- Pack 5 opens with the cost arithmetic, not the sequence: $64.5m of packages against the $46m of ALT-B.
- Ten phase checklists, every line an 'every X has a Y' assertion checkable in under a minute.
- Weeks one and two touch no sheet. Week three adds the four governance sheets and the seven columns.
- Show the documentation contradictions at week 2, four executive-scored capability areas at week 6, a computing gap register at week 13.
- If you only do five things: change control, the four mechanical faults, capability at level 3, data authority, the interface columns.

SOURCES FOR THIS CHAPTER

- **1.** Banking_TOGAF_Metamodel.xlsx, sheets O10 Actor, O11 Role and O57 Governance Body
- **2.** Banking_TOGAF_Metamodel.xlsx, sheets O13 Business Capability, O20 Data Entity, O23 Application Component and R05 Application manages Data En
- **3.** Banking_Architecture_Comparison_Tool_TEMPLATE.xlsx, sheets Review and Gap Candidates (design specification, October 2026)
- **4.** Banking_TOGAF_Metamodel.xlsx, sheets O30 Work Package, O52 Implementation Factor, O53 Transition Architecture, O54 Architecture Alternative and O66 Architecture Increment
- **5.** Banking_TOGAF_Metamodel.xlsx, sheets O60 Governance Decision, O61 Governance Change Request and ZZ Checks
- **6.** The Open Group, TOGAF Standard, Architecture Development Method https://pubs.opengroup.org/architecture/togaf-standard/

APPENDIX D

The repository in ArchiMate 4

One hundred and five sheets of rows, forty-two elements, eleven relationships — and the rule that when they disagree, the rows are right.

Why this mapping exists

You have been handed two descriptions of one architecture and they do not resemble each other on the page. `Banking_TOGAF_Metamodel.xlsx` is a TOGAF content metamodel written out as rows: 105 sheets, 878 records, 465 resolved relationship instances, the first column of every object sheet headed `Ref` [1]. `Banking_ArchiMate4_Views.pptx` is a pack of pictures. Nothing in either file says how one becomes the other. This appendix is that bridge. Ardent Union Bank is an invented South African composite bank and its data is real only inside this workbook.

The repository is the **record**. It answers by filter and formula: which of the 32 requirements reaches no building block, which entity has two golden sources, how many components are past support. A diagram cannot answer those at all.

An ArchiMate 4 view is the **argument**. It takes six rows out of eight hundred and shows the shape: this driver pushes this goal, this outcome realises it, this requirement traces to this block, and nothing funds the block. That is a sentence you can say to a Chief Risk Officer in thirty seconds, and no filter over `RPT06 Requirements Traceability` says it as fast. Neither artefact replaces the other, and authority runs one way.

> **THE RULE**
>
> The repository is authoritative. A view that disagrees with it is wrong — not a variant, not a simplification, not "the version we drew for the board". Where a view shows an element the repository does not hold, either the repository is missing a record and you raise a row on `O61 Governance Change Request`, or the view is inventing architecture and you delete it. There is no third case.

That rule matters more here than in most repositories, because this one has no change control. There is no `01 Version Control` sheet, no `02 Change Log`, and no `Version`, `Last Changed` or `Approved By` column anywhere on the 64 `O` sheets. The only record of a model change is whatever rows somebody put on `O61`, which holds five. Two views drawn a quarter apart cannot be reconciled by date — only by `Ref`. The `Ref` on the label is the whole of the audit trail.

Three working consequences.

Every element on a view carries its `Ref`. `A04 Loan Origination`, not "Loan Origination". A box with a name only cannot be reconciled against anything, and four of this repository's sheets hold rows whose names are identical to rows on another sheet.

Views are generated, not drawn. Every view below is a filter over one object sheet and one or two relationship sheets. This repository has **no export sheets** — the insurance set shipped three and banking ships none — so each extract is a filter, a copy and a paste. Write the filter beside the view so the next person can re-run it.

The loss runs one way. Everything on a view exists in the repository; much of the repository cannot be drawn. `Criticality`, `Current`, `Target`, `Disposition`, `Health`, `Conformance`, `Tested`, `Managed` and `Logged` have no expression in the notation. That is why the repository exists.

Volume 1, chapter 16, *Modelling banking in ArchiMate*, teaches the notation — *Five decisions that settle most of the model*, *The mapping*, *The honest limits*. This appendix assumes you have read it and maps *these sheets* onto it. Where the two disagree about an element, volume 1 is right. Where they disagree about a sheet, open the sheet.

The object mapping — all 65 object sheets

The arithmetic first, because the file's own index gets it half right. `00 Index` and `ZZ Checks` both state **65 object types**. The workbook holds **64 sheets prefixed `O`** — `001` to `066` with `034` and `051` absent, which are real gaps and not an export error — plus **`EXT Industry Extension`** and its six types. Sixty-four sheets plus that one tab is the 65 the index counts. Say "64 `O` sheets" when you mean sheets and "65 object types" when you mean the index.

Two differences between the repository's domain labels and ArchiMate 4's layers matter. `013 Business Capability` and `014 Value Stream` carry the subtitle *Business domain* and belong to the **strategy** layer. And nine ArchiMate 4 elements carry no layer at all — Role, Collaboration, Process, Function, Service, Event, Path, Grouping and Location — so the four service sheets `015`, `024`, `025` and `028` all collapse onto **Service** and the layer column reads "none" for all four. Without the `Ref` on the label, `BSV-01` and `ISS-01` are the same box.

Twenty-two of the 64 `O` sheets have no ArchiMate 4 element of their own. Eight substitute an element that is honest about what it is; fourteen stay in the repository, and a view that needs one says in its notes that it cannot show it. A near fit is worse than nothing, because the near fit gets believed.

Motivation — eight sheets

Sheet	ArchiMate 4 element	Layer	Why this element	Watch out for
`001 Driver`	Driver	Motivation	A condition in the environment that motivates intent	Seven of the 13 are the same obligations as `CO-01` – `CO-07` on `063`. Model one, not both
`002 Goal`	Goal	Motivation	Intent with no end date in the notation	`Owner` holds a role name, not a `Ref` into `011`, so Assignment needs a manual lookup
`003 Objective`	Outcome	Motivation	Time-bounded and measurable: `Horizon` reads 12 months to 3 years	Not a Goal. Collapse the two and the realisation chain disappears
`004 Measure`	No element	—	No metric element exists, and Value is not a number	`Baseline`, `Unit` and `Cadence` become properties on the Outcome. Movement lives in `RPT01 Dashboard`
`005 Principle`	Principle	Motivation	A rule informing how the enterprise acts	Group the 24 by `Category`. `Status` reads Draft on five, and the notation cannot say "draft"
`006 Requirement`	Requirement	Motivation	A statement a design satisfies or fails	32 rows. The `Driver` column duplicates `R08`; trust the sheet, draw from the relationship
`007 Constraint`	No element — carry as Requirement	Motivation	The language holds no Constraint	A Requirement with `Type` said plainly in the notes. Eight rows, five of them real blockers
`008 Assumption`	No element	—	An assumption is not an analysis outcome, so Assessment misstates it	Repository only. `ASM-03` and `ASM-04` read At risk and belong in the view notes

Strategy — three sheets

Sheet	ArchiMate 4 element	Layer	Why this element	Watch out for
`013 Business Capability`	Capability	Strategy	An ability with an owner, a maturity and a target	120 rows, three levels, 12 at level 1. One level per view; maturity is a property
`014 Value Stream`	Value Stream	Strategy	Activity ending in a result for a stakeholder	`Stages` is a count ("6 stages"), not rows. Draw stages and you are inventing them
`054 Architecture Alternative`	Course of Action	Strategy	An approach chosen to reach a goal	Publish `ALT-B`, the recommended one. Keep the four rejected in the repository, rejected

> **`014` AND `R02` DISAGREE, AND ONE OF THEM IS WRONG**
>
> `014`'s `Capabilities` column and `R02 Value stream uses Capability` are two records of the same fact. They agree on `VS01`, `VS02` and `VS03`. On `VS04 Arrears to Resolution` the column reads `C01, C02, C03` and `R02` reads `C04`, `C05`, `C06`; on `VS05 Trade to Report` the column reads `C02, C03, C04` and `R02` reads `C05`, `C06`, `C07`. Draw from the relationship sheet, raise the column as a correction on `O61`, and never load a value stream view from both.

Organisation and business — ten sheets

Sheet	ArchiMate 4 element	Layer	Why this element	Watch out for
`009 Organization Unit`	Business Actor	Business	A structure that performs behaviour	`Head` holds a title string. `Headcount` cannot be drawn. Not a licence to model the org chart
`010 Actor`	Business Actor	Business	Identity, not position	All 18 rows are position names identical to `011`. This sheet holds roles; model `011` and resolve people when they are named
`011 Role`	Role	none	A position an actor occupies	There is no "Business Role" element. `Role` has no layer, so it sits anywhere
`012 Location`	Location	none	A place	Eight rows mixing offices, centres and data centres. `Scale` is a headcount, undrawably
`015 Business Service`	Service	none	Behaviour visible outside, with a consumer	`Exposure` reads Internal on nine of 12, which contradicts "externally consumable" in the description
`016 Business Process`	Process	none	A sequence producing a defined outcome	The `Value stream` column is a relationship held in a column. No `R` sheet carries it
`017 Business Function`	Function	none	Behaviour with no fixed order	All 12 share a name with their level-1 capability in `013`. Both on one view says the same thing twice
`018 Product`	Product	Business	What is sold, priced and contracted	Eight products; `X01` on `EXT` is the same concept. Model the sheet, not the extension row
`019 Contract`	No element — carry as Business Object	Business	The language holds no Contract	`Counterparty` names an application, not a company, so the vendor is nowhere in the repository
`035 Business Building Block`	Grouping	none	A package of required ability, not a thing that exists	Realization to the `Capability` named in column three. Duplicated verbatim on `055`

Data — eleven sheets

Sheet	ArchiMate 4 element	Layer	Why this element	Watch out for
`020 Data Entity`	Data Object; Business Object on a conceptual view	Application	An encapsulation of data	`Master system` holds a name, not a `Ref`. `Lineage` reads Absent on `E08`, `E10` and `E11`
`021 Logical Data Component`	Grouping	none	A boundary over entities, not a system	One entity per component, so the Grouping adds nothing a Data Object does not carry. Keep it for migration scoping
`022 Physical Data Component`	Artifact	Technology	A deployed store is a deployable	`Type` names a storage class, not a product, so there is no System Software to put beneath it
`036 Data Entity Relationship`	No element — a relationship	—	Relationships living on an object sheet	The same 11 rows as `R19`. Association between Data Objects; `Cardinality` is lost
`037 Data Attribute`	No element	—	Attributes are properties of an element	60 rows, never drawn. `Classification` is the filter behind the personal-information view
`038 Data Set`	Data Object	Application	A named collection with a consumer	Six sets, no relationship sheet to anything. Every one is an orphan and `RPT13` misses them
`039 Data Migration`	No element	—	A movement is not a construct	Properties on the Work Package doing the move, or a Grouping over source and target Artifacts
`040 Source To Target Mapping`	No element	—	A transformation rule is not architecture	Nearest is a Flow between Artifacts. `Rule` — Direct, Lookup, Transform — is lost
`042 Metadata Element`	No element	—	Metadata about the model is not in the model	Repository only. `Coverage` reads "Recorded, not held" on four rows and no notation says that
`043 Master Data Rule`	No element — carry as Requirement	Motivation	A survivorship rule is policy, not structure	`Golden source` disagrees with `020`'s `Master system` on eight of ten entities. That is the finding
`044 Data Building Block`	Grouping	none	As `035`, one layer down	Realization to the `Entity`. Duplicated verbatim on `055`

Application — three sheets

Sheet	ArchiMate 4 element	Layer	Why this element	Watch out for
`023 Application Component`	Application Component	Application	Encapsulated functionality with an interface	`A01 Core Banking` is one row and three systems — volume 1, chapter 16, *The core banking system, which is three systems wearing one name*
`024 Information System Service`	Service	none	Automated behaviour with a consumer	The realising application sits in the `Application` column. No `R` sheet carries application to service; that edge is authored
`045 Application Building Block`	Grouping	none	What must exist before anything is chosen	Ten rows, one per application, so each block is really a disposition target. Duplicated on `055`

Technology — eight sheets

Sheet	ArchiMate 4 element	Layer	Why this element	Watch out for
`025 Technology Service`	Service	none	A technical capability with a consumer	Twelve rows and no relationship sheet to anything. The whole sheet is unreferenced
`026 Logical Technology Component`	Node	Technology	Vendor-independent infrastructure	Node, not Device, at logical level. `LTC-06` reads Target and must not appear on a baseline view
`027 Physical Technology Component`	Node, Device, System Software or Communication Network by `Class`	Technology	The row names a product class, so name its kind	`Support end` drives `RPT10 Technology Currency`, not the picture. Three are past support today
`028 Platform Service`	Service	none	A grouped capability consumed by applications	Eight rows, also unreferenced by any `R` sheet. Serving to a component is an authored edge
`046 Technology Building Block`	Grouping	none	As `035` and `045`	Column three is also headed `Ref` and holds `T01` – `T07`, which resolve to nothing. A broken key
`048 Environment`	Grouping	none	A named set of deployed things, not a thing	Aggregation over the Nodes in it. `Personal data present` reads Yes on `ENV-02` and `ENV-04`
`049 Resilience Tier`	No element	—	A recovery objective is a property	Properties on the Application Component in column two. Ten of 12 read Never tested
`050 Network Link`	Path, realised by a Communication Network	Technology	Path is the connection; the network realises it	`From` and `To` hold `012` names. `NWL-01` and `NWL-09` are the same pair twice with different answers

Implementation and migration — eleven sheets

Sheet	ArchiMate 4 element	Layer	Why this element	Watch out for
`029 Gap`	No element — carry as Assessment	Motivation	The implementation aspect holds only Work Package, Deliverable and Plateau	**Zero records.** Every gap on every view in this volume is authored until you populate it
`030 Work Package`	Work Package	Implementation	A set of actions to achieve a defined result	`Delivery` reads Past end date on four of 12 and the notation cannot say so
`031 Deliverable`	Deliverable	Implementation	A specified, signed-off output	20 rows keyed by ADM `Phase`. No relationship sheet joins them to a package
`032 Standard`	Requirement, with the specification as an Artifact	Motivation	A standard is testable against a component	`Conformance` comes from `R18`, never from the element. Three of 16 read Conformant
`033 Architecture Contract`	No element — carry as Business Object	Business	No Contract element, and the useful fact is the verdict	`Compliance status` reads "Not yet assessable" on six of eight. Show `059` instead, when it has rows
`052 Implementation Factor`	No element	—	Planning rationale is not architecture	Repository only. `Deduction` belongs in the view notes, where a reader will act on it
`053 Transition Architecture`	Plateau	Implementation	A state the enterprise can stop at	`Exit criteria` is a count ("6 exit criteria"), not the criteria. The criteria are nowhere
`055 Building Block`	No element of its own	—	A composed view over `035`, `044`, `045` and `046`	Model it separately and every block appears twice. Filter with it; never draw it
`056 Project`	Work Package	Implementation	A chartered unit of delivery is the same element	Twelve projects against twelve packages, one to one. Draw both and the cost is counted twice
`062 Delivery Risk`	No element	—	A delivery risk is not an architecture statement	Repository only. Three read Certain, which makes them issues, not risks
`066 Architecture Increment`	No element	—	The notation models states, not deltas	`INC-1`, `INC-2` and `INC-4` share names with `TA-1`, `TA-2` and `TA-3`. A Grouping over what each adds

Governance and security — ten sheets

Sheet	ArchiMate 4 element	Layer	Why this element	Watch out for
`041 Security Control`	No element — carry as Requirement	Motivation	The language holds no control	A control reading Absent is a Requirement with no Realization edge, and that absence is the finding. Four of eight
`047 Reference Model Entry`	No element	—	It records what should be chosen, not what exists	`Status` becomes a property on the component. Seven of 11 duplicate `026` row for row
`057 Governance Body`	Collaboration; Business Actor where it is one unit	Business	Four of five draw members from several units	`GB-05 Architecture Function` is a unit, not a collaboration. `Mandate` cannot be drawn and is the column that matters
`058 Governance Gate`	Event	none	A point at which work is assessed	The assessing behaviour is a Process assigned to the body. `What it tests` is documentation
`059 Compliance Assessment`	Assessment	Motivation	A verdict reached by analysis	**Zero records.** Phase G has no evidence in this repository until you fill it
`060 Governance Decision`	No element	—	The effect of a decision is in the model; the decision is not	Repository only. Four of seven read Outstanding and three of those block Phase E
`061 Governance Change Request`	No element	—	It is about the model, not in it	Repository only, and the only change mechanism this workbook has. `CR-G01` expires 2029-06
`063 Control Objective`	Requirement	Motivation	What must be true is a need statement	The obligation is already `DRV-01` – `DRV-07`. Four of seven have no mechanism or a mechanism in breach
`064 Security Policy`	Principle	Motivation	A board-language commitment	`Position` reads Breached on `POL-03` and the notation cannot say "breached"
`065 Enforcement Point`	No element of its own — the row names an existing one	varies	A gateway, identity provider, platform or gate is already modelled	`EP-03 API gateway` reads "Product not yet selected", so there is nothing to map it to

`EXT Industry Extension` — six banking object types

`EXT` answers a question the TOGAF content metamodel does not ask: what does banking need that the core does not hold. Six rows, `X01` – `X06`, third column headed `What it holds and why the core metamodel is not enough` [1]. It is a **declaration**, not a sheet of records — five of the six already have

rows elsewhere in the workbook. Read it as the list of concepts you must be able to answer for, then find where the answers live.

Row	What it holds	ArchiMate 4	Where the records already are	What to do
`X01 Product`	A product with terms, pricing and eligibility	Product	`O18 Product`, 8 rows	Model `O18`. The extension row is a pointer, not a second product catalogue
`X02 Arrangement`	An instance of a product held by a customer	Business Object	`E05 Arrangement` on `O20`	Business Object on a conceptual view, Data Object on a data view. Never both on one view
`X03 Exposure`	Amount at risk to a counterparty, by facility	No equivalent	`E08 Credit Exposure` on `O20`	The entity is a Data Object; the *amount* is a number and no element carries numbers
`X04 Limit`	An authorised ceiling on exposure or activity	No equivalent	`E10 Limit` on `O20`	Data Object for the record; the ceiling itself is a Requirement if it must be tested at a gate
`X05 Regulatory Report`	A supervisory submission with its lineage	Business Object; Deliverable where it is signed	`E13 Regulatory Report` on `O20`, `A12` on `O23`	Model the submission as a Business Object and its lineage as Access edges, not as one box
`X06 Payment Scheme`	A clearing and settlement arrangement	No single equivalent	Nowhere. No sheet holds a scheme	Business Actor for the operator plus a Business Interface for participation, and raise the absence

Three of the six — `X03`, `X04` and `X06` — have no faithful element, for the reason volume 1, chapter 16 gives in *The honest limits*: ArchiMate 4 has no quantity, no money and no time. An exposure is a number against a counterparty at a moment; a limit is a number with an authority behind it. Both are properties, and the thing they are properties of is the entity.

`X06 Payment Scheme` is the serious absence. South African banking runs on shared rails and no sheet holds them — `DRV-04 Payment scheme rules` is a driver, `CO-04` is an obligation reading "No mechanism", and no object anywhere names a scheme the bank participates in. Volume 1, chapter 15, *The payment system, and who writes the rules now*, says why that matters. Extend `EXT` by adding the row **and** the `O` sheet that holds the records, with a `Ref` prefix nothing else uses. Adding a row to `EXT` alone records an intention and changes nothing.

The relationship mapping — all 25 `R` sheets

The relationship sheets run `R01` to `R25` with no gaps and are named after **what they mean** rather than after their endpoints: `R01 Capability decomposition`, not "R01 CAP-CAP". Several tab names are cut off at Excel's 31-character limit — `R05 Application manages Data En`, `R23 Increment`

`delivered by Proj` — and the full sentence sits in the first cell of the sheet. Spell the tab as the tab spells it; use the full sentence when you explain it.

The sheet's direction is the author's sentence, not an ArchiMate direction. **Thirteen of the 25 reverse when drawn**, because the notation's arrow runs the other way: a service realises a capability, an outcome realises a goal, an actor is assigned to a role, a location is assigned to the thing standing in it. Draw a reversed one as the sheet reads and the model validates and means the opposite.

Sheet	From → To	ArchiMate 4 relationship	Direction in the model	Note
`R01 Capability decomposition`	Parent → Child	Composition	As sheet	108 instances over three levels. `Level` is on the row; one level per view
`R02 Value stream uses Capabilit`	Value stream → Capability	Serving	**Reversed** — Capability serves Value Stream	Disagrees with `O14`'s own column on `VS04` and `VS05`
`R03 Capability realised by Busi`	Capability → Business service	Realization	**Reversed** — Service realises Capability	Twelve rows, one per level-1 capability, so coverage looks perfect and is untested
`R04 Business Service supported`	Business service → Application	Realization	**Reversed** — Component realises Service	Twelve rows pairing `BSV-nn` with `A-nn` positionally; `BSV-01 Customer Management` to `A01 Core Banking` is the only pairing that reads true
`R05 Application manages Data En`	Application → Data entity	Access	As sheet	`Role` sets the type: Master writes, Consumer reads. The first 14 rows hold the application *name* where a `Ref` belongs
`R06 Application interfaces with`	Source → Target	Flow	As sheet	32 rows, `INT-001` – `INT-032`. `Type` reads Direct DB read on ten; `Managed` and `Logged` are properties and both are mostly No
`R07 Application deployed on Tec`	Application → Platform component	Assignment	**Reversed** — Node assigned to Component	The notation assigns the platform to the thing running on it. `Environment` reads Production on all 14
`R08 Requirement derives from Dr`	Requirement → Driver	Influence	**Reversed** — Driver influences Requirement	32 rows, the cleanest trace in the repository. The alternative is to route through `O03`, as the motivation view does
`R09 Requirement satisfied by Bu`	Requirement → Building block	Realization	**Reversed** — Grouping realises Requirement	`Traced` is the column `RPT06` reads. This is the edge an auditor asks for
`R10 Objective measured by Measu`	Objective → Measure	**Not expressible**	—	Measure is not an element. The values sit on the Outcome and the movement sits in `RPT01`
`R11 Goal realised by Objective`	Goal → Objective	Realization	**Reversed** — Outcome realises Goal	The sheet reads "is realised by"; drawn that way the whole motivation chain inverts
`R12 Gap closed by Work Package`	Gap → Work package	Association	As sheet	Not Realization: a package does not realise a gap. **Zero instances**, because `O29` is empty

`R13 Work Package delivers Build`	Work package → Building block	Realization	As sheet	37 rows, one per `055` block. The standard implementation edge; nothing is lost
`R14 Project delivers Work Packa`	Project → Work package	Composition	As sheet	Both ends map to Work Package. One to one across all 12, so the composition carries no information
`R15 Transition contains Work Pa`	Transition → Work package	Realization	**Reversed** — Work Package realises Plateau	A plateau is a state; the package brings it about. Seven of 12 packages appear; five sit in no transition
`R16 Control Objective met by Co`	Control objective → Control	Realization	**Reversed** — the control Requirement realises the objective Requirement	Both ends are Requirements under this mapping. Pairing is positional: `CO-01 Basel III / IV` to `SEC-01 Identity lifecycle` is not a true statement
`R17 Policy enforced at Enforcem`	Policy → Enforcement point	Realization where it enforces, Association where it does not	**Reversed** — the point realises the policy	Four rows. `Position` reads enforcement failure on three of them
`R18 Standard applies to Applica`	Standard → Application	Realization where conformant, otherwise Association	**Reversed** — the component realises the standard	Only the rows reading Conformant earn a Realization. Most read Partial or Breach
`R19 Data Entity relates to Data`	From → To	Association	As sheet	Eleven rows, identical to `036`. `Cardinality` cannot be drawn and carries the business rule
`R20 Application located at Loca`	Application → Location	Assignment	**Reversed** — Location assigned to Component	The notation assigns the place to the thing standing in it
`R21 Role performed by Actor`	Role → Actor	Assignment	**Reversed** — Actor assigned to Role	All 18 pair a `ROL-nn` with an identically named `ACT-nn`, which is a structural artefact, not information
`R22 Org Unit owns Capability`	Org unit → Capability	Assignment	As sheet	Twelve rows of ownership, one per level-1 capability. The other 108 capabilities have no owner
`R23 Increment delivered by Proj`	Increment → Project	**Not expressible** as drawn	—	No Increment element. Substitute a Grouping over what the project adds, or a Plateau and accept the overlap with `053`
`R24 Compliance Assessment tests`	Assessment → Building block	Association	As sheet	The Assessment carries no verdict; `Verdict` stays in the sheet. **Zero instances**

`R25 Application supports Produc`	Product → Application	Serving	**Reversed from the tab name**	The tab says application supports product; the columns run `Product` first. Route through `O24` and `O15` if the view needs a chain

> **WHAT YOU CANNOT DRAW, AND WHAT YOU MUST SAY INSTEAD**
>
> `R10` and `R23` have no ArchiMate 4 expression at either end. `R12` and `R24` are expressible and empty. `R16` and `R17` are expressible only by substituting Requirements and Principles for controls and policies. Six of 25 sheets, then, where a reader of the views alone reaches a wrong conclusion — that no measure tracks an objective, that no increment is delivered, that there are no gaps. Name the missing sheets in every view's notes. "This view cannot show whether `OBJ-10` is moving; see `MSR-10` on `O04`" is one line and it saves a meeting.

Three absences you will meet on your first morning, because the view you want needs an edge no sheet carries.

No driver to goal. `R08` runs requirement to driver and `R11` runs goal to objective. Nothing joins a driver to a goal, which is the first edge of every motivation view ever drawn. Author it, label it authored, raise the sheet on `O61`.

No role to process. `R21` pairs roles with actors and that is all. Nothing assigns a role to a process and `O16` has no role column, so the RACI that volume 1, chapter 19, *Process to role, as RACI*, pre-drafts has nowhere to land. This is the most useful sheet the repository is missing.

No application to information system service. `O24`'s `Application` column holds it and no relationship sheet does, so the cleanest edge in the layer is a column lookup. Same for `O25` and `O28`, which no sheet references at all.

The view set, by ADM phase

For each phase: the views to draw, what they contain, the sheets that populate them, and who asks. Where a standard ArchiMate 4 viewpoint fits, it is named; where none fits, the view is marked *named here* — yours to define and yours to defend. Fifty views across ten tables, and you will not produce them all; the note at the end says which five to start with.

Preliminary

View	Elements and relationships	Populated from	For
Organisation (Organization viewpoint)	Business Actors, Roles, Aggregation, Assignment	`009`, `010`, `011`, `R21` reversed	The sponsor, to agree whose architecture this is
Principles (Motivation viewpoint)	24 Principles in five Groupings by `Category`	`005`	`GB-01 Architecture Board`
Governance (*named here*)	Collaborations for bodies, Events for gates, the assessment Process between	`057`, `058`	`GB-04 Executive Committee`
Repository scope (*named here*)	A Grouping per domain over the sheets you will actually fill	`00 Index`, `ZZ Checks`	The architecture function, once, and then never again

Phase A — Architecture Vision

View	Elements and relationships	Populated from	For
Motivation (Motivation viewpoint)	Drivers, Goals, Outcomes, Principles, Requirements	`001` – `006`, `R08` reversed, `R11` reversed	The sponsor and `GB-02 Investment Committee`
Goal realisation (Goal Realization viewpoint)	One Goal, its Outcomes, the Requirements behind them	`002`, `003`, `006`, `R11` reversed	The executive owning that goal — six goals, six views
Stakeholder (Stakeholder viewpoint)	Stakeholders, the Drivers each carries, Assessments	`011`, `001`, plus authored Assessments	The Chief Enterprise Architect
Capability map, level 1 (Capability Map viewpoint)	The 12 level-1 Capabilities, shaded by `Current` against `Target`	`013`, `R01`, `R22`, mirrors `RPT03`	`GB-02 Investment Committee`
Value streams (Value Stream viewpoint)	Five Value Streams and the Capabilities serving each	`014`, `013`, `R02` reversed	The Chief Operating Officer
Solution concept (Layered viewpoint)	One line of business, motivation to technology, twenty elements at most	Everything, filtered hard	The board

Phase B — Business Architecture

View	Elements and relationships	Populated from	For
Business process co-operation	Processes, Triggering between stages, the Services realised	`O16`, `O15`, `R03` reversed	Operations, and the head of the value stream
Business function (Organization viewpoint)	Functions aggregating Processes	`O17`, `O16`	The Chief Operating Officer
Product (Product viewpoint)	Product aggregating Services, the Contract composed into it	`O18`, `O15`, `O19`, `R25`	Product heads and conduct compliance
Capability map, levels 1–3	120 Capabilities with owners and realising Services	`O13`, `R01`, `R22`, `R03` reversed	`GB-02 Investment Committee`
Service coverage (*named here*)	Services realising Capabilities, duplication and absence visible	`R03`, `R04`, both reversed	The Chief Enterprise Architect
Role and authority (*named here*)	Roles, the Processes each is accountable for, the limit on each	`O11`, `O16` — **and an authored edge, because no sheet joins them**	The Chief Risk Officer. Volume 1, chapter 16 calls this the view that earns the function its seat

Phase C — Data Architecture

View	Elements and relationships	Populated from	For
Information structure (Information Structure viewpoint)	14 Data Objects and the Associations between them	`O20`, `R19`	The Chief Data Officer
Data realisation (*named here*)	Entity, its logical Grouping, the Artifact that holds it	`O20`, `O21`, `O22`	Data architects and the migration lead
Authority (*named here*)	Data Objects with every Access edge marked Master, and the golden source beside it	`O20`, `O43`, `R05`, mirrors `RPT05`	`GB-03 Data Governance Council`
Lineage (*named here*)	The reporting entity, the Artifacts between it and source, Access and Flow	`O20`, `O40`, `O42`, `R05`	Finance and regulatory reporting
Classified data (*named here*)	Data Objects filtered to Confidential and Restricted, with the controls over them	`O20`, `O37`, `O41`, `O48`	The Information Officer
Migration (*named here*)	Source Artifact, target Artifact, a Flow per entity moving	`O39`, `O40`, `O22`	The programme board

Phase C — Application Architecture

View	Elements and relationships	Populated from	For
Application co-operation (Application Co-operation viewpoint)	14 Components, 32 Flows, dashed where `Managed` reads Unmanaged	`023`, `R06`, mirrors `RPT04`	The integration lead
Application usage (Application Usage viewpoint)	Components realising Services, Services serving Processes	`023`, `024`, `015`, `R04` reversed	The Chief Information Officer
Application structure (Application Structure viewpoint)	One Component decomposed into its real parts	`023`, `045`	The modernisation business case
Application and data (*named here*)	Components with Access edges to Data Objects, Master and Consumer typed	`R05`, mirrors `RPT05`	Data and application architects, together, in one room
Portfolio (*named here*)	Four Groupings by `Disposition`, labelled with `Health` and `Criticality`	`023`, mirrors `RPT02`	`GB-02 Investment Committee`
Direct database reads (*named here*)	Only the ten `R06` rows typed Direct DB read, as Access from component to Artifact	`R06`, `022`	The Architecture Board, at the first gate it holds

Phase D — Technology Architecture

View	Elements and relationships	Populated from	For
Technology (Technology viewpoint)	Nodes, Devices, System Software, Services, Realization	`025`, `026`, `027`	The infrastructure lead
Technology usage (Technology Usage viewpoint)	Nodes assigned to Components, Platform Services serving them	`R07` reversed, `028`	The Chief Information Officer
Deployment (Implementation and Deployment viewpoint)	Nodes at Locations, Paths between sites, Environments as Groupings	`027`, `012`, `048`, `050`, `R20` reversed	Operational resilience, and the Joint Standard evidence pack
Standards conformance (*named here*)	Requirements standing for Standards, Realization only where conformant	`032`, `R18` reversed, mirrors `RPT12`	`GB-01 Architecture Board`
Currency (*named here*)	Nodes shaded by `Position`, with `Support end` on the label	`027`, mirrors `RPT10`	The Chief Information Officer and the Chief Risk Officer together
Resilience (*named here*)	Application Components carrying `Tier`, `Recovery objective` and `Tested`	`049`, `023`, `050`	The Chief Risk Officer

Phase E — Opportunities and Solutions

View	Elements and relationships	Populated from	For
Gaps and building blocks (*named here*)	Assessments for gaps, Groupings for blocks, Association between	`029` once populated, `055`, `R12`	The programme board
Building block coverage (*named here*)	Blocks with the Work Package realising each, and the unfunded ones alone	`055`, `030`, `R13`	`GB-02 Investment Committee`
Architecture alternatives (Strategy viewpoint)	Five Courses of Action against the Goals each reaches	`054`, `002`	The board, once, and then it is decided
Transition overview (Migration viewpoint)	Three Plateaus, the packages realising each, Triggering between	`053`, `R15` reversed	The sponsor

Phase F — Migration Planning

View	Elements and relationships	Populated from	For
Implementation and migration (Implementation and Migration viewpoint)	Work Packages realising Plateaus and Deliverables	`030`, `031`, `053`, `R15` reversed	The programme board
Project (Project viewpoint)	Projects composing Work Packages	`056`, `R14`	The portfolio office
Sequencing (*named here*)	Work Packages with Triggering, `Start` and `End` on the label	`030`, `052`, mirrors `RPT08`	The programme director
Increment (*named here*)	A Grouping per increment over what it adds, against the Plateau it lands on	`066`, `R23` by substitution	The architecture function

Phase G — Implementation Governance

View	Elements and relationships	Populated from	For
Compliance (*named here*)	Assessments against block Groupings at gate Events	`059` once populated, `058`, `055`, `R24`	`GB-01 Architecture Board`
Gate and body (*named here*)	Collaborations, the Events each holds, the Process it runs	`057`, `058`, `060`	`GB-04 Executive Committee`
Enforcement (*named here*)	Principles for policies, Realization to the points that enforce them	`064`, `065`, `R17` reversed	The Chief Information Security Officer
Regulatory coverage (*named here*)	Drivers, the Requirements standing for control objectives, the controls behind them	`001`, `063`, `041`, `R16` reversed, mirrors `RPT09`	Internal audit and the supervisor

Phase H — Architecture Change Management

View	Elements and relationships	Populated from	For
Change impact (*named here*)	The element a request touches, plus every element one edge away	`061`, then the `R` sheet that names the element	`GB-01 Architecture Board`
Baseline against target (*named here*)	Two Plateaus and the classified differences between them	The comparison tool's gap candidates, then `029`	The Chief Enterprise Architect
Requirements realisation (Requirements Realization viewpoint)	Requirements with the blocks realising them, and the 32nd that realises nothing	`006`, `R09` reversed, mirrors `RPT06`	Internal audit and the Chief Risk Officer
Orphans (*named here*)	Every element with no edge, on one page, with nothing else on it	`RPT13`, plus `025`, `028` and `038`, which `RPT13` misses	The architecture function, monthly

> **FIVE VIEWS, NOT FIFTY**
>
> Start with the Phase A motivation view, the level-1 capability map, the application co-operation view, the authority view and the transition overview. Those five answer most of what gets asked in the first six months, each is a filter over sheets that are already populated, and each fits on a page. Draw one of the other forty-five when somebody asks a question that needs it.

Four views from the repository

Each is built from rows that exist in `Banking_TOGAF_Metamodel.xlsx`, with their `Ref`s. Where an edge had to be authored because no sheet carries it, the figure says so — that is usually the most useful thing the picture has to tell you.

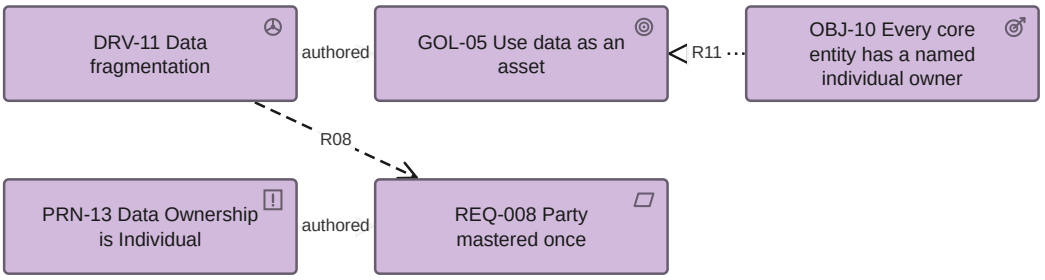

Figure 16.1 Motivation chain for Ardent Union Bank from O01, O02, O03, O05 and O06. Only two of the four edges come from a relationship sheet: R08 and R11, both drawn against the direction the sheet reads.

The chain reads outward in intent and inward in realisation. `DRV-11 Data fragmentation` influences `GOL-05 Use data as an asset`; `OBJ-10 Every core entity has a named individual owner` realises that goal, which is `R11` drawn backwards; `REQ-008 Party mastered once` is influenced by `DRV-11`, which is `R08` drawn backwards; and `PRN-13 Data Ownership is Individual` shapes the

requirement. Two of those four edges are authored. There is no driver-to-goal sheet and no principle-to-requirement sheet in this repository, so a generated motivation view arrives with half its edges missing and an architect fills them in by hand every quarter. Note also what is absent: `MSR-10 Entities with a named owner` stands at a baseline of 26 and no element here can carry the number.

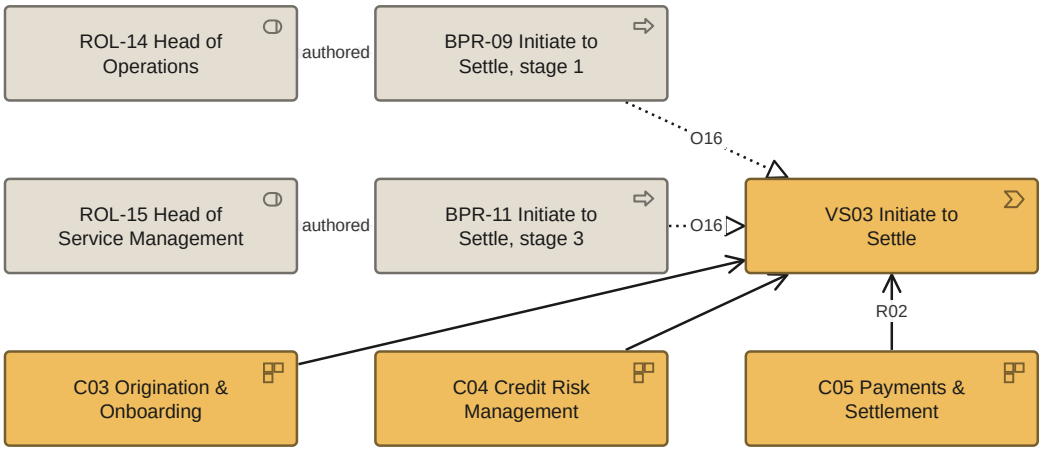

Figure 16.2 VS03 Initiate to Settle with the three capabilities serving it and two of its six process stages. All three Serving edges are R02 reversed; the Assignment edges are authored, because no sheet in this repository joins a role to a process.

`VS03 Initiate to Settle`, three capabilities serving it, two of its six process stages below. The Serving edges are `R02` reversed and they are the only edges here that come from a relationship sheet; the process-to-stream edges come from the `Value stream` column of `O16`. The Assignment edges from `ROL-14 Head of Operations` and `ROL-15 Head of Service Management` are **authored**, because nothing in this repository joins a role to a process. That is the figure's real content: the view a Chief Operating Officer most wants is the one the repository is least able to generate, and the fix is a new relationship sheet, not a better drawing tool.

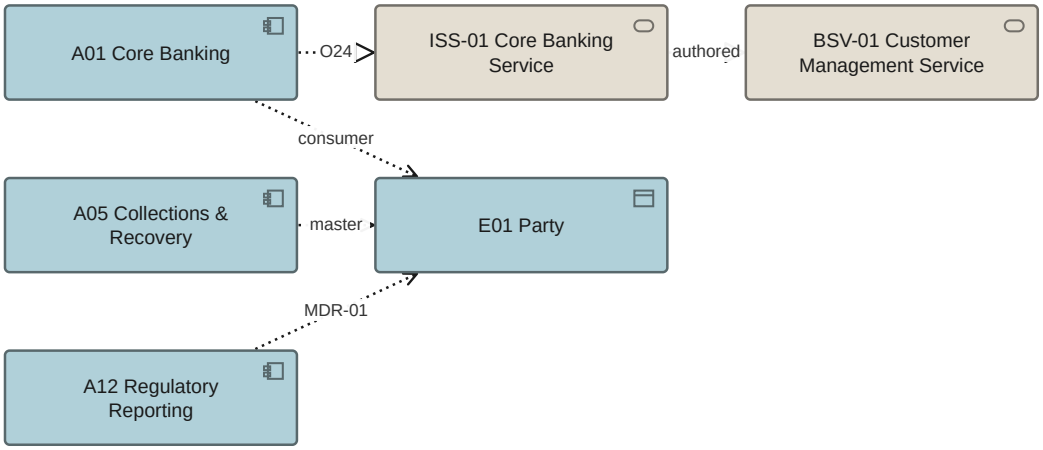

Figure 16.3 A01 Core Banking through two Services that are the same element, and E01 Party mastered twice. The master and consumer edges are R05; O20 names Collections and Recovery as master, while MDR-01 on O43 names Regulatory Reporting as golden source.

`A01 Core Banking` realises `ISS-01 Core Banking Service`, which realises `BSV-01 Customer Management Service`. Both services are **Service** in ArchiMate 4, so the `Ref`s on the labels do the work that colour used to do; neither edge comes from a relationship sheet — the first is the `Application` column of `O24` and the second is `R04` reversed, collapsed through the information system service. Then the point of the figure. `E01 Party` is mastered twice. `O20` names `Collections & Recovery` as its `Master system`; `MDR-01` on `O43` names `Regulatory Reporting` as its `Golden source`; `R05` records `Collections & Recovery` as Master, `A01` as Consumer, and says nothing about `A12`. Three sheets, two answers, and the notation cannot express contested authority — every Access edge looks alike. What the view does is make the count visible, and the count is the gap. Eight of the ten entities on `O43` disagree with `O20` the same way.

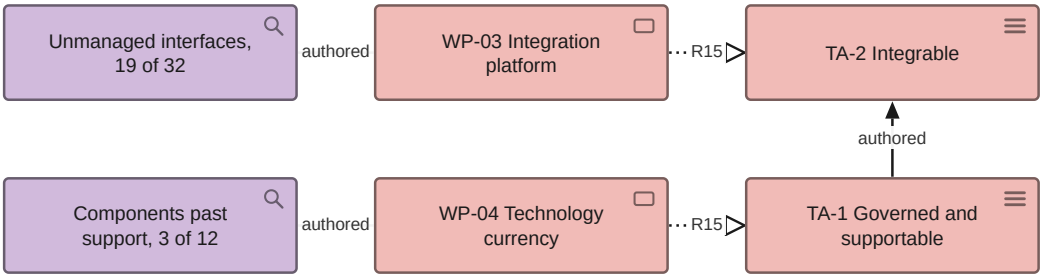

Figure 16.4 Two findings from ZZ Checks carried as Assessments, the two packages that address them, and two of the three plateaus. Every gap edge here is authored, because O29 Gap holds no records and R12 therefore holds none either.

Two Assessments, two Work Packages, two Plateaus. Every gap edge here is authored, because `O29 Gap` holds **zero records** and `R12 Gap closed by Work Package` therefore holds zero instances. The two Assessments are taken from `ZZ Checks`, which counts 19 unmanaged interfaces against a threshold of 0 and three platform components past vendor support against a threshold of 0 — real findings with real numbers and no `Ref`, because no row exists to carry one. The package-to-plateau edges are `R15` reversed. The Triggering between `TA-1 Governed and supportable` and `TA-2 Integrable` is authored: `O53` has no sequence column. The view cannot say that `WP-03 Integration platform` costs $10.1m, that it is not yet approved, or that `ACO-03` records its contract as "Not yet assessable". All three are in the repository and none is drawable. And the first instruction this figure gives you is not about drawing at all: populate `O29`.

Keeping views and the repository in step

Four rules, short because they are either kept or the arrangement collapses inside two quarters.

Regenerate, never hand-maintain. A view is a query result with a layout. When a row changes, re-run the query. Hand-editing a view to match a changed sheet works twice and fails the third time, and nobody notices until a committee reads a stale picture and makes a decision on it. This repository has no export sheets, so "re-run the query" means re-filter and re-paste. Write the filter in the view's notes: sheet, column, value.

The view carries the `Ref`, not just the name. `WP-02 Data ownership and mastering`, not "data ownership". Names are edited in `O30` without anybody's approval; `Ref`s are not. And this repository

is full of same-named rows across sheets: all 18 `O10` actors share names with `O11` roles, all 12 `O17` functions share names with `O13` capabilities, three `O66` increments share names with `O53` transitions, and seven `O47` entries share names with `O26` components. A label without a `Ref` is ambiguous in four different ways at once.

An element the repository does not hold is a change request waiting to be raised. Not an error to fix quietly — a row on `O61 Governance Change Request` with an `Against` and a `Type`, so that the decision to add it is made by somebody entitled to make it. `CR-G03 Raise unfunded building blocks as gaps` is exactly this mechanism working correctly, and it is one of five rows where there should be dozens. The usual cause of an invented element is a workshop that agreed something true and never wrote it down. Worth capturing; not worth capturing by drawing.

And the reverse: a record on no view is a question, not a problem. `RPT13 Orphan Analysis` names two — `A13 Data Platform` and `A14 Fraud Detection` — and misses every row of `O25`, `O28` and `O38`, which no relationship sheet references at all. Some of those rows are unnecessary; some are unrealised, which is the finding. Establish which before deleting anything.

Then the test, which runs in a minute. Take a published view, list its element `Ref`s, look each one up on the sheet its prefix implies. Check that every relationship appears on its `R` sheet with the same two endpoints and that every reversal above has been applied. Run `ZZ Checks` and confirm the counts have not moved since the view was drawn. If that holds, the view is a true statement about the repository. If it does not, you do not have a disagreement. You have two architectures.

THE CHAPTER IN TEN LINES

- The repository is the record and an ArchiMate 4 view is the argument; where they disagree the repository wins and the view is wrong, not a variant.
- Every element on a view carries its `Ref`, because this repository has no change control at all and the `Ref` is the whole of the audit trail.
- Sixty-four sheets are prefixed `O`; with `EXT` that is the 65 object types the index counts. `O34` and `O51` are real absences, not an export error.
- Twenty-two of the 64 `O` sheets have no ArchiMate 4 element of their own; eight substitute honestly and fourteen stay in the repository.
- `O03 Objective` is an Outcome, not a Goal. Collapse the two and the realisation chain the whole motivation view rests on disappears.
- Thirteen of the 25 relationship sheets reverse when drawn: `R02`, `R03`, `R04`, `R07`, `R08`, `R09`, `R11`, `R15`, `R16`, `R17`, `R18`, `R20` and `R21`.
- `R10` and `R23` cannot be drawn at all, and `R12` and `R24` are drawable and empty because `O29` and `O59` hold no records.
- Three edges every architect wants are on no sheet: driver to goal, role to process, and application to information system service.
- Fifty views cover the phases; five of them answer most of what gets asked in the first six months.
- Run `ZZ Checks` before and after every view set, and raise an invented element on `O61` rather than correcting it quietly.

SOURCES FOR THIS CHAPTER

- **1.** Banking_TOGAF_Metamodel.xlsx — 105 sheets: 00 Index, 64 object sheets O01–O66 (O34 and O51 absent), EXT Industry Extension, 25 relationship sheets R01–R25, 13 reports RPT01–RPT13 and ZZ Checks
- **2.** Banking_TOGAF_Metamodel.xlsx, sheets O20 Data Entity, O43 Master Data Rule and R05 Application manages Data Entity — the contested mastering of E01 Party
- **3.** The Open Group, TOGAF Standard, 10th Edition — Architecture Development Method https://pubs.opengroup.org/architecture/togaf-standard/
- **4.** The Open Group, ArchiMate Forum — language overview https://www.opengroup.org/archimate-forum/archimate-overview

APPENDIX E

Where to look in volume 1

The volume read backwards: start from the step you are stuck on and find the chapter and section where volume 1 already worked the judgement through.

This appendix runs the other way round from the rest of the volume. Everywhere else you start from a phase and work forwards. Here you start from the step you are stuck on and work back to the place in volume 1 where the judgement behind it has already been argued through.

Volume 1 is the why; this volume is the what, the where and the who signs. Work the steps without the teaching and you will fill cells correctly and defend them badly — which is worse, because a wrong cell gets corrected and a badly defended right one gets overruled.

In the step index the third column gives the volume 1 **chapter number** and then, in italics, the **section heading inside that chapter**. A dash means volume 1 has nothing to say about that step. That is honest and common: **sixty-two of the 157 steps carry a dash**, two in every five. The repository, the comparison tool and the gate regime are this volume's own subject.

Where volume 1 carries pre-drafted content — chapter 19, appendix A — its identifiers are its own. `P-01` – `P-12`, `C-01` – `C-20`, `R-01` – `R-33`, `D-01` – `D-20` and `A-01` – `A-25` are teaching sets. The repository runs `C01` – `C12` decomposed to `C12.03.02`, 20 `BPR-` rows, 18 `ROL-`, 14 `E` entities and 14 `A` applications. The two schemes look similar enough to tempt a join and they do not line up. **Match on name. Never on number.** `A08` in volume 1's conceptual model and `A08` in `O23 Application Component` are different things, and a step that gets this wrong produces a traceability table that is confidently false.

Ardent Union Bank, the bank inside the repository, and Table Bay Bank, volume 1's, are both inventions and their data is not interchangeable.

The step index

One hundred and fifty-seven steps, eleven chapters, in the order you would meet them.

Chapter 2 · Standing the practice up (Preliminary)

Step	What it asks you to decide	Volume 1 chapter and section
2.1	Which licensed entities and units are inside the boundary	15 · *The Banks Act, and why a licence is a data attribute*
2.2	Whether the record can be changed safely at all	—
2.3	What the baseline is, and the date it was cut	—
2.4	Which integrity defects to fix before anything else	18 · *Six numbers that tell you whether the estate is healthy*
2.5	Which of the 64 object sheets you will actually fill	19 · *What these are, and what they are not*
2.6	What banking needs that the core metamodel does not hold	16 · *The mapping*
2.7	Whether to add a seventh extension type, and how	13 · *The three ways architects misuse one*
2.8	What an `Owner` cell is allowed to contain	10 · *Ownership and lineage*
2.9	Which of the 24 principles survive	15 · *The Joint Standards, with the numbering got right*
2.10	Who may decide what, and who may not	5 · *The architect's real counterparties*
2.11	What each gate tests, and who holds it	—
2.12	Where the architecture function sits in its own model	18 · *The three artefacts that buy you credibility*
2.13	How good the practice is today	7 · *What the operating model means for architecture*
2.14	Whether the motivation layer joins up end to end	1 · *The seven things an architect must understand*
2.15	Whether the Preliminary Phase is closed	18 · *What week thirteen should look like*

Chapter 3 · Architecture Vision (Phase A)

Step	What it asks you to decide	Volume 1 chapter and section
3.1	Who is asking, and what authority they are granting	18 · *Week one: eight documents, and where they live*
3.2	Whether an actor and a role are the same list	5 · *The org chart will not help you*
3.3	Which stakeholders a bank's map always misses	5 · *Treasury and markets*
3.4	Whose agreement the vision actually needs	5 · *Four questions*
3.5	How the bank makes money, on one page	4 · *The canvas for a full-service bank*
3.6	Where value begins and where it lands	16 · *Capabilities and value streams*
3.7	Breadth, depth, horizon and domain scope	2 · *The twelve lines*
3.8	What you cannot change, and until when	15 · *What the regulator actually asks to see*
3.9	What you are guessing, and when you will know	14 · *The corrections — get these right, because the rest of the market gets them wrong*
3.10	The one-page argument for changing	17 · *The motivation view: why this bank is changing*
3.11	What the architecture function will and will not do	7 · *What the operating model means for architecture*
3.12	Whether Phase A earned its mandate	13 · *What a reference architecture is actually for*

Chapter 4 · Business Architecture (Phase B)

Step	What it asks you to decide	Volume 1 chapter and section
4.1	Which streams and which capability levels Phase B covers	—
4.2	The organisation as fact, not as architecture	5 · *The org chart will not help you*
4.3	Which rows are positions and which are people	19 · *Roles*
4.4	Which role may approve what, to what value	5 · *The credit mandate ladder*
4.5	Where value begins, and which capabilities it uses	7 · *The value chain, end to end*
4.6	Whether a row is a capability at all	19 · *Business capabilities*
4.7	That the levels decompose cleanly	2 · *The twelve lines*
4.8	One named accountable owner per capability	—
4.9	How mature each capability really is	—
4.10	Target, criticality and disposition, as three decisions	—
4.11	Which capabilities make the Phase E shortlist	—
4.12	What is a function and what is a process	16 · *Is it a service or a process?*
4.13	Which level the process inventory sits at	8 · *The L1 process map*
4.14	Whether a service is channel independent	7 · *Channels, and what each one can actually do*
4.15	Which products exist, and what constrains them	16 · *Product, and the catalogue underneath it*
4.16	Whether Phase B can be handed on	—

Chapter 5 · Data Architecture (Phase C)

Step	What it asks you to decide	Volume 1 chapter and section
5.1	How many entities are in scope, at which layer	9 · *How many*
5.2	Which entities are missing from the catalogue	19 · *Conceptual data entities*
5.3	Whether Party and Customer are one entity or two	9 · *Party, and why Customer is usually a bad entity*
5.4	Who owns each entity, and which system is authoritative	10 · *Ownership and lineage*
5.5	Which of three sheets to believe about the system of record	19 · *Application to data, with the system of record*
5.6	Cardinality that carries the business rule	9 · *Agreement, Product and the thing called Account*
5.7	Which logical store holds each entity, and which physical one	12 · *15. The data platform*
5.8	Which attributes are worth recording	10 · *Attributes, where they earn their place*
5.9	Classification and retention, entity by entity	15 · *POPIA, with the banking bite*
5.10	The lawful basis for each data set	—
5.11	What a golden record is, per entity	10 · *Reference and master data*
5.12	What CoDI's single customer view demands of the data	15 · *CoDI, and a regulation that is really a data specification*
5.13	What metadata the bank holds, and what it does not	—
5.14	Which application masters which entity, with a usable key	19 · *Application to data, with the system of record*
5.15	Which data has to move, and under what rule	—
5.16	Whether the data layer can be handed to the application phase	—

Chapter 6 · Application Architecture (Phase C)

Step	What it asks you to decide	Volume 1 chapter and section
6.1	Which conceptual boxes the estate actually fills	11 · *The table*
6.2	Every application anyone in the bank can name	—
6.3	Owner and criticality, defensible in one sentence	12 · *The shape of the estate*
6.4	Which five columns the sheet does not have	—
6.5	The applications nobody declared	—
6.6	What each application is worth, on both axes	11 · *As the basis for the application portfolio*
6.7	Invest, retain, consolidate, replace or retire	—
6.8	Which automated services exist, and which are published	—
6.9	Which application supports which business service	—
6.10	Where coverage is thin and support is duplicated	19 · *Capability to application — the coverage map*
6.11	Which application is the system of record	19 · *Application to data, with the system of record*
6.12	How many interfaces are unmanaged, and how many unlogged	12 · *How these systems actually talk*
6.13	Which requirement each application answers	—
6.14	Conformance, application by application	—
6.15	Whether the estate matches the reference model, and where BIAN fits	13 · *BIAN — the Banking Industry Architecture Network*
6.16	What the portfolio catalogue says publicly	—

Chapter 7 · Technology Architecture (Phase D)

Step	What it asks you to decide	Volume 1 chapter and section
7.1	Which technical capabilities the estate offers	14 · *The research is dated, and here is what goes stale first*
7.2	What each application actually runs on	—
7.3	The vendor-independent components, against the reference model	—
7.4	Support dates taken from the contract, not the slide	14 · *The cost lines that are not the licence*
7.5	Which specifications are governed, and where conformance lives	13 · *ISO 20022 — a reference data model wearing a message format's clothes*
7.6	Conformance, component by component	—
7.7	What goes out of support, and when	14 · *What a core replacement costs, and how long it takes*
7.8	Which platform services are standard, and who consumes them	—
7.9	Where the estate physically sits	—
7.10	What each environment holds, including personal data	—
7.11	What each network link carries	—
7.12	Recovery objectives, and whether they have been tested	15 · *Resolution, and the test you take over a weekend*
7.13	Which controls exist, and which are absent	15 · *POPIA, with the banking bite*
7.14	Whether Phase D can be signed off	—

Chapter 8 · Opportunities and Solutions (Phase E)

Step	What it asks you to decide	Volume 1 chapter and section
8.1	What the comparison tool needs, produced by hand	—
8.2	Which question the comparison is answering	—
8.3	Which records appear, vanish or get renumbered	—
8.4	What the tool thinks each difference is	15 · *What the regulator actually asks to see*
8.5	Whether each difference was governed	—
8.6	How severe each gap candidate is	—
8.7	Which capability gaps the tool cannot see	19 · *Capability to application — the coverage map*
8.8	The five capabilities a bank is usually missing entirely	11 · *The patterns*
8.9	Which integrity findings become gaps with numbers attached	1 · *5. Payments is shared infrastructure, and in South Africa it is its own country*
8.10	The gap numbering scheme, before the first row	—
8.11	What must exist, before who supplies it	14 · *Build, buy or configure*
8.12	Which package delivers which block, exactly once	—
8.13	What gets funded together	12 · *The legacy question*
8.14	Which states you can stop at safely	16 · *The roadmap, and what is true at the end of each year*
8.15	Which three options the board chooses between	14 · *What a core replacement costs, and how long it takes*
8.16	What the roadmap says, and what it shows	6 · *The four numbers an executive watches*

Chapter 9 · Migration Planning (Phase F)

Step	What it asks you to decide	Volume 1 chapter and section
9.1	What becomes a chartered project, and who leads it	5 · *The architect's real counterparties*
9.2	What will shape delivery whatever you plan	7 · *What the operating model means for architecture*
9.3	The sequence, and the dependency the sheet cannot hold	—
9.4	Where each cost number came from, in one currency	14 · *The cost lines that are not the licence*
9.5	Whether a benefit traces to a measure	6 · *Cost-to-income and jaws*
9.6	What is true at the end of each increment	16 · *The roadmap, and what is true at the end of each year*
9.7	The consequence of each delivery risk	14 · *The two fully documented failures, and what they actually teach*
9.8	Which project a constraint or an assumption holds up	15 · *The Joint Standards, with the numbering got right*
9.9	Which weeks the business will actually give you	7 · *The reporting and regulatory calendar*
9.10	How each entity moves, and how history survives the move	10 · *Time: three dates, not two*
9.11	Field to field, with the rule written out	10 · *Classification, consent and POPIA*
9.12	Whether `Delivery` and `End` can both be true	—
9.13	Whether the plan is funded	15 · *The BA returns, and the change that moved them out of the law*

Chapter 10 · Implementation Governance (Phase G)

Step	What it asks you to decide	Volume 1 chapter and section
10.1	Whether the repository can be governed at all	—
10.2	Who holds each gate, and on what evidence	—
10.3	What the delivery organisation signed up to	—
10.4	Whether the criteria were inherited or invented	16 · *Capabilities and value streams*
10.5	Whether the design will meet the criteria	12 · *How these systems actually talk*
10.6	A verdict per criterion, never per project	—
10.7	Whether a contract may claim compliance	—
10.8	Which obligations have no mechanism behind them	15 · *What the regulator actually asks to see*
10.9	What each body may not decide	—
10.10	What is excused, by whom, until when	—
10.11	Whether the benefit arrived	—
10.12	Which columns evidence the board's own duty	15 · *King IV, and the board's version of the same question*
10.13	Which Joint Standard is which	15 · *The Joint Standards, with the numbering got right*
10.14	What the gate calendar holds this quarter	—

Chapter 11 · Architecture Change Management (Phase H)

Step	What it asks you to decide	Volume 1 chapter and section
11.1	Whether a change may be made at all	—
11.2	The before and after of every field	—
11.3	Which body decides, and on what papers	—
11.4	Simplification, incremental or re-architecting	—
11.5	What the two extracts must contain	—
11.6	What changed with no change request behind it	—
11.7	When the sweeps run, and off what trigger	—
11.8	When a baseline is worth cutting	—
11.9	The three health numbers and their targets	18 · *Six numbers that tell you whether the estate is healthy*
11.10	Which maps a process or system change invalidates	19 · *Keeping these alive*
11.11	Which banking events force a change cycle	15 · *The payment system, and who writes the rules now*
11.12	What to do about a support expiry inside a year	14 · *The cost lines that are not the licence*
11.13	What the board is asked for, once a year	15 · *King IV, and the board's version of the same question*

Step	What it asks you to decide	Volume 1 chapter and section
12.1	What the inherited register actually contains	—
12.2	Whether it can be tested at a gate	16 · *The honest limits*
12.3	Which objective the requirement serves	16 · *A regulation, from driver to control*
12.4	Which requirements are preferences	—
12.5	Which principle governs it, and how	13 · *BCBS 239 — the principles that force the data architecture*
12.6	Which principles govern nothing	—
12.7	What the obligation actually requires	15 · *What the regulator actually asks to see*
12.8	Which South African instrument, not which category	15 · *CoDI, and a regulation that is really a data specification*
12.9	Who is funded to deliver it	—
12.10	Traced against ever tested	—
12.11	Whether to accept it into this phase	7 · *The reporting and regulatory calendar*
12.12	Which rows are no longer live	—

> **WHERE THE DASHES CLUSTER**
>
> They are not spread evenly. Chapter 3 has none at all and chapter 9 has two, because Phase A and Phase F are arguments about the business and volume 1 is an argument about the business. Chapters 10 and 11 run at nine of fourteen and eight of thirteen, chapter 6 at nine of sixteen, chapter 7 at eight of fourteen. Those four chapters are where this volume works alone: the gate regime, the change-control spine the repository does not have, the portfolio arithmetic, and the repair of columns that contradict each other. If you are stuck in one of them, stop looking in volume 1 and read the step again.

The reverse map

The same thing from the other end: each of volume 1's twenty-two chapters, where this volume draws on it, and the one thing to carry away if you read nothing else in it. Chapter numbers in the middle column are **workbook** chapters.

Volume 1 chapter	Workbook chapters that use it	The one thing to take from it
1 · *The industry in one chapter*	2, 8	The seven things that decide every later judgement. Read it again the week before Phase A, not only on day one.
2 · *The lines of business*	3, 4	Twelve lines, and three things you will be told are lines and are not. Scope that does not say which lines is not scope.
3 · *The vocabulary*	4, 5	The five words that mean different things in the same bank. Half of all catalogue arguments are vocabulary arguments.
4 · *How the business model works*	3	The canvas and the four journeys, which is where value streams come from and where four of them have no home in the repository.
5 · *Who does what*	2, 3, 4, 9	Who may decide what, and the credit mandate ladder. The org chart will not tell you and the repository has nowhere to put it.
6 · *How banks make money*	8, 9	The four numbers an executive watches. A benefit that does not land on one of them is not a benefit.
7 · *The operating model*	3, 4, 9, 12	The reporting and regulatory calendar. It decides which weeks your plan is allowed to have.
8 · *The process model*	4	L1 and L2 are different artefacts. Loading L3 into `016` is the commonest Phase B error.
9 · *The conceptual data model*	5	Party, agreement, account — and the several things the word "Account" means before anybody argues.
10 · *The information model*	2, 5, 9	Three dates, not two. No migration plan written without bitemporality has ever held.
11 · *The conceptual application model*	6, 8	The twenty-five boxes that must exist before any vendor is named, and the five that are always shared and therefore always unfunded.
12 · *The application landscape*	5, 6, 8, 10	How these systems actually talk, and the batch clock behind it. The estate you are cataloguing.
13 · *The reference architecture*	2, 3, 6, 7, 12	What BIAN costs and what it does not give you free, and the three ways architects misuse a reference architecture.
14 · *Technology providers*	3, 7, 8, 9, 11	What rots first, what a core replacement really costs, and the fifteen corrections the market gets wrong.
15 · *Regulation, risk and what the regulator inspects*	2, 3, 5, 7, 9, 10, 11, 12	The obligations in substance. This workbook records them as seven rows on `063`; it does not teach them.
16 · *Modelling banking in ArchiMate*	2, 3, 4, 8, 10, 12, appendix D	Five decisions that settle most of a model, and four things the notation cannot do. Appendix D maps the sheets; this teaches the language.
17 · *Table Bay Bank, end to end*	3, 8	What a complete, consistent model looks like — including the two first drafts that were wrong and why.
18 · *Your first ninety days*	2, 3, 11	Six numbers that tell you whether the estate is healthy, and what week thirteen should look like.

19 · Appendix A, *The cross-maps*	2, 4, 5, 6, 8, 11	Pre-drafted catalogues and the argument for keeping them alive. Match on name, never on number.
20 · Appendix B, *Glossary*	all	The arbiter when two people in a workshop mean different things by the same word. 325 of them.
21 · Appendix C, *Standards and bodies*	7, 12	Where the standards in `032` come from, and which of them will date fastest.
22 · Appendix D, *References*	all	The sources. Cite from here rather than from memory when a number is challenged in a room.

Two patterns in that middle column. Chapter 15 is used in eight workbook chapters and chapter 14 in five, which is the right shape: regulation and the vendor market are the two things a banking architect is never done with. And chapters 20 and 22 read *all*, because a glossary and a reference list are consulted under pressure rather than read.

One absence worth naming. Volume 1 chapter 8, *The process model*, is used by one workbook chapter. That is not because process matters less in a bank — it is because `016 Business Process` holds 20 rows of generated stage names and no sheet in this repository joins a process to a role, a system or a data entity. Volume 1 has thirteen BPMN level-2 models and the repository has nowhere to put them. If one chapter of volume 1 is underused by this volume, that is the chapter, and the reason is a gap in the record rather than a gap in the teaching.

What each volume deliberately leaves out

Neither volume is a shortened version of the other, and knowing the boundary saves you looking in the wrong place.

Volume 1 covers, and this volume does not:

- **The industry itself.** What a bank is, why it is not a shop with money, how the two revenue engines are computed differently. This volume assumes you know.
- **The economics.** Margin, the endowment effect, funds transfer pricing, jaws, return on equity, IFRS 9 impairment. `004 Measure` holds 16 baselines and no opinion about where a credible number comes from. Chapter 6 has the opinion.
- **The vendor landscape.** Who sells what, which platforms have landed in South Africa, what a core replacement costs. `027 Physical Technology Component` names twelve product classes and says nothing about the market around them.
- **The regulation in substance.** The Banks Act, the BA returns, Basel, IFRS 9, CoDI, FICA, the NCA, the Conduct Standard, POPIA, the Joint Standards. `063 Control Objective` holds seven obligations as rows; what each obliges you to do is chapter 15, and no row can carry it.
- **A complete worked model.** Chapter 17 draws Table Bay Bank end to end, including the two drafts that were wrong. This volume draws fragments, because its subject is the record.
- **The notation.** Chapter 16 teaches ArchiMate 4 for banking. Appendix D maps this repository's sheets onto it and assumes the teaching.

This volume covers, and volume 1 does not:

- **The repository.** One hundred and five sheets: which columns carry judgement, which are derived, which contradict each other, and the naming rules that keep every lookup resolving. Appendix A is the whole of it.
- **The comparison tool.** Five paste targets, the `Classification` column that separates a design delta from governed drift from ungoverned change, and the gap candidates that fall out. Chapter 8 teaches it, chapter 11 sets the cadence, volume 1 does not mention it.
- **The ADM as a sequence of acts.** Not the phases in the abstract but 157 steps, each with a sheet, a column set, a signatory, a testable completion condition and the report it shows up in. This volume is where somebody can check that you did the work.
- **Refusal.** Four gates, dispensations that expire on a date, architecture contracts, and what each body may *not* decide. Chapter 10 exists because a practice that cannot refuse has not implemented the method.
- **What the record is missing.** No change control, no gap records, no compliance assessments, no role-to-process relationship, no payment scheme object. Volume 1 could not have known; this volume has to say so and say what to add.
- **The ninety-day calendar against the record.** Volume 1 chapter 18 says what to read and who to ask. Appendix C here says which sheets you touch in week three.

One closing instruction, and it is the only thing in this appendix worth memorising. Where a step and volume 1 disagree about the business, volume 1 is probably right and your sheet is stale. Where they disagree about the repository — a count, a column name, a `Ref` prefix — the repository is right and both books are describing it. Open the sheet before you argue with either.

THE CHAPTER IN TEN LINES

- Start from the step, not the phase: this index runs step to teaching, not teaching to step.
- The third column gives a volume 1 chapter number and then the section heading inside it, and every heading cited has been checked against the chapter.
- A dash is honest. Sixty-two of the 157 steps have no volume 1 counterpart, two in every five.
- The dashes cluster in chapters 6, 7, 10 and 11 — the portfolio arithmetic, the column repairs, the gates and the change-control spine.
- Volume 1 appendix A is a teaching set. Its `C-01`…`C-20` and `A-01`…`A-25` are not the repository's `C01`…`C12` and `A01`…`A14`. Match on name, never on number.
- Chapter 15 is used in eight workbook chapters and chapter 14 in five. Regulation and the vendor market never close.
- Chapter 8, the process model, is used by one chapter, because `O16` has 20 generated rows and no sheet joins a process to anything.
- Volume 1 owns the industry, the economics, the vendors, the regulation and the notation in substance.
- This volume owns the repository, the comparison tool, the gates, the 157 steps and the list of what the record is missing.
- Disagreement about the business: believe volume 1. Disagreement about the repository: open the sheet.

APPENDIX F

The forms

Thirty printable worksheets, eleven sets of the questions you must answer before you can fill one honestly, and the discipline that gets the writing off the paper and into the sheet.

This appendix is paper. Everything before it tells you which sheet and which column. This one hands you the sheet of paper you carry into the room and bring back with writing on it.

Ardent Union Bank is an invented bank. Its data is real inside this repository and nowhere else. Every example row below is marked as an example and is there to be crossed out.

How to use these forms

Print the forms for the phase you are in, and only those. There are thirty of them and nobody needs all thirty in a week. The index overleaf gives the form number, its title, the sheet it transcribes into and the ADM phase it belongs to. Find your phase, print those pages, and put them in the folder with the agenda.

Fill them by hand, in the room, while the people who know the answer are still in front of you. Transcribe into the repository the same day — not the same week. Keep the paper until the row has been approved by whoever the form says signs it. Then retire the paper the way the last section of this appendix describes.

Why paper beats a laptop in a workshop

A laptop does three things to a room, all of them bad. It puts a screen between you and the person you are asking, so you stop watching their face and miss the hesitation that is the real finding. It turns an argument into a typing queue: the room moves at the speed of the typist, and the typist starts steering towards the cells that are easy to fill. And it tempts you to type into the repository live, which puts a half-agreed judgement into a sheet and in front of a report before anybody has signed it.

Paper is also faster at the thing workshops are for. Two people fill two forms at once. A form can be slid across a table to the executive who owns the capability, and they will initial a row they would never have agreed to by email. A crossing-out stays visible, which matters: on Form B-2 the score somebody changed under pressure is more interesting than the score they settled on. And when the room disagrees you write both numbers in the cell, which no spreadsheet lets you do.

The cost of paper is transcription, and transcription is where the discipline has to live. That is the last section of this appendix and it is the part to read twice.

A form that does not transcribe into a real sheet is useless

Every column on every form below is a real column of a real sheet of `Banking_TOGAF_Metamodel.xlsx`, spelled the way the sheet spells it, in the order the sheet holds it

[1]. That is not tidiness. It is the property that makes a form worth filling: the person transcribing reads left to right across the paper and left to right across the sheet, and nothing has to be interpreted on the way.

Which means the forms carry some things that look like faults and are not. The first column is `Ref`, never `ID`. `009 Organization Unit` spells its tab with an American *z* and its second column with a British *s* — `Organisation unit` — and the form prints both the way the file does. Several relationship tabs are cut off at Excel's thirty-one-character limit: `026 Logical Technology Componen`, `R05 Application manages Data En`, `R06 Application interfaces with`, `R09 Requirement satisfied by Bu`, `R18 Standard applies to Applica`. Those truncations are the sheet names. Type them as they are, because a lookup written against the full name will not resolve.

Where a sheet has more than six columns, the form splits. Six is what fits the 170mm text column and still leaves a cell wide enough to write a date in. So `013 Business Capability` becomes Form B-1, identify, and Form B-2, assess; `020 Data Entity` becomes C-1 and C-2; `023 Application Component` becomes CA-1 and CA-2. Each form says which columns it covers and which are typed straight into Excel. The ones left to Excel are, with few exceptions, sentences — `Description`, `Definition`, `Statement`, `Mandate` — because nobody should be writing a paragraph into a printed cell.

Two forms have no sheet behind them at all, and say so on their face. Form A-1, the stakeholder register, feeds nothing: the repository holds sixty-five object sheets and not one of them is a stakeholder sheet, so the register lives in the Statement of Architecture Work and each line keys into `010 Actor` or `011 Role`. Form H-2 feeds four sheets that do not exist yet, because this repository ships **no version control, no change log, no change-request register and no baseline sheet**, and **no `Change Ref`, `Changed By` or `Approved By` column on any object sheet**. You create those before you load a record. The form prescribes the columns.

The eleven question sets

Between the forms are eleven `ask` sets. Each sits before the form it prepares and holds three to five questions with ruled space for the answers. They are not warm-ups. They are the questions that decide whether the form can be filled honestly — who can rule a legal entity out of scope, which capability this bank would pay to be better at than its competitors, which balance the reconciliations clerk actually trusts. Answer them on paper before the session. If you cannot answer them, the session will produce rows that look like data and are not.

The index of forms

Form	Title	Feeds	Phase
P-1	Scope, organisation units and sites	`O09 Organization Unit`, `O12 Location`	Preliminary
P-2	Principles — adopt, edit, retire	`O05 Principle`	Preliminary
P-3	Governance bodies and gates	`O57 Governance Body`, `O58 Governance Gate`	Preliminary
P-4	Drivers, goals and objectives	`O01 Driver`, `O02 Goal`, `O03 Objective`	Preliminary
P-5	Measures and the deliverable plan	`O04 Measure`, `O31 Deliverable`	Preliminary
A-1	Stakeholder register, actors and roles	no sheet — keys into `O10 Actor`, `O11 Role`	A
A-2	Value streams	`O14 Value Stream`	A
A-3	Constraints and assumptions	`O07 Constraint`, `O08 Assumption`	A
B-1	Capability identify	`O13 Business Capability` — structure columns	B
B-2	Capability assess	`O13 Business Capability` — judgement columns	B
B-3	Functions, processes and business services	`O17`, `O16`, `O15`	B
B-4	Products, contracts and what supports them	`O18 Product`, `O19 Contract`, `R25`	B
C-1	Data entity identify	`O20 Data Entity` — identity columns	C data
C-2	Data entity assess	`O20 Data Entity` — classification and lineage	C data
C-3	Entity relationships, data sets and master data rules	`O36`, `O38`, `O43`	C data
CA-1	Application identify	`O23 Application Component` — identity columns	C application
CA-2	Application assess	`O23 Application Component` — judgement columns	C application
CA-3	Application to data entity	`R05 Application manages Data En`	C application
CA-4	Interface register	`R06 Application interfaces with`	C application
D-1	Technology services and components	`O25`, `O26 Logical Technology Componen`, `O27 Physical Technology Compone`	D
D-2	Standards and conformance	`O32 Standard`, `R18 Standard applies to Applica`	D
D-3	Resilience tiers	`O49 Resilience Tier`	D

E-1	Gap register	`O29 Gap`	E
E-2	Building blocks and acceptance criteria	`O55 Building Block`, and `O35`, `O44`, `O45`, `O46`	E
E-3	Work packages and transition architectures	`O30 Work Package`, `O53 Transition Architecture`	E
F-1	Projects and implementation factors	`O56 Project`, `O52 Implementation Factor`	F
G-1	Architecture contract and compliance assessment	`O33 Architecture Contract`, `O59 Compliance Assessment`	G
H-1	Governance decision and change request	`O60 Governance Decision`, `O61 Governance Change Request`	H
H-2	The change log and change-request sheets you must create	four sheets that do not exist — you build them	H
R-1	Requirements, obligations and the acceptance criterion	`O06`, `O63`, `R09 Requirement satisfied by Bu`	Requirements

Four things you might expect on that list are not there. `O37 Data Attribute` has no form: sixty rows of entity, attribute, type and key, which is a data-modelling tool's work and not a workshop's. `O40 Source To Target Mapping` has no form for the same reason, and because a mapping rule written by hand is a mapping rule nobody can test. `EXT Industry Extension` has no form, because its six rows are the metamodel's own shape rather than content — you change them by argument, not by filling in a sheet. And none of the thirteen `RPT` sheets has a form, because every row in them is read from an object sheet: a form that fed a report would be a form that falsified it.

Preliminary Phase

Five forms, filled before you have the authority to fill anything else. They are about scope, power and dates rather than about architecture, which is why a sponsor reads them more carefully than anything in Phase B.

BEFORE YOU DRAW THE SCOPE BOUNDARY

Which legal entities does this architecture bind, and which does it merely describe? State the difference in one sentence.

Who, by name, can rule an entity out of scope — and would they say so in front of the group executive?

Name one organisation unit nobody will claim. What happens to its capabilities when you reach Form B-1?

`012 Location` holds eight sites and a `Scale` column that counts people. Which site would stop the clearing day if it stopped, and what is the evidence rather than the assumption?

Form P-1 — Scope, organisation units and sites

Feeds `009 Organization Unit` — `Ref`, `Organisation unit`, `Type`, `Head`, `Headcount` — and `012 Location` — `Ref`, `Location`, `Type`, `Scale`. **Nothing is typed into Excel**: these are the whole of both sheets. **First pass** twelve units and eight sites at Ardent Union; a mid-tier South African bank with a branch estate will need two copies of the second table. **Signed by** the Chief Executive, or `GB-04 Executive Committee` sitting as a body. Nobody below that may rule a legal entity out of scope. **Note** `009` has no `Parent` column, so a group structure cannot be expressed here — record the parent in `Type` as 'Group' or 'Subsidiary' and keep the legal-entity tree in the Statement of Architecture Work.

Ref	Organisation unit	Type	Head	Headcount
ORG-01	*example — delete*	Group	Chief Executive	97

`012 Location`. `Scale` is a free-text size in the shipped data ('61 people'). Put the headcount and, where it differs, what the site carries: a data centre with four people in it is not a small site.

Ref	Location	Type	Scale
LOC-01	*example — delete*	Corporate	61 people

BEFORE YOU TOUCH THE PRINCIPLE SET

Which of the twenty-four shipped principles has been quoted in a real decision at this bank in the last year? Give the decision.

Which one has never been quoted? Say why you would keep it rather than retire it.

Name two principles that will conflict on a live question — a payments cut-off against a change freeze will do — and say what breaks the tie.

`005 Principle` has no `Tier` column and `Status` offers only Approved and Draft. What is your tie-break until you add one? Write the sentence you will say to the Board.

Form P-2 — Principles — adopt, edit, retire

Feeds `005 Principle` — `Ref`, `Principle`, `Category`, `Owner`, `Status`. **Typed into Excel** `Statement`, which is a sentence; use the second table for any principle the room re-words and write it out in full. **The decision lives in** `Status`, and the sheet ships only two values, Approved and Draft. Retired is a third value you add; there is no separate decision column and you should not create one. **`Category` ships five values** — Business, Application, Data, Technology, Security. A Governance principle needs a sixth, which appendix G adds. **First pass** twenty-four shipped principles, three copies. **Signed by** `GB-01 Architecture Board`. A principle nobody has signed is a preference.

Ref	Principle	Category	Owner	Status
PRN-01	*example — delete*	Business	Architecture Board	Approved

Re-worded statements. Write the statement, not an instruction to the typist. If it needs two sentences it is probably two principles.

Ref	Statement
PRN-04	*example — delete*

BEFORE YOU WRITE THE GOVERNANCE BODIES DOWN

Which of the five shipped bodies has met in the last quarter, with a minute and a quorum?

For each that has not met: what decision is waiting on it, and who is taking that decision instead?

`GB-05 Architecture Function` has `Cadence` = Continuous. Name one thing it has decided that `GB-01 Architecture Board` should have decided.

At which of `GT-0` to `GT-3` would a project that ignored the architecture actually be stopped, and has that ever happened here?

Form P-3 — Governance bodies and gates

Feeds `057 Governance Body` — `Ref`, `Governance body`, `Cadence`, `Mandate` — and `058 Governance Gate` — `Ref`, `Gate`, `When`, `What it tests`. **Typed into Excel** `057 Membership`, a list of posts that will not fit a printed cell. **First pass** five bodies and four gates, which is the whole of both sheets. **Signed by** `GB-04 Executive Committee`. A governance body cannot constitute itself. **Note** neither sheet has a column for whether the body has actually met, so a body that exists on paper and nowhere else reads as healthy. Record the last quorate sitting in `Cadence` — 'Monthly, last sat 2027-02' — until you add a column for it.

Ref	Governance body	Cadence	Mandate
GB-01	*example — delete*	Monthly	Holds G1 and G2; approves dispensations with an expiry

`058 Governance Gate`. Ardent Union runs four: `GT-0` charter, `GT-1` design, `GT-2` build complete, `GT-3` benefit. Add a fifth only if you can say what it tests that no other gate tests. In a bank the candidate is a scheme-readiness gate before a rail change goes to the clearing house.

Ref	Gate	When	What it tests
GT-0	*example — delete*	Before work starts	Scope, acceptance criteria and contract exist

BEFORE YOU NAME A DRIVER

For each regulatory driver, name the instrument and the section — 'POPIA s72', 'Joint Standard 2 of 2024' — not the regulator. Which three can you not name yet?

`001 Driver` has an `Origin` column that ships External or Internal. Which of your drivers is genuinely internal, and who would defend it as a driver rather than a preference?

The sheet ships thirteen drivers and six goals. Which driver has no goal behind it, and is that the finding or the omission?

Form P-4 — Drivers, goals and objectives

Feeds three sheets. `001 Driver` — `Ref`, `Driver`, `Type`, `Origin`, `Owner`, `Pressure`. `002 Goal` — `Ref`, `Goal`, `Owner`, `Status`. `003 Objective` — `Ref`, `Objective`, `Goal`, `Horizon`, `Status`. **Typed into Excel** `Description` on `001` and `002`. **First pass** thirteen drivers, six goals, fourteen objectives — three copies. **Signed by** `GB-04 Executive Committee` for goals, the named executive for each driver and objective. **`Type` on `001`** ships Regulatory, and the regulatory rows are where the bank's money goes; name the instrument in `Driver`, not the authority.

Ref	Driver	Type	Origin	Owner	Pressure
DRV-01	*example — delete*	Regulatory	External	Chief Compliance Officer	High

`002 Goal`. Drivers outnumber goals, always. `Status` ships Active.

Ref	Goal	Owner	Status
GOL-01	*example — delete*	Chief Executive	Active

`003 Objective`. `Horizon` is a duration in the shipped data ('2 years', '18 months'), not a date. Keep it a duration or convert the whole column; a sheet with both is a sheet nobody can sort.

Ref	Objective	Goal	Horizon	Status
OBJ-01	*example — delete*	GOL-01	2 years	Not started

Form P-5 — Measures and the deliverable plan

Feeds `004 Measure` — `Ref`, `Measure`, `Unit`, `Domain`, `Baseline`, `Cadence` — and `031 Deliverable` — `Ref`, `Deliverable`, `Phase`, `Status`. **Nothing is typed into Excel. First pass** sixteen measures and twenty deliverables. **Signed by** the measure's owner, who is promising that a number can be produced at the stated `Cadence` without a spreadsheet; `GB-01 Architecture Board` signs the deliverable plan as the thing it will be held to. **Note** `004` has no `Target` and no `Owner` column, so a measure here is a baseline and nothing else. Put the target on the objective in `003` and keep the owner in the Statement of Architecture Work until you add the column.

Ref	Measure	Unit	Domain	Baseline	Cadence
MSR-01	*example — delete*	%	Compliance	23	Monthly

`031 Deliverable`. Twenty rows: every deliverable the first cycle will produce, named and phased before you start. `Phase` takes the single-letter ADM phase as the sheet does — A, B, C, D, E, F, G, H. The one Preliminary table a sponsor reads line by line.

Ref	Deliverable	Phase	Status
DLV-01	*example — delete*	A	Approved

Phase A — Architecture Vision

Three forms. The repository barely holds Phase A, and Form A-1 says so on its face.

BEFORE YOU FILL THE STAKEHOLDER REGISTER

`O10 Actor` and `O11 Role` ship eighteen rows each with the same eighteen names. Which of the two are you modelling, and what will you put in the other?

Which stakeholder can stop this work at `GT-1` without anybody's agreement? Name them.

Who on your list has not agreed to be on it? Say so before the sponsor signs.

The Chief Risk Officer and the Chief Compliance Officer will read the same obligation differently. Which obligation, and which of them wins?

Form A-1 — Stakeholder register, actors and roles

Feeds nothing directly. The repository has no stakeholder sheet. Sixty-five object sheets and not one of them is `Stakeholder`. The register lives in the Statement of Architecture Work as a document, and each line keys into the repository through a `ROL-` reference in `011 Role` or an `ACT-` reference in `010 Actor`. **Transcribe** the `Key` column into whichever of the two sheets the row belongs to; the concern and the plan stay in the document. **First pass** twelve to twenty names. **Signed by** the sponsor.

Name and post	Key	Concern, one line	Can stop	Class	Plan
T. Mahlangu, CRO	ROL-07	*example — delete*	GT-1	Keep satisfied	Monthly, 30 min

`010 Actor` — `Ref`, `Actor`, `Type`, `Status`. `Status` ships Filled, and a vacancy is the point of the column: `ZZ Checks` counts vacant architecture roles and reads zero, which is only true because every row says Filled.

Ref	Actor	Type	Status
ACT-01	*example — delete*	Executive	Filled

`011 Role` — `Ref`, `Role`, `Type`, `Performed by`. `Performed by` ships Person. A role performed by a team, a vendor or a shared service is where accountability leaks, so write what is true.

Ref	Role	Type	Performed by
ROL-01	*example — delete*	Executive	Person

Form A-2 — Value streams

Feeds `014 Value Stream` — `Ref`, `Value stream`, `Scope`, `Stages`, `Capabilities`. **Nothing is typed into Excel. First pass** five streams, which is what the repository ships. More than ten and you are writing processes, not streams. **Signed by** the Chief Operating Officer, with each stream's owning executive agreeing their own stream in the room. **`Stages` holds a count, not the stage names** — the shipped rows read '4 stages', '5 stages' — so the stage names live nowhere in the repository. Write them in order on the back of this sheet, numbered, and keep them in the deliverable: Phase B will need them when it names processes in `016 Business Process`, where the shipped rows are called 'Prospect to Customer — stage 1'. **`Capabilities`** is a comma-separated list of `C` references in one cell; keep it short and put the real mapping in `R02 Value stream uses Capabilit`.

Ref	Value stream	Scope	Stages	Capabilities
VS01	*example — delete*	Marketing through application, KYC and first account	4 stages	C01, C02, C03

The stages, in order, for each stream above. These feed `016` process names, so get the words right here and you will not rename twenty processes later.

Ref	1	2	3	4
VS01				

Form A-3 — Constraints and assumptions

Feeds `007 Constraint` — `Ref`, `Constraint`, `Description`, `Type` — and `008 Assumption` — `Ref`, `Assumption`, `Status`. **Nothing is typed into Excel**, which is why `Description` is on the form and the rows are tall. **First pass** eight constraints and six assumptions. **Signed by** the Chief Enterprise Architect, with the sponsor accepting the assumption list, because an assumption is a risk the sponsor is carrying without having said so. **Neither sheet has an expiry column.** `007` has no 'applies until' and `008` has no 'validate by', and a constraint with no expiry becomes a principle by neglect. Put the date in `Description` and `Assumption` as the first six characters — '2028-06: ' — until you add the columns.

Ref	Constraint	Description	Type
CON-01	*example — delete*	Run cost consumes most of the technology budget.	Financial

`008 Assumption`. `Status` ships Open. An assumption that is Open and under a funded work package is the one to validate first.

Ref	Assumption	Status
ASM-01	*example — delete*	Open

Phase B — Business Architecture

Four forms, and the two capability forms are the ones you will print most. `013 Business Capability` ships 120 rows on three levels with every row scored, so it splits: B-1 is structure and takes an hour at a desk, B-2 is judgement and takes three hours in a room with an executive in it.

Form B-1 — Capability identify

Feeds `013 Business Capability`, structure columns only — `Ref`, `Capability`, `Level`, `Parent`. **Typed into Excel** `Description`. The five judgement columns are Form B-2 and a different room. **First pass** the level 2 and level 3 rows under one level 1 capability — nine rows at the shipped shape, so this form runs twelve times. **Signed by** the executive who owns the level 1 area. **Read this before you print it.** The shipped level 2 and level 3 names are machine-generated: `C01.01 Customer Management Administration`, then `C01.01.01 Customer Management Administration — Define` and `C01.01.02 … — Operate`. That is a shape, not a capability map. Every one of the 108 rows below level 1 needs a real banking name — 'Mandate registration', 'Affordability assessment', 'Scheme settlement reconciliation' — and volume 1 appendix A, *Business capabilities*, drafts a set to argue with. Map it by name, never by number: its `C-01…C-20` are its own local identifiers and do not line up with this sheet's `C01…C12`.

Ref	Capability	Level	Parent
C05.02	*example — delete*	2	C05

BEFORE YOU SCORE A SINGLE CAPABILITY

Which of these capabilities would this bank pay to be better at than its competitors? Name the executive who says so, not the strategy document.

Which capability is Critical and scores Current 1 or 2 at the same time? Say it now, because the room will not admit both after lunch.

What would you see if you watched this capability being done for a week? That observation is what justifies the `Current` score, and nothing else does.

The shipped data scores every one of the 120 rows, including level 3. How many level 3 rows are you prepared to score yourself, and what will you do with the rest?

`RPT03 Capability Heatmap` reads level 1 only and computes `Gap` as target minus current. Which level 1 score will you have to defend to the Board, and on what evidence?

Form B-2 — Capability assess

Feeds `013 Business Capability`, judgement columns only — `Ref`, `Criticality`, `Current`, `Target`, `Disposition`. `Capability` is printed so the room knows which row it is scoring and is not retyped. **Values the sheet uses**: `Criticality` Critical, High, Medium, Low; `Current` and `Target` integers 1 to 5; `Disposition` Sustain, Build. **First pass** one level 1 area and everything under it, three hours. **Signed by** the owning executive, in the room, at the end of the session, because the scores drift by Friday. **Goes wrong when** a level 3 row scores higher than its parent — the shipped data does this repeatedly, `C01.01.01` at Current 4 under `C01.01` at Current 2 — and nobody notices because no report rolls level 3 up.

Ref	Capability	Criticality	Current	Target	Disposition
C05.02	*example — delete*	Low	1	2	Sustain

Form B-3 — Functions, processes and business services

Feeds three sheets. `017 Business Function` — `Ref`, `Business function`, `Capability`. `016 Business Process` — `Ref`, `Business process`, `Value stream`, `Automation`. `015 Business Service` — `Ref`, `Business service`, `Capability`, `Exposure`, `Criticality`. **Typed into Excel** `Description` on all three. **First pass** twelve functions, twenty processes, twelve services — three copies. **Signed by** the Chief Operating Officer. **There is no role-to-process sheet.** The repository has `R21 Role performed by Actor` and nothing that says who is accountable for a process, so a RACI cannot be held here; keep it in the process deliverable and say so in the minute. **`Automation`** ships Manual, Partial, Automated and is the column Phase C will come back and argue with.

Ref	Business function	Capability
BFN-01	*example — delete*	C01

`016 Business Process`. The shipped names are stream-and-stage placeholders. Replace them with what the bank calls the process — 'Register a DebiCheck mandate', 'Run a Registered Mandates collection' — and keep the stream reference.

Ref	Business process	Value stream	Automation
BPR-01	*example — delete*	VS01	Manual

`015 Business Service`. `Exposure` ships Internal and External. External means a customer, a counterparty or a scheme consumes it, which changes the resilience argument and the conduct argument at once.

Ref	Business service	Capability	Exposure	Criticality
BSV-01	*example — delete*	C01	Internal	Low

Form B-4 — Products, contracts and what supports them

Feeds `018 Product` — `Ref`, `Product`, `Type` — `019 Contract` — `Ref`, `Contract`, `Counterparty`, `Type`, `Expiry` — and `R25 Application supports Produc` — `Product`, `Product name`, `Application`, `Application name`. **Typed into Excel** `018 Description`. **First pass** eight products, ten contracts, eight support rows. **Signed by** the product head for products, the Chief Financial Officer and procurement for contracts. **`018` carries no regulatory classification** — no National Credit Act subtype, no `BA 900` instrument type, no deposit-insurance qualifying flag — and in a South African bank those three decide what the return says and what CoDI covers. Add the columns, or the product catalogue cannot answer the first question a supervisor asks. **`019` has no notice period**, which is what makes an exit real; put it in `Type` — 'Perpetual, 12 months notice' — until you add a column.

Ref	Product	Type
PRD-01	*example — delete*	Deposit

`019 Contract`. `Expiry` runs year-month in the shipped data. A perpetual agreement still has a review date and a notice period; write both.

Ref	Contract	Counterparty	Type	Expiry
CTR-01	*example — delete*	Core Banking	Perpetual	2027-01

`R25 Application supports Produc`. One row per product-and-application pair. A product supported by nothing is either retired or administered in a spreadsheet, and both are findings.

Product	Product name	Application	Application name
PRD-01	*example — delete*	A01	Core Banking

Phase C — Data Architecture

Three forms. `O20 Data Entity` splits the way the capability sheet does, because identity is a desk exercise and classification is an argument with the information officer in the room.

BEFORE YOU NAME A MASTER SYSTEM

Which record do the people who do the work actually trust? Ask the reconciliations clerk, not the head of department, and write what they said.

The shipped sheet masters `Party` in `Collections & Recovery`, `Customer` in `Regulatory Reporting` and `Financial Instrument` in `Fraud Detection`. Before you laugh: which of your own fourteen is as wrong, and who would have signed it?

`Customer` ships with `Classification` = Public. What is the classification of a customer record at this bank, and which POPIA provision decides it?

Which entity has four claimants and no master? Name it, and name who will be told the answer is 'undecided' in writing.

For each master you are about to designate: what breaks tomorrow if that system is wrong, and who finds out first?

Form C-1 — Data entity identify

Feeds `O20 Data Entity`, identity columns only — `Ref`, `Data entity`, `Class`, `Owner`, `Master system`. **Typed into Excel** `Definition`, one sentence per entity and the hardest sentence on the sheet. **Values the sheet uses**: `Class` Master, Transactional, Derived. **`Owner` must be a person**, and the shipped rows hold post titles — 'Head of Customer Management' — which is as far as a printed cell goes; write the name beside it and carry the name in the deliverable. **`Master system` holds an application name, not an `A` reference**, so it will not resolve against `O23 Application Component`; use the exact `Application` string from `O23` or the lookup fails. **First pass** fourteen entities, which is the whole sheet. **Signed by** `GB-03 Data Governance Council`, which the repository says designates data owners and resolves disputes. **Look it up.** Volume 1, chapter 9, *The spine: a party, an agreement, an account*, and chapter 10, *Reference and master data*.

Ref	Data entity	Class	Owner	Master system
E01	*example — delete*	Master	Head of Customer Management	Core Banking

Where ownership is contested. **This table feeds no sheet** — `O20` has one `Owner` cell and no way to record a dispute — so it goes in the minute and into `O60 Governance Decision` as a decision for `GB-03`.

Ref	Who claims it	Who decides	By when
E01	*example — delete*	GB-03	

Form C-2 — Data entity assess

Feeds `O20 Data Entity`, assessment columns only — `Ref`, `Classification`, `Lineage`. `Data entity` is printed for reading. **Values the sheet uses**: `Classification` Public, Internal, Confidential, Restricted; `Lineage` Full, Partial, Absent. **`Lineage` is the column a supervisor will test**, and `RPT05 Data Ownership` prints it next to the owner and the master system, so an entity with a named owner and `Absent` lineage is the shape that gets written up. The shipped data has three — `Credit Exposure`, `Limit`, `Financial Instrument` — and `ZZ Checks` counts them against a threshold of zero. **First pass** fourteen entities. **Signed by** the information officer for `Classification`, `GB-03 Data Governance Council` for `Lineage`. **Note** classification here is one value for the whole entity, which is too coarse for a bank: a card primary account number and a card expiry date are not the same thing under PCI DSS. Classify at the attribute level in `O37 Data Attribute`, which has its own `Classification` column, and keep the entity value as the floor.

Ref	Data entity	Classification	Lineage
E01	*example — delete*	Confidential	Partial

Retention, which `O20` has no column for and a South African bank cannot do without. One row per entity, the longest statutory clock that runs on it, and the instrument that sets it. **Transcribe into** a column you add, not into `Definition`.

Ref	Longest clock	Instrument	Disposal rule
E03	*example — delete*	*example — delete*	

Form C-3 — Entity relationships, data sets and master data rules

Feeds `036 Data Entity Relationship` — `Ref`, `From entity`, `Relationship`, `To entity`, `Cardinality` — `038 Data Set` — `Ref`, `Data set`, `Description` — and `043 Master Data Rule` — `Ref`, `Entity`, `Rule type`, `Description`, `Golden source`. **Nothing is typed into Excel. First pass** eleven relationships, six data sets, ten rules — three copies. **Signed by** `GB-03 Data Governance Council`. **`036` names entities by name, not by `Ref`** — the shipped rows read 'Party has Customer' — and `R19 Data Entity relates to Data` holds the same eleven rows with the same columns. Fill one and copy; two sheets disagreeing about the same eleven relationships is the commonest inconsistency in this repository. **`Rule type` on `043`** ships Validation and Survivorship. Survivorship is the one that matters in a bank: when two party records merge, which address, which risk rating, which FICA status survives.

Ref	From entity	Relationship	To entity	Cardinality
REL-01	*example — delete*	has	Customer	1:1

`038 Data Set`. Six rows. A data set is a named collection with a purpose — the regulatory reporting set, the customer analytics set — and the purpose is what POPIA makes you state before you reuse it.

Ref	Data set	Description
DST-01	*example — delete*	Data supporting statutory submissions

`043 Master Data Rule`. `Golden source` holds an application name, same trap as `020 Master system`. Where the rule and the master disagree, the rule is wrong or the master is.

Ref	Entity	Rule type	Description	Golden source
MDR-01	*example — delete*	Survivorship	Rule governing party mastering.	Core Banking

Phase C — Application Architecture

Four forms. `O23 Application Component` has ten columns and splits; `R06 Application interfaces with` is the sheet that will tell you the most about this estate and the one most likely to be filled in by guesswork.

Form CA-1 — Application identify

Feeds `O23 Application Component`, identity columns only — `Ref`, `Application`, `Type`, `In service`, `Users`, `Owner`. **Typed into Excel** `Description`. The judgement columns are Form CA-2. **Values the sheet uses**: `Type` Package, and whatever else your estate holds — write Build, SaaS or Package, and be strict, because a package that has been modified for fifteen years is not a package any more. **`In service`** is a four-digit year. **`Owner`** holds an organisation name in the shipped data, not a person; write the post and the name. **First pass** fourteen applications, which is the whole sheet. A real mid-tier bank will have between 300 and 900, so this form runs many times and you print it by domain, not by alphabet. **Signed by** the Chief Information Officer, with each `Owner` confirming their own row.

Ref	Application	Type	In service	Users	Owner
A01	*example — delete*	Package	2012	144	Executive

BEFORE YOU ASSESS AN APPLICATION

`Health` and `Criticality` are different questions. Which of your applications is Critical and Poor at once, and who has known that for longest?

The shipped data has `A02 Payment Hub` at `Criticality` Low. Which of your own criticality values would a supervisor laugh at?

For each Replace disposition: what is the decommissioning date, and what is still reading from it the day after?

Which application has nobody under fifty who can change it? Write its name here and nowhere else until you have told the Chief Information Officer.

Form CA-2 — Application assess

Feeds `023 Application Component`, judgement columns only — `Ref`, `Disposition`, `Criticality`, `Health`. `Application` is printed for reading. **Values the sheet uses**: `Disposition` Retain, Invest, Replace; `Criticality` High, Medium, Low; `Health` At risk, Poor, and whatever your scale adds — fix the vocabulary before the session and write it at the top of the form. **Shows up in** `RPT02 Application Portfolio`, which prints all nine columns side by side, and `RPT13 Orphan Analysis`, which names the applications no business service maps to. **First pass** fourteen rows. **Signed by** the Chief Information Officer; `Criticality` countersigned by the business service owner, because criticality is a business judgement wearing a technical column heading.

Ref	Application	Disposition	Criticality	Health
A01	*example — delete*	Retain	High	At risk

Form CA-3 — Application to data entity

Feeds `R05 Application manages Data En` — `Application`, `Application name`, `Data entity`, `Entity name`, `Role`. **Nothing is typed into Excel. Values** `Role` Master, Consumer. One Master row per entity and as many Consumer rows as there are readers. **First pass** twenty-eight rows at the shipped shape — fourteen Master, fourteen Consumer. **Signed by** `GB-03 Data Governance Council` for the Master rows; the application owner for the Consumer rows. **Fix this on the way in.** In the shipped sheet the fourteen Master rows carry the application *name* in the `Application` column where the fourteen Consumer rows carry the `A` reference. Half the sheet will not resolve. Put the `A` reference in `Application` on every row and the name in `Application name`, and check it before you add anything. **Goes wrong when** two rows claim Master for the same entity. That is not a data-entry error, it is the finding, and it belongs on Form E-1 as a gap.

Application	Application name	Data entity	Entity name	Role
A01	*example — delete*	E01	Party	Master

BEFORE YOU WRITE AN INTERFACE DOWN

`ZZ Checks` says nineteen of thirty-two interfaces are unmanaged and twenty have no logging. Which of yours carries customer money and no log?

Ten rows are `Direct DB read`. For each, who would know if the schema changed on Friday night?

`Managed` means routed through managed integration. Who decides that, and have they looked, or are they telling you what the design document says?

Which interface exists that nobody will admit to? Name the file drop, the service account and the server it runs on.

Form CA-4 — Interface register

Feeds `R06 Application interfaces with` — `Ref`, `Source`, `Target`, `Type`, `Managed`, `Logged`. **Typed into Excel** `Source name`, `Target name` and `Frequency`; the two name columns are echoes you fill by lookup from `023`, not by hand. **Values the sheet uses**: `Type` Direct DB read, Event, File, API; `Frequency` Nightly, Hourly, Real-time; `Managed` Managed, Unmanaged; `Logged` Yes, No. **First pass** thirty-two interfaces at the shipped shape; a real bank will fill several copies and still be short, because the file drops nobody owns are the ones missing. **Signed by** the Chief Information Officer, with the integration lead confirming `Managed` by looking at the integration platform rather than at the design. **Shows up in** `RPT04 Interface Catalogue` and in three lines of `ZZ Checks` — unmanaged, unlogged, direct database reads — each with a threshold of zero. Those three lines are the most useful numbers in the repository, and they only work if this form was filled honestly.

Ref	Source	Target	Type	Managed	Logged
INT-001	*example — delete*	A03	Direct DB read	Unmanaged	No

Continue on a second copy of this form. Number the copies, because a renumbered `INT-` reference breaks `RPT04`.

Phase D — Technology Architecture

Three forms, and two of the three sheet names are truncated at Excel's thirty-one characters. Spell them as the tab spells them.

Form D-1 — Technology services and components

Feeds `025 Technology Service`, `026 Logical Technology Componen` and `027 Physical Technology Compone`, columns as printed. **Typed into Excel** `025` `Description` and `027` `Location`. **First pass** twelve services, seven logical, twelve physical — three copies. **Signed by** the Chief Information Officer; `Support end` countersigned from the vendor agreement, not from the relationship manager. **`Support end` and `Position` are the pair that matters**: `RPT10 Technology Currency` prints them together and `ZZ Checks` counts components past support against a threshold of zero. The shipped data has three.

Ref	Technology service	Category
TSV-01	*example — delete*	Platform

`026 Logical Technology Componen`. A logical component is a capability of the platform, not a product: 'ISO 20022 message backbone', not the vendor's name for it. `Status` ships Current.

Ref	Logical technology component	Rationale	Status
LTC-01	*example — delete*	24×7 availability with sub-second response.	Current

`027 Physical Technology Compone`. Write `Position` as Supported, Extended or Past support, and date `Support end` even where the vendor will not commit.

Ref	Physical technology component	Vendor	Class	Support end	Position
PTC-01	*example — delete*	IBM-class	Mainframe	2028-05	Supported

Form D-2 — Standards and conformance

Feeds `O32 Standard` — `Ref`, `Standard`, `Category`, `Status`, `Conformance` — and `R18 Standard applies to Applica` — `Standard`, `Standard name`, `Application`, `Application name`, `Conformance`. **Nothing is typed into Excel. Values the sheet uses**: `Category` Integration, Data, Security, Platform, Delivery, Governance; `Status` Current, Emerging, Deprecated; `Conformance` Conformant, Partial, Breach, Not assessed. **First pass** sixteen standards and sixteen application assessments; appendix G proposes a fuller catalogue, including the ones a South African bank cannot avoid — ISO 20022, the scheme mandates, PCI DSS, the SACRRA layouts, FinSurv category capture. **Signed by** `GB-01 Architecture Board`. **`Conformance` appears on both sheets and means two different things**: on `O32` it is the estate-wide verdict, on `R18` it is this application against this standard. Fill `R18` first and let `O32` follow, or the headline will not match the detail. **A `Deprecated` standard with `Partial` conformance** — the shipped data has two — is a standard being cited at gates on the authority of nobody.

Ref	Standard	Category	Status	Conformance
STD-01	*example — delete*	Integration	Current	Partial

`R18 Standard applies to Applica`. One row per standard-and-application pair. **Not assessed is not conformance.** Count the applications in scope, not the ones you assessed, or the conformance rate rises while the estate gets worse.

Standard	Standard name	Application	Application name	Conformance
STD-01	*example — delete*	A01	Core Banking	Partial

Form D-3 — Resilience tiers

Feeds `049 Resilience Tier` — `Ref`, `Application`, `Tier`, `Recovery objective`, `Tested`. **Nothing is typed into Excel. First pass** twelve rows, one per application in scope. **Signed by** the Chief Operating Officer and the Chief Information Officer jointly, with the business service owner agreeing the recovery objective, because a recovery objective nobody in the business has agreed is a number the technology team made up. **Fix the scale before you fill this in.** The shipped data has `Tier 1` applications at a twenty-four-hour recovery objective and a `Tier 3` application at fifteen minutes, which means the tier numbering and the objectives disagree. Decide whether Tier 1 is the most critical or the least, write it at the top of the form, and make the objectives monotonic. **`Tested` is the column that costs money and saves careers**: eleven of the twelve shipped rows read 'Never tested'. A tier with no test is an aspiration, and under the operational-resilience obligation in `063 Control Objective` row `CO-06` it is an aspiration a supervisor will ask to see evidence for. **`Application` holds a name, not an `A` reference** — same trap as `020` and `043`.

Ref	Application	Tier	Recovery objective	Tested
RES-01	*example — delete*	Tier 1	15 min	Never tested

The test record, which `049` has no columns for. One row per test. **Transcribe into** columns you add — a date and what was exercised — because 'Tested' with no date is 'tested once, by somebody who has left'.

Ref	Date tested	What was exercised	Failed on
RES-01		*example — delete*	

Phase E — Opportunities and Solutions

Three forms. `029 Gap` ships with **zero records**, which makes `RPT07 Gap Analysis` and `R12 Gap closed by Work Package` empty, and that is the largest hole in the repository. Form E-1 is the one that fills it.

BEFORE YOU RAISE A GAP

A gap is the distance between baseline and target, not a complaint. For your first ten, write the baseline fact and the target fact in the same sentence.

`ZZ Checks` hands you nine gaps before you start: unmanaged interfaces, missing logging, direct database reads, components past support, absent security controls, control objectives with no evidence, entities with no lineage, work packages past their end date. Which of those are you raising, and which are you consolidating?

Which gap will somebody senior ask you to downgrade, and what is the evidence that holds the severity?

The sheet has a `Due` column. Who agreed the date, and what happens on the day after it?

Form E-1 — Gap register

Feeds `029 Gap` — `Ref`, `Gap`, `Domain`, `Severity`, `Status`, `Due`. **Typed into Excel** `Building block`, which stays empty until Form E-2 exists. **Values**: `Domain` Business, Data, Application, Technology; `Severity` Critical, High, Medium, Low; `Status` Open, In progress, Closed. **First pass** fifteen to twenty-five gaps. The repository ships none, so every row here is new. **Signed by** `GB-01 Architecture Board`, with the domain owner accepting the severity. **Shows up in** `RPT07 Gap Analysis`, which prints the gap, its severity and the work package that closes it, and in `R12 Gap closed by Work Package`. A gap with no work package is the number the Board should be given, and it should be given it every month. **Write the gap as a fact, not as a wish**: 'Nineteen of thirty-two interfaces are not routed through managed integration', not 'integration needs improvement'. The first can be closed and tested. The second cannot be closed at all.

Ref	Gap	Domain	Severity	Status	Due
GAP-01	*example — delete*	Application	Critical	Open	2028-03

Form E-2 — Building blocks and acceptance criteria

Feeds `055 Building Block` — `Ref`, `Domain`, `Building block`, `What it must do`, `Acceptance criteria`. **The same thirty-seven rows also live on four domain sheets** — `035 Business Building Block`, `044 Data Building Block`, `045 Application Building Block`, `046 Technology Building Block` — each with the domain column replaced by a pointer to the thing it uplifts. Fill `055` and copy down; filling five sheets by hand guarantees five versions. **Watch `046`**: it has **two columns both headed `Ref`**, the second holding a `T` reference to the logical technology component. The form prints it as `Ref (T)` so the transcriber puts it in the right place. **First pass** thirty-seven blocks — ten business, ten data, ten application, seven technology. **Signed by** `GB-01 Architecture Board`. **`Acceptance criteria` is the column that decides whether the block can ever be signed off**, and it is where the requirement's test lives, because `006 Requirement` has no acceptance-criteria column of its own. If you cannot write a test, you have written an intention and `GT-2` will have nothing to assess.

Ref	Domain	Building block	What it must do	Acceptance criteria
BBB-01	*example — delete*	*example — delete*	*example — delete*	*example — delete*

`046 Technology Building Block` only — the second `Ref` column, which points at `026 Logical Technology Componen`.

Ref	Ref (T)	Technology building block
TBB-01	T01	*example — delete*

Form E-3 — Work packages and transition architectures

Feeds `030 Work Package` — `Ref`, `Work package`, `Domains`, `Cost`, `Start`, `End` — and `053 Transition Architecture` — `Ref`, `Transition architecture`, `Work packages`, `Exit criteria`. **Typed into Excel** `030 Approval` and `Delivery`, both of which change weekly and should not be on paper; `053 Description`. **First pass** twelve work packages and three transitions. **Signed by** `GB-02 Investment Committee` for cost and dates, `GB-01 Architecture Board` for the transition boundaries. **`Cost` ships in dollars** in the shipped data — '$7.7m' — which for a South African bank is wrong and should be rand. Change the whole column or none of it. **`Exit criteria` holds a count, not the criteria** — the shipped rows read '5 exit criteria' — so write the criteria themselves on the second table and keep them with the transition deliverable, because an exit criterion you cannot read is an exit nobody can refuse. **`Delivery` has a value 'Past end date'** and four shipped rows carry it; `ZZ Checks` counts them against zero.

Ref	Work package	Domains	Cost	Start	End
WP-01	*example — delete*	Business	R7.7m	2028-09	2030-05

`053 Transition Architecture`. Three transitions at Ardent Union: governed and supportable, integrable, then the business change. The exit criteria, written out, one per line.

Ref	Transition architecture	Work packages	Exit criterion, written out
TA-1	*example — delete*	WP-01, WP-02	*example — delete*

Phase F — Migration Planning

One form, two sheets. The roadmap itself is `RPT08 Roadmap` and computes from `O30`, so there is nothing to fill there.

Form F-1 — Projects and implementation factors

Feeds `056 Project` — `Ref`, `Project`, `Work package`, `Start`, `End`, `Lead` — and `052 Implementation Factor` — `Ref`, `Implementation factor`, `Category`, `Deduction`. **Typed into Excel** `056 Cost`, which duplicates `030 Work Package Cost` and should be copied rather than re-estimated. **First pass** twelve projects and eight factors. **Signed by** `GB-02 Investment Committee` for projects, the Chief Enterprise Architect for factors. **`Lead` is a post title** in the shipped data and must be a person who knows they have the job. **`052` is the sheet nobody fills and everybody needs**: a dependency, a resource constraint or a political fact, with the deduction it forces on the plan. The shipped rows are honest about the shape — 'Vendor selection not made', 'Data owners not designated' — and `Deduction` says what it blocks. Write the deduction as a consequence, not as a risk: 'Blocks mastering work until Q3', not 'may cause delay'. **There is no sheet for delivery risk and implementation factor together**; `062 Delivery Risk` is separate, with `Likelihood` and `Impact`, and a factor is not a risk — a factor is a fact you must plan around.

Ref	Project	Work package	Start	End	Lead
PRJ-01	*example — delete*	WP-01	2028-09	2030-05	Chief Information Officer

`052 Implementation Factor`. `Category` ships Dependency. Add Resource, Capability, Political, Regulatory — and be willing to write Political down, because the plan that ignores it fails on it.

Ref	Implementation factor	Category	Deduction
IF-01	*example — delete*	Dependency	Blocks integration work until resolved

Phase G — Implementation Governance

One form, two sheets, and the second of them ships empty. `O59 Compliance Assessment` with zero rows means no building block has ever been tested against its acceptance criterion, which is the whole of Phase G.

Form G-1 — Architecture contract and compliance assessment

Feeds `033 Architecture Contract` — `Ref`, `Architecture contract`, `Work package`, `Compliance status` — and `059 Compliance Assessment` — `Ref`, `Building block`, `Acceptance criterion`, `Gate`, `Verdict`, `Evidence`. **Typed into Excel** `059` `Block name`, an echo you fill by lookup from `055`. **Values**: `033` `Compliance status` Non-compliant, Not yet assessable, Compliant; `059` `Gate` takes a `GT-` reference; `Verdict` Pass, Fail, Partial. **First pass** eight contracts and one assessment row per acceptance criterion per gate, which at thirty-seven blocks and two assessing gates is seventy-four rows — several copies. **Signed by** `GB-01 Architecture Board`, which the repository says holds `GT-1` and `GT-2`. **`Evidence` is the column that makes the verdict real.** Name the artefact: a test report with a date, a screenshot of the integration platform, a signed minute. 'Confirmed by the project' is not evidence, it is an assertion with a name on it. **A Fail with no gap row on Form E-1 is a Fail nobody will act on**, and `R24 Compliance Assessment tests` carries the link.

Ref	Architecture contract	Work package	Compliance status
ACO-01	*example — delete*	WP-01	Not yet assessable

`059 Compliance Assessment`. One row per criterion per gate. The sheet ships empty; every row is new.

Ref	Building block	Acceptance criterion	Gate	Verdict	Evidence
CA-01	BBB-01	*example — delete*	GT-1	Partial	

Phase H — Architecture Change Management

Two forms. The first feeds sheets that exist. The second feeds four that do not, and that is the point of it.

Form H-1 — Governance decision and change request

Feeds `O60 Governance Decision` — `Ref`, `Decision`, `Body`, `Status` — and `O61 Governance Change Request` — `Ref`, `Change request`, `Against`, `Type`, `Status`, `Expires`. **Nothing is typed into Excel. Values**: `O60` `Status` Outstanding, Taken; `O61` `Type` Dispensation, Scope; `Status` Granted, Approved, Refused; `Against` takes a `WP-` reference. **First pass** seven decisions and five change requests. **Signed by** the body named in `Body`, and the minute reference goes in `Decision` where there is no column for it. **`Expires` is compulsory on a Dispensation and the shipped data leaves it blank on the Scope rows**, which is correct — a scope change does not expire, a dispensation must. A dispensation with no expiry is a permanent exception granted by accident. **This sheet records a decision, not its reasoning.** `O60` has four columns and no rationale, so the architecture decision record lives outside the repository and `Decision` must carry enough words to find it: 'Adopt alternative B — see ADR-014'. **Goes wrong when** a dispensation expires and nothing happens. Put the expiry date in the `GB-01` standing agenda the day you grant it.

Ref	Decision	Body	Status
DEC-01	*example — delete*	Investment Committee	Outstanding

`O61 Governance Change Request`. One row per request. Read the four shipped rows before you add to them: they are a good model of how short a change request can be and still be auditable.

Ref	Change request	Against	Type	Status	Expires
CR-G01	*example — delete*	WP-04	Dispensation	Granted	2029-06

Form H-2 — The change log and change-request sheets you must create

Feeds four sheets that do not exist in this repository. `Banking_TOGAF_Metamodel.xlsx` ships no version control, no change log, no change-request register and no baseline sheet, and no object sheet carries a `Change Ref`, `Changed By` or `Approved By` column. That absence is a finding, not an inconvenience: as shipped, a judgement can be changed by anybody with the file and nothing records that it was. **Before you load a record**, add two columns — `Change Ref` and `Approved By` — to every object sheet, and add two sheets with the columns below. Name them so they sort after `ZZ Checks` and not between the numbered sheets — a change log inserted at `R26` shifts every report range that reads past it. This appendix calls them the change log and the change-request register; the Preliminary chapter of this volume settles the names, and the only wrong answer is a different name on each architect's copy. **This form is for changes made away from the keyboard**: a correction a signatory dictates, a decision taken in a corridor. At a desk, type into the sheet. **Signed by** `GB-01 Architecture Board` for the sheet design, once; then by whoever the changed row's own form says signs it.

The change log. **One line per field changed** — not one per record and not one per sitting. Six columns changed on nine rows is fifty-four lines, and there is no report to catch you shortcutting it, which is exactly why you must not.

Change	Sheet	Record	Field	Previous value	New value
CHG-001	O20	E02	Classification	Public	Confidential

The change request. One per session, raised **before** the session, with `Scope` naming the sheets the session may touch. Every row the session produces carries that `CR` reference in `Change Ref`.

Change request	Raised	Scope — sheets	Status	Approved by	Date
CR-001	2027-03-14	O13, R22	Open	GB-01	

Requirements Management

One form, used in every phase. Requirements arrive between the phases, which is why it lives at the back where you can find it without knowing which phase you are in.

BEFORE YOU WRITE A REQUIREMENT DOWN

Which driver does this requirement derive from? If you cannot name one, whose preference is it?

Write the test that closes it. `006 Requirement` has no acceptance-criteria column, so the test has to live on a building block — which one?

`RPT06 Requirements Traceability` has a column called `Reaches a block`. How many of your requirements will read No, and which of those are Must?

Which obligation in `063 Control Objective` has no requirement behind it yet? There are seven obligations and four of them have no mechanism or a mechanism in breach.

Form R-1 — Requirements, obligations and the acceptance criterion

Feeds `006 Requirement`, `063 Control Objective` and `R09 Requirement satisfied by Bu`, columns as printed. **Typed into Excel** `006 Statement`. **First pass** thirty-two requirements, seven obligations, thirty-two trace rows — four copies. **Signed by** `GB-01 Architecture Board` for the set; the Chief Risk Officer and the compliance officer jointly for `063`. **`006` has no acceptance-criteria column and no owner column.** The test lives in `Acceptance criteria` on the building block that satisfies it, through `R09`; the owner lives nowhere, so add the column. **`Source` on `063` takes a section, not a regulator** — 'POPIA s72', 'Joint Standard 2 of 2024' — and the shipped rows hold a description where the section should be.

Ref	Requirement	Category	Priority	Driver	Status
REQ-001	*example — delete*	Regulatory	Must	DRV-01	Approved

`063 Control Objective`. **No mechanism is worse than In breach**: a breach has a control that failed, no mechanism has nothing at all. Four of the seven shipped rows are one or the other.

Ref	Obligation	Source	Control objective	Evidence
CO-01	*example — delete*	POPIA s72	*example — delete*	No mechanism

`R09 Requirement satisfied by Bu`. A Must requirement with `Traced` = No is the most expensive blank in the repository; `RPT06 Requirements Traceability` prints it last.

Requirement	Requirement name	Building block	Block name	Traced
REQ-001	*example — delete*	BBB-01	*example — delete*	No

Transcription — the part that makes the paper worth filling

A filled form is not a record. It is a promise to make a record, and the promise has a shelf life of about a day.

Form to sheet the same day

Transcribe before you go home. Not out of diligence but because of memory: the cells that are hard to read are the cells where the room argued, and you are the only person who remembers what the argument was. By Thursday you will be guessing at your own handwriting, and a guess transcribed looks exactly like a decision.

Book the hour with the session. A three-hour capability workshop needs an hour after it, in the diary, before the workshop is booked. If you cannot find the hour, run a shorter session: twelve rows transcribed beat twenty rows in a folder.

Who types

The person who held the pen types. Not an assistant, not the domain architect who was not in the room, not the graduate. The pen-holder is the only person who can tell a 3 from a crossed-out 4, and the only one who knows which cell carries a caveat the paper does not show. Where volume makes that impossible — `R01 Capability decomposition` at 108 rows, `R09` at thirty-two — the pen-holder sits beside whoever types and reads the rows aloud. It is slower, and it is still faster than fixing it twice.

Type into the sheet, not into a staging spreadsheet. A staging file is a second repository with no change control, and it will develop opinions of its own.

Run `ZZ Checks` before and after

`ZZ Checks` is twenty-one lines and it is the cheapest thing in the repository. Note the numbers before you transcribe and again after. Three of its lines move whenever you touch the interface register, three when you touch the gap register, one when you touch resilience. A line that moved and should not have is a transcription error you have just caught for free; a line that did not move and should have is a form you forgot to type up.

Do the same with `RPT01 Dashboard`, which carries the same counts in a form an executive will read. A dashboard that changed without a change log line is the position this repository ships in, and Form H-2 is how you leave it.

The change reference on every row

Every row you add or change carries a `Change Ref` — the column you added on Form H-2. One change request per session, raised before the session rather than after it, with `Scope` naming the sheets the session may touch. Then every row from that session carries the same `CR` reference, and every field change gets a line in the change log against it.

There is no report in this repository that catches you skipping this, which is the opposite of reassuring. In the insurance repository a version-control report compares records changed against changes logged, sheet by sheet. Here nothing does. A sheet showing nine records changed and no changes logged will look identical to a sheet nobody has touched, and the first person to ask where a number came from will not get an answer. Until you build the check, the discipline is the only control there is.

`Approved By` stays empty until it is true. An `Approved By` filled in hope is the one entry in the repository that is actively dishonest.

When the room disagrees

Record both and mark the row Draft. This is the most useful habit in this appendix.

On the paper, write both values in the cell — '2 / 4', with the initials of whoever argued each. In the repository, put the one the owner signed, set `Status` to Draft where the sheet has a `Status` column, and put the other in the change log `Reason` field you created. Then name the disagreement: head of payments 2, chief operating officer 4, owner signed 2, unresolved.

A disagreement recorded is worth more than an agreement invented. On `O13 Business Capability` it shows up in `RPT03 Capability Heatmap` as a `Gap` that satisfies nobody, which is the correct outcome and will bring the right people back to the table. What you must not do is split the difference. A `Current` of 3 that nobody in the room said is a number you invented, and you will be asked to defend it.

Put a date on every Draft row. A row that has been Draft for two quarters is a decision somebody is avoiding.

Retiring a filled form

Keep the paper until three things are true: the rows are in the sheet, the change log lines exist with the change request against them, and `Approved By` has a name in it. Until then the paper is the only evidence of what the room decided, and losing it means running the session again.

When all three are true, do this. Write the change-request reference and the transcription date across the top of the form. Scan it. File the scan where the deliverable lives, named after the form number and the session date rather than after the sheet, and record the location against the row in `O31 Deliverable` — which has no location column, so add one. Then destroy the paper, because a filled Form C-2 carries classification and retention decisions about personal information, and a filled Form A-1 carries named people and what they are worried about. Neither belongs in a drawer.

One exception. The forms signed as evidence at a gate — G-1, and any Form E-2 acceptance criterion a gate relied on — keep their originals in ink for as long as the architecture contract runs. Those are the ones somebody outside the architecture function may one day ask to see.

THE CHAPTER IN TEN LINES

- Print the forms for the phase you are in, not all thirty.
- Fill them by hand in the room; a laptop steers the conversation.
- Every column is a real column of a real sheet, in sheet order, with the tab names truncated as Excel truncated them.
- Six columns maximum — a wider sheet splits into identify and assess.
- Form A-1 feeds nothing: sixty-five object sheets and no stakeholder sheet.
- Form H-2 feeds four sheets you have to build, because this repository has no change control at all.
- Answer the ask set before the session, not during it.
- Transcribe the same day, and the person who held the pen types.
- Run `ZZ Checks` before and after; three of its lines move when you touch the interface register.
- When the room disagrees, record both values and mark the row Draft.

SOURCES FOR THIS CHAPTER

- **1.** Banking_TOGAF_Metamodel.xlsx — sheet names, column headings and shipped values as read from the workbook

APPENDIX G

The principles and standards starter set

Forty-five principles and thirty-one standards somebody has already argued with, a tier column the repository does not have and cannot work without, and the ninety minutes that turn an inherited set into an adopted one.

Drafting principles from nothing is the most reliably wasted week in the Preliminary Phase. Put five bank executives in a room with a blank page and they will write forty statements nobody can argue with, in no order, with no test, and the catalogue will be quoted in one steering committee and never again. The failure is structural rather than a failure of effort: a principle written to be agreed is written to be unarguable, and an unarguable statement cannot break a tie.

The useful work is the other thing — adopting, editing and rejecting a set somebody else has already argued with. Argument is cheaper against a position than against a blank page. "Reuse, then buy, then build" takes thirty seconds to read and provokes an immediate objection from the person holding a scheme mandate with a fixed date and nothing to reuse, and that objection is the beginning of a tier decision. The same person given a blank page writes "we will make sound sourcing decisions", and nothing follows from it.

So this appendix hands you a set. Forty-five principles and thirty-one standards, each marked with where it came from. `Banking_TOGAF_Metamodel.xlsx` ships twenty-four principles on `005 Principle` and sixteen standards on `032 Standard` [1]. Of the twenty-four principles, eight are kept unchanged, fifteen are edited, one is recommended for retirement, and twenty-one are new — added for things a South African bank needs and the inherited set does not cover. Of the standards, sixteen come from the repository and fifteen are new, and eleven of those are specific to banking in this country.

Ardent Union Bank, the bank in that repository, is invented. The shape of what it ships is not.

> **ADOPTING THE SET UNCHANGED IS THE SECOND FAILURE, NOT THE SAFE OPTION**
>
> Running this set through a workshop and emerging with forty-five adoptions and no edits means you tested nothing against your own bank. It looks like efficiency and it reaches the same place as the blank page, faster. The adoption workshop later in this appendix sets a floor: one rejection or edit per category, or the session does not close.

What makes a principle real

A principle is real when three things are true of it. **It can be broken** — there is a decision somebody could take next Tuesday that would breach it, and you can describe that decision. **You can point at the evidence** — a sheet, a report, a demonstration that shows whether it is being followed. **Something**

happens when it is broken — a refusal at `GT-1`, a dispensation with an expiry in `061 Governance Change Request`, a row in `029 Gap` with a severity on it.

Most inherited principles fail the second test. They carry a statement, which makes them a position, and an owner, which makes them somebody's position. What turns a position into a control is a named test: the thing you would ask to be shown. `005 Principle` has no column for it — `Ref`, `Principle`, `Category`, `Statement`, `Owner`, `Status`, and that is all [1] — so the test lives in the adoption record until you add the column. Add the column.

The three-question motherhood test

#	Question	Pass	Fail
1	Would any sane executive argue the opposite?	Somebody does argue it, regularly, and you can name when	Nobody argues against it. Add the arguable clause, or drop it
2	Does it say what to do when two good things conflict?	It names what it yields to, or what yields to it	It states a good outcome with no priority. Add the tier and the exception
3	Can you name a decision it would have changed in the last year?	You can name the decision, the date and what changed	You cannot. It is decoration, or nobody consulted it

Question three has no mechanical answer in this repository, and that is worth sitting with. There is **no relationship sheet binding a principle to anything** — twenty-five relationship sheets, and not one of them has a principle at either end — and no report over `005`. So nothing counts how many requirements a principle governs, and a principle that governs nothing looks exactly like a principle that governs everything. The last section of this appendix says what to add.

Two worked sharpenings

Weak. `PRN-04 Business Continuity` — *operations continue despite the loss of any single component* [1]. Nobody argues against it, so it fails question one. There is nothing in the repository that could fail it either: `049 Resilience Tier` holds twelve rows and eleven of them read `Tested` = Never tested, and the principle is silent on that because it talks about design rather than about proof.

Sharpened. *Operations continue despite the loss of any single component, site or supplier, demonstrated by a test inside the tier's stated cadence rather than by a plan.* The second clause is argued against every time a business owner declines a test window in the collections month-end. **The test:** the last recovery test for one Tier 1 application, its date, what it exercised, and the `029 Gap` row raised by anything it failed. `049` already carries `Tier`, `Recovery objective` and `Tested`, so the test costs nothing to run — and it will read badly, which is the point.

Weak. `PRN-10 Data is Shared` — *data is shared through published interfaces, not copies* [1]. Agreed by everyone, and the repository's own `ZZ Checks` says ten interfaces are direct database reads and nineteen of thirty-two are unmanaged [1]. A principle that the file it ships in contradicts on its own check sheet is not being applied.

Sharpened. *Data is shared through a published, versioned interface; the system holding a record may not refuse access to the designated owner, and no application reads another application's store.* Three clauses, three

different opponents: the integration team that wants a file drop by Friday, the business unit that will not release its customer file, and the reporting team that already has a read replica. **The test:** `RPT04 Interface Catalogue` filtered to `Type` = Direct DB read, with a dated containment or a gap row against every one. And the principle moves to Tier 3, because it loses predictably to purpose limitation and to classification — which is the honest position and makes it usable rather than ornamental.

The set

Six categories, forty-five rows. `Ref`, `Principle` and `Statement (condensed)` transcribe into `005 Principle` columns A, B and D; `Category` comes from the heading above each table; `Owner` and `Status` come off Form PS-1. `Tier` and `Source` are **not `005` columns** — `Tier` is a column you add, and `Source` is working information for the adoption pass that you drop when you paste.

> **TWO COLUMNS THE SHEET DOES NOT HAVE, AND ONE VALUE IT DOES NOT HAVE**
>
> `005 Principle` has **no `Tier` column**, so the conflict rule in `PRN-42` has nothing to read. Add `Tier` before you load the set; it is one integer per row and it is the difference between a catalogue and a control. `005` also has **no `Category` value for Governance** — the shipped data uses Business, Application, Data, Technology and Security — so the four governance principles below need a sixth value added to the column. And `Status` ships only Approved and Draft: Retired is a third value you add, because deleting a retired principle breaks every decision that cited it.

Business

Ref	Principle	Statement (condensed)	Tier	Source
PRN-01	Primacy of Principles	Applies to every part of the bank and every initiative, including scheme-delivered, outsourced and software-as-a-service work	1	edited
PRN-02	Maximise Benefit to the Enterprise	Enterprise benefit outranks the fastest option for one line, and the cost of the slower path is stated and accepted	3	edited
PRN-03	Compliance is Designed, Not Added	Obligations are architecture requirements from the outset, with the evidence designed in and named	1	repository
PRN-04	Business Continuity	Operations survive the loss of any component, site or supplier, demonstrated by a test inside the tier's cadence	2	edited
PRN-25	Every Product Carries Its Regulatory Classification	Credit-agreement subtype, return instrument type and deposit-insurance eligibility are product attributes, derivable for every account open	1	new
PRN-26	A Payment Carries Its Clearing Stream	Every payment records the stream it travelled; rulebook, cut-off, return window and dispute process are read from it, never assumed	2	new
PRN-27	Customer Outcomes are Evidenced, Not Asserted	Product governance decisions, target market and delivered outcomes are held as data at product and cohort grain	2	new
PRN-28	Delegated Work Keeps the Record Here	Business written or serviced by a third party is held in the bank's own records at account and transaction grain, on the bank's own clock	2	new

Data

Ref	Principle	Statement (condensed)	Tier	Source
PRN-09	Data is an Asset	Every data entity has an owner, a definition and a lifecycle, and the lifecycle includes disposal	3	edited
PRN-10	Data is Shared	Shared through a published versioned interface; the holder may not refuse the designated owner; no application reads another's store	3	edited
PRN-11	Data is Accessible	Those entitled can reach it without a project, within the classification that governs it	3	repository
PRN-12	Common Vocabulary	One agreed definition per concept; where two parts of the bank disagree, the disagreement is recorded and visible	2	edited
PRN-13	Data Ownership is Individual	An owner is a named person, not a committee and not a department	2	repository
PRN-14	Classification Travels with Data	Handling follows the data, not the document, the report or the system it happens to sit in	2	repository
PRN-29	The Ledger is Immutable and Triple-Dated	Entries are never amended; a correction is a new entry; every entry carries its effective, posting and recorded dates	1	new
PRN-30	The Reporting Layer Sits Finer Than Any Return	Data is held at a grain finer than any current return requires, and the mapping to a return is metadata, not code	2	new
PRN-31	One Reported Figure, One Derivation	Each reported figure has one definition and one derivation path; a second basis is derived from the first, never separately sourced	2	new
PRN-32	The Single Customer View is a Specified Structure	The deposit-insurance view is built to the regulator's field set and aggregation rules, reproducible as at any reporting date	1	new
PRN-33	Economic Purpose is Captured, Never Inferred	Where a reporting category encodes purpose, it is captured from the customer at the point of transacting, in every channel that can originate	1	new
PRN-34	Retention is the Longest Clock	Every entity carries the longest statutory retention that runs on it, with the instrument named and a disposal rule that actually runs	2	new

Application

Ref	Principle	Statement (condensed)	Tier	Source
PRN-05	Common Use Applications	Prefer a shared capability to a duplicated one — except where a scheme or statutory date cannot wait for one to be built	3	edited
PRN-06	Technology Independence	Capability and data are not permanently bound to a platform or a supplier, and the exit is costed before selection	3	edited
PRN-07	Ease of Use	A system that is hard to use will be worked around, and the workaround becomes the real process	3	repository
PRN-08	Service Orientation	Capability is exposed as a versioned, documented service with a contract; the contract outlives the implementation	2	edited
PRN-35	Translate at the Boundary, Never Inward	External message standards are translated at the edge; the internal model is never restructured to match a scheme's dialect	2	new
PRN-36	A Mandate is a Long-Lived Object	A collection mandate is mastered once as a long-lived object with its own lifecycle; instructions reference it and never re-establish it	2	new
PRN-37	One Component Owns Each Rail	Each clearing stream has exactly one component that speaks to it; no second path to a rail, however urgent	3	new

Technology

Ref	Principle	Statement (condensed)	Tier	Source
PRN-15	Requirements-Based Change	Change is driven by a stated requirement that traces to a driver; an initiative with neither is somebody's preference	3	edited
PRN-16	Responsive Change Management	*Recommended for retirement — see below*	—	retire
PRN-17	Control Technical Diversity	Each capability has one approved technology unless a departure is recorded with a date	2	repository
PRN-18	Interoperability	Standards are chosen for interoperability with the rails and counterparties the bank actually uses, not for preference	2	edited
PRN-24	Sustainable by Design	Energy and resource cost is assessed and recorded, including where it loses	3	repository
PRN-38	Data Residency is Decided Before Deployment	Workload and data placement is a recorded decision with a transfer basis per data category per destination, taken before deployment	2	new
PRN-39	The Business Day is Designed, Not Inherited	Cut-offs, batch windows and the end-of-day position are a designed property of the estate with a named owner, not a consequence of history	3	new

Security & Risk

Ref	Principle	Statement (condensed)	Tier	Source
PRN-19	Security is Built In	Controls are a design input proportionate to classification and criticality, and the proportionality judgement is recorded	2	edited
PRN-20	Least Privilege	Minimum access for a stated role, reviewed on a stated cycle, with the review evidenced	2	edited
PRN-21	Attributable Access	Every access to sensitive data is attributable to an individual, not to a service account or a shared credential	2	repository
PRN-22	Defence in Depth	No single control is relied on alone, and the second control is named	2	edited
PRN-23	Models are Governed	Any model influencing a customer or capital outcome is registered, explainable, monitored and owned by a person	2	edited
PRN-40	Automated Customer Decisions are Declared and Reconstructible	Every customer decision records whether it was wholly automated, on which inputs, under which model version, with a human-review path that exists	1	new
PRN-41	Screening Runs Continuously and On the Clock	Sanctions and watchlist screening runs on change to either side — list or party — within a stated clock, and the clock is monitored	2	new

Governance

Ref	Principle	Statement (condensed)	Tier	Source
PRN-42	Principles Are Applied as a Set	Higher tier prevails; at equal tier the more specific prevails; otherwise the Architecture Board decides and the decision is a precedent	1	new
PRN-43	A Gate Must Be Able To Say No	A gate that has never refused anything is not a gate; its mandate is reviewed and its refusals are counted	2	new
PRN-44	Dispensations Expire	Every departure is explicit, dated, owned by an executive, and the expiry has a stated consequence	2	new
PRN-45	A Decision the Repository Cannot See Did Not Happen	Architecture decisions are recorded where the repository points at them, with the principles the decision engaged named	2	new

`PRN-16 Responsive Change Management` — *the estate changes at the pace the business needs* — is the one row to retire. Nobody argues the opposite, nothing could fail it, and in practice it is cited only by whoever wants a gate skipped. Retire it by setting `Status` to Retired rather than deleting the row, and point anything that cited it at `PRN-15` and `PRN-43`.

The twelve that carry the weight

For each of these: why this bank needs it, what somebody has to do differently on Monday, and **the test** — the thing you ask to be shown, naming the sheet or report. Volume 1 carries the teaching behind each; this does not repeat it.

PRN-01 Primacy of Principles (Tier 1, edited). A bank's estate is substantially operated by other people: a clearing utility, two card schemes, a credit bureau hub, a cloud provider, a collections agency. A set that applies only to internal build governs a shrinking minority of it, and the Joint Standard on information-technology governance expects the board's arrangements to cover arrangements with third parties, not only the parts the bank writes itself [2]. **Monday:** every vendor agreement, outsourcing arrangement and scheme participation carries the principles as a schedule, and a counterparty that cannot meet one needs a dispensation with an expiry in the bank's own `O61 Governance Change Request`. **The test:** one contract in `O19 Contract` whose schedule carries the principles, and one third-party dispensation with a date in `Expires`. Where neither exists, the set governs internal build only, whatever it says.

PRN-03 Compliance is Designed, Not Added (Tier 1, kept). `O63 Control Objective` ships seven obligations, and four of them read `Evidence` = No mechanism or In breach [1]. That distribution is normal and it is what retrofitting looks like. **Monday:** every obligation gets a control objective and a named evidence artefact before the build starts, and `RPT09 Regulatory Coverage` is read at `GT-1` rather than at audit. **The test:** `RPT09` with no row reading No mechanism, and for each row an artefact you could put in front of a supervisor this week.

PRN-25 Every Product Carries Its Regulatory Classification (Tier 1, new). Three different classifications hang off a South African banking product and none of them is the marketing name. The credit-agreement subtype governs the fee and interest caps and the disclosure set. The return instrument type decides which line of the prudential return the balance lands on, and the return specification is now determined in writing by the Authority rather than fixed in the Regulations — Government Notice 5802 removed the forms on 1 February 2025, and a directive published 21 April 2026 sets new returns from 1 July 2026 [3][4]. And the deposit-insurance qualifying flag decides whether the balance is covered. **Monday:** `O18 Product` grows three columns and nobody opens a product without them. **The test:** take one account opened yesterday and derive all three classifications from product and party master data without asking a human. If you cannot, the return is being assembled in spreadsheets.

PRN-26 A Payment Carries Its Clearing Stream (Tier 2, new). The rulebook, the cut-off, the return window, the dispute process and — since the payment system management body's functions were split between the Reserve Bank and PayInc through 2026 — the governing body all differ by stream [5]. Registered Mandates is a different rail from DebiCheck, not a rename of it: it is ISO 20022, it runs a late collection window, and it supports business and dual-signatory accounts that DebiCheck cannot [6]. A payment model with a single "EFT" type cannot express that. **Monday:** the stream is a mandatory attribute on `E07 Payment Instruction` and every downstream rule reads it. **The test:** one query that returns last month's collections by stream, with the return window and the applicable rulebook version against each.

PRN-29 The Ledger is Immutable and Triple-Dated (Tier 1, new). An architect arriving from insurance will reach for bitemporal versioning on every entity, and in a bank that is the wrong first move. What a bank needs is an immutable entry ledger carrying effective, posting and recorded dates, versioned reference and master data around it, and immutable period-end snapshots over the top. Expected-credit-loss measurement needs the initial-recognition credit state of every exposure to survive; a home-loan agreement and its initial state have to be retrievable under evidential conditions for about three decades. **Monday:** nobody amends a posted entry, and the correction pattern is a reversal plus a new entry with its own dates. **The test:** reproduce last March's impairment number from the ledger as it stood in March, without a restatement and without a spreadsheet. **Look it up.** Volume 1, chapter 10, *Time: three dates, not two*.

PRN-31 One Reported Figure, One Derivation (Tier 2, new). The accuracy principle of the Basel committee's risk-data standard expressed as a build requirement. Most large banks can produce their monthly pack; very few can answer a question nobody has asked before within days, and that gap is why risk-data remediation sits in every large bank's portfolio under a different name. **Monday:** a figure with two derivations is a defect with a gap row, not a reconciliation exercise. **The test:** `RPT05 Data Ownership` with `Lineage` = Full on every entity that feeds a submission, and one figure traced from the return line back to the source entry in front of you.

PRN-32 The Single Customer View is a Specified Structure (Tier 1, new). The deposit-insurance single customer view is the clearest case in South African banking regulation of a regulator specifying a data structure rather than stating a principle: aggregation rules, joint-account shares, beneficiary look-through, a mandated field set and a monthly submission, with cover at R100 000 per qualifying depositor per bank, gross [7][8]. One coverage rule is a pure architecture requirement in disguise — the exclusion of the bank's own executives, directors, their related parties and lead audit partners means the bank must identify those people *in its depositor data*, as a party-to-party relationship resolvable at a reporting date. **Monday:** the view is built once, from the deposit systems, on a schedule, not assembled when asked. **The test:** produce the view as at the last month-end, with the excluded-depositor flags populated, and name the system each field came from. **Look it up.** Volume 1, chapter 15, *CoDI, and a regulation that is really a data specification*.

PRN-33 Economic Purpose is Captured, Never Inferred (Tier 1, new). The balance-of-payments category on a cross-border payment encodes why the money moved, and no system can infer it — not from the amount, the beneficiary, the country or the account. It has to be captured at the point of transacting, from the customer, which makes it a mandatory field in every channel that can originate a cross-border payment: application, branch, host-to-host file, corporate portal, programming interface [9]. **Monday:** no channel ships a cross-border capability without the field, and the field is validated against the category list rather than free text. **The test:** the rejection and warning counts on the cross-border submissions for the last two months, and the number of payments where the category was supplied by the reporting team rather than by the customer. **Look it up.** Volume 1, chapter 15, *Exchange control, and the field no system can infer*.

PRN-35 Translate at the Boundary, Never Inward (Tier 2, new). South Africa is an early ISO 20022 adopter at the high-value and new-rail end and a fixed-width batch estate at the retail end, at the same time: the high-value settlement system migrated on 17 September 2022 and the instant payment rail is native to the standard [10][11], while the retail clearing layouts remain file-based and are not

public. A canonical internal format designed around the fixed-width world cannot carry structured remittance data or structured addresses. **Monday:** the scheme dialect stops at the adapter, and no internal entity grows a field because a scheme asked for one. **The test:** name the component where each external standard is translated — ISO 20022, the card scheme dialects, the credit-bureau layout — and show one internal schema that has not changed in response to a scheme release.

PRN-36 A Mandate is a Long-Lived Object (Tier 2, new). The authenticated-collection rails make the mandate a first-class object with its own lifecycle: registered, amended, suspended, cancelled, and referenced by every collection instruction thereafter. Estates that model the mandate as a flag on the instruction end up re-establishing it, and a re-established mandate is a customer complaint and a conduct finding. **Monday:** one component holds the mandate; collections reference it and never recreate it. **The test:** the count of mandates registered more than once for the same account and creditor. It should be zero and it almost never is.

PRN-40 Automated Customer Decisions are Declared and Reconstructible (Tier 1, new). Section 71 of the privacy statute restricts decisions with legal or substantial consequences based *solely* on automated processing where profiling is involved — creditworthiness explicitly — with exceptions that require appropriate measures, including an opportunity to make representations and sufficient information about the underlying logic [12]. The credit statute arrives at the same place from the other direction: it requires reasons for a decline and the retention of the affordability assessment, its inputs and the norm-table version used. Machine-learned scorecards are not prohibited; opacity is. **Monday:** a digital lending design that promises ninety-five per cent automated decisioning has acquired a designed fifth path, with a service level and an audit trail. **The test:** take one decline from last month and render the reason, the inputs as at the decision, the model version and the human-review route. **Look it up.** Volume 1, chapter 15, *POPIA, with the banking bite*.

PRN-42 Principles Are Applied as a Set (Tier 1, new). This is the conflict rule, and without it the other forty-three are applied selectively by whoever is in the room. It is new because the repository ships no equivalent, and it is Tier 1 because a breach of it invalidates every other verdict. **Monday:** `005 Principle` has a `Tier` column, and every refusal cites a rung. **The test:** one `060 Governance Decision` row that names two conflicting principles and the rung that settled it. Where none exists, the ladder below has never been used, whatever the catalogue says.

The tier problem, and the tie-break ladder

`PRN-42` says the higher tier prevails. The repository gives it nothing to read: `005 Principle` has no `Tier` column at all, and twenty-two of its twenty-four rows have `Status` = Approved with two at Draft [1]. So as shipped, every principle is equal and the conflict rule is decorative.

The failure mode once you add the column is the opposite and just as predictable. Put nineteen of forty-four at Tier 1 and the rule resolves almost nothing, because nearly every pair is Tier 1 against Tier 1. Then the rule gets abandoned, and after that principles are applied selectively anyway.

Tier is not a measure of importance. It is a tie-break mechanism, and a tie-break mechanism with no discrimination is not one.

The proposed distribution

Band	Count	Share	What qualifies
Tier 1 — the apex	8	18%	Breach is a licence matter, a legal breach, or the loss of reproducibility on a regulated number
Tier 2 — the working set	25	57%	The body of the practice: ownership, lineage, interfaces, mandates, resilience, security, governance records
Tier 3 — yields	11	25%	Principles that should predictably lose, and say so

The eight at Tier 1: `PRN-01` (the set applies to work you do not control), `PRN-03` (obligations are requirements), `PRN-25` (product classification), `PRN-29` (the ledger), `PRN-32` (the specified customer view), `PRN-33` (captured purpose), `PRN-40` (automated decisions), `PRN-42` (the conflict rule itself).

The admission test for Tier 1: would you stop a release for it? Not "is it important" — everything in the catalogue is important, which is exactly why importance cannot be the criterion. Eight is small enough that an executive can recall the band without the sheet, which is the property that makes rung one work without a meeting.

Tier 3 is the band people get wrong. It is not a bin for weak statements. It is the explicit band of principles that should lose, and that say what they lose to: sourcing preference to a scheme date, technology independence to a vendor's only viable option, accessibility to classification, sharing to purpose limitation, one-component-per-rail to a mandated migration window, the designed business day to a clearing-house cut-off nobody can move. Saying so in the tier is what makes them usable, because a principle overridden quietly stops being consulted at all.

The ladder

Rung	Rule	What it resolves	How it is recorded
1	The higher tier prevails	Tier 1 against anything; Tier 2 against Tier 3	Mechanical. No judgement, no meeting
2	At equal tier, the more specific prevails	Most Tier 2 against Tier 2 conflicts	The architect applies it; the `O60 Governance Decision` row names the rung
3	Otherwise the Architecture Board decides, and the decision is a precedent	Genuine ties at equal specificity	`O60 Governance Decision` with both principles named, cited next time rather than re-argued

Rung two does the heaviest lifting, because general-against-specific is the common shape of a real conflict: a principle about the payment ledger beats a principle about data generally; a principle about this rail beats a principle about integration.

> **TIER SETTLES ABOUT A THIRD OF THE CONFLICTS YOU WILL ACTUALLY MEET**
>
> Be honest about this with the Board, because somebody will test the arithmetic. Forty-four live principles make 946 possible pairs. Of those, 563 cross a tier boundary and rung one settles them mechanically, which sounds decisive. It is not, because conflicts do not arrive evenly: the working body of the practice is Tier 2, and 300 of the 383 same-tier pairs are Tier 2 against Tier 2. Those are the arguments you have in real life — ownership against delivery date, security against usability, lineage against cost, the designed business day against the clearing cut-off — and rung one is silent on every one of them. Count the conflicts you meet rather than the pairs you could draw, and tier settles roughly a third. That is a property of any set this size, and it is why rungs two and three are written down rather than improvised.

Re-tiering happens by recorded decision, never by preference. `PRN-06 Technology Independence` is the obvious candidate at most banks: it arrives at Tier 1 on inherited authority, and it belongs at Tier 3 the moment a treasury system or a card platform has one viable supplier in this market. Move it with an `O60 Governance Decision` row and the Board's reasoning, and it becomes a principle that does work — it now loses, predictably, to something named.

Reviewing the distribution is annual work, run alongside the maturity assessment in appendix H. Three checks. Can an executive name the Tier 1 principles without the sheet? Has any Tier 3 principle won a conflict — if so it is mis-tiered. Has any Tier 1 principle never been cited — if so it is either not a principle or nobody is consulting the set.

Settle Tier 1 **before** the workshop, with the Chief Risk Officer and the Chief Compliance Officer. That band is where the regulatory argument sits, and it is the only band that should not be decided by a show of hands.

The standards catalogue

Thirty-one standards, shaped for `032 Standard` — `Ref`, `Standard`, `Category`, `Status`, `Conformance` [1]. Sixteen come from the repository and fifteen are new; eleven are specific to banking in South Africa and are marked. `Category` takes the six values the sheet already uses: Integration, Data, Security, Platform, Delivery, Governance. The `Status` and `Conformance` vocabularies are set out after the tables, and both matter more than the catalogue itself.

Integration

Ref	Standard	Status	Conformance	SA bank	Source
STD-01	API design standard	Current	Partial	—	edited
STD-02	Event schema standard	Deprecated	Partial	—	repository
STD-17	ISO 20022 message and data standard	Current	Not assessed	yes	new
STD-18	ISO 8583 card message standard, named by scheme and edition	Current	Not assessed	yes	new
STD-21	PayInc file and mandate specifications, by stream	Current	Not assessed	yes	new
STD-26	Interface management and routing standard	Emerging	Breach	—	new

`STD-17` is the one standard that genuinely unifies, and the one with hard external dates. It is not a message format: it is a methodology plus a repository of business models from which messages are generated, so adopting it is a data-model decision wearing a messaging decision's clothes. The domestic high-value settlement system migrated on 17 September 2022, the first real-time gross settlement system in Africa to do so, and the instant payment rail is native to it [10][11]. Conformance here means the canonical internal model can carry structured remittance and structured address data, not that an adapter exists.

`STD-18` must name the scheme and the edition or it specifies nothing. The card message standard is currently in its 2023 edition, and its own reference literature says it is not typically used directly: the schemes base their authorisation messaging on it and add network-specific fields and usages, and implementations do not interpret message-type meanings identically [13]. So there is a Visa interface, a Mastercard interface, a domestic switch interface, and three mapping layers. A design document that writes "ISO 8583" without the scheme and the version has specified nothing.

`STD-21` carries an honest gap and it stays there. The clearing utility's file submission times and layouts sit in product and service manuals supplied to participants and in a fee book available under

non-disclosure; they are not public [14]. PayInc is the renamed BankservAfrica, now half-owned by the Reserve Bank, and the electronic funds transfer, authenticated collection, instant payment, registered mandate and real-time clearing houses moved to it on 2 September 2026 [5][15]. Obtain the specifications under agreement and put them on the critical path. Do not let a programme plan assume a layout version nobody has published.

Data

Ref	Standard	Status	Conformance	SA bank	Source
STD-03	Data classification standard	Emerging	Conformant	—	repository
STD-04	Data retention standard	Current	Breach	yes	edited
STD-10	Database standard	Emerging	Not assessed	—	repository
STD-14	Naming convention	Deprecated	Breach	—	repository
STD-22	Credit-bureau submission layout standard	Current	Not assessed	yes	new
STD-23	Prudential return specification and submission standard	Current	Breach	yes	new
STD-24	Cross-border category capture standard	Current	Breach	yes	new
STD-25	Effective dating and triple-dated history standard	Emerging	Not assessed	yes	new
STD-31	Payment reference data standard	Emerging	Not assessed	yes	new

`STD-04` is not one period. A bank's retention schedule is the maximum of several statutory clocks running on the same record, and the longest is longer than most architects expect — the signed home-loan agreement, its security documents, the statutory default notice and its proof of delivery, and the initial credit state of the exposure all have to be retrievable under evidential conditions about thirty years later. The repository ships this standard as `Current` with `Conformance` = Breach, which is probably the most honest cell in the file.

`STD-22` governs the monthly submission to the credit-reporting hub, which describes itself as the sole legally recognised central data transmission hub and routes member submissions to the bureaux authorised to hold payment profile information [16]. For a South African bank these layouts are a mandatory interchange standard with a monthly deadline — more operationally binding than anything a global reference model publishes, and absent from every one of them. The current layout version is members-only; say so rather than guessing it.

`STD-23` changed shape in 2025 and the change is the point. Government Notice 5802, dated 31 January 2025 and effective 1 February 2025, removed the return forms from the Regulations; returns are now determined by the Authority in writing, and twenty-five forms were deleted from the regulation text [3]. A proposed directive published 21 April 2026 sets returns to be submitted from 1 July 2026 [4]. So the source-to-return mapping has to be metadata a business analyst can change, the return has to be a versioned entity so a figure submitted under the old form stays reproducible, and the data layer has to sit finer than any current form. An estate built to the shape of today's returns is

an estate that gets rebuilt. This is **not** the Basel post-crisis reform instrument, which took effect 1 July 2025 through separate amended Regulations [17] — commentary that merges the two will tell you the capital rules changed in February 2025, and they did not.

`STD-24` is the capture standard behind `PRN-33`: the category list, the validation, and which channels must present it. The submission clock is two business days from the value date, rejections and warnings have their own one-business-day clock, and the reporter must not pre-convert currency because the central bank applies the mid-rate [9].

Security and platform

Ref	Standard	Status	Conformance	SA bank	Source
STD-05	Identity and access standard	Current	Breach	—	repository
STD-06	Encryption standard	Current	Breach	—	edited
STD-07	Logging and monitoring standard	Current	Not assessed	—	repository
STD-08	Cloud landing zone standard	Current	Not assessed	—	edited
STD-09	Container image standard	Current	Partial	—	repository
STD-11	Network segmentation standard	Emerging	Partial	—	repository
STD-16	Supplier security standard	Current	Conformant	—	repository
STD-20	Card data security standard	Current	Partial	yes	new
STD-27	Key management standard	Emerging	Not assessed	—	new
STD-28	Data residency and transfer basis standard	Emerging	Breach	yes	new
STD-29	Resilience tiering and test standard	Emerging	Breach	—	new

`STD-20` is a hard dated constraint rather than a design preference. The card data security standard's version 4 future-dated requirements became effective 31 March 2025, among them requirements addressing script integrity on payment pages [18]. It also settles a data-model question: the card primary account number may be stored only rendered unreadable, and sensitive authentication data — full track data, the verification values, the encrypted personal identification block — must not be stored after authorisation at all, even encrypted. Entity-level classification cannot express that, which is why `PRN-14` has to reach the attribute.

`STD-28` is new and it is the one most likely to be discovered late. Cross-border transfer of personal information requires one of the listed bases — a law, binding corporate rules or a binding agreement providing adequate protection; the data subject's consent; necessity for a contract with or in their interest; or their benefit where consent is not practicable — so cloud and offshore processing need a mapped transfer basis per data category per destination, and the two authorities issued a joint communication on cloud computing and the offshoring of data in July 2025 [19][20]. Retrofitting residency is a migration, not a configuration change.

`STD-29` exists because `049 Resilience Tier` ships eleven of twelve rows at Never tested and the operational-resilience obligation in `063` is one of the four reading No mechanism or In breach [1]. The standard is what makes the tier mean something: a cadence per tier, a test scope per tier, and a gap row for anything a test fails.

Governance and delivery

Ref	Standard	Status	Conformance	SA bank	Source
STD-12	Coding and review standard	Deprecated	Not assessed	—	repository
STD-13	Environment standard	Emerging	Conformant	—	repository
STD-15	Records management standard	Current	Partial	—	repository
STD-19	Card scheme mandate standard	Current	Not assessed	yes	new
STD-30	Model documentation standard	Emerging	Not assessed	—	new
STD-32	Exit and portability standard	Emerging	Not assessed	—	new

`STD-19` is the standard nobody writes down and everybody obeys. The card schemes issue mandates on a release calendar, with compliance dates, and the rules go to participants rather than to the public. A bank that does not hold the mandate calendar as architecture input discovers it as an unplanned project twice a year. The standard says where the calendar lives, who reads it, and which gate it is presented at.

The status vocabulary

A standard has a `Status`. A component has a position against it. The assessment between the two is a conformance verdict, and the two vocabularies get confused constantly [1].

Value	Used on	What it obliges	What it is not
Current	`032 Status`	Refuse a non-conforming new design; raise a gap against existing non-conformance	Not a claim the estate conforms. A long non-conformant list means enforcement, not re-drafting
Emerging	`032 Status`	Refuse non-conforming new work; leave existing components until they change anyway	Not a draft and not optional. One you will not enforce on new work should be withdrawn
Deprecated	`032 Status`	Stop citing it at gates; point references at the replacement; keep the row	Not deleted, and not harmless — the repository ships three Deprecated standards, two of them with live conformance verdicts against them

The conformance vocabulary

Verdict	Recorded on	What it obliges	What it is not
Conformant	`R18 Standard applies to Applica Conformance`	Nothing, until the next assessment or a change to either side	Not permanent. An undated verdict is an assertion
Partial	`R18 Conformance`	A note naming exactly what is unmet, and a dated remediation	Not a pass. Partial at `GT-1` is a refusal unless a dispensation carries an expiry
Breach	`R18 Conformance`	A dated containment with an executive owner, or an `029 Gap` row with a severity. One of the two	Not automatically a defect. Some non-conformance is the right economic answer; undecided never is
Not assessed	`R18` row absent, component in scope	Count it. The position is meaningless without the denominator	Not conformance. An unassessed component is not conforming by default

> **THE DENOMINATOR RULE**
>
> Report conformance as assessed-and-conformant over **in scope**, never over assessed. A catalogue that assesses only what it expects to pass produces a rising conformance rate and a falling estate quality, and `RPT12 Standards Conformance` cannot tell the two apart — five of the sixteen shipped standards read `Not assessed`, so the headline rate is computed over two-thirds of the catalogue. The same arithmetic applies to review: a review either reaffirms, edits or retires, and cannot defer. A deferred review is how Emerging becomes permanent.

The adoption workshop

Ninety minutes, six people, one change request out the other end. Appendix C holds the facilitation craft — keeping the most senior voice from settling a question of fact, time-boxing an argument, what to send as a pre-read — and this does not repeat it. What follows is the running order and the recording discipline specific to a principles pass.

In the room. The Chief Enterprise Architect chairs and does not advocate: a chair arguing for a principle guarantees consent rather than adoption. The Chief Information Officer settles whether the sourcing order, the interface rule and the one-component-per-rail principle survive contact with the delivery portfolio. The Chief Data Officer settles whether one named owner per entity is a decision this bank will take, and who takes it. The Chief Risk Officer settles the Tier 1 band and the resilience and model principles. The Chief Compliance Officer settles whether the conduct, privacy and reporting principles carry the obligations, and is the person most likely to add one in the room. The Chief Financial Officer settles `PRN-29` and `PRN-31`, because the ledger and the derivation rule land on the finance function and nowhere else.

Not in the room. Solution architects, project managers and vendors. They are affected and they are not the owners. Bring their objections as a written, attributed pre-read. A principles workshop with twenty people in it produces a vocabulary, not a decision.

Segment	Minutes	What happens
Frame it	10	Two slides: what a principle is for, and the three-rung ladder. Then the session rule — silence is adoption, anyone may reject anything, and a rejection must name the decision the principle would have got wrong
The eight Tier 1 principles	20	One at a time, two minutes each. No re-tiering here; re-tiering is a recorded decision, not a show of hands
Everything else, by category	35	Reference and statement read aloud, nothing else. Objections get a two-minute box, then edit, reject or defer. Six categories, six minutes each
Bind Tier 1 to requirements	15	One `006 Requirement` row per Tier 1 principle. Where none exists, the requirement is missing, not the link
Close	10	Read back the counts. Name the change request, the date `Status` moves off Draft, and who re-points anything that cited `PRN-16`

Step 1 — Pre-mark the whole set and settle Tier 1 before the room sits

Produce. A marked copy of the forty-five rows with Adopt, Edit, Reject or Defer against each, circulated as a pre-read — plus an agreed Tier 1 band of eight.

You need. One day of your own reading. Ninety minutes each with the Chief Risk Officer and the Chief Compliance Officer. The twenty-four shipped rows of `005 Principle` open beside this appendix.

> **005 Principle**
>
> Nothing is written to the sheet at this step. Work on Form PS-1 and keep the `Tier` and `Source` columns on paper until the Board has signed.

How.

1. Read the twenty-four shipped statements and mark each one against the three-question test. Most will fail question one.
2. Mark the twenty-one new rows the same way, and strike any you would not defend yourself.
3. Settle the eight Tier 1 rows with the two risk executives, separately, before the workshop.
4. Circulate the marked set at least three working days ahead, with the ladder on one page.

Signed by nobody yet. This step produces a position, not a decision.

Done when every one of the forty-five rows carries a mark and the Tier 1 band has eight members, agreed by two executives who were not in a group setting when they agreed it.

Shows up in nothing yet. That is deliberate: a principle in the repository before it is adopted is a principle the Board will read as already decided.

Goes wrong when you walk in without the Tier 1 band settled. The room will then spend forty minutes on tier and never reach the other thirty-six rows.

Step 2 — Run the ninety minutes to the running order above

Produce. Forty-five decisions: Adopt, Edit, Reject or Defer, each on Form PS-1, with every rejection naming the decision the principle would have got wrong.

You need. The six people named above. Form PS-1, five copies. A floor: one rejection or edit per category, or the session does not close.

> **005 Principle**
>
> Captured on paper. `Ref`, `Principle`, `Category`, `Owner`, `Status` from the first table of Form PS-1; re-worded `Statement` from the second.

How.

1. State the session rule in the first two minutes: silence is adoption, and a rejection must name a decision.
2. Take the eight Tier 1 rows first, two minutes each, with no re-tiering permitted.
3. Take the other categories at six minutes each, reading the reference and the statement and nothing else.

4. Where the room will not settle, mark Defer with a named owner and a date. Defer is a legitimate outcome; a vague adoption is not.
5. Read the counts back before anybody leaves.

Signed by `GB-01 Architecture Board` — which per `057 Governance Body` holds `GT-1` and `GT-2` and approves dispensations with an expiry, so it is the body whose refusals the set has to support.

Done when every row has a decision, the counts have been read aloud, and the change request reference is written on every copy of the form.

Shows up in `005 Principle` once transcribed, with `Status` moving from Draft to Approved only for the rows the Board adopted.

Goes wrong when the chair advocates. You get forty-five adoptions, no edits, and a catalogue nobody will cite.

Step 3 — Bind every Tier 1 principle to a requirement, in the room

Produce. Eight bindings — one `006 Requirement` reference per Tier 1 principle — and a list of the ones where no requirement exists.

You need. The thirty-two shipped rows of `006 Requirement` printed, and `RPT06 Requirements Traceability` open.

> **`006 Requirement`**
>
> No new column here. The binding itself has nowhere to live — see the next step — so record it against the requirement on Form R-1 of appendix F and in the session minute.

How.

1. Read each Tier 1 principle and ask which requirement it mandates.
2. Where a requirement exists, write its reference against the principle.
3. Where none exists, write the requirement there and then on Form R-1; the principle has just found a hole.
4. Count the Tier 1 principles with no requirement. That number is the honest headline of the session.

Signed by `GB-01 Architecture Board`, accepting the new requirements as a set.

Done when all eight Tier 1 principles either carry a requirement reference or carry a new requirement drafted in the room.

Shows up in `RPT06 Requirements Traceability`, where the new requirements will read `Reaches a block` = No until Phase E. That is correct and it is the next piece of work.

Goes wrong when the binding is left for afterwards. It never happens afterwards, because the people who could say which requirement have gone back to their day.

Step 4 — Add the sheet that lets a principle govern something

Produce. One relationship sheet binding a principle to a requirement and to a gap, because the repository has neither.

You need. Agreement from whoever owns the file's structure, and the naming decision from the Preliminary chapter of this volume.

> **`A sheet that does not exist yet`**
>
> Twenty-five relationship sheets ship with this file and not one has a principle at either end. `R08 Requirement derives from Dr` binds a requirement to a driver; `R09 Requirement satisfied by Bu` binds it to a building block. Nothing binds either to `O05 Principle`. Add one sheet with five columns: the principle reference, its name, the target reference, the target name, and the nature of the link — Mandates, Supports or Violated.

How.

1. Add the sheet, with the two name columns as lookups so a renamed principle does not orphan a row.
2. Load the eight bindings from step 3 as Mandates.
3. Load every gap raised on Form E-1 against the principles it breaches, as Violated.
4. Keep the row count visible: a principle with no rows is the finding, and there is no report to find it for you.

Signed by `GB-01 Architecture Board`, once, for the structure.

Done when every Tier 1 principle has at least one Mandates row and every Critical gap names the principles it breaches.

Shows up in nothing, until you build the report. Build it: a count per principle of Mandates rows, Violated rows, and a verdict of Applied, Violated or Not yet applied.

Goes wrong when the binding is recorded in the `Statement` column as prose. Prose does not count, and a count is the only thing that catches a decorative principle.

Step 5 — Run the standards pass, and assess before you adopt

Produce. Thirty-one standards on `O32 Standard` with a `Status`, and an `R18 Standard applies to Applica` row for every standard-and-application pair in scope.

You need. Form PS-3. The fourteen rows of `O23 Application Component`. Somebody from the integration team who will look at the platform rather than at the design document.

> **`O32 Standard`**
>
> `Ref`, `Standard`, `Category`, `Status`, `Conformance`. Fill `R18` first and let `O32 Conformance` follow from it, or the headline will contradict the detail.

How.

1. Decide `Status` per standard: Current, Emerging or Deprecated. Withdraw anything you will not enforce on new work.
2. For each standard, list the applications in scope — the denominator — before assessing any of them.
3. Assess each pair and record Conformant, Partial, Breach or Not assessed on `R18`.
4. For every Breach, record either a dated containment with an executive owner or an `O29 Gap` row. One of the two, never neither.
5. Set a review date per standard outside the repository, because `O32` has no review column, and put the dates in the `GB-01` standing agenda.

Signed by `GB-01 Architecture Board`; `STD-20`, `STD-28` and `STD-29` also need the Chief Risk Officer.

Done when every in-scope pair has a verdict and no Breach is undecided.

Shows up in `RPT12 Standards Conformance`, which prints the catalogue with `Status` and `Conformance` side by side. Read it with the denominator in mind: the five `Not assessed` rows are not passes.

Goes wrong when the catalogue is adopted and conformance is never assessed, so `R18` stays at sixteen rows and `RPT12` reports on a third of the estate.

Making the principles do work

A principle with nothing bound to it is decoration. In this repository every principle is in that position on the day you open the file, because there is no sheet that could bind one — which means the catalogue is decoration by default and step 4 above is not optional.

Once the sheet exists, read it in two minutes, in this order. **Principles with a Violated row** — that is the work, and it should be owned; a violation with a gap record behind it is the healthy shape, because the principle detected a breach and the breach has a reference, a severity and a work package. **Principles with nothing at all** — zero mandated, zero violated; that is the catalogue's own defect, and it reads as the harmless verdict, which is why it survives. **Tier 1 principles violated** — that is the board conversation, because by this appendix's own admission test a violated Tier 1 principle is a licence matter, a legal breach or a loss of reproducibility on a regulated number.

The worst combination is a violation with nothing mandated: broken, and mandating nothing, so it has no mechanism to stop the next breach. The worst single row is a violation against `PRN-01`, the principle that makes the set apply at all — it means the rule making the catalogue binding is itself

being breached, which invalidates every other verdict. Where that appears, stop reading and go and find out whether any gate has ever refused anything.

A WORKED BINDING, END TO END

`PRN-29`, Tier 1 — the immutable triple-dated ledger. Bind it to the requirement that the general ledger carry effective, posting and recorded dates on every entry, as Mandates. Then bind it to two gaps from Form E-1: `E12 General Ledger Entry` has `Lineage` = Partial on `RPT05 Data Ownership`, and `O23` shows the ledger reading from `A09 Credit Risk Engine` as a direct database read on `RPT04 Interface Catalogue`. Both are Violated rows.

Read that as the principle working. Two different gaps engaging one principle means it is describing a real structural property of the estate rather than a coincidence of wording. What it adds to gaps already marked Critical is the argument for sequencing them first, and the reason `GT-2` cannot pass on the impairment work while the ledger cannot reproduce a past position. Resist softening the statement so the count looks better.

The forms

Three forms and two question sets. The forms transcribe into `005 Principle`, `032 Standard` and the change-request sheet appendix F says you must create.

BEFORE THE WORKSHOP — ANSWER THESE ON PAPER

The repository ships twenty-four principles with no tier and no test. Which three would you reject outright, and for each, name the decision it would have got wrong.

Which principle does this bank break routinely, and who benefits from it being broken?

For each of the eight proposed Tier 1 principles: would you stop a release for it? Name the ones where the answer is no.

Who, by name, owns `PRN-25` product classification and `PRN-33` captured purpose — and have they agreed to own it?

`PRN-16` is recommended for retirement. Who will object, and what will they say instead?

Form PS-1 — Principle decision sheet

Feeds `005 Principle` — `Ref`, `Principle`, `Category`, `Owner`, `Status`. **The decision lives in** `Status`: Approved, Draft, or Retired once you add the value. Record Adopt, Edit, Reject or Defer in the `Decision` column below, which is working information and not a repository column — do not add one. **`Tier` is a column you must add** to `005` before this form is worth transcribing; it is on the form so the room can see it. **Typed into Excel** `Statement` for any row the room re-words — use the second table and write it out in full. **First pass** forty-five principles, five copies. **Signed by** `GB-01 Architecture Board`. A principle nobody has signed is a preference.

Ref	Principle	Category	Tier	Decision	Status
PRN-01	*example — delete*	Business	1	Adopt	Approved

Re-worded statements, and rejections with their reason. If a statement needs two sentences it is probably two principles.

Ref	Statement, or the decision this principle would have got wrong
PRN-10	*example — delete*

Form PS-2 — A new principle the bank adds

Feeds `005 Principle`, one row — but not until the rationale, the implications and the test are written. **No new principle is accepted in the adoption session.** This form captures the proposal, and the proposal goes to the next `GB-01 Architecture Board` sitting. **First pass** two or three; if the room proposes eight, the set was not the problem. **Signed by** `GB-01 Architecture Board`, after the owner named below has written the three paragraphs.

Field	Fill
`Ref`	
`Principle`	
`Category` — one of the six, or say which you are adding	
`Tier` — and would you stop a release for it?	
`Owner` — a named executive role, not a committee	
Proposed by, and date	

`Statement` — one sentence of consequence, written as it will appear in the sheet.

Why this bank needs it, and who argues the opposite.

What somebody has to do differently on Monday.

The test — what you would ask to be shown, naming the sheet or the report.

Which existing principle does it overlap, and which of the two yields?

Form PS-3 — Standards decision sheet

Feeds `032 Standard` — `Ref`, `Standard`, `Category`, `Status`, `Conformance`. **Values**: `Category` Integration, Data, Security, Platform, Delivery, Governance; `Status` Current, Emerging, Deprecated; `Conformance` Conformant, Partial, Breach, Not assessed. **Fill** `R18 Standard applies to Applica` **first** on Form D-2 of appendix F and let the `Conformance` column here follow from it. `032` **has no review date column**, so the review cycle lives in the `GB-01` standing agenda and the date goes on the second table below. **First pass** thirty-one standards, three copies. **Signed by** `GB-01 Architecture Board`; the card data, residency and resilience standards also need the Chief Risk Officer.

Ref	Standard	Category	Status	Conformance	Decision
STD-17	*example — delete*	Integration	Current	Not assessed	Adopt

Containment and review. One row per standard where existing non-conformance is permitted for a period. **No row without a date**, because Emerging with no review and Breach with no exit are the two commonest dishonesties in a catalogue.

Ref	What is non-conformant	Exit date	Executive owner	Review due
STD-05	*example — delete*			

SIX MONTHS ON — IS THE SET DOING WORK?

How many principles have nothing bound to them? Which ones, and what will you bind each to?

Name one design refused on a principle in the last six months, and the `060 Governance Decision` row that records it.

Has any Tier 3 principle won a conflict? If so it is mis-tiered — which way, and who signs the re-tier?

Can your Chief Information Officer name the eight Tier 1 principles without the sheet? Ask them, and write what they say.

How many rows in `061 Governance Change Request` name the principle they depart from, and how many have passed `Expires` with nothing happening?

Which standards read `Not assessed` on `RPT12 Standards Conformance`, and who has been citing them at `GT-1` anyway?

One thing this set does not give you

It does not tell you what your principles should be. It tells you what forty-four defensible ones look like, where each would be contested at a South African bank, and how to run the argument. A set adopted without an argument is a set that will lose its first conflict and never be opened again.

And it does not ship as a spreadsheet. The insurance volume in this series has a companion principles-and-standards workbook; the banking set does not, and inventing one here would be worse

than saying so. Everything above pastes into `O05 Principle` and `O32 Standard` by hand, which at forty-five and thirty-one rows is an afternoon.

THE CHAPTER IN TEN LINES

- Arguing with somebody else's set is cheaper than arguing with a blank page.
- A principle is real when it can be broken, the evidence exists, and something happens when it is.
- The repository ships 24 principles: 8 kept, 15 edited, 1 retired, 21 added.
- `O05 Principle` has no `Tier` column, so the conflict rule has nothing to read — add it first.
- Eight principles at Tier 1, and the admission test is whether you would stop a release.
- Tier 3 is the band of principles that should lose, and that name what they lose to.
- Thirty-one standards, eleven of them specific to banking in South Africa.
- Report conformance over what is in scope, never over what you assessed.
- Twenty-five relationship sheets and not one has a principle at either end — add the sheet that binds one.
- A principle with nothing bound to it is decoration, and here that is the default.

SOURCES FOR THIS CHAPTER

- **1.** Banking_TOGAF_Metamodel.xlsx — sheets O05 Principle, O32 Standard, O49 Resilience Tier, O63 Control Objective, R18, RPT12 and ZZ Checks, as read from the workbook
- **2.** Joint Standard 1 of 2023 — IT governance and risk management (commenced 15 November 2024) https://lawlibrary.org.za/akn/za/act/standard/sarb/2023/1/eng@2024-11-15
- **3.** Government Notice 5802, Government Gazette 52013 — amendments to the Regulations relating to Banks (31 January 2025, effective 1 February 2025) https://compliancelibrary.co.za/wp/wp-content/uploads/2025/02/Banks-Act-Amendments-to-Regulations.pdf
- **4.** Prudential Authority, Proposed Directive — returns to be submitted from 1 July 2026 (published 21 April 2026) https://www.resbank.co.za/en/home/publications/publication-detail-pages/prudential-authority/pa-documents-issued-for-comment
- **5.** South African Reserve Bank, payment system management body transition https://www.resbank.co.za/en/home/what-we-do/payments-and-settlements/psmb
- **6.** Precium, "RMS to Registered Mandates: beyond just a technical upgrade" https://www.precium.com/blog/rms-to-registered-mandates-beyond-just-a-technical-upgrade
- **7.** CoDI Coverage and Reporting Rules (R100 000 per qualifying depositor; excluded depositors) https://www.resbank.co.za/content/dam/sarb/what-we-do/financial-stability/resolution-planning/CoDI-coverage-and-reporting-rules.pdf
- **8.** CoDI, single customer view calculations document v0.2 https://www.resbank.co.za/content/dam/sarb/what-we-do/deposit-insurance/SCV%20calculations%20document.pdf
- **9.** SARB Financial Surveillance, FinSurv Reporting System business and technical specifications https://www.resbank.co.za/content/dam/sarb/what-we-do/financial-surveillance/authorised-dealers/FinSurv-Reporting-System.pdf
- **10.** SWIFT, "South Africa market infrastructure SAMOS goes live on ISO 20022" https://www.swift.com/news-events/news/south-africa-market-infrastructure-samos-goes-live-iso-20022
- **11.** Stitch, "Real-time payments in South Africa: the state of PayShap in 2026" https://stitch.money/blog/real-time-payments-in-south-africa-the-state-of-payshap-in-2026
- **12.** POPIA section 71 — automated decision making https://popia.co.za/section-71-automated-decision-making/
- **13.** ISO 8583 overview, including the note that the standard is not typically used directly by systems or networks https://en.wikipedia.org/wiki/ISO_8583
- **14.** BankservAfrica, "Key Rules and Procedures" (file submission times in participant manuals; fee book under non-disclosure) https://www.bankservafrica.com/website/assets/key-rule-and-procedures.pdf
- **15.** ITWeb, "SARB takes 50% PayInc stake to drive digital payments" (11 November 2025) https://www.itweb.co.za/article/sarb-takes-50-payinc-stake-to-drive-digital-payments/WnpNgM219EP7bQQL
- **16.** SACRRA — Central Data Transmission Hub and the bureaux authorised to hold payment profile information https://sacrra.org.za/
- **17.** Prudential Authority Annual Report 2025/26, policy and regulation chapter (Basel III post-crisis reforms effective 1 July 2025) https://www.resbank.co.za/content/dam/sarb/publications/reports/pa-annual-reports/2026/chapters
- **18.** PCI Security Standards Council, guidance on PCI DSS e-commerce script requirements effective 31 March 2025 https://blog.pcisecuritystandards.org/coffee-with-the-council-podcast-guidance-for-pci-dss-e-commerce-requirements
- **19.** POPIA section 72 — transborder information flows https://popia.co.za/section-72-transfers-of-personal-information-outside-republic/
- **20.** Prudential Authority and FSCA, Joint Communication 2 of 2025 on cloud computing and the offshoring of data (July 2025) https://www.resbank.co.za/en/home/publications/publication-detail-pages/prudential-authority/pa-public-awareness/joint-communications

APPENDIX H

Running the maturity assessment

Twenty-six dimensions, two scores that stay apart, and the rule that makes both defensible: no score without the evidence that supports it.

A maturity assessment is a way of finding out where to look. It is not a score for its own sake and it is not a report card for the architecture function, although it will be read as one.

Run it three times. **At the start**, in your first ninety days, to find out which of the twenty-six dimensions below are so weak that the first cycle of the ADM has to address them before anything else. **Before a funding round**, because a sponsor who will not fund a capability map will fund a dimension scoring 1 with a named consequence. **Annually thereafter**, alongside the principle review in appendix G, to show movement — and to show where there has been none.

Do not run it when somebody has asked for a benchmark. There is no defensible banking benchmark for most of this, and a score of 2.4 against an invented industry average of 3.1 is a number that will be quoted at you for two years. Score your own bank against observable descriptors and let the movement be the comparison.

Ardent Union Bank, the bank in the repository this volume works, is invented. Where a shipped number is quoted below it comes from that file [1].

> **THE ASSESSMENT TELLS YOU WHERE TO LOOK. IT DOES NOT TELL YOU WHAT IS THERE**
>
> A dimension scoring 2 is an instruction to go and investigate, not a finding in itself. The finding is what you discover when you look, and only that goes into `029 Gap`. An assessment that produces twenty-six gap rows has produced none, because nobody can act on twenty-six.

Two assessments, kept separate

There are two different questions and conflating them is the commonest way this exercise gets discredited.

EA practice maturity asks: is the architecture function any good? Does it have a mandate, a repository anybody trusts, gates that have refused something, traceability from obligation to building block? Eleven dimensions. The answers are about the practice and the architect is the subject.

Banking architecture health asks: is this bank's architecture any good? Can it change a product in weeks, master a party once, reproduce an impairment number, produce a prudential return without a reconciliation spreadsheet, settle on every rail it participates in? Fifteen dimensions. The answers are about the estate and the bank is the subject.

Keep them in separate tables, score them separately, and present them separately. A practice can be excellent inside a terrible estate — that is a well-run architecture function two years into a ten-year

remediation — and a bank can have a healthy estate with no architecture practice at all, which happens when one very good engineer has held the line for a decade and is about to retire. Averaging the two produces a middling number that describes neither and funds nothing.

> **WHERE THIS SITS AGAINST THE REPOSITORY'S OWN NUMBERS**
>
> `013 Business Capability` already carries `Current` and `Target` on every row, and `RPT03 Capability Heatmap` prints them for the twelve level 1 capabilities [1]. That is **capability** maturity: how good the bank is at doing a thing. The banking health dimensions below are architecture health: whether the estate underneath those capabilities can be changed, reproduced and evidenced. They are related and they are not the same, and where a health dimension corresponds to a level 1 capability the two scores should be close. Where they are three apart, one of them is wrong and the gap between them is the most interesting thing in the assessment.

The scoring rule

Five levels, scored per dimension, whole numbers only. No halves. A 2.5 is a disagreement you have hidden; record both scores and say who gave each.

Level	Name	What you observe
1	Absent	Nothing exists, or what exists is one person's private knowledge. No artefact you could be shown
2	Ad hoc	It exists where somebody needed it. Inconsistent, undated, and not maintained between the projects that created it
3	Defined	It exists, it is written down, it is owned by a named person, and it is used in the work it was written for
4	Managed	It is used routinely, it is measured, and the measure is reported to somebody who acts on it
5	Optimising	It is measured, the measure drives change, and the mechanism itself is reviewed and improved on a cycle

Level 3 is the honest target for most dimensions in a first cycle. Level 4 requires a measure and a reader, and level 5 requires the measure to have changed something — which in architecture usually takes two annual cycles to demonstrate.

NO SCORE IS RECORDED WITHOUT THE EVIDENCE THAT SUPPORTS IT

Every score carries one of three kinds of evidence, named on the score sheet. **A repository sheet or report** — `RPT06 Requirements Traceability`, `ZZ Checks` line 6, `RPT05 Data Ownership` — cited by sheet and, where it matters, by row. **A document** — a signed minute with a date, a test report, a gate pack, a return submission. **A person who showed you** — named, with the date, and what they put on the screen.

"The team says it is about a 3" is not evidence. Neither is a slide. Neither is the architect's own opinion, however well founded, because the whole point of the exercise is to produce something somebody else can check.

Where there is no evidence, the score is 1 or 2 and the finding is the absence. That rule is what makes the assessment survive contact with an executive who does not like a number.

(a) The EA practice dimensions

Eleven dimensions, weighted to 100. The weights are a starting position: move them if your bank's situation demands it, but move them **before** you score, write down why, and keep them stable between runs or the trend means nothing.

Ref	Dimension	Weight	Where the evidence comes from
EA-01	Mandate and sponsorship	10	`O57 Governance Body`, `O60 Governance Decision`, the signed Statement of Architecture Work
EA-02	Governance that can refuse	10	`O58 Governance Gate`, `O59 Compliance Assessment`, `O61 Governance Change Request`
EA-03	Repository currency and integrity	10	`ZZ Checks`, `RPT01 Dashboard`, `RPT13 Orphan Analysis`
EA-04	Principles and standards doing work	8	`O05 Principle`, `O32 Standard`, `RPT12 Standards Conformance`
EA-05	Requirements traceability	10	`RPT06 Requirements Traceability`, `R08`, `R09 Requirement satisfied by Bu`
EA-06	Gap and roadmap discipline	10	`RPT07 Gap Analysis`, `RPT08 Roadmap`, `R12 Gap closed by Work Package`
EA-07	Capability model fitness	8	`O13 Business Capability`, `RPT03 Capability Heatmap`, `R01 Capability decomposition`
EA-08	Application portfolio truth	9	`RPT02 Application Portfolio`, `RPT04 Interface Catalogue`, `RPT13 Orphan Analysis`
EA-09	Data ownership and lineage	10	`RPT05 Data Ownership`, `O43 Master Data Rule`, `R05 Application manages Data En`
EA-10	Obligation coverage	10	`RPT09 Regulatory Coverage`, `O63 Control Objective`, `R16 Control Objective met by Co`
EA-11	Change control over the repository	5	**Nothing. There is no version-control sheet, no change log and no `Change Ref` column**

`EA-11` carries the lowest weight and the most uncomfortable finding. This repository ships with no change control of any kind, so the dimension cannot score above 1 on the day you open the file, and it will not move until you build what appendix F Form H-2 prescribes. Leave the dimension in. A weight of 5 on a dimension stuck at 1 costs four points of the total, and those four points are the argument for an afternoon's work.

The span, pole to pole

Ref	Level 1 looks like	Level 5 looks like
EA-01	No signed statement of work; the architect's scope is whatever the last project asked for	A signed mandate, an executive sponsor who chairs a body that sits, and a recorded decision in the last quarter that went against a delivery date
EA-02	Four gates on paper; `059 Compliance Assessment` holds zero rows, as shipped	Every acceptance criterion assessed at the gate that tests it, refusals counted, and dispensations expiring with a consequence
EA-03	A file somebody updated for a workshop eighteen months ago	`ZZ Checks` run before and after every phase, every line moving only when a change log line says why
EA-04	Twenty-four principles with no tier and nothing bound to them	A tier column, a binding sheet, and a principle that has refused a design in the last six months
EA-05	Requirements in a document; `Reaches a block` reads No on most of them	Every Must requirement traced to a building block with an acceptance criterion a gate has tested
EA-06	`029 Gap` holds zero records and `RPT07` is empty, as shipped	Every gap owned, dated, closed by a named work package, and the overdue count reported monthly
EA-07	120 capability rows with machine-generated level 2 and level 3 names	A map the business argues with, scored by owners, re-scored annually, driving investment
EA-08	An application count nobody can reconcile and two orphans nothing maps to	A portfolio verified by decommissioning, every interface managed and logged, orphans questioned within a quarter
EA-09	Fourteen entities with post titles for owners and three with `Lineage` = Absent	Every entity owned by a person, mastered once, lineage traceable to a reported figure without reconstruction
EA-10	Seven obligations, four with no mechanism or a mechanism in breach	Every obligation with a control, named evidence, and a date it was last produced for somebody outside the bank
EA-11	No change log, no `Change Ref`, no `Approved By` — nothing records who changed a judgement	Every row traceable to a change request and a signatory, with a report that catches the omission

(b) The banking architecture health dimensions

Fifteen dimensions, weighted to 100. These are the industry-specific half and they are what makes this assessment worth running at a bank rather than anywhere else. Each cross-references volume 1 rather than re-teaching it.

Ref	Dimension	Weight	Volume 1
BH-01	Core and product change lead time	8	ch 12, *The legacy question*
BH-02	Party and customer mastering	8	ch 9, *The spine: a party, an agreement, an account*
BH-03	Account and ledger integrity	8	ch 10, *Time: three dates, not two*
BH-04	The payments estate and rail readiness	9	ch 12, *10. The payments hub and the rail interfaces*
BH-05	Card and scheme currency	6	ch 12, *7–9. Cards, end to end*
BH-06	Credit origination and decisioning	7	ch 8, *8. Retail credit, application to decision*
BH-07	Collections and recoveries	5	ch 8, *11. Collections and recoveries*
BH-08	The impairment data chain	7	ch 15, *IFRS 9, and the most data-hungry accounting standard a bank faces*
BH-09	Regulatory reporting and the return cycle	8	ch 15, *The BA returns, and the change that moved them out of the law*
BH-10	Deposit-insurance single customer view readiness	6	ch 15, *CoDI, and a regulation that is really a data specification*
BH-11	Financial crime and screening	6	ch 15, *Sanctions screening, and the twenty-four-hour clock*
BH-12	Fraud and the social-engineering problem	5	ch 12, *18. Fraud*
BH-13	Treasury and asset-liability management	5	ch 6, *Liquidity, and why a treasury system exists at all*
BH-14	Resilience against criticality	6	ch 15, *Resolution, and the test you take over a weekend*
BH-15	Conduct and data protection	6	ch 15, *POPIA, with the banking bite*

The span, pole to pole

Ref	Level 1 looks like	Level 5 looks like
BH-01	A new product variant needs a core release, and the release train runs twice a year	Cover, pricing, fees and eligibility are business-maintained configuration; a variant ships in a fortnight with its classifications derivable
BH-02	Four party masters, no common identifier, and a customer timeline nobody can assemble	One party master with a durable identifier, survivorship rules recorded, and every consumer reading it rather than copying it
BH-03	Entries amended in place; a past position reproducible only by restatement	An immutable entry ledger with effective, posting and recorded dates, versioned reference data, and reproducible period-end snapshots
BH-04	One "EFT" payment type; the clearing stream inferred downstream; batch windows nobody owns	Stream carried on every instruction; one component per rail; structured remittance data end to end; cut-offs owned and monitored
BH-05	The scheme message layer written per scheme with no mapping document, and the mandate calendar discovered by surprise	Scheme dialects isolated behind one adapter, the mandate calendar an architecture input, token-to-account mapping answerable
BH-06	Decision inputs not retained; a decline reason reconstructed by hand	Every decision reconstructible — inputs as at the decision, model version, rule set, affordability calculation, human-review route
BH-07	Arrears state held in three places and the hand-off to recoveries done by spreadsheet	One arrears position, a recorded treatment history, legal-stage evidence retained to the statutory clock
BH-08	The initial credit state of an exposure not retained, so a prior-period impairment cannot be rebuilt	Staging, initial recognition state and the forward-looking inputs all retained and dated; a prior-period number reproduced on demand
BH-09	Returns assembled in spreadsheets; the source-to-return mapping is code	Mapping is metadata a business analyst changes; returns are versioned entities; the data layer sits finer than any current form
BH-10	No consolidated depositor view; joint accounts and excluded depositors not identifiable in the data	The view produced on schedule to the specified field set, with excluded-depositor relationships resolvable at any reporting date
BH-11	Screening at onboarding only, lists refreshed when somebody remembers	Screening on change to either side within a monitored clock, with alert disposition and reporting evidenced
BH-12	Fraud rules and authorisation rules maintained by different teams with no combined view of what gets declined	One view of declines across fraud and authorisation; the social-engineering cases distinguished in the data, not merged into card fraud
BH-13	Positions reconciled by hand between the dealing system and the ledger; liquidity reported on a lag	Deal capture to settlement to ledger without manual reconciliation; the liquidity position available intraday
BH-14	`049 Resilience Tier` with eleven of twelve rows reading Never tested, as shipped	Tiers set against business criticality, objectives tested inside cadence, and every failed test raising a gap

BH-15	Consent and purpose not held per channel; cross-border transfers with no mapped basis	Purpose, consent and transfer basis held per party per channel per destination, and the disposal rule actually runs

LEVEL 1 ON BH-14 IS THE DEFAULT, NOT THE EXCEPTION

Eleven of twelve resilience rows in the shipped repository read Never tested, with a Tier 1 application at a twenty-four-hour recovery objective and a Tier 3 application at fifteen minutes [1]. That is not a quirk of invented data. It is what most resilience registers look like before somebody runs the test, and the reason the dimension is weighted at 6 rather than 3 is that the operational-resilience obligation is one of the four in `O63 Control Objective` reading No mechanism or In breach.

How to run it

Three weeks of elapsed time and about six working days of effort. Do not compress it into a workshop: a room scoring twenty-six dimensions in an afternoon produces consensus, and consensus is what you are trying to avoid.

1. **Fix the weights and the scale, in writing, before you look at anything.** Two tables, twenty-six rows, the weights above or your own. Circulate them. Changing a weight after you have seen a score is how an assessment becomes an argument about method.
2. **Run `ZZ Checks` and read every report.** Half a day. Twenty-one check lines and thirteen reports, and between them they evidence most of the eleven practice dimensions without you speaking to anybody. Write the numbers down with the date you read them, because they move.
3. **Score the eleven practice dimensions from the repository alone**, and mark each score provisional. You are the subject of this half, so a second architect or the Chief Information Officer should re-score it independently before it goes anywhere.
4. **Interview for the fifteen health dimensions.** Four sets of interviews, grouped by who you are talking to, using the question sets at the back of this appendix. Eight to twelve conversations of forty-five minutes. Ask for the artefact in the room: "show me last month's return", not "do you produce the return reliably".
5. **Score each health dimension against the descriptor, with its evidence named.** One line of evidence per score, in the form a repository sheet, a dated document, or a named person and what they showed you.
6. **Write the findings, not the scores.** For each dimension scoring 1 or 2, one sentence of fact: what you observed, how many of them, and what it prevents. That sentence is what becomes a gap row.

When a dimension has no evidence

Score it low. The absence of evidence is the finding, and it is usually a more useful finding than a middling score.

This is the rule people argue with, so hold it plainly: if nobody can show you that the bank tests its recovery objectives, the bank does not demonstrably test its recovery objectives, and the dimension scores 1 or 2 regardless of what anybody believes. You are not asserting the thing does not happen. You are recording that it cannot be evidenced, which — for an obligation a supervisor will ask about — is the same operational position.

Write the absence precisely. "No recovery test record was produced for any Tier 1 application; the Chief Information Officer's office was asked on 14 March and again on 21 March" is a finding somebody can refute by producing the record. "Resilience testing is immature" is not.

Getting it signed

An unsigned assessment is an opinion. Three signatures, and ask for them in this order.

The Chief Enterprise Architect signs the method — the weights, the scale and the evidence rule — before any score is shown to anybody. That signature is what stops the method being renegotiated

once the numbers are unwelcome.

Each dimension's evidence owner signs their own evidence, not the score. The head of regulatory reporting confirms that what you wrote down about the return cycle is what you were shown. This is the step that takes the most time and saves the most argument: a score somebody disputes is survivable, a piece of evidence somebody denies is not.

`GB-01 Architecture Board` signs the assessment as a whole, with the findings attached and the gap rows drafted. Per `057 Governance Body` that body holds `GT-1` and `GT-2` and approves dispensations with an expiry, which makes it the right body: it is the one that will have to refuse something on the strength of these findings.

Then send it to the sponsor with the six or eight gap rows, not the twenty-six scores. The scores are an appendix to your own findings.

Turning a low score into a gap

A score is not a gap. A gap is a fact about the estate, in `029 Gap` — `Ref`, `Gap`, `Domain`, `Building block`, `Severity`, `Status`, `Due` [1] — and it has to be written so that it can be closed and tested.

Four rules. **Name the quantity**: nineteen interfaces, three components, eleven of twelve rows. A gap with no number cannot be measured as it closes. **Name what it prevents**, in the bank's own terms, not in architecture terms. **Set `Severity` from consequence**, not from how much work it is: Critical means a licence, a legal or a reproducibility consequence. **Set `Due` with whoever will be held to it**, in the room, before you write it down.

THE WORKED EXAMPLE, END TO END

`BH-14 Resilience against criticality` scores 1. The evidence line reads: `049 Resilience Tier`, twelve rows, eleven reading `Tested` = Never tested; `RES-03` Card Management at `Tier 1` with a 24-hour recovery objective while `RES-02` Payment Hub sits at `Tier 3` with 15 minutes; no test record produced on request on two dates [1].

That becomes two gap rows, not one, because they close differently.

Ref	Gap	Domain	Severity	Status	Due
GAP-14	Eleven of twelve applications have never had their recovery objective tested, so no impact tolerance can be evidenced	Technology	Critical	Open	2028-06
GAP-15	The resilience tier scale and the recovery objectives disagree: Tier 1 applications carry longer objectives than Tier 3	Technology	High	Open	2027-12

Then do the three things that make the rows real. Point them at the technology building block that closes them through `029 Building block` and `R12 Gap closed by Work Package` — here, the resilience tiering and test standard from appendix G. Bind them to the principle they breach, on the sheet appendix G says you must add: `PRN-04`, which now reads *demonstrated by a test inside the tier's stated cadence*. And give `GAP-15` an earlier date than `GAP-14`, because fixing the scale is a morning's work and testing eleven applications is a year's — and a programme that schedules them together will do neither.

`GAP-15` is the more interesting row. It is free to fix, nobody will resist it, and until it is fixed every conversation about `GAP-14` will be conducted in a vocabulary where Tier 1 means two different things.

How many gaps. Six to ten from a first assessment. Not twenty-six. Take every dimension scoring 1, consolidate where two share a cause — `BH-02` party mastering and `BH-10` the single customer view will usually be the same gap wearing two hats — and hold the rest in the assessment document as context. The twenty-six scores are your map; the eight gaps are what the Board can act on. An assessment that raises a gap per dimension is an assessment nobody will commission twice.

Re-running it

Annually, in the same month, with the same weights, by the same method. Keep the previous scores beside the new ones on the sheet and present the movement rather than the level: a dimension that went from 2 to 3 is the sentence the sponsor remembers.

Four things to watch on a re-run.

A score that rose with no work behind it is a scoring drift, not an improvement. Ask what evidence supports the new score and whether it is the same kind of evidence as last year. The commonest cause is a more generous assessor.

A score that did not move on a funded dimension is the most valuable finding of the second run, and the one the programme will want softened. Write it plainly and name the work package from `O30`

`Work Package` that was supposed to move it.

A dimension nobody can evidence two years running should have its weight questioned, not its score. Either it does not matter here, or nobody owns it — and the second is a finding about the practice, which belongs in `EA-01`.

`EA-11` should be the first dimension to move, from 1 to 3, because building the change log is an afternoon. If it is still 1 at the second run, nothing else on the practice side can be trusted, because every score rests on a repository nobody is recording changes to.

The interview sets

Four sets, grouped by who is in front of you. Print the set, not the whole appendix, and leave the ruled space blank until they have answered — writing while they are still talking is how you get the answer they think you want.

SET 1 — THE CHIEF ENTERPRISE ARCHITECT AND THE CHAIR OF THE ARCHITECTURE BOARD (EA-01, EA-02, EA-11)

Name the last design this function refused, the date, and what happened next. If there is none, name the last one it should have refused.

`059 Compliance Assessment` holds zero rows. When was an acceptance criterion last tested at a gate, and who holds the record?

Who changed a judgement in this repository last, what did they change, and how would anybody know?

Which decision has the architecture function taken that `GB-02 Investment Committee` should have taken, and which has it avoided taking?

SET 2 — THE CHIEF DATA OFFICER, THE INFORMATION OFFICER AND THE DATA STEWARDS (EA-09, BH-02, BH-03, BH-15)

How many systems believe they master the party? Name them, and name the one the reconciliations clerks actually trust.

Take one customer who holds a current account, a card and a home loan. How long to assemble their full position, and how much of it is done by hand?

Can you reproduce the general ledger position as at the last month-end, from the ledger, without a restatement? Show me.

For every cross-border data flow in the estate: where is the transfer basis recorded, per data category per destination?

Which personal data has passed its retention clock and is still here? What stops the disposal rule running?

SET 3 — FINANCE, CREDIT RISK AND REGULATORY REPORTING (BH-06, BH-08, BH-09, BH-13)

Take last month's prudential return. How many manual adjustments, and who holds the working papers?

Reproduce the impairment number as at the prior reporting date. What is retained that makes it possible — and what has to be rebuilt?

For one credit decline last month: render the reason, the inputs as at the decision, the model version and the human-review route.

When the return specification changes — and it now changes by written determination rather than by gazette — who makes the change, and is it a configuration or a release?

How is the treasury position reconciled to the ledger, and how long after the dealing day?

SET 4 — PAYMENTS, CARDS, OPERATIONS AND FINANCIAL CRIME (BH-01, BH-04, BH-05, BH-07, BH-11, BH-12, BH-14)

How long from a product change being agreed to it being live, and which step is the long one?

Which clearing streams does this bank participate in, and is the stream carried on the instruction or inferred later?

Name the component that speaks to each rail. Where there are two paths to one rail, which was the emergency?

How does the scheme mandate calendar reach the architecture function, and when did it last arrive as a surprise?

What is the screening clock on a list change, who monitors it, and what was the longest breach last quarter?

Which declines are fraud and which are authorisation, and who can see both in one place?

When was the last recovery test, on which application, and what failed?

Form MA-1 — EA practice maturity score sheet

Feeds no repository sheet. There is no maturity sheet in `Banking_TOGAF_Metamodel.xlsx` and you should not create one: the scores belong in the assessment deliverable, registered in `031 Deliverable`, and only the findings reach the repository, as `029 Gap` rows on Form MA-3. **Eleven rows, the whole assessment.** `Weight` is fixed before scoring and must total 100. `Score` is a whole number 1 to 5. **`Evidence seen` is the column that makes the score defensible** — a sheet and row, a dated document, or a named person and what they put on the screen. A blank in that column means the score is 1 or 2, whatever is written in `Score`. **Signed by** the Chief Enterprise Architect for the method, a second architect or the Chief Information Officer for the scores, `GB-01 Architecture Board` for the set.

Ref	Dimension	Weight	Score	Evidence seen	Scored by
EA-01	*example — delete*	10	2	O57, 5 bodies, 1 minuted sitting	

Weighted total, and last year's. Carry both forward on every re-run.

This run	Weighted total	Last run	Movement	Dimensions at 1	Dimensions unevidenced

Form MA-2 — Banking architecture health score sheet

Feeds no repository sheet, same as MA-1. **Fifteen rows.** `Weight` totals 100 and is fixed before scoring. **`Volume 1` is printed so the scorer can check the descriptor** against the teaching rather than against their own assumptions; it is not transcribed anywhere. **Where a dimension corresponds to a level 1 capability in `013 Business Capability`**, write that capability's `Current` score in the margin and compare: three apart means one of the two is wrong, and finding out which is worth half a day. **Signed by** the executive who owns the estate the dimension describes — the head of payments for `BH-04`, the head of regulatory reporting for `BH-09` — each signing their own evidence, not the score. `GB-01 Architecture Board` signs the set.

Ref	Dimension	Weight	Score	Evidence seen	Signed
BH-04	*example — delete*	9	2	Shown by head of payments, 2027-03-18	

Form MA-3 — Findings, shaped for O29 Gap

Feeds `O29 Gap` — `Ref`, `Gap`, `Domain`, `Severity`, `Status`, `Due`. **Typed into Excel** `Building block`, which stays blank until the building block that closes the gap exists on Form E-2 of appendix F. **Six to ten rows from a first assessment, not twenty-six.** Consolidate where two dimensions share a cause and say which dimensions each row came from on the second table. **Write the gap as a fact with a number in it**: 'Eleven of twelve applications have never had their recovery objective tested', not 'resilience is weak'. **`Severity` comes from consequence** — Critical for a licence, legal or reproducibility consequence — not from effort. **`Due` is agreed in the room with the person who will be held to it**, before it is written down. **Signed by** `GB-01 Architecture Board`, with each `Domain` owner accepting the severity on their own rows. **Shows up in** `RPT07 Gap Analysis` and `R12 Gap closed by Work Package` — and until a work package exists, every row here is a gap with nothing closing it, which is the number to give the Board monthly.

Ref	Gap	Domain	Severity	Status	Due
GAP-01	*example — delete*	Technology	Critical	Open	2028-06

Which dimensions each gap came from, and the evidence line behind it. **This table feeds no sheet** — `O29 Gap` has no column for provenance — so it stays in the assessment deliverable, and it is the first thing anybody will ask for when they want to dispute a severity.

Gap	Dimensions	Evidence line	Scored
GAP-01	BH-14	*example — delete*	1

THE CHAPTER IN TEN LINES

- Run it at the start, before a funding round, and annually — never as a benchmark.
- Two assessments, kept apart: is the practice any good, and is the estate any good.
- Five levels, whole numbers only; a 2.5 is a disagreement you have hidden.
- No score without evidence — a sheet and row, a dated document, or a named person who showed you.
- Eleven practice dimensions weighted to 100, evidenced from `ZZ Checks` and the thirteen reports.
- `EA-11` cannot score above 1 as shipped, because this repository has no change control at all.
- Fifteen banking health dimensions, each cross-referenced to volume 1 rather than re-taught.
- A dimension with no evidence scores low: the absence is the finding.
- Six to ten gap rows from a first assessment, consolidated — not one per dimension.
- On a re-run, present the movement, and treat a rise with no work behind it as drift.

SOURCES FOR THIS CHAPTER

- **1.** Banking_TOGAF_Metamodel.xlsx — sheets O05, O13, O29, O49, O57, O59, O63, the thirteen RPT reports and ZZ Checks, as read from the workbook